

Coral

1.8.0

Volume I · Linguagem e Desenvolvimento

Sintaxe, bibliotecas, ferramentas, instalação, arquitetura e referência consolidada

EDIÇÃO 1.8.0

Revisão de leitura QoL 1 · outubro de 2026

Português corrente controlado. Contratos verificáveis. Ferramentas integradas.

Coleção oficial: consulte o relatório de qualidade e pendências editoriais.

Como usar este livro

Este volume ensina Coral em 42 capítulos e inclui glossário, respostas e índice remissivo. Cada capítulo apresenta objetivos, revisão e prática. As seções de aprofundamento podem ser lidas depois. Código Coral, terminal e saída usam rótulos distintos. Trechos dentro dos parágrafos têm fundo próprio. O capítulo 8 explica o realce de leitura. As respostas aparecem junto dos exercícios e no Apêndice B.

Caixas laterais destacam notas, compatibilidade e limites de evidência. Exemplos em Coral são validados pelo compilador da própria árvore antes da release.

Sumário navegável

Como usar este livro 2

Sumário navegável 3

Coral 1.8.0 · Volume I 5

Sobre esta edição 5

PARTE I: COMEÇAR COM CORAL 6

1. O que é Coral 6

2. Instalando Coral 7

3. Linux 8

4. Outras plataformas suportadas 10

5. VS Code 10

6. Projetos Coral 13

7. Primeiro programa (passo a passo) 14

8. Como ler exemplos e fichas de API 16

PARTE II: FUNDAMENTOS DA LINGUAGEM 18

9. Valores e variáveis 18

10. Tipos e tipagem opcional 20

11. Operadores e expressões 22

12. Condições 23

13. Repetições 24

14. Coleções, índices e fatias 26

15. Funções e escopo 28

16. Classes e objetos 32

17. Verificando tipos na prática 36

18. Erros, padrões e contextos 41

19. Geradores 44

20. Assíncrono 45

21. Programa principal 59

PARTE III: BIBLIOTECA FUNDAMENTAL COM ESPECIFICAÇÕES 61

22. Entrada e saída 61

23. Conversões 66

24. Texto 71

25. Coleções 83

26. Arquivos e caminhos 87

27. JSON e serialização 90

28. Matemática, datas e aleatoriedade 106

29. Persistência 111

PARTE IV: DESENVOLVENDO APLICAÇÕES 118

30. Projetos, módulos e coral.toml 118

31. Testes 121

32. Regras, eventos e tempo 123

33. Mundo 128

34. RPG 140

35. Jogos 142

36. Espaço e geometria 155

37. Procedural 157

38. Computação numérica e científica 160

39. Gráficos 163

40. Web 174

41. Sistema e hardware 183

42. Laboratório 186

APÊNDICE A: Glossário 622

APÊNDICE B: Respostas dos exercícios 626

APÊNDICE C: Índice remissivo 635

APÊNDICE D: Migração desde a Coral 1.7.0 638

Migração para a Coral 1.7.4 638

Migração para a Coral 1.7.6 638

APÊNDICE E: Mapa de recursos novos 640

Coral 1.8.0 · Volume I

Coral: Linguagem e Desenvolvimento

Sobre esta edição

Esta coleção pode ser consultada sem internet. O Volume I ensina e o Volume II reúne a referência e a arquitetura. O site pode usar as mesmas fontes e fichas; os livros não dependem dele. Coleção oficial em dois volumes para o runtime 1.8.0. O relatório de validação registra as lacunas editoriais conhecidas.

PARTE I: COMEÇAR COM CORAL

1. O que é Coral

Objetivo de leitura

Entender o que é um programa e reconhecer uma instrução Coral.

Antes de começar

Nenhum conhecimento de programação é necessário.

Coral é uma linguagem de programação em português corrente controlado. Um programa é uma sequência de instruções que o computador executa. Antes de estudar instalação e ferramentas, veja um programa inteiro:

Código Coral

```
mostre "Olá, Coral"
```

Saída esperada

```
Olá, Coral
```

O comando mostra `Olá, Coral` e termina a linha. O texto entre aspas é um valor; `mostre` é a instrução que o exibe. No Cap. 7 você executa e modifica esse programa passo a passo.

A Coral é usada em scripts, projetos com vários arquivos, simulações, dados e aplicações. A escrita natural tem formas definidas: você aprende essas formas em exemplos e consulta seus contratos nas fichas. Não é preciso saber Python para iniciar; a integração com Python é apresentada no Volume II, Parte V.

Este volume foi organizado para estudantes, professores e pessoas que desejam construir programas reais. Siga os capítulos na ordem ou use o índice remissivo para consultar um recurso. As seções Para ir além aprofundam o tema sem interromper a primeira leitura.

O que você aprendeu

Um programa reúne instruções. Em `mostre "Olá, Coral"`, a instrução é `mostre` e o valor é o texto entre aspas.

Prática

Escreva um programa que mostre seu nome e o nome Coral em duas linhas.

Conferir a resposta comentada

Use duas instruções `mostre`, uma com o seu nome entre aspas e outra com `"Coral"`. Cada instrução termina sua saída com uma quebra de linha.

Resposta do capítulo 1 no Apêndice B.

2. Instalando Coral

Objetivo de leitura

Escolher um modo de instalação e preparar os arquivos necessários.

Antes de começar

Leia o capítulo 1; tenha acesso ao pacote de distribuição.

Há quatro caminhos oficiais, todos apoiados pela mesma base de runtime.

Coral Setup, recomendado para uso comum

Abra `Instalacao/Coral_Setup.cmd` no Windows ou `Instalacao/Coral_Setup.sh` em sistemas POSIX. A interface usa o contrato `coral.manager/1`: não existe um segundo instalador escondido na GUI.

Coral Manager, recomendado para automação e diagnóstico

Terminal

```
python Instalacao/Runtime/coral-1.7.6.pyz manager diagnosticar --json
python Instalacao/Runtime/coral-1.7.6.pyz manager instalar --runtime coral-1.7.6.pyz
  --perfil estudante --json
python Instalacao/Runtime/coral-1.7.6.pyz manager instalar --runtime coral-1.7.6.pyz
  --perfil estudante --aplicar --json
```

Operações mutáveis produzem primeiro um plano. A alteração é aplicada apenas quando o usuário pede explicitamente.

Runtime portátil

Terminal

```
python3 Instalacao/Runtime/coral-1.7.6.pyz --versao
python3 Instalacao/Runtime/coral-1.7.6.pyz meu_programa.coral
```

Desenvolvimento do compilador

Terminal

```
python -m venv .venv
# Linux/macOS
source .venv/bin/activate
# Windows PowerShell
.\.venv\Scripts\Activate.ps1
python -m pip install -e .
python -m coral --self-check
```

A instalação persistente continua disponível pelos assistentes em `Instalacao/Assistentes`.

Uso sem internet

O pacote completo pode ser transferido por outro computador, dispositivo de armazenamento ou rede local. O runtime portátil `Instalacao/Runtime/coral-1.7.6.pyz` usa o Python disponível e inclui o núcleo Coral. Na máquina alvo, execute `python3 Instalacao/Runtime/coral-1.7.6.pyz --versao` e depois um arquivo Coral.

Isso não significa que toda opção do Setup seja offline. Perfis com dependências opcionais exigem que os pacotes estejam previamente disponíveis; a instalação por pip com downloads não funciona desconectada. Para o editor, transfira também o VSIX da distribuição e instale o arquivo localmente. Consulte o Cap. 5.

A instalação nativa e os pacotes de sistema possuem requisitos próprios. Consulte o Volume II, Parte III, seção Instalação, antes de escolher um fluxo para máquinas sem conexão.

O que você aprendeu

Setup orienta a instalação comum; Manager permite automação; o runtime portátil executa arquivos. Dependências opcionais precisam ser obtidas separadamente quando usadas.

Prática

Escolha a instalação para uma máquina sem internet e identifique o que precisa transferir antes.

Conferir a resposta comentada

Transfira o pacote completo da release e, quando houver dependências opcionais, os pacotes dessas dependências para a plataforma alvo. O runtime portátil do pacote não precisa baixar dependências do núcleo. A instalação com pip só é offline quando todos os wheels necessários estão disponíveis.

Resposta do capítulo 2 no Apêndice B.

3. Linux

Objetivo de leitura

Instalar em Linux, conferir a versão e identificar componentes opcionais.

Antes de começar

Tenha o pacote da Coral, apresentado no capítulo 2.

3.1 Requisitos

O executável Linux x86_64 inclui o pacote Coral com Python e pip privados. O modo portátil continua aceitando Python 3.10 ou superior quando o usuário escolhe executar o PYZ diretamente.

3.2 Setup ou assistente

Para instalar a versão atual pelo launcher da distribuição, execute:

Terminal

```
bash Instalacao/Coral_Setup.sh
```

O pacote `Projeto/dist/nativos/coral-setup_1.7.0_amd64.deb` é histórico da 1.7.0 e não deve ser confundido com um instalador nativo novo da 1.7.5. Os launchers `Instalacao/Coral_Setup.sh` e os assistentes de terminal continuam disponíveis para o fluxo portátil e de desenvolvimento.

3.3 Validar

Terminal

```
coral --versao
coral --self-check
coral --gate-1-3
coral --gate-1-4
coral diagnostico --completo
```

3.4 VS Code

O VSIX corrente fica em `Instalacao/Extensao_VSCode/coral-language-0.83.0.vsix`. Instale pela interface **Extensions: Install from VSIX...** ou pelo CLI do editor.

3.5 Componentes opcionais

Use `coral componentes listar`, `coral componentes verificar` e `coral componentes instalar`. Perfis como estudante, científico, hardware, jogos e completo agrupam capacidades sem transformar dependências opcionais em requisitos do núcleo.

O que você aprendeu

Confira o runtime antes de executar exemplos. O núcleo portátil e os componentes opcionais têm requisitos diferentes; o diagnóstico ajuda a identificar o ambiente.

Prática

Confirme no terminal a versão de Coral e execute um arquivo local.

Conferir a resposta comentada

Execute `coral --versao` e depois `coral programa.coral`. Se usar o runtime portátil, execute `python3 Instalacao/Runtime/coral-1.7.6.pyz --versao` e substitua `coral` por essa chamada.

Resposta do capítulo 3 no Apêndice B.

4. Outras plataformas suportadas

Objetivo de leitura

Distinguir portabilidade do núcleo e validação de uma plataforma.

Antes de começar

Conheça os caminhos de instalação do capítulo 2.

Linux é a plataforma autoritativa da Coral 1.7.6. Os fluxos e scripts preservados para Windows não representam uma qualificação nativa dessa plataforma nesta release. macOS também não recebe, nesta edição, a mesma declaração de evidência Linux.

O núcleo é escrito em Python e possui caminho portátil, mas portabilidade do código não comprova funcionamento de instaladores, janelas e drivers num sistema específico. Para usar outra plataforma, consulte a referência de instalação e registre sua validação local. O Volume II, Parte V, distingue compatibilidade de evidência de campo.

Este capítulo informa o estado efetivo da release; não depende de suporte futuro para permitir o estudo em Linux.

O que você aprendeu

Linux é a plataforma autoritativa desta edição. Um script disponível em outro sistema não comprova sua execução nem o funcionamento de drivers e janelas.

Prática

Explique por que possuir um script Windows não comprova a qualificação daquela plataforma.

Conferir a resposta comentada

O script registra um fluxo de instalação, mas a qualificação exige sua execução na plataforma. A 1.7.5 considera Linux autoritativo e não promete evidência nativa Windows.

Resposta do capítulo 4 no Apêndice B.

5. VS Code

Objetivo de leitura

Selecionar o runtime no VS Code e usar execução, depuração e auxílio à escrita.

Antes de começar

Instale Coral e VS Code; consulte os capítulos 2 e 3.

A extensão oficial é **Coral Language 0.83.0**.

Instalar

Use `coral-language-0.83.0.vsix` da pasta `Instalacao/Extensao_VSCode`. Depois execute **Developer: Reload Window**.

Confirmar o runtime

Use **Coral: Mostrar runtime selecionado**. A extensão respeita configuração explícita e, na ausência dela, consulta a instalação persistente.

Executar

Coral: Executar arquivo no terminal usa o runtime selecionado e mantém entrada e saída no terminal.

Depurar com F5

Abra um arquivo `.coral`, coloque um breakpoint e pressione F5. O DAP suporta breakpoints no arquivo principal e em módulos `.coral` locais importados, pilha, variáveis, evaluate e step in/over/out.

Estudar o código

O Modo Aprender e o Coral Ensinar explicam o programa sem exigir alteração do código fonte. O estudo automático no editor é opcional e pode ser ligado ou desligado pelo usuário.

Componentes e diagnóstico

A extensão também oferece comandos para verificar suporte opcional, instalar componentes pelo gerenciador oficial, abrir o Coral Manager e diagnosticar a integração VS Code.

Reconhecer símbolos pelas cores

A Coral 1.7.4 informa ao editor o papel dos símbolos que consegue resolver: classes, tipos, parâmetros, propriedades, variáveis, funções, métodos e módulos. A aparência depende do tema do VS Code. Para ligar ou desligar esse recurso, use `coral.semanticColoring.enable`.

Em `dieta do tipo Dieta`, o primeiro nome é um parâmetro e o segundo é uma classe. Em `defina seu dieta como dieta`, `seu dieta` é uma propriedade e a última ocorrência é o parâmetro. A diferença ajuda a ler o código sem mudar sua execução.

Se o código estiver incompleto ou a análise não puder comprovar um vínculo, o realce básico continua disponível. Uma cor não comprova que o programa está correto.

Escrever blocos e completar nomes

A opção `coral.editor.autoCloseBlocks` insere um corpo indentado e `fim` alinhado ao pressionar Enter após uma abertura completa. O completamento usa o contexto para sugerir nomes. Confira a sugestão selecionada antes de aceitar com Tab.

Em um `coral.toml` vazio, o comando **Coral: Inserir estrutura básica do coral.toml** cria o manifesto inicial. O arquivo possui um ícone próprio que acompanha temas claros e escuros.

Exemplo: parâmetro, propriedade e classe

Código Coral

```
crie a classe Dieta
fim
crie a classe Pessoa
    ao criar uma Pessoa com dieta do tipo Dieta
        defina seu dieta como dieta
    fim
fim
defina plano como Dieta()
defina pessoa como Pessoa(plano)
mostre "Pessoa criada"
```

Saída esperada

```
Pessoa criada
```

No construtor, `Dieta` designa a classe do parâmetro `dieta`. A atribuição guarda esse argumento na propriedade do objeto. O editor diferencia esses papéis quando consegue resolver os símbolos.

Consultas de tipo e chamadas em bloco na 1.7.5

Depois de `animal for do tipo`, o autocomplete oferece tipos nativos e classes válidas disponíveis no escopo. O nome da classe recebe realce de tipo; quando seu vínculo é comprovado, hover apresenta a declaração e a hierarquia, e ir para definição abre a declaração.

Ao pressionar Enter depois de `chame alvo com`, o editor acompanha a indentação e o argumento corrente. O fechamento automático respeita `coral.editor.autoCloseBlocks`. A ajuda de assinatura também atende construtores e métodos. A tolerância do editor a uma chamada incompleta serve para continuar digitando; o compilador ainda exige o fechamento correto.

Documentos abertos prevalecem sobre as cópias no disco. Ao fechar o arquivo, o editor volta a considerar a versão salva. Alterações de interface revalidam os consumidores afetados; não é preciso renomear arquivos ou reiniciar o editor para obter a atualização normal do índice.

O que você aprendeu

Executar mostra o comportamento do programa; depurar permite acompanhar seu estado. Completamento e cores auxiliam a escrita, mas não comprovam correção.

Prática

Localize a configuração de runtime da extensão e compare executar com depurar.

Conferir a resposta comentada

Executar inicia o programa no terminal. Depurar por F5 também conecta o adaptador e permite observar pontos de parada, pilha e variáveis. O runtime escolhido precisa corresponder à versão da aplicação.

Resposta do capítulo 5 no Apêndice B.

6. Projetos Coral

Objetivo de leitura

Criar um projeto a partir de um template e reconhecer seus arquivos principais.

Antes de começar

Consiga executar o comando `coral` no terminal.

Coral oferece templates oficiais:

Template	Objetivo
basico	Ponto de partida mínimo
terminal	Aplicativo interativo de terminal
jogo-2d	Projeto usando `coral.jogos`
aventura	Mundo, RPG e Regras Coral
simulacao	Estados e entidades
didatico	Exemplos graduais

Liste e crie:

Terminal

```
coral templates
coral novo MeuProjeto --template basico
cd MeuProjeto
coral executar
```

Projetos usam `coral.toml` para identificar entrada, versão e módulos. O VS Code oferece **Coral: Novo Projeto** e um Project Explorer próprio.

Fluxo recomendado de desenvolvimento

Referência para leitura

```
Editar .coral
   ↓
coral verificar / diagnósticos do LSP
   ↓
coral testar
   ↓
F5 / Debug Adapter
   ↓
coral naturalidade .
   ↓
coral documentar
```

O que você aprendeu

Um projeto organiza entrada, módulos e testes. O arquivo `coral.toml` registra sua configuração; editar, verificar e testar são etapas distintas.

Prática

Separe modelos e aplicação em arquivos diferentes e identifique a entrada no projeto.

Conferir a resposta comentada

Coloque classes e funções reutilizáveis em `modelos.coral` e importe os símbolos necessários em `principal.coral`. Configure o arquivo de entrada no `coral.toml`; esse caminho é independente do bloco opcional `programa principal`.

Resposta do capítulo 6 no Apêndice B.

7. Primeiro programa (passo a passo)

Objetivo de leitura

Salvar, executar e modificar seu primeiro programa.

Antes de começar

Tenha um runtime instalado e um editor de texto.

Salvar e executar

Crie um arquivo chamado `primeiro.coral` no editor e copie o programa abaixo. Salve o arquivo. Depois abra o terminal na pasta em que ele foi salvo e execute:

Terminal

```
coral primeiro.coral
```

Se estiver usando o runtime portátil, substitua `coral` pelo comando apresentado no capítulo 2. O bloco abaixo é o conteúdo do arquivo, não um comando para colar diretamente no terminal.

O programa completo

Código Coral

```
defina idade como 17

defina mensagem como ""

se idade for maior ou igual a 18 então
    defina mensagem como "maior de idade"
senão
    defina mensagem como "menor de idade"
fim

mostre mensagem
```

Saída esperada

```
menor de idade
```

- `defina` cria ou atualiza um nome.
- `como` separa o nome do valor.
- `se ... então` abre uma condição.
- `for maior ou igual a` é uma comparação escrita como português.
- `senão` fornece o caminho alternativo.
- `fim` encerra o bloco.
- `mostre` produz saída.

Experimente:

Terminal

```
coral explicar primeiro.coral
coral ensinar primeiro.coral
coral naturalidade primeiro.coral
```

`explicar`/Modo Aprender **executa** e rastreia valores. `ensinar` faz análise estática e não executa o programa. `naturalidade` verifica estilo e idioma da superfície Coral; não é uma prova de correção lógica.

O que você aprendeu

Salve o código em um arquivo `.coral` e execute esse arquivo. Aspas delimitam textos; `fim` encerra o bloco ao qual pertence.

Prática

Altere a mensagem do primeiro programa e preveja a saída.

Conferir a resposta comentada

Troque apenas o texto entre aspas. O comando `mostre` imprime o novo texto; as aspas não aparecem na saída.

Resposta do capítulo 7 no Apêndice B.

8. Como ler exemplos e fichas de API

Objetivo de leitura

Distinguir explicação, código, terminal, saída e contrato de uma função.

Antes de começar

Reconheça um exemplo simples do capítulo 7.

O que escrever e o que observar

A Coral usa palavras em português, mas segue formas definidas. A frase de uma explicação e a instrução de um programa têm papéis diferentes. Neste livro, trechos como `defina pontos como 10` recebem fonte própria e fundo discreto, mesmo dentro de um parágrafo.

Os exemplos em bloco possuem rótulos:

Rótulo	Como usar
Código Coral	Escreva ou copie em um arquivo `.coral`
Terminal	Execute no terminal, na pasta indicada
Saída esperada	Compare com o resultado; não copie para o programa
Configuração	Salve no arquivo de configuração indicado
Assinatura para consulta	Consulte os argumentos; não execute a assinatura como programa

Quando um exemplo não imprime texto, o livro informa isso em uma nota. O programa ainda pode criar valores, salvar arquivos ou alterar estado.

Ler o realce do código

Nos blocos Coral, palavras da linguagem, tipos e valores especiais recebem negrito. Instruções como `defina` se distinguem de nomes como `pontos`, escolhidos pelo programador. Aspas identificam textos; números mantêm sua escrita literal; comentários aparecem em tom discreto. A fonte monoespaçada, o fundo creme e o rótulo Código Coral distinguem o programa da explicação. O vermelho da marca aparece na faixa de anéis, sem reduzir o contraste do texto.

O realce do livro reconhece a escrita dos tokens. Ele não descobre o tipo de todo objeto nem valida o programa. A extensão do editor pode acrescentar informação semântica quando consegue resolver os símbolos.

Ler uma assinatura

Uma assinatura mostra como chamar uma função. Em `inteiro(valor, padrao=<ausente>)`, `valor` é obrigatório e `padrao` é opcional. Na chamada `inteiro("12")`, o texto `"12"` é o argumento. O retorno é o valor devolvido, neste caso `12`.

Termo	Significado
Parâmetro	Nome previsto na declaração da função
Argumento	Valor fornecido em uma chamada
Valor padrão	Alternativa usada quando o argumento opcional é omitido
Retorno	Valor que a operação devolve ao programa

As assinaturas extraídas do runtime usam notação Python para mostrar o contrato completo. `None` corresponde ao valor `nulo` da escrita Coral. `<ausente>` significa que o argumento não foi fornecido, o que é diferente de fornecer `nulo`. `*` separa parâmetros que precisam de nome; `/` identifica os apenas posicionais; `*partes` aceita vários argumentos posicionais.

Consultar uma ficha sem interromper o estudo

Primeiro entenda a intenção e o exemplo do capítulo. Depois consulte a ficha para conferir assinatura, tipos, obrigatoriedade, valores padrão, retorno, erros e observações. As fichas permanecem completas; no HTML, você pode abrir a seção de consulta quando precisar.

Uma forma natural é uma expressão oficial que corresponde a uma operação. `converta "12" para inteiro`, por exemplo, corresponde a `inteiro("12")`. A Coral não tenta adivinhar qualquer frase em português.

No HTML, ajuste fonte, largura e tema na barra de leitura. O botão Copiar copia apenas o conteúdo do bloco. As respostas ficam recolhidas para você tentar a atividade antes de conferir. No PDF, respostas e aprofundamentos aparecem por inteiro.

O Apêndice A reúne termos; o Apêndice C ajuda a localizar recursos. O Volume II contém a referência normativa e a arquitetura.

O que você aprendeu

Código é o que você escreve; saída é o que observa. Parâmetro é o nome declarado; argumento é o valor fornecido; retorno é o valor devolvido pela operação.

Prática

Diferencie parâmetro, argumento, padrão e retorno em uma ficha.

Conferir a resposta comentada

Parâmetro é o nome previsto pela função; argumento é o valor fornecido numa chamada; padrão é o valor usado quando um argumento opcional é omitido; retorno é o valor devolvido ao chamador.

Resposta do capítulo 8 no Apêndice B.

PARTE II: FUNDAMENTOS DA LINGUAGEM

9. Valores e variáveis

Objetivo de leitura

Criar uma variável, alterar seu valor e mostrar o resultado.

Antes de começar

Conheça `mostre`, apresentado no capítulo 7.

Guardar um valor

Uma variável associa um nome a um valor. Podemos usar o nome `pontos` para guardar a pontuação de um jogador. Para começar com dez pontos, escrevemos:

Código Coral

```
defina pontos como 10
```

Este exemplo não imprime texto no terminal.

Leia a instrução por partes:

Trecho	O que significa
`defina`	Inicia a instrução de atribuição
`pontos`	É o nome que receberá o valor
`como`	Separa o nome do valor
`10`	É o valor inicial

O nome `pontos` foi escolhido por nós. Você pode usar outro nome, desde que o escreva da mesma maneira nas instruções seguintes. Guardar um valor não significa mostrá lo no terminal.

Alterar e mostrar

Agora vamos acrescentar cinco pontos e retirar dois. A instrução `mostre pontos` exibe o valor que ficou guardado.

Arquivo completo de apoio:
`Exemplos/Citados/Fundamentos/01_fluxo_e_colecoes.coral`.

Código Coral

```
defina pontos como 10
adicione 5 a pontos
subtraia 2 de pontos
mostre pontos
```

Saída esperada

```
13
```

O programa executa as instruções em ordem:

Depois da instrução	Valor de `pontos`
`defina pontos como 10`	`10`
`adicione 5 a pontos`	`15`
`subtraia 2 de pontos`	`13`

Atenção ao nome e ao texto `mostre pontos` exibe o valor da variável. Já `mostre "pontos"` exibe a palavra entre aspas.

Atribuir outro valor

Se `pontos` já existir, `defina pontos como 20` substitui seu valor atual por `20`. A forma alternativa `pontos recebe 20` também faz atribuição.

Um nome precisa existir antes de ser consultado. Neste exemplo, a primeira instrução define `pontos`; as outras usam esse nome. Mais adiante, você verá nomes introduzidos por parâmetros de funções e por repetições.

O que você aprendeu

O nome permite consultar um valor. As instruções são executadas em ordem: definir 10, adicionar 5 e subtrair 2 deixa 13.

Prática

Troque o valor inicial de `10` por `20` no exemplo de pontos. Antes de executar, preveja o resultado.

Conferir a resposta comentada

A saída será `23`: o programa começa com vinte pontos, adiciona cinco e retira dois. A ordem das instruções continua sendo a mesma.

Resposta do capítulo 9 no Apêndice B.

10. Tipos e tipagem opcional

Objetivo de leitura

Reconhecer tipos de valor e escrever anotações opcionais sem confundi las com conversão.

Antes de começar

Conheça variáveis e listas simples; consulte o capítulo 9.

Tipos de valor

Categoria	Exemplos	Observação
Inteiro	`0`, `19`, `-3`	Contagens e valores sem parte fracionária
Decimal	`2.5`, `0.01`	Usa ponto decimal no código
Texto	`"Coral"`, `'Ana'`	Preserva os espaços dentro das aspas
Booleano	`verdadeiro`, `falso`	Condições e resultados lógicos
Ausência	`nulo`	Representa ausência de valor
Coleções	`[1, 2]`, `{"nome": "Ana"}`	Agrupam vários valores

A execução usa objetos Python. Por isso, a impressão direta de certos valores segue sua representação, como `True`, `False` e `None`. Isso não muda a grafia recomendada no código Coral.

Tipagem opcional

Arquivo: `Exemplos/Citados/Fundamentos/02_funcoes_tipos_e_escopo.coral`.

Código Coral

```
defina idade do tipo inteiro como 19
crie a função dobro com numero do tipo inteiro retornando inteiro
    retorne numero vezes 2
fim
mostre dobro(idade)
```

Saída esperada

```
38
```

Uma anotação informa o tipo que você espera usar. Ela registra essa intenção para a análise e para o editor. A Coral continua dinamicamente tipada; uma anotação não converte automaticamente os dados nem garante a rejeição estática de toda incompatibilidade. Use conversão explícita quando o dado chega como texto.

Tipos reconhecidos nas anotações incluem `inteiro`, `decimal`, `texto`, `booleano`, `lista`, `dicionário`, `conjunto`, `tupla`, `qualquer` e nomes de classes. Os tipos concretos e as operações seguem os objetos usados em execução.

Coleções parametrizadas

As quatro formas estruturais novas são listas, conjuntos, dicionários e tuplas de posições fixas.

Código Coral

```
defina notas do tipo lista de decimal como [8.0, 9.5]
defina tags do tipo conjunto de texto como {"coral", "linguagem"}
defina idades do tipo dicionário de texto para inteiro como {"Ana": 19}
defina pessoa do tipo tupla de (texto, inteiro) como ("Ana", 19)
```

Este exemplo não imprime texto no terminal.

Tipos podem ser compostos. Por exemplo, `dicionário de texto para lista de inteiro` representa um dicionário cujos valores conhecidos são listas de inteiros.

Para ir além

Nulidade em tipos compostos

A forma `T ou nulo` indica que o valor pode ter o tipo informado ou ser nulo.

Código Coral

```
defina apelido do tipo texto ou nulo como nulo
```

Este exemplo não imprime texto no terminal.

A posição dos parênteses importa. `lista de inteiro ou nulo` é uma lista que também pode ser nula. `lista de (inteiro ou nulo)` é uma lista cujos elementos podem ser nulos.

A 1.7.3 não introduz uniões gerais. Formas como `inteiro ou texto` continuam fora do contrato.

Verificação conservadora

As anotações ajudam a análise estática e o editor. Elas não inserem coerções nem validações profundas em tempo de execução. Quando a Coral não consegue provar o tipo de um valor, a informação permanece desconhecida em vez de produzir uma certeza inventada.

Coleções mutáveis são tratadas de modo conservador. Uma `lista de inteiro` não é assumida compatível com `lista de decimal` apenas porque os tipos numéricos individuais possam ter relações úteis em outros contextos.

O que a análise consegue comprovar

A tipagem permanece opcional. Na 1.7.4, a análise também usa contratos públicos de módulos Coral importados no projeto. Uma assinatura conhecida permite conferir argumentos e retorno, inclusive em construtores e métodos.

Esses diagnósticos continuam conservadores: um tipo desconhecido não é tratado como incompatível. Código Python importado e atributos sem informação suficiente podem

permanecer sem conclusão. As advertências de tipos ajudam a revisar o programa; não substituem os testes.

O que você aprendeu

A anotação comunica a intenção e ajuda a análise. Ela não converte valores nem valida profundamente todos os dados durante a execução.

Prática

Diferencie uma lista que pode ser nula de uma lista cujos itens podem ser nulos.

Conferir a resposta comentada

`lista de inteiro ou nulo` permite nulo no valor externo; `lista de (inteiro ou nulo)` permite nulo dentro da lista. Essas anotações não convertem nem validam profundamente os valores em execução.

Resposta do capítulo 10 no Apêndice B.

11. Operadores e expressões

Objetivo de leitura

Combinar valores em expressões e controlar a ordem das operações.

Antes de começar

Conheça valores numéricos e variáveis, dos capítulos 9 e 10.

Operadores combinam valores. `mais`, `menos`, `vezes` e `dividido por` são formas de leitura corrente. Operadores simbólicos continuam disponíveis, especialmente `%`, `//` e `**`.

Intenção	Forma Coral	Forma simbólica
Soma	`a mais b`	`a + b`
Subtração	`a menos b`	`a - b`
Multiplicação	`a vezes b`	`a * b`
Divisão	`a dividido por b`	`a / b`
Igualdade	`a for igual a b`	`a == b`
Diferença	`a for diferente de b`	`a != b`
Ordem	`a for maior que b`	`a > b`
Limite incluso	`a for menor ou igual a b`	`a <= b`
Pertencimento	`a estiver em valores`	Forma natural recomendada

Não use `=` como teste de igualdade. Ele aparece em argumentos nomeados, enquanto comparações usam `for igual a` ou `==`.

O que você aprendeu

Uma expressão produz um valor. Use parênteses para explicitar a ordem; comparação usa `for igual a` ou `==`, e não `=`.

Prática

Calcule o dobro da soma de 3 e 4, preservando a ordem desejada.

Conferir a resposta comentada

Use `(3 mais 4) vezes 2`. O resultado é 14. Parênteses tornam a ordem explícita.

Resposta do capítulo 11 no Apêndice B.

12. Condições

Objetivo de leitura

Escolher um caminho com condições e combinar perguntas lógicas.

Antes de começar

Conheça expressões e comparações do capítulo 11.

Escolher um caminho

Arquivo: `Exemplos/Citados/Fundamentos/01_fluxo_e_colecoes.coral`.

Código Coral

```
defina media como 7
se media for maior ou igual a 7 então
    mostre "Aprovado"
senão se media for maior ou igual a 5 então
    mostre "Recuperação"
senão
    mostre "Reprovado"
fim
```

Saída esperada

```
Aprovado
```

O programa testa os ramos em ordem e executa apenas o primeiro cuja condição for verdadeira. `senão` cobre os demais casos e é opcional. `fim` fecha a decisão inteira. A indentação comunica a estrutura, mas o encerramento explícito é indispensável.

Combinar condições

Use `e`, `ou` e `não`. Prefira parênteses ao combinar comparações com negação: `não (idade for menor que 18)`. O parser não interpreta toda frase possível em português; siga uma forma reconhecida.

A precedência implementada, da menor para a maior, é: `ou`, `e`, comparação, soma e subtração, produto e divisão, potência, operadores unários e operações de chamada, atributo e índice. Não presuma que todo caso de precedência seja idêntico ao Python; use parênteses para tornar explícita sua intenção, principalmente com potência e sinal negativo.

O que você aprendeu

A decisão testa os ramos em ordem e executa o primeiro verdadeiro. `senão` trata os demais casos; `fim` fecha a decisão.

Prática

Altere a média do exemplo para 6 e para 4; acrescente rejeição de valor negativo.

Conferir a resposta comentada

Para 6, o ramo de recuperação é escolhido; para 4, o de reprovação. Antes da comparação de aprovação, teste se a média é menor que zero e informe valor inválido.

Resposta do capítulo 12 no Apêndice B.

13. Repetições

Objetivo de leitura

Repetir instruções, percorrer intervalos e garantir o encerramento de um laço.

Antes de começar

Conheça variáveis e condições, dos capítulos 9 e 12.

Intervalos e condição de continuidade

Arquivo: `Exemplos/Citados/Fundamentos/01_fluxo_e_colecoes.coral`.

Código Coral

```
defina total como 0
para numero de 1 até 4 faça
    adicione numero a total
fim
mostre total
defina restante como 3
enquanto restante for maior que 0 faça
    subtraia 1 de restante
fim
mostre restante
```

Saída esperada

```
10
0
```

`para numero de 1 até 4 faça` inclui o limite final: o total é 1 + 2 + 3 + 4. `até antes de 4` excluiria 4. O laço `enquanto` repete enquanto a condição permanece verdadeira; ele precisa alterar algum estado ou ter outra forma de encerramento quando a condição depende desse estado.

`para cada item em colecao faça` percorre os elementos de uma coleção ou de outro iterável. `repita 3 vezes` repete um bloco por quantidade, sem exigir que você mantenha o contador manualmente.

Interromper e pular uma passagem

Arquivo: `Exemplos/Citados/Fundamentos/01_fluxo_e_colecoes.coral`.

Código Coral

```
para n de 0 até antes de 5 faça
    se n for igual a 1 então
        continue
    fim
    se n for igual a 3 então
        pare
    fim
    mostre n
fim
repita 2 vezes
    mostre "Coral"
fim
```

Saída esperada

```
0
2
Coral
Coral
```

`continue` pula o restante da passagem atual. `pare` encerra o laço mais próximo. Ambos exigem um contexto de repetição. O último bloco demonstra repetição por quantidade. Não escreva `pare` para encerrar uma função; nesse caso, use `retorne`.

Pensar na terminação

Antes de executar um `enquanto`, responda: qual estado muda? Qual condição permite sair? O corpo pode deixar de alcançar essa mudança? Isso evita laços infinitos. Para interromper um programa de terminal que não termina, use Ctrl+C.

O que você aprendeu

`até` inclui o limite; `até antes de` o exclui. Em `enquanto`, identifique o estado que muda e a condição que permite terminar.

Prática

Mostre os inteiros de 0 até antes de 3 e explique a terminação.

Conferir a resposta comentada

Use `para n de 0 até antes de 3 faça`, `mostre n` e `fim`. A saída é 0, 1 e 2 em linhas separadas; o limite superior não é visitado.

Resposta do capítulo 13 no Apêndice B.

14. Coleções, índices e fatias

Objetivo de leitura

Escolher uma coleção e acessar itens, chaves e fatias.

Antes de começar

Conheça variáveis e repetições, dos capítulos 9 e 13.

Lista: sequência mutável

Arquivo: `Exemplos/Citados/Fundamentos/01_fluxo_e_colecoes.coral`.

Código Coral

```
defina notas como [7, 8, 9]
coloque 10 em notas
mostre quantidade de notas
mostre primeiro item de notas
mostre notas[1:3]
```

Saída esperada

```
4
7
[8, 9]
```

Uma lista preserva a ordem e pode crescer. `coloque` acrescenta um elemento; `estenda` incorpora os elementos de outra sequência. Os índices começam em zero. Na fatia `[1:3]`, o início é incluído e o fim é excluído; a lista original não é alterada por essa consulta.

Dicionário: valores por chave

Arquivo: `Exemplos/Citados/Fundamentos/01_fluxo_e_colecoes.coral`.

Código Coral

```
defina aluno como {"nome": "Ana", "nota": 8}
defina a chave "cidade" de aluno como "Timon"
mostre valor de "nome" em aluno
mostre aluno["cidade"]
```

Saída esperada

```
Ana
Timon
```

Uma chave identifica um valor. A expressão `aluno["nome"]` é equivalente à consulta natural mostrada. Consultar uma chave inexistente gera erro; confira a existência ou use uma operação apropriada ao seu caso. Remova com `remova a chave "nota" de aluno`.

Outras coleções

Tuplas, como `(10, 20)`, são sequências imutáveis. Uma tupla com um único item precisa da vírgula: `(10,)`. Conjuntos guardam elementos únicos, sem contrato de ordem para apresentação. `{}` representa dicionário vazio; use `um conjunto vazio` para um conjunto sem elementos.

Arquivo: `Exemplos/Citados/Fundamentos/01_fluxo_e_colecoes.coral`.

Código Coral

```
defina numeros como [1, 2, 3, 4]
defina pares como [n vezes 2 para cada n em numeros se n % 2 for igual a 0]
mostre pares
defina unicos como {1, 1, 2}
mostre quantidade de unicos
```

Saída esperada

```
[4, 8]
2
```

A compreensão produz uma nova coleção a partir de outra. Primeiro percorre os valores, aplica o filtro e só então transforma cada valor aceito. No exemplo, apenas 2 e 4 passam pelo filtro e são dobrados.

Referências e mutabilidade

Atribuir uma lista a outro nome não cria necessariamente uma cópia independente. Os dois nomes podem apontar para o mesmo objeto. Quando a separação é importante, use uma cópia explícita, como uma fatia completa `valores[:]`, e considere que objetos aninhados continuam compartilhados numa cópia superficial.

Desempacotamento é permitido, como `defina x e y como (10, 20)`, respeitando a quantidade de valores. Evite misturar elementos de tipos incompatíveis em operações de ordenação ou aritmética.

O que você aprendeu

Listas usam índices a partir de zero; dicionários usam chaves. Uma atribuição pode compartilhar o objeto, por isso alterar e copiar são operações diferentes.

Prática

Diferencie alterar uma lista e produzir outra lista.

Conferir a resposta comentada

Uma alteração muda o objeto compartilhado por referências. Uma operação que produz nova lista preserva a original; confira o campo Atenção da ficha antes de assumir qualquer uma das duas situações.

Resposta do capítulo 14 no Apêndice B.

15. Funções e escopo

Objetivo de leitura

Declarar funções, passar argumentos e usar o retorno em outra expressão.

Antes de começar

Conheça variáveis, expressões e condições.

Uma função dá nome a uma operação reutilizável. Parâmetros são nomes que recebem argumentos a cada chamada. `retorne` encerra a função e devolve um valor; mostrar um resultado na tela não é a mesma coisa que retorná lo.

Uma função simples

Vamos receber um número e devolver seu dobro. O nome `numero` é um parâmetro; na chamada `dobro(21)`, o argumento é `21`.

Código Coral

```
crie a função dobro com numero
    retorne numero vezes 2
fim
mostre dobro(21)
```

Saída esperada

```
42
```

O cabeçalho declara a função e seu parâmetro. O corpo calcula o dobro; `retorne` entrega o resultado. Fora da função, `mostre` exibe esse resultado.

Dar uma forma natural à chamada

Arquivo: `Exemplos/Citados/Fundamentos/02_funcoes_tipos_e_escopo.coral`.

Código Coral

```
crie a função dobro com numero chamada como "dobre {numero}"
    retorne numero vezes 2
fim
mostre dobre 21
mostre (dobre 4) mais (dobre 5)
```

Saída esperada

```
42
18
```

`chamada como` declara uma forma natural. Os campos entre chaves precisam corresponder aos parâmetros. A leitura natural pode participar de outras expressões; os parênteses no segundo resultado deixam a composição evidente. Não existe interpretação livre por IA.

Parâmetros opcionais e nomeados

Arquivo: `Exemplos/Citados/Fundamentos/02_funcoes_tipos_e_escopo.coral`.

Código Coral

```
crie a função saudar com nome e prefixo opcional "Olá"
    retorne prefixo mais ", " mais nome
fim
mostre saudar("Ana")
mostre saudar(nome="Bia", prefixo="Oi")
crie a função contar com vários valores
    retorne quantidade de valores
fim
mostre contar(10, 20, 30)
```

Saída esperada

```
Olá, Ana
Oi, Bia
3
```

Um valor padrão permite omitir o argumento. Argumentos nomeados associam explicitamente o nome ao valor. `vários valores` aceita uma quantidade variável de argumentos posicionais. A forma `várias opcoes nomeadas` aceita argumentos nomeados variáveis.

Use `execute funcao(...)` quando deseja chamar uma função apenas por seu efeito. Use a chamada dentro de `defina` ou `mostre` quando deseja consumir o retorno. Uma expressão de chamada solta não é uma instrução universalmente aceita na superfície Coral.

Escopo e funções internas

Arquivo: `Exemplos/Citados/Fundamentos/02_funcoes_tipos_e_escopo.coral`.

Código Coral

```
crie a função criar_contador
    defina valor como 0
    crie a função proximo
        use valor do escopo externo
        adicione 1 a valor
        retorne valor
    fim
    retorne proximo
fim
defina contar como criar_contador()
mostre contar()
mostre contar()
```

Saída esperada

```
1
2
```

O estado `valor` pertence à função externa e continua acessível pela função retornada. `use valor do escopo externo` permite alterar essa ligação. Sem a declaração adequada, uma atribuição interna pode criar outra variável ou produzir um erro de escopo.

As formas declarativas de chamada natural são registradas durante a análise da fonte. Em projetos com módulos separados, importações seletivas também podem disponibilizar as formas naturais exportadas. Confira o capítulo 30 para aliases e ambiguidades; não presuma que toda forma interna do módulo seja exportada.

Funções como valores

Uma função pode ser atribuída a uma variável, recebida como argumento ou retornada. Uma função anônima pode ser escrita como `função anônima com x que retorna x vezes 2`. Isso é uma expressão; atribua o resultado a um nome antes de usar, quando precisar de uma operação identificável.

Decoradores são declarados com `decore com decorador` antes da função. Eles exigem um objeto chamável previamente definido e são um recurso avançado de interoperabilidade. Prefira funções comuns até existir uma razão concreta para modificar o comportamento de outras funções.

Não existe uma promessa de cobertura estática equivalente a linguagens com verificação obrigatória de tipos. Testes e validação de entradas continuam necessários.

Chamadas explícitas em várias linhas

Quando uma chamada tem muitos argumentos, coloque cada argumento numa linha. `chame somar com` abre o corpo porque `com` termina a linha sem argumentos; `fim` encerra essa chamada.

Código Coral

```
crie a função somar com primeiro, segundo
    retorne primeiro mais segundo
fim
defina total como chame somar com
    10
    chame somar com
        2
        3
    fim
fim
mostre total
```

Saída esperada

```
15
```

O primeiro `fim` encerra a chamada interna e o segundo encerra a externa. Os argumentos são avaliados uma vez, na ordem escrita. A versão compacta `chame somar com 10, 5` continua válida.

Cada entrada aceita uma expressão completa. Comentários e linhas vazias não contam como argumentos. Uma vírgula final é opcional. Argumentos nomeados e expansões

seguem as regras da chamada compacta. Para uma chamada sem argumentos, use a forma compacta sem `com`; abrir um bloco vazio com `com` é erro.

A chamada em bloco também pode criar objetos e chamar métodos. Ela pode aparecer em atribuições, retornos e argumentos de outra chamada. Se fizer parte de uma expressão maior, use os agrupamentos da linguagem para deixar a continuação explícita. O fechamento deve ocupar sua própria linha.

Use `chame` quando a chamada participa da organização natural do programa; `execute funcao(...)` continua disponível para executar a função por seu efeito. As duas formas invocam a operação; consumir ou ignorar o retorno depende do contexto. O suporte de chamadas usa seu próprio `fim`, enquanto a lista de parâmetros da seção seguinte usa `fim com`.

Parâmetros em linhas separadas

Na Coral 1.7.6, uma declaração que termina em `com` abre uma lista de parâmetros. Escreva um parâmetro por linha e feche a lista com `fim com`. O corpo começa depois desse fechamento e conserva seu próprio `fim`.

Código Coral

```
crie a função somar com
    primeiro do tipo inteiro
    segundo do tipo inteiro opcional 2
fim com retornando inteiro
    retorne primeiro mais segundo
fim
mostre somar(3)
```

Saída esperada

```
5
```

A forma compacta continua válida: `crie a função somar com primeiro do tipo inteiro, segundo do tipo inteiro opcional 2 retornando inteiro`. As duas formas têm os mesmos parâmetros, padrões e retorno. Um padrão é avaliado uma vez na criação da função, conforme o contrato existente; mover sua escrita para outra linha não muda esse momento.

Métodos, construtores e declarações assíncronas também aceitam a lista explícita. Os parâmetros implícitos do método continuam implícitos. Tipos estruturais e parâmetros variádicos conservam suas regras. Uma chamada multilinha usada como padrão fecha com seu próprio `fim`, antes do `fim com` da declaração. Uma função sem parâmetros omite `com`; uma lista vazia é diagnosticada.

Essa organização é opcional. O editor oferece a conversão de cabeçalhos elegíveis como uma ação explícita; a formatação normal não transforma toda assinatura compacta em um bloco.

O que você aprendeu

Uma função reúne uma operação reutilizável. `retorne` devolve um valor; `mostre` exibe texto. Parâmetros recebem os argumentos da chamada.

Prática

Crie uma função que receba um inteiro e devolva seu dobro.

Conferir a resposta comentada

Declare `crie a função dobro com numero do tipo inteiro retornando inteiro`, use `retorne numero vezes 2` e feche com `fim`. Uma chamada com 21 devolve 42.

Resposta do capítulo 15 no Apêndice B.

16. Classes e objetos

Objetivo de leitura

Criar objetos com estado próprio e reconhecer classe, atributo, método e herança.

Antes de começar

Conheça funções e escopo do capítulo 15.

Objetos combinam estado e operações. Uma classe define como criar e manipular esses objetos. Cada instância pode manter valores próprios.

Arquivo: `Exemplos/Citados/Fundamentos/03_objetos_e_erros.coral`.

Código Coral

```
crie a classe Contador
    ao criar uma Contador com inicial
        defina seu valor como inicial
    fim
    método somar com quantidade
        adicione quantidade a seu valor
    fim
    propriedade atual
        retorne seu valor
    fim
fim
crie contador como uma Contador com 10
execute contador.somar(5)
mostre contador.atual
```

Saída esperada

```
15
```

O construtor `ao criar` recebe o estado inicial. `seu valor` acessa o atributo da instância atual. O método altera esse atributo; a propriedade oferece leitura sem exigir uma chamada com parênteses.

Herdar comportamento

Arquivo: `Exemplos/Citados/Fundamentos/03_objetos_e_erros.coral`.

Código Coral

```
crie a classe Pessoa
    ao criar uma Pessoa com nome
        defina seu nome como nome
    fim
fim
crie a classe Aluno herda de Pessoa
    ao criar uma Aluno com nome e nota
        inicialize a classe pai com nome
        defina sua nota como nota
    fim
fim
crie aluno como uma Aluno com "Ana" e 9
mostre aluno.nome
mostre aluno.nota
```

Saída esperada

```
Ana
9
```

`Aluno` reutiliza a estrutura de `Pessoa`. O comando `inicialize a classe pai` chama a inicialização herdada, e o construtor acrescenta a nota. Herança é útil quando a relação de especialização é real; para apenas guardar ou usar outro objeto, composição costuma ser mais direta.

Referência de membros

Construção	Papel
`método nome com parametros`	Operação de instância; `self` é implícito
`método estático nome`	Operação sem instância implícita
`método de classe nome`	Operação que recebe a classe como `cls`
`propriedade nome`	Getter acessado como atributo
`ao definir a propriedade nome com valor`	Setter
`ao converter para texto`	Representação textual do objeto
`ao obter tamanho`	Protocolo de tamanho
`ao iterar` e `ao obter próximo`	Protocolo de iteração

Esses cabeçalhos são trechos de referência: devem aparecer dentro de uma classe, com corpo e `fim` próprios. As formas `seu`, `sua`, `seus` e `suas` referem se a atributos do objeto atual. O nome do atributo precisa coincidir exatamente com o que foi definido.

Para ir além

Herança múltipla e ordem dos métodos

Uma classe pode declarar mais de uma base em ordem explícita:

Código Coral

```
crie a classe Identificavel
    passe
fim
crie a classe Publicavel
    passe
fim
crie a classe Livro herda de Identificavel e Publicavel
    passe
fim
```

Este exemplo não imprime texto no terminal.

A Coral usa a ordem de resolução C3, a mesma regra aplicada pelo backend Python. Em uma hierarquia em diamante, a base comum aparece uma única vez na ordem de resolução.

Bases repetidas, ciclos, bases conhecidas que não são classes e hierarquias comprovadamente inconsistentes recebem diagnóstico antes da execução. Quando uma base é opaca para a análise local, a Coral evita inventar uma ordem estática.

Chamada cooperativa ao pai

Dentro de métodos de instância ou de classe, `chame o método pai` continua a cadeia cooperativa segundo a MRO.

Código Coral

```
crie a classe Raiz
    método nomes retornando lista de texto
        retorne ["Raiz"]
    fim
fim
crie a classe A herda de Raiz
    método nomes retornando lista de texto
        retorne ["A"] mais (chame o método pai nomes)
    fim
fim
```

Este exemplo não imprime texto no terminal.

Construtores podem usar `inicialize a classe pai`. A Coral não chama automaticamente todos os construtores das bases. A cooperação precisa estar explícita nas classes que participam da cadeia.

Exemplo: uma célula que guarda conteúdo

O objeto abaixo pode guardar grama ou um animal. Seus métodos fazem perguntas sobre esse conteúdo sem precisar comparar textos ou repetir regras de herança.

Código Coral

```
crie a classe Celula
    ao criar uma Celula com conteudo
        defina seu conteudo como conteudo
    fim
    método tem_grama
        retorne seu conteudo for do tipo Grama
    fim
    método tem_herbivoro
        retorne seu conteudo for do tipo Herbivoro
    fim
    método tem_carnivoro
        retorne seu conteudo for do tipo Carnivoro
    fim
fim
crie a classe Grama
fim
crie a classe Animal
fim
crie a classe Herbivoro herda de Animal
fim
crie a classe Carnivoro herda de Animal
fim
crie a classe Onivoro herda de Herbivoro e Carnivoro
fim

defina celula como chame Celula com
    Onivoro()
fim
mostre celula.tem_grama()
mostre celula.tem_herbivoro()
mostre celula.tem_carnivoro()
```

Saída esperada

```
False
True
True
```

As classes consultadas nos métodos podem ser declaradas depois da classe Celula; elas precisam estar disponíveis quando esses métodos forem executados. `Onivoro` pertence às duas classes ancestrais do exemplo. Para a aplicação organizada em módulos e testes, consulte `Exemplos/Citados/Analise_1_7_5/Ecossistema`.

Bases de herança em linhas separadas

Quando uma classe tem várias bases, `herda de` no fim da declaração abre a lista. `fim herança` fecha apenas o cabeçalho; o corpo da classe conserva seu próprio `fim`. A forma sem acento `fim heranca` também é aceita.

Código Coral

```
crie a classe Animal
fim
crie a classe Identificavel
fim
crie a classe Herbivoro herda de
    Animal
    Identificavel
fim herança
fim
defina animal como Herbivoro()
mostre animal for do tipo Animal
```

Saída esperada

```
True
```

A ordem das bases permanece a ordem escrita. O contrato de herança múltipla e a MRO C3 são os mesmos da declaração compacta. Repetir uma base, usar um valor que não representa uma classe ou criar uma hierarquia inconsistente continua produzindo diagnóstico. A lista não aceita expressões arbitrárias no lugar dos nomes das bases.

O que você aprendeu

A classe descreve os objetos; cada instância pode guardar seu próprio estado. Métodos operam sobre esse estado, e a herança reutiliza comportamentos.

Prática

Explique a diferença entre instância, classe e atributo.

Conferir a resposta comentada

A classe descreve a estrutura e as operações; a instância é um objeto criado a partir dela; um atributo guarda parte do estado desse objeto. Duas instâncias podem manter estados diferentes.

Resposta do capítulo 16 no Apêndice B.

17. Verificando tipos na prática

Objetivo de leitura

Verificar tipos nativos e pertencimento a classes do programa.

Antes de começar

Conheça tipos e classes, dos capítulos 10 e 16.

Às vezes um programa recebe dados cujo tipo precisa verificar. A verificação responde uma pergunta; a conversão tenta obter outro valor. Esses papéis são diferentes. Aqui verificamos; o Cap. 23 ensina conversões.

Código Coral

```
de coral.tipos importe e_tipo, nome_tipo

defina valor como 12
mostre nome_tipo(valor)
se valor for do tipo inteiro então
    mostre "Pode ser usado como contagem"
fim
mostre e_tipo(verdadeiro, "inteiro")
```

Saída esperada

```
inteiro
Pode ser usado como contagem
False
```

A saída é `inteiro`, `Pode ser usado como contagem` e `False`, em linhas separadas. Booleano não é inteiro implícito no teste oficial de Coral.

`tipo_de` devolve um descritor `TipoCoral`; `nome_tipo` devolve texto e `e_tipo` devolve um booleano. Nomes compostos das anotações do Cap. 10 pertencem à análise opcional; não confunda aquela análise estática com uma validação profunda feita por `e_tipo`. Para uma classe criada pelo programa, prefira a expressão natural com o nome da classe. A API explícita continua disponível e recebe a própria classe, não um texto com seu nome.

Consultar classes do programa

Uma consulta de tipo pergunta se o objeto pertence à classe indicada. Uma instância de uma subclasse também pertence às classes das quais ela herda. A consulta não compara apenas nomes e não converte o objeto.

Código Coral

```
de coral.tipos importe e_tipo
crie a classe Animal
fim
crie a classe Herbivoro herda de Animal
fim
defina animal como Herbivoro()
mostre animal for do tipo Herbivoro
mostre animal for do tipo Animal
mostre animal não for do tipo Animal
mostre e_tipo(animal, Animal)
```

Saída esperada

```
True
True
False
True
```

`animal for do tipo Animal` tem o mesmo resultado de `e_tipo(animal, Animal)`. A negação equivale a `não e_tipo(animal, Animal)`. A expressão pode aparecer em uma condição, atribuição, repetição ou retorno. As formas nativas, como `valor for do tipo inteiro`, continuam iguais. Em classes importadas, use o nome ou o alias disponível no escopo.

Depois de `do tipo`, forneça uma classe real. Se `Grama` guardar o número 10, o diagnóstico informa que Grama existe, mas não representa um tipo ou classe. Se o nome não existir, a análise informa o tipo inexistente e pode sugerir um nome próximo. Essas mensagens são advertências conservadoras de análise; não substituem a execução e os testes. Evite ocultar um nome de classe com outra variável.

Para herança múltipla, a consulta conserva o contrato de `e_tipo` e reconhece os ancestrais válidos da classe. A ordem de resolução de métodos continua sendo a MRO C3 apresentada no Cap. 16, seção Para ir além.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`tipo_de(...)` · **módulo** `coral.tipos`

O que faz

Obtém o descritor da categoria do valor.

Assinatura

Assinatura para consulta

```
tipo_de(valor: 'Any') -> 'TipoCoral'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

TipoCoral; o atributo nome contém o nome da categoria.

Exemplo

Código Coral

```
de coral.tipos importe tipo_de
mostre tipo_de(12).nome
```

Resultado

Saída esperada

```
inteiro
```

Forma natural

`(o tipo de 12).nome` consulta o mesmo contrato.

Atenção

Booleano não é inteiro implícito. Não é um validador profundo de coleções parametrizadas.

`nome_tipo(...)` · módulo `coral.tipos`

O que faz

Obtém o nome Coral da categoria do valor.

Assinatura

Assinatura para consulta

```
nome_tipo(valor: 'Any') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor, classe Python ou TipoCoral	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.tipos importe nome_tipo
mostre nome_tipo(12)
```

Resultado

Saída esperada

```
inteiro
```

Forma natural

`o nome do tipo de 12` consulta o mesmo contrato.

Atenção

Booleano não é inteiro implícito. Não é um validador profundo de coleções parametrizadas.

`e_tipo(...)` · módulo `coral.tipos`

O que faz

Testa se o valor pertence à categoria informada.

Assinatura

Assinatura para consulta

```
e_tipo(valor: 'Any', esperado: 'str | type | TipoCoral') -> 'bool'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor	sim	sem padrão	Valor sobre o qual a operação atua.
esperado	texto de tipo, classe Python ou TipoCoral	sim	sem padrão	Nome, tipo ou descritor aceito pelo teste de tipo.

Retorno

Booleano.

Exemplo

Código Coral

```
de coral.tipos importe e_tipo
mostre e_tipo(verdadeiro, "inteiro")
```

Resultado

Saída esperada

```
False
```

Forma natural

`valor for do tipo inteiro` e `animal for do tipo Animal` usam o mesmo contrato. A negação é `animal não for do tipo Animal`.

Erros

Um nome de classe ausente ou ocultado por outro valor pode receber diagnóstico de análise. A API não transforma um nome textual arbitrário numa classe do programa.

Atenção

Booleano não é inteiro implícito. Classes são passadas como objetos reais, não como texto. Subclasses pertencem aos ancestrais válidos, inclusive na herança múltipla. Não valida profundamente coleções parametrizadas.

Veja também

Cap. 17, seção Consultar classes do programa; Cap. 16, seção Exemplo: uma célula que guarda conteúdo.

O que você aprendeu

Uma consulta de tipo responde uma pergunta e não converte o valor. Uma instância também pertence aos ancestrais válidos de sua classe.

Prática

Verifique se verdadeiro é do tipo inteiro.

Conferir a resposta comentada

O teste oficial `e_tipo(verdadeiro, "inteiro")` devolve falso. A superfície Coral não trata booleano como inteiro implícito.

Resposta do capítulo 17 no Apêndice B.

18. Erros, padrões e contextos

Objetivo de leitura

Tratar falhas previstas, selecionar padrões e encerrar recursos.

Antes de começar

Conheça condições e funções. Os exemplos de arquivo serão retomados no capítulo 26.

Tratar falhas esperadas

Arquivo: `Exemplos/Citados/Fundamentos/03_objetos_e_erros.coral`.

Código Coral

```
tente
    defina resultado como 10 dividido por 0
se der erro do tipo ZeroDivisionError
    mostre "Não é possível dividir por zero"
finalmente
    mostre "Operação encerrada"
fim
```

Saída esperada

```
Não é possível dividir por zero
Operação encerrada
```

`se der erro do tipo` seleciona a exceção a tratar. `finalmente` é executado ao sair do bloco, inclusive quando houve erro. Uma captura ampla pode esconder falhas que você deveria corrigir; prefira capturar o tipo que seu programa realmente espera.

Uma exceção própria pode ser declarada com `crie a exceção Nome`, corpo e `fim`. Use `lance um erro do tipo Nome com "mensagem"` para lançá la. `garanta que condicao com mensagem "..."` expressa uma verificação, frequentemente usada em testes.

Selecionar por padrão

Arquivo: `Exemplos/Citados/Fundamentos/01_fluxo_e_colecoes.coral`.

Código Coral

```
defina comando como "iniciar"
combinar comando
    caso "iniciar"
        mostre "Começando"
    caso "sair" ou "parar"
        mostre "Encerrando"
    caso padrão
        mostre "Comando desconhecido"
fim
```

Saída esperada

```
Começando
```

`combinar` avalia casos em ordem. Há suporte a literais, alternativas, capturas, sequências, mapeamentos e outros padrões estruturais. O caso padrão cobre o que não foi reconhecido. Não confunda uma captura de nome com uma comparação contra uma variável: padrões possuem regras próprias.

Liberar recursos

Arquivo: `Exemplos/Citados/Dados/02_arquivos_json_e_caminhos.coral`.

Código Coral

```
de coral.arquivos importe escrever_texto
execute escrever_texto("dados.txt", "Coral")
com arquivo "dados.txt" aberto para leitura como arquivo faça
    defina conteudo como arquivo.read()
fim
mostre conteudo
```

Saída esperada

```
Coral
```

O gerenciador de contexto fecha o arquivo ao sair do bloco. A leitura por `arquivo.read()` é uma chamada avançada do objeto de arquivo. Para operações simples, a biblioteca padrão de arquivos geralmente é suficiente.

Distingua erros de parsing, que impedem compreender o código, de erros de execução, como divisão por zero ou arquivo ausente. `coral verificar` não executa o programa e, por isso, não comprova que um caminho existe ou que uma operação é válida para todos os dados.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`escrever_texto(...)` **· módulo** `coral.arquivos`

O que faz

Escreve texto no arquivo escolhido.

Assinatura

Assinatura para consulta

```
escrever_texto(caminho, texto, codificacao='utf-8')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho do arquivo local, interpretado na pasta de trabalho.
texto	qualquer valor convertido com str	sim	sem padrão	Texto a consultar ou transformar.
codificacao	texto de codificação	não	'utf-8'	Codificação de texto usada na leitura ou escrita.

Retorno

Nulo.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_texto
mostre escrever_texto("dados.txt", "Coral")
```

Resultado

Saída esperada

```
None
```

Forma natural

`escreva "Coral" em "dados.txt"` devolve nulo; o caminho é avaliado antes do texto.

Erros

PermissionError para acesso recusado; erros de caminho ou codificação quando a operação não é possível.

Atenção

Cria ou substitui o arquivo, com efeito no sistema de arquivos. Confira o caminho antes de escrever.

O que você aprendeu

Capture o erro que você espera. `combinar` seleciona padrões; um bloco de contexto cuida do encerramento do recurso ao sair.

Prática

Capture uma divisão por zero e imprima uma mensagem ao encerrar.

Conferir a resposta comentada

Use `tente`, a divisão, `se der erro do tipo ZeroDivisionError` com a mensagem, e `finalmente` para a mensagem final. O bloco final roda tanto no caso normal quanto na falha tratada.

Resposta do capítulo 18 no Apêndice B.

19. Geradores

Objetivo de leitura

Produzir uma sequência sob demanda com uma função geradora.

Antes de começar

Conheça funções e repetições, dos capítulos 13 e 15.

Produzir valores sob demanda

Arquivo: `Exemplos/Citados/Fundamentos/04_geradores_e_assincrono.coral`.

Código Coral

```
crie a função numeros com limite
    para n de 0 até antes de limite faça
        produza n
    fim
fim
para cada n em numeros(3) faça
    mostre n
fim
```

Saída esperada

```
0
1
2
```

`produza` suspende a execução de uma função geradora e entrega um valor por vez. Na próxima solicitação, a função continua de onde parou. Isso permite percorrer uma sequência sem construir todos os valores antecipadamente. `produza de outra_colecao` delega a produção a outro iterável.

O que você aprendeu

`produza` entrega um valor e suspende a função. Na próxima solicitação, a execução continua de onde parou.

Prática

Explique por que produzir números pode consumir menos memória que montar uma lista inteira.

Conferir a resposta comentada

Uma função geradora entrega um valor a cada solicitação e mantém apenas seu estado de execução. A economia existe quando o consumidor também processa os valores progressivamente.

Resposta do capítulo 19 no Apêndice B.

20. Assíncrono

Objetivo de leitura

Coordenar operações assíncronas e usar `aguarde` no contexto adequado.

Antes de começar

Conheça funções; complete primeiro os exemplos síncronos.

Programação assíncrona permite suspender uma operação enquanto outra pode avançar no mesmo ciclo de eventos. Ela não torna automaticamente todo cálculo paralelo nem elimina o custo de operações bloqueantes.

Arquivo: `Exemplos/Citados/Fundamentos/04_geradores_e_assincrono.coral`.

Código Coral

```
crie a função assíncrona dobro com numero
    espere 0 segundos
    retorne numero vezes 2
fim
crie a função assíncrona principal
    defina resultado como aguarde dobro(21)
    mostre resultado
fim
execute assincronamente principal()
```

Saída esperada

```
42
```

A função `dobro` é assíncrona. `aguarde` obtém seu resultado dentro de outra função assíncrona. `execute assincronamente` inicia a execução no nível principal. `espere 0 segundos` cede ao ciclo de eventos sem introduzir uma demora perceptível no exemplo.

Regras de contexto

`aguarde`, laços assíncronos e contextos assíncronos exigem função ou método assíncrono. Não copie apenas uma linha `aguarde` para o topo de um arquivo e espere que funcione. O exemplo inclui a estrutura necessária.

`crie tarefa como uma tarefa para operacao()` registra uma tarefa dentro de um contexto com ciclo de eventos ativo. Depois, `aguarde tarefa` obtém o resultado. Laços podem usar `para cada item assincronamente em fonte faça`; a fonte precisa implementar um iterador assíncrono.

Classes avançadas podem implementar `ao iterar assincronamente`, `ao obter próximo assincronamente`, `ao entrar assincronamente` e `ao sair assincronamente`. Esses nomes não transformam um objeto comum em assíncrono sem um comportamento correspondente.

Filas controladas na Coral 1.7.10

Quando há vários trabalhos independentes, criar todas as tarefas diretamente pode dificultar limites, progresso e sucesso parcial. `FilaTarefas` organiza um lote sem substituir `aguarde` nem criar uma nova sintaxe assíncrona.

Código Coral

```
de coral.assincrono importe FilaTarefas

crie a função assíncrona dobro com valor
    espere 0 segundos
    retorne valor vezes 2
fim

crie a função assíncrona principal
    defina fila como FilaTarefas(1)
    execute fila.adicionar(dobro(3), "dobro")
    defina relatorio como aguarde fila.executar()
    mostre relatorio.progresso.sucessos
    mostre relatorio.resultados[0].valor
fim

execute assincronamente principal()
```

Saída esperada

```
1
6
```

O primeiro argumento de `FilaTarefas` é `limite_ativas`. Ele determina quantas tarefas podem avançar simultaneamente. A fila preserva a ordem de submissão no relatório mesmo quando as conclusões ocorrem em outra ordem.

Na Coral 1.7.11, o cancelamento cobre também a janela entre a reserva de um item e o primeiro passo real da tarefa. Se o cancelamento chegar nesse intervalo, uma fábrica de awaitable ainda não materializada não é chamada. Falhas do callback de progresso continuam sendo propagadas, mas a fila fecha os itens reservados e pendentes antes de sair, evitando corrotinas órfãs e relatórios incompletos.

A política padrão `continuar` registra uma falha e continua o lote. Com `politica_falha="parar"`, a primeira falha observada impede o início de novos itens, mas trabalhos que já começaram podem terminar. `cancelar()` é mais forte: solicita cancelamento cooperativo das tarefas ativas e impede novos inícios.

`timeout_tarefa` limita cada item. `timeout_grupo` limita a execução do lote. Nenhum dos dois promete interromper à força código síncrono que ficou bloqueado. Para uma função síncrona potencialmente bloqueante, use `aguarde executar_bloqueante(funcao, argumentos...)`. Essa ponte desloca a chamada para uma thread auxiliar, mas cancelar a espera não encerra à força a função que já começou.

`ProgressoTarefas`, `ResultadoTarefa` e `RelatorioTarefas` são dados estruturados. Eles podem alimentar terminal, arquivo de log ou interface gráfica sem que o executor dependa de uma dessas interfaces. O projeto `Exemplos/Projetos/Automacao_Arquivos` mostra dois arquivos processados com sucesso e uma falha deliberada no mesmo lote.

Para programas iniciais de terminal, prefira fluxo síncrono. Use o modelo assíncrono quando houver uma necessidade concreta de coordenar espera e atividades, e teste tratamento de erro e encerramento das tarefas.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`FilaTarefas(...)` · **módulo** `coral.assincrono`

O que faz

Organiza um lote de awaitables com limite de simultaneidade, políticas explícitas de falha, cancelamento cooperativo e relatório ordenado.

Construtor

Assinatura para consulta

```
FilaTarefas(limite_ativas: 'int' = 4, capacidade: 'int | None' = None, politica_falha:
  'str' = 'continuar', timeout_tarefa: 'float | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
limite_ativas	inteiro positivo	não	4	Quantidade máxima de tarefas que podem estar ativas ao mesmo tempo.
capacidade	inteiro positivo ou nulo	não	None	Quantidade máxima de tarefas que podem ser adicionadas; nulo deixa o lote sem teto adicional de itens.
politica_falha	texto	não	'continuar'	`continuar` mantém sucesso parcial; `parar` impede novos inícios depois da primeira falha observada.
timeout_tarefa	decimal positivo ou nulo	não	None	Limite padrão em segundos para cada tarefa; nulo desativa o limite individual padrão.

Atributos importantes

Atributo	Significado
`limite_ativas`	Limite efetivo de trabalhos ativos.
`capacidade`	Teto de itens submetidos.
`politica_falha`	Política usada quando um item falha.
`fechada`	Indica que novas tarefas não podem ser adicionadas.
`executando`	Indica que o lote está em execução.

Métodos principais

Assinatura para consulta

```
adicionar(self, esperavel: 'Awaitable[Any] | Callable[[], Awaitable[Any]]', nome: 'str |
   None' = None, timeout: 'float | None' = None) -> 'int'
```

Referência para leitura

```
fechar(self) -> 'bool'
```

Referência para leitura

```
cancelar(self, motivo: 'str | None' = None) -> 'bool'
```

Referência para leitura

```
progresso(self) -> 'ProgressoTarefas'
```

Referência para leitura

```
executar(self, timeout_grupo: 'float | None' = None, ao_progresso:
  'Callable[[ProgressoTarefas], Any] | None' = None) -> 'RelatorioTarefas'
```

Retorno

Uma instância `FilaTarefas`. `aguarde fila.executar()` devolve `RelatorioTarefas`.

Exemplo

Código Coral

```
de coral.assincrono importe FilaTarefas
crie a função assíncrona dobro com valor
    espere 0 segundos
    retorne valor vezes 2
fim
crie a função assíncrona principal
    defina fila como FilaTarefas(1)
    execute fila.adicionar(dobro(3), "dobro")
    defina relatorio como aguarde fila.executar()
    mostre relatorio.progresso.sucessos
    mostre relatorio.resultados[0].valor
fim
execute assincronamente principal()
```

Resultado

Saída esperada

```
1
6
```

Forma natural

Não há forma natural separada. A fila compõe as construções assíncronas existentes da Coral.

Erros

ErroValor para limites ou política inválidos; ErroFilaCheia ao exceder `capacidade`; ErroFilaFechada ao adicionar depois do fechamento ou executar a mesma fila novamente. Falhas das tarefas são registradas em `ResultadoTarefa`.

Atenção

Cancelamento e timeout são cooperativos. Código síncrono bloqueante que não devolve controle ao ciclo de eventos não é interrompido à força.

`executar_fila(...)` · módulo `coral.assincrono`

O que faz

Executa um conjunto de awaitables com as mesmas políticas de `FilaTarefas` e devolve um relatório estruturado.

Assinatura

Assinatura para consulta

```
executar_fila(*esperaveis: 'Awaitable[Any]', limite_ativas: 'int' = 4, politica_falha:
  'str' = 'continuar', timeout_tarefa: 'float | None' = None, timeout_grupo: 'float |
  None' = None, ao_progresso: 'Callable[[ProgressoTarefas], Any] | None' = None) ->
  'RelatorioTarefas'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
esperaveis	um ou mais awaitables	não	sem padrão	Um ou mais awaitables a executar.
limite_ativas	inteiro positivo	não	4	Quantidade máxima de tarefas ativas.
politica_falha	texto	não	'continuar'	Política `continuar` ou `parar`.
timeout_tarefa	decimal positivo ou nulo	não	None	Limite padrão por tarefa em segundos.
timeout_grupo	decimal positivo ou nulo	não	None	Limite para o lote inteiro em segundos.
ao_progresso	função ou função assíncrona ou nulo	não	None	Callback opcional que recebe instantâneos de `ProgressoTarefas`.

Retorno

`RelatorioTarefas` com resultados na ordem de submissão, progresso final e estado do lote.

Exemplo

Código Coral

```
de coral.assincrono importe executar_fila
crie a função assíncrona dobro com valor
    espere 0 segundos
    retorne valor vezes 2
fim
crie a função assíncrona principal
    defina relatorio como aguarde executar_fila(dobro(2), dobro(3), limite_ativas=1)
    mostre relatorio.resultados[0].valor
    mostre relatorio.resultados[1].valor
fim
execute assincronamente principal()
```

Resultado

Saída esperada

```
4
6
```

Forma natural

Não há forma natural separada; use `aguarde executar_fila(...)` dentro de contexto assíncrono.

Erros

ErroValor para limites ou política inválidos. Falhas dos trabalhos são representadas nos resultados conforme a política escolhida.

Atenção

Para compatibilidade simples, `grupo_tarefas` continua disponível e propaga exceções como antes. Use `executar_fila` quando precisar de limite, progresso ou sucesso parcial.

`executar_bloqueante(...)` · **módulo** `coral.assincrono`

O que faz

Executa uma função síncrona em uma thread auxiliar para evitar bloquear diretamente o ciclo de eventos.

Assinatura

Assinatura para consulta

```
executar_bloqueante(funcao: 'Callable[..., Any]', *args: 'Any', **kwargs: 'Any') ->
  'Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
funcao	função chamável	sim	sem padrão	Função síncrona a executar.
args	zero ou mais valores	não	sem padrão	Argumentos posicionais repassados à função.
kwargs	zero ou mais argumentos nomeados	não	sem padrão	Argumentos nomeados repassados à função.

Retorno

O valor retornado pela função síncrona fornecida.

Exemplo

Código Coral

```
de coral.assincrono importe executar_bloqueante
de coral.texto importe maiusculas
crie a função assíncrona principal
    defina resultado como aguarde executar_bloqueante(maiusculas, "coral")
    mostre resultado
fim
execute assincronamente principal()
```

Resultado

Saída esperada

```
CORAL
```

Forma natural

Não há forma natural separada. A chamada explícita torna visível a passagem de uma operação síncrona para uma thread auxiliar.

Erros

ErroValor se `funcao` não for chamável. Exceções da função executada são propagadas ao awaitable e podem ser registradas por `FilaTarefas`.

Atenção

Cancelar quem aguarda não encerra à força uma função que já começou na thread auxiliar. Não use esta API como promessa de cancelamento forte.

`ResultadoTarefa(...)` · módulo `coral.assincrono`

O que faz

Representa o resultado de um item da fila com índice estável, nome, estado, valor ou erro e duração.

Construtor

Assinatura para consulta

```
ResultadoTarefa(indice: 'int', nome: 'str', estado: 'str', valor: 'Any' = None, erro:
  'BaseException | None' = None, segundos: 'float' = 0.0) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
indice	inteiro	sim	sem padrão	Posição original de submissão.
nome	texto	sim	sem padrão	Rótulo do item.
estado	texto	sim	sem padrão	`sucesso`, `falha` ou `cancelada`.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor ou nulo	não	None	Valor devolvido quando houve sucesso.
erro	exceção ou nulo	não	None	Exceção associada a falha ou cancelamento.
segundos	decimal	não	0.0	Duração observada da tarefa em segundos.

Atributos importantes

Atributo	Significado
`ok`	Verdadeiro somente quando `estado` é `sucesso`.

Métodos principais

Assinatura para consulta

```
para_dict(self) -> 'dict[str, Any]'
```

Retorno

Uma instância imutável `ResultadoTarefa`.

Exemplo

Código Coral

```
de coral.assincrono importe ResultadoTarefa
defina resultado como ResultadoTarefa(0, "item", "sucesso", 42)
mostre resultado.ok
mostre resultado.valor
```

Resultado

Saída esperada

```
True
42
```

Forma natural

Não há forma natural separada; normalmente o objeto é recebido de um relatório da fila.

Erros

A classe não valida semanticamente os textos de estado quando construída manualmente; use normalmente os objetos produzidos por `FilaTarefas`.

Atenção

`erro` preserva o objeto de exceção no processo. `para_dict()` converte o erro para tipo e mensagem e só inclui `valor` quando o item teve sucesso.

`ProgressoTarefas(...)` · módulo `coral.assincrono`

O que faz

Instantâneo imutável dos contadores de progresso de uma fila de tarefas.

Construtor

Assinatura para consulta

```
ProgressoTarefas(total: 'int', iniciadas: 'int', concluidas: 'int', sucessos: 'int',
  falhas: 'int', canceladas: 'int', pendentes: 'int', ativas: 'int') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
total	inteiro	sim	sem padrão	Quantidade total de itens.
iniciadas	inteiro	sim	sem padrão	Quantidade de itens que chegaram a iniciar.
concluidas	inteiro	sim	sem padrão	Quantidade finalizada em qualquer estado.
sucessos	inteiro	sim	sem padrão	Quantidade concluída com sucesso.
falhas	inteiro	sim	sem padrão	Quantidade concluída com falha.
canceladas	inteiro	sim	sem padrão	Quantidade cancelada ou não iniciada por encerramento.
pendentes	inteiro	sim	sem padrão	Quantidade ainda não iniciada.
ativas	inteiro	sim	sem padrão	Quantidade ativa no instante observado.

Atributos importantes

Atributo	Significado
`fracao`	Fração concluída entre 0 e 1.

Métodos principais

Assinatura para consulta

```
para_dict(self) -> 'dict[str, Any]'
```

Retorno

Uma instância `ProgressoTarefas`; `fracao` varia de 0 a 1 e vale 1 para uma fila vazia ou totalmente concluída.

Exemplo

Código Coral

```
de coral.assincrono importe ProgressoTarefas
defina progresso como ProgressoTarefas(2, 2, 2, 2, 0, 0, 0, 0)
mostre progresso.concluidas
mostre progresso.fracao
```

Resultado

Saída esperada

```
2
1.0
```

Forma natural

Não há forma natural separada.

Erros

A classe é um objeto de dados e não substitui as validações da fila; normalmente não deve ser construída manualmente.

Atenção

Use instantâneos recebidos de `fila.progresso()` ou do callback `ao_progresso` para apresentar estado sem acoplar a fila à interface.

`RelatorioTarefas(...)` · módulo `coral.assincrono`

O que faz

Reúne resultados ordenados, progresso final, estado do lote e motivo de encerramento.

Construtor

Assinatura para consulta

```
RelatorioTarefas(resultados: 'tuple[ResultadoTarefa, ...]', progresso:
  'ProgressoTarefas', estado: 'str', motivo: 'str | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
resultados	tupla de ResultadoTarefa	sim	sem padrão	Sequência ordenada de `ResultadoTarefa`.
progresso	ProgressoTarefas	sim	sem padrão	Instantâneo final `ProgressoTarefas`.
estado	texto	sim	sem padrão	`concluida`, `falha`, `cancelada` ou `timeout`.
motivo	texto ou nulo	não	None	Explicação opcional do encerramento não normal.

Atributos importantes

Atributo	Significado
`ok`	Verdadeiro apenas para conclusão sem falhas nem cancelamentos.

Métodos principais

Assinatura para consulta

```
para_dict(self) -> 'dict[str, Any]'
```

Retorno

Uma instância imutável `RelatorioTarefas`.

Exemplo

Código Coral

```
de coral.assincrono importe RelatorioTarefas, ProgressoTarefas
defina progresso como ProgressoTarefas(0, 0, 0, 0, 0, 0, 0, 0)
defina relatorio como RelatorioTarefas([], progresso, "concluida")
mostre relatorio.estado
mostre relatorio.ok
```

Resultado

Saída esperada

```
concluida
True
```

Forma natural

Não há forma natural separada.

Erros

A classe não substitui as políticas de execução; normalmente é produzida por `FilaTarefas.executar` ou `executar_fila`.

Atenção

`para_dict()` é a forma indicada para serialização ou apresentação genérica do relatório.

`ErroFilaTarefas(...)` · módulo `coral.assincrono`

O que faz

Base pública das falhas controladas relacionadas ao ciclo de vida da fila.

Construtor

Assinatura para consulta

```
ErroFilaTarefas(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mensagem	texto	sim	sem padrão	Mensagem explicativa da falha.
causa	exceção ou nulo	não	None	Exceção técnica opcional preservada como causa.

Atributos importantes

Atributo	Significado
`mensagem`	Mensagem pública da falha.
causa	Causa técnica opcional.

Retorno

Uma exceção `ErroFilaTarefas` pronta para ser lançada ou inspecionada.

Exemplo

Código Coral

```
de coral.assincrono importe ErroFilaTarefas
defina erro como ErroFilaTarefas("exemplo")
mostre erro.mensagem
```

Resultado

Saída esperada

```
exemplo
```

Forma natural

Não há forma natural separada.

Erros

Construir a exceção não lança o erro. Ela é lançada pelas operações da fila nos casos correspondentes.

Atenção

Prefira capturar a classe específica quando precisar distinguir capacidade, fechamento ou outra falha de fila.

`ErroFilaCheia(...)` · módulo `coral.assincrono`

O que faz

Indica que uma adição excedeu a capacidade configurada da fila.

Construtor

Assinatura para consulta

```
ErroFilaCheia(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mensagem	texto	sim	sem padrão	Mensagem explicativa da falha.
causa	exceção ou nulo	não	None	Exceção técnica opcional preservada como causa.

Atributos importantes

Atributo	Significado
`mensagem`	Mensagem pública da falha.
causa	Causa técnica opcional.

Retorno

Uma exceção `ErroFilaCheia` pronta para ser lançada ou inspecionada.

Exemplo

Código Coral

```
de coral.assincrono importe ErroFilaCheia
defina erro como ErroFilaCheia("exemplo")
mostre erro.mensagem
```

Resultado

Saída esperada

```
exemplo
```

Forma natural

Não há forma natural separada.

Erros

Construir a exceção não lança o erro. Ela é lançada pelas operações da fila nos casos correspondentes.

Atenção

Prefira capturar a classe específica quando precisar distinguir capacidade, fechamento ou outra falha de fila.

`ErroFilaFechada(...)` · módulo `coral.assincrono`

O que faz

Indica tentativa de adicionar ou executar fora do ciclo de vida permitido da fila.

Construtor

Assinatura para consulta

```
ErroFilaFechada(mensagem: 'str', *, causa: 'BaseException | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mensagem	texto	sim	sem padrão	Mensagem explicativa da falha.
causa	exceção ou nulo	não	None	Exceção técnica opcional preservada como causa.

Atributos importantes

Atributo	Significado
`mensagem`	Mensagem pública da falha.
causa	Causa técnica opcional.

Retorno

Uma exceção `ErroFilaFechada` pronta para ser lançada ou inspecionada.

Exemplo

Código Coral

```
de coral.assincrono importe ErroFilaFechada
defina erro como ErroFilaFechada("exemplo")
mostre erro.mensagem
```

Resultado

Saída esperada

```
exemplo
```

Forma natural

Não há forma natural separada.

Erros

Construir a exceção não lança o erro. Ela é lançada pelas operações da fila nos casos correspondentes.

Atenção

Prefira capturar a classe específica quando precisar distinguir capacidade, fechamento ou outra falha de fila.

O que você aprendeu

Uma função assíncrona pode ceder a execução enquanto espera. Isso não transforma automaticamente um cálculo em trabalho paralelo.

Prática

Explique o papel de aguarde dentro de uma função assíncrona.

Conferir a resposta comentada

`aguarde` suspende a função até que a operação entregue um resultado, permitindo ao ciclo de eventos avançar outras tarefas. Não transforma um cálculo bloqueante em cálculo paralelo.

Resposta do capítulo 20 no Apêndice B.

21. Programa principal

Objetivo de leitura

Organizar a entrada da aplicação e distinguir execução direta de importação.

Antes de começar

Conheça funções e módulos básicos.

Definir a entrada da aplicação

A Coral aceita uma entrada principal opcional para organizar aplicações sem obrigar todo arquivo a usar esse modelo.

Código Coral

```
programa principal
    mostre "Olá"
fim
```

Saída esperada

```
Olá
```

Quando o arquivo é executado diretamente, a Coral inicializa primeiro o módulo inteiro e depois inicia o bloco `programa principal` uma única vez. Isso permite declarar funções e classes abaixo do bloco e utilizá las dentro dele. Quando o mesmo arquivo é importado como módulo, a entrada não é iniciada.

O bloco não recebe parâmetros, não possui anotação de retorno e não cria uma função pública chamada `principal`. Uma função comum com esse nome continua válida e precisa ser chamada normalmente. Também não é permitido devolver um valor da entrada. Um `retorne` vazio pode encerrar o bloco antecipadamente.

Organização sugerida

Código Coral

```
programa principal
    chame apresentar com "Ana"
fim

crie a função apresentar com nome
    mostre "Olá, " mais nome
fim
```

Saída esperada

```
Olá, Ana
```

A entrada é opcional. Scripts lineares continuam aceitos exatamente como antes:

Código Coral

```
mostre "Olá"
```

Saída esperada

```
Olá
```

`chame` e `execute`

`chame nome com argumentos` é a forma corrente da Coral para invocar uma função pelo nome e por seus argumentos, especialmente quando a intenção é realizar uma ação e ignorar o valor devolvido.

`execute EXPRESSAO` executa uma expressão de chamada por efeito. É útil para chamadas explícitas como `execute objeto.metodo(1)` ou quando a forma de expressão é mais adequada. Para consumir um retorno, use a chamada dentro de uma expressão, por exemplo em `defina` ou `mostre`.

Nenhuma dessas formas é usada para iniciar `programa principal`. A própria execução direta do arquivo inicia o bloco automaticamente.

Importação

Um arquivo que contém `programa principal` ainda pode ser usado como módulo. A importação disponibiliza seus símbolos públicos, mas não inicia a aplicação. Isso evita efeitos inesperados em testes, bibliotecas e projetos com vários módulos.

Editor

A extensão Coral reconhece a entrada no outline, hover, dobramento, indentação e realce. Ao pressionar Enter no cabeçalho, o fechamento com `fim` segue a configuração de fechamento automático de blocos. O snippet de entrada principal permanece uma escolha contextual e não substitui automaticamente identificadores digitados pelo usuário.

Onde a entrada pode aparecer

A entrada não pode ficar dentro de condição, repetição, função, classe, `tente` ou `combinar`. Só pode existir uma entrada principal válida por módulo. REPL e execução interativa de trechos rejeitam esse bloco porque não representam a execução completa do módulo.

O que você aprendeu

O bloco `programa principal` é opcional. A execução direta o inicia após inicializar o módulo; uma importação não inicia esse bloco.

Prática

Coloque a função chamada abaixo do bloco programa principal. O que acontece ao importar o arquivo?

Conferir a resposta comentada

A execução direta inicializa o módulo inteiro e só depois inicia a entrada, portanto a função já está disponível. A importação disponibiliza os símbolos, mas não inicia o bloco principal.

Resposta do capítulo 21 no Apêndice B.

PARTE III: BIBLIOTECA FUNDAMENTAL COM ESPECIFICAÇÕES

22. Entrada e saída

Objetivo de leitura

Ler uma resposta no teclado e mostrar uma mensagem com o texto recebido.

Antes de começar

Conheça variáveis e concatenação, dos capítulos 9 e 11.

Pedir uma resposta

Um programa de terminal pode mostrar uma pergunta e esperar uma resposta. A função `ler_linha`, do módulo `coral.entrada`, devolve o que foi digitado como texto.

Salve este programa em `saudacao.coral` e execute com `coral saudacao.coral`:

Código Coral

```
# Exemplo interativo: forneça Ana quando o programa pedir um nome.
de coral.entrada importe ler_linha

defina nome como ler_linha("Nome: ")
mostre "Olá, " mais nome
```

Saída esperada

```
Nome: Olá, Ana
```

O resultado acima registra o texto escrito pelo programa quando a entrada fornecida é `Ana`. Em um terminal comum, a digitação também aparece: você verá `Nome: Ana`, pressionará Enter e depois verá `Olá, Ana`. O eco da digitação pertence ao terminal, não à saída produzida pelo programa.

Leia o exemplo em três passos:

1. A importação disponibiliza `ler_linha`.
2. A função mostra `Nome: `, espera a resposta e devolve texto. A atribuição guarda esse texto em `nome`.
3. `mais` une a saudação e o nome; `mostre` exibe a mensagem.

Erro comum Uma entrada com aparência numérica ainda é texto. Para somar uma idade digitada, converta o valor antes do cálculo, como ensina o capítulo 23.

Repetir a entrada em um teste

Ao testar automaticamente, podemos fornecer uma resposta conhecida sem digitar. `FonteEntradaSequencial` guarda as respostas e `usar_fonte_entrada` seleciona essa fonte durante o bloco.

Arquivo completo de apoio: `Exemplos/Citados/Dados/01_entrada_conversoes_e_texto.coral`.

Código Coral

```
de coral.entrada importe ler_linha, FonteEntradaSequencial, usar_fonte_entrada

defina entrada como FonteEntradaSequencial(["Ana"])
com usar_fonte_entrada(entrada) faça
    defina nome como ler_linha("Nome: ")
fim
mostre "Olá, " mais nome
```

Saída esperada

```
Olá, Ana
```

A fonte sequencial não reproduz o eco de um terminal. Ela permite verificar a mesma lógica com dados previsíveis. `ler_linha` traduz o fim da fonte para o erro Coral `FimDeEntrada`; um teste precisa fornecer respostas suficientes.

A superfície de entrada também pode ser usada por REPL, depurador e outras ferramentas. Para começar um programa comum, a leitura manual do primeiro exemplo é suficiente.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`ler_linha(...)` · **módulo** `coral.entrada`

O que faz

Lê uma linha da fonte explícita, do contexto de entrada ou do terminal.

Assinatura

Assinatura para consulta

```
ler_linha(mensagem: 'str | None' = None, *, fonte: 'FonteEntrada | FonteEntradaFuncao |
  None' = None) -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mensagem	texto ou nulo	não	None	Texto mostrado antes de ler a entrada; nulo equivale a texto vazio.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
fonte	FonteEntrada ou FonteEntradaFuncao ou nulo	não	None	Fonte explícita de entrada; omitida, usa o contexto ou o terminal.

Retorno

Texto sem a quebra de linha final.

Exemplo

Código Coral

```
de coral.entrada importe ler_linha
mostre ler_linha("Nome: ")
```

Resultado

Saída esperada

```
Nome: Ana
```

Forma natural

`leia uma linha com "Nome: "` tem o mesmo contrato. Veja o Cap. 22.

Erros

FimDeEntrada quando a fonte termina; não é um texto vazio.

`formatar_valor(...)` · módulo `coral.formatacao`

O que faz

Formata um valor básico usando sua grafia Coral.

Assinatura

Assinatura para consulta

```
formatar_valor(valor: 'Any') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor representável como texto Coral	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.formatacao importe formatar_valor
mostre formatar_valor(verdadeiro)
```

Resultado

Saída esperada

```
verdadeiro
```

`montar_texto(...)` · módulo `coral.formatacao`

O que faz

Monta texto a partir de vários valores, separador e final.

Assinatura

Assinatura para consulta

```
montar_texto(*partes: 'Any', separador: 'str' = '', final: 'str' = '') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
partes	argumentos variáveis, representáveis como texto Coral	não	sem padrão	Valores convertidos e reunidos no texto final.
separador	valor representável como texto Coral	não	''	Texto que separa as partes.
final	valor representável como texto Coral	não	''	Texto acrescentado após a última parte.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.formatacao importe montar_texto
mostre montar_texto("Coral", 3, separador=" ")
```

Resultado

Saída esperada

```
Coral 3
```

`FonteEntradaSequencial(...)` · módulo `coral.entrada`

O que faz

Fornece linhas pré definidas para repetir uma leitura de entrada em testes.

Construtor

Assinatura para consulta

```
FonteEntradaSequencial(linhas: 'Iterable[str]') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
linhas	iterável de textos	sim	sem padrão	Linhas fornecidas à fonte em ordem de leitura.

Atributos importantes

Atributo	Significado
`linhas`	Coleção de textos fornecida ao criar a fonte.

Métodos principais

Assinatura para consulta

```
ler_linha(self, mensagem: 'str' = '') -> 'str'
```

Retorno

Instância de FonteEntradaSequencial.

Exemplo

Código Coral

```
de coral.entrada importe FonteEntradaSequencial
defina fonte como FonteEntradaSequencial(["Ana"])
mostre fonte.ler_linha()
```

Resultado

Saída esperada

```
Ana
```

Erros

FimDeEntrada quando não há mais linhas.

`usar_fonte_entrada(...)` · módulo `coral.entrada`

O que faz

Escolhe uma fonte de entrada apenas durante um bloco de contexto.

Assinatura

Assinatura para consulta

```
usar_fonte_entrada(fonte: 'FonteEntrada | FonteEntradaFuncao')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
fonte	FonteEntrada	sim	sem padrão	Fonte usada temporariamente no contexto de entrada.

Retorno

Gerenciador de contexto; entrar nele devolve a fonte.

Exemplo

Código Coral

```
de coral.entrada importe FonteEntradaSequencial, usar_fonte_entrada, ler_linha
defina fonte como FonteEntradaSequencial(["Ana"])
com usar_fonte_entrada(fonte) faça
    mostre ler_linha()
fim
```

Resultado

Saída esperada

```
Ana
```

Atenção

Restaura a fonte anterior ao sair do bloco, inclusive em caso de erro.

O que você aprendeu

`ler_linha` devolve texto. Para calcular com uma entrada, converta o valor explicitamente; uma saudação pode usar o texto como foi lido.

Prática

Leia um nome e mostre uma saudação.

Conferir a resposta comentada

Leia uma linha com `ler_linha("Nome: ")` ou a forma natural equivalente. Guarde o texto e use `mostre "Olá, " mais nome`. A conversão é desnecessária para um nome.

Resposta do capítulo 22 no Apêndice B.

23. Conversões

Objetivo de leitura

Converter valores e escolher como tratar uma entrada inválida.

Antes de começar

Conheça tipos e entrada de teclado, dos capítulos 10 e 22.

Uma entrada de teclado é texto. Converter significa produzir um valor com outro tipo antes de usá lo no cálculo. Coral oferece `inteiro`, `decimal`, `texto` e `booleano`.

Código Coral

```
de coral.conversoes importe inteiro, decimal
mostre inteiro("12")
mostre decimal("2,5")
mostre tente converter "inválido" para inteiro, senão nulo
```

Saída esperada

```
12
2.5
None
```

A saída é 12, 2.5 e None, em linhas separadas. A conversão estrita não transforma um número fracionário em inteiro por truncamento e não aceita booleano como inteiro implícito. Em texto decimal, ponto ou vírgula são aceitos, mas sua mistura é ambígua.

Um valor padrão é devolvido quando a conversão falha. Sem padrão, o erro público é `ErroConversao`. A forma natural e a API explícita têm o mesmo contrato. Argumentos são avaliados uma vez; o padrão também é avaliado numa conversão bem sucedida. Consulte o Cap. 18 para tratamento de erros.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`inteiro(...)` · **módulo** `coral.conversoes`

O que faz

Converte um inteiro, um decimal exato ou uma representação textual inteira.

Assinatura

Assinatura para consulta

```
inteiro(valor: 'Any', padrao: 'Any' = <ausente>) -> 'int | Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	inteiro, decimal finito sem fração ou texto de inteiro, exceto booleano	sim	sem padrão	Valor sobre o qual a operação atua.
padrao	qualquer valor	não	<ausente>	Valor devolvido quando a operação não pode obter o resultado normal. Omitir é diferente de fornecer nulo.

Retorno

Inteiro, ou o padrão explicitamente fornecido.

Exemplo

Código Coral

```
de coral.conversoes importe inteiro
mostre inteiro("12")
```

Resultado

Saída esperada

```
12
```

Forma natural

`converta "12" para inteiro` é equivalente. O padrão natural usa `tente converter ..., senão ...`.

Erros

ErroConversao se a entrada é inválida e o padrão foi omitido.

Atenção

Não trunca decimais fracionários e rejeita booleanos. Textos admitem sinal e espaços nas extremidades, sem separadores de milhar. O padrão é retornado como foi fornecido, inclusive nulo.

`decimal(...)` · módulo `coral.conversoes`

O que faz

Converte um número ou texto para decimal finito, aceitando ponto ou vírgula decimal.

Assinatura

Assinatura para consulta

```
decimal(valor: 'Any', padrao: 'Any' = <ausente>) -> 'float | Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	inteiro, decimal finito ou texto numérico, exceto booleano	sim	sem padrão	Valor sobre o qual a operação atua.
padrao	qualquer valor	não	<ausente>	Valor devolvido quando a operação não pode obter o resultado normal. Omitir é diferente de fornecer nulo.

Retorno

Decimal finito, ou o padrão fornecido.

Exemplo

Código Coral

```
de coral.conversoes importe decimal
mostre decimal("2,5")
```

Resultado

Saída esperada

```
2.5
```

Forma natural

`converta "2,5" para decimal` é equivalente. O padrão natural usa `tente converter ..., senão ...`.

Erros

ErroConversao se a entrada é inválida e o padrão foi omitido.

Atenção

Aceita ponto ou vírgula decimal, mas rejeita a mistura dos dois no mesmo texto. Rejeita NaN, infinitos e booleanos. O padrão é retornado como foi fornecido, inclusive nulo.

`texto(...)` · módulo `coral.conversoes`

O que faz

Produz texto com a grafia Coral para os valores básicos.

Assinatura

Assinatura para consulta

```
texto(valor: 'Any', padrao: 'Any' = <ausente>) -> 'str | Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor representável por str, incluindo nulo e booleanos	sim	sem padrão	Valor sobre o qual a operação atua.
padrao	qualquer valor	não	<ausente>	Valor devolvido quando a operação não pode obter o resultado normal. Omitir é diferente de fornecer nulo.

Retorno

Texto, ou o padrão se sua produção falhar.

Exemplo

Código Coral

```
de coral.conversoes importe texto
mostre texto(nulo)
```

Resultado

Saída esperada

```
nulo
```

Forma natural

`converta nulo para texto` é equivalente. O padrão natural usa `tente converter ..., senão ...`.

Erros

ErroConversao se a entrada é inválida e o padrão foi omitido.

Atenção

nulo, verdadeiro e falso usam grafia Coral; os demais valores usam str. Se a representação falha, devolve o padrão fornecido, inclusive nulo, ou levanta ErroConversao quando o padrão foi omitido.

`booleano(...)` · módulo `coral.conversoes`

O que faz

Converte representações explícitas de verdadeiro e falso.

Assinatura

Assinatura para consulta

```
booleano(valor: 'Any', padrao: 'Any' = <ausente>) -> 'bool | Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	booleano, número 0/1 ou texto booleano explícito	sim	sem padrão	Valor sobre o qual a operação atua.
padrao	qualquer valor	não	<ausente>	Valor devolvido quando a operação não pode obter o resultado normal. Omitir é diferente de fornecer nulo.

Retorno

Booleano, ou o padrão fornecido.

Exemplo

Código Coral

```
de coral.conversoes importe booleano
mostre booleano("sim")
```

Resultado

Saída esperada

```
True
```

Forma natural

`converta "sim" para booleano` é equivalente. O padrão natural usa `tente converter ..., senão ...`.

Erros

ErroConversao se a entrada é inválida e o padrão foi omitido.

Atenção

Aceita verdadeiro/falso, números exatamente 0/1 e textos normalizados: verdadeiro,true,sim,1; falso,false,nao,não,0. Não usa a regra Python de qualquer objeto não vazio ser verdadeiro. O padrão é retornado como foi fornecido, inclusive nulo.

O que você aprendeu

Conversão produz outro valor. Sem padrão, uma falha produz `ErroConversao`; com padrão, a operação devolve o valor alternativo indicado.

Prática

Converta 12 e depois uma entrada inválida com padrão nulo.

Conferir a resposta comentada

`inteiro("12")` devolve 12. `inteiro("inválido", nulo)` devolve nulo. Sem o padrão, a entrada inválida produz `ErroConversao`.

Resposta do capítulo 23 no Apêndice B.

24. Texto

Objetivo de leitura

Limpar espaços e transformar textos sem alterar o valor original.

Antes de começar

Conheça textos e chamadas de função.

Texto guarda caracteres e preserva espaços dentro de aspas. Para limpar uma entrada, use `aparar`; para comparações com uma regra de caixa explícita, use `maiusculas` ou `minusculas`.

Código Coral

```
de coral.texto importe aparar, maiusculas
mostre maiusculas(aparar("  Coral  "))
mostre "coral" começa com "co"
```

Saída esperada

```
CORAL
True
```

A saída é CORAL e True, em linhas separadas. Essas operações produzem texto ou respostas novas; não alteram a string original. Prefixos e sufixos são comparados exatamente como fornecidos. Normalização Unicode, caixa e equivalência cultural são decisões distintas; não remova acentos ou normalize nomes automaticamente sem um motivo.

Índices e fatias são ensinados no Cap. 14. Para a família completa, consulte o Volume II, Parte II, seção Biblioteca `coral.texto`.

Textos com várias linhas

Na Coral 1.7.9, três aspas duplas ou três aspas simples delimitam um texto multilinha. O recurso é geral: serve para SQL, HTML, Markdown, mensagens extensas e outros conteúdos textuais. Ele não executa nem interpreta o conteúdo.

Quando a abertura termina a linha e o conteúdo começa na linha seguinte, essa primeira quebra é estrutural. Se o fechamento também fica sozinho em sua linha, a última quebra estrutural é removida. O recuo comum é retirado e o recuo relativo interno é preservado.

Código Coral

```
defina mensagem como """
    primeira linha
      segunda linha
    terceira linha
"""
mostre mensagem
```

Saída esperada

```
primeira linha
  segunda linha
terceira linha
```

O corpo é opaco para a estrutura do programa. Palavras como `fim`, parênteses, comentários aparentes e nomes que parecem imports continuam sendo caracteres do texto. Strings de uma linha continuam com o mesmo comportamento das versões anteriores. Se o fechamento faltar, o editor aponta o início do literal com o diagnóstico P180.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`maiusculas(...)` · **módulo** `coral.texto`

O que faz

Produz a versão em maiúsculas do texto.

Assinatura

Assinatura para consulta

```
maiusculas(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe maiusculas
mostre maiusculas("Coral")
```

Resultado

Saída esperada

```
CORAL
```

Forma natural

`maiúsculas de "Coral"` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`minusculas(...)` · módulo `coral.texto`

O que faz

Produz a versão em minúsculas do texto.

Assinatura

Assinatura para consulta

```
minusculas(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe minusculas
mostre minusculas("Coral")
```

Resultado

Saída esperada

```
coral
```

Forma natural

`minúsculas de "Coral"` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`aparar(...)` · módulo `coral.texto`

O que faz

Retira espaços em branco nas extremidades do texto.

Assinatura

Assinatura para consulta

```
aparar(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe aparar
mostre aparar("  Coral  ")
```

Resultado

Saída esperada

```
Coral
```

Forma natural

`apare o texto de " Coral "` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`comeca_com(...)` · módulo `coral.texto`

O que faz

Verifica se o texto começa com o prefixo informado.

Assinatura

Assinatura para consulta

```
comeca_com(valor, prefixo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
prefixo	qualquer valor convertido com str	sim	sem padrão	Texto procurado no início.

Retorno

Booleano.

Exemplo

Código Coral

```
de coral.texto importe comeca_com
mostre comeca_com("coral", "co")
```

Resultado

Saída esperada

```
True
```

Forma natural

`"coral" começa com "co"` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`termina_com(...)` · módulo `coral.texto`

O que faz

Verifica se o texto termina com o sufixo informado.

Assinatura

Assinatura para consulta

```
termina_com(valor, sufixo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
sufixo	qualquer valor convertido com str	sim	sem padrão	Texto procurado no final.

Retorno

Booleano.

Exemplo

Código Coral

```
de coral.texto importe termina_com
mostre termina_com("coral", "al")
```

Resultado

Saída esperada

```
True
```

Forma natural

`"coral" termina com "al"` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`substituir(...)` · módulo `coral.texto`

O que faz

Produz um texto substituindo ocorrências do trecho informado.

Assinatura

Assinatura para consulta

```
substituir(valor, antigo, novo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
antigo	qualquer valor convertido com str	sim	sem padrão	Trecho a substituir.
novo	qualquer valor convertido com str	sim	sem padrão	Texto que ocupa o lugar do trecho encontrado.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe substituir
mostre substituir("casa", "a", "o")
```

Resultado

Saída esperada

```
coso
```

Forma natural

`substitua "a" por "o" em "casa"` é a forma natural equivalente para estes argumentos.

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`dividir(...)` · módulo `coral.texto`

O que faz

Divide o texto em partes.

Assinatura

Assinatura para consulta

```
dividir(valor, separador=None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
separador	texto não vazio ou nulo	não	None	Texto que separa as partes.

Retorno

Lista de textos.

Exemplo

Código Coral

```
de coral.texto importe dividir
mostre dividir("a,b", ",")
```

Resultado

Saída esperada

```
['a', 'b']
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`juntar(...)` · módulo `coral.texto`

O que faz

Reúne textos com um separador.

Assinatura

Assinatura para consulta

```
juntar(separador, valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
separador	qualquer valor convertido com str	sim	sem padrão	Texto que separa as partes.
valores	iterável de valores convertidos com str	sim	sem padrão	Coleção de entrada.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.texto importe juntar
mostre juntar(",", ["a", "b"])
```

Resultado

Saída esperada

```
a,b
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`linhas(...)` · módulo `coral.texto`

O que faz

Obtém as linhas de um texto.

Assinatura

Assinatura para consulta

```
linhas(valor, *, manter_quebras=False)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
manter_quebras	booleano	não	False	Se verdadeiro, mantém os caracteres de quebra de linha nas partes devolvidas.

Retorno

Lista de textos.

Exemplo

Código Coral

```
de coral.texto importe linhas
mostre linhas("a\nb")
```

Resultado

Saída esperada

```
['a', 'b']
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`localizar(...)` · módulo `coral.texto`

O que faz

Procura um trecho no texto.

Assinatura

Assinatura para consulta

```
localizar(valor, trecho, inicio=0)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
trecho	qualquer valor convertido com str	sim	sem padrão	Texto procurado a partir do índice inicial.
inicio	valor convertível com int	não	0	Índice inicial da busca.

Retorno

Índice inteiro; menos um quando não encontrado.

Exemplo

Código Coral

```
de coral.texto importe localizar
mostre localizar("coral", "ra")
```

Resultado

Saída esperada

```
2
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`contar(...)` · módulo `coral.texto`

O que faz

Conta ocorrências de um trecho.

Assinatura

Assinatura para consulta

```
contar(valor, trecho, inicio=0, fim=None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
trecho	qualquer valor convertido com str	sim	sem padrão	Texto cujas ocorrências serão contadas.
inicio	valor convertível com int	não	0	Índice inicial da busca.
fim	valor convertível com int ou nulo	não	None	Limite final da busca.

Retorno

Quantidade inteira.

Exemplo

Código Coral

```
de coral.texto importe contar
mostre contar("banana", "a")
```

Resultado

Saída esperada

```
3
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`remover_prefixo(...)` · módulo `coral.texto`

O que faz

Retira o prefixo se estiver presente.

Assinatura

Assinatura para consulta

```
remover_prefixo(valor, prefixo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
prefixo	qualquer valor convertido com str	sim	sem padrão	Texto procurado no início.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe remover_prefixo
mostre remover_prefixo("coral", "co")
```

Resultado

Saída esperada

```
ral
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`remover_sufixo(...)` · módulo `coral.texto`

O que faz

Retira o sufixo se estiver presente.

Assinatura

Assinatura para consulta

```
remover_sufixo(valor, sufixo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
sufixo	qualquer valor convertido com str	sim	sem padrão	Texto procurado no final.

Retorno

Texto novo.

Exemplo

Código Coral

```
de coral.texto importe remover_sufixo
mostre remover_sufixo("coral", "al")
```

Resultado

Saída esperada

```
cor
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

`normalizar_unicode(...)` · módulo `coral.texto`

O que faz

Normaliza a representação Unicode com a forma escolhida.

Assinatura

Assinatura para consulta

```
normalizar_unicode(valor, forma='NFC')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	qualquer valor convertido com str	sim	sem padrão	Valor sobre o qual a operação atua.
forma	texto NFC, NFD, NFKC ou NFKD	não	'NFC'	Forma de normalização Unicode.

Retorno

Texto normalizado.

Exemplo

Código Coral

```
de coral.texto importe normalizar_unicode
mostre normalizar_unicode("Coral")
```

Resultado

Saída esperada

```
Coral
```

Atenção

O texto original é preservado. A comparação respeita o conteúdo e a caixa fornecidos.

O que você aprendeu

As operações retornam novos valores. Retirar espaços, mudar caixa e normalizar caracteres são decisões diferentes.

Prática

Remova espaços externos de um texto e converta para maiúsculas.

Conferir a resposta comentada

`aparar(" Coral ")` devolve `"Coral"`; passar esse resultado para `maiusculas` devolve `"CORAL"`. O texto original não é modificado.

Resposta do capítulo 24 no Apêndice B.

25. Coleções

Objetivo de leitura

Ordenar, contar e remover repetições de coleções.

Antes de começar

Conheça as estruturas do capítulo 14.

Uma coleção agrupa valores. Além de acessar seus itens, podemos ordenar, remover repetições, contar ocorrências e consultar o primeiro elemento.

Código Coral

```
de coral.colecoes importe unicos, ordenar, contar

defina valores como [3, 1, 3, 2]
mostre unicos(valores)
mostre ordenar(valores)
mostre contar(valores)
mostre valores
```

Saída esperada

```
[3, 1, 2]
[1, 2, 3, 3]
{3: 2, 1: 1, 2: 1}
[3, 1, 3, 2]
```

A saída é [3, 1, 2], [1, 2, 3, 3], {3: 2, 1: 1, 2: 1} e [3, 1, 3, 2], em linhas separadas. A origem permanece igual. `unicos` mantém a primeira ocorrência e precisa de itens utilizáveis como chaves. `contar` produz um dicionário de frequências; `primeiro` devolve nulo quando a coleção é vazia.

No Cap. 14 você encontra a estrutura das coleções; aqui encontra as operações. Tipos parametrizados ficam no Cap. 10.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`ordenar(...)` · **módulo** `coral.colecoes`

O que faz

Ordena os valores e produz uma nova lista.

Assinatura

Assinatura para consulta

```
ordenar(valores, reverso=False)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de itens comparáveis	sim	sem padrão	Coleção de entrada.
reverso	booleano	não	False	Se verdadeiro, inverte a ordem da classificação.

Retorno

Lista ordenada.

Exemplo

Código Coral

```
de coral.colecoes importe ordenar
mostre ordenar([3, 1, 2])
```

Resultado

Saída esperada

```
[1, 2, 3]
```

Forma natural

`ordene os valores de [3, 1, 2]` é equivalente para estes argumentos.

Atenção

Retorna uma lista nova; não modifica a coleção original. Requer que os itens possam ser comparados entre si.

`unicos(...)` · módulo `coral.colecoes`

O que faz

Conserva a primeira ocorrência de cada valor.

Assinatura

Assinatura para consulta

```
unicos(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de itens hashable	sim	sem padrão	Coleção de entrada.

Retorno

Lista sem duplicatas.

Exemplo

Código Coral

```
de coral.colecoes importe unicos
mostre unicos([3, 1, 3, 2])
```

Resultado

Saída esperada

```
[3, 1, 2]
```

Forma natural

`os valores únicos de [3, 1, 3, 2]` é equivalente para estes argumentos.

Atenção

A origem não é alterada. Chaves e elementos usados na eliminação de duplicatas ou contagem precisam ser utilizáveis como chaves.

`contar(...)` · módulo `coral.colecoes`

O que faz

Conta as ocorrências de cada valor.

Assinatura

Assinatura para consulta

```
contar(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de itens hashable	sim	sem padrão	Coleção de entrada.

Retorno

Dicionário de frequências.

Exemplo

Código Coral

```
de coral.colecoes importe contar
mostre contar([3, 1, 3])
```

Resultado

Saída esperada

```
{3: 2, 1: 1}
```

Forma natural

`conte as ocorrências de [3, 1, 3]` é equivalente para estes argumentos.

Atenção

A origem não é alterada. Chaves e elementos usados na eliminação de duplicatas ou contagem precisam ser utilizáveis como chaves.

`primeiro(...)` · módulo `coral.colecoes`

O que faz

Obtém o primeiro elemento ou o padrão se não houver elemento.

Assinatura

Assinatura para consulta

```
primeiro(valores, padrao=None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável	sim	sem padrão	Coleção de entrada.
padrao	qualquer valor	não	None	Valor devolvido quando a operação não pode obter o resultado normal. Omitir é diferente de fornecer nulo.

Retorno

Elemento ou padrão; por padrão, nulo.

Exemplo

Código Coral

```
de coral.colecoes importe primeiro
mostre primeiro([])
```

Resultado

Saída esperada

```
None
```

Forma natural

`o primeiro de []` é equivalente para estes argumentos.

Atenção

A origem não é alterada. Chaves e elementos usados na eliminação de duplicatas ou contagem precisam ser utilizáveis como chaves.

`agrupar_pares(...)` · módulo `coral.colecoes`

O que faz

Combina chaves e valores por posição para construir um dicionário.

Assinatura

Assinatura para consulta

```
agrupar_pares(chaves, valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
chaves	iterável de chaves hashable	sim	sem padrão	Coleção de chaves.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de valores	sim	sem padrão	Coleção de valores, combinada por posição com as chaves.

Retorno

Dicionário com os pares; chaves repetidas mantêm o último valor.

Exemplo

Código Coral

```
de coral.colecoes importe agrupar_pares
mostre agrupar_pares(["a", "b"], [1, 2])
```

Resultado

Saída esperada

```
{'a': 1, 'b': 2}
```

Atenção

Consome chaves e valores até terminar o iterável mais curto. Chaves repetidas ficam com o último valor correspondente.

O que você aprendeu

`unicos` conserva a primeira ocorrência; `ordenar` organiza valores; `contar` produz frequências. Confira na ficha se a operação altera a origem.

Prática

Remova valores repetidos de [3, 1, 3, 2] e ordene o resultado.

Conferir a resposta comentada

`unicos` produz [3, 1, 2], conservando a primeira ocorrência. `ordenar` produz [1, 2, 3]. A lista original permanece igual.

Resposta do capítulo 25 no Apêndice B.

26. Arquivos e caminhos

Objetivo de leitura

Gravar e ler texto, reconhecendo a pasta usada por um caminho relativo.

Antes de começar

Conheça textos, funções e tratamento de erros.

Gravar e ler dados

Arquivo: `Exemplos/Citados/Dados/02_arquivos_json_e_caminhos.coral`.

Código Coral

```
de coral.arquivos importe escrever_texto e ler_texto
execute escrever_texto("mensagem.txt", "Olá, arquivo!")
defina conteudo como ler_texto("mensagem.txt")
mostre conteudo
```

Saída esperada

```
Olá, arquivo!
```

O caminho relativo é resolvido a partir da pasta de trabalho, não necessariamente da pasta em que o arquivo `.coral` está salvo. Se você executa de outra pasta, um caminho como `mensagem.txt` aponta a essa outra pasta. Para um projeto, comece pelo terminal na raiz e organize dados em diretórios conhecidos.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`ler_texto(...)` · **módulo** `coral.arquivos`

O que faz

Lê todo o texto de um arquivo.

Assinatura

Assinatura para consulta

```
ler_texto(caminho, codificacao='utf-8')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho do arquivo local, interpretado na pasta de trabalho.
codificacao	texto de codificação	não	'utf-8'	Codificação de texto usada na leitura ou escrita.

Retorno

Texto.

Exemplo

Código Coral

```
de coral.arquivos importe ler_texto
de coral.arquivos importe escrever_texto
execute escrever_texto("dados.txt", "Coral")
mostre ler_texto("dados.txt")
```

Resultado

Saída esperada

```
Coral
```

Forma natural

`leia o texto de "dados.txt"` é equivalente.

Erros

A leitura pode produzir FileNotFoundError ou PermissionError. Consulte o Cap. 18.

Atenção

O exemplo cria seu próprio arquivo na pasta de trabalho.

`existe(...)` · módulo `coral.arquivos`

O que faz

Verifica se o caminho existe, usando a normalização oficial.

Assinatura

Assinatura para consulta

```
existe(caminho)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou os.PathLike[str] nativo	sim	sem padrão	Caminho do arquivo local, interpretado na pasta de trabalho.

Retorno

Booleano.

Exemplo

Código Coral

```
de coral.arquivos importe existe
de coral.arquivos importe escrever_texto
execute escrever_texto("dados.txt", "Coral")
mostre existe("dados.txt")
```

Resultado

Saída esperada

```
True
```

Forma natural

`existe o caminho "dados.txt"` é equivalente.

Erros

A leitura pode produzir FileNotFoundError ou PermissionError. Consulte o Cap. 18.

Atenção

O exemplo cria seu próprio arquivo na pasta de trabalho.

O que você aprendeu

Um caminho relativo parte da pasta de trabalho. Permissões, codificação e existência do arquivo fazem parte do uso real da operação.

Prática

Escreva Coral num arquivo e leia o conteúdo.

Conferir a resposta comentada

Use `escrever_texto("dados.txt", "Coral")` e `ler_texto("dados.txt")`. A escrita tem efeito no sistema de arquivos e a leitura devolve texto. Considere permissão, codificação e pasta de trabalho.

Resposta do capítulo 26 no Apêndice B.

27. JSON e serialização

Objetivo de leitura

Ler e escrever arquivos e validar registros externos com conversão explícita e diagnósticos por campo.

Antes de começar

Conheça dicionários, listas e textos.

Serializar estruturas

Arquivo: `Exemplos/Citados/Dados/02_arquivos_json_e_caminhos.coral`.

Código Coral

```
de coral.json importe para_json e de_json
defina original como {"nome": "Ana", "nota": 9}
defina documento como para_json(original)
defina recuperado como de_json(documento)
mostre recuperado["nome"]
mostre recuperado["nota"]
```

Saída esperada

```
Ana
9
```

JSON preserva estruturas básicas como objetos, listas, textos, números, booleanos e ausência. Ele não serializa automaticamente qualquer classe personalizada. `de_json` interpreta dados JSON; não executa código Coral.

Processar arquivos e CSV aos poucos

Quando os dados crescem, um iterador permite processar um registro de cada vez. Um lote reúne uma quantidade limitada de registros para uma etapa de trabalho. Isso reduz a necessidade de guardar todos os dados na memória, embora o tamanho de cada registro ainda importe.

Código Coral

```
de coral.arquivos importe escrever_texto e ler_lotes_csv
execute escrever_texto("dados.csv", "nome,valor\nAna,10\nBia,20\nCaio,30\n")
com ler_lotes_csv("dados.csv", tamanho_lote=2) como lotes faça
    para cada lote em lotes faça
        mostre tamanho de lote
    fim
fim
```

Saída esperada

```
2
1
```

A abertura acontece no primeiro avanço, não na construção do leitor. O leitor é de passagem única e fecha no fim ou em erro. Para parar antes, use `com` ou `fechar()`; não dependa da coleta de lixo. Linhas de texto conservam suas quebras; registros CSV podem conter vírgulas e quebras dentro de aspas. Cabeçalhos devem ser únicos, não vazios, e as linhas precisam manter a largura.

Os escritores exigem textos, consomem gradualmente e substituem o destino. Não inserem quebras em `escrever_linhas` nem convertem valores implicitamente em `escrever_csv`; um erro pode deixar destino parcial. Um leitor usado como entrada de um escritor deve continuar sob o `com` externo do chamador. `coral.formatos.de_csv` continua materializando dados, conforme seu contrato anterior.

Veja o projeto executável `Exemplos/Projetos/Importador_Gradual`: lotes de dois registros, filtro `ativo` igual a `sim`, nomes aparados e saída `nome,valor`. As fichas deste capítulo detalham cada contrato. A referência completa está no Volume II e em `Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md`.

Para sequências de campos, nomes não são inferíveis: sem colunas explícitas, não é emitido cabeçalho mesmo com cabecalho=True. Objetos permitem inferir nomes na ordem do primeiro registro; colunas explícitas definem nomes e ordem.

Validar registros externos com esquemas

Na Coral 1.7.8, `coral.esquemas` descreve contratos de cadastro, pesquisa e inventário com quatro chamadas explícitas: `campo`, `esquema`, `validar_registro` e `validar_registros`. Nenhuma forma natural nova foi criada. A validação aceita ou relata dados; não converte textos, não preenche valores padrão e não altera a entrada.

Ausência significa que a chave não existe. Nulo significa que a chave existe com `nulo`. Um campo opcional pode estar ausente e ainda recusar nulo; um campo obrigatório e anulável exige a chave, mas aceita nulo. Inteiro rejeita booleanos; decimal aceita números finitos e rejeita booleanos. Limites são inclusivos.

Código Coral

```
de coral.esquemas importe campo, esquema, validar_registro
de coral.conversoes importe inteiro

defina cadastro como esquema({"idade": campo("inteiro", minimo=0), "apelido":
  campo("texto", obrigatorio=falso)}, permitir_extras=falso)
defina bruto como {"idade": "18"}
mostre validar_registro(bruto, cadastro)["erros"][0]["regra"]
defina convertido como {"idade": inteiro(bruto["idade"])}
mostre validar_registro(convertido, cadastro)["valido"]
mostre validar_registro({"idade": 18, "apelido": nulo}, cadastro)["erros"][0]["regra"]
```

Saída esperada

```
tipo
True
nulo
```

A conversão `inteiro` é explícita e pode falhar antes da validação; preserve esse erro no relatório de importação. JSON também precisa validar dados após `de_json`. Não confunda texto CSV vazio, chave ausente e nulo JSON. O projeto `Exemplos/Projetos/Importador_Validado` combina conversão, validação e relatório JSON.

Código Coral

```
de coral.esquemas importe campo, esquema, validar_registros

defina pesquisa como esquema({"medida": campo("decimal", minimo=0), "grupo":
  campo("texto", permitidos=["controle", "ensaio"])})
defina resultado como validar_registros([{"medida": -1, "grupo": "outro"}, {"medida": 2,
   "grupo": "ensaio"}], pesquisa, limite_erros=1, linha_inicial=2, origem="pesquisa.csv")
mostre resultado["erros"][0]["campo"]
mostre resultado["erros"][0]["regra"]
mostre resultado["erros"][0]["registro"]
mostre resultado["erros"][0]["linha"]
mostre resultado["erros"][0]["origem"]
mostre resultado["verificados"]
mostre resultado["interrompido"]
```

Saída esperada

```
medida
minimo
1
2
pesquisa.csv
1
True
```

Erros contêm `campo`, `regra`, `mensagem`, `registro`, `linha` e `origem`, sem copiar valores sensíveis. A ordem declarada dos campos determina a ordem do diagnóstico; extras recusados vêm depois, na ordem do registro. Tipo incompatível não avalia limites desse campo. `modo="primeiro"` para no primeiro erro; `modo="coletar"` para ao atingir `limite_erros`, inteiro positivo que não aceita booleano. `interrompido` sinaliza a política, sem alegar que o restante foi examinado. `verificados` conta somente registros consumidos.

`linha_inicial` informa a linha lógica do primeiro registro e os seguintes somam 1; o índice de registro começa em 1. CSV com campos multilinha não permite deduzir linha física pelo

índice. `origem` é fornecida pelo chamador. A validação consome uma passagem e guarda apenas erros, sem fechar iteradores alheios. Ao usar `ler_csv`, mantenha o leitor dentro de `com`, inclusive quando o limite interromper a validação. Esquema e opções inválidos falham antes do consumo; falhas de leitura/conversão propagam ao chamador.

Consulte as quatro fichas completas neste capítulo, a referência no Volume II e `Documentacao/Usuario/Coral_Esquemas_1_7_8.md`.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`para_json(...)` · **módulo** `coral.json`

O que faz

Transforma valores compatíveis com JSON em texto JSON.

Assinatura

Assinatura para consulta

```
para_json(valor, identar=2)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	valor serializável pelo json do Python	sim	sem padrão	Valor sobre o qual a operação atua.
identar	inteiro, texto ou nulo	não	2	Quantidade de espaços de recuo; nulo produz a representação sem recuo.

Retorno

Texto JSON.

Exemplo

Código Coral

```
de coral.json importe para_json
mostre para_json([1, 2])
```

Resultado

Saída esperada

```
[
  1,
  2
]
```

Forma natural

`serialize json de [1, 2]` é equivalente.

Erros

TypeError para objetos não serializáveis.

Atenção

Preserva Unicode. Chaves compatíveis não textuais são convertidas para texto. NaN e Infinity são permitidos pelo serializador padrão, embora consumidores de JSON estrito possam recusá-los.

`de_json(...)` · módulo `coral.json`

O que faz

Interpreta um texto JSON e reconstrói seus valores.

Assinatura

Assinatura para consulta

```
de_json(texto)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
texto	str, bytes ou bytearray com JSON	sim	sem padrão	Texto a consultar ou transformar.

Retorno

Valor compatível com JSON: lista, dicionário, texto, número, booleano ou nulo.

Exemplo

Código Coral

```
de coral.json importe de_json
mostre de_json("[1, 2]")
```

Resultado

Saída esperada

```
[1, 2]
```

Forma natural

`interprete json de "[1, 2]"` é equivalente.

Erros

JSONDecodeError para texto inválido.

`ler_linhas(...)` · módulo `coral.arquivos`

O que faz

Iterador de textos, preservando terminações de linha. Processamento gradual sem materializar a entrada inteira.

Assinatura

Assinatura para consulta

```
ler_linhas(caminho, codificacao='utf-8') -> 'Iterator[str]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou caminho de arquivo	sim	sem padrão	Caminho do arquivo, relativo à pasta de trabalho ou absoluto.
codificacao	texto	não	'utf-8'	Codificação textual usada na abertura; padrão utf-8.

Retorno

Iterador de textos, preservando terminações de linha.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_texto e ler_linhas
execute escrever_texto("linhas.txt", "Ana\nBia")
com ler_linhas("linhas.txt") como linhas faça
    para cada linha em linhas faça
        mostre tamanho de linha
    fim
fim
```

Resultado

Saída esperada

```
4
3
```

Erros

Erros de caminho, permissão ou codificação; TypeError para tipos inválidos e ValueError para opções/dados fora do contrato. CSV malformado é rejeitado.

Atenção

Leitura preguiçosa, passagem única; EOF e erro fecham. Use com ou fechar()/close() ao interromper antes. CSV exige cabeçalhos únicos não vazios e largura coerente; aceita aspas, separadores e campos multilinha.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md.

`ler_csv(...)` · módulo `coral.arquivos`

O que faz

Iterador de objetos com chaves e valores textuais se cabecalho=True; listas de textos caso contrário. Processamento gradual sem materializar a entrada inteira.

Assinatura

Assinatura para consulta

```
ler_csv(caminho, codificacao='utf-8', *, delimitador=',', cabecalho=True) ->
  'Iterator[dict[str, str] | list[str]]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou caminho de arquivo	sim	sem padrão	Caminho do arquivo, relativo à pasta de trabalho ou absoluto.
codificacao	texto	não	'utf-8'	Codificação textual usada na abertura; padrão utf-8.
delimitador	texto de um caractere	não	','	Caractere separador CSV, inclusive entre campos; padrão vírgula.
cabecalho	booleano	não	True	Se verdadeiro, lê ou escreve a linha de nomes das colunas.

Retorno

Iterador de objetos com chaves e valores textuais se cabecalho=True; listas de textos caso contrário.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_texto e ler_csv
execute escrever_texto("dados.csv", "nome,valor\nAna,10\n")
com ler_csv("dados.csv") como registros faça
    para cada registro em registros faça
        mostre registro["nome"]
    fim
fim
```

Resultado

Saída esperada

```
Ana
```

Erros

Erros de caminho, permissão ou codificação; TypeError para tipos inválidos e ValueError para opções/dados fora do contrato. CSV malformado é rejeitado.

Atenção

Leitura preguiçosa, passagem única; EOF e erro fecham. Use com ou fechar()/close() ao interromper antes. CSV exige cabeçalhos únicos não vazios e largura coerente; aceita aspas, separadores e campos multilinha.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md.

`ler_lotes_csv(...)` · módulo `coral.arquivos`

O que faz

Iterador de listas de registros; lote final possivelmente menor. Processamento gradual sem materializar a entrada inteira.

Assinatura

Assinatura para consulta

```
ler_lotes_csv(caminho, tamanho_lote=100, codificacao='utf-8', *, delimitador=',',
  cabecalho=True) -> 'Iterator[list[dict[str, str] | list[str]]]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou caminho de arquivo	sim	sem padrão	Caminho do arquivo, relativo à pasta de trabalho ou absoluto.
tamanho_lote	inteiro positivo, não booleano	não	100	Máximo de registros por lote: inteiro positivo, exceto booleano.
codificacao	texto	não	'utf-8'	Codificação textual usada na abertura; padrão utf-8.
delimitador	texto de um caractere	não	','	Caractere separador CSV, inclusive entre campos; padrão vírgula.
cabecalho	booleano	não	True	Se verdadeiro, lê ou escreve a linha de nomes das colunas.

Retorno

Iterador de listas de registros; lote final possivelmente menor.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_texto e ler_lotes_csv
execute escrever_texto("dados.csv", "nome,valor\nAna,10\nBia,20\nCaio,30\n")
com ler_lotes_csv("dados.csv", tamanho_lote=2) como lotes faça
    para cada lote em lotes faça
        mostre tamanho de lote
    fim
fim
```

Resultado

Saída esperada

```
2
1
```

Erros

Erros de caminho, permissão ou codificação; TypeError para tipos inválidos e ValueError para opções/dados fora do contrato. CSV malformado é rejeitado.

Atenção

Leitura preguiçosa, passagem única; EOF e erro fecham. Use com ou fechar()/close() ao interromper antes. CSV exige cabeçalhos únicos não vazios e largura coerente; aceita aspas, separadores e campos multilinha.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md.

`escrever_linhas(...)` · módulo `coral.arquivos`

O que faz

Nulo; o destino recebe os textos sem quebras inseridas. Processamento gradual sem materializar a entrada inteira.

Assinatura

Assinatura para consulta

```
escrever_linhas(caminho, linhas: 'Iterable[str]', codificacao='utf-8') -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou caminho de arquivo	sim	sem padrão	Caminho do arquivo, relativo à pasta de trabalho ou absoluto.
linhas	iterável de textos	sim	sem padrão	Iterável de textos com as terminações desejadas; sem conversão implícita.
codificacao	texto	não	'utf-8'	Codificação textual usada na abertura; padrão utf-8.

Retorno

Nulo; o destino recebe os textos sem quebras inseridas.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_linhas e ler_texto
execute escrever_linhas("saida.txt", ["Ana\n", "Bia"])
mostre ler_texto("saida.txt")
```

Resultado

Saída esperada

```
Ana
Bia
```

Erros

Erros de caminho, permissão ou codificação; TypeError para tipos inválidos e ValueError para opções/dados fora do contrato. CSV malformado é rejeitado.

Atenção

Escrita substitui o destino e pode ficar parcial em erro; campos precisam ser textos. O chamador fecha leitores de entrada com com externo. Objetos CSV exigem chaves exatas e sequências largura exata.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md.

`escrever_csv(...)` · módulo `coral.arquivos`

O que faz

Nulo; o destino recebe CSV com ordem de colunas explícita ou inferida. Processamento gradual sem materializar a entrada inteira.

Assinatura

Assinatura para consulta

```
escrever_csv(caminho, registros: 'Iterable[Mapping[str, str] | Sequence[str]]',
  codificacao='utf-8', *, delimitador=',', colunas=None, cabecalho=True) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou caminho de arquivo	sim	sem padrão	Caminho do arquivo, relativo à pasta de trabalho ou absoluto.
registros	iterável de objetos ou sequências textuais	sim	sem padrão	Iterável de objetos texto/texto ou sequências de textos de largura uniforme.
codificacao	texto	não	'utf-8'	Codificação textual usada na abertura; padrão utf-8.
delimitador	texto de um caractere	não	','	Caractere separador CSV, inclusive entre campos; padrão vírgula.
colunas	sequência de textos ou nulo	não	None	Sequência ordenada de nomes; nulo infere nomes do primeiro objeto.
cabecalho	booleano	não	True	Se verdadeiro, lê ou escreve a linha de nomes das colunas.

Retorno

Nulo; o destino recebe CSV com ordem de colunas explícita ou inferida.

Exemplo

Código Coral

```
de coral.arquivos importe escrever_csv e ler_csv
execute escrever_csv("saida.csv", [{"nome": "Ana", "valor": "10"}], colunas=["nome",
  "valor"])
com ler_csv("saida.csv") como registros faça
    para cada registro em registros faça
        mostre registro["nome"]
        mostre registro["valor"]
    fim
fim
```

Resultado

Saída esperada

```
Ana
10
```

Erros

Erros de caminho, permissão ou codificação; TypeError para tipos inválidos e ValueError para opções/dados fora do contrato. CSV malformado é rejeitado.

Atenção

Escrita substitui o destino e pode ficar parcial em erro; campos precisam ser textos. O chamador fecha leitores de entrada com com externo. Objetos CSV exigem chaves exatas e sequências largura exata. Para sequências de campos, nomes não são inferíveis: sem colunas explícitas, não é emitido cabeçalho mesmo com cabecalho=True. Objetos permitem inferir nomes na ordem do primeiro registro; colunas explícitas definem nomes e ordem.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Arquivos_CSV_1_7_7.md.

`campo(...)` · **módulo** `coral.esquemas`

O que faz

Cria a especificação declarativa de um campo; não converte nem preenche dados.

Assinatura

Assinatura para consulta

```
campo(tipo: 'str', *, obrigatorio: 'bool' = True, anulavel: 'bool' = False, minimo: 'int
   | float | None' = None, maximo: 'int | float | None' = None, tamanho_minimo: 'int |
  None' = None, tamanho_maximo: 'int | None' = None, permitidos: 'Iterable | None' = None)
   -> 'dict'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tipo	texto	sim	sem padrão	Tipo canônico: inteiro, decimal, texto, booleano, lista ou dicionario.
obrigatorio	booleano	não	True	Exige presença da chave; verdadeiro por padrão.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
anulavel	booleano	não	False	Aceita nulo quando a chave está presente; falso por padrão.
minimo	inteiro ou decimal finito ou nulo	não	None	Limite numérico inferior inclusivo; nulo desativa.
maximo	inteiro ou decimal finito ou nulo	não	None	Limite numérico superior inclusivo; nulo desativa.
tamanho_minimo	inteiro não negativo, exceto booleano, ou nulo	não	None	Comprimento mínimo inclusivo; nulo desativa.
tamanho_maximo	inteiro não negativo, exceto booleano, ou nulo	não	None	Comprimento máximo inclusivo; nulo desativa.
permitidos	iterável de valores ou nulo	não	None	Iterável de valores aceitos; nulo desativa, coleção vazia não aceita nenhum valor não nulo.

Retorno

Dicionário de especificação, copiado e novamente conferido pelo consumidor.

Exemplo

Código Coral

```
de coral.esquemas importe campo
defina idade como campo("inteiro", minimo=0, maximo=120)
mostre idade["tipo"]
```

Resultado

Saída esperada

```
inteiro
```

Forma natural

Não há forma natural específica para esta API. Use a chamada explícita importada de coral.esquemas; não há nova gramática.

Erros

TypeError para tipos de opções inválidos, incluindo limites numéricos não finitos ou booleanos; ValueError para tipo canônico desconhecido, tamanho negativo, regras incompatíveis ou limites contraditórios.

Atenção

Inteiro rejeita booleanos; decimal aceita inteiro/decimal finitos e rejeita booleanos. Regras numéricas só servem para inteiro/decimal; comprimento só para texto/lista/dicionario. Limites são inclusivos. Ausência e nulo são independentes: opcional permite ausência, anulavel permite nulo presente. Nulo aceito não percorre regras de valor. Especificações não retêm referências mutáveis do chamador.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Esquemas_1_7_8.md; Exemplos/Projetos/Importador_Validado.

`esquema(...)` · módulo `coral.esquemas`

O que faz

Compõe campos em um esquema de registro com ordem declarada de diagnóstico.

Assinatura

Assinatura para consulta

```
esquema(campos: 'Mapping[str, dict]', *, permitir_extras: 'bool' = True) -> 'dict'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
campos	mapeamento texto/especificação de campo	sim	sem padrão	Mapeamento de nomes textuais não vazios para especificações de campo.
permitir_extras	booleano	não	True	Permite chaves fora da declaração; verdadeiro por padrão.

Retorno

Dicionário de esquema com especificações copiadas; sem dados do registro.

Exemplo

Código Coral

```
de coral.esquemas importe campo, esquema, validar_registro
defina cadastro como esquema({"nome": campo("texto", tamanho_minimo=1)},
  permitir_extras=falso)
mostre validar_registro({"nome": "Ana"}, cadastro)["valido"]
```

Resultado

Saída esperada

```
True
```

Forma natural

Não há forma natural específica para esta API. Use a chamada explícita importada de coral.esquemas; não há nova gramática.

Erros

TypeError para campos ou opções de tipo inválido; ValueError para nome vazio ou especificação inválida, incluindo regras e limites contraditórios.

Atenção

A ordem dos campos declarados define a ordem dos erros; extras recusados aparecem depois, na ordem do registro. Não há campos aninhados, preenchimento padrão, conversão

ou mutação dos dados. Use especificações produzidas por campo. A conferência ocorre também ao consumir o esquema.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Esquemas_1_7_8.md; Exemplos/Projetos/Importador_Validado.

`validar_registro(...)` · módulo `coral.esquemas`

O que faz

Confere um registro e relata falhas sem incluir o valor sensível nem modificar a entrada.

Assinatura

Assinatura para consulta

```
validar_registro(registro: 'Mapping', esquema: 'Mapping', *, indice: 'int' = 1, linha:
  'int | None' = None, origem: 'str | None' = None, modo: 'str' = 'coletar', limite_erros:
   'int' = 100) -> 'dict'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
registro	mapeamento	sim	sem padrão	Mapeamento com os dados; outra entrada resulta em erro de regra registro.
esquema	mapeamento de esquema	sim	sem padrão	Esquema criado por esquema e reconferido no consumo.
indice	inteiro positivo, exceto booleano	não	1	Identificador lógico do registro, inteiro positivo base 1; padrão 1.
linha	inteiro positivo, exceto booleano, ou nulo	não	None	Linha informada pelo chamador, inteiro positivo ou nulo; não inferida.
origem	texto ou nulo	não	None	Rótulo textual opcional fornecido pelo chamador; nunca inferido.
modo	texto: primeiro ou coletar	não	'coletar'	primeiro interrompe no primeiro erro; coletar usa o limite global.
limite_erros	inteiro positivo, exceto booleano	não	100	Máximo positivo de erros; inteiro excluindo booleano, padrão 100.

Retorno

Dicionário com valido (booleano), erros (lista) e interrompido (booleano). Cada erro contém campo, regra, mensagem, registro (índice base 1), linha e origem; não contém valor.

Exemplo

Código Coral

```
de coral.esquemas importe campo, esquema, validar_registro
defina contrato como esquema({"apelido": campo("texto", obrigatorio=falso)})
mostre validar_registro({}, contrato)["valido"]
defina resultado como validar_registro({"apelido": nulo}, contrato, indice=3, linha=8,
  origem="cadastro.json")
mostre resultado["erros"][0]["regra"]
mostre resultado["erros"][0]["registro"]
mostre resultado["erros"][0]["linha"]
```

Resultado

Saída esperada

```
True
nulo
3
8
```

Forma natural

Não há forma natural específica para esta API. Use a chamada explícita importada de coral.esquemas; não há nova gramática.

Erros

TypeError/ValueError para esquema ou opções inválidos. Dados inválidos são diagnósticos, não exceções: registro, ausente, nulo, tipo, minimo, maximo, tamanho_minimo, tamanho_maximo, permitido e extra.

Atenção

Tipo errado encerra a avaliação daquele campo. Ausente opcional é válido; nulo presente exige anulavel. primeiro e coletar ao alcançar limite marcam interrompido, sem afirmar exame do restante. Erros seguem ordem declarada e depois extras. Não há conversão; nenhum dado de entrada é alterado.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Esquemas_1_7_8.md; Exemplos/Projetos/Importador_Validado.

`validar_registros(...)` · módulo `coral.esquemas`

O que faz

Valida um iterável em passagem única e acumula apenas erros, com limite global.

Assinatura

Assinatura para consulta

```
validar_registros(registros: 'Iterable[Mapping]', esquema: 'Mapping', *, modo: 'str' =
  'coletar', limite_erros: 'int' = 100, linha_inicial: 'int | None' = None, origem: 'str |
   None' = None) -> 'dict'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
registros	iterável de mapeamentos	sim	sem padrão	Iterável de registros; cada item não mapeamento produz erro registro.
esquema	mapeamento de esquema	sim	sem padrão	Esquema validado antes de avançar a entrada.
modo	texto: primeiro ou coletar	não	'coletar'	primeiro interrompe no primeiro erro; coletar usa o limite global.
limite_erros	inteiro positivo, exceto booleano	não	100	Máximo positivo de erros; inteiro excluindo booleano, padrão 100.
linha_inicial	inteiro positivo, exceto booleano, ou nulo	não	None	Linha lógica do primeiro registro; nulo omite linha. O índice seguinte soma 1.
origem	texto ou nulo	não	None	Rótulo textual opcional fornecido pelo chamador; nunca inferido.

Retorno

Dicionário com valido (booleano), erros (lista), verificados (número de registros consumidos) e interrompido (booleano). Os erros têm campo, regra, mensagem, registro, linha e origem.

Exemplo

Código Coral

```
de coral.esquemas importe campo, esquema, validar_registros
defina inventario como esquema({"quantidade": campo("inteiro", minimo=0)})
defina resultado como validar_registros([{"quantidade": -1}, {"quantidade": -2},
  {"quantidade": 4}], inventario, limite_erros=1, linha_inicial=2,
  origem="inventario.csv")
mostre resultado["verificados"]
mostre resultado["interrompido"]
mostre resultado["erros"][0]["linha"]
```

Resultado

Saída esperada

```
1
True
2
```

Forma natural

Não há forma natural específica para esta API. Use a chamada explícita importada de coral.esquemas; não há nova gramática.

Erros

TypeError/ValueError para esquema, iterável ou opções inválidos; esquema e opções falham antes do primeiro consumo. Falhas do iterador de entrada propagam ao chamador. Dados incompatíveis produzem as regras de validar_registro.

Atenção

Índice de registro é base 1 e linha = linha_inicial + índice - 1. Isso é contagem lógica, não linha física de CSV multilinha. origem não é inferida. Limite positivo é global; nenhum registro adicional é lido após parar. Não guarda registros nem fecha iteradores de terceiros; o chamador mantém o contexto com e o fechamento. Consumo não altera dados; conversão é explícita antes da validação.

Veja também

Cap. 27; Volume II; Documentacao/Usuario/Coral_Esquemas_1_7_8.md; Exemplos/Projetos/Importador_Validado.

O que você aprendeu

Arquivos podem ser processados em passagem única. JSON conserva estruturas básicas; CSV oferece textos que exigem conversão explícita. Esquemas distinguem chave ausente de nulo e acumulam erros com limite global; o chamador mantém o fechamento do leitor.

Prática

Converta um dicionário para JSON e reconstrua o valor.

Conferir a resposta comentada

Use `para_json` para obter texto JSON e `de_json` para recuperar o dicionário. A ida e volta preserva os valores compatíveis com JSON, não objetos arbitrários ou identidade de instância.

Resposta do capítulo 27 no Apêndice B.

28. Matemática, datas e aleatoriedade

Objetivo de leitura

Usar cálculos fundamentais e distinguir datas, durações e aleatoriedade.

Antes de começar

Conheça números e coleções.

A biblioteca fundamental reúne cálculo, datas e aleatoriedade. Use `coral.matematica` para operações escalares e reduções simples. Use `coral.numerico`, apresentado no Cap. 38, para matrizes e estatística mais ampla.

Código Coral

```
de coral.matematica importe soma, media, raiz
mostre soma([2, 4, 6])
mostre media([2, 4, 6])
mostre raiz(9)
```

Saída esperada

```
12
4.0
3.0
```

A saída é 12, 4.0 e 3.0, em linhas separadas. A média exige observações; a raiz real exige domínio válido. Evite concluir que toda redução aceita vazio ou todo cálculo aceita infinito.

Datas e durações têm papéis diferentes: uma data identifica um dia, uma duração representa um intervalo. A referência de `coral.datas` no Volume II, Parte II, mostra os formatos e fusos.

Uma fonte pseudoaleatória com semente ajuda a repetir simulações. Ela não serve para senhas ou tokens. A semente e a versão do algoritmo fazem parte do contexto de reprodução. O Cap. 37 usa esse princípio em receitas procedurais; `coral.criptografia` fornece operações explícitas para valores secretos.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`soma(...)` · **módulo** `coral.matematica`

O que faz

Soma os valores da coleção.

Assinatura

Assinatura para consulta

```
soma(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável de valores somáveis	sim	sem padrão	Coleção de entrada.

Retorno

Número.

Exemplo

Código Coral

```
de coral.matematica importe soma
mostre soma([2, 4, 6])
```

Resultado

Saída esperada

```
12
```

Erros

Entradas fora do domínio da operação produzem erro. Média não aceita ausência de observações; raiz real não aceita número negativo.

`media(...)` · módulo `coral.matematica`

O que faz

Calcula a média aritmética de observações.

Assinatura

Assinatura para consulta

```
media(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	iterável não vazio de valores numéricos	sim	sem padrão	Coleção de entrada.

Retorno

Número decimal.

Exemplo

Código Coral

```
de coral.matematica importe media
mostre media([2, 4, 6])
```

Resultado

Saída esperada

```
4.0
```

Erros

Entradas fora do domínio da operação produzem erro. Média não aceita ausência de observações; raiz real não aceita número negativo.

`raiz(...)` · módulo `coral.matematica`

O que faz

Calcula a raiz quadrada real.

Assinatura

Assinatura para consulta

```
raiz(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número real não negativo	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Número decimal.

Exemplo

Código Coral

```
de coral.matematica importe raiz
mostre raiz(9)
```

Resultado

Saída esperada

```
3.0
```

Erros

Entradas fora do domínio da operação produzem erro. Média não aceita ausência de observações; raiz real não aceita número negativo.

`absoluto(...)` · módulo `coral.matematica`

O que faz

Obtém a magnitude sem o sinal.

Assinatura

Assinatura para consulta

```
absoluto(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	número ou objeto com __abs__	sim	sem padrão	Valor sobre o qual a operação atua.

Retorno

Número.

Exemplo

Código Coral

```
de coral.matematica importe absoluto
mostre absoluto(-3)
```

Resultado

Saída esperada

```
3
```

Erros

Entradas fora do domínio da operação produzem erro. Média não aceita ausência de observações; raiz real não aceita número negativo.

`potencia(...)` · módulo `coral.matematica`

O que faz

Eleva a base ao expoente.

Assinatura

Assinatura para consulta

```
potencia(base, expoente)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
base	valor compatível com potência	sim	sem padrão	Base da potência.
expoente	expoente compatível com a base	sim	sem padrão	Expoente aplicado à base.

Retorno

Número produzido pela potência; o tipo depende dos operandos.

Exemplo

Código Coral

```
de coral.matematica importe potencia
mostre potencia(2, 3)
```

Resultado

Saída esperada

```
8
```

Erros

Entradas fora do domínio da operação produzem erro. Média não aceita ausência de observações; raiz real não aceita número negativo.

O que você aprendeu

Cada cálculo tem um domínio válido. Datas identificam momentos; durações medem intervalos; sementes ajudam a reproduzir simulações.

Prática

Calcule a média de 2, 4 e 6.

Conferir a resposta comentada

A soma é 12 e a quantidade é 3, portanto a média é 4. `coral.matematica.media([2, 4, 6])` devolve 4.0. Não use uma coleção vazia.

Resposta do capítulo 28 no Apêndice B.

29. Persistência

Objetivo de leitura

Salvar estado recuperável e distinguir dados persistentes de recursos ativos.

Antes de começar

Conheça JSON e objetos, dos capítulos 16 e 27.

A superfície pública `coral.persistencia/1` usa um envelope versionado com contrato, versão de esquema, tipo Coral e carga útil compatível com JSON. Mundo, RPG, Jogos e Laboratório podem persistir estado sem serializar callbacks, janelas, listeners vivos ou identidade acidental de objetos Python.

Em Jogos, por exemplo:

Código Coral

```
crie um jogo chamado aventura sem janela
salve o jogo aventura em "partida.coral-save"
carregue o jogo de "partida.coral-save" como aventura_reaberta
```

Este exemplo não imprime texto no terminal.

Um jogo reaberto usa `BackendNulo`, começa parado e restaura somente estado persistente. Combates contextuais preservam participantes, ordem, índice e rodada utilizando as entidades reconstruídas do Mundo.

SQLite local na Coral 1.7.9

Para aplicações que precisam consultar dados depois de encerrar o programa, `coral.sqlite` oferece uma camada pequena sobre SQLite. SQL permanece explícito. Valores entram por parâmetros separados do texto SQL; parâmetros não substituem nomes de tabela, coluna ou fragmentos de consulta.

Código Coral

```
de coral.sqlite importe abrir

defina banco como abrir(":memoria:")
defina criar_tabela como """
    CREATE TABLE itens (
        id INTEGER PRIMARY KEY,
        nome TEXT NOT NULL,
        quantidade INTEGER NOT NULL
    )
"""
execute banco.executar(criar_tabela)
execute banco.executar("INSERT INTO itens(nome, quantidade) VALUES (?, ?)", ["Ana", 3])
execute banco.confirmar()
defina item como banco.consultar_um("SELECT nome, quantidade FROM itens WHERE nome = ?",
   ["Ana"])
mostre item["nome"]
mostre item["quantidade"]
execute banco.fechar()
```

Saída esperada

```
Ana
3
```

`abrir(caminho, timeout=5.0)` devolve uma `Conexao`. O caminho especial `:memoria:` cria um banco temporário em memória. `timeout` deve ser numérico, não negativo, e booleanos não são aceitos como números nesse contrato.

Operação da conexão	Contrato de uso
`executar(sql, parametros=())`	Executa um comando e retorna `linhas_afetadas` e `ultimo_id`.
`executar_muitos(sql, parametros)`	Executa o mesmo comando para um iterável de sequências ou mapeamentos.
`consultar(sql, parametros=())`	Materializa todas as linhas como lista de dicionários.
`consultar_um(sql, parametros=())`	Retorna um dicionário ou `nulo` quando não há linha.
`confirmar()`	Confirma alterações pendentes.
`reverter()`	Desfaz alterações pendentes desde a última confirmação.
`transacao()`	Contexto explícito: confirma na saída normal e reverte se o bloco falhar.
`fechar()`	Encerra a conexão; é idempotente. Uso posterior da conexão falha.

Parâmetros aceitam sequência ou mapeamento. Texto e bytes isolados são recusados para evitar vinculação acidental caractere a caractere. `executar_muitos` exige um iterável de conjuntos de parâmetros. Erros operacionais do SQLite são expostos como `ErroOperacionalSQLite`; demais falhas da camada usam `ErroSQLite`, preservando a causa original.

Uma transação contextual não pode ser aninhada e não começa se já houver alteração pendente. A API não cria savepoints implícitos. Para um banco em arquivo, confirme antes de fechar quando quiser conservar as alterações. O projeto `Exemplos/Projetos/Catalogo_SQLite` demonstra cadastro, consulta, atualização, remoção e reabertura persistente.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`abrir(...)` · **módulo** `coral.sqlite`

O que faz

Abre uma conexão SQLite local com timeout explícito e resultados adaptados à API Coral.

Assinatura

Assinatura para consulta

```
abrir(caminho: 'str | Path', *, timeout: 'float' = 5.0) -> 'Conexao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou Path	sim	sem padrão	Caminho do banco. O texto :memoria: cria um banco somente em memória.
timeout	inteiro ou decimal não negativo, exceto booleano	não	5.0	Tempo máximo, em segundos, aguardado pelo SQLite em bloqueios; número não negativo, exceto booleano.

Retorno

Conexao aberta. O chamador deve confirmar ou reverter alterações e fechar a conexão.

Exemplo

Código Coral

```
de coral.sqlite importe abrir
defina banco como abrir(":memoria:")
mostre banco.fechada
execute banco.fechar()
mostre banco.fechada
```

Resultado

Saída esperada

```
False
True
```

Forma natural

Não há forma natural específica. Use abrir(...) de coral.sqlite e mantenha SQL explícito.

Erros

TypeError para caminho/timeout incompatíveis; ValueError para caminho vazio ou timeout negativo; ErroOperacionalSQLite quando o driver não consegue abrir o banco.

Atenção

Parâmetros SQL pertencem às operações da conexão, não a abrir. :memoria: não persiste após o fechamento.

Veja também

Cap. 29; Volume II; Documentacao/Usuario/Coral_SQLite_Textos_Multilinha_1_7_9.md; Exemplos/Projetos/Catalogo_SQLite.

`Conexao(...)` · **módulo** `coral.sqlite`

O que faz

Representa uma conexão SQLite com consultas em dicionários, confirmação, reversão e transação contextual explícitas.

Construtor

Assinatura para consulta

```
Conexao(caminho: 'str | Path', *, timeout: 'float' = 5.0)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	texto ou Path	sim	sem padrão	Caminho do banco. O texto :memoria: cria um banco somente em memória.
timeout	inteiro ou decimal não negativo, exceto booleano	não	5.0	Tempo máximo, em segundos, aguardado pelo SQLite em bloqueios; número não negativo, exceto booleano.

Atributos importantes

Atributo	Significado
`fechada`	Booleano somente leitura que indica se fechar() já encerrou a conexão.
`em_transacao`	Booleano somente leitura que reflete a transação atual; exige conexão aberta.

Métodos principais

Assinatura para consulta

```
executar(self, sql: 'str', parametros=()) -> 'dict[str, int | None]'
```

Referência para leitura

```
executar_muitos(self, sql: 'str', parametros) -> 'dict[str, int | None]'
```

Referência para leitura

```
consultar(self, sql: 'str', parametros=()) -> 'list[dict[str, Any]]'
```

Referência para leitura

```
consultar_um(self, sql: 'str', parametros=()) -> 'dict[str, Any] | None'
```

Referência para leitura

```
confirmar(self) -> 'None'
```

Referência para leitura

```
reverter(self) -> 'None'
```

Referência para leitura

```
fechar(self) -> 'None'
```

Referência para leitura

```
transacao(self) -> "Iterator['Conexao']"
```

Retorno

Instância de Conexao pronta para uso até fechar().

Exemplo

Código Coral

```
de coral.sqlite importe Conexao
defina banco como Conexao(":memoria:")
mostre banco.fechada
execute banco.fechar()
mostre banco.fechada
```

Resultado

Saída esperada

```
False
True
```

Forma natural

Normalmente obtenha a conexão por abrir(...); construir Conexao(...) diretamente é equivalente no contrato atual.

Erros

ErroSQLite para uso após fechamento ou estado transacional incompatível; ErroOperacionalSQLite para falhas operacionais do driver; TypeError/ValueError para argumentos fora do contrato.

Atenção

executar/consultar usam parâmetros separados do SQL. transacao() não aceita aninhamento nem começa sobre alteração pendente. fechar() é idempotente.

Veja também

Cap. 29; Volume II; Documentacao/Usuario/Coral_SQLite_Textos_Multilinha_1_7_9.md; Exemplos/Projetos/Catalogo_SQLite.

`ErroSQLite(...)` · módulo `coral.sqlite`

O que faz

Exceção base da camada SQLite Coral, usada para estado inválido e erros SQLite não classificados como operacionais.

Construtor

Assinatura para consulta

```
ErroSQLite(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Instância de exceção usada para identificar falhas SQLite na camada Coral.

Exemplo

Código Coral

```
de coral.sqlite importe ErroSQLite
de coral.tipos importe e_tipo
defina erro como ErroSQLite("falha")
mostre e_tipo(erro, ErroSQLite)
```

Resultado

Saída esperada

```
True
```

Forma natural

Não há forma natural específica para construir este erro; normalmente ele é produzido pela camada coral.sqlite.

Erros

A própria classe representa erro; não introduz validação adicional ao construtor herdado de RuntimeError.

Atenção

Prefira tratar esta classe ou ErroOperacionalSQLite em torno de operações de banco, em vez de depender diretamente das exceções sqlite3.

Veja também

Cap. 29; Volume II; Documentacao/Usuario/Coral_SQLite_Textos_Multilinha_1_7_9.md.

`ErroOperacionalSQLite(...)` · módulo `coral.sqlite`

O que faz

Especialização de ErroSQLite para falhas operacionais do driver, como bloqueio, abertura ou SQL rejeitado pelo SQLite.

Construtor

Assinatura para consulta

```
ErroOperacionalSQLite(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Instância de exceção usada para identificar falhas SQLite na camada Coral.

Exemplo

Código Coral

```
de coral.sqlite importe ErroSQLite e ErroOperacionalSQLite
de coral.tipos importe e_tipo
defina erro como ErroOperacionalSQLite("bloqueado")
mostre e_tipo(erro, ErroOperacionalSQLite)
mostre e_tipo(erro, ErroSQLite)
```

Resultado

Saída esperada

```
True
True
```

Forma natural

Não há forma natural específica para construir este erro; normalmente ele é produzido pela camada coral.sqlite.

Erros

A própria classe representa erro operacional; não introduz validação adicional ao construtor herdado.

Atenção

É subclasse de ErroSQLite. Operações coral.sqlite traduzem sqlite3.OperationalError para esta classe preservando a causa original.

Veja também

Cap. 29; Volume II; Documentacao/Usuario/Coral_SQLite_Textos_Multilinha_1_7_9.md.

O que você aprendeu

Persistência guarda estado declarativo com contrato e esquema. Janelas, funções de reação e recursos ativos não são serializados como se fossem dados comuns.

Prática

Explique por que persistência não deve guardar callbacks e janelas ativas.

Conferir a resposta comentada

Esses objetos dependem de execução e recursos externos. O formato portátil guarda estado declarativo, com contrato e esquema, e reconstrói apenas os tipos que possuem adaptadores explícitos.

Resposta do capítulo 29 no Apêndice B.

PARTE IV: DESENVOLVENDO APLICAÇÕES

30. Projetos, módulos e coral.toml

Objetivo de leitura

Separar módulos, configurar a entrada e executar um projeto completo.

Antes de começar

Conheça funções, classes e o projeto básico do capítulo 6.

Criar a estrutura

Terminal

```
coral novo MinhaFerramenta --template basico
cd MinhaFerramenta
coral executar
coral testar
```

`coral.toml` identifica a raiz do projeto. Ele pode definir nome, versão, arquivo de entrada, requisito de versão Coral, caminhos de módulos e caminhos de testes.

Configuração TOML

```
[projeto]
nome = "minha-ferramenta"
versao = "0.1.0"
[coral]
versao = ">=1.7.4"
[entrada]
arquivo = "principal.coral"
[modulos]
caminhos = ["."]
[testes]
caminhos = ["testes"]
```

Caminhos são relativos à raiz configurada. `coral executar` compila os módulos para `.coral_build` e executa a entrada. Essa pasta contém saída regenerável. Não a use como local dos seus arquivos fonte.

Importar funções locais

Um arquivo `calculos.coral` pode declarar uma função `media`. Em `principal.coral`, use `de calculos importe media` e depois `media(notas)`. Nomeie seus módulos de forma clara e evite colidir com `coral`, `json`, `sys` e outros módulos do ambiente.

Na 1.5.8, a interface pública do módulo também carrega formas naturais exportadas. Uma função declarada com uma forma como `"dobre {numero}"` pode ser usada pelo consumidor

após uma importação seletiva. Alias de importação preserva a forma adaptada, enquanto duas formas externas ambíguas produzem diagnóstico em vez de uma escolha silenciosa.

O build guarda hashes de fonte e de interface. Sem mudança, módulos já válidos são reutilizados. Uma mudança apenas de implementação recompila o módulo afetado; uma mudança de interface também invalida os consumidores. O resultado de um build incremental deve ser equivalente ao de um build limpo.

Recursos de projeto precisam ser declarados no `coral.toml`. O build copia somente os recursos declarados e o runtime resolve seus caminhos a partir do pacote construído, de modo que a pasta de build possa ser movida sem perder o vínculo com esses arquivos.

A execução de um arquivo isolado não equivale à construção de um projeto: para módulos locais Coral, prefira `coral executar` na raiz do projeto. O comando `coral arquivo.coral` é apropriado a programas independentes.

Projeto curto: Cadastro em três arquivos

Separe as classes, a aplicação e os testes de um cadastro. Declare o arquivo de entrada no coral.toml.

A orientação de solução está no Apêndice B, seção Projeto 30.

Contratos entre módulos na 1.7.4

Quando uma classe ou função possui tipos declarados, a análise do consumidor aproveita essa informação. Imports seletivos, aliases e reexportações preservam a origem do símbolo quando ela pode ser resolvida. Métodos herdados e o construtor herdado também podem fornecer contratos.

O editor usa os arquivos abertos para analisar alterações ainda não salvas. Ao mudar o contrato de um módulo, os consumidores abertos recebem diagnósticos atualizados. A análise não executa os módulos importados para descobrir seus tipos.

O build incremental considera o contrato público das dependências transitivas. Por isso, mudar uma assinatura atrás de um módulo que a reexporta também invalida o consumidor. Alterações somente no corpo preservam o contrato público.

O projeto completo de demonstração está em `Exemplos/Citados/Analise_1_7_4/Cadastro`: leia `modelos.coral`, `fachada.coral` e `principal.coral` e execute seus testes na raiz do projeto.

Imports seletivos em linhas separadas

Uma linha que termina em `de modulo importe` abre uma lista de nomes exportados. Cada linha recebe um nome, com alias opcional; `fim` fecha a importação. Esse bloco não possui corpo de execução.

Código Coral

```
de math importe
    sqrt como raiz
    floor
fim
mostre raiz(9)
mostre floor(2.8)
```

Saída esperada

```
3.0
2
```

Isso equivale a `de math importe sqrt como raiz, floor`. A resolução do módulo, os aliases e a ordem de importação permanecem os mesmos. A lista explícita não aceita curingas nem itens vazios. O editor mantém a navegação entre cada item e sua declaração de origem.

Um ambiente próprio para o projeto

Na Coral 1.7.6, `[dependencias].python` declara requisitos Python e `[ambiente].python` limita o interpretador. O fluxo resolve wheels para um alvo específico, registra versões e hashes em `coral.lock`, prepara uma geração isolada e verifica essa geração antes de executar. A execução de arquivo, projeto, build e testes pela CLI usa o mesmo executor quando o projeto adota esse lock.

Configuração TOML

```
[projeto]
nome = "meu-projeto"

[coral]
versao = ">=1.7.6"

[dependencias]
python = ["biblioteca-didatica==1.0"]

[ambiente]
python = ">=3.10,<3.15"
```

O nome acima é ilustrativo. Para um fluxo executável sem registro remoto, use `Exemplos/Projetos/Ambiente_Reproduzivel`, que gera suas próprias wheels didáticas. Com uma pasta de wheels compatíveis, o fluxo é:

Terminal

```
coral manager ambiente resolver --projeto . --cache wheels --offline --json
coral manager ambiente preparar --projeto . --offline --aplicar --json
coral manager ambiente verificar --projeto . --json
coral executar
```

Sem `--permitir-rede`, a resolução não consulta um registro remoto. Preparar sem `--aplicar` apenas apresenta o plano. Uma falha na instalação conserva a geração ativa anterior; repetir a preparação reutiliza uma geração válida. Um ambiente ausente, alterado ou incompatível impede a execução de um projeto que adotou lock. Projetos antigos sem lock conservam seu fluxo anterior.

Esta edição aceita wheels e requisitos de nome e versão. URLs, caminhos, VCS, extras e marcadores escritos no manifesto ficam fora desse contrato inicial. O lock pertence ao alvo registrado: não copie uma venv pronta para outra máquina ou caminho. Veja `Documentacao/Usuario/Coral_Ambientes_Reproduziveis_1_7_6.md` para restrições, cancelamento e os limites de qualificação.

Integração opcional com o desktop Linux

Na Coral 1.7.11, o runtime pode registrar `*.coral` como `text/x-coral` e o nome exato `coral.toml` como `text/x-coral-project` no desktop Linux. A instalação por usuário é opcional e reversível:

Referência para leitura

```
coral desktop instalar --usuario
coral desktop status --usuario
coral desktop remover --usuario
```

A integração não escolhe aplicativo padrão, não altera `mimeapps.list`, não associa todos os arquivos TOML e não muda o tema de ícones do VS Code. Arquivos preexistentes ou modificados depois da instalação são preservados na remoção.

O que você aprendeu

O manifesto organiza o projeto. Imports disponibilizam símbolos; alterações de contrato podem exigir reconstruir consumidores.

Prática

Diferencie arquivo de entrada, importação e bloco programa principal.

Conferir a resposta comentada

O projeto escolhe qual arquivo executar; a importação carrega um módulo; o bloco principal organiza a ação da execução direta. Bibliotecas reutilizáveis não devem iniciar a aplicação durante a importação.

Resposta do capítulo 30 no Apêndice B.

31. Testes

Objetivo de leitura

Escrever verificações e executar testes sem depender de sua ordem.

Antes de começar

Conheça funções e projetos, dos capítulos 15 e 30.

Testes nativos

Desde a linha 1.2, um bloco `teste "nome"` contém verificações e termina com `fim`. Execute com `coral testar`; simplesmente executar um arquivo de testes como programa não é o mesmo fluxo. Na 1.5.8, o runner prepara o projeto uma vez por sessão e reutiliza esse build entre arquivos de teste.

O bloco de teste pode declarar casos parametrizados, erro esperado, motivo de pulo, semente, limite de duração, grupo e recursos temporários. Hooks explícitos de preparação e encerramento continuam separados do teste básico, de modo que um caso simples não precisa carregar toda a infraestrutura avançada.

O conjunto `Exemplos/Citados/Projetos_Completos/Notas/` traz um projeto completo com `calculos.coral`, `principal.coral` e `testes/basico.coral`. Copie essa pasta para seu diretório de trabalho e execute:

Terminal

```
coral executar
coral testar
coral dependencias
coral documentar
```

A execução imprime `8.0` e o teste da média deve passar. `coral documentar` gera uma referência de símbolos; isso não substitui o manual de uso nem inventa explicações das regras de negócio.

O runner isola os módulos locais durante o carregamento de cada arquivo de testes. Testes no mesmo arquivo ainda podem compartilhar objetos importados. Evite depender da ordem de execução.

Projeto curto: Contrato de uma função

Teste o dobro para zero, um valor positivo e um negativo. Acrescente uma entrada textual inválida se a aplicação recebe texto.

A orientação de solução está no Apêndice B, seção Projeto 31.

O que você aprendeu

Testes conferem comportamentos esperados. `coral testar` usa o fluxo próprio do runner; executar um arquivo de testes como programa é outro fluxo.

Prática

Escreva uma verificação para uma função dobro e provoque uma falha intencional.

Conferir a resposta comentada

Use `garanta que dobro(3) for igual a 6`. Trocar 6 por 7 provoca a falha da verificação. Reponha o valor correto antes de considerar o teste aprovado.

Resposta do capítulo 31 no Apêndice B.

32. Regras, eventos e tempo

Objetivo de leitura

Reagir a eventos e mudanças e controlar o tempo lógico.

Antes de começar

Conheça condições e estado, dos capítulos 9 e 12.

A ideia principal da programação reativa em Coral é simples: **algo acontece e uma regra reage**. O programa não precisa transformar cada comportamento em um loop manual nem em callbacks de baixo nível. A superfície natural descreve o acontecimento, a condição opcional e a reação; o runtime mantém ordem determinística, tempo lógico e rastreamento.

O exemplo canônico mais curto está em `Exemplos/Regras/01_eventos_observacao_e_tempo.coral`:

Código Coral

```
defina ultimo_dano como 0
quando ocorrer "dano" como evento se evento.quantidade for maior que 3
    defina ultimo_dano como evento.quantidade
    mostre evento.origem
fim

emita o evento "dano" com quantidade 5 e origem "espada"
```

Saída esperada

```
espada
```

O objeto `evento` expõe os dados nomeados do acontecimento. Nesse exemplo, `quantidade` é carga útil e `origem` identifica quem publicou o evento. A emissão é local e determinística: ela não cria uma thread, não inicia uma tarefa em segundo plano e não transforma o programa em um sistema distribuído.

Observar mudanças sem inventar eventos manuais

Quando a intenção é reagir a um valor que mudou, diga isso diretamente:

Código Coral

```
defina vida como 10
quando vida mudar
    mostre vida
fim

avalie as regras
defina vida como 9
avalie as regras
```

Saída esperada

```
9
```

Também existem cruzamentos de limite:

Código Coral

```
quando temperatura passar de 40
    mostre "Temperatura alta."
fim

quando temperatura cair abaixo de 30
    mostre "Temperatura baixa."
fim
```

Este exemplo não imprime texto no terminal.

O exemplo executável é `Exemplos/Regras/01_eventos_observacao_e_tempo.coral`. Uma atribuição que mantém o mesmo valor não conta como mudança. `passar de` e `cair abaixo de` reagem ao cruzamento da fronteira, não a toda avaliação posterior que permaneça do mesmo lado.

Prioridade, cancelamento e consumo

Prioridades nomeadas evitam números mágicos: `baixa`, `normal`, `alta` e `crítica`. Reações de maior prioridade são consideradas primeiro; empates preservam a ordem de registro.

Código Coral

```
quando ocorrer "rpg.dano.antes" como evento com prioridade crítica
    cancele evento
fim
```

Este exemplo não imprime texto no terminal.

Cancelar só tem efeito sobre acontecimentos declarados como canceláveis e deve ocorrer **antes** do efeito. A Coral não tenta desfazer uma alteração já aplicada. O RPG usa esse contrato em `rpg.dano.antes`: se o evento for cancelado, a vida não é modificada. Veja `Exemplos/Regras/02_prioridade_e_cancelamento.coral`.

`consuma evento` tem outro significado: impede que o acontecimento continue sua propagação para níveis superiores, mas não representa por si só o cancelamento da operação de domínio.

Limites, cooldown e debounce

Uma reação pode executar somente uma vez ou um número máximo de vezes:

Código Coral

```
uma vez quando ocorrer "visita"
    mostre "Primeira visita."
fim
```

Este exemplo não imprime texto no terminal.

Cooldown limita a frequência de um evento:

Código Coral

```
quando ocorrer "ataque" no máximo uma vez a cada 2 segundos
    mostre "Ataque aceito."
fim
```

Este exemplo não imprime texto no terminal.

Debounce é adequado quando a ação deve ocorrer somente depois de um período sem novas mudanças:

Código Coral

```
quando texto mudar espere 300 milissegundos sem novas mudanças faça
    mostre texto
fim
```

Este exemplo não imprime texto no terminal.

Essas formas usam o tempo lógico do motor. Não usam `sleep` escondido.

Tempo determinístico e relógio de teste

Temporizadores continuam disponíveis com `depois de` e `a cada`. Em testes, o relógio pode ser avançado instantaneamente:

Código Coral

```
defina pulsos como 0
a cada 2 segundos faça
    adicione 1 a pulsos
fim

avance o relógio de teste em 5 segundos
mostre pulsos
```

Saída esperada

```
2
```

O exemplo `Exemplos/Regras/01_eventos_observacao_e_tempo.coral` valida tempo sem esperar cinco segundos reais. Isso mantém testes rápidos e reproduzíveis.

Grupos e regras temporárias

Regras nomeadas podem ser reunidas em grupos:

Código Coral

```
regra "chuva_venenosa"
    quando falso
    então
        passe
fim
regra "raios"
    quando falso
    então
        passe
fim
grupo tempestade faça
    use chuva_venenosa
    use raios
fim

desative grupo tempestade
ative grupo tempestade
```

Este exemplo não imprime texto no terminal.

Uma regra desativada também pode ser ligada por duração ou até uma condição:

Código Coral

```
defina temperatura como 35
regra "alerta"
    quando temperatura for maior que 30
    então
        passe
fim
ative alerta durante 30 segundos
ative alerta até temperatura for menor que 30
```

Este exemplo não imprime texto no terminal.

A duração é cooperativa: o motor aplica a expiração quando seu tempo lógico avança.

Sequências de dois acontecimentos

A 1.5.6 reconhece sequências deliberadamente simples A → B:

Código Coral

```
quando ocorrer "botao_a" e depois ocorrer "botao_b" dentro de 2 segundos
    mostre "Combinação reconhecida."
fim
```

Este exemplo não imprime texto no terminal.

A janela temporal é opcional. Esta edição não transforma a linguagem em um processador geral de eventos complexos; sequências arbitrariamente longas permanecem fora do escopo.

Eventos espaciais em 2D e 3D

Os observadores espaciais reutilizam a baseline geral da linguagem:

Código Coral

```
defina jogador como posição(-1, 5)
defina floresta como região(retângulo(0, 0, 10, 10))

quando jogador entrar em floresta
    mostre "Entrou na floresta."
fim
```

Este exemplo não imprime texto no terminal.

Também há `sair de`, proximidade, toque e interseção. As mesmas ideias funcionam em 3D; misturar 2D e 3D continua exigindo conversão explícita. O exemplo canônico é `Exemplos/Regras/03_eventos_espaciais.coral`.

Estados observáveis

Para uma pequena máquina de estados, declare a intenção sem construir uma classe auxiliar:

Código Coral

```
estado porta começa como fechada

quando ocorrer "abrir_porta"
    mude porta para aberta
fim
```

Este exemplo não imprime texto no terminal.

Esse recurso é um estado observável leve. Ele não é um statechart hierárquico e não cria estados paralelos implicitamente.

Histórico e explicação

O comando:

Código Coral

```
mostre histórico de eventos
```

Saída esperada

```
Nenhum evento ou regra foi registrado no histórico reativo.
```

ativa rastreamento detalhado desde o início do programa e apresenta a cadeia de decisões reativas. O runtime registra o resultado de uma condição quando ela é avaliada; o modo Ensinar usa esse registro para explicar por que uma regra executou ou foi suprimida **sem reavaliar a condição** e sem repetir efeitos colaterais.

Motores e avaliação

`avalie as regras` processa condições e observadores no motor padrão. `atualize as regras por ... segundos` também avança tempo lógico, timers, cooldowns, debounce e ativações temporárias. Jogos podem atualizar seu próprio motor durante o ciclo de quadros, enquanto Mundo e RPG publicam acontecimentos de domínio no mesmo modelo de eventos.

Quando nenhum motor específico é informado, a superfície natural usa o motor padrão. Formas históricas como `quando ocorrer "moeda" recebendo valor` continuam válidas por compatibilidade.

API avançada

A API orientada a objetos permanece em `coral.regras`, `coral.tempo_eventos` e módulos auxiliares para ferramentas, integrações e casos que a superfície natural não cobre. Código introdutório deve preferir as frases deste capítulo. `regra`, eventos, observadores, sequências, grupos e controles reativos são nós do parser e da AST Coral; não dependem de substituição textual frágil nem de interpretação por IA.

Projeto curto: Alerta de estoque

Modele uma quantidade de itens e uma regra que informe quando o estoque ficar abaixo de três. Controle o tempo com relógio de teste.

A orientação de solução está no Apêndice B, seção Projeto 32.

O que você aprendeu

Uma regra reage a um acontecimento. Ordem, prioridade e tempo são explícitos; cancelar um evento e consumir sua propagação têm papéis diferentes.

Prática

Diferencie evento, regra e relógio de teste.

Conferir a resposta comentada

O evento registra um acontecimento; a regra escolhe como reagir; o relógio de teste permite controlar o tempo explicitamente. Uma passagem de tempo simulada não deve depender de espera real.

Resposta do capítulo 32 no Apêndice B.

33. Mundo

Objetivo de leitura

Organizar valores em células, camadas e regiões de um mapa.

Antes de começar

Conheça coleções e posição como conjunto de coordenadas.

A Coral 1.5.7 introduz uma infraestrutura discreta de mapas em `coral.mundo`. Ela não é uma engine de jogos: um mapa organiza **valores no espaço**, permite consultar relações locais e comunica mudanças usando a infraestrutura reativa já existente. O mesmo contrato serve para simulações, ciência, robótica, logística, autômatos celulares e, opcionalmente, jogos.

A ideia central é separar cinco coisas que costumam ser confundidas:

- uma **posição** é uma localização contínua no espaço, como `(2.4, 7.1)`;
- uma **célula** é uma localização discreta inteira, como `(2, 7)`;
- uma **camada** diz que tipo de informação está sendo armazenada;
- o **valor** é o dado guardado naquela célula e camada;
- uma **região** agrupa células ou associa uma forma espacial a uma parte do mapa.

Célula e posição

Células usam coordenadas inteiras. Uma topologia quadrada usa coluna e linha; uma topologia hexagonal usa `q` e `r`; e um mapa voxel usa `x`, `y` e `z`. Posições contínuas continuam pertencendo à baseline espacial apresentada no capítulo anterior.

Código Coral

```
crie um mapa chamado clima com topologia quadrada
adicione uma camada chamada temperatura ao mapa clima
defina 31.5 na célula (4, 2) da camada temperatura do mapa clima
defina leitura como o valor da célula (4, 2) da camada temperatura do mapa clima
garanta que leitura for igual a 31.5
```

Este exemplo não imprime texto no terminal.

Ausência não significa `nulo`, zero, falso ou texto vazio. Uma célula pode simplesmente não possuir conteúdo em determinada camada.

Topologia

A topologia define a dimensionalidade, a forma de vizinhança e a conversão entre célula e coordenada de mundo.

Código Coral

```
crie um mapa chamado colmeia com topologia hexagonal
defina vizinhas como as células vizinhas de (2, menos 1) no mapa colmeia
garanta que quantidade de vizinhas for igual a 6
```

Este exemplo não imprime texto no terminal.

Código Coral

```
crie um mapa chamado volume com topologia voxel
adicione uma camada chamada densidade ao mapa volume
defina 0.82 na célula (3, 4, 5) da camada densidade do mapa volume
defina amostra como o valor da célula (3, 4, 5) da camada densidade do mapa volume
garanta que amostra for igual a 0.82
```

Este exemplo não imprime texto no terminal.

A topologia quadrada possui quatro vizinhos ortogonais por padrão. A hexagonal possui seis vizinhos axiais. A voxel possui seis vizinhos ortogonais em três dimensões. Mapas podem ser finitos ou não limitados; uma célula ausente em um mapa não limitado continua sendo uma coordenada válida, apenas sem conteúdo.

Camada e valor

Camadas são nomeadas, ordenadas e esparsas. Elas podem armazenar números, textos, booleanos, listas, dicionários e outros valores portáteis reconhecidos pela persistência Coral. `BlocoMapa` existe como conveniência para terreno ou material, mas não é a unidade obrigatória do sistema.

Código Coral

```
crie um mapa chamado laboratorio com topologia quadrada
adicione uma camada chamada temperatura ao mapa laboratorio
adicione uma camada chamada umidade ao mapa laboratorio
defina 22.0 na célula (1, 0) da camada temperatura do mapa laboratorio
defina 0.61 na célula (1, 0) da camada umidade do mapa laboratorio
defina t como o valor da célula (1, 0) da camada temperatura do mapa laboratorio
defina u como o valor da célula (1, 0) da camada umidade do mapa laboratorio
garanta que t for igual a 22.0
garanta que u for igual a 0.61
```

Este exemplo não imprime texto no terminal.

Vizinhança

Vizinhança pertence à topologia, não ao domínio do programa. Isso permite que o mesmo comando seja usado por um autômato celular, um robô ou um jogo.

Código Coral

```
crie um mapa chamado grade com topologia quadrada
defina proximas como as células vizinhas de (4, 2) no mapa grade
garanta que quantidade de proximas for igual a 4
```

Este exemplo não imprime texto no terminal.

Consultas de mapa são expressões e podem participar de cálculos e comparações. Por exemplo, `soma mais o valor da célula celula da camada temperatura do mapa laboratorio` permanece uma única expressão estrutural da linguagem.

Região

Uma região pode ser formada por células explícitas ou, pela API avançada, por uma forma espacial serializável. Regiões discretas servem para setores de armazém, biomas, zonas de segurança, territórios ou qualquer agrupamento irregular.

Código Coral

```
crie um mapa chamado clima com topologia quadrada
adicione uma camada chamada temperatura ao mapa clima
crie uma região chamada zona_quente no mapa clima com as células [(4, 2), (5, 2), (5,
  3)]
defina presentes como as regiões da célula (5, 2) no mapa clima
garanta que quantidade de presentes for igual a 1
```

Este exemplo não imprime texto no terminal.

Em mapas não limitados, uma região definida por forma não precisa materializar uma quantidade infinita de células. Quando uma enumeração puder ser infinita, a API exige limites explícitos.

Travessia

Travessia é opcional e genérica. Um conteúdo pode permitir ou bloquear passagem, declarar custo não negativo e registrar um motivo. A 1.5.7 fornece adjacência e custos, mas não introduz um algoritmo oficial de caminhos.

Código Coral

```
crie um mapa chamado deposito com topologia quadrada
adicione uma camada chamada ocupacao ao mapa deposito
coloque o bloco "estante" na célula (8, 3) da camada ocupacao do mapa deposito
bloqueie a travessia pela célula (8, 3) da camada ocupacao do mapa deposito porque
  "corredor ocupado"
```

Este exemplo não imprime texto no terminal.

Uma consulta pura de travessia não publica eventos. Uma tentativa explícita pode publicar `mapa.travessia_bloqueada` no barramento já utilizado pelo mundo.

Eventos de mapa

Mapas associados a `Mundo` usam **o mesmo barramento** `mundo.regras.eventos`; não existe um motor de regras oculto por mapa. Um mapa independente também funciona sem barramento quando o programa não precisa de reatividade.

Código Coral

```
crie um mapa chamado clima com topologia quadrada
adicione uma camada chamada temperatura ao mapa clima
defina ultima como 0

quando o valor da célula (4, 2) da camada temperatura do mapa clima mudar como evento
    defina ultima como evento.valor_novo
fim

defina 31.5 na célula (4, 2) da camada temperatura do mapa clima
garanta que ultima for igual a 31.5
```

Este exemplo não imprime texto no terminal.

Também é possível observar entrada e saída de regiões. O resolvedor de localização é relido a cada avaliação, portanto mover o alvo não congela a posição inicial.

Persistência

`Mapa` participa do envelope `coral.persistencia/1` com o identificador `coral.mundo/Mapa`. O formato recomendado para um mapa isolado é `.coralmapa`. Topologia, camadas, conteúdos portáteis e regiões são preservados; Pygame, superfícies, callbacks e recursos gráficos abertos não fazem parte do arquivo.

Na API avançada, a persistência genérica já é suficiente:

Código Python

```
from coral.persistencia import carregar, salvar

salvar(mapa, "clima.coralmapa")
restaurado = carregar("clima.coralmapa")
```

`Mundo` também persiste seus mapas nomeados. Documentos de versões anteriores que não possuem o campo `mapas` continuam sendo carregados com zero mapas.

Visualização opcional com Jogos

`coral.jogos` não define o significado de um mapa. Ele oferece apenas conversões de tela e um adaptador 2D para mapas quadrados e hexagonais. Mapas voxel continuam válidos no núcleo, mas não recebem uma projeção 3D implícita.

Código Coral

```
de coral.jogos importe Camera e RenderizadorMapa2D e BackendNulo

crie um mapa chamado sala com topologia quadrada
adicione uma camada chamada terreno ao mapa sala
coloque o bloco "parede" na célula (1, 1) da camada terreno do mapa sala

defina camera como Camera(0, 0, 64, 64)
defina backend como BackendNulo()
defina renderizador como RenderizadorMapa2D(sala, camera=camera)
defina desenhadas como renderizador.desenhar(backend)
garanta que desenhadas for igual a 1
```

Este exemplo não imprime texto no terminal.

O conjunto de exemplos oficiais inclui uma grade de temperatura, um autômato celular e um mapa de ocupação robótica justamente para provar que visualização é um consumidor opcional e não a fundação da API.

`coral.mundo` é a API oficial da Coral 1.2 para mundos, entidades, atributos, inventários, relações e pequenos modelos de simulação. Internamente, relações formam um grafo dirigido, mas a superfície recomendada é escrita como Coral corrente.

Forma recomendada

Código Coral

```
crie um mundo chamado campanha com nome "Minha Campanha"

crie uma entidade chamada ana com nome "Ana" no mundo campanha
crie uma entidade chamada castelo com nome "Castelo" no mundo campanha

relacione ana com castelo como "viaja para" no mundo campanha

encontre um caminho de ana até castelo no mundo campanha como rota
```

Este exemplo não imprime texto no terminal.

A intenção é que a leitura revele o modelo sem exigir que o iniciante conheça construtores, métodos ou a implementação em grafos.

Entidades

Entidades podem guardar atributos, tags e inventário. A API de baixo nível permanece disponível para código avançado, interoperabilidade e extensão da biblioteca.

Relações e caminhos

`relacione` cria uma relação dirigida. `encontre um caminho` usa a busca de caminho fornecida por `Mundo`, atualmente baseada em busca em largura para caminhos não ponderados.

Eventos e regras

Cada mundo possui um `MotorRegras`, portanto pode hospedar regras e eventos próprios:

Código Coral

```
crie um mundo chamado campanha com nome "Campanha"
defina vida como 2
regra "perigo" em campanha.regras
    quando vida for menor ou igual a 3
    então
        mostre "Perigo!"
fim
```

Este exemplo não imprime texto no terminal.

API orientada a objetos

Formas como `Mundo(...)`, `mundo.criar_entidade(...)` e `mundo.relacionar(...)` continuam válidas. Elas são consideradas superfície avançada e não a forma principal dos templates e tutoriais oficiais quando existe frase Coral equivalente.

Precisões preservadas na linha 1.2

As relações são dirigidas, mas `Mundo.caminho` e a frase natural de busca consideram ambas as direções por padrão (`bidirecional=True`). A API avançada aceita `bidirecional=falso` numa expressão Coral para seguir somente saídas. A implementação atual rejeita entidades externas que tenham apenas o mesmo nome de uma entidade local. `para_json` exporta o estado genérico do mundo; não é um formato completo de persistência de personagens e combates.

Um mundo reúne entidades e relações. Entidades têm identidade própria: dois objetos com o mesmo nome não são intercambiáveis. A biblioteca recusa relações com objetos externos que apenas repetem o nome de uma entidade registrada.

Arquivo: `Exemplos/Citados/Mundo/01_mundo_e_rpg.coral`.

Código Coral

```
crie um mundo chamado campanha com nome "Campanha"
crie uma entidade chamada vila no mundo campanha
crie uma entidade chamada bosque no mundo campanha
relacione vila com bosque como "estrada" no mundo campanha
encontre um caminho de bosque até vila no mundo campanha como rota
mostre quantidade de rota
```

Saída esperada

```
2
```

A relação criada é dirigida, mas a busca padrão considera ambas as direções. Por isso, o exemplo encontra um caminho de bosque para vila. A API avançada `campanha.caminho(bosque, vila, bidirecional=falso)` permite seguir apenas a direção de saída. A busca usa BFS para caminhos não ponderados.

Personagens e turnos

Arquivo: `Exemplos/Citados/Mundo/01_mundo_e_rpg.coral`.

Código Coral

```
crie um mundo chamado campanha com nome "Campanha"
crie um personagem chamado heroina no mundo campanha com 12 de vida
crie um personagem chamado monstro no mundo campanha com 6 de vida
role um dado de 6 lados usando a semente 7 como ataque
cause ataque.total de dano a monstro
cure 1 de vida de monstro
mostre monstro.vida
crie um combate chamado batalha com heroina e monstro
inicie o combate batalha usando a semente 4
mostre batalha.rodada
```

Saída esperada

```
4
1
```

O dano reduz vida sem deixá la negativa; a cura respeita o limite máximo. A semente torna a rolagem reproduzível para o exemplo. O combate organiza iniciativa e turnos, sem fornecer um sistema completo de regras de RPG. Use `avance o combate batalha` para avançar ao participante seguinte.

Referência dos componentes

Componente	Elementos principais
`coral.mundo`	`Mundo`, `Entidade`, `Relacao`, `criar_mundo`
Entidade	atributos, tags, inventário, `obter`, `definir`, `guardar`
Mundo	`criar_entidade`, `relacionar`, `relacionados`, `caminho`, `atualizar`
`coral.rpg`	`Personagem`, `Rolagem`, `CombateTurnos`, `rolar`
Rolagem	expressão, resultados individuais, modificador e `total`
Combate	participantes, ordem, índice, rodada e participante atual

`Mundo.para_json` exporta o estado genérico do mundo. Não é um salvamento integral de campanha: vida específica de personagem, estado de combate e lógica de retorno precisam de modelagem própria. Não prometa persistência completa usando somente essa função.

Projeto curto: Mapa de ocupação

Crie uma camada de ocupação num mapa quadrado e consulte uma célula ocupada e outra livre, sem janela.

A orientação de solução está no Apêndice B, seção Projeto 33.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`BackendNulo(...)` · **módulo** `coral.jogos`

O que faz

Simula entrada e desenho sem criar uma janela.

Construtor

Assinatura para consulta

```
BackendNulo(*, dt_fixo: 'float | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
dt_fixo	float | None	não	None	Intervalo de tempo fixo do backend, quando fornecido.

Atributos importantes

Atributo	Significado
`quadros`	Quantidade de quadros apresentados na simulação.

Métodos principais

Assinatura para consulta

```
definir_teclas(self, *teclas: 'str') -> 'None'
```

Referência para leitura

```
definir_mouse(self, *, posicao: 'tuple[int, int]' = (0, 0), pressionados=(),
  clicados=(), soltos=(), roda: 'tuple[int, int]' = (0, 0)) -> 'None'
```

Referência para leitura

```
processar_eventos(self) -> 'bool'
```

Retorno

Instância de BackendNulo.

Exemplo

Código Coral

```
de coral.jogos importe BackendNulo
defina backend como BackendNulo()
mostre backend.quadros
```

Resultado

Saída esperada

```
0
```

Atenção

A criação do objeto de dados não inicia a execução de um jogo. A simulação sem janela não comprova comportamento de uma janela ou dispositivo físico.

`Camera(...)` · módulo `coral.jogos`

O que faz

Controla a posição e a transformação da região visualizada.

Construtor

Assinatura para consulta

```
Camera(x: 'float' = 0.0, y: 'float' = 0.0, largura: 'float' = 800.0, altura: 'float' =
  450.0, alvo: 'Any | None' = None, suavidade: 'float' = 1.0, zoom: 'float' = 1.0,
  rotacao: 'float' = 0.0, zoom_minimo: 'float | None' = None, zoom_maximo: 'float | None'
  = None, deslocamento_visual_x: 'float' = 0.0, deslocamento_visual_y: 'float' = 0.0,
  constante_tempo: 'float | None' = None, zona_morta: 'tuple[float, float] | None' = None,
   enquadramento_x: 'float' = 0.0, enquadramento_y: 'float' = 0.0, antecipacao: 'float' =
  0.0, limites: 'Retangulo | None' = None, travar_x: 'bool' = False, travar_y: 'bool' =
  False) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	float	não	0.0	Coordenadas horizontais da série, ou posição horizontal do objeto.
y	float	não	0.0	Coordenadas verticais da série, ou posição vertical do objeto.
largura	float	não	800.0	Largura em unidades da apresentação.
altura	float	não	450.0	Altura em unidades da apresentação.
alvo	Any | None	não	None	Objeto que a câmera acompanha.
suavidade	float	não	1.0	Fator de suavização usado no acompanhamento.
zoom	float	não	1.0	Fator de ampliação da câmera.
rotacao	float	não	0.0	Ângulo de rotação da apresentação.
zoom_minimo	float | None	não	None	Limite inferior permitido para zoom.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
zoom_maximo	float | None	não	None	Limite superior permitido para zoom.
deslocamento_visual_x	float	não	0.0	Deslocamento horizontal adicional da apresentação.
deslocamento_visual_y	float	não	0.0	Deslocamento vertical adicional da apresentação.
constante_tempo	float | None	não	None	Constante temporal da suavização quando configurada.
zona_morta	tuple[float, float] | None	não	None	Região em que o alvo pode se mover sem deslocar a câmera.
enquadramento_x	float	não	0.0	Deslocamento horizontal desejado no enquadramento.
enquadramento_y	float	não	0.0	Deslocamento vertical desejado no enquadramento.
antecipacao	float	não	0.0	Fator de antecipação do movimento do alvo.
limites	Retangulo | None	não	None	Retângulo que limita o deslocamento da câmera.
travar_x	bool	não	False	Impede acompanhamento horizontal.
travar_y	bool	não	False	Impede acompanhamento vertical.

Atributos importantes

Atributo	Significado
`x`	Posição horizontal da câmera.
`y`	Posição vertical da câmera.
`largura`	Largura da área visualizada.
`altura`	Altura da área visualizada.
`zoom`	Fator de ampliação.

Métodos principais

Assinatura para consulta

```
seguir(self, sprite: 'Sprite', suavidade: 'float' = 1.0, *, constante_tempo: 'float |
  None' = None, zona_morta: 'tuple[float, float] | None' = None, enquadramento:
  'tuple[float, float] | None' = None, antecipacao: 'float | None' = None, travar_x: 'bool
   | None' = None, travar_y: 'bool | None' = None)
```

Referência para leitura

```
atualizar(self, dt: 'float | None' = None)
```

Referência para leitura

```
mundo_para_tela(self, x: 'float', y: 'float') -> 'tuple[float, float]'
```

Referência para leitura

```
tela_para_mundo(self, x: 'float', y: 'float') -> 'tuple[float, float]'
```

Retorno

Instância de Camera.

Exemplo

Código Coral

```
de coral.jogos importe Camera
defina camera como Camera(0, 0, 64, 64)
mostre camera.largura
```

Resultado

Saída esperada

```
64.0
```

Atenção

A criação do objeto de dados não inicia a execução de um jogo. A simulação sem janela não comprova comportamento de uma janela ou dispositivo físico.

`RenderizadorMapa2D(...)` · módulo `coral.jogos`

O que faz

Converte as células de um mapa em comandos de desenho 2D.

Construtor

Assinatura para consulta

```
RenderizadorMapa2D(mapa: 'Mapa', *, camadas: 'Iterable[str] | None' = None, camera:
  'Camera | None' = None, resolvedor_estilo: 'ResolvedorEstiloMapa | None' = None,
  area_visivel: 'tuple[float, float, float, float] | None' = None) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mapa	Mapa	sim	sem padrão	Mapa que deve ser visualizado.
camadas	Iterable[str] | None	não	None	Nomes das camadas a considerar.
camera	Camera | None	não	None	Câmera que controla a região e transformação da visualização.
resolvedor_estilo	ResolvedorEstiloMapa | None	não	None	Função que define a aparência de uma célula.
area_visivel	tuple[float, float, float, float] | None	não	None	Retângulo de mundo que delimita a consulta visual.

Atributos importantes

Atributo	Significado
`mapa`	Mapa consultado pelo renderizador.
`camera`	Câmera fornecida, quando houver.
`area_visivel`	Limite explícito de consulta quando usado.

Métodos principais

Assinatura para consulta

```
celulas_visiveis(self) -> 'tuple[Any, ...]'
```

Referência para leitura

```
desenhar(self, backend: 'BackendJogos') -> 'int'
```

Retorno

Instância de RenderizadorMapa2D.

Exemplo

Código Coral

```
de coral.jogos importe RenderizadorMapa2D, BackendNulo
crie um mapa chamado sala com topologia quadrada
adicione uma camada chamada terreno ao mapa sala
coloque o bloco "parede" na célula (1, 1) da camada terreno do mapa sala
defina renderizador como RenderizadorMapa2D(sala, area_visivel=(0, 0, 64, 64))
mostre renderizador.desenhar(BackendNulo())
```

Resultado

Saída esperada

```
1
```

Atenção

A criação do objeto de dados não inicia a execução de um jogo. A simulação sem janela não comprova comportamento de uma janela ou dispositivo físico.

O que você aprendeu

Uma célula é discreta; uma posição pode ser contínua. Camadas separam informações, e uma célula ausente não equivale a guardar zero ou `nulo`.

Prática

Modele dois objetos e uma relação fora do domínio de jogos.

Conferir a resposta comentada

Um laboratório pode modelar equipamentos e medições, com uma relação indicando o equipamento que gerou cada medição. Mundo oferece entidades e relações sem exigir janela ou combate.

Resposta do capítulo 33 no Apêndice B.

34. RPG

Objetivo de leitura

Criar personagens e simular ações com estado e sementes explícitos.

Antes de começar

Conheça Mundo e regras, dos capítulos 32 e 33.

`coral.rpg` é uma camada pequena construída sobre `coral.mundo`. O objetivo é oferecer peças úteis para aventuras, protótipos e demonstrações, sem tentar substituir sistemas completos de RPG.

Personagens

Código Coral

```
crie um mundo chamado campanha com nome "Campanha"
crie um personagem chamado heroina com nome "Coralina" no mundo campanha com 12 de vida
crie um personagem chamado slime com nome "Slime de Musgo" no mundo campanha com 6 de
  vida

defina o atributo "forca" de heroina como 2
```

Este exemplo não imprime texto no terminal.

Dados

Código Coral

```
role um dado de 20 lados como ataque
role 3 dados de 6 lados como dano
role um dado de 6 lados usando a semente 7 como dado
```

Este exemplo não imprime texto no terminal.

Sementes permitem exemplos e testes reproduzíveis.

Dano e cura

Código Coral

```
crie um mundo chamado campanha com nome "Campanha"
crie um personagem chamado heroina no mundo campanha com 12 de vida
crie um personagem chamado slime no mundo campanha com 20 de vida
role um dado de 6 lados usando a semente 42 como dado
cause 3 de dano a slime
cause dado.total de dano a slime como dano_causado
cure 2 de vida de heroina
```

Este exemplo não imprime texto no terminal.

Combate por turnos

Código Coral

```
crie um mundo chamado campanha com nome "Campanha"
crie um personagem chamado heroina no mundo campanha com 12 de vida
crie um personagem chamado slime no mundo campanha com 20 de vida
crie um combate chamado batalha com heroina e slime
inicie o combate batalha usando a semente 4
avance o combate batalha
```

Este exemplo não imprime texto no terminal.

A camada fornece rolagens, personagens, vida, atributos, testes e ordem de turnos. O modelo de mundo, relações e eventos continua vindo de `coral.mundo`.

API avançada

Classes e funções como `Personagem`, `CombateTurnos` e `rolar(...)` permanecem disponíveis para extensão e interoperabilidade. A documentação introdutória prioriza as frases naturais acima.

Projeto curto: Duelo reproduzível

Prepare dois personagens com estado inicial fixo e uma semente conhecida para as jogadas. Registre turnos e estado final.

A orientação de solução está no Apêndice B, seção Projeto 34.

O que você aprendeu

RPG acrescenta vida, dados e turnos sobre o modelo de Mundo. Uma semente conhecida permite reproduzir as rolagens sob o mesmo contrato.

Prática

Identifique os dados que devem ser reproduzíveis numa simulação de combate.

Conferir a resposta comentada

Estado inicial, participantes, regras, ordem de turno e semente ou resultados das jogadas precisam ser controlados. Uma vitória observada numa única execução não comprova equilíbrio.

Resposta do capítulo 34 no Apêndice B.

35. Jogos

Objetivo de leitura

Reconhecer jogo, cena e sprite e testar a lógica antes da apresentação gráfica.

Antes de começar

Conheça eventos e objetos, dos capítulos 16 e 32.

A Coral usa Pygame CE como backend gráfico oficial, mas a superfície principal é `coral.jogos`. O programa deve poder ser lido como uma descrição do jogo, sem obrigar o usuário a escrever o código repetitivo necessário para configurar o backend gráfico.

Instalação do backend gráfico

Terminal

```
python -m pip install pygame-ce
coral jogos-check
```

Exemplo recomendado

Código Coral

```
de coral.jogos importe AZUL e AMARELO

crie um jogo chamado jogo com largura 800 e altura 450 sem janela
crie uma cena chamada principal
crie um sprite chamado jogador em 100 e 180 com largura 40 e altura 40 e cor AMARELO

adicione jogador à cena principal
mude jogo para a cena principal

quando o jogo jogo atualizar com dt
    se a tecla "direita" estiver pressionada no jogo jogo então
        mova jogador 180 vezes dt para a direita
    fim

    se a tecla "esquerda" estiver pressionada no jogo jogo então
        mova jogador 180 vezes dt para a esquerda
    fim
fim

quando o jogo jogo desenhar
    limpe jogo com AZUL
    desenhe o texto "Use as setas" em 20 e 20 no jogo jogo
fim

execute jogo.executar(max_quadros=2)
```

Este exemplo não imprime texto no terminal.

Recursos atuais

A API inclui:

- janela e loop de jogo;
- teclado;
- desenho de formas e texto;
- sprites;
- cenas;
- colisão retangular;
- assets;
- animações;
- câmera simples;
- áudio;
- regras e eventos integrados ao loop.

Geometria compartilhada

Neste modelo, `coral.jogos` continua 2D, mas colisões de sprites reutilizam as relações geométricas gerais da base espacial. A direção da dependência permanece única: Jogos conhece a infraestrutura espacial; a base espacial não conhece sprites, cenas nem Pygame. O contrato histórico de Jogos também é preservado: sobreposição interior conta como colisão, enquanto apenas encostar pelas bordas não conta.

O exemplo `Exemplos/Jogos/01_geometria_compartilhada.coral` executa sem janela e sem Pygame. Ele compara dois sprites sobrepostos, um terceiro apenas tangente e usa, no mesmo programa, uma consulta espacial geral sobre posições comuns. A versão equivalente na biblioteca geral fica em `Exemplos/Citados/Jogos/04_geometria_compartilhada.coral`.

Mouse e tela cheia

A Coral oferece mouse e tela cheia parte do contrato público de `coral.jogos`. `criar_jogo(..., tela_cheia=verdadeiro)` permite iniciar no modo de tela cheia pela API avançada, e `Jogo` pode consultar ou alternar esse estado durante a execução. O comportamento padrão permanece em janela quando a opção é omitida.

A entrada de mouse diferencia estado contínuo de eventos do quadro. `posicao_mouse()` e `botao_mouse_pressionado(...)` consultam o estado atual; `mouse_clicado(...)`, `mouse_solto(...)` e `roda_mouse()` representam eventos observados no quadro corrente. Essa distinção evita confundir manter um botão pressionado com um novo clique. Os nomes de botão aceitos incluem `esquerdo`, `meio` e `direito`.

No `BackendNulo`, `definir_mouse(...)` fixa um estado e `programar_mouse(...)` descreve quadros determinísticos, permitindo testar interação sem abrir janela. O mesmo backend registra mudanças de tela cheia. A abertura real de janela, a entrada do sistema e o modo de tela cheia continuam dependendo do Pygame CE e do ambiente gráfico.

Folhas de sprites e transformações visuais

A biblioteca oferece folhas de sprites para uso sem janela sem tornar Pygame uma dependência do núcleo. `Spritesheet` descreve uma folha ou atlas, `QuadroSprite` representa um recorte imutável, e animações podem consumir esses quadros sem abandonar o formato antigo baseado em caminhos de imagens.

`Sprite` oferece transformação visual independente da colisão e do estado de `coral.mundo`: escala uniforme, rotação em graus, espelhamento horizontal e vertical, origem e pivô local. As origens estruturadas reconhecidas são `topo_esquerdo`, `centro` e `centro_inferior`. Um pivô personalizado é informado como `(x, y)` nas unidades locais anteriores à escala. O ponto escolhido permanece ancorado na posição do sprite enquanto a representação visual é transformada.

A chamada estruturada `Jogo.desenhar_imagem(...)` aceita os mesmos parâmetros opcionais. Omiti los preserva a semântica anterior. O `BackendNulo` registra comandos transformados sem abrir arquivos ou janela; o backend Pygame aplica as operações à imagem carregada ou ao quadro recortado. A abertura de janela e o resultado visual físico continuam sujeitos a teste de campo com Pygame CE.

Estas interfaces pertencem à camada avançada e interoperável. Elas foram mantidas como fundação técnica até os gates de contrato, erros e regressão ficarem verdes; só então partes com ganho pedagógico claro começaram a receber frases Coral naturais.

Na implementação atual, a integração com `coral.mundo` ganha uma ponte visual opcional do lado de `coral.jogos`. `Cena.representar(...)` associa uma entidade a um `Sprite` por atributos de posição explicitamente nomeados, mantendo a entidade como fonte de verdade. A dependência não é invertida: `coral.mundo` continua sem importar jogos, sprites ou Pygame, de modo que a mesma entidade pode existir em simulações sem janela e em aplicações sem interface.

A biblioteca atual não mantém mais o antigo laboratório versionado de desenvolvimento. O exemplo `Exemplos/Citados/Jogos/02_sprites_transformacoes_e_animacao.coral` concentra a superfície estável de sprites, transformações e automações em modo sem janela; validações históricas de spritesheets, recortes e estados permanecem na suíte de regressão.

Na implementação atual, uma primeira fatia da superfície natural é promovida para parser/AST: `carregue a spritesheet ...`, `pegue o quadro ...`, `crie uma animação ...`, `use o quadro ...`, `use a animação ...` e `represente ... com o sprite ... na cena ...`. Essas frases compilam para a API estrutural já estabilizada e são conhecidas pela semântica, tabela de símbolos, Naturalidade, Ensinar e LSP. Métodos diretos continuam válidos como API avançada, mas a documentação para iniciantes prioriza as frases Coral.

O exemplo `Exemplos/Citados/Jogos/02_sprites_transformacoes_e_animacao.coral` demonstra a superfície estável de sprites sem janela e sem Pygame CE.

Na implementação atual, as transformações visuais estabilizadas recebem a superfície natural `ajuste a escala do sprite ...`, `gire o sprite ... em ... graus`, `espelhe o sprite ...` e `ancore o sprite ...`. A ancoragem por origem aceita topo esquerdo, centro

e centro inferior; selecionar uma origem limpa um pivô personalizado anterior. `ancore ... no pivô X e Y` volta a definir um ponto local personalizado que prevalece na transformação. O editor e as ferramentas de análise reconhecem essas formas.

O mesmo exemplo demonstra escala, rotação, espelhamento, ancoragem, colisão associada e automação matemática sem janela.

Na implementação atual, o modelo direcional deixa de tratar `cima`, `baixo`, `esquerda` e `direita` como uma lista estrutural fixa. A base compartilhada passa a ser um conjunto arbitrário de direções nomeadas, cada uma associada a um vetor bidimensional. Duas, quatro e oito direções são apenas predefinições convenientes do mesmo mecanismo. Vetores diagonais são normalizados para preservar a distância efetiva do movimento.

Código Coral

```
de coral.jogos importe Sprite
defina jogador como Sprite(0, 0, 10, 10)
crie um conjunto de direções chamado diagonais
    direção nordeste com vetor 1 e -1
    direção sudoeste com vetor -1 e 1
fim
use movimento com diagonais em jogador
mova jogador 10 para nordeste
```

Este exemplo não imprime texto no terminal.

A animação direcional usa os mesmos nomes livres e usa um bloco declarativo:

Código Coral

```
de coral.jogos importe carregar_spritesheet
defina folha como carregar_spritesheet("heroi.png", 192, 64, 32, 32)
crie uma animação direcional chamada andar usando folha
    direção esquerda na linha 0
    direção direita na linha 1
    quadros 0 até 5
    velocidade 10 quadros por segundo
    repetir
fim
```

Este exemplo não imprime texto no terminal.

Movimento e apresentação não precisam possuir a mesma quantidade de direções. `use movimento em 8 direções no jogador` pode ser ligado a uma animação com apenas esquerda e direita por `ligue a animação andar ao movimento de jogador no sprite visual`. A ponte visual escolhe a direção disponível mais próxima do vetor lógico. O estado também separa direção de movimento e direção de olhar: `pare o movimento de jogador` limpa somente o movimento atual e conserva para onde o alvo estava olhando.

O núcleo compartilhado do movimento direcional vive internamente em `coral.direcoes` e ainda não constitui uma nova biblioteca pública estável. `coral.mundo` continua sem importar sprites, animações ou Pygame, enquanto `coral.jogos` consome o estado direcional apenas para apresentação. O exemplo `Exemplos/Citados/Jogos/02_sprites_transformacoes_e_animacao.coral` reúne a demonstração atual em modo sem janela.

BackendNulo

`BackendNulo` executa a lógica sem abrir uma janela. Ele existe para testes automatizados, CI, simulações e desenvolvimento em ambientes sem vídeo. Isso permite testar a maior parte da API mesmo sem Pygame CE instalado.

Pygame CE

Pygame CE é o backend gráfico oficial de `coral.jogos`. A documentação do Pygame CE cobre janela/display, desenho, eventos, teclado, imagens, áudio, tempo e sprites, que são as capacidades sobre as quais a camada Coral é construída.

API avançada

O runtime também pode ser manipulado por classes e métodos (`Jogo`, `Cena`, `Sprite`, `jogo.executar()`, etc.). Essa camada é válida para extensão da engine e interoperabilidade, mas os templates oficiais priorizam a sintaxe natural Coral quando ela existe.

Instalar a dependência correta

Pygame CE é o backend gráfico oficial. O runtime básico funciona sem ele. Instale Pygame usando o Python da instalação persistente. No Linux, com a pasta padrão:

Terminal

```
"$HOME/.local/share/coral/ambiente/bin/python" -m pip install pygame-ce
coral jogos-check
```

Se você mudou `XDG_DATA_HOME` ou o destino, use o caminho mostrado pelo diagnóstico. A detecção de `pygame` comprova importação, mas não comprova abertura de janela, áudio ou comportamento de drivers.

Criar um jogo

Terminal

```
coral novo MeuJogo --template jogo-2d
cd MeuJogo
coral executar
```

O template contém as importações de cores, uma cena e um sprite controlado pelo teclado. No evento de atualização, `dt` representa o tempo do quadro em segundos. Multiplicar a velocidade por `dt` permite expressar deslocamento por segundo em vez de deslocamento fixo por quadro.

Ler a arquitetura do jogo

Peça	Responsabilidade
Jogo	janela, eventos, ritmo de atualização e encerramento
Cena	agrupar sprites e organizar entrada, saída, atualização e desenho
Sprite	posição, dimensões, aparência, velocidade e colisão retangular
Assets	localizar arquivos de imagem e som dentro de uma raiz
Animacao	alternar quadros ao longo do tempo
Camera	transformar coordenadas do mundo para a tela
MotorRegras	avaliar regras e temporizadores durante o ciclo
BackendNulo	exercitar lógica e registrar comandos sem janela

Cores como `AZUL` e `AMARELO` precisam ser importadas. Um sprite com imagem depende de um arquivo existente. Mantenha caminhos de assets relativos à pasta de trabalho prevista. O build de projetos não copia automaticamente os arquivos de mídia.

Teste sem janela

Arquivo: `Exemplos/Citados/Jogos/01_jogo_e_entrada_sem_janela.coral`.

Código Coral

```
de coral.jogos importe criar_jogo e BackendNulo
defina backend como BackendNulo()
defina jogo como criar_jogo("Teste", backend=backend)
execute jogo.executar(max_quadros=2)
mostre backend.quadros
```

Saída esperada

```
2
```

Essa é uma demonstração deliberada da API avançada para testes. Ela limita a execução a dois quadros e verifica o contador do backend. Não prova que o mesmo ambiente possui suporte gráfico. Para desenvolvimento de um jogo real, teste também janela, teclado, encerramento e áudio no seu computador.

A sintaxe natural completa de criação e callbacks está exemplificada no template `jogo-2d` e em `Documentacao/Desenvolvimento/Projeto/JOGOS.md`. Os componentes `Sprite`, `Cena`, `Assets`, `Animacao` e `Camera` possuem interfaces avançadas em `coral/jogos/objetos.py`.

Construir o primeiro jogo, passo a passo

O arquivo `principal.coral` do template `jogo-2d` tem o programa abaixo. Esta versão de estudo executa dois quadros sem janela, para permitir a verificação automática. Para abrir uma janela, retire `sem janela` da criação do jogo e substitua a última instrução por `inicie jogo`. Esse modo exige Pygame CE e ambiente gráfico e permanece ativo até o encerramento do jogo.

Código Coral

```
# Versão de estudo: executa dois quadros sem janela.
de coral.jogos importe AZUL e AMARELO

crie um jogo chamado jogo com largura 800 e altura 450 sem janela
crie uma cena chamada principal
crie um sprite chamado jogador em 100 e 180 com largura 40 e altura 40 e cor AMARELO
adicione jogador à cena principal
mude jogo para a cena principal

quando o jogo jogo atualizar com dt
    se a tecla "direita" estiver pressionada no jogo jogo então
        mova jogador 180 vezes dt para a direita
    fim
    se a tecla "esquerda" estiver pressionada no jogo jogo então
        mova jogador 180 vezes dt para a esquerda
    fim
fim

quando o jogo jogo desenhar
    limpe jogo com AZUL
    desenhe o texto "Use ← e -> para mover" em 20 e 20 no jogo jogo com tamanho 24
fim

execute jogo.executar(max_quadros=2)
```

Este exemplo não imprime texto no terminal.

1. A importação disponibiliza as duas cores. A criação do jogo define dimensões de 800 por 450 pixels, mesmo neste modo sem janela.
2. A cena reúne o conteúdo visível. O sprite começa em x=100 e y=180, com largura e altura de 40 pixels. Adicioná lo à cena e selecionar a cena são operações separadas.
3. O evento de atualização consulta o estado das setas a cada quadro. `180 vezes dt` desloca o jogador de acordo com o tempo decorrido. As coordenadas de tela crescem para a direita e para baixo.
4. O evento de desenho limpa o fundo e escreve a instrução. Em seguida, o ciclo desenha a cena, incluindo seus sprites. Desenho deve ocorrer novamente a cada quadro.
5. `execute jogo.executar(max_quadros=2)` limita o estudo a dois quadros. No modo com janela, `inicie jogo` entra no ciclo de execução até o encerramento. Para começar, altere a cor, a velocidade e a posição inicial, uma mudança por vez.

O programa não impede o jogador de sair da janela: limites e regras de colisão pertencem à lógica do jogo. Também não carrega imagens ou sons. Acrescente esses recursos depois de conferir o movimento básico.

Referência prática dos objetos gráficos

A superfície natural é a primeira opção para iniciar jogos. As assinaturas abaixo documentam a API de componentes para integração e comportamentos mais específicos. `objeto.metodo(...)` usa a instância que você criou. `self` não é passado nas chamadas.

Operação	Contrato
`criar_jogo(titulo, largura=800, altura=450, fps=60, backend=None)`	Cria o jogo; backend omitido seleciona Pygame
`jogo.executar(max_quadros=None)`	Executa até o encerramento; o limite é útil em testes
`jogo.parar()`	Solicita encerramento do ciclo
`jogo.mudar_cena(cena)`	Sai da cena anterior e entra na nova
`jogo.tecla_pressionada(tecla)`	Consulta uma tecla no backend
`jogo.tocar_som(caminho, volume=1.0)`	Reproduz o arquivo pelo backend; depende de áudio disponível
`jogo.desenhar_imagem(caminho, x, y, largura=None, altura=None, ...)`	Desenha imagem ou `QuadroSprite`; na API atual aceita escala, rotação, espelhamento, origem e pivô opcionais
`Cena(nome)`	Cria uma coleção de sprites e callbacks
`cena.adicionar(sprite)`	Adiciona e devolve o sprite
`cena.remover(sprite)`	Remove se estiver presente
`Sprite(x, y, largura, altura, imagem=None, cor=BRANCO)`	Parâmetros iniciais do sprite; também aceita velocidades, visibilidade, nome, dados, animação e, na API atual, escala, rotação, espelhamento, origem e pivô
`sprite.mover(dx, dy)`	Soma deslocamentos à posição
`sprite.atualizar(dt)`	Avança velocidades e animação; a cena chama essa operação
`sprite.colide_com(outro)`	Testa sobreposição retangular; não resolve fisicamente a colisão
`Assets(raiz="assets")`	Define a raiz dos arquivos de mídia
`assets.imagem(nome)` / `assets.som(nome)`	Resolve um arquivo existente dentro da raiz; rejeita saída da raiz
`Animacao(quadros, fps=10.0, repetir=True)`	Exige ao menos um quadro e fps positivo
`animacao.atualizar(dt)`	Avança o tempo e devolve o quadro atual
`animacao.reiniciar()`	Retorna ao primeiro quadro e zera o tempo
`Camera(x=0, y=0, largura=800, altura=450)`	Define o deslocamento e a área da câmera
`camera.seguir(sprite, suavidade=1.0)`	Define o alvo; suavidade é limitada ao intervalo de 0 a 1
`camera.tela(x, y)`	Converte uma posição do mundo em posição da tela

Para usar uma câmera, atribua sua instância a `cena.camera`. A cena atualiza a câmera e transforma as posições ao desenhar. Para animar, atribua uma `Animacao` a `sprite.animacao`; seus quadros representam caminhos de imagens. A atribuição de

`sprite.velocidade_x` dispensa movimentá lo manualmente por essa mesma velocidade no callback: fazer ambos soma os dois deslocamentos.

Callbacks do jogo recebem `(jogo)`, exceto atualização, que recebe `(jogo, dt)`. Callbacks da cena recebem `(jogo, cena)`; atualização da cena recebe `(jogo, cena, dt)`. Registre funções com `quando_iniciar`, `quando_atualizar`, `quando_desenhar` e `quando_encerrar` no jogo, ou com os registradores correspondentes de entrada, saída, atualização e desenho na cena. Uma assinatura incompatível causa erro na execução.

Projeto curto: Movimento sem janela

Execute dois quadros com BackendNulo e verifique a posição de um sprite antes e depois de um movimento.

A orientação de solução está no Apêndice B, seção Projeto 35.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`Sprite(...)` **· módulo** `coral.jogos`

O que faz

Mantém posição, tamanho, apresentação e dados de um elemento visual.

Construtor

Assinatura para consulta

```
Sprite(x: 'float', y: 'float', largura: 'float', altura: 'float', imagem: 'str | Path |
  QuadroSprite | None' = None, cor: 'Cor' = Cor(vermelho=255, verde=255, azul=255,
  alfa=255), velocidade_x: 'float' = 0.0, velocidade_y: 'float' = 0.0, visivel: 'bool' =
  True, nome: 'str' = 'sprite', dados: 'dict[str, Any]' = <factory>, animacao: 'Any' =
  None, escala: 'float' = 1.0, rotacao: 'float' = 0.0, espelhado_horizontalmente: 'bool' =
   False, espelhado_verticalmente: 'bool' = False, origem: 'str' = 'topo_esquerdo', pivo:
  'tuple[float, float] | None' = None, opacidade: 'float' = 1.0, colisao_acompanha_escala:
   'bool' = False, colisao_acompanha_rotacao: 'bool' = False) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	float	sim	sem padrão	Coordenadas horizontais da série, ou posição horizontal do objeto.
y	float	sim	sem padrão	Coordenadas verticais da série, ou posição vertical do objeto.
largura	float	sim	sem padrão	Largura em unidades da apresentação.
altura	float	sim	sem padrão	Altura em unidades da apresentação.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
imagem	str | Path | QuadroSprite | None	não	None	Imagem ou quadro do sprite; nulo usa sua representação de cor.
cor	Cor	não	Cor(vermelho=255, verde=255, azul=255, alfa=255)	Cor de apresentação do sprite.
velocidade_x	float	não	0.0	Velocidade horizontal.
velocidade_y	float	não	0.0	Velocidade vertical.
visivel	bool	não	True	Se verdadeiro, o sprite participa da apresentação.
nome	str	não	'sprite'	Nome da série ou do objeto.
dados	dict[str, Any]	não	<factory>	Dicionário livre de dados associados ao sprite; a fábrica cria um dicionário independente.
animacao	Any	não	None	Animação associada ao sprite.
escala	float	não	1.0	Fator de escala visual.
rotacao	float	não	0.0	Ângulo de rotação da apresentação.
espelhado_horizontalmente	bool	não	False	Se verdadeiro, inverte horizontalmente a imagem.
espelhado_verticalmente	bool	não	False	Se verdadeiro, inverte verticalmente a imagem.
origem	str	não	'topo_esquerdo'	Ponto de referência da posição visual.
pivo	tuple[float, float] | None	não	None	Ponto usado como pivô de transformações.
opacidade	float	não	1.0	Intensidade de opacidade entre zero e um.
colisao_acompanha_escala	bool	não	False	Se verdadeiro, a forma de colisão acompanha a escala visual.
colisao_acompanha_rotacao	bool	não	False	Se verdadeiro, a forma de colisão acompanha a rotação visual.

Atributos importantes

Atributo	Significado
`x`	Posição horizontal.
`y`	Posição vertical.
`largura`	Largura da forma base.
`altura`	Altura da forma base.

Atributo	Significado
`dados`	Dados associados ao objeto.

Métodos principais

Assinatura para consulta

```
mover(self, dx: 'float', dy: 'float') -> 'None'
```

Referência para leitura

```
atualizar(self, dt: 'float') -> 'None'
```

Referência para leitura

```
colide_com(self, outro: "'Sprite'") -> 'bool'
```

Retorno

Instância de Sprite.

Exemplo

Código Coral

```
de coral.jogos importe Sprite
defina jogador como Sprite(0, 0, 10, 10)
mostre jogador.largura
```

Resultado

Saída esperada

```
10
```

Atenção

A criação do objeto de dados não inicia a execução de um jogo. A simulação sem janela não comprova comportamento de uma janela ou dispositivo físico.

`criar_jogo(...)` · módulo `coral.jogos`

O que faz

Cria um jogo com dimensões, backend e fontes explicitamente configuráveis.

Assinatura

Assinatura para consulta

```
criar_jogo(titulo: 'str', largura: 'int' = 800, altura: 'int' = 450, fps: 'int' = 60, *,
   backend=None, mundo=None, relogio=None, eventos=None, tela_cheia: 'bool' = False) ->
  'Jogo'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
titulo	str	sim	sem padrão	Valor titulo usado na operação.
largura	int	não	800	Largura em unidades da apresentação.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
altura	int	não	450	Altura em unidades da apresentação.
fps	int	não	60	Frequência alvo de quadros por segundo.
backend	valor dinâmico	não	None	Backend que realiza desenho e entrada; BackendNulo permite executar sem janela.
mundo	valor dinâmico	não	None	Mundo associado ao jogo, quando houver.
relogio	valor dinâmico	não	None	Fonte de tempo que o jogo usa.
eventos	valor dinâmico	não	None	Motor de eventos do jogo.
tela_cheia	bool	não	False	Se verdadeiro, pede modo de tela cheia ao backend.

Retorno

Jogo, ainda sem iniciar seu laço.

Exemplo

Código Coral

```
de coral.jogos importe criar_jogo, BackendNulo
defina jogo como criar_jogo("Teste", backend=BackendNulo())
execute jogo.executar(max_quadros=2)
mostre jogo.backend.quadros
```

Resultado

Saída esperada

```
2
```

`carregar_spritesheet(...)` · módulo `coral.jogos`

O que faz

Define uma folha de sprites dividida em quadros numa grade declarada.

Assinatura

Assinatura para consulta

```
carregar_spritesheet(caminho: 'str | Path', largura_imagem: 'int', altura_imagem: 'int',
   largura_quadro: 'int', altura_quadro: 'int', *, margem: 'int' = 0, espacamento: 'int' =
   0) -> 'Spritesheet'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	str | Path	sim	sem padrão	Caminho local do arquivo de saída ou da imagem de origem.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
largura_imagem	int	sim	sem padrão	Largura total da imagem declarada.
altura_imagem	int	sim	sem padrão	Altura total da imagem declarada.
largura_quadro	int	sim	sem padrão	Largura de cada quadro da grade.
altura_quadro	int	sim	sem padrão	Altura de cada quadro da grade.
margem	int	não	0	Margem entre a borda da imagem e a grade.
espacamento	int	não	0	Separação entre quadros.

Retorno

Spritesheet.

Exemplo

Código Coral

```
de coral.jogos importe carregar_spritesheet
defina folha como carregar_spritesheet("heroi.png", 96, 64, 32, 32)
mostre len(folha)
```

Resultado

Saída esperada

```
6
```

Atenção

O exemplo descreve a grade. A renderização da imagem exige o recurso correspondente; não está sendo aberta uma janela.

O que você aprendeu

A atualização avança o estado; o desenho apresenta esse estado. `BackendNulo` permite testes sem janela; uma janela real exige o backend gráfico.

Prática

Explique a vantagem de testar um jogo com `` `BackendNulo` ``.

Conferir a resposta comentada

O backend permite exercitar o estado e o fluxo sem janela nem renderização física. Ele não comprova desempenho gráfico ou funcionamento do dispositivo real.

Resposta do capítulo 35 no Apêndice B.

36. Espaço e geometria

Objetivo de leitura

Representar posições e consultar distâncias e relações em 2D e 3D.

Antes de começar

Conheça valores numéricos e coleções.

A Coral trata posição, distância, direção, formas, movimento e relações espaciais como conceitos gerais da linguagem. Eles pertencem à base espacial da linguagem e estão disponíveis **sem importação**. Não existe um módulo público separado para habilitar esses recursos. Jogos é apenas um consumidor 2D dessa infraestrutura.

Posições e formas em 2D

Código Coral

```
defina origem como posição(0, 0)
defina destino como posição(3, 4)
defina distancia_total como distância entre origem e destino
defina area_util como retângulo(0, 0, 100, 60)
mostre distancia_total
mostre area_util
```

Saída esperada

```
5.0
Retangulo(origem=Posicao(coordenadas=(0.0, 0.0)), largura=100.0, altura=60.0)
```

Exemplo completo: `Exemplos/Espaco/01_espaco_2d_e_3d.coral`.

As mesmas operações em 3D

Adicionar uma terceira coordenada cria um valor tridimensional real. A dimensão faz parte do contrato do valor; uma posição 2D não ganha `z = 0` implicitamente.

Código Coral

```
defina origem como posição(0, 0, 0)
defina destino como posição(2, 3, 6)
defina volume como caixa(0, 0, 0, 20, 30, 40)
defina distancia_total como distância entre origem e destino
mostre distancia_total
```

Saída esperada

```
7.0
```

Exemplo completo: `Exemplos/Espaco/01_espaco_2d_e_3d.coral`.

Relações geométricas

As relações são matemáticas e independentes de jogos. `intersecta` aceita contato de borda; `sobrepõe` exige interior comum; `toca` descreve contato sem sobreposição interior. Formas e regiões podem ser usadas em simulações, dados científicos, interfaces, mapas abstratos ou qualquer outro domínio que precise raciocinar espacialmente.

Consultas espaciais sobre coleções comuns

Uma consulta não exige cena, mundo nem classe base especial. A coleção pode conter valores espaciais diretamente ou objetos dos quais a Coral consiga obter posição ou forma.

Código Coral

```
defina sensores como [posição(1, 1), posição(4, 4), posição(20, 0)]
defina proximos como procure em sensores os que estão a até 6 de posição(0, 0)
defina mais_perto como procure em sensores o mais próximo de posição(3, 3)
mostre proximos
mostre mais_perto
```

Saída esperada

```
[Posicao(coordenadas=(1.0, 1.0)), Posicao(coordenadas=(4.0, 4.0))]
Posicao(coordenadas=(4.0, 4.0))
```

Exemplo completo: `Exemplos/Espaco/02_consultas_e_movimento.coral`. O mesmo arquivo mostra a consulta em 2D e 3D; a dimensionalidade vem dos dados.

Movimento também é geral

Velocidade, aceleração, integração por `dt` e limites não pertencem a `coral.jogos`. Isso permite modelar sondas, partículas, robôs, corpos simulados ou trajetórias sem abrir uma janela.

Código Coral

```
defina posicao_inicial como posição(0, 0, 200)
defina velocidade_inicial como velocidade(12, 4, menos 2)
defina gravidade como aceleração(0, 0, menos 9.8)
defina estado como integrar_movimento(posicao_inicial, velocidade_inicial, gravidade,
  dt=0.5)
mostre estado.posicao
```

Saída esperada

```
Posicao(coordenadas=(6.0, 2.0, 197.775))
```

Exemplo completo: `Exemplos/Espaco/02_consultas_e_movimento.coral`.

Relação com coral.jogos

`coral.jogos` permanece 2D nesta versão. Sprites e colisores reutilizam as formas e relações gerais, mas uma forma geométrica não é sinônimo de colisor. Isso permite que a mesma matemática seja útil dentro e fora de jogos sem transformar o restante da linguagem em uma API de game engine.

Projeto curto: Distâncias no espaço

Compare a distância de dois pontos em 2D com um exemplo em 3D e identifique a dimensão de cada dado.

A orientação de solução está no Apêndice B, seção Projeto 36.

O que você aprendeu

Geometria é geral e está disponível sem importação. Dados 2D e 3D têm dimensões próprias; contato de borda e sobreposição interior são relações diferentes.

Prática

Diferencie um vetor de posição e uma transformação em 3D.

Conferir a resposta comentada

O vetor representa coordenadas; a transformação especifica como alterar coordenadas. Em matrizes homogêneas, pontos e direções têm papéis diferentes e não devem ser confundidos.

Resposta do capítulo 36 no Apêndice B.

37. Procedural

Objetivo de leitura

Produzir dados reproduzíveis com sementes, campos e receitas.

Antes de começar

Conheça funções, coleções e sementes.

Uma operação procedural produz dados a partir de uma regra. Quando preservamos a receita e a semente, podemos repetir o resultado sob o mesmo contrato. Isso é útil para mapas, simulações e dados sintéticos.

Semente, campo e receita

`coral.procedural` é uma biblioteca geral de geração reproduzível. Ela não depende de Jogos, RPG nem interface gráfica. `Semente`, `GeradorProcedural`, `SequenciaProcedural` e `CampoProcedural` permitem derivar domínios independentes, consultar valores por chave ou coordenada sem depender da ordem de consultas anteriores, compor campos e distribuir pontos sob limites finitos. Na 1.6.15, pesos inválidos recebem diagnóstico explícito, campos declaram dimensionalidade e intervalos conhecidos, e a ordem de operações não comutativas permanece observável.

`ReceitaProcedural`, `receita_procedural`, `reconstruir_receita` e `diagnosticar_custo_receita` formam a camada declarativa portátil. As receitas suportadas registram algoritmo, versão, semente e parâmetros e podem ser preservadas

por `coral.persistencia`. Callbacks não entram na forma portátil. Reconstrução e materializações grandes possuem limites de custo explícitos, sem esconder trabalho ou alocação potencialmente desproporcional.

O módulo serve a simulações, mapas, ecologia, dados sintéticos e outros domínios. Jogos e Laboratório podem consumi lo, mas não são sua finalidade obrigatória.

Na 1.6.17, `amostrar_campo` e `matriz_campo` tornam a observação do campo uma operação explícita e limitada. `anotar_pontos` combina distribuições espaciais com grandezas como altitude, umidade ou risco. `proveniencia_procedural` fornece metadados portáteis sem afirmar que um callback arbitrário pode ser reconstruído.

A ponte com Jogos vive no consumidor: receitas de `distribuir_pontos` podem preencher cenas com sprites reprodutíveis e campos podem escrever valores em `sprite.dados`, sem abrir janela. A ponte com Laboratório trata receita e semente procedural como parte do desenho experimental, separadas da semente usada pelo próprio protocolo de medição. `planejar_experimento_procedural` calcula execuções e custo máximo antes de reconstruir a receita; `executar_experimento_procedural` preserva a receita base e as sementes nos metadados do resultado.

Procedural em jogos e pesquisa

A fundação procedural permite que o mesmo processo determinístico possa ser usado na construção de cenários de jogo e em experimentos reproduzíveis.

A regra central continua simples: uma semente e uma receita descrevem como produzir algo. Jogos decide como apresentar esse resultado. Laboratório decide como medir e comparar execuções. Procedural não precisa conhecer nenhum dos dois.

Usando pontos em uma cena

Uma receita de distribuição pode ser passada a `sprites_de_receita` ou `popular_cena_procedural`. O adaptador cria sprites nas mesmas posições sempre que a receita, a semente e a versão do contrato forem as mesmas. Nenhuma janela é aberta por essa conversão.

Campos procedurais podem ser aplicados aos sprites depois. O valor fica em `sprite.dados`, permitindo que um programa use, por exemplo, relevo para escolher aparência, fertilidade para crescimento ou perigo para regras de encontro.

Medindo uma receita

`planejar_experimento_procedural` combina sementes, parâmetros e repetições sem executar a receita. Além do número de execuções, ele mostra o custo máximo declarado. Isso é especialmente útil quando uma distribuição possui muitas tentativas possíveis.

`executar_experimento_procedural` reconstrói a receita para cada semente e entrega o produto à função estudada. Cada medição preserva a semente procedural, e o resultado

guarda a receita base nos metadados. Assim é possível comparar diferentes sementes sem perder de vista exatamente qual processo gerou cada observação.

Enviando campos a Gráficos

`amostrar_campo` cria registros gerais. Eles podem ser enviados a `series_de_registros`. `matriz_campo` cria uma matriz 2D que pode ser enviada diretamente a `mapa_calor`. A integração não exige que Gráficos conheça `CampoProcedural`.

Quando usar cada caminho

Use `amostrar_campo` quando você precisa preservar coordenadas individuais. Use `matriz_campo` para grades 2D finitas. Use `anotar_pontos` quando uma distribuição espacial precisa receber grandezas ambientais. Use as funções de Jogos quando o resultado vai virar objeto visual. Use as funções de Laboratório quando sementes ou parâmetros precisam ser comparados sob um protocolo experimental explícito.

Projeto curto: Campo com receita

Gere um campo procedural com receita e semente registradas. Repita a consulta em ordem diferente.

A orientação de solução está no Apêndice B, seção Projeto 37.

O que você aprendeu

Uma receita registra como gerar o resultado. Preserve algoritmo, versão, semente e parâmetros; limites de custo evitam materializações desproporcionais.

Prática

Explique como semente e receita ajudam a reproduzir um campo procedural.

Conferir a resposta comentada

A semente controla o gerador e a receita registra algoritmo, versão e parâmetros. A reconstrução só é garantida para operações e versões suportadas; callbacks arbitrários não viram receita portátil.

Resposta do capítulo 37 no Apêndice B.

38. Computação numérica e científica

Objetivo de leitura

Escolher operações numéricas e reconhecer seus requisitos e limites de interpretação.

Antes de começar

Conheça matemática e coleções; estatística básica ajuda nas seções avançadas.

`coral.numerico` reúne operações sobre séries, vetores e matrizes. Comece escolhendo o cálculo necessário e conferindo o formato dos dados. As fichas deste capítulo apresentam as assinaturas e exemplos completos; as seções de estatística avançada podem ser consultadas depois.

Uma série é uma sequência de valores. Uma matriz organiza valores em linhas e colunas. Dados finitos excluem valores como infinito e `NaN`, que representa um resultado numérico indefinido. Quando uma operação exige NumPy, o pacote precisa estar instalado; importar o módulo não carrega essa dependência antecipadamente.

Validar os dados antes de calcular

A API normaliza erros públicos relevantes como `ErroNumerico`, valida matrizes, estatísticas, intervalos e reduções vazias e evita vazar detalhes acidentais do backend NumPy para a superfície Coral. As estatísticas numéricas rejeitam dados ausentes, `NaN` e infinito com diagnóstico público em português. `selecionar_coluna` permite escolher explicitamente se registros ausentes geram erro ou são ignorados. Percentis e quartis usam interpolação linear declarada; correlação rejeita variância zero; moda preserva empates em vez de inventar uma única categoria vencedora.

A mesma política cobre álgebra linear e geometria 3D. `norma` e `normalizar` trabalham com vetores finitos; `posto` e `minimos_quadrados` validam formas; `produto_vetorial` é restrito a três dimensões. `rotacionar3` usa a fórmula de Rodrigues e ângulo em radianos. `transformar_ponto3` recebe uma matriz afim homogênea 4 × 4 e usa vetor coluna, mantendo a coordenada homogênea apenas como detalhe interno.

Nas operações estatísticas, `erro_padrao` usa desvio padrão amostral e exige pelo menos duas observações. `assimetria` e `curtose` usam momentos centrais padronizados e rejeitam variância zero; curtose é devolvida em excesso. `matriz_covariancia` interpreta linhas como observações e colunas como variáveis e `matriz_correlacao` rejeita colunas constantes. `regressao_linear` devolve inclinação, intercepto, resíduos, R^2 e quantidade de observações.

`intervalo_bootstrap` implementa intervalo percentil por reamostragem e `teste_permutacao` implementa teste bilateral por diferença da estatística escolhida pelo programa. Ambos usam um gerador local por semente, registram repetição e custo estimado e recusam trabalho acima do limite interno. Intervalos e valores p são resultados numéricos; a biblioteca não os transforma automaticamente em causalidade, superioridade ou significância substantiva.

A camada tridimensional oferece `matriz_rotacao3`, `matriz_translacao3`, `matriz_escala3`, `compor_transformacoes3` e `transformar_vetor3`. As relações ponto plano são cobertas por `distancia_ponto_plano3`, `projetar_ponto_plano3` e `intersecao_reta_plano3`. `coordenadas_esfericas3` e `ponto_de_esfericas3` fazem a conversão explícita entre representações, enquanto `area_triangulo3` e `volume_tetraedro3` tratam medidas geométricas sem recorrer ao backend gráfico.

Estatística avançada em `coral.numerico`

As operações desta seção fazem parte do runtime 1.7.6.

A implementação permanece em `coral.numerico`, com NumPy opcional na importação e exigido somente quando uma operação numérica que dele depende é chamada. SciPy não é dependência desta versão.

Medidas de forma e incerteza

`erro_padrao(valores)` calcula o erro padrão da média usando o desvio padrão amostral, com divisor `n - 1`, e exige ao menos duas observações finitas.

`assimetria(valores)` calcula o terceiro momento central padronizado sem correção de viés amostral. `curtose(valores)` calcula o quarto momento padronizado e devolve curtose em excesso, subtraindo 3. As duas operações rejeitam variância zero porque o denominador seria indefinido.

Matrizes de associação

`matriz_covariancia(dados, populacional=False)` interpreta cada linha como uma observação e cada coluna como uma variável. O padrão é amostral; `populacional=True` troca o divisor para `n`.

`matriz_correlacao(dados)` devolve a matriz de correlação de Pearson para as colunas observadas conjuntamente. Colunas constantes são rejeitadas explicitamente. As rotinas usam centralização escalada para reduzir transbordamentos intermediários evitáveis.

Regressão linear

`regressao_linear(x, y)` ajusta `y = intercepto + inclinacao * x` por mínimos quadrados simples. O retorno é `ResultadoRegressaoLinear` e inclui:

- `inclinacao`;
- `intercepto`;
- `residuos`;
- `r_quadrado`;
- `quantidade`.

As séries precisam ter o mesmo comprimento, conter ao menos duas observações e usar somente valores finitos. Uma série `x` constante não identifica uma inclinação e produz

`ErroNumerico`. Uma série `y` constante é válida e resulta em inclinação zero e R² igual a 1 para o ajuste constante exato.

Bootstrap

`intervalo_bootstrap(valores, estatistica, confianca, repeticoes, semente)` usa reamostragem com reposição e devolve `ResultadoBootstrap`.

O método disponível nesta edição é o intervalo percentil. A função estatística é fornecida pelo programa, deve ser chamável e precisa produzir um número real finito. A semente é inteira e alimenta uma instância local de `random.Random`, portanto o gerador aleatório global do programa não é modificado.

O resultado registra estimativa observada, limites inferior e superior, confiança, repetições, semente, tamanho da amostra, custo estimado e o identificador `bootstrap_percentil`.

Teste de permutação

`teste_permutacao(a, b, estatistica, repeticoes, semente)` estima bilateralmente a probabilidade de obter uma diferença de magnitude pelo menos igual à observada quando os rótulos dos dois grupos são permutáveis sob a hipótese nula.

O valor p usa a correção `(extremos + 1) / (repeticoes + 1)`, evitando zero artificial em uma simulação finita. O resultado registra diferença observada, valor p, tamanhos dos grupos, repetições, semente, custo estimado e o identificador `permutacao_bilateral`.

Limites e interpretação

Bootstrap e permutação possuem limite interno de 5 000 000 de observações processadas por chamada para impedir custos acidentais muito grandes. O limite é validado antes de iniciar a repetição.

Intervalos e valores p são resultados de métodos declarados, não conclusões automáticas. A Coral não afirma causalidade, não classifica um efeito como importante e não transforma um limiar arbitrário em verdade científica. A interpretação continua sob responsabilidade do programa e do contexto da análise.

Generalidade

As rotinas aceitam séries e matrizes comuns e não dependem de `coral.laboratorio`. Resultados de experimentos podem ser convertidos em séries pelo programa, mas as fórmulas não assumem uma origem específica para os dados.

Projeto curto: Resumo de medições

Calcule um resumo dos valores 20, 22, 24 e 26 e identifique o que acontece com dados ausentes.

A orientação de solução está no Apêndice B, seção Projeto 38.

O que você aprendeu

Confira formato, finitude e quantidade dos dados. Uma medida estatística descreve um cálculo; sua interpretação depende do método e do contexto.

Prática

Explique por que correlação com coluna constante é inválida.

Conferir a resposta comentada

O cálculo divide pela dispersão de cada variável. Se uma coluna não varia, essa dispersão é zero e a correlação não pode ser definida pelo contrato; a biblioteca produz diagnóstico.

Resposta do capítulo 38 no Apêndice B.

39. Gráficos

Objetivo de leitura

Criar uma especificação de gráfico e escolher um formato de apresentação.

Antes de começar

Conheça coleções numéricas e arquivos.

`coral.graficos` recebe coleções comuns e produz especificações imutáveis. A criação da especificação é separada da renderização e não abre janela. Linhas e dispersão exigem séries alinhadas e finitas; barras mantêm zero na escala vertical por padrão. Registros comuns podem ser adaptados por campos escolhidos com `series_de_registros`.

Histogramas e caixas não mantêm uma segunda implementação estatística: `grafico_histograma` usa o contrato de `coral.numerico.histograma`, e `grafico_caixa` usa os quartis de Numérico. Isso mantém contagens, bordas e quartis consistentes entre cálculo e apresentação.

`grafico_funcao` limita a quantidade de amostras e representa avaliações inválidas como lacunas para não ligar uma falha por uma linha enganosa. `configurar` retorna cópia com título, rótulos, unidades, legenda, tema, proveniência e texto alternativo. `para_svg` e `salvar` produzem SVG 2D autossuficiente usando somente a biblioteca padrão.

Código Coral

```
de coral.graficos importe linha, configurar, salvar

defina figura como linha([0, 1, 2], [20, 21, 23])
defina figura como configurar(figura, titulo="Temperatura", eixos=["Tempo",
  "Temperatura"], unidades=["min", "°C"])
execute salvar(figura, "temperatura.svg")
```

Este exemplo não imprime texto no terminal.

Na 1.6.11, `mapa_calor` representa matrizes retangulares e `mapa_correlacao` reutiliza o cálculo de `coral.numerico.matriz_correlacao`. `barras_erro` e `faixa_incerteza`

representam incerteza sem chamar toda faixa de intervalo de confiança. `grafico_experimento` importa Laboratório apenas durante a chamada explícita e converte medições em séries comuns preservando proveniência. `painel` combina múltiplos gráficos sem alterar as especificações filhas. SVG 2D continua disponível somente com a biblioteca padrão; PNG e PDF usam Matplotlib opcional carregado sob demanda.

Na 1.6.12, `dispersao3d` e `trajetoria3d` criam séries tridimensionais sem alterar a ordem ou as coordenadas de entrada. `superficie3d` representa uma grade retangular explícita e não interpola dados ausentes. A especificação 3D continua independente do backend. `para_svg` recusa reduzir 3D a uma projeção implícita; `salvar` usa Matplotlib sob demanda para SVG, PNG e PDF tridimensionais. A superfície espacial dedicada da linha 1.6.x termina em 3D; não há uma API pública planejada para quatro dimensões espaciais.

Gráficos interativos locais

Na 1.6.13, `para_html` gera HTML interativo autossuficiente e `salvar_html` grava o resultado localmente. Plotly é carregado somente nessas operações ou em `explorar`; importar `coral.graficos` continua sem carregar o backend. O arquivo final não usa CDN e contém os dados e o JavaScript necessários à interação.

Código Coral

```
de coral.graficos importe dispersao, configurar, salvar_html

defina figura como dispersao([1, 2, 3], [2, 4, 6])
defina figura como configurar(figura, titulo="Relação entre medidas")
execute salvar_html(figura, "medidas_interativas.html")
```

Este exemplo não imprime texto no terminal.

`explorar` abre a visualização somente quando solicitado. A ausência de sessão gráfica produz diagnóstico que orienta `salvar_html`. `amostrar_grafico` é a única forma pública de redução automática planejada nesta edição e exige decisão explícita; a exportação nunca elimina pontos silenciosamente.

Painéis mistos podem conter gráficos 2D e 3D. Zoom, inspeção e rotação da câmera são recursos de apresentação e não alteram coordenadas, malhas ou resultados estatísticos. Texto alternativo, avisos e proveniência permanecem disponíveis na saída, e o HTML informa que os dados ficam incorporados no arquivo.

Gráficos 2D

`coral.graficos` é a primeira camada pública de visualização científica da Coral. O módulo separa três responsabilidades: Numérico calcula, Laboratório produz medições e Gráficos apresenta dados. Criar uma especificação não abre janela e não acessa rede.

Séries diretas

`linha(x, y)` e `dispersao(x, y)` recebem duas coleções numéricas de mesmo comprimento. Valores ausentes, `NaN` e infinito são recusados explicitamente.

`barras(categorias, valores)` apresenta categorias e mantém zero na escala vertical por padrão, evitando truncamento enganoso.

Estatística visual

`grafico_histograma(valores, faixas)` delega o cálculo a `coral.numerico.histograma`, portanto contagens e bordas são exatamente as mesmas do módulo Numérico.

`grafico_caixa(grupos)` usa os quartis de `coral.numerico` e bigodes de Tukey de 1,5 × IQR. Pontos extremos ficam preservados e grupos com menos de cinco observações recebem aviso.

Registros comuns

`series_de_registros(registros, campo_x, campo_y)` aceita mapeamentos e objetos com atributos. O adaptador retorna duas séries alinhadas sem importar `coral.laboratorio`. Campo inexistente ou valor ausente é erro, nunca descarte silencioso.

Funções e descontinuidades

`grafico_funcao(funcao, inicio, fim, amostras)` amostra uma função escalar em intervalo fechado. Avaliações que falham ou produzem valores não finitos viram lacunas e não são ligadas por uma linha. O limite é 100000 amostras por chamada.

Configuração

`configurar` devolve uma nova especificação e preserva os dados originais. É possível definir título, rótulos de eixos, unidades, legenda, tema, proveniência e texto alternativo. Os temas iniciais são `claro`, `escuro` e `alto_contraste`.

SVG

`para_svg` produz SVG autossuficiente em memória. `salvar` grava SVG 2D pela biblioteca padrão; PNG e PDF usam Matplotlib opcional. A renderização usa a biblioteca padrão e não carrega Matplotlib, Plotly, Pygame ou Tkinter.

Compor gráficos e representar incerteza

A superfície gráfica continua geral. Ela recebe listas, matrizes e séries comuns sem exigir `coral.laboratorio`. O adaptador de experimentos é acionado apenas quando `grafico_experimento` é chamado.

`mapa_calor(matriz)` representa valores já calculados em uma matriz retangular. `mapa_correlacao(dados)` delega o cálculo de Pearson a

`coral.numerico.matriz_correlacao` e apenas apresenta o resultado. Assim, a definição estatística continua pertencendo ao módulo numérico.

`barras_erro(x, y, erros)` aceita incertezas simétricas ou pares inferior e superior. `faixa_incerteza(x, centro, inferior, superior)` recebe limites absolutos. A faixa não é chamada automaticamente de intervalo de confiança porque seu significado vem do método que produziu os limites.

`grafico_experimento(resultado, eixo_x, metrica)` converte medições bem sucedidas de um `ResultadoExperimento` para a mesma `Serie2D` usada pelos gráficos gerais. Nome do experimento, sementes e protocolo de medição ficam registrados na proveniência da especificação.

`painel(graficos, colunas)` compõe várias especificações sem alterar seus dados ou escalas. `para_svg` continua funcionando apenas com a biblioteca padrão e também aceita painéis.

`salvar(grafico, caminho)` reconhece `.svg`, `.png` e `.pdf`. SVG continua sendo o backend básico e autossuficiente. PNG e PDF carregam Matplotlib somente quando solicitados. Se o backend opcional não estiver instalado, a Coral devolve erro orientando a instalação em vez de falhar durante `import coral.graficos`.

A importação de `coral.graficos` não carrega Matplotlib, Plotly, NumPy nem Laboratório antecipadamente. HTML interativo usa Plotly opcional, descrito adiante.

Gráficos 3D

`coral.graficos` passa a aceitar especificações tridimensionais sem transformar visualização em cálculo geométrico. `coral.numerico` continua responsável por vetores, transformações, planos, coordenadas esféricas, áreas e volumes. Gráficos recebe coordenadas já definidas e as apresenta.

Pontos e trajetórias

`dispersao3d(x, y, z, nome=None)` cria uma coleção de pontos tridimensionais. `trajetoria3d(x, y, z, nome=None)` preserva a mesma sequência de coordenadas e apenas indica que os pontos devem ser ligados na ordem fornecida.

Os três eixos precisam ter o mesmo comprimento. Valores ausentes, `NaN` e infinito são rejeitados. As coleções de entrada não são modificadas nem reordenadas.

Superfícies

`superficie3d(x, y, z, nome=None)` representa uma grade retangular explícita. Há duas formas aceitas.

Quando `x` e `y` são eixos unidimensionais, `z` deve possuir `len(y)` linhas e `len(x)` colunas. Quando `x` e `y` são grades bidimensionais, as três grades precisam ter exatamente a mesma forma.

A função não interpola pontos ausentes, não corrige grades irregulares e não inventa valores entre amostras. Uma superfície descreve somente os pontos presentes na grade fornecida.

Configuração dos eixos

`configurar` aceita dois ou três rótulos e unidades. Em gráficos 3D, a forma `eixos=("x", "y", "z")` e `unidades=("m", "m", "m")` também configura o terceiro eixo. Mapeamentos podem usar as chaves `x`, `y` e `z`.

Renderização estática

Criar `dispersao3d`, `trajetoria3d` ou `superficie3d` não importa Matplotlib, não abre janela e não escolhe uma projeção escondida. `para_svg` continua sendo o renderizador próprio da Coral para especificações 2D e, por isso, rejeita gráficos 3D em vez de achatá los silenciosamente.

`salvar` usa Matplotlib somente quando uma renderização 3D é solicitada. Gráficos 3D podem ser salvos em SVG, PNG ou PDF quando o backend opcional está instalado. A importação de `coral.graficos` permanece independente de Matplotlib.

Fronteira de responsabilidade

A câmera e a projeção visual usadas pelo backend servem apenas à apresentação. As coordenadas originais guardadas em `Serie3D` e `Superficie3D` permanecem inalteradas. Operações geométricas como rotação, transformação afim, projeção sobre plano e interseção reta plano pertencem a `coral.numerico`.

A linha 1.6.x não possui uma superfície pública dedicada a espaço 4D. A decisão da 1.6.12 concentra a evolução espacial da Coral em 3D.

Consultar os gráficos interativos

`coral.graficos` pode transformar as especificações científicas já existentes em HTML interativo autossuficiente. Plotly continua opcional e só é carregado quando a capacidade interativa é solicitada.

HTML local e offline

`para_html(grafico)` devolve um documento HTML completo em memória. `salvar_html(grafico, caminho)` grava o mesmo documento de forma atômica. O JavaScript necessário é incorporado no próprio arquivo, portanto a visualização não depende de CDN nem de conexão com a internet.

O HTML preserva os valores já presentes na especificação. O backend não recalcula estatística, não interpola superfícies, não suaviza trajetórias e não reduz coleções silenciosamente.

Exploração explícita

`explorar(grafico)` cria uma prévia local e pede ao navegador do sistema que a abra. A operação só acontece quando chamada explicitamente. Criar um gráfico ou importar `coral.graficos` não abre janela, navegador, servidor ou conexão.

Em sessão sem interface gráfica, `explorar` produz um diagnóstico orientando o uso de `salvar_html`. Nenhuma porta é aberta como alternativa implícita.

Painéis 2D e 3D

Painéis podem combinar gráficos 2D e 3D na mesma saída interativa. Cada subgráfico mantém título, eixos, unidades e tipo de cena. Séries de `grafico_experimento` continuam séries comuns depois da adaptação e preservam avisos e proveniência no texto acessível do documento.

Para 3D, dispersões, trajetórias e superfícies preservam as coordenadas originais. Rotação, zoom e posição da câmera pertencem apenas à apresentação.

Limites e amostragem

A exportação HTML possui um limite de observações ou células para evitar materialização acidental de documentos desproporcionais. Ao exceder o limite, a operação falha antes da renderização e explica as alternativas disponíveis.

`amostrar_grafico(grafico, max_pontos)` oferece uma redução uniforme explícita apenas para linhas, dispersões e séries 3D simples. A especificação devolvida registra a quantidade original, a quantidade mantida e um aviso de amostragem. A função nunca é chamada automaticamente por `para_html`.

Mapas, superfícies, caixas, barras e faixas não são reduzidos automaticamente porque uma seleção genérica poderia alterar sua semântica. Nesses casos, o programa deve preparar uma representação menor antes de construir o gráfico.

Privacidade e acessibilidade

O HTML autossuficiente contém os dados do gráfico incorporados. A página inclui um aviso visível lembrando que dados sensíveis devem ser revisados antes de compartilhar o arquivo.

A visualização recebe papel de imagem e descrição acessível. `texto_alternativo`, avisos e proveniência participam dessa descrição. Em painéis, as descrições dos filhos também são preservadas.

Integração com servidor local

`coral.web.servidor.resposta_grafico(grafico)` converte explicitamente um gráfico em `RespostaServidorHTTP`. A função não cria nem inicia servidor. Ela apenas prepara a resposta HTML e usa `Cache-Control: no-store` por padrão porque o documento contém os próprios dados.

Para publicar localmente, o programa cria um servidor, registra uma rota que devolve `resposta_grafico` e então chama `iniciar()` de forma explícita. Assim Gráficos não passa a depender de Web e Web não inicia rede durante a adaptação.

Dependência opcional

Plotly 6.0 ou superior pertence ao componente opcional `graficos_interativos`. A ausência do pacote não impede `import coral.graficos`, SVG, PNG, PDF ou as especificações 2D e 3D. O erro aparece apenas ao solicitar HTML interativo.

Projeto curto: Gráfico de temperatura

Crie uma linha com quatro medições e salve SVG com título e unidades. Preserve a lista de origem.

A orientação de solução está no Apêndice B, seção Projeto 39.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`linha(...)` · **módulo** `coral.graficos`

O que faz

Cria uma especificação imutável de gráfico de linhas.

Assinatura

Assinatura para consulta

```
linha(x: 'Iterable[Any]', y: 'Iterable[Any]', nome: 'str | None' = None) -> 'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	Iterable[Any]	sim	sem padrão	Coordenadas horizontais da série, ou posição horizontal do objeto.
y	Iterable[Any]	sim	sem padrão	Coordenadas verticais da série, ou posição vertical do objeto.
nome	str | None	não	None	Nome da série ou do objeto.

Retorno

Grafico 2D.

Exemplo

Código Coral

```
de coral.graficos importe linha
defina figura como linha([0, 1], [2, 3])
mostre figura.elementos[0].estilo
```

Resultado

Saída esperada

```
linha
```

Erros

ErroGrafico para dados, configuração, limites ou backend inválidos.

Atenção

Criar a especificação não abre janela. Exportar escreve no caminho informado. HTML incorpora dados e exige o backend opcional Plotly; outros formatos têm requisitos próprios.

`dispersao(...)` · módulo `coral.graficos`

O que faz

Cria uma especificação imutável de gráfico de pontos.

Assinatura

Assinatura para consulta

```
dispersao(x: 'Iterable[Any]', y: 'Iterable[Any]', nome: 'str | None' = None) ->
  'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	Iterable[Any]	sim	sem padrão	Coordenadas horizontais da série, ou posição horizontal do objeto.
y	Iterable[Any]	sim	sem padrão	Coordenadas verticais da série, ou posição vertical do objeto.
nome	str | None	não	None	Nome da série ou do objeto.

Retorno

Grafico 2D.

Exemplo

Código Coral

```
de coral.graficos importe dispersao
defina figura como dispersao([0, 1], [2, 3])
mostre figura.elementos[0].estilo
```

Resultado

Saída esperada

```
dispersao
```

Erros

ErroGrafico para dados, configuração, limites ou backend inválidos.

Atenção

Criar a especificação não abre janela. Exportar escreve no caminho informado. HTML incorpora dados e exige o backend opcional Plotly; outros formatos têm requisitos próprios.

`configurar(...)` · módulo `coral.graficos`

O que faz

Cria uma cópia do gráfico com as opções solicitadas.

Assinatura

Assinatura para consulta

```
configurar(grafico: 'Grafico', titulo: 'str | None' = None, eixos: 'Any' = None,
  unidades: 'Any' = None, legenda: 'bool | None' = None, tema: 'str | None' = None, *,
  proveniencia: 'Iterable[Any] | None' = None, texto_alternativo: 'str | None' = None) ->
  'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grafico	Grafico	sim	sem padrão	Especificação de gráfico a configurar ou salvar.
titulo	str | None	não	None	Valor titulo usado na operação.
eixos	Any	não	None	Rótulos dos eixos; a quantidade precisa corresponder à dimensionalidade.
unidades	Any	não	None	Unidades dos eixos; acompanham a dimensionalidade do gráfico.
legenda	bool | None	não	None	Se verdadeiro, inclui a legenda.
tema	str | None	não	None	Identificador do tema visual suportado.
proveniencia	Iterable[Any] | None	não	None	Metadados que explicam de onde vêm os dados.
texto_alternativo	str | None	não	None	Descrição textual para acessibilidade.

Retorno

Nova especificação Grafico.

Exemplo

Código Coral

```
de coral.graficos importe configurar
de coral.graficos importe linha
defina figura como configurar(linha([0, 1], [2, 3]), titulo="Medidas")
mostre figura.titulo
```

Resultado

Saída esperada

```
Medidas
```

Erros

ErroGrafico para dados, configuração, limites ou backend inválidos.

Atenção

Criar a especificação não abre janela. Exportar escreve no caminho informado. HTML incorpora dados e exige o backend opcional Plotly; outros formatos têm requisitos próprios.

`salvar(...)` · módulo `coral.graficos`

O que faz

Renderiza um gráfico num arquivo escolhido pelo formato de saída.

Assinatura

Assinatura para consulta

```
salvar(grafico: 'Grafico | PainelGraficos', caminho: 'str | Path') -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grafico	Grafico | PainelGraficos	sim	sem padrão	Especificação de gráfico a configurar ou salvar.
caminho	str | Path	sim	sem padrão	Caminho local do arquivo de saída ou da imagem de origem.

Retorno

Path do arquivo salvo.

Exemplo

Código Coral

```
de coral.graficos importe salvar
de coral.graficos importe linha
execute salvar(linha([0, 1], [2, 3]), "figura.svg")
de coral.arquivos importe existe
mostre existe("figura.svg")
```

Resultado

Saída esperada

```
True
```

Erros

ErroGrafico para dados, configuração, limites ou backend inválidos.

Atenção

Criar a especificação não abre janela. Exportar escreve no caminho informado. HTML incorpora dados e exige o backend opcional Plotly; outros formatos têm requisitos próprios.

`salvar_html(...)` · **módulo** `coral.graficos`

O que faz

Exporta um gráfico interativo autossuficiente, sem CDN.

Assinatura

Assinatura para consulta

```
salvar_html(grafico: 'Any', caminho: 'str | Path', *, limite_pontos: 'int' = 200000) ->
  'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grafico	Any	sim	sem padrão	Especificação de gráfico a configurar ou salvar.
caminho	str | Path	sim	sem padrão	Caminho local do arquivo de saída ou da imagem de origem.
limite_pontos	int	não	200000	Teto explícito de observações aceitas na exportação HTML.

Retorno

Path do HTML salvo.

Exemplo

Código Coral

```
de coral.graficos importe salvar_html
de coral.graficos importe linha
execute salvar_html(linha([0, 1], [2, 3]), "figura.html")
de coral.arquivos importe existe
mostre existe("figura.html")
```

Resultado

Saída esperada

```
True
```

Erros

ErroGrafico para dados, configuração, limites ou backend inválidos.

Atenção

Criar a especificação não abre janela. Exportar escreve no caminho informado. HTML incorpora dados e exige o backend opcional Plotly; outros formatos têm requisitos próprios.

O que você aprendeu

Criar um gráfico não abre uma janela. SVG 2D usa a biblioteca padrão; outras saídas podem exigir backends opcionais. Unidades e proveniência ajudam a interpretar os dados.

Prática

Separe criar uma especificação de gráfico e salvar sua imagem.

Conferir a resposta comentada

Criar `linha` ou `dispersao` descreve os dados sem abrir janela. `salvar` escolhe o formato e renderiza. A disponibilidade do backend depende do formato solicitado.

Resposta do capítulo 39 no Apêndice B.

40. Web

Objetivo de leitura

Distinguir HTML local, cliente HTTP e servidor HTTP.

Antes de começar

Conheça textos, funções e tratamento de erros.

A base Web separa geração local de HTML e URLs, cliente HTTP e servidor HTTP. Escolha apenas a capacidade que sua aplicação precisa.

Montar HTML e manipular URLs

`coral.web.html` e `coral.web.url` formam a base Web local da Coral. Nesta versão as duas superfícies trabalham somente em memória: importar os módulos não abre conexão, porta, navegador ou janela. As requisições HTTP usam o módulo próprio, apresentado mais adiante.

HTML seguro por padrão

`escapar_texto(valor)` codifica texto para o conteúdo de elementos. `escapar_atributo(valor)` codifica texto para atributos entre aspas. Escape não transforma uma URL perigosa em URL segura, por isso atributos de URL são validados separadamente.

`elemento(nome, atributos, filhos)` cria um `ElementoHTML` imutável. Nomes de elemento e atributo são validados, atributos de evento inline como `onclick` são recusados e esquemas executáveis como `javascript:` não são aceitos em `href`, `src`, `action` e atributos equivalentes. Texto comum em `filhos` nunca é interpretado como marcação confiável.

`fragmento(filhos)` agrupa nós sem inserir um contêiner extra. `documento(titulo, corpo, idioma)` cria um `DocumentoHTML` completo com UTF 8 e idioma declarado. `renderizar(no)` produz a serialização em memória e não escreve arquivo.

Tabelas de registros gerais

`tabela(registros, colunas)` aceita mapeamentos ou objetos com atributos. As colunas são explícitas e mantêm a ordem fornecida. Um campo ausente produz erro em vez de desaparecer silenciosamente, e o valor de cada célula continua sujeito ao escape de texto. Isso permite apresentar dados comuns e, por conversão explícita, dados produzidos por outros módulos sem importar `coral.laboratorio`.

Consultas e URLs

`codificar_consulta(parametros)` aceita mapeamento ou sequência ordenada de pares. Chaves repetidas são preservadas e o texto usa UTF 8. `ler_consulta(texto)` devolve os pares na mesma ordem, preservando valores vazios e duplicatas.

`decompor(endereco)` exige esquema explícito e devolve `EnderecoURL`, com esquema, autoridade, host, porta, caminho, consulta e fragmento. A propriedade `parametros` decodifica a consulta sem perder chaves repetidas. `juntar(base, referencia)` resolve uma referência relativa contra uma base usando apenas regras textuais. Nenhuma dessas operações resolve DNS ou acessa a rede.

Exemplo

O exemplo `Exemplos/Citados/Web/01_html_e_url.coral` monta um documento com tabela, escapa conteúdo potencialmente ambíguo e prepara uma URL com consulta repetida. Ele pode ser compilado e executado sem conexão de rede.

Escolher o módulo adequado

HTML e URL trabalham em memória. Para acessar um endereço, use `coral.web.http`; para atender requisições locais, use `coral.web.servidor`. Montar uma página não a hospeda automaticamente.

Fazer uma requisição HTTP

`coral.web.http` é o primeiro cliente HTTP geral da Coral. Importar o módulo não abre conexão, não inicia servidor e não consulta recursos externos. A rede só é usada quando `requisitar`, `obter` ou `enviar` é chamada. O módulo não depende de `coral.laboratorio`.

Resposta preservada em bytes

`RespostaHTTP` preserva `status`, `cabecalhos`, `corpo_bytes`, `url`, `metodo` e a quantidade de `redirecionamentos`. A propriedade `sucesso` apenas indica status entre 200 e 299. Respostas 3xx, 4xx e 5xx continuam inspecionáveis e não são transformadas automaticamente em erro de transporte.

`texto(codificacao)` decodifica o corpo sob demanda. O corpo recebido nunca é convertido silenciosamente em texto. `json()` interpreta JSON sob demanda e usa `ErroFormato` para conteúdo inválido, separando erro de formato de falha de transporte.

Requisições

`requisitar(metodo, url, cabecalhos, corpo, tempo_limite)` é a operação geral. `obter(url, ...)` é o atalho para GET e `enviar(url, corpo, ...)` é o atalho para POST. Texto enviado exige `codificacao_corpo` explícita; bytes são enviados sem conversão.

O cliente aceita somente URLs `http` e `https`, rejeita credenciais embutidas e valida nomes e valores de cabeçalhos antes de montar a requisição. A execução usa a biblioteca padrão e não requer pacote externo.

Timeout e limite de resposta

Toda requisição tem `tempo_limite` finito. `limite_resposta` limita o corpo materializado em memória; se `Content-Length` já declara tamanho acima do teto ou se a leitura excede o limite, a operação falha com `ErroLimiteHTTP` em vez de truncar silenciosamente.

Esses limites controlam o cliente, não substituem políticas da aplicação. O programa ainda decide quais hosts pode acessar, que dados envia e que respostas considera confiáveis.

Redirecionamentos

GET, HEAD e OPTIONS seguem redirecionamentos por padrão, com `max_redirecionamentos` finito. Métodos com potencial de efeito, como POST, não repetem a requisição após redirecionamento sem `seguir_redirecionamentos=true` explícito. Apenas destinos `http` e `https` podem ser seguidos.

Formulários

`codificar_formulario(campos)` produz `application/x-www-form-urlencoded` em bytes ASCII com valores codificados em UTF 8. Sequências de pares preservam ordem e chaves repetidas. Multipart, upload de arquivos, cookies persistentes e autenticação automática não fazem parte desta base.

TLS e proxy

HTTPS usa `ssl.create_default_context()` quando nenhum `contexto_tls` é fornecido. Isso mantém verificação de certificado e de hostname habilitadas por padrão. Um `SSLContext` explícito pode ser passado por aplicações que realmente precisam controlar a política TLS.

`proxy` é explícito. Quando não é fornecido, o cliente não depende de uma camada Coral de proxy. Quando configurado, aceita uma URL textual para HTTP e HTTPS ou um mapeamento por esquema.

Exemplo local

`Exemplos/Citados/Web/02_cliente_http_loopback.coral` recebe a URL de um servidor local efêmero como argumento do programa, faz GET de JSON e POST de formulário e inspeciona status e corpo. O gate da release executa esse fluxo somente em loopback, sem serviço externo.

Limites do cliente

O cliente não oferece upload multipart, cookies persistentes, cache ou autenticação automática. O servidor é um módulo separado. HTML e URL continuam utilizáveis sem conexão.

Servidor HTTP local da Coral

`coral.web.servidor` oferece um servidor pequeno para desenvolvimento, demonstrações e testes locais. Importar o módulo não abre porta. `criar_servidor()` apenas cria a configuração; a escuta começa com `iniciar()` ou ao usar o objeto em um bloco de contexto.

Por padrão, o servidor usa `127.0.0.1` e porta `0`. A porta zero pede ao sistema uma porta efêmera livre. Exposição fora do loopback exige `permitir_externo=True`, portanto não acontece por acidente.

Rotas literais

`adicionar_rota` é o método `ServidorHTTP.adicionar_rota(metodo, caminho, tratador)` e registra uma combinação literal como `GET /estado`. A 1.6.9 não interpreta parâmetros de rota, curingas ou expressões. O tratador recebe `RequisicaoHTTP` e deve retornar `RespostaServidorHTTP`.

A requisição preserva `metodo`, `caminho`, `consulta`, `cabecalhos`, `corpo_bytes` e `cliente`. `texto()`, `json()` e `formulario()` interpretam o corpo somente quando o programa pede. Consultas e formulários preservam ordem e chaves repetidas.

Respostas

Use `resposta_bytes`, `resposta_texto`, `resposta_html` ou `resposta_json`. HTML reutiliza a serialização segura de `coral.web.html`; texto comum não vira marcação por acidente. JSON recusa valores que exigiriam conversão silenciosa e não aceita NaN como JSON válido.

Limite de corpo

`limite_corpo` é aplicado a `Content-Length` antes de o tratador ser chamado. Corpos acima do limite recebem status 413. `tempo_limite_corpo` impede que uma leitura parcial mantenha a conexão indefinidamente. O servidor básico não implementa corpo de requisição com `Transfer-Encoding`.

Arquivos sob raiz declarada

Ao criar o servidor, `raiz_arquivos` pode apontar para uma pasta explícita. `adicionar_arquivo` associa um arquivo relativo a uma rota literal. Caminhos absolutos, `..` que saiam da raiz e links simbólicos que saiam da raiz declarada são recusados. `limite_arquivo` impede leitura acidental de arquivos acima do teto escolhido.

Encerramento

`encerrar()` faz `shutdown`, fecha o socket e aguarda a thread de atendimento. A chamada é idempotente. Usar `com servidor:` também encerra o serviço ao sair do bloco.

O servidor é uma ferramenta de desenvolvimento local. Ele não é um servidor de produção, não oferece TLS de servidor, autenticação, balanceamento, HTTP/2 ou gestão de processos. O cliente HTTP da Coral permanece em `coral.web.http` e continua independente do servidor.

Resposta de gráfico interativo

Na 1.6.13, `resposta_grafico(grafico)` adapta uma especificação de `coral.graficos` para uma `RespostaServidorHTTP` com HTML interativo autossuficiente. A adaptação é explícita e não inicia servidor, não abre porta e não acessa recursos remotos.

A resposta usa `Cache-Control: no-store` por padrão porque o HTML contém os dados da figura. O programa continua responsável por criar o servidor, registrar a rota e chamar `iniciar()` quando realmente quiser publicar a página no loopback.

Exemplo guiado sem serviço externo

Este programa inicia um servidor apenas no loopback, consulta sua própria rota e encerra o servidor em finalmente. A resposta esperada é 200 e Coral local em linhas separadas.

Código Coral

```
# Exemplo de HTTP no loopback
de coral.web.servidor importe criar_servidor, resposta_texto
de coral.web.http importe obter

crie a função pagina com requisicao
    retorne resposta_texto("Coral local")
fim

defina servidor como criar_servidor()
execute servidor.adicionar_rota("GET", "/", pagina)
execute servidor.iniciar()
tente
    defina resposta como obter(servidor.url_base + "/", tempo_limite=2,
      limite_resposta=4096)
    mostre resposta.status
    mostre resposta.texto()
finalmente
    execute servidor.encerrar()
fim
```

Saída esperada

```
200
Coral local
```

Projeto curto: Página de estado local

Adapte o exemplo de loopback para devolver um texto de estado diferente e confirme o encerramento.

A orientação de solução está no Apêndice B, seção Projeto 40.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`criar_servidor(...)` · **módulo** `coral.web.servidor`

O que faz

Cria a configuração de um servidor HTTP pequeno para desenvolvimento local. Não começa a escuta até iniciar.

Assinatura

Assinatura para consulta

```
criar_servidor(host: 'str' = '127.0.0.1', porta: 'int' = 0, **opcoes: 'Any') ->
  'ServidorHTTP'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
host	str	não	'127.0.0.1'	Endereço de escuta; o padrão limita o servidor ao loopback.
porta	int	não	0	Porta de escuta; zero pede uma porta efêmera livre.
opcoes	Any	não	sem padrão	Opções nomeadas aceitas pelo contrato chamado, conforme a referência de ServidorHTTP ou requisitar nesta parte.

Retorno

ServidorHTTP.

Exemplo

Código Coral

```
# Exemplo de HTTP no loopback
de coral.web.servidor importe criar_servidor, resposta_texto
de coral.web.http importe obter

crie a função pagina com requisicao
    retorne resposta_texto("Coral local")
fim

defina servidor como criar_servidor()
execute servidor.adicionar_rota("GET", "/", pagina)
execute servidor.iniciar()
tente
    defina resposta como obter(servidor.url_base + "/", tempo_limite=2,
      limite_resposta=4096)
    mostre resposta.status
    mostre resposta.texto()
finalmente
    execute servidor.encerrar()
fim
```

Resultado

Saída esperada

```
200
Coral local
```

Erros

ErroHTTP para falha de transporte ou limites no cliente. O servidor de desenvolvimento não substitui um servidor de produção.

Atenção

Este exemplo completo usa somente o loopback e encerra o servidor no bloco finalmente. HTTP local não comprova TLS ou segurança de um serviço público. Consulte o Volume II, Parte II, seções coral.web.http e coral.web.servidor.

`resposta_texto(...)` · **módulo** `coral.web.servidor`

O que faz

Cria uma resposta HTTP cujo corpo é texto, com tipo de conteúdo e codificação definidos.

Assinatura

Assinatura para consulta

```
resposta_texto(texto: 'Any', status: 'int' = 200, cabecalhos: 'Any' = None, *,
  codificacao: 'str' = 'utf-8') -> 'RespostaServidorHTTP'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
texto	Any	sim	sem padrão	Texto a enviar no corpo da resposta.
status	int	não	200	Código HTTP da resposta.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
cabecalhos	Any	não	None	Cabeçalhos adicionais da resposta.
codificacao	str	não	'utf-8'	Codificação de texto usada na resposta.

Retorno

RespostaServidorHTTP.

Exemplo

Código Coral

```
# Exemplo de HTTP no loopback
de coral.web.servidor importe criar_servidor, resposta_texto
de coral.web.http importe obter

crie a função pagina com requisicao
    retorne resposta_texto("Coral local")
fim

defina servidor como criar_servidor()
execute servidor.adicionar_rota("GET", "/", pagina)
execute servidor.iniciar()
tente
    defina resposta como obter(servidor.url_base + "/", tempo_limite=2,
      limite_resposta=4096)
    mostre resposta.status
    mostre resposta.texto()
finalmente
    execute servidor.encerrar()
fim
```

Resultado

Saída esperada

```
200
Coral local
```

Erros

ErroHTTP para falha de transporte ou limites no cliente. O servidor de desenvolvimento não substitui um servidor de produção.

Atenção

Este exemplo completo usa somente o loopback e encerra o servidor no bloco finalmente. HTTP local não comprova TLS ou segurança de um serviço público. Consulte o Volume II, Parte II, seções coral.web.http e coral.web.servidor.

`obter(...)` · módulo `coral.web.http`

O que faz

Faz um pedido HTTP GET com limites explícitos.

Assinatura

Assinatura para consulta

```
obter(url: 'str', cabecalhos: 'Any' = None, tempo_limite: 'float' = 10.0, **opcoes:
  'Any') -> 'RespostaHTTP'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
url	str	sim	sem padrão	URL do recurso HTTP.
cabecalhos	Any	não	None	Cabeçalhos adicionais da resposta.
tempo_limite	float	não	10.0	Tempo limite finito da operação, em segundos.
opcoes	Any	não	sem padrão	Opções nomeadas aceitas pelo contrato chamado, conforme a referência de ServidorHTTP ou requisitar nesta parte.

Retorno

RespostaHTTP, com status, cabeçalhos e corpo em bytes; a interpretação do conteúdo é solicitada por métodos da resposta.

Exemplo

Código Coral

```
# Exemplo de HTTP no loopback
de coral.web.servidor importe criar_servidor, resposta_texto
de coral.web.http importe obter

crie a função pagina com requisicao
    retorne resposta_texto("Coral local")
fim

defina servidor como criar_servidor()
execute servidor.adicionar_rota("GET", "/", pagina)
execute servidor.iniciar()
tente
    defina resposta como obter(servidor.url_base + "/", tempo_limite=2,
      limite_resposta=4096)
    mostre resposta.status
    mostre resposta.texto()
finalmente
    execute servidor.encerrar()
fim
```

Resultado

Saída esperada

```
200
Coral local
```

Erros

ErroHTTP para falha de transporte ou limites no cliente. O servidor de desenvolvimento não substitui um servidor de produção.

Atenção

Este exemplo completo usa somente o loopback e encerra o servidor no bloco finalmente. HTTP local não comprova TLS ou segurança de um serviço público. Consulte o Volume II, Parte II, seções coral.web.http e coral.web.servidor.

O que você aprendeu

Montar HTML não acessa a rede. O cliente faz requisições; o servidor atende rotas. Conexões e início da escuta são ações explícitas.

Prática

Diferencie HTML local, cliente HTTP e servidor HTTP.

Conferir a resposta comentada

HTML local é conteúdo de apresentação; o cliente faz pedidos; o servidor responde a pedidos. Gerar HTML não abre conexão nem inicia um servidor.

Resposta do capítulo 40 no Apêndice B.

41. Sistema e hardware

Objetivo de leitura

Consultar capacidades do sistema e reconhecer limites de testes simulados.

Antes de começar

Conheça funções e tratamento de erros.

Um script pode consultar o ambiente em que está sendo executado e, quando necessário, iniciar outro programa. São ações diferentes: descobrir se um executável existe não significa executá lo.

Comece pelas operações de consulta nas fichas. Depois estude captura de saída, prazo de execução e hardware. Um teste simulado verifica a lógica, mas o comportamento de uma placa física precisa ser observado no equipamento.

Inspeção e execução

`coral.sistema` separa inspeção de capacidade de execução. `localizar_programa` e `programa_disponivel` consultam executáveis sem iniciá los; `quantidade_processadores`, `pasta_temporaria` e `terminal_interativo` descrevem capacidades do hospedeiro sem prometer recursos efetivos sob quotas. `ambiente_selecionado` copia apenas nomes explícitos e `variavel_ambiente_obrigatoria` falha sem incluir valores em diagnósticos. `argumentos_programa` expõe somente os argumentos do programa Coral, enquanto `caminho_executavel` identifica o executável que hospeda o runtime.

`executar_comando` continua sem shell implícito e agora aceita `codificacao`, política explícita de `erros_decodificacao` e `limite_saida_bytes`. O limite é aplicado enquanto stdout e stderr são drenados, e `saida_truncada` e `erro_truncado` registram descarte sem transformar truncamento em falha. Timeout tem prioridade sobre código de saída. `ResultadoComando` preserva seus campos históricos e acrescenta leituras por linhas, `codigo_saida`, `tempo_decorrido` e `exigir_sucesso`, que produz `ComandoFalhou` preservando o resultado completo.

Hardware e serial

Inventário de hardware degrada de forma previsível quando dependências opcionais não estão instaladas. A serial possui um backend simulado determinístico usado por contratos e testes. Comunicação com Arduino ou ESP32 físicos continua uma evidência de campo separada e não é declarada aprovada por simulação.

Coral Sistema e processos

`coral.sistema` reúne informações portáteis do processo Coral e operações explícitas sobre programas do sistema operacional. A biblioteca não substitui `coral.hardware`: ela descreve capacidades úteis para scripts, ferramentas, automação e aplicações, sem tentar inventariar profundamente CPU, memória ou dispositivos.

Consultar capacidades

`localizar_programa(nome)` resolve um executável usando o ambiente corrente e devolve seu caminho ou `nulo`. `programa_disponivel(nome)` usa a mesma resolução e apenas informa se o programa pode ser encontrado. Nenhuma das duas funções inicia o executável.

`quantidade_processadores()` devolve a quantidade lógica observada pelo runtime ou `nulo` quando o hospedeiro não informa esse valor. O número não deve ser interpretado como garantia de capacidade efetiva quando o processo estiver sob quotas, contêineres ou outras restrições.

`pasta_temporaria()` informa a pasta temporária configurada sem criar arquivos. `terminal_interativo(canal)` aceita `entrada`, `saida` ou `erro` e informa se o fluxo está associado a um terminal.

Ambiente

`ambiente_selecionado(nomes)` copia somente as variáveis explicitamente pedidas e omite as ausentes. O resultado é uma nova estrutura, portanto alterá lo não modifica o ambiente do processo.

`variavel_ambiente_obrigatoria(nome)` devolve a variável solicitada ou produz `ErroSistema` quando ela não existir. A mensagem de erro informa o nome, mas não inclui valor secreto.

Programa Coral e executável

`argumentos_programa()` devolve uma cópia dos argumentos fornecidos ao programa Coral. Ao usar o runtime pela linha de comando, separe esses argumentos com `--`:

Referência para leitura

```
python coral.pyz meu_programa.coral -- entrada.txt modo-seguro
```

Nesse caso o programa recebe `entrada.txt` e `modo-seguro`; opções internas do interpretador Coral não aparecem na lista.

`caminho_executavel()` informa o executável que hospeda o runtime, normalmente o Python usado para iniciar o PYZ. `processo_atual()` informa separadamente o executável, o programa ou PYZ corrente, os argumentos e a pasta atual.

ResultadoComando

Além dos campos históricos `codigo`, `saida`, `erro` e `duracao_segundos`, um resultado oferece:

- `linhas_saida()` e `linhas_erro()` para leitura por linhas sem alterar o texto original;
- `codigo_saida()` como leitura natural de `codigo`;
- `tempo_decorrido()` como leitura natural de `duracao_segundos`;
- `exigir_sucesso()` que devolve o próprio resultado quando o código é zero e produz `ComandoFalhou` caso contrário, preservando o resultado e o stderr.

Codificação e captura limitada

`executar_comando` continua executando sem shell. `codificacao` escolhe a codificação textual e `erros_decodificacao` aceita `estrito`, `substituir` ou `ignorar`. O padrão é estrito, preservando o comportamento histórico de falhar quando bytes inválidos não podem ser decodificados.

`limite_saida_bytes` limita separadamente a quantidade mantida em memória para stdout e stderr. O runtime aplica o teto enquanto lê os fluxos e continua drenando o excedente, evitando que um processo de saída contínua bloqueie apenas porque o conteúdo deixou de ser armazenado. `saida_truncada` e `erro_truncado` registram se houve descarte.

Timeout tem prioridade: quando o prazo termina, a Coral encerra o grupo de processos quando a plataforma oferece esse mecanismo e produz `TempoLimiteExcedido`. Truncamento não transforma código de saída zero em falha. Código não zero continua disponível em `ResultadoComando` e só vira exceção quando `exigir_sucesso()` é chamado.

Limites

A biblioteca não executa comandos por meio de shell implicitamente, não interpreta metacaracteres, não estima memória livre e não promete que a quantidade de processadores represente recursos realmente disponíveis sob quotas. Arquivos continuam sob responsabilidade de `coral.arquivos`.

Projeto curto: Relatório de capacidades

Consulte a quantidade de processadores e a disponibilidade de um programa sem iniciá lo.

A orientação de solução está no Apêndice B, seção Projeto 41.

O que você aprendeu

Consultar um executável não o inicia. O inventário observado não garante recursos efetivos, e uma simulação serial não comprova comunicação física.

Prática

Explique por que inventário simulado não comprova comunicação com uma placa física.

Conferir a resposta comentada

A simulação verifica o contrato do programa sem exercer cabo, porta, driver ou dispositivo. A comunicação física exige uma sessão de campo com aquele hardware.

Resposta do capítulo 41 no Apêndice B.

42. Laboratório

Objetivo de leitura

Planejar, registrar e comparar execuções de um experimento.

Antes de começar

Conheça funções, sementes e medições.

Um experimento executa uma operação sob configurações conhecidas e registra medições. Antes de executar, decida quais parâmetros variar, quantas repetições fazer e o que medir. Duas configurações com três repetições produzem seis execuções.

O planejamento ajuda a perceber o custo antes de iniciar. Depois, preserve resultados e contexto para repetir o procedimento e comparar as medições.

Da configuração ao resultado

Experimentos aceitam repetições, parâmetros, sementes e métricas. A implementação preserva os estados aleatórios globais do chamador e não encerra um `tracemalloc` que já estivesse ativo antes do experimento. Na 1.6.10, `planejar_experimento` calcula dimensões, combinações e execuções previstas antes da execução; `selecionar_medicoes` filtra resultados sem alterar o original; e `resumir_por_parametro` apresenta contagens, falhas, tempos e memória por cenário.

`validar_resultado` produz diagnósticos estruturados antes de persistir ou comparar. O JSON completo usa `coral.laboratorio.resultado/1`, esquema 1, e `carregar_resultado_json` reconstrói tipos portáteis e reconhece explicitamente o formato

histórico conhecido até a 1.6.9. CSV e Markdown continuam formatos de relatório, enquanto JSON é a representação recuperável. `comparar_experimentos` descreve diferenças de configuração, sucessos, tempo e memória usando `b - a`, mas não calcula significância estatística, causalidade nem superioridade geral.

Na 1.6.17, receitas procedurais podem ser planejadas e executadas como fator experimental. O plano registra quantidade de sementes, grade, repetições e custo preventivo. Cada medição preserva `semente_procedural`, enquanto o resultado guarda a receita base em metadados. Esse mecanismo mede configurações; ele não escolhe automaticamente a melhor semente nem transforma diferenças descritivas em conclusão científica.

Organizar um experimento

`coral.laboratorio` permite planejar, executar, filtrar, resumir, persistir e comparar experimentos reproduzíveis. O módulo registra e descreve resultados; a inferência estatística exige métodos próprios.

Planejar antes de executar

`planejar_experimento(parametros, repeticoes=1, limite=None)` calcula dimensões, combinações e execuções previstas sem construir toda a grade cartesiana. Quando `limite` é informado, o plano registra se o total o excede e em quanto. A função nunca executa o código estudado.

Selecionar e resumir medições

`selecionar_medicoes(resultado, criterio)` aceita uma função de decisão ou um mapeamento de igualdade. Use `parametros.nome` e `metricas.nome` para tornar a origem de uma chave explícita. A seleção preserva a ordem original e não altera o resultado.

`resumir_por_parametro(resultado, nome)` agrupa medições pelo valor do parâmetro e informa execuções, sucessos, falhas, tempo médio, tempo mediano e médias de memória. A ordem dos grupos segue a primeira aparição no resultado.

Validar, salvar e reabrir

`validar_resultado(resultado)` devolve diagnósticos estruturados para estrutura e portabilidade. JSONs novos usam o contrato `coral.laboratorio.resultado/1` e esquema 1. `carregar_resultado_json(caminho)` restaura resultados desse contrato e reconhece explicitamente o formato histórico conhecido até a 1.6.9; contratos ou esquemas desconhecidos são recusados. Valores portáteis, incluindo bytes, são reconstruídos em vez de convertidos silenciosamente para texto.

CSV e Markdown continuam exportações de leitura e análise externa. O JSON versionado é a representação completa destinada à recuperação de um `ResultadoExperimento`.

Comparar experimentos

`comparar_experimentos(a, b)` apresenta diferenças de configuração, quantidade de execuções, sucessos, falhas, tempos e memória. Diferenças numéricas são expressas como `b - a`. A função é deliberadamente descritiva: não calcula valor p, intervalo de confiança, causalidade nem superioridade geral. Para percentis, correlação ou outras operações numéricas, selecione as medições desejadas e use `coral.numerico`.

Superfície principal

`ResultadoExperimento.resumo()` continua sendo a visão geral. As operações novas complementam essa visão: `planejar_experimento`, `selecionar_medicoes`, `resumir_por_parametro`, `validar_resultado`, `carregar_resultado_json` e `comparar_experimentos`.

Projeto curto: Experimento com seis execuções

Planeje duas configurações e três repetições antes de executar. Defina como registrar falhas.

A orientação de solução está no Apêndice B, seção Projeto 42.

Fichas de API

Consulte os contratos abaixo para conferir tipos, argumentos, valores padrão, retorno e erros. Todas as fichas permanecem disponíveis para uso real.

Abrir as fichas completas de consulta

`versao_corrente(...)` · **módulo** `coral`

O que faz

Consulta a versão semântica publicada pela instalação Coral em execução.

Assinatura

Assinatura para consulta

```
versao_corrente() -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Texto no formato maior.menor.correção, sem depender de arquivos externos.

Exemplo

Código Coral

```
de coral importe versao_corrente
mostre versao_corrente()
```

Resultado

Saída esperada

```
1.8.0
```

Erros

Falha de identidade se o módulo Coral ainda não disponibilizar uma versão semântica válida.

`versao_anterior(...)` · módulo `coral`

O que faz

Determina a versão anterior usando a regra de patch e a âncora explícita nas transições de linha.

Assinatura

Assinatura para consulta

```
versao_anterior(atual: 'str | None' = None, *, ancora_linha_anterior: 'str | None' =
  None) -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
atual	str | None	não	None	Versão de referência; quando omitida utiliza a versão corrente.
ancora_linha_anterior	str | None	não	None	Última versão comprovada da linha anterior; necessária nas transições sem âncora interna.

Retorno

Texto da versão imediatamente anterior segundo a política oficial de releases.

Exemplo

Código Coral

```
de coral importe versao_anterior
mostre versao_anterior("1.8.3")
```

Resultado

Saída esperada

```
1.8.2
```

Erros

ValueError para versão inválida, âncora ausente ou âncora incompatível.

`bases_release(...)` · módulo `coral`

O que faz

Obtém as bases de comparação para o pacote anterior e o pacote estável.

Assinatura

Assinatura para consulta

```
bases_release(atual: 'str | None' = None, *, ancora_linha_anterior: 'str | None' = None)
   -> 'dict[str, str]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
atual	str | None	não	None	Versão cujas bases se deseja consultar.
ancora_linha_anterior	str | None	não	None	Versão validada da linha precedente quando a versão é o início de uma linha.

Retorno

Dicionário com chaves anterior e estavel, ambas resolvidas conforme a regra da versão.

Exemplo

Código Coral

```
de coral importe bases_release
defina bases como bases_release("1.8.3")
mostre bases["anterior"]
```

Resultado

Saída esperada

```
1.8.2
```

Erros

ValueError para versões ou âncoras incompatíveis.

`identidade_release(...)` · módulo `coral`

O que faz

Descreve o contrato de identidade usado na atualização da release.

Assinatura

Assinatura para consulta

```
identidade_release(atual: 'str | None' = None, *, ancora_linha_anterior: 'str | None' =
  None) -> 'dict[str, object]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
atual	str | None	não	None	Versão que se quer identificar; omitida, utiliza a corrente.
ancora_linha_anterior	str | None	não	None	Âncora da linha anterior quando necessária.

Retorno

Dicionário com contrato, versão, anterior, estável e critério de derivação.

Exemplo

Código Coral

```
de coral importe identidade_release
defina identificacao como identidade_release("1.8.3")
mostre identificacao["versao"]
```

Resultado

Saída esperada

```
1.8.3
```

Erros

ValueError se a versão ou a âncora não respeitarem a ordem semântica.

`pyproject_declara_versao_corrente(...)` · módulo `coral`

O que faz

Verifica se um trecho de metadados declara a versão ativa de modo literal ou dinâmico compatível.

Assinatura

Assinatura para consulta

```
pyproject_declara_versao_corrente(texto: 'str') -> 'bool'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
texto	str	sim	sem padrão	Conteúdo textual de metadados a examinar, preservando as linhas.

Retorno

Booleano; verdadeiro apenas quando o texto satisfaz a regra publicada.

Exemplo

Código Coral

```
de coral importe pyproject_declara_versao_corrente
mostre pyproject_declara_versao_corrente("version = \"1.8.0\"")
```

Resultado

Saída esperada

```
True
```

Erros

Entradas que não sejam texto resultam em falso; não modifica nem interpreta todo o arquivo TOML.

`traduzir_expressao(...)` · módulo `coral`

O que faz

Traduz uma única expressão Coral para sua representação Python de acordo com o parser.

Assinatura

Assinatura para consulta

```
traduzir_expressao(expr: 'str') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
expr	str	sim	sem padrão	Expressão escrita na sintaxe Coral.

Retorno

Texto de expressão Python; não executa o resultado.

Exemplo

Código Coral

```
de coral importe traduzir_expressao
mostre traduzir_expressao("1 + 2")
```

Resultado

Saída esperada

```
(1 + 2)
```

Erros

ErroDeTraducao quando a expressão não é aceita pela gramática.

`traduzir_codigo(...)` · módulo `coral`

O que faz

Traduz o código fonte Coral em memória sem executá-lo.

Assinatura

Assinatura para consulta

```
traduzir_codigo(codigo: 'str', nome_fonte: 'str' = '<memoria>') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
codigo	str	sim	sem padrão	Código fonte Coral que será traduzido.
nome_fonte	str	não	'<memoria>'	Nome informativo para diagnósticos e rastreamento da origem.

Retorno

Texto do código Python gerado pelo compilador.

Exemplo

Código Coral

```
de coral importe traduzir_codigo
defina python_gerado como traduzir_codigo("mostre 7")
mostre python_gerado != nulo
```

Resultado

Saída esperada

```
True
```

Erros

Erro de compilação ou sintaxe para programas Coral inválidos.

`analisar_expressao_coral(...)` · módulo `coral`

O que faz

Constrói a árvore sintática de uma expressão Coral, preservando a estrutura para inspeção.

Assinatura

Assinatura para consulta

```
analisar_expressao_coral(expr: 'str')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
expr	str	sim	sem padrão	Texto da expressão a analisar.

Retorno

Objeto de árvore sintática; não executa a expressão.

Exemplo

Código Coral

```
de coral importe analisar_expressao_coral
defina expressao como analisar_expressao_coral("1 + 2")
mostre expressao != nulo
```

Resultado

Saída esperada

```
True
```

Erros

Erro de expressão se a sintaxe for inválida.

`analisar(...)` · módulo `coral.api`

O que faz

Solicita diagnósticos de um programa Coral pela API pública legada 1.0.

Assinatura

Assinatura para consulta

```
analisar(codigo: 'str', nome_fonte: 'str' = '<api>') -> 'ResultadoAnalise'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
codigo	str	sim	sem padrão	Código Coral em memória a analisar.
nome_fonte	str	não	'<api>'	Nome do arquivo lógico exibido nos diagnósticos.

Retorno

ResultadoAnalise com a coleção de diagnósticos e indicação de erros.

Exemplo

Código Coral

```
de coral.api importe analisar
defina resultado como analisar("mostre 7")
mostre resultado.possui_erros
```

Resultado

Saída esperada

```
False
```

Erros

Erros de entrada podem provocar diagnósticos; não executa o programa.

`analisar_snapshot(...)` · módulo `coral.api`

O que faz

Cria um snapshot reutilizável da análise de um programa, evitando refazer o parse em outras etapas.

Assinatura

Assinatura para consulta

```
analisar_snapshot(codigo: 'str', nome_fonte: 'str' = '<api>', *, versao: 'int | str' =
  1) -> 'SnapshotAnalise'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
codigo	str	sim	sem padrão	Código Coral a analisar.
nome_fonte	str	não	'<api>'	Nome lógico do arquivo de origem.
versao	int | str	não	1	Versão contratual do snapshot solicitada; padrão 1.

Retorno

SnapshotAnalise com AST e diagnóstico para reuso no compilador.

Exemplo

Código Coral

```
de coral.api importe analisar_snapshot
defina foto como analisar_snapshot("mostre 7")
mostre foto != nulo
```

Resultado

Saída esperada

```
True
```

Erros

Versão de snapshot não suportada ou entrada inválida é rejeitada.

`compilar(...)` · módulo `coral.api`

O que faz

Compila um programa Coral em memória, preservando informações de mapeamento de origem.

Assinatura

Assinatura para consulta

```
compilar(codigo: 'str', nome_fonte: 'str' = '<api>') -> 'ResultadoCompilacao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
codigo	str	sim	sem padrão	Texto fonte do programa Coral.
nome_fonte	str	não	'<api>'	Nome lógico para rastreamento de erros.

Retorno

ResultadoCompilacao com código gerado, sem executar o programa.

Exemplo

Código Coral

```
de coral.api importe compilar
defina resultado como compilar("mostre 7")
mostre resultado.codigo != nulo
```

Resultado

Saída esperada

```
True
```

Erros

ErroCoral ou ErroDeTraducao para sintaxe ou construção semântica inválida.

`compilar_de_snapshot(...)` · módulo `coral.api`

O que faz

Gera código a partir de um snapshot de análise previamente criado, evitando um segundo parse.

Assinatura

Assinatura para consulta

```
compilar_de_snapshot(snapshot: 'SnapshotAnalise') -> 'ResultadoCompilacao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
snapshot	SnapshotAnalise	sim	sem padrão	SnapshotAnalise produzido pela API de análise.

Retorno

ResultadoCompilacao referente ao snapshot fornecido.

Exemplo

Código Coral

```
de coral.api importe analisar_snapshot, compilar_de_snapshot
defina foto como analisar_snapshot("mostre 7")
defina resultado como compilar_de_snapshot(foto)
mostre resultado.codigo != nulo
```

Resultado

Saída esperada

```
True
```

Erros

Rejeita snapshots incompatíveis ou árvores inválidas durante a compilação.

`formatar(...)` · módulo `coral.api`

O que faz

Aplica a formatação oficial ao código Coral sem solicitar execução.

Assinatura

Assinatura para consulta

```
formatar(codigo: 'str') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
codigo	str	sim	sem padrão	Texto de origem que será formatado.

Retorno

Código Coral formatado como texto; pode incluir quebra de linha final.

Exemplo

Código Coral

```
de coral.api importe formatar
mostre formatar("mostre 7")
```

Resultado

Saída esperada

```
mostre 7
```

Erros

Código de entrada inválido ou ambíguo pode ser rejeitado pelo formatador.

`info_api(...)` · módulo `coral.api`

O que faz

Consulta as versões da API pública e as operações disponíveis para ferramentas externas.

Assinatura

Assinatura para consulta

```
info_api() -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Dicionário com versão histórica, versão mais recente e funções anunciadas.

Exemplo

Código Coral

```
de coral.api importe info_api
defina info como info_api()
mostre info["api"]
```

Resultado

Saída esperada

```
1.0
```

Erros

Não recebe parâmetros e não acessa o sistema de arquivos.

`parse_expressao(...)` · módulo `coral.api`

O que faz

Analisa uma expressão da gramática Coral e produz a AST de expressão.

Assinatura

Assinatura para consulta

```
parse_expressao(texto: 'str')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
texto	str	sim	sem padrão	Expressão a analisar.

Retorno

Objeto AST com informações estruturais, sem avaliação de valores.

Exemplo

Código Coral

```
de coral.api importe parse_expressao
defina expr como parse_expressao("2 + 3")
mostre expr != nulo
```

Resultado

Saída esperada

```
True
```

Erros

Erro de análise para expressão inválida.

`parse_programa(...)` · módulo `coral.api`

O que faz

Analisa um programa Coral completo e produz sua árvore sintática.

Assinatura

Assinatura para consulta

```
parse_programa(codigo: 'str') -> 'Programa'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
codigo	str	sim	sem padrão	Texto fonte Coral a converter em AST.

Retorno

Programa, que representa a AST estrutural do código.

Exemplo

Código Coral

```
de coral.api importe parse_programa
defina programa como parse_programa("mostre 7")
mostre programa != nulo
```

Resultado

Saída esperada

```
True
```

Erros

Erro de sintaxe se o programa não respeitar a gramática.

`soma(...)` · módulo `coral.numerico`

O que faz

Soma os elementos de uma coleção numérica pela redução do NumPy.

Assinatura

Assinatura para consulta

```
soma(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Coleção de números a somar.

Retorno

Escalar ou agregado numérico; a precisão segue o tipo do conjunto.

Exemplo

Código Coral

```
de coral.numerico importe soma
mostre soma([1, 2, 3])
```

Resultado

Saída esperada

```
6
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

Entradas incompatíveis podem provocar erro de conversão do NumPy.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`media(...)` · módulo `coral.numerico`

O que faz

Calcula a média aritmética de uma coleção de números finitos e não vazia.

Assinatura

Assinatura para consulta

```
media(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Observações finitas para a média.

Retorno

Número decimal com a média aritmética.

Exemplo

Código Coral

```
de coral.numerico importe media
mostre media([2, 4, 6])
```

Resultado

Saída esperada

```
4.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico se a coleção estiver vazia, contiver ausentes ou valores não finitos.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`mediana(...)` · módulo `coral.numerico`

O que faz

Calcula a mediana de valores numéricos, interpolando os dois centrais quando a quantidade é par.

Assinatura

Assinatura para consulta

```
mediana(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Observações finitas da amostra.

Retorno

Valor mediano como resultado numérico do NumPy.

Exemplo

Código Coral

```
de coral.numerico importe mediana
mostre mediana([1, 3, 9])
```

Resultado

Saída esperada

```
3.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para dados vazios, ausentes ou não finitos.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`variancia(...)` · módulo `coral.numerico`

O que faz

Mede a dispersão ao quadrado; por padrão divide pelo total de observações.

Assinatura

Assinatura para consulta

```
variancia(valores, populacional=True)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Observações numéricas finitas.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
populacional	valor dinâmico	não	True	Verdadeiro usa divisor N; falso usa N menos um.

Retorno

Variância populacional ou amostral conforme a opção.

Exemplo

Código Coral

```
de coral.numerico importe variancia
mostre variancia([1, 3, 5])
```

Resultado

Saída esperada

```
2.6666666666666665
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para dados vazios ou não finitos; a forma amostral exige pelo menos dois valores.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`desvio_padrao(...)` · módulo `coral.numerico`

O que faz

Obtém a raiz da variância, adotando por padrão a convenção populacional.

Assinatura

Assinatura para consulta

```
desvio_padrao(valores, populacional=True)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Observações numéricas finitas.
populacional	valor dinâmico	não	True	Verdadeiro usa convenção populacional, falso usa amostral.

Retorno

Desvio padrão da série na mesma unidade dos valores.

Exemplo

Código Coral

```
de coral.numerico importe desvio_padrao
mostre desvio_padrao([2, 2, 2])
```

Resultado

Saída esperada

```
0.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para dados vazios ou não finitos; amostra requer pelo menos dois valores.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`minimo(...)` · módulo `coral.numerico`

O que faz

Obtém o menor elemento de uma coleção numérica não vazia.

Assinatura

Assinatura para consulta

```
minimo(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Coleção finita da qual extrair o mínimo.

Retorno

Menor número observado.

Exemplo

Código Coral

```
de coral.numerico importe minimo
mostre minimo([5, 2, 9])
```

Resultado

Saída esperada

```
2
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico quando a coleção estiver vazia ou tiver observações inválidas.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`maximo(...)` · módulo `coral.numerico`

O que faz

Obtém o maior elemento de uma coleção numérica não vazia.

Assinatura

Assinatura para consulta

```
maximo(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Coleção finita da qual extrair o máximo.

Retorno

Maior número observado.

Exemplo

Código Coral

```
de coral.numerico importe maximo
mostre maximo([5, 2, 9])
```

Resultado

Saída esperada

```
9
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico quando a coleção estiver vazia ou tiver observações inválidas.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`percentil(...)` · módulo `coral.numerico`

O que faz

Interpola um percentil entre zero e cem em uma série numérica finita.

Assinatura

Assinatura para consulta

```
percentil(valores, percentual)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Série finita da qual obter o percentil.
percentual	valor dinâmico	sim	sem padrão	Posição percentual desejada, inclusive os extremos 0 e 100.

Retorno

Valor correspondente ao percentil solicitado.

Exemplo

Código Coral

```
de coral.numerico importe percentil
mostre percentil([0, 10, 20], 50)
```

Resultado

Saída esperada

```
10.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para percentual fora de 0 a 100, dados vazios ou não finitos.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`covariancia(...)` · módulo `coral.numerico`

O que faz

Calcula a covariância de duas séries de mesmo comprimento, com divisor populacional por padrão.

Assinatura

Assinatura para consulta

```
covariancia(a, b, populacional=True)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	valor dinâmico	sim	sem padrão	Primeira série de observações.
b	valor dinâmico	sim	sem padrão	Segunda série pareada de observações.
populacional	valor dinâmico	não	True	Verdadeiro usa N; falso usa N menos um.

Retorno

Número positivo, negativo ou zero segundo a variação conjunta.

Exemplo

Código Coral

```
de coral.numerico importe covariancia
mostre covariancia([1, 2, 3], [2, 4, 6])
```

Resultado

Saída esperada

```
1.3333333333333335
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para comprimentos distintos, dados não finitos ou amostra insuficiente.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`correlacao(...)` · módulo `coral.numerico`

O que faz

Calcula a correlação de Pearson entre séries finitas pareadas.

Assinatura

Assinatura para consulta

```
correlacao(a, b)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	valor dinâmico	sim	sem padrão	Primeira série de dados.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
b	valor dinâmico	sim	sem padrão	Segunda série da mesma dimensão.

Retorno

Coeficiente adimensional de correlação entre menos um e um.

Exemplo

Código Coral

```
de coral.numerico importe correlacao
mostre correlacao([1, 2, 3], [1, 2, 3])
```

Resultado

Saída esperada

```
1.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para amostra insuficiente, comprimentos diferentes ou variância nula.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`mediana_desvios_absolutos(...)` · módulo `coral.numerico`

O que faz

Obtém a mediana das distâncias absolutas em relação à própria mediana, medida robusta de dispersão.

Assinatura

Assinatura para consulta

```
mediana_desvios_absolutos(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Observações finitas para medir dispersão robusta.

Retorno

Mediana dos desvios absolutos.

Exemplo

Código Coral

```
de coral.numerico importe mediana_desvios_absolutos
mostre mediana_desvios_absolutos([2, 3, 4])
```

Resultado

Saída esperada

```
1.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para observações vazias ou inválidas.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`media_ponderada(...)` · módulo `coral.numerico`

O que faz

Combina observações segundo pesos não negativos de soma positiva.

Assinatura

Assinatura para consulta

```
media_ponderada(valores, pesos)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Valores a agregar.
pesos	valor dinâmico	sim	sem padrão	Importância de cada valor na mesma ordem e comprimento.

Retorno

Média ponderada, considerando apenas pesos fornecidos.

Exemplo

Código Coral

```
de coral.numerico importe media_ponderada
mostre media_ponderada([10, 20], [1, 3])
```

Resultado

Saída esperada

```
17.5
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para dimensões diferentes, pesos negativos, soma de pesos nula ou valores não finitos.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`media_geometrica(...)` · módulo `coral.numerico`

O que faz

Calcula a raiz de ordem N do produto de N observações estritamente positivas.

Assinatura

Assinatura para consulta

```
media_geometrica(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Valores reais positivos.

Retorno

Média geométrica positiva.

Exemplo

Código Coral

```
de coral.numerico importe media_geometrica
mostre media_geometrica([4, 4])
```

Resultado

Saída esperada

```
4.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para dados vazios, não finitos ou valores menores ou iguais a zero.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`media_harmonica(...)` · módulo `coral.numerico`

O que faz

Calcula a razão entre a quantidade de valores e a soma de seus recíprocos.

Assinatura

Assinatura para consulta

```
media_harmonica(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Observações estritamente positivas.

Retorno

Média harmônica positiva.

Exemplo

Código Coral

```
de coral.numerico importe media_harmonica
mostre media_harmonica([2, 2])
```

Resultado

Saída esperada

```
2.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para dados vazios, não finitos ou não positivos.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`amplitude(...)` · módulo `coral.numerico`

O que faz

Calcula a diferença entre o maior e o menor valor de uma série numérica.

Assinatura

Assinatura para consulta

```
amplitude(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Série de valores finitos.

Retorno

Amplitude total não negativa.

Exemplo

Código Coral

```
de coral.numerico importe amplitude
mostre amplitude([2, 9, 4])
```

Resultado

Saída esperada

```
7.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para amostra vazia, não finita ou resultado fora do intervalo numérico.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`intervalo_interquartil(...)` · módulo `coral.numerico`

O que faz

Subtrai o primeiro quartil do terceiro quartil para medir a dispersão central de uma série.

Assinatura

Assinatura para consulta

```
intervalo_interquartil(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Valores ordenáveis e finitos da amostra.

Retorno

Intervalo interquartil, não negativo.

Exemplo

Código Coral

```
de coral.numerico importe intervalo_interquartil
mostre intervalo_interquartil([1, 2, 3, 4, 5])
```

Resultado

Saída esperada

```
2.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para amostra vazia, não finita ou fora do intervalo suportado.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`produto_escalar(...)` · módulo `coral.numerico`

O que faz

Multiplica componentes correspondentes e soma seus produtos para obter um produto escalar.

Assinatura

Assinatura para consulta

```
produto_escalar(a, b)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	valor dinâmico	sim	sem padrão	Primeiro vetor numérico.
b	valor dinâmico	sim	sem padrão	Segundo vetor de dimensão compatível.

Retorno

Escalar numérico quando os argumentos são vetores compatíveis.

Exemplo

Código Coral

```
de coral.numerico importe produto_escalar
mostre produto_escalar([1, 2], [3, 4])
```

Resultado

Saída esperada

```
11
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para dimensões que não permitem o produto; outros erros NumPy para tipos incompatíveis.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`distancia_vetores(...)` · módulo `coral.numerico`

O que faz

Mede a distância euclidiana entre dois vetores finitos de mesma dimensão.

Assinatura

Assinatura para consulta

```
distancia_vetores(a, b)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	valor dinâmico	sim	sem padrão	Primeiro vetor.
b	valor dinâmico	sim	sem padrão	Segundo vetor da mesma dimensão.

Retorno

Distância não negativa no espaço euclidiano.

Exemplo

Código Coral

```
de coral.numerico importe distancia_vetores
mostre distancia_vetores([0, 0], [3, 4])
```

Resultado

Saída esperada

```
5.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para dimensões incompatíveis, vetores vazios ou dados não finitos.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`norma(...)` · módulo `coral.numerico`

O que faz

Obtém o comprimento euclidiano de um vetor finito.

Assinatura

Assinatura para consulta

```
norma(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Componentes numéricas de um vetor.

Retorno

Comprimento não negativo.

Exemplo

Código Coral

```
de coral.numerico importe norma
mostre norma([3, 4])
```

Resultado

Saída esperada

```
5.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para vetor vazio ou com componentes não finitos.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`vetor3(...)` · módulo `coral.numerico`

O que faz

Constrói explicitamente um vetor de três coordenadas numéricas.

Assinatura

Assinatura para consulta

```
vetor3(x, y, z)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	valor dinâmico	sim	sem padrão	Componente do eixo X.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
y	valor dinâmico	sim	sem padrão	Componente do eixo Y.
z	valor dinâmico	sim	sem padrão	Componente do eixo Z.

Retorno

Vetor tridimensional com as coordenadas informadas.

Exemplo

Código Coral

```
de coral.numerico importe vetor3
mostre vetor3(1, 2, 3)
```

Resultado

Saída esperada

```
[1. 2. 3.]
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico se uma componente não puder ser interpretada numericamente.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`produto_vetorial(...)` · módulo `coral.numerico`

O que faz

Calcula o vetor perpendicular a dois vetores tridimensionais pelo produto vetorial orientado.

Assinatura

Assinatura para consulta

```
produto_vetorial(a, b)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	valor dinâmico	sim	sem padrão	Primeiro vetor 3D.
b	valor dinâmico	sim	sem padrão	Segundo vetor 3D.

Retorno

Vetor tridimensional ortogonal aos dois argumentos.

Exemplo

Código Coral

```
de coral.numerico importe produto_vetorial
mostre produto_vetorial([1, 0, 0], [0, 1, 0])
```

Resultado

Saída esperada

```
[0. 0. 1.]
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para dimensões diferentes de três ou componentes inválidas.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`area_triangulo3(...)` · módulo `coral.numerico`

O que faz

Obtém a área do triângulo determinado por três pontos tridimensionais.

Assinatura

Assinatura para consulta

```
area_triangulo3(a, b, c)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	valor dinâmico	sim	sem padrão	Primeiro vértice 3D.
b	valor dinâmico	sim	sem padrão	Segundo vértice 3D.
c	valor dinâmico	sim	sem padrão	Terceiro vértice 3D.

Retorno

Área não negativa em unidades quadradas.

Exemplo

Código Coral

```
de coral.numerico importe area_triangulo3
mostre area_triangulo3([0, 0, 0], [1, 0, 0], [0, 1, 0])
```

Resultado

Saída esperada

```
0.5
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico quando as coordenadas são inválidas ou fora da dimensão 3D.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`volume_paralelepipedo3(...)` · módulo `coral.numerico`

O que faz

Calcula o volume absoluto do sólido formado por três vetores tridimensionais.

Assinatura

Assinatura para consulta

```
volume_paralelepipedo3(a, b, c)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	valor dinâmico	sim	sem padrão	Primeira aresta.
b	valor dinâmico	sim	sem padrão	Segunda aresta.
c	valor dinâmico	sim	sem padrão	Terceira aresta.

Retorno

Volume não negativo do paralelepípedo.

Exemplo

Código Coral

```
de coral.numerico importe volume_paralelepipedo3
mostre volume_paralelepipedo3([1, 0, 0], [0, 1, 0], [0, 0, 1])
```

Resultado

Saída esperada

```
1.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico se um argumento não for um vetor 3D finito.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`volume_tetraedro3(...)` · módulo `coral.numerico`

O que faz

Obtém o volume absoluto do tetraedro por quatro pontos tridimensionais.

Assinatura

Assinatura para consulta

```
volume_tetraedro3(a, b, c, d)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	valor dinâmico	sim	sem padrão	Primeiro vértice 3D.
b	valor dinâmico	sim	sem padrão	Segundo vértice 3D.
c	valor dinâmico	sim	sem padrão	Terceiro vértice 3D.
d	valor dinâmico	sim	sem padrão	Quarto vértice 3D.

Retorno

Volume não negativo do tetraedro.

Exemplo

Código Coral

```
de coral.numerico importe volume_tetraedro3
mostre volume_tetraedro3([0, 0, 0], [1, 0, 0], [0, 1, 0], [0, 0, 1])
```

Resultado

Saída esperada

```
0.16666666666666666
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ErroNumerico para argumentos incompatíveis com pontos 3D.

Atenção

A disponibilidade de bibliotecas opcionais deve ser considerada quando a implementação exigir dependências externas.

`onda_seno(...)` · módulo `coral.jogos`

O que faz

Retorna uma onda senoidal com período um, recebendo a fase em ciclos.

Assinatura

Assinatura para consulta

```
onda_seno(fase: 'float') -> 'float'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
fase	float	sim	sem padrão	Posição no ciclo, em que 1 equivale a uma volta completa.

Retorno

Amplitude senoidal entre menos um e um.

Exemplo

Código Coral

```
de coral.jogos importe onda_seno
mostre onda_seno(0)
```

Resultado

Saída esperada

```
0.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

A conversão de fase para float falha em entradas não numéricas.

Atenção

O valor de progresso não é limitado automaticamente por todas as curvas; use normalizar_progresso quando essa política for desejada.

`onda_cosseno(...)` · módulo `coral.jogos`

O que faz

Retorna uma onda cossenoidal de período um para uso em animações.

Assinatura

Assinatura para consulta

```
onda_cosseno(fase: 'float') -> 'float'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
fase	float	sim	sem padrão	Posição da oscilação em ciclos completos.

Retorno

Amplitude cossenoidal entre menos um e um.

Exemplo

Código Coral

```
de coral.jogos importe onda_cosseno
mostre onda_cosseno(0)
```

Resultado

Saída esperada

```
1.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

A conversão de fase para float falha em entradas não numéricas.

Atenção

O valor de progresso não é limitado automaticamente por todas as curvas; use normalizar_progresso quando essa política for desejada.

`onda_triangular(...)` · módulo `coral.jogos`

O que faz

Gera uma onda triangular contínua, repetindo sua forma em cada ciclo.

Assinatura

Assinatura para consulta

```
onda_triangular(fase: 'float') -> 'float'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
fase	float	sim	sem padrão	Fração do ciclo, repetida periodicamente.

Retorno

Amplitude triangular normalizada entre menos um e um.

Exemplo

Código Coral

```
de coral.jogos importe onda_triangular
mostre onda_triangular(0.5)
```

Resultado

Saída esperada

```
1.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

A conversão de fase para float falha para valores incompatíveis.

Atenção

O valor de progresso não é limitado automaticamente por todas as curvas; use normalizar_progresso quando essa política for desejada.

`onda_serra(...)` · módulo `coral.jogos`

O que faz

Produz uma rampa periódica crescente que retorna ao início a cada ciclo.

Assinatura

Assinatura para consulta

```
onda_serra(fase: 'float') -> 'float'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
fase	float	sim	sem padrão	Fração do ciclo em andamento.

Retorno

Amplitude dente de serra no intervalo de menos um inclusive até um exclusive.

Exemplo

Código Coral

```
de coral.jogos importe onda_serra
mostre onda_serra(0.25)
```

Resultado

Saída esperada

```
-0.5
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

A conversão de fase para float falha para valores incompatíveis.

Atenção

O valor de progresso não é limitado automaticamente por todas as curvas; use normalizar_progresso quando essa política for desejada.

`onda_pulso(...)` · módulo `coral.jogos`

O que faz

Alterna entre amplitudes positiva e negativa a cada metade de ciclo.

Assinatura

Assinatura para consulta

```
onda_pulso(fase: 'float') -> 'float'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
fase	float	sim	sem padrão	Fase normalizada em ciclos; valores são repetidos módulo 1.

Retorno

Um enquanto a fase estiver na primeira metade, menos um na segunda.

Exemplo

Código Coral

```
de coral.jogos importe onda_pulso
mostre onda_pulso(0.25)
```

Resultado

Saída esperada

```
1.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

A conversão de fase para float falha para valores incompatíveis.

Atenção

O valor de progresso não é limitado automaticamente por todas as curvas; use normalizar_progresso quando essa política for desejada.

`curva_linear(...)` · módulo `coral.jogos`

O que faz

Preserva o progresso de uma interpolação sem aceleração nem desaceleração.

Assinatura

Assinatura para consulta

```
curva_linear(progresso: 'float') -> 'float'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
progresso	float	sim	sem padrão	Fração da duração da animação.

Retorno

O mesmo progresso convertido em ponto flutuante.

Exemplo

Código Coral

```
de coral.jogos importe curva_linear
mostre curva_linear(0.5)
```

Resultado

Saída esperada

```
0.5
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ValueError ou TypeError para valores não conversíveis a float.

Atenção

O valor de progresso não é limitado automaticamente por todas as curvas; use normalizar_progresso quando essa política for desejada.

`curva_entrada_suave(...)` · módulo `coral.jogos`

O que faz

Aplica progressão quadrática crescente, começando com velocidade menor.

Assinatura

Assinatura para consulta

```
curva_entrada_suave(progresso: 'float') -> 'float'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
progresso	float	sim	sem padrão	Fração temporal a ser elevada ao quadrado.

Retorno

Quadrado do progresso fornecido.

Exemplo

Código Coral

```
de coral.jogos importe curva_entrada_suave
mostre curva_entrada_suave(0.5)
```

Resultado

Saída esperada

```
0.25
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ValueError ou TypeError se o progresso não puder ser convertido.

Atenção

O valor de progresso não é limitado automaticamente por todas as curvas; use normalizar_progresso quando essa política for desejada.

`curva_saida_suave(...)` · módulo `coral.jogos`

O que faz

Aplica progressão quadrática com desaceleração próxima do fim.

Assinatura

Assinatura para consulta

```
curva_saida_suave(progresso: 'float') -> 'float'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
progresso	float	sim	sem padrão	Fração temporal de referência.

Retorno

Um menos o quadrado da distância restante até um.

Exemplo

Código Coral

```
de coral.jogos importe curva_saida_suave
mostre curva_saida_suave(0.5)
```

Resultado

Saída esperada

```
0.75
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ValueError ou TypeError se o progresso não puder ser convertido.

Atenção

O valor de progresso não é limitado automaticamente por todas as curvas; use normalizar_progresso quando essa política for desejada.

`curva_entrada_saida_suave(...)` · módulo `coral.jogos`

O que faz

Interpola com aceleração e desaceleração suaves pelo polinômio cúbico suave.

Assinatura

Assinatura para consulta

```
curva_entrada_saida_suave(progresso: 'float') -> 'float'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
progresso	float	sim	sem padrão	Fração temporal que entra na curva suave.

Retorno

Progresso transformado segundo $3p^2$ menos $2p^3$.

Exemplo

Código Coral

```
de coral.jogos importe curva_entrada_saida_suave
mostre curva_entrada_saida_suave(0.5)
```

Resultado

Saída esperada

```
0.5
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ValueError ou TypeError se o progresso não puder ser convertido.

Atenção

O valor de progresso não é limitado automaticamente por todas as curvas; use normalizar_progresso quando essa política for desejada.

`curva_quadratica(...)` · módulo `coral.jogos`

O que faz

Eleva o progresso ao quadrado para uma progressão não linear.

Assinatura

Assinatura para consulta

```
curva_quadratica(progresso: 'float') -> 'float'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
progresso	float	sim	sem padrão	Valor cuja potência quadrática será calculada.

Retorno

Progresso ao quadrado.

Exemplo

Código Coral

```
de coral.jogos importe curva_quadratica
mostre curva_quadratica(0.5)
```

Resultado

Saída esperada

```
0.25
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ValueError ou TypeError para entradas não convertíveis em float.

Atenção

O valor de progresso não é limitado automaticamente por todas as curvas; use normalizar_progresso quando essa política for desejada.

`curva_cubica(...)` · módulo `coral.jogos`

O que faz

Eleva o progresso ao cubo para uma progressão mais lenta no início.

Assinatura

Assinatura para consulta

```
curva_cubica(progresso: 'float') -> 'float'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
progresso	float	sim	sem padrão	Progresso temporal a elevar ao cubo.

Retorno

Progresso ao cubo.

Exemplo

Código Coral

```
de coral.jogos importe curva_cubica
mostre curva_cubica(0.5)
```

Resultado

Saída esperada

```
0.125
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ValueError ou TypeError para entradas não convertíveis em float.

Atenção

O valor de progresso não é limitado automaticamente por todas as curvas; use normalizar_progresso quando essa política for desejada.

`curva_degrau(...)` · módulo `coral.jogos`

O que faz

Retorna zero antes do fim da animação e um ao alcançar progresso um.

Assinatura

Assinatura para consulta

```
curva_degrau(progresso: 'float') -> 'float'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
progresso	float	sim	sem padrão	Fração temporal comparada ao limiar final 1.

Retorno

Zero antes do limite e um a partir de progresso um.

Exemplo

Código Coral

```
de coral.jogos importe curva_degrau
mostre curva_degrau(1)
```

Resultado

Saída esperada

```
1.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ValueError ou TypeError se o progresso não for conversível em float.

Atenção

O valor de progresso não é limitado automaticamente por todas as curvas; use normalizar_progresso quando essa política for desejada.

`normalizar_progresso(...)` · módulo `coral.jogos`

O que faz

Limita o progresso temporal ao intervalo unitário, salvo quando a extrapolação é solicitada.

Assinatura

Assinatura para consulta

```
normalizar_progresso(progresso: 'float', *, extrapolar: 'bool' = False) -> 'float'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
progresso	float	sim	sem padrão	Progresso que será validado como número finito.
extrapolar	bool	não	False	Verdadeiro conserva valores além de zero a um; falso os limita.

Retorno

Número finito original ou limitado ao intervalo de zero a um.

Exemplo

Código Coral

```
de coral.jogos importe normalizar_progresso
mostre normalizar_progresso(2)
```

Resultado

Saída esperada

```
1.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ValueError para NaN, infinito, booleano ou entrada não numérica.

Atenção

O valor de progresso não é limitado automaticamente por todas as curvas; use normalizar_progresso quando essa política for desejada.

`interpolar(...)` · módulo `coral.jogos`

O que faz

Interpola um valor entre início e fim segundo uma curva temporal nomeada ou fornecida.

Assinatura

Assinatura para consulta

```
interpolar(inicio: 'float', fim: 'float', progresso: 'float', *, curva: 'str |
  Callable[[float], float]' = 'linear', extrapolar: 'bool' = False) -> 'float'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
inicio	float	sim	sem padrão	Valor no início da transição.
fim	float	sim	sem padrão	Valor de destino.
progresso	float	sim	sem padrão	Fração temporal da transição.
curva	str | Callable[[float], float]	não	'linear'	Nome de curva cadastrada ou função personalizada.
extrapolar	bool	não	False	Permite progresso fora do intervalo unitário quando verdadeiro.

Retorno

Número interpolado com a curva escolhida.

Exemplo

Código Coral

```
de coral.jogos importe interpolar
mostre interpolar(0, 10, 0.5)
```

Resultado

Saída esperada

```
5.0
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ValueError para curva desconhecida ou parâmetros numéricos inválidos.

Atenção

O valor de progresso não é limitado automaticamente por todas as curvas; use normalizar_progresso quando essa política for desejada.

`avaliar_curva_temporal(...)` · módulo `coral.jogos`

O que faz

Avalia uma curva temporal cadastrada ou uma função personalizada no progresso indicado.

Assinatura

Assinatura para consulta

```
avaliar_curva_temporal(curva: 'str | Callable[[float], float]', progresso: 'float', *,
  extrapolar: 'bool' = False) -> 'float'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
curva	str | Callable[[float], float]	sim	sem padrão	Nome da curva temporal ou função de transformação.
progresso	float	sim	sem padrão	Instante normalizado da interpolação.
extrapolar	bool	não	False	Permite extrapolação temporal quando verdadeiro.

Retorno

Valor de progresso transformado pela curva.

Exemplo

Código Coral

```
de coral.jogos importe avaliar_curva_temporal
mostre avaliar_curva_temporal("linear", 0.5)
```

Resultado

Saída esperada

```
0.5
```

Forma natural

Não possui uma forma natural distinta; importe a função pública do módulo indicado.

Erros

ValueError para curva inexistente ou progresso não finito.

Atenção

O valor de progresso não é limitado automaticamente por todas as curvas; use normalizar_progresso quando essa política for desejada.

`traduzir_tipo(...)` · módulo `coral.tipos`

O que faz

Converte um tipo escrito em português Coral para a anotação equivalente na representação Python.

Assinatura

Assinatura para consulta

```
traduzir_tipo(tipo: 'str') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tipo	str	sim	sem padrão	Expressão de tipo da Coral em português.

Retorno

Texto com a expressão de tipo Python resultante da gramática de tipos.

Exemplo

Código Coral

```
de coral.tipos importe traduzir_tipo
mostre traduzir_tipo("inteiro")
```

Resultado

Saída esperada

```
int
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

A análise estrutural rejeita sintaxes e identificadores de tipo inválidos.

`politica_persistencia(...)` · módulo `coral.persistencia`

O que faz

Consulta as regras executáveis e a versão dos formatos persistentes suportados.

Assinatura

Assinatura para consulta

```
politica_persistencia() -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Dicionário com limites, versões e regras de persistência.

Exemplo

Código Coral

```
de coral.persistencia importe politica_persistencia
mostre politica_persistencia()
```

Resultado

Saída esperada

```
{'contrato': 'coral.persistencia/1', 'leitura': [1], 'escrita': 1, 'migracao':
  'recusa_explicita_de_esquema_desconhecido', 'compatibilidade': 'esquema_1_estavel'}
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Não acessa arquivos; não altera o registro de adaptadores.

`texto_canonico(...)` · módulo `coral.persistencia`

O que faz

Converte valores portáteis ao JSON determinístico usado em hashes e comparação.

Assinatura

Assinatura para consulta

```
texto_canonico(valor: 'Any') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	Any	sim	sem padrão	Valor que precisa ser representado sem ambiguidades.

Retorno

Texto JSON com chaves ordenadas e sem espaços de apresentação.

Exemplo

Código Coral

```
de coral.persistencia importe texto_canonico
mostre texto_canonico(10)
```

Resultado

Saída esperada

```
10
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

ErroPersistencia para valores sem codificação portátil ou não finitos.

`serializar_canonico(...)` · módulo `coral.persistencia`

O que faz

Codifica o JSON canônico em bytes UTF 8, mantendo a ordem e representação estáveis.

Assinatura

Assinatura para consulta

```
serializar_canonico(valor: 'Any') -> 'bytes'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	Any	sim	sem padrão	Valor portátil a serializar em bytes.

Retorno

Sequência de bytes da serialização canônica.

Exemplo

Código Coral

```
de coral.persistencia importe serializar_canonico
mostre serializar_canonico(10)
```

Resultado

Saída esperada

```
b'10'
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Propaga as falhas do codificador portátil para entradas não suportadas.

`valor_portatil(...)` · módulo `coral.persistencia`

O que faz

Produz representação portátil de um valor isolado sem envelope de documento.

Assinatura

Assinatura para consulta

```
valor_portatil(valor: 'Any') -> 'Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	Any	sim	sem padrão	Objeto isolado a converter para o contrato portátil.

Retorno

Valor primitivo ou estrutura JSON portátil, conforme o tipo de entrada.

Exemplo

Código Coral

```
de coral.persistencia importe valor_portatil
mostre valor_portatil(10)
```

Resultado

Saída esperada

```
10
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

ErroPersistencia quando o valor contém dados não serializáveis.

`para_dados(...)` · módulo `coral.persistencia`

O que faz

Encapsula um valor em documento persistente versionado e tipado.

Assinatura

Assinatura para consulta

```
para_dados(objeto: 'Any') -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Valor ou objeto com contrato de persistência.

Retorno

Dicionário de documento com contrato e conteúdo codificado.

Exemplo

Código Coral

```
de coral.persistencia importe para_dados
mostre para_dados(10)
```

Resultado

Saída esperada

```
{'contrato': 'coral.persistencia/1', 'versao_esquema': 1, 'tipo': 'coral.base/Valor',
  'dados': {'valor': 10}}
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

ErroPersistencia para tipos sem adaptador de persistência.

`de_dados(...)` · módulo `coral.persistencia`

O que faz

Reconstrói o objeto a partir de um documento do formato persistente canônico.

Assinatura

Assinatura para consulta

```
de_dados(documento: 'dict[str, Any]') -> 'Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
documento	dict[str, Any]	sim	sem padrão	Dicionário produzido pelo codificador para_dados.

Retorno

Valor ou objeto recuperado e validado.

Exemplo

Código Coral

```
de coral.persistencia importe de_dados
mostre de_dados({"contrato": "coral.persistencia/1", "versao_esquema": 1, "tipo":
  "coral.base/Valor", "dados": {"valor": 10}})
```

Resultado

Saída esperada

```
10
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

ErroPersistencia em documentos malformados ou com identificadores desconhecidos.

`caminho(...)` · módulo `coral.sistema`

O que faz

Constrói um caminho de arquivo juntando partes sem tocar no sistema de arquivos.

Assinatura

Assinatura para consulta

```
caminho(*partes: 'Any') -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
partes	Any	não	sem padrão	Componentes de caminho em ordem, sem criar arquivos.

Retorno

Objeto Path representando o caminho composto.

Exemplo

Código Coral

```
de coral.sistema importe caminho
mostre caminho("pasta", "arquivo.txt")
```

Resultado

Saída esperada

```
pasta/arquivo.txt
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Erros de conversão para componentes incompatíveis com o construtor.

`variavel_ambiente(...)` · módulo `coral.sistema`

O que faz

Lê uma variável do ambiente atual com fallback explícito para ausência.

Assinatura

Assinatura para consulta

```
variavel_ambiente(nome: 'str', padrao: 'str | None' = None) -> 'str | None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome exato da variável de ambiente.
padrao	str | None	não	None	Texto retornado quando a variável não está definida.

Retorno

Texto do ambiente, ou o padrão fornecido.

Exemplo

Código Coral

```
de coral.sistema importe variavel_ambiente
mostre variavel_ambiente("CORAL_LIVRO_AUSENTE_9081", "sem valor")
```

Resultado

Saída esperada

```
sem valor
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Entradas inválidas de nome são rejeitadas pelo ambiente.

`ambiente_selecionado(...)` · módulo `coral.sistema`

O que faz

Copia somente as variáveis presentes cujos nomes foram indicados.

Assinatura

Assinatura para consulta

```
ambiente_selecionado(nomes: 'Iterable[Any]') -> 'dict[str, str]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nomes	Iterable[Any]	sim	sem padrão	Coleção dos nomes a consultar, evitando expor todo o ambiente.

Retorno

Dicionário de nomes e valores presentes; não inclui nomes ausentes.

Exemplo

Código Coral

```
de coral.sistema importe ambiente_selecionado
mostre ambiente_selecionado(["CORAL_LIVRO_AUSENTE_9081"])
```

Resultado

Saída esperada

```
{}
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Nomes incompatíveis podem ser rejeitados pela normalização do ambiente.

`argumentos_programa(...)` · módulo `coral.sistema`

O que faz

Obtém os argumentos adicionais recebidos pelo programa, excluindo o executável.

Assinatura

Assinatura para consulta

```
argumentos_programa() -> 'tuple[str, ...]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Tupla dos argumentos da linha de comando.

Exemplo

Código Coral

```
de coral.sistema importe argumentos_programa
mostre argumentos_programa() == argumentos_programa()
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Não inicia subprocessos nem altera o ambiente.

`programa_disponivel(...)` · módulo `coral.sistema`

O que faz

Verifica se um executável com o nome fornecido é resolvido no ambiente atual.

Assinatura

Assinatura para consulta

```
programa_disponivel(nome: 'str') -> 'bool'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome do executável a procurar no PATH.

Retorno

Booleano verdadeiro quando a busca encontra um programa executável.

Exemplo

Código Coral

```
de coral.sistema importe programa_disponivel
mostre programa_disponivel("coral_programa_que_nao_existe_9081")
```

Resultado

Saída esperada

```
False
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Entrada incompatível pode provocar erro de validação ou do sistema.

`localizar_programa(...)` · módulo `coral.sistema`

O que faz

Procura um programa executável sem iniciá-lo e devolve seu caminho, se existir.

Assinatura

Assinatura para consulta

```
localizar_programa(nome: 'str') -> 'Path | None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome exato do programa procurado.

Retorno

Path do executável localizado ou nulo.

Exemplo

Código Coral

```
de coral.sistema importe localizar_programa
mostre localizar_programa("coral_programa_que_nao_existe_9081")
```

Resultado

Saída esperada

```
None
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Consulta somente o ambiente; nomes inválidos podem ser recusados.

`terminal_interativo(...)` · módulo `coral.sistema`

O que faz

Consulta se um fluxo padrão está conectado a um terminal interativo.

Assinatura

Assinatura para consulta

```
terminal_interativo(canal: 'str') -> 'bool'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
canal	str	sim	sem padrão	Um dos canais suportados de entrada, saída ou erro.

Retorno

Booleano indicativo de TTY para entrada, saída ou erro.

Exemplo

Código Coral

```
de coral.sistema importe terminal_interativo
mostre terminal_interativo("saida")
```

Resultado

Saída esperada

```
False
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Canal não reconhecido é rejeitado com erro de valor.

`remover_variavel_ambiente(...)` · módulo `coral.sistema`

O que faz

Remove uma variável do ambiente do processo quando ela existe.

Assinatura

Assinatura para consulta

```
remover_variavel_ambiente(nome: 'str') -> 'bool'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome da variável a remover do processo atual.

Retorno

Booleano indicando se houve remoção.

Exemplo

Código Coral

```
de coral.sistema importe remover_variavel_ambiente
mostre remover_variavel_ambiente("CORAL_LIVRO_AUSENTE_9081")
```

Resultado

Saída esperada

```
False
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Nome inválido pode ser rejeitado pelo sistema operacional.

`variaveis_ambiente(...)` · módulo `coral.sistema`

O que faz

Produz uma cópia do ambiente corrente sem modificar seus valores.

Assinatura

Assinatura para consulta

```
variaveis_ambiente() -> 'dict[str, str]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Dicionário dos nomes e valores conhecidos do processo.

Exemplo

Código Coral

```
de coral.sistema importe variaveis_ambiente
mostre variaveis_ambiente() == variaveis_ambiente()
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Não expõe um mecanismo de alteração global.

`pasta_atual(...)` · módulo `coral.sistema`

O que faz

Retorna o diretório corrente do processo sem criar novos arquivos.

Assinatura

Assinatura para consulta

```
pasta_atual() -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Path do diretório de trabalho atual.

Exemplo

Código Coral

```
de coral.sistema importe pasta_atual
mostre pasta_atual() == pasta_atual()
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Propaga erros do sistema caso o diretório deixe de existir.

`pasta_temporaria(...)` · módulo `coral.sistema`

O que faz

Consulta o diretório temporário da configuração do processo.

Assinatura

Assinatura para consulta

```
pasta_temporaria() -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Path do diretório temporário sem criar arquivos.

Exemplo

Código Coral

```
de coral.sistema importe pasta_temporaria
mostre pasta_temporaria() == pasta_temporaria()
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Propaga erros na determinação do diretório temporário.

`pasta_usuario(...)` · módulo `coral.sistema`

O que faz

Retorna o diretório pessoal do usuário identificado pelo processo.

Assinatura

Assinatura para consulta

```
pasta_usuario() -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Path do diretório pessoal sem criar arquivos.

Exemplo

Código Coral

```
de coral.sistema importe pasta_usuario
mostre pasta_usuario() == pasta_usuario()
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Pode refletir configurações específicas do sistema e ambiente.

`psutil_disponivel(...)` · módulo `coral.hardware`

O que faz

Verifica se a integração opcional psutil pode ser carregada no ambiente.

Assinatura

Assinatura para consulta

```
psutil_disponivel() -> 'bool'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Booleano de disponibilidade da dependência opcional.

Exemplo

Código Coral

```
de coral.hardware importe psutil_disponivel
mostre psutil_disponivel() == psutil_disponivel()
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Não requer inicialização de sensores nem acesso administrativo.

`serial_disponivel(...)` · módulo `coral.hardware`

O que faz

Verifica a disponibilidade da integração serial opcional.

Assinatura

Assinatura para consulta

```
serial_disponivel() -> 'bool'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Booleano para a dependência serial presente no processo.

Exemplo

Código Coral

```
de coral.hardware importe serial_disponivel
mostre serial_disponivel() == serial_disponivel()
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Não abre portas nem faz transmissão de dados.

`auditar_hardware(...)` · módulo `coral.hardware`

O que faz

Retorna o inventário de contratos de hardware e seus critérios de coesão.

Assinatura

Assinatura para consulta

```
auditar_hardware() -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Dicionário de diagnósticos do módulo Hardware.

Exemplo

Código Coral

```
de coral.hardware importe auditar_hardware
mostre auditar_hardware() == auditar_hardware()
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Não conecta dispositivos nem escreve configurações.

`diagnosticar_hardware(...)` · módulo `coral.hardware`

O que faz

Produz um diagnóstico das capacidades e dependências de hardware observadas.

Assinatura

Assinatura para consulta

```
diagnosticar_hardware() -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Dicionário de capacidades observáveis, sujeito ao ambiente.

Exemplo

Código Coral

```
de coral.hardware importe diagnosticar_hardware
mostre diagnosticar_hardware() == diagnosticar_hardware()
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Dependências ausentes são comunicadas conforme o contrato do diagnóstico.

`semente(...)` · módulo `coral.procedural`

O que faz

Normaliza uma semente portátil em objeto gerador determinístico.

Assinatura

Assinatura para consulta

```
semente(valor: 'Any' = 0) -> 'Semente'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	Any	não	0	Valor determinístico inicial, inclusive texto ou número.

Retorno

Instância Semente com origem explícita.

Exemplo

Código Coral

```
de coral.procedural importe semente
mostre semente(42).inteiro == semente(42).inteiro
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

TypeError para tipos não portáteis e ValueError para decimais não finitos.

`derivar_semente(...)` · módulo `coral.procedural`

O que faz

Deriva uma sub semente determinística a partir do valor e rótulos de domínio.

Assinatura

Assinatura para consulta

```
derivar_semente(valor: 'Any', *rotulos: 'Any') -> 'Semente'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	Any	sim	sem padrão	Semente raiz da geração.
rotulos	Any	não	sem padrão	Rótulos que separam fluxos determinísticos.

Retorno

Instância Semente independente e reproduzível.

Exemplo

Código Coral

```
de coral.procedural importe derivar_semente
mostre derivar_semente(42, "terreno").inteiro == derivar_semente(42, "terreno").inteiro
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

TypeError para rótulos não portáteis ou semente inválida.

`campo_constante(...)` · módulo `coral.procedural`

O que faz

Cria um campo procedural que devolve o mesmo valor finito em todas as coordenadas.

Assinatura

Assinatura para consulta

```
campo_constante(valor: 'float', *, nome: 'str' = 'constante') -> 'CampoProcedural'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	float	sim	sem padrão	Valor constante de cada amostra.
nome	str	não	'constante'	Identificador do campo para fins de documentação e proveniência.

Retorno

Objeto CampoProcedural com valor e metadados de origem.

Exemplo

Código Coral

```
de coral.procedural importe campo_constante
mostre campo_constante(3).valor(1, 2)
```

Resultado

Saída esperada

```
3.0
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

ValueError para entradas não numéricas ou não finitas.

`campo_ruido(...)` · módulo `coral.procedural`

O que faz

Configura campo de ruído de valor suave e determinístico em uma, duas ou três dimensões.

Assinatura

Assinatura para consulta

```
campo_ruido(semente_raiz: 'Any' = 0, *, escala: 'float' = 1.0, oitavas: 'int' = 1,
  persistencia: 'float' = 0.5, lacunaridade: 'float' = 2.0, nome: 'str' = 'ruido') ->
  'CampoProcedural'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
semente_raiz	Any	não	0	Valor raiz para a derivação estável da amostra.
escala	float	não	1.0	Escala espacial do ruído, positiva e finita.
oitavas	int	não	1	Número de níveis de detalhe.
persistencia	float	não	0.5	Razão de amplitude entre oitavas.
lacunaridade	float	não	2.0	Razão de frequência entre oitavas.
nome	str	não	'ruido'	Nome semântico atribuído ao campo.

Retorno

CampoProcedural com parâmetros de ruído validados.

Exemplo

Código Coral

```
de coral.procedural importe campo_ruido
mostre campo_ruido(42).valor(0, 0) == campo_ruido(42).valor(0, 0)
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

ValueError para escala inválida, oitavas ou parâmetros fora do contrato.

`codificar_formulario(...)` · módulo `coral.web.http`

O que faz

Transforma pares de campos em corpo de formulário web percentualmente codificado em UTF 8.

Assinatura

Assinatura para consulta

```
codificar_formulario(campos: 'Any') -> 'bytes'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
campos	Any	sim	sem padrão	Mapeamento ou sequência de pares, mantendo repetições quando aplicável.

Retorno

Bytes no formato application/x-www-form-urlencoded.

Exemplo

Código Coral

```
de coral.web.http importe codificar_formulario
mostre codificar_formulario({"nome": "Ana"})
```

Resultado

Saída esperada

```
b'nome=Ana'
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

ErroFormato para campos com representação incompatível.

`resposta_json(...)` · módulo `coral.web.servidor`

O que faz

Converte um valor JSON para resposta HTTP em memória, sem iniciar servidor.

Assinatura

Assinatura para consulta

```
resposta_json(valor: 'Any', status: 'int' = 200, cabecalhos: 'Any' = None) ->
  'RespostaServidorHTTP'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	Any	sim	sem padrão	Valor serializável em JSON sem perda.
status	int	não	200	Código HTTP válido, padrão 200.
cabecalhos	Any	não	None	Cabeçalhos adicionais, preservando tipos de conteúdo existentes.

Retorno

RespostaServidorHTTP com bytes JSON, status e cabeçalho de tipo.

Exemplo

Código Coral

```
de coral.web.servidor importe resposta_json
mostre resposta_json({"ok": verdadeiro}).status
```

Resultado

Saída esperada

```
200
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

ErroFormato para valores não compatíveis com JSON ou não finitos.

`resposta_bytes(...)` · módulo `coral.web.servidor`

O que faz

Monta resposta HTTP binária com status e tipo de conteúdo declarados.

Assinatura

Assinatura para consulta

```
resposta_bytes(corpo: 'Any', status: 'int' = 200, cabecalhos: 'Any' = None, *,
  tipo_conteudo: 'str' = 'application/octet-stream') -> 'RespostaServidorHTTP'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
corpo	Any	sim	sem padrão	Conteúdo aceito pelo normalizador binário.
status	int	não	200	Código de resposta HTTP válido.
cabecalhos	Any	não	None	Cabeçalhos HTTP adicionais.
tipo_conteudo	str	não	'application/octet-stream'	Valor de Content Type quando não fornecido nos cabeçalhos.

Retorno

RespostaServidorHTTP com bytes imutáveis e cabeçalhos normalizados.

Exemplo

Código Coral

```
de coral.web.servidor importe resposta_bytes
mostre resposta_bytes(bytes([97, 98, 99])).status
```

Resultado

Saída esperada

```
200
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

ErroValor para status HTTP inválido e ErroFormato para corpo incompatível.

`normalizar_valor(...)` · **módulo** `coral.inspecao`

O que faz

Converte dados de inspeção para uma estrutura JSON segura com limites de profundidade e coleção.

Assinatura

Assinatura para consulta

```
normalizar_valor(valor: 'Any', *, profundidade: 'int' = 7, limite_colecao: 'int' = 100,
  _vistos: 'set[int] | None' = None) -> 'Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	Any	sim	sem padrão	Objeto a reduzir a estrutura segura.
profundidade	int	não	7	Limite de aninhamento antes de truncar.
limite_colecao	int	não	100	Tamanho máximo de cada coleção.
_vistos	set[int] | None	não	None	Controle interno de detecção de ciclos, normalmente omitido.

Retorno

Valor JSON representável, com marcadores de truncamento quando exigidos.

Exemplo

Código Coral

```
de coral.inspecao importe normalizar_valor
mostre normalizar_valor([1, 2])
```

Resultado

Saída esperada

```
[1, 2]
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Objetos desconhecidos recebem representação textual conservadora.

`inspecionar(...)` · módulo `coral.inspecao`

O que faz

Constrói um snapshot normalizado e de somente leitura de um objeto observável.

Assinatura

Assinatura para consulta

```
inspecionar(objeto: 'Any', *, nome: 'str | None' = None, tipo: 'str | None' = None) ->
  'SnapshotInspecao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto a inspecionar sem mutação.
nome	str | None	não	None	Nome explícito opcional para o snapshot.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tipo	str | None	não	None	Tipo explícito opcional para o snapshot.

Retorno

SnapshotInspecao com tipo, nome, dados e avisos.

Exemplo

Código Coral

```
de coral.inspecao importe inspecionar
mostre inspecionar(10).tipo
```

Resultado

Saída esperada

```
int
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Objetos de domínio devem expor estado_inspecao para fornecer campos próprios.

`inspecionar_dict(...)` · módulo `coral.inspecao`

O que faz

Expõe o snapshot como dicionário JSON seguro sem criar modelo paralelo.

Assinatura

Assinatura para consulta

```
inspecionar_dict(objeto: 'Any', **opcoes: 'Any') -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto cujo snapshot será transformado em dicionário.
opcoes	Any	não	sem padrão	Opções nomeadas repassadas à inspeção.

Retorno

Dicionário de campos do snapshot de inspeção.

Exemplo

Código Coral

```
de coral.inspecao importe inspecionar_dict
mostre inspecionar_dict(10)["tipo"]
```

Resultado

Saída esperada

```
int
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Opções adicionais seguem o contrato de inspecionar.

`consultar_inspecao(...)` · módulo `coral.inspecao`

O que faz

Consulta um caminho pontuado ou com índices dentro dos dados normalizados de um snapshot.

Assinatura

Assinatura para consulta

```
consultar_inspecao(objeto: 'Any', caminho: 'str', padrao: 'Any' = <ausente>) -> 'Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto inspecionável ou snapshot normalizado.
caminho	str	sim	sem padrão	Expressão do caminho dentro do campo dados.
padrao	Any	não	<ausente>	Valor devolvido se o caminho não existir; omissão gera erro.

Retorno

Valor selecionado, ou padrão quando o caminho está ausente.

Exemplo

Código Coral

```
de coral.inspecao importe consultar_inspecao
mostre consultar_inspecao({"pontos": 3}, "valor.pontos", 0)
```

Resultado

Saída esperada

```
3
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

KeyError para caminho inexistente sem padrão explícito.

`selecionar_inspecao(...)` · módulo `coral.inspecao`

O que faz

Extrai vários caminhos de inspeção em ordem e devolve um dicionário.

Assinatura

Assinatura para consulta

```
selecionar_inspecao(objeto: 'Any', caminhos: 'list[str] | tuple[str, ...]') ->
  'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto inspecionável ou snapshot existente.
caminhos	list[str] | tuple[str, ...]	sim	sem padrão	Sequência ordenada dos caminhos a consultar.

Retorno

Mapeamento de caminhos para valores encontrados.

Exemplo

Código Coral

```
de coral.inspecao importe selecionar_inspecao
mostre selecionar_inspecao({"pontos": 3}, ["valor.pontos"])
```

Resultado

Saída esperada

```
{'valor.pontos': 3}
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

KeyError se algum caminho solicitado não existir.

`visoes_inspecao(...)` · módulo `coral.inspecao`

O que faz

Lista as visões semânticas disponíveis para determinado objeto de inspeção.

Assinatura

Assinatura para consulta

```
visoes_inspecao(objeto: 'Any') -> 'tuple[str, ...]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto a partir do qual serão inferidas as visões disponíveis.

Retorno

Tupla de identificadores de visão reconhecidos para o objeto.

Exemplo

Código Coral

```
de coral.inspecao importe visoes_inspecao
mostre visoes_inspecao(10)
```

Resultado

Saída esperada

```
()
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Não cria nem modifica visões no objeto original.

`DiagnosticoInspecao(...)` · módulo `coral.inspecao`

O que faz

Conserva o resultado estrutural da validação do contrato de inspeção, com mensagens e visões disponíveis.

Construtor

Assinatura para consulta

```
DiagnosticoInspecao(ok: 'bool', tipo: 'str', nome: 'str | None', assinatura: 'str |
  None', visoes: 'tuple[str, ...]' = (), avisos: 'tuple[str, ...]' = (), erro: 'str |
  None' = None, contrato: 'str' = 'coral.inspecao.diagnostico/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
ok	bool	sim	sem padrão	Resultado da validação do snapshot.
tipo	str	sim	sem padrão	Tipo registrado para o alvo.
nome	str | None	sim	sem padrão	Nome opcional do alvo.
assinatura	str | None	sim	sem padrão	Assinatura estável opcional do snapshot.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
visoes	tuple[str, ...]	não	()	Visões semânticas disponíveis.
avisos	tuple[str, ...]	não	()	Mensagens não fatais da inspeção.
erro	str | None	não	None	Erro de validação quando houver.
contrato	str	não	'coral.inspecao.diagnostico/1'	Identificador de versão do contrato do diagnóstico.

Retorno

Instância imutável de DiagnosticoInspecao.

Exemplo

Código Coral

```
de coral.inspecao importe DiagnosticoInspecao
mostre DiagnosticoInspecao(verdadeiro, "numero", nulo, nulo).ok
```

Resultado

Saída esperada

```
True
```

Erros

O construtor mantém os campos recebidos; dados de tipo incorreto podem falhar ao consumir ou serializar.

`DiferencaInspecao(...)` · módulo `coral.inspecao`

O que faz

Agrupa os hashes ou identificadores de dois snapshots e suas mudanças estruturadas.

Construtor

Assinatura para consulta

```
DiferencaInspecao(antes: 'str', depois: 'str', mudancas: 'tuple[MudancaInspecao, ...]' =
   (), contrato: 'str' = 'coral.inspecao.diferenca/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
antes	str	sim	sem padrão	Assinatura do estado anterior.
depois	str	sim	sem padrão	Assinatura do estado posterior.
mudancas	tuple[MudancaInspecao, ...]	não	()	Tupla de mudanças individuais encontradas.
contrato	str	não	'coral.inspecao.diferenca/1'	Identificador versionado do relatório de diferenças.

Retorno

Instância de DiferencaInspecao.

Exemplo

Código Coral

```
de coral.inspecao importe DiferencaInspecao
mostre DiferencaInspecao("antes", "depois").antes
```

Resultado

Saída esperada

```
antes
```

Erros

As mudanças devem obedecer ao formato do relatório para os consumidores posteriores.

`EventoObservabilidade(...)` · módulo `coral.inspecao`

O que faz

Representa um evento de observabilidade independente de editor, depurador e transporte.

Construtor

Assinatura para consulta

```
EventoObservabilidade(sequencia: 'int', tipo: 'str', alvo: 'str | None' = None,
  instante: 'float | None' = None, caminho: 'str | None' = None, valor: 'Any' = None,
  anterior: 'Any' = None, detalhes: 'Mapping[str, Any]' = <factory>, contrato: 'str' =
  'coral.inspecao.evento/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
sequencia	int	sim	sem padrão	Número de ordem da ocorrência.
tipo	str	sim	sem padrão	Identificador semântico da ocorrência.
alvo	str | None	não	None	Alvo opcional do evento.
instante	float | None	não	None	Marca temporal opcional.
caminho	str | None	não	None	Caminho do dado observado.
valor	Any	não	None	Novo valor observado.
anterior	Any	não	None	Valor anterior à mudança.
detalhes	Mapping[str, Any]	não	<factory>	Dados complementares portáteis.
contrato	str	não	'coral.inspecao.evento/1'	Versão do contrato de evento.

Retorno

EventoObservabilidade com sequência, tipo e contexto do evento.

Exemplo

Código Coral

```
de coral.inspecao importe EventoObservabilidade
mostre EventoObservabilidade(1, "alteracao").tipo
```

Resultado

Saída esperada

```
alteracao
```

Erros

O evento é transportado como dados; consumidores devem validar o significado do tipo e detalhes.

`ExplicacaoRegra(...)` · módulo `coral.inspecao`

O que faz

Descreve o último estado observável de uma regra por meio de dados estruturados e mensagem legível.

Construtor

Assinatura para consulta

```
ExplicacaoRegra(nome: 'str | None', estado: 'str', mensagem: 'str', dados: 'Mapping[str,
   Any]' = <factory>, contrato: 'str' = 'coral.inspecao.explicacao_regra/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str | None	sim	sem padrão	Identificador opcional da regra.
estado	str	sim	sem padrão	Estado normalizado da regra.
mensagem	str	sim	sem padrão	Explicação textual legível.
dados	Mapping[str, Any]	não	<factory>	Dados adicionais associados ao estado.
contrato	str	não	'coral.inspecao.explicacao_regra/1'	Versão do contrato de explicação.

Retorno

ExplicacaoRegra com nome, estado, mensagem e dados.

Exemplo

Código Coral

```
de coral.inspecao importe ExplicacaoRegra
mostre ExplicacaoRegra(nulo, "ativa", "aprovada").estado
```

Resultado

Saída esperada

```
ativa
```

Erros

Dados extras devem manter representação portátil quando serializados.

`MudancaInspecao(...)` · módulo `coral.inspecao`

O que faz

Identifica uma alteração estrutural em um caminho observado, incluindo valores anterior e posterior.

Construtor

Assinatura para consulta

```
MudancaInspecao(caminho: 'str', tipo: 'str', antes: 'Any' = None, depois: 'Any' = None)
  -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	str	sim	sem padrão	Caminho dentro de dados do snapshot.
tipo	str	sim	sem padrão	Natureza da alteração.
antes	Any	não	None	Valor antes da ocorrência.
depois	Any	não	None	Valor após a ocorrência.

Retorno

MudancaInspecao com tipo de alteração e caminho.

Exemplo

Código Coral

```
de coral.inspecao importe MudancaInspecao
mostre MudancaInspecao("pontos", "alterado").caminho
```

Resultado

Saída esperada

```
pontos
```

Erros

O construtor não calcula diferenças; caminhos precisam seguir o contrato de consulta na interpretação.

`RelatorioInspecao(...)` · módulo `coral.inspecao`

O que faz

Conserva o relatório de apresentação derivado de um snapshot de inspeção existente.

Construtor

Assinatura para consulta

```
RelatorioInspecao(tipo: 'str', nome: 'str | None', dados: 'Mapping[str, Any]', visao:
  'str | None' = None, assinatura_origem: 'str | None' = None, contrato: 'str' =
  'coral.inspecao.relatorio/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tipo	str	sim	sem padrão	Tipo do objeto resumido.
nome	str | None	sim	sem padrão	Nome opcional do alvo.
dados	Mapping[str, Any]	sim	sem padrão	Mapeamento dos dados apresentados.
visao	str | None	não	None	Identificador da visão selecionada.
assinatura_origem	str | None	não	None	Assinatura do snapshot original.
contrato	str	não	'coral.inspecao.relatorio/1'	Identificador versionado de relatório.

Retorno

RelatorioInspecao com valores e metadados de origem.

Exemplo

Código Coral

```
de coral.inspecao importe RelatorioInspecao
mostre RelatorioInspecao("int", nulo, {}).tipo
```

Resultado

Saída esperada

```
int
```

Erros

A semântica do conteúdo depende da visão usada na geração do relatório.

`RespostaInspecao(...)` · módulo `coral.inspecao`

O que faz

Representa a resposta de um comando de inspeção com resultado ou erro explícito.

Construtor

Assinatura para consulta

```
RespostaInspecao(operacao: 'str', ok: 'bool', resultado: 'Any' = None, erro: 'str |
  None' = None, contrato: 'str' = 'coral.inspecao.resposta/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
operacao	str	sim	sem padrão	Nome do comando solicitado.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
ok	bool	sim	sem padrão	Indicador de conclusão com sucesso.
resultado	Any	não	None	Conteúdo retornado pelo comando.
erro	str | None	não	None	Mensagem de erro, quando aplicável.
contrato	str	não	'coral.inspecao.resposta/1'	Versão do contrato de resposta.

Retorno

RespostaInspecao com indicador de sucesso e conteúdo da operação.

Exemplo

Código Coral

```
de coral.inspecao importe RespostaInspecao
mostre RespostaInspecao("consultar", verdadeiro).operacao
```

Resultado

Saída esperada

```
consultar
```

Erros

O resultado pode exigir validação de portabilidade para transporte entre processos.

`SnapshotInspecao(...)` · módulo `coral.inspecao`

O que faz

Registra o estado observável de um objeto sem conceder mutabilidade ou acesso a detalhes privados do domínio.

Construtor

Assinatura para consulta

```
SnapshotInspecao(tipo: 'str', nome: 'str | None', dados: 'Mapping[str, Any]' =
  <factory>, avisos: 'tuple[str, ...]' = (), contrato: 'str' = 'coral.inspecao/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tipo	str	sim	sem padrão	Identificador semântico do objeto.
nome	str | None	sim	sem padrão	Nome opcional de apresentação.
dados	Mapping[str, Any]	não	<factory>	Mapeamento do estado observado.
avisos	tuple[str, ...]	não	()	Avisos da coleta.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
contrato	str	não	'coral.inspecao/1'	Versão de snapshot reconhecida pelos consumidores.

Retorno

SnapshotInspecao com dados normalizáveis em JSON.

Exemplo

Código Coral

```
de coral.inspecao importe SnapshotInspecao
mostre SnapshotInspecao("inteiro", nulo).tipo
```

Resultado

Saída esperada

```
inteiro
```

Erros

Dados arbitrários exigem normalização antes do transporte e da persistência.

`VisaoInspecao(...)` · módulo `coral.inspecao`

O que faz

Armazena uma projeção semântica de um snapshot, sem modificar o estado original.

Construtor

Assinatura para consulta

```
VisaoInspecao(visao: 'str', tipo: 'str', nome: 'str | None', dados: 'Mapping[str, Any]'
  = <factory>, assinatura_origem: 'str | None' = None, avisos: 'tuple[str, ...]' = (),
  contrato: 'str' = 'coral.inspecao.visao/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
visao	str	sim	sem padrão	Nome da representação semântica.
tipo	str	sim	sem padrão	Tipo de objeto representado.
nome	str | None	sim	sem padrão	Nome opcional do objeto.
dados	Mapping[str, Any]	não	<factory>	Dados específicos da visão.
assinatura_origem	str | None	não	None	Hash de origem para detectar mudanças.
avisos	tuple[str, ...]	não	()	Avisos de projeção.
contrato	str	não	'coral.inspecao.visao/1'	Versão do contrato de visão.

Retorno

VisaoInspecao com rótulo, tipo e dados filtrados.

Exemplo

Código Coral

```
de coral.inspecao importe VisaoInspecao
mostre VisaoInspecao("resumo", "inteiro", nulo).visao
```

Resultado

Saída esperada

```
resumo
```

Erros

A assinatura da origem é apenas referência e não autoriza mutação no objeto.

`VigiaInspecao(...)` · módulo `coral.inspecao`

O que faz

Define uma condição de watchpoint de leitura sobre caminho do contrato de inspeção.

Construtor

Assinatura para consulta

```
VigiaInspecao(nome: 'str', caminho: 'str', *, operador: 'str' = 'mudou', valor: 'Any' =
  None, ativo: 'bool' = True) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Identificador da vigia.
caminho	str	sim	sem padrão	Caminho observado na estrutura dados.
operador	str	não	'mudou'	Operação de comparação ou detecção de mudança.
valor	Any	não	None	Valor de referência quando exigido pelo operador.
ativo	bool	não	True	Indica se a condição participa das avaliações.

Retorno

VigiaInspecao com operador e valor de referência.

Exemplo

Código Coral

```
de coral.inspecao importe VigiaInspecao
mostre VigiaInspecao("limite", "pontos").nome
```

Resultado

Saída esperada

```
limite
```

Erros

O caminho e o operador são validados durante a avaliação da vigia.

`VersaoSemantica(...)` · módulo `coral`

O que faz

Representa uma versão semântica por três componentes inteiros, permitindo operações explícitas de versão.

Construtor

Assinatura para consulta

```
VersaoSemantica(major: 'int', minor: 'int', patch: 'int') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
major	int	sim	sem padrão	Versão principal incompatível.
minor	int	sim	sem padrão	Versão secundária.
patch	int	sim	sem padrão	Revisão corretiva.

Retorno

VersaoSemantica com major, minor e patch.

Exemplo

Código Coral

```
de coral importe VersaoSemantica
mostre VersaoSemantica(1, 8, 0).minor
```

Resultado

Saída esperada

```
8
```

Erros

A coerência dos valores deve ser preservada ao comparar versões.

`DiagnosticoEstruturado(...)` · módulo `coral.api`

O que faz

Transporta um diagnóstico técnico com código, severidade, localização e sugestão opcional.

Construtor

Assinatura para consulta

```
DiagnosticoEstruturado(codigo: 'str', nivel: 'str', mensagem: 'str', linha: 'int | None'
   = None, coluna: 'int | None' = None, sugestao: 'str | None' = None, origem: 'str' =
  'coral', trecho: 'str | None' = None, categoria: 'str' = 'codigo', detalhe:
  'Mapping[str, Any]' = <factory>) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
codigo	str	sim	sem padrão	Identificador estável do diagnóstico.
nivel	str	sim	sem padrão	Grau de severidade.
mensagem	str	sim	sem padrão	Descrição legível do problema.
linha	int | None	não	None	Linha opcional de origem.
coluna	int | None	não	None	Coluna opcional da ocorrência.
sugestao	str | None	não	None	Proposta opcional de correção.
origem	str	não	'coral'	Componente emissor.
trecho	str | None	não	None	Fragmento de origem associado.
categoria	str	não	'codigo'	Categoria para agrupamento.
detalhe	Mapping[str, Any]	não	<factory>	Metadados adicionais do diagnóstico.

Retorno

DiagnosticoEstruturado tipado para ferramentas de desenvolvimento.

Exemplo

Código Coral

```
de coral.api importe DiagnosticoEstruturado
mostre DiagnosticoEstruturado("T100", "erro", "tipo invalido").codigo
```

Resultado

Saída esperada

```
T100
```

Erros

Campos opcionais podem ser nulos; consumidores devem validar offsets contra a fonte.

`ResultadoAnalise(...)` · **módulo** `coral.api`

O que faz

Agrupa diagnósticos de uma análise para consumo por CLI, testes e ferramentas.

Construtor

Assinatura para consulta

```
ResultadoAnalise(diagnosticos: 'tuple[DiagnosticoEstruturado, ...]') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
diagnosticos	tuple[DiagnosticoEstruturado, ...]	sim	sem padrão	Tupla de diagnósticos produzidos pela análise.

Retorno

ResultadoAnalise com sequência de diagnósticos.

Exemplo

Código Coral

```
de coral.api importe ResultadoAnalise
mostre ResultadoAnalise(()).diagnosticos
```

Resultado

Saída esperada

```
()
```

Erros

Os itens fornecidos devem seguir o contrato de DiagnosticoEstruturado.

`ResultadoCompilacao(...)` · módulo `coral.api`

O que faz

Mantém código Python traduzido e indicadores de cobertura do backend com mapa de linhas.

Construtor

Assinatura para consulta

```
ResultadoCompilacao(codigo: 'str', mapa_linhas: 'dict[int, int]' = <factory>,
  folhas_legadas: 'int' = 0, blocos_estruturais: 'int' = 0, folhas_genericas: 'int' = 0,
  blocos_genericos: 'int' = 0) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
codigo	str	sim	sem padrão	Código de saída gerado pelo backend.
mapa_linhas	dict[int, int]	não	<factory>	Mapeamento entre linhas de origem e saída.
folhas_legadas	int	não	0	Quantidade de folhas traduzidas pelo caminho legado.
blocos_estruturais	int	não	0	Quantidade de blocos gerados estruturalmente.
folhas_genericas	int	não	0	Quantidade de folhas tratadas pela rota genérica.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
blocos_genericos	int	não	0	Quantidade de blocos tratados por fallback genérico.

Retorno

ResultadoCompilacao contendo código, mapa e contadores.

Exemplo

Código Coral

```
de coral.api importe ResultadoCompilacao
mostre ResultadoCompilacao("print(3)").codigo
```

Resultado

Saída esperada

```
print(3)
```

Erros

Contadores apenas descrevem a tradução, não garantem ausência de erros em execução.

`Grafico(...)` · módulo `coral.graficos`

O que faz

Especificação imutável de gráfico com tipo, séries, escala e metadados de apresentação.

Construtor

Assinatura para consulta

```
Grafico(tipo: 'str', elementos: 'tuple[Any, ...]', titulo: 'str' = '', rotulo_x: 'str' =
   '', rotulo_y: 'str' = '', rotulo_z: 'str' = '', unidade_x: 'str' = '', unidade_y: 'str'
   = '', unidade_z: 'str' = '', legenda: 'bool' = True, tema: 'str' = 'claro',
  proveniencia: 'tuple[str, ...]' = (), texto_alternativo: 'str' = '', avisos: 'tuple[str,
   ...]' = ()) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tipo	str	sim	sem padrão	Identificador do tipo de gráfico.
elementos	tuple[Any, ...]	sim	sem padrão	Tupla de séries e elementos do gráfico.
titulo	str	não	''	Título do gráfico.
rotulo_x	str	não	''	Legenda do eixo horizontal.
rotulo_y	str	não	''	Legenda do eixo vertical.
rotulo_z	str	não	''	Legenda do eixo de profundidade.
unidade_x	str	não	''	Unidade do eixo X.
unidade_y	str	não	''	Unidade do eixo Y.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
unidade_z	str	não	''	Unidade do eixo Z.
legenda	bool	não	True	Habilita a indicação das séries.
tema	str	não	'claro'	Tema visual esperado pelo renderizador.
proveniencia	tuple[str, ...]	não	()	Origens dos dados do gráfico.
texto_alternativo	str	não	''	Descrição acessível do gráfico.
avisos	tuple[str, ...]	não	()	Mensagens relevantes para representação.

Retorno

Grafico declarativo, sem abrir janelas ou renderizar automaticamente.

Exemplo

Código Coral

```
de coral.graficos importe Grafico
mostre Grafico("linha", ()).tipo
```

Resultado

Saída esperada

```
linha
```

Erros

Renderizadores podem recusar tipos e escalas incompatíveis com a projeção escolhida.

`FaixaIncerteza(...)` · módulo `coral.graficos`

O que faz

Agrupa valores centrais e limites inferiores e superiores explicitamente calculados para cada ponto.

Construtor

Assinatura para consulta

```
FaixaIncerteza(x: 'tuple[float, ...]', centro: 'tuple[float, ...]', inferior:
  'tuple[float, ...]', superior: 'tuple[float, ...]', nome: 'str | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	tuple[float, ...]	sim	sem padrão	Eixo das posições amostradas.
centro	tuple[float, ...]	sim	sem padrão	Valores centrais.
inferior	tuple[float, ...]	sim	sem padrão	Limites inferiores absolutos.
superior	tuple[float, ...]	sim	sem padrão	Limites superiores absolutos.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str | None	não	None	Rótulo opcional da faixa.

Retorno

FaixaIncerteza com quatro séries alinhadas.

Exemplo

Código Coral

```
de coral.graficos importe FaixaIncerteza
mostre FaixaIncerteza((1,), (2,), (1,), (3,)).centro
```

Resultado

Saída esperada

```
(2,)
```

Erros

Os vetores devem ter dimensões compatíveis e limites coerentes para representação.

`Serie2D(...)` · módulo `coral.graficos`

O que faz

Conserva pontos cartesianos para gráfico 2D, incluindo lacunas de amostragem explicitadas.

Construtor

Assinatura para consulta

```
Serie2D(estilo: 'str', x: 'tuple[float, ...]', y: 'tuple[float | None, ...]', nome: 'str
   | None' = None, origem: 'str' = 'direta') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
estilo	str	sim	sem padrão	Estilo de apresentação da série.
x	tuple[float, ...]	sim	sem padrão	Coordenadas horizontais ordenadas.
y	tuple[float | None, ...]	sim	sem padrão	Valores verticais, admitindo lacunas explícitas quando aplicável.
nome	str | None	não	None	Rótulo de série.
origem	str	não	'direta'	Marca a proveniência da amostragem.

Retorno

Serie2D com estilo e coordenadas X e Y.

Exemplo

Código Coral

```
de coral.graficos importe Serie2D
mostre Serie2D("linha", (0, 1), (2, 3)).estilo
```

Resultado

Saída esperada

```
linha
```

Erros

Os vetores devem ter comprimentos compatíveis para plotagem.

`Serie3D(...)` · módulo `coral.graficos`

O que faz

Conserva amostras de coordenadas X, Y e Z sem projeção implícita.

Construtor

Assinatura para consulta

```
Serie3D(estilo: 'str', x: 'tuple[float, ...]', y: 'tuple[float, ...]', z: 'tuple[float,
  ...]', nome: 'str | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
estilo	str	sim	sem padrão	Estilo de visualização em três dimensões.
x	tuple[float, ...]	sim	sem padrão	Valores do eixo X.
y	tuple[float, ...]	sim	sem padrão	Valores do eixo Y.
z	tuple[float, ...]	sim	sem padrão	Valores do eixo Z.
nome	str | None	não	None	Identificação opcional da série.

Retorno

Serie3D com três eixos de igual comprimento.

Exemplo

Código Coral

```
de coral.graficos importe Serie3D
mostre Serie3D("pontos", (0,), (1,), (2,)).estilo
```

Resultado

Saída esperada

```
pontos
```

Erros

A renderização deve validar dimensões e finitude das coordenadas.

`SeriesRegistros(...)` · módulo `coral.graficos`

O que faz

Armazena duas sequências numéricas alinhadas extraídas de registros comuns.

Construtor

Assinatura para consulta

```
SeriesRegistros(x: 'tuple[float, ...]', y: 'tuple[float, ...]', campo_x: 'str', campo_y:
    'str') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	tuple[float, ...]	sim	sem padrão	Valores lidos do primeiro campo.
y	tuple[float, ...]	sim	sem padrão	Valores correspondentes do segundo campo.
campo_x	str	sim	sem padrão	Nome do campo horizontal.
campo_y	str	sim	sem padrão	Nome do campo vertical.

Retorno

SeriesRegistros com eixos e nomes de campo utilizados.

Exemplo

Código Coral

```
de coral.graficos importe SeriesRegistros
mostre SeriesRegistros((1,), (3,), "ano", "valor").campo_x
```

Resultado

Saída esperada

```
ano
```

Erros

Campos ausentes ou registros incompatíveis devem ser recusados na extração.

`Bateria(...)` · módulo `coral.hardware`

O que faz

Representa o estado conhecido da bateria, incluindo casos em que medição ou tempo são desconhecidos.

Construtor

Assinatura para consulta

```
Bateria(percentual: 'float | None', conectada: 'bool | None', segundos_restantes: 'int |
    None') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
percentual	float | None	sim	sem padrão	Porcentagem observada de carga, quando disponível.
conectada	bool | None	sim	sem padrão	Estado da alimentação externa ou nulo.
segundos_restantes	int | None	sim	sem padrão	Estimativa de autonomia, se informada.

Retorno

Bateria com nível, conexão e autonomia opcional.

Exemplo

Código Coral

```
de coral.hardware importe Bateria
mostre Bateria(75, verdadeiro, nulo).percentual
```

Resultado

Saída esperada

```
75
```

Erros

Os valores refletem uma medição; não implicam que uma bateria esteja presente.

`CPU(...)` · módulo `coral.hardware`

O que faz

Agrupa identificação e informações observáveis de núcleos e frequência da CPU.

Construtor

Assinatura para consulta

```
CPU(nome: 'str', nucleos_logicos: 'int | None', nucleos_fisicos: 'int | None',
  frequencia_mhz: 'float | None') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome exposto pela plataforma.
nucleos_logicos	int | None	sim	sem padrão	Quantidade lógica detectada.
nucleos_fisicos	int | None	sim	sem padrão	Quantidade física detectada.
frequencia_mhz	float | None	sim	sem padrão	Frequência observada quando a plataforma fornece valor.

Retorno

CPU com nome e métricas opcionais.

Exemplo

Código Coral

```
de coral.hardware importe CPU
mostre CPU("processador", 4, 2, nulo).nome
```

Resultado

Saída esperada

```
processador
```

Erros

As métricas podem estar ausentes segundo o SO e as permissões de consulta.

`Semente(...)` · módulo `coral.procedural`

O que faz

Conserva uma semente portátil que pode derivar fluxos independentes por contexto.

Construtor

Assinatura para consulta

```
Semente(valor: 'Any' = 0) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	Any	não	0	Valor portátil usado como raiz do gerador.

Retorno

Semente imutável capaz de produzir inteiro derivado e gerador de eventos.

Exemplo

Código Coral

```
de coral.procedural importe Semente
mostre Semente(42).valor
```

Resultado

Saída esperada

```
42
```

Erros

TypeError para tipo não portátil, ValueError para decimal não finito.

`GeradorProcedural(...)` · módulo `coral.procedural`

O que faz

Expõe geração sem estado, derivada da semente e de um namespace escolhido.

Construtor

Assinatura para consulta

```
GeradorProcedural(semente_raiz: 'Any' = 0, *, namespace: 'str' = 'gerador') -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
semente_raiz	Any	não	0	Semente raiz do domínio.
namespace	str	não	'gerador'	Identificador de espaço de geração independente.

Retorno

GeradorProcedural que pode obter números e amostras por chaves.

Exemplo

Código Coral

```
de coral.procedural importe GeradorProcedural
mostre GeradorProcedural(42).namespace
```

Resultado

Saída esperada

```
gerador
```

Erros

Parâmetros das operações numéricas devem ser finitos e limites válidos.

`TipoCoral(...)` · módulo `coral.tipos`

O que faz

Representa um nome de tipo da Coral, seus tipos correspondentes e nomes alternativos.

Construtor

Assinatura para consulta

```
TipoCoral(nome: 'str', tipos_python: 'tuple[type, ...]' = (), aliases: 'tuple[str, ...]'
    = ()) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome canônico do tipo.
tipos_python	tuple[type, ...]	não	()	Tupla de classes Python associadas ao tipo.
aliases	tuple[str, ...]	não	()	Denominações equivalentes aceitas.

Retorno

TipoCoral com operação aceita para teste de pertinência.

Exemplo

Código Coral

```
de coral.tipos importe TipoCoral
mostre TipoCoral("inteiro").nome
```

Resultado

Saída esperada

```
inteiro
```

Erros

Tipos não reconhecidos não passam a ser válidos só por construir a descrição.

`Rolagem(...)` · módulo `coral.rpg`

O que faz

Guarda a expressão de dados, resultados individuais e modificador de uma rolagem concluída.

Construtor

Assinatura para consulta

```
Rolagem(expressao: 'str', dados: 'tuple[int, ...]', modificador: 'int' = 0) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
expressao	str	sim	sem padrão	Expressão original de dados.
dados	tuple[int, ...]	sim	sem padrão	Tupla dos resultados de cada dado.
modificador	int	não	0	Bônus ou penalidade somado à rolagem.

Retorno

Rolagem com os dados registrados e modificador aplicado.

Exemplo

Código Coral

```
de coral.rpg importe Rolagem
mostre Rolagem("1d6", (4,)).expressao
```

Resultado

Saída esperada

```
1d6
```

Erros

Não gera números automaticamente; a função rolar fornece a operação aleatória.

`ResultadoComando(...)` · módulo `coral.sistema`

O que faz

Agrupa argumentos, código de retorno, saídas capturadas e duração de um subprocesso.

Construtor

Assinatura para consulta

```
ResultadoComando(comando: 'tuple[str, ...]', codigo: 'int', saida: 'str', erro: 'str',
  duracao_segundos: 'float', saida_truncada: 'bool' = False, erro_truncado: 'bool' =
  False) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
comando	tuple[str, ...]	sim	sem padrão	Sequência de argumentos do processo.
codigo	int	sim	sem padrão	Código de saída do processo.
saida	str	sim	sem padrão	Texto capturado do stdout.
erro	str	sim	sem padrão	Texto capturado do stderr.
duracao_segundos	float	sim	sem padrão	Tempo medido de execução.
saida_truncada	bool	não	False	Indica corte por limite de stdout.
erro_truncado	bool	não	False	Indica corte por limite de stderr.

Retorno

ResultadoComando com metadados de limites de captura.

Exemplo

Código Coral

```
de coral.sistema importe ResultadoComando
mostre ResultadoComando(("echo",), 0, "ok", "", 0.1).codigo
```

Resultado

Saída esperada

```
0
```

Erros

Construir o resultado não executa comandos e não valida previamente o sistema.

`RespostaServidorHTTP(...)` · módulo `coral.web.servidor`

O que faz

Representa resposta HTTP já materializada, incluindo status, cabeçalhos e corpo binário.

Construtor

Assinatura para consulta

```
RespostaServidorHTTP(status: 'int', cabecalhos: 'tuple[tuple[str, str], ...]',
  corpo_bytes: 'bytes') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
status	int	sim	sem padrão	Código de status HTTP.
cabecalhos	tuple[tuple[str, str], ...]	sim	sem padrão	Sequência de pares nome e valor de cabeçalhos.
corpo_bytes	bytes	sim	sem padrão	Conteúdo da resposta em bytes.

Retorno

RespostaServidorHTTP pronta para envio por servidor explícito.

Exemplo

Código Coral

```
de coral.web.servidor importe RespostaServidorHTTP
mostre RespostaServidorHTTP(200, (), bytes([65])).status
```

Resultado

Saída esperada

```
200
```

Erros

O objeto não aplica validações de roteamento ou inicia a rede.

`RequisicaoHTTP(...)` · módulo `coral.web.servidor`

O que faz

Conserva uma requisição HTTP recebida e normalizada antes de chamar a rota.

Construtor

Assinatura para consulta

```
RequisicaoHTTP(metodo: 'str', caminho: 'str', consulta: 'tuple[tuple[str, str], ...]',
  cabecalhos: 'tuple[tuple[str, str], ...]', corpo_bytes: 'bytes', cliente: 'str') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
metodo	str	sim	sem padrão	Método HTTP recebido.
caminho	str	sim	sem padrão	Caminho solicitado.
consulta	tuple[tuple[str, str], ...]	sim	sem padrão	Parâmetros da query em pares.
cabecalhos	tuple[tuple[str, str], ...]	sim	sem padrão	Cabeçalhos da requisição.
corpo_bytes	bytes	sim	sem padrão	Corpo recebido em bytes.
cliente	str	sim	sem padrão	Identificação textual do cliente.

Retorno

RequisicaoHTTP com método, caminho, consulta, cabeçalhos, corpo e cliente.

Exemplo

Código Coral

```
de coral.web.servidor importe RequisicaoHTTP
mostre RequisicaoHTTP("GET", "/", (), (), bytes([]), "local").metodo
```

Resultado

Saída esperada

```
GET
```

Erros

O parsing e os limites de tamanho devem ser validados pelo servidor antes da construção.

`RespostaHTTP(...)` · módulo `coral.web.http`

O que faz

Preserva resposta HTTP cliente com status, cabeçalhos, corpo original e URL efetiva.

Construtor

Assinatura para consulta

```
RespostaHTTP(status: 'int', cabecalhos: 'tuple[tuple[str, str], ...]', corpo_bytes:
  'bytes', url: 'str', metodo: 'str', redirecionamentos: 'int' = 0) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
status	int	sim	sem padrão	Código HTTP retornado.
cabecalhos	tuple[tuple[str, str], ...]	sim	sem padrão	Pares de cabeçalho retornados.
corpo_bytes	bytes	sim	sem padrão	Conteúdo bruto recebido.
url	str	sim	sem padrão	URL efetiva da resposta.
metodo	str	sim	sem padrão	Método empregado.
redirecionamentos	int	não	0	Quantidade de redirecionamentos seguidos.

Retorno

RespostaHTTP com bytes e metadados de redirecionamentos.

Exemplo

Código Coral

```
de coral.web.http importe RespostaHTTP
mostre RespostaHTTP(200, (), bytes([65]), "https://exemplo.org", "GET").status
```

Resultado

Saída esperada

```
200
```

Erros

Construção não realiza a requisição, que é responsabilidade de requisitar.

`DiagnosticoInspecao(...)` · módulo `coral`

O que faz

Conserva o resultado estrutural da validação do contrato de inspeção, com mensagens e visões disponíveis. Também exportado diretamente por `coral`, mantendo o mesmo objeto e comportamento do módulo `coral.inspecao`.

Construtor

Assinatura para consulta

```
DiagnosticoInspecao(ok: 'bool', tipo: 'str', nome: 'str | None', assinatura: 'str |
  None', visoes: 'tuple[str, ...]' = (), avisos: 'tuple[str, ...]' = (), erro: 'str |
  None' = None, contrato: 'str' = 'coral.inspecao.diagnostico/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
ok	bool	sim	sem padrão	Resultado da validação do snapshot.
tipo	str	sim	sem padrão	Tipo registrado para o alvo.
nome	str | None	sim	sem padrão	Nome opcional do alvo.
assinatura	str | None	sim	sem padrão	Assinatura estável opcional do snapshot.
visoes	tuple[str, ...]	não	()	Visões semânticas disponíveis.
avisos	tuple[str, ...]	não	()	Mensagens não fatais da inspeção.
erro	str | None	não	None	Erro de validação quando houver.
contrato	str	não	'coral.inspecao.diagnostico/1'	Identificador de versão do contrato do diagnóstico.

Retorno

Instância imutável de DiagnosticoInspecao.

Exemplo

Código Coral

```
de coral importe DiagnosticoInspecao
mostre DiagnosticoInspecao(verdadeiro, "numero", nulo, nulo).ok
```

Resultado

Saída esperada

```
True
```

Erros

O construtor mantém os campos recebidos; dados de tipo incorreto podem falhar ao consumir ou serializar.

`DiferencaInspecao(...)` · módulo `coral`

O que faz

Agrupa os hashes ou identificadores de dois snapshots e suas mudanças estruturadas. Também exportado diretamente por `coral`, mantendo o mesmo objeto e comportamento do módulo `coral.inspecao`.

Construtor

Assinatura para consulta

```
DiferencaInspecao(antes: 'str', depois: 'str', mudancas: 'tuple[MudancaInspecao, ...]' =
   (), contrato: 'str' = 'coral.inspecao.diferenca/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
antes	str	sim	sem padrão	Assinatura do estado anterior.
depois	str	sim	sem padrão	Assinatura do estado posterior.
mudancas	tuple[MudancaInspecao, ...]	não	()	Tupla de mudanças individuais encontradas.
contrato	str	não	'coral.inspecao.diferenca/1'	Identificador versionado do relatório de diferenças.

Retorno

Instância de DiferencaInspecao.

Exemplo

Código Coral

```
de coral importe DiferencaInspecao
mostre DiferencaInspecao("antes", "depois").antes
```

Resultado

Saída esperada

```
antes
```

Erros

As mudanças devem obedecer ao formato do relatório para os consumidores posteriores.

`EventoObservabilidade(...)` · módulo `coral`

O que faz

Representa um evento de observabilidade independente de editor, depurador e transporte. Também exportado diretamente por `coral`, mantendo o mesmo objeto e comportamento do módulo `coral.inspecao`.

Construtor

Assinatura para consulta

```
EventoObservabilidade(sequencia: 'int', tipo: 'str', alvo: 'str | None' = None,
  instante: 'float | None' = None, caminho: 'str | None' = None, valor: 'Any' = None,
  anterior: 'Any' = None, detalhes: 'Mapping[str, Any]' = <factory>, contrato: 'str' =
  'coral.inspecao.evento/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
sequencia	int	sim	sem padrão	Número de ordem da ocorrência.
tipo	str	sim	sem padrão	Identificador semântico da ocorrência.
alvo	str | None	não	None	Alvo opcional do evento.
instante	float | None	não	None	Marca temporal opcional.
caminho	str | None	não	None	Caminho do dado observado.
valor	Any	não	None	Novo valor observado.
anterior	Any	não	None	Valor anterior à mudança.
detalhes	Mapping[str, Any]	não	<factory>	Dados complementares portáteis.
contrato	str	não	'coral.inspecao.evento/1'	Versão do contrato de evento.

Retorno

EventoObservabilidade com sequência, tipo e contexto do evento.

Exemplo

Código Coral

```
de coral importe EventoObservabilidade
mostre EventoObservabilidade(1, "alteracao").tipo
```

Resultado

Saída esperada

```
alteracao
```

Erros

O evento é transportado como dados; consumidores devem validar o significado do tipo e detalhes.

`ExplicacaoRegra(...)` · módulo `coral`

O que faz

Descreve o último estado observável de uma regra por meio de dados estruturados e mensagem legível. Também exportado diretamente por `coral`, mantendo o mesmo objeto e comportamento do módulo `coral.inspecao`.

Construtor

Assinatura para consulta

```
ExplicacaoRegra(nome: 'str | None', estado: 'str', mensagem: 'str', dados: 'Mapping[str,
   Any]' = <factory>, contrato: 'str' = 'coral.inspecao.explicacao_regra/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str | None	sim	sem padrão	Identificador opcional da regra.
estado	str	sim	sem padrão	Estado normalizado da regra.
mensagem	str	sim	sem padrão	Explicação textual legível.
dados	Mapping[str, Any]	não	<factory>	Dados adicionais associados ao estado.
contrato	str	não	'coral.inspecao.explicacao_regra/1'	Versão do contrato de explicação.

Retorno

ExplicacaoRegra com nome, estado, mensagem e dados.

Exemplo

Código Coral

```
de coral importe ExplicacaoRegra
mostre ExplicacaoRegra(nulo, "ativa", "aprovada").estado
```

Resultado

Saída esperada

```
ativa
```

Erros

Dados extras devem manter representação portátil quando serializados.

`MudancaInspecao(...)` · módulo `coral`

O que faz

Identifica uma alteração estrutural em um caminho observado, incluindo valores anterior e posterior. Também exportado diretamente por `coral`, mantendo o mesmo objeto e comportamento do módulo `coral.inspecao`.

Construtor

Assinatura para consulta

```
MudancaInspecao(caminho: 'str', tipo: 'str', antes: 'Any' = None, depois: 'Any' = None)
  -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	str	sim	sem padrão	Caminho dentro de dados do snapshot.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tipo	str	sim	sem padrão	Natureza da alteração.
antes	Any	não	None	Valor antes da ocorrência.
depois	Any	não	None	Valor após a ocorrência.

Retorno

MudancaInspecao com tipo de alteração e caminho.

Exemplo

Código Coral

```
de coral importe MudancaInspecao
mostre MudancaInspecao("pontos", "alterado").caminho
```

Resultado

Saída esperada

```
pontos
```

Erros

O construtor não calcula diferenças; caminhos precisam seguir o contrato de consulta na interpretação.

`RelatorioInspecao(...)` · módulo `coral`

O que faz

Conserva o relatório de apresentação derivado de um snapshot de inspeção existente. Também exportado diretamente por `coral`, mantendo o mesmo objeto e comportamento do módulo `coral.inspecao`.

Construtor

Assinatura para consulta

```
RelatorioInspecao(tipo: 'str', nome: 'str | None', dados: 'Mapping[str, Any]', visao:
  'str | None' = None, assinatura_origem: 'str | None' = None, contrato: 'str' =
  'coral.inspecao.relatorio/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tipo	str	sim	sem padrão	Tipo do objeto resumido.
nome	str | None	sim	sem padrão	Nome opcional do alvo.
dados	Mapping[str, Any]	sim	sem padrão	Mapeamento dos dados apresentados.
visao	str | None	não	None	Identificador da visão selecionada.
assinatura_origem	str | None	não	None	Assinatura do snapshot original.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
contrato	str	não	'coral.inspecao.relatorio/1'	Identificador versionado de relatório.

Retorno

RelatorioInspecao com valores e metadados de origem.

Exemplo

Código Coral

```
de coral importe RelatorioInspecao
mostre RelatorioInspecao("int", nulo, {}).tipo
```

Resultado

Saída esperada

```
int
```

Erros

A semântica do conteúdo depende da visão usada na geração do relatório.

`RespostaInspecao(...)` · módulo `coral`

O que faz

Representa a resposta de um comando de inspeção com resultado ou erro explícito. Também exportado diretamente por `coral`, mantendo o mesmo objeto e comportamento do módulo `coral.inspecao`.

Construtor

Assinatura para consulta

```
RespostaInspecao(operacao: 'str', ok: 'bool', resultado: 'Any' = None, erro: 'str |
  None' = None, contrato: 'str' = 'coral.inspecao.resposta/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
operacao	str	sim	sem padrão	Nome do comando solicitado.
ok	bool	sim	sem padrão	Indicador de conclusão com sucesso.
resultado	Any	não	None	Conteúdo retornado pelo comando.
erro	str | None	não	None	Mensagem de erro, quando aplicável.
contrato	str	não	'coral.inspecao.resposta/1'	Versão do contrato de resposta.

Retorno

RespostaInspecao com indicador de sucesso e conteúdo da operação.

Exemplo

Código Coral

```
de coral importe RespostaInspecao
mostre RespostaInspecao("consultar", verdadeiro).operacao
```

Resultado

Saída esperada

```
consultar
```

Erros

O resultado pode exigir validação de portabilidade para transporte entre processos.

`SnapshotInspecao(...)` · módulo `coral`

O que faz

Registra o estado observável de um objeto sem conceder mutabilidade ou acesso a detalhes privados do domínio. Também exportado diretamente por `coral`, mantendo o mesmo objeto e comportamento do módulo `coral.inspecao`.

Construtor

Assinatura para consulta

```
SnapshotInspecao(tipo: 'str', nome: 'str | None', dados: 'Mapping[str, Any]' =
  <factory>, avisos: 'tuple[str, ...]' = (), contrato: 'str' = 'coral.inspecao/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tipo	str	sim	sem padrão	Identificador semântico do objeto.
nome	str | None	sim	sem padrão	Nome opcional de apresentação.
dados	Mapping[str, Any]	não	<factory>	Mapeamento do estado observado.
avisos	tuple[str, ...]	não	()	Avisos da coleta.
contrato	str	não	'coral.inspecao/1'	Versão de snapshot reconhecida pelos consumidores.

Retorno

SnapshotInspecao com dados normalizáveis em JSON.

Exemplo

Código Coral

```
de coral importe SnapshotInspecao
mostre SnapshotInspecao("inteiro", nulo).tipo
```

Resultado

Saída esperada

```
inteiro
```

Erros

Dados arbitrários exigem normalização antes do transporte e da persistência.

`VigiaInspecao(...)` · módulo `coral`

O que faz

Define uma condição de watchpoint de leitura sobre caminho do contrato de inspeção. Também exportado diretamente por `coral`, mantendo o mesmo objeto e comportamento do módulo `coral.inspecao`.

Construtor

Assinatura para consulta

```
VigiaInspecao(nome: 'str', caminho: 'str', *, operador: 'str' = 'mudou', valor: 'Any' =
  None, ativo: 'bool' = True) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Identificador da vigia.
caminho	str	sim	sem padrão	Caminho observado na estrutura dados.
operador	str	não	'mudou'	Operação de comparação ou detecção de mudança.
valor	Any	não	None	Valor de referência quando exigido pelo operador.
ativo	bool	não	True	Indica se a condição participa das avaliações.

Retorno

VigiaInspecao com operador e valor de referência.

Exemplo

Código Coral

```
de coral importe VigiaInspecao
mostre VigiaInspecao("limite", "pontos").nome
```

Resultado

Saída esperada

```
limite
```

Erros

O caminho e o operador são validados durante a avaliação da vigia.

`VisaoInspecao(...)` · módulo `coral`

O que faz

Armazena uma projeção semântica de um snapshot, sem modificar o estado original. Também exportado diretamente por `coral`, mantendo o mesmo objeto e comportamento do módulo `coral.inspecao`.

Construtor

Assinatura para consulta

```
VisaoInspecao(visao: 'str', tipo: 'str', nome: 'str | None', dados: 'Mapping[str, Any]'
  = <factory>, assinatura_origem: 'str | None' = None, avisos: 'tuple[str, ...]' = (),
  contrato: 'str' = 'coral.inspecao.visao/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
visao	str	sim	sem padrão	Nome da representação semântica.
tipo	str	sim	sem padrão	Tipo de objeto representado.
nome	str | None	sim	sem padrão	Nome opcional do objeto.
dados	Mapping[str, Any]	não	<factory>	Dados específicos da visão.
assinatura_origem	str | None	não	None	Hash de origem para detectar mudanças.
avisos	tuple[str, ...]	não	()	Avisos de projeção.
contrato	str	não	'coral.inspecao.visao/1'	Versão do contrato de visão.

Retorno

VisaoInspecao com rótulo, tipo e dados filtrados.

Exemplo

Código Coral

```
de coral importe VisaoInspecao
mostre VisaoInspecao("resumo", "inteiro", nulo).visao
```

Resultado

Saída esperada

```
resumo
```

Erros

A assinatura da origem é apenas referência e não autoriza mutação no objeto.

`consultar_inspecao(...)` · módulo `coral`

O que faz

Consulta um caminho pontuado ou com índices dentro dos dados normalizados de um snapshot. Também exportado diretamente por `coral`, mantendo o mesmo objeto e comportamento do módulo `coral.inspecao`.

Assinatura

Assinatura para consulta

```
consultar_inspecao(objeto: 'Any', caminho: 'str', padrao: 'Any' = <ausente>) -> 'Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto inspecionável ou snapshot normalizado.
caminho	str	sim	sem padrão	Expressão do caminho dentro do campo dados.
padrao	Any	não	<ausente>	Valor devolvido se o caminho não existir; omissão gera erro.

Retorno

Valor selecionado, ou padrão quando o caminho está ausente.

Exemplo

Código Coral

```
de coral importe consultar_inspecao
mostre consultar_inspecao({"pontos": 3}, "valor.pontos", 0)
```

Resultado

Saída esperada

```
3
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

KeyError para caminho inexistente sem padrão explícito.

`inspecionar(...)` · módulo `coral`

O que faz

Constrói um snapshot normalizado e de somente leitura de um objeto observável. Também exportado diretamente por `coral`, mantendo o mesmo objeto e comportamento do módulo `coral.inspecao`.

Assinatura

Assinatura para consulta

```
inspecionar(objeto: 'Any', *, nome: 'str | None' = None, tipo: 'str | None' = None) ->
  'SnapshotInspecao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto a inspecionar sem mutação.
nome	str | None	não	None	Nome explícito opcional para o snapshot.
tipo	str | None	não	None	Tipo explícito opcional para o snapshot.

Retorno

SnapshotInspecao com tipo, nome, dados e avisos.

Exemplo

Código Coral

```
de coral importe inspecionar
mostre inspecionar(10).tipo
```

Resultado

Saída esperada

```
int
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Objetos de domínio devem expor estado_inspecao para fornecer campos próprios.

`inspecionar_dict(...)` · módulo `coral`

O que faz

Expõe o snapshot como dicionário JSON seguro sem criar modelo paralelo. Também exportado diretamente por `coral`, mantendo o mesmo objeto e comportamento do módulo `coral.inspecao`.

Assinatura

Assinatura para consulta

```
inspecionar_dict(objeto: 'Any', **opcoes: 'Any') -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto cujo snapshot será transformado em dicionário.
opcoes	Any	não	sem padrão	Opções nomeadas repassadas à inspeção.

Retorno

Dicionário de campos do snapshot de inspeção.

Exemplo

Código Coral

```
de coral importe inspecionar_dict
mostre inspecionar_dict(10)["tipo"]
```

Resultado

Saída esperada

```
int
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Opções adicionais seguem o contrato de inspecionar.

`selecionar_inspecao(...)` · módulo `coral`

O que faz

Extrai vários caminhos de inspeção em ordem e devolve um dicionário. Também exportado diretamente por `coral`, mantendo o mesmo objeto e comportamento do módulo `coral.inspecao`.

Assinatura

Assinatura para consulta

```
selecionar_inspecao(objeto: 'Any', caminhos: 'list[str] | tuple[str, ...]') ->
  'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto inspecionável ou snapshot existente.
caminhos	list[str] | tuple[str, ...]	sim	sem padrão	Sequência ordenada dos caminhos a consultar.

Retorno

Mapeamento de caminhos para valores encontrados.

Exemplo

Código Coral

```
de coral importe selecionar_inspecao
mostre selecionar_inspecao({"pontos": 3}, ["valor.pontos"])
```

Resultado

Saída esperada

```
{'valor.pontos': 3}
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

KeyError se algum caminho solicitado não existir.

`visoes_inspecao(...)` · módulo `coral`

O que faz

Lista as visões semânticas disponíveis para determinado objeto de inspeção. Também exportado diretamente por `coral`, mantendo o mesmo objeto e comportamento do módulo `coral.inspecao`.

Assinatura

Assinatura para consulta

```
visoes_inspecao(objeto: 'Any') -> 'tuple[str, ...]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto a partir do qual serão inferidas as visões disponíveis.

Retorno

Tupla de identificadores de visão reconhecidos para o objeto.

Exemplo

Código Coral

```
de coral importe visoes_inspecao
mostre visoes_inspecao(10)
```

Resultado

Saída esperada

```
()
```

Forma natural

Sem forma natural alternativa exclusiva: importe a operação pública do módulo correspondente.

Erros

Não cria nem modifica visões no objeto original.

`angulo_entre_vetores(...)` · módulo `coral.numerico`

O que faz

Calcula o ângulo entre dois vetores não nulos pelo cosseno de sua orientação.

Assinatura

Assinatura para consulta

```
angulo_entre_vetores(a, b)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	valor dinâmico	sim	sem padrão	Primeiro vetor finito.
b	valor dinâmico	sim	sem padrão	Segundo vetor da mesma dimensão.

Retorno

Ângulo em radianos entre os vetores.

Exemplo

Código Coral

```
de coral.numerico importe angulo_entre_vetores
mostre angulo_entre_vetores([1, 0], [0, 1])
```

Resultado

Saída esperada

```
1.5707963267948966
```

Erros

ErroNumerico para dimensão incompatível ou vetor de norma zero.

`assimetria(...)` · módulo `coral.numerico`

O que faz

Quantifica o desvio de simetria de uma amostra em torno de sua média.

Assinatura

Assinatura para consulta

```
assimetria(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Valores observados, finitos e numéricos.

Retorno

Coeficiente numérico de assimetria amostral.

Exemplo

Código Coral

```
de coral.numerico importe assimetria
mostre assimetria([1, 2, 3, 4, 5])
```

Resultado

Saída esperada

```
3.679904200040098e-16
```

Erros

ErroNumerico para dados insuficientes ou inválidos.

`autovalores(...)` · módulo `coral.numerico`

O que faz

Obtém os autovalores de uma matriz quadrada.

Assinatura

Assinatura para consulta

```
autovalores(m)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
m	valor dinâmico	sim	sem padrão	Matriz quadrada cujos autovalores serão calculados.

Retorno

Sequência numérica, possivelmente com partes complexas.

Exemplo

Código Coral

```
de coral.numerico importe autovalores
mostre autovalores([[2, 0], [0, 3]])
```

Resultado

Saída esperada

```
[2. 3.]
```

Erros

ErroNumerico para matriz não quadrada ou tipos incompatíveis.

`concatenar(...)` · módulo `coral.numerico`

O que faz

Concatena coleções ou matrizes compatíveis ao longo de um eixo escolhido.

Assinatura

Assinatura para consulta

```
concatenar(valores, eixo=0)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Lista de arranjos a combinar.
eixo	valor dinâmico	não	0	Eixo onde ocorre a concatenação, começando em zero.

Retorno

Arranjo formado pela justaposição das entradas.

Exemplo

Código Coral

```
de coral.numerico importe concatenar
mostre concatenar([[1, 2], [3, 4]])
```

Resultado

Saída esperada

```
[1 2 3 4]
```

Erros

ErroNumerico ou erro de dimensão para matrizes de formas incompatíveis.

`coordenadas_esfericas3(...)` · módulo `coral.numerico`

O que faz

Converte um vetor cartesiano 3D em raio e ângulos esféricos.

Assinatura

Assinatura para consulta

```
coordenadas_esfericas3(ponto)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
ponto	valor dinâmico	sim	sem padrão	Vetor cartesiano tridimensional finito.

Retorno

Coordenadas esféricas ordenadas segundo o contrato 3D.

Exemplo

Código Coral

```
de coral.numerico importe coordenadas_esfericas3
mostre coordenadas_esfericas3([1, 0, 0])
```

Resultado

Saída esperada

```
(1.0, 0.0, 0.0)
```

Erros

ErroNumerico para coordenadas inválidas ou dimensão diferente de três.

`curtose(...)` · módulo `coral.numerico`

O que faz

Mede o grau relativo de concentração de valores em caudas de uma distribuição.

Assinatura

Assinatura para consulta

```
curtose(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Observações numéricas finitas.

Retorno

Coeficiente numérico de curtose sobre a amostra.

Exemplo

Código Coral

```
de coral.numerico importe curtose
mostre curtose([1, 2, 3, 4, 5])
```

Resultado

Saída esperada

```
-1.3000000000000005
```

Erros

ErroNumerico para amostra insuficiente ou degenerada.

`determinante(...)` · módulo `coral.numerico`

O que faz

Calcula o determinante de uma matriz quadrada pelo núcleo numérico.

Assinatura

Assinatura para consulta

```
determinante(m)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
m	valor dinâmico	sim	sem padrão	Matriz quadrada, numérica e finita.

Retorno

Escalar do determinante.

Exemplo

Código Coral

```
de coral.numerico importe determinante
mostre determinante([[2, 0], [0, 3]])
```

Resultado

Saída esperada

```
6.0
```

Erros

ErroNumerico para matriz não quadrada ou componentes inválidas.

`diagnosticar_numerico(...)` · módulo `coral.numerico`

O que faz

Descreve a configuração e disponibilidade dos recursos numéricos opcionais.

Assinatura

Assinatura para consulta

```
diagnosticar_numerico() -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Dicionário de capacidades e dependências, sem alterar configurações.

Exemplo

Código Coral

```
de coral.numerico importe diagnosticar_numerico
mostre diagnosticar_numerico() == diagnosticar_numerico()
```

Resultado

Saída esperada

```
True
```

Erros

Não executa operações científicas nem instala dependências.

`distancia_ponto_plano3(...)` · módulo `coral.numerico`

O que faz

Obtém a distância perpendicular de um ponto tridimensional a um plano.

Assinatura

Assinatura para consulta

```
distancia_ponto_plano3(ponto, ponto_plano, normal)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
ponto	valor dinâmico	sim	sem padrão	Coordenadas do ponto consultado.
ponto_plano	valor dinâmico	sim	sem padrão	Ponto de referência pertencente ao plano.
normal	valor dinâmico	sim	sem padrão	Vetor normal não nulo ao plano.

Retorno

Distância euclidiana não negativa.

Exemplo

Código Coral

```
de coral.numerico importe distancia_ponto_plano3
mostre distancia_ponto_plano3([0, 0, 2], [0, 0, 0], [0, 0, 1])
```

Resultado

Saída esperada

```
2.0
```

Erros

ErroNumerico para normal nula ou vetores de dimensões incompatíveis.

`erro_padrao(...)` · módulo `coral.numerico`

O que faz

Estima o erro padrão da média de uma amostra numérica.

Assinatura

Assinatura para consulta

```
erro_padrao(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Observações numéricas independentes da amostra.

Retorno

Desvio padrão da média estimada.

Exemplo

Código Coral

```
de coral.numerico importe erro_padrao
mostre erro_padrao([2, 4, 6, 8])
```

Resultado

Saída esperada

```
1.2909944487358056
```

Erros

ErroNumerico para amostra inválida ou pequena demais.

`espaco_linear(...)` · módulo `coral.numerico`

O que faz

Gera pontos igualmente espaçados num intervalo incluindo as extremidades.

Assinatura

Assinatura para consulta

```
espaco_linear(inicio, fim, quantidade=50)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
inicio	valor dinâmico	sim	sem padrão	Extremidade inicial do intervalo.
fim	valor dinâmico	sim	sem padrão	Extremidade final incluída.
quantidade	valor dinâmico	não	50	Quantidade de amostras equidistantes.

Retorno

Arranjo de pontos de início a fim.

Exemplo

Código Coral

```
de coral.numerico importe espaco_linear
mostre espaco_linear(0, 1, 3)
```

Resultado

Saída esperada

```
[0.  0.5 1. ]
```

Erros

ErroNumerico para quantidade inválida ou limites não finitos.

`forma(...)` · módulo `coral.numerico`

O que faz

Consulta as dimensões de um vetor ou matriz sem modificar seus valores.

Assinatura

Assinatura para consulta

```
forma(valor) -> 'tuple[int, ...]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	valor dinâmico	sim	sem padrão	Arranjo cuja forma será inspecionada.

Retorno

Tupla de comprimentos por eixo.

Exemplo

Código Coral

```
de coral.numerico importe forma
mostre forma([[1, 2], [3, 4]])
```

Resultado

Saída esperada

```
(2, 2)
```

Erros

ErroNumerico para dados irregulares que não admitem uma forma.

`frequencias(...)` · módulo `coral.numerico`

O que faz

Conta quantas vezes cada valor distinto aparece numa coleção.

Assinatura

Assinatura para consulta

```
frequencias(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Observações categóricas ou numéricas da contagem.

Retorno

Dicionário de valores observados e suas frequências.

Exemplo

Código Coral

```
de coral.numerico importe frequencias
mostre frequencias([1, 1, 2])
```

Resultado

Saída esperada

```
{1: 2, 2: 1}
```

Erros

ErroNumerico para amostras inválidas ou valores não suportados.

`histograma(...)` · módulo `coral.numerico`

O que faz

Agrupa dados por faixas numéricas explícitas, preservando contagens.

Assinatura

Assinatura para consulta

```
histograma(valores, faixas)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Amostra numérica a agrupar.
faixas	valor dinâmico	sim	sem padrão	Limites das faixas ou especificação de sua distribuição.

Retorno

Histograma estruturado com intervalos e contagens.

Exemplo

Código Coral

```
de coral.numerico importe histograma
mostre histograma([0, 1, 2, 3], [0, 2, 4])
```

Resultado

Saída esperada

```
(array([2, 2]), array([0., 2., 4.]))
```

Erros

ErroNumerico para limites de faixa inválidos.

`identidade(...)` · módulo `coral.numerico`

O que faz

Constrói matriz identidade de ordem indicada.

Assinatura

Assinatura para consulta

```
identidade(ordem)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
ordem	valor dinâmico	sim	sem padrão	Quantidade de linhas e colunas da matriz quadrada.

Retorno

Matriz quadrada com 1 na diagonal principal e 0 fora dela.

Exemplo

Código Coral

```
de coral.numerico importe identidade
mostre identidade(2)
```

Resultado

Saída esperada

```
[[1. 0.]
 [0. 1.]]
```

Erros

ErroNumerico para ordem negativa ou dimensão inválida.

`intervalo(...)` · **módulo** `coral.numerico`

O que faz

Gera uma sequência de valores numéricos com passo explícito, sem incluir o limite final.

Assinatura

Assinatura para consulta

```
intervalo(inicio, fim=None, passo=1, tipo=None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
inicio	valor dinâmico	sim	sem padrão	Valor inicial, ou limite quando fim é omitido.
fim	valor dinâmico	não	None	Limite exclusivo opcional.
passo	valor dinâmico	não	1	Distância entre elementos consecutivos.
tipo	valor dinâmico	não	None	Tipo numérico de materialização quando informado.

Retorno

Arranjo numérico dos elementos do intervalo.

Exemplo

Código Coral

```
de coral.numerico importe intervalo
mostre intervalo(0, 5, 2)
```

Resultado

Saída esperada

```
[0 2 4]
```

Erros

ErroNumerico para passo zero ou condições incompatíveis com o tipo.

`inversa(...)` · módulo `coral.numerico`

O que faz

Calcula a inversa de uma matriz quadrada não singular.

Assinatura

Assinatura para consulta

```
inversa(m)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
m	valor dinâmico	sim	sem padrão	Matriz quadrada que admite inversão.

Retorno

Matriz inversa para a multiplicação matricial.

Exemplo

Código Coral

```
de coral.numerico importe inversa
mostre inversa([[2, 0], [0, 2]])
```

Resultado

Saída esperada

```
[[0.5 0. ]
 [0.  0.5]]
```

Erros

ErroNumerico para matriz singular, não quadrada ou inválida.

`matriz(...)` · módulo `coral.numerico`

O que faz

Constrói uma matriz numérica a partir de linhas de valores.

Assinatura

Assinatura para consulta

```
matriz(linhas: 'Iterable[Iterable[Any]]', tipo: 'Any | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
linhas	Iterable[Iterable[Any]]	sim	sem padrão	Coleção ordenada das linhas da matriz.
tipo	Any | None	não	None	Tipo dos elementos quando for solicitado explicitamente.

Retorno

Matriz com a forma e tipo numérico definidos.

Exemplo

Código Coral

```
de coral.numerico importe matriz
mostre matriz([[1, 2], [3, 4]])
```

Resultado

Saída esperada

```
[[1 2]
 [3 4]]
```

Erros

ErroNumerico para linhas de comprimentos incompatíveis.

`matriz_correlacao(...)` · módulo `coral.numerico`

O que faz

Calcula as correlações de Pearson entre colunas de dados.

Assinatura

Assinatura para consulta

```
matriz_correlacao(dados)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
dados	valor dinâmico	sim	sem padrão	Matriz de observações e variáveis numéricas.

Retorno

Matriz simétrica dos coeficientes de correlação.

Exemplo

Código Coral

```
de coral.numerico importe matriz_correlacao
mostre matriz_correlacao([[1, 2], [2, 4], [3, 6]])
```

Resultado

Saída esperada

```
[[1. 1.]
 [1. 1.]]
```

Erros

ErroNumerico para amostras insuficientes ou colunas degeneradas.

`matriz_covariancia(...)` · módulo `coral.numerico`

O que faz

Calcula covariâncias entre variáveis de uma matriz de observações.

Assinatura

Assinatura para consulta

```
matriz_covariancia(dados, populacional=False)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
dados	valor dinâmico	sim	sem padrão	Conjunto tabular de variáveis numéricas.
populacional	valor dinâmico	não	False	Determina divisor populacional quando verdadeiro.

Retorno

Matriz simétrica de covariâncias.

Exemplo

Código Coral

```
de coral.numerico importe matriz_covariancia
mostre matriz_covariancia([[1, 2], [2, 4], [3, 6]])
```

Resultado

Saída esperada

```
[[1. 2.]
 [2. 4.]]
```

Erros

ErroNumerico para tamanho amostral insuficiente.

`matriz_escala3(...)` · módulo `coral.numerico`

O que faz

Cria transformação homogênea de escala dos três eixos cartesianos.

Assinatura

Assinatura para consulta

```
matriz_escala3(sx, sy=None, sz=None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
sx	valor dinâmico	sim	sem padrão	Escala do eixo X.
sy	valor dinâmico	não	None	Escala opcional do eixo Y.
sz	valor dinâmico	não	None	Escala opcional do eixo Z.

Retorno

Matriz 4×4 de escala em três dimensões.

Exemplo

Código Coral

```
de coral.numerico importe matriz_escala3
mostre matriz_escala3(2, 3, 4)
```

Resultado

Saída esperada

```
[[2. 0. 0. 0.]
 [0. 3. 0. 0.]
 [0. 0. 4. 0.]
 [0. 0. 0. 1.]]
```

Erros

ErroNumerico para escalas inválidas ou não finitas.

`matriz_rotacao3(...)` · módulo `coral.numerico`

O que faz

Cria matriz de rotação em torno de um eixo tridimensional.

Assinatura

Assinatura para consulta

```
matriz_rotacao3(eixo, angulo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
eixo	valor dinâmico	sim	sem padrão	Eixo tridimensional de rotação.
angulo	valor dinâmico	sim	sem padrão	Ângulo da rotação em radianos.

Retorno

Matriz 4×4 de rotação 3D.

Exemplo

Código Coral

```
de coral.numerico importe matriz_rotacao3
mostre matriz_rotacao3([0, 0, 1], 0)
```

Resultado

Saída esperada

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

Erros

ErroNumerico para eixo nulo ou ângulo inválido.

`matriz_translacao3(...)` · módulo `coral.numerico`

O que faz

Cria matriz homogênea para translação espacial.

Assinatura

Assinatura para consulta

```
matriz_translacao3(dx, dy, dz)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
dx	valor dinâmico	sim	sem padrão	Translação no eixo X.
dy	valor dinâmico	sim	sem padrão	Translação no eixo Y.
dz	valor dinâmico	sim	sem padrão	Translação no eixo Z.

Retorno

Matriz 4×4 com deslocamentos X, Y e Z.

Exemplo

Código Coral

```
de coral.numerico importe matriz_translacao3
mostre matriz_translacao3(1, 2, 3)
```

Resultado

Saída esperada

```
[[1. 0. 0. 1.]
 [0. 1. 0. 2.]
 [0. 0. 1. 3.]
 [0. 0. 0. 1.]]
```

Erros

ErroNumerico para deslocamentos não finitos.

`moda(...)` · módulo `coral.numerico`

O que faz

Encontra o valor mais frequente de uma amostra.

Assinatura

Assinatura para consulta

```
moda(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Observações cujas frequências serão comparadas.

Retorno

Valor modal conforme o critério determinístico da implementação.

Exemplo

Código Coral

```
de coral.numerico importe moda
mostre moda([1, 2, 2, 3])
```

Resultado

Saída esperada

```
[2]
```

Erros

ErroNumerico para coleção vazia ou valores incompatíveis.

`multiplicar_matrizes(...)` · módulo `coral.numerico`

O que faz

Calcula produto matricial com soma de produtos internos dos eixos.

Assinatura

Assinatura para consulta

```
multiplicar_matrizes(a, b)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	valor dinâmico	sim	sem padrão	Matriz à esquerda.
b	valor dinâmico	sim	sem padrão	Matriz à direita com dimensão interna compatível.

Retorno

Matriz do produto A×B.

Exemplo

Código Coral

```
de coral.numerico importe multiplicar_matrizes
mostre multiplicar_matrizes([[1, 2]], [[3], [4]])
```

Resultado

Saída esperada

```
[[11]]
```

Erros

ErroNumerico para dimensões incompatíveis.

`normalizar(...)` · módulo `coral.numerico`

O que faz

Transforma um vetor numérico para norma unitária.

Assinatura

Assinatura para consulta

```
normalizar(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Componentes do vetor a normalizar.

Retorno

Vetor normalizado preservando sua direção.

Exemplo

Código Coral

```
de coral.numerico importe normalizar
mostre normalizar([3, 4])
```

Resultado

Saída esperada

```
[0.6 0.8]
```

Erros

ErroNumerico para vetor nulo ou componentes inválidas.

`numpy_disponivel(...)` · módulo `coral.numerico`

O que faz

Verifica a presença da dependência NumPy sem criar arrays.

Assinatura

Assinatura para consulta

```
numpy_disponivel() -> 'bool'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Booleano de disponibilidade da capacidade numérica opcional.

Exemplo

Código Coral

```
de coral.numerico importe numpy_disponivel
mostre numpy_disponivel() == numpy_disponivel()
```

Resultado

Saída esperada

```
True
```

Erros

Não instala NumPy nem importa ferramentas de interface.

`ponto_de_esfericas3(...)` · módulo `coral.numerico`

O que faz

Converte raio, azimute e elevação em coordenadas cartesianas tridimensionais.

Assinatura

Assinatura para consulta

```
ponto_de_esfericas3(raio, azimute, elevacao)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
raio	valor dinâmico	sim	sem padrão	Distância radial à origem.
azimute	valor dinâmico	sim	sem padrão	Ângulo de direção horizontal em radianos.
elevacao	valor dinâmico	sim	sem padrão	Ângulo vertical em radianos.

Retorno

Vetor cartesiano 3D correspondente.

Exemplo

Código Coral

```
de coral.numerico importe ponto_de_esfericas3
mostre ponto_de_esfericas3(1, 0, 0)
```

Resultado

Saída esperada

```
[1. 0. 0.]
```

Erros

ErroNumerico para raio inválido ou ângulo não finito.

`posto(...)` · módulo `coral.numerico`

O que faz

Calcula o posto de uma matriz numérica segundo tolerâncias do núcleo.

Assinatura

Assinatura para consulta

```
posto(matriz)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
matriz	valor dinâmico	sim	sem padrão	Matriz numérica cujo posto será calculado.

Retorno

Número inteiro de dimensões lineares independentes.

Exemplo

Código Coral

```
de coral.numerico importe posto
mostre posto([[1, 0], [0, 1]])
```

Resultado

Saída esperada

```
2
```

Erros

ErroNumerico para matriz malformada ou tipos incompatíveis.

`produto_misto3(...)` · módulo `coral.numerico`

O que faz

Calcula o volume orientado formado por três vetores em 3D.

Assinatura

Assinatura para consulta

```
produto_misto3(a, b, c)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	valor dinâmico	sim	sem padrão	Primeiro vetor 3D.
b	valor dinâmico	sim	sem padrão	Segundo vetor 3D.
c	valor dinâmico	sim	sem padrão	Terceiro vetor 3D.

Retorno

Escalar do produto misto, com sinal da orientação.

Exemplo

Código Coral

```
de coral.numerico importe produto_misto3
mostre produto_misto3([1, 0, 0], [0, 1, 0], [0, 0, 1])
```

Resultado

Saída esperada

```
1.0
```

Erros

ErroNumerico para dimensão diferente de três.

`projecao_vetorial(...)` · módulo `coral.numerico`

O que faz

Projeta um vetor sobre a direção de um segundo vetor não nulo.

Assinatura

Assinatura para consulta

```
projecao_vetorial(a, sobre)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	valor dinâmico	sim	sem padrão	Vetor a ser projetado.
sobre	valor dinâmico	sim	sem padrão	Vetor que estabelece a direção da projeção.

Retorno

Vetor componente paralelo à direção de referência.

Exemplo

Código Coral

```
de coral.numerico importe projecao_vetorial
mostre projecao_vetorial([3, 4], [1, 0])
```

Resultado

Saída esperada

```
[3. 0.]
```

Erros

ErroNumerico quando a referência é nula ou as dimensões não coincidem.

`projetar_ponto_plano3(...)` · módulo `coral.numerico`

O que faz

Projeta um ponto na superfície de um plano tridimensional.

Assinatura

Assinatura para consulta

```
projetar_ponto_plano3(ponto, ponto_plano, normal)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
ponto	valor dinâmico	sim	sem padrão	Ponto cuja projeção se deseja.
ponto_plano	valor dinâmico	sim	sem padrão	Ponto pertencente ao plano.
normal	valor dinâmico	sim	sem padrão	Normal não nula ao plano.

Retorno

Ponto 3D com componente perpendicular removida.

Exemplo

Código Coral

```
de coral.numerico importe projetar_ponto_plano3
mostre projetar_ponto_plano3([1, 2, 3], [0, 0, 0], [0, 0, 1])
```

Resultado

Saída esperada

```
[1. 2. 0.]
```

Erros

ErroNumerico para normal nula ou entrada não finita.

`quartis(...)` · módulo `coral.numerico`

O que faz

Calcula os três quartis que dividem os valores ordenados em quatro partes.

Assinatura

Assinatura para consulta

```
quartis(valores)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Valores finitos a partir dos quais serão calculados os quartis.

Retorno

Três estatísticas do primeiro, segundo e terceiro quartil.

Exemplo

Código Coral

```
de coral.numerico importe quartis
mostre quartis([1, 2, 3, 4, 5])
```

Resultado

Saída esperada

```
(np.float64(2.0), np.float64(3.0), np.float64(4.0))
```

Erros

ErroNumerico para amostra vazia ou inválida.

`regressao_linear(...)` · módulo `coral.numerico`

O que faz

Ajusta uma reta pelo método de mínimos quadrados simples.

Assinatura

Assinatura para consulta

```
regressao_linear(x, y)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	valor dinâmico	sim	sem padrão	Valores da variável explicativa.
y	valor dinâmico	sim	sem padrão	Valores correspondentes da variável resposta.

Retorno

ResultadoRegressaoLinear com inclinação, intercepto, resíduos e R^2.

Exemplo

Código Coral

```
de coral.numerico importe regressao_linear
mostre regressao_linear([1, 2, 3], [2, 4, 6]).inclinacao
```

Resultado

Saída esperada

```
2.0
```

Erros

ErroNumerico para tamanhos incompatíveis ou variância X nula.

`remodelar(...)` · módulo `coral.numerico`

O que faz

Reorganiza um arranjo em nova forma conservando a ordem dos elementos.

Assinatura

Assinatura para consulta

```
remodelar(valores, forma)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Elementos de entrada.
forma	valor dinâmico	sim	sem padrão	Tupla das dimensões de destino.

Retorno

Arranjo com novas dimensões.

Exemplo

Código Coral

```
de coral.numerico importe remodelar
mostre remodelar([1, 2, 3, 4], (2, 2))
```

Resultado

Saída esperada

```
[[1 2]
 [3 4]]
```

Erros

ErroNumerico quando o total de elementos não corresponde à forma solicitada.

`resolver_sistema(...)` · módulo `coral.numerico`

O que faz

Resolve sistema linear Ax=b quando a matriz admite solução única.

Assinatura

Assinatura para consulta

```
resolver_sistema(coeficientes, termos)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
coeficientes	valor dinâmico	sim	sem padrão	Matriz A dos coeficientes.
termos	valor dinâmico	sim	sem padrão	Vetor b dos termos independentes.

Retorno

Vetor solução das incógnitas.

Exemplo

Código Coral

```
de coral.numerico importe resolver_sistema
mostre resolver_sistema([[1, 0], [0, 1]], [2, 3])
```

Resultado

Saída esperada

```
[2. 3.]
```

Erros

ErroNumerico para sistema singular ou dimensões incompatíveis.

`rotacionar3(...)` · módulo `coral.numerico`

O que faz

Rotaciona um vetor tridimensional em torno de um eixo usando um ângulo em radianos.

Assinatura

Assinatura para consulta

```
rotacionar3(vetor, eixo, angulo)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
vetor	valor dinâmico	sim	sem padrão	Vetor de entrada tridimensional.
eixo	valor dinâmico	sim	sem padrão	Eixo fixo de rotação não nulo.
angulo	valor dinâmico	sim	sem padrão	Ângulo da rotação em radianos.

Retorno

Vetor 3D transformado.

Exemplo

Código Coral

```
de coral.numerico importe rotacionar3
mostre rotacionar3([1, 0, 0], [0, 0, 1], 0)
```

Resultado

Saída esperada

```
[1. 0. 0.]
```

Erros

ErroNumerico para eixo nulo e componentes inválidas.

`selecionar_coluna(...)` · módulo `coral.numerico`

O que faz

Extrai os valores de uma coluna de registros ordenados.

Assinatura

Assinatura para consulta

```
selecionar_coluna(registros, campo, ausentes='erro')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
registros	valor dinâmico	sim	sem padrão	Coleção de mapeamentos ou registros com propriedades.
campo	valor dinâmico	sim	sem padrão	Campo a selecionar.
ausentes	valor dinâmico	não	'erro'	Política quando um registro não apresenta o campo.

Retorno

Sequência da coluna preservando a ordem de registros.

Exemplo

Código Coral

```
de coral.numerico importe selecionar_coluna
mostre selecionar_coluna([{"x": 2}, {"x": 3}], "x")
```

Resultado

Saída esperada

```
[2, 3]
```

Erros

ErroNumerico quando campo ausente e política de ausência for erro.

`transposta(...)` · módulo `coral.numerico`

O que faz

Troca os eixos de linhas e colunas de uma matriz.

Assinatura

Assinatura para consulta

```
transposta(valor)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	valor dinâmico	sim	sem padrão	Matriz cuja orientação será trocada.

Retorno

Matriz transposta.

Exemplo

Código Coral

```
de coral.numerico importe transposta
mostre transposta([[1, 2], [3, 4]])
```

Resultado

Saída esperada

```
[[1 3]
 [2 4]]
```

Erros

ErroNumerico para dados irregulares ou não matriciais.

`uns(...)` · módulo `coral.numerico`

O que faz

Constrói arranjo numérico com valor um em todas as posições.

Assinatura

Assinatura para consulta

```
uns(forma, tipo=<class 'float'>)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
forma	valor dinâmico	sim	sem padrão	Dimensões do arranjo a criar.
tipo	valor dinâmico	não	float	Tipo numérico escolhido para os elementos.

Retorno

Arranjo com a forma e o tipo de elementos solicitados.

Exemplo

Código Coral

```
de coral.numerico importe uns
mostre uns((2, 2))
```

Resultado

Saída esperada

```
[[1. 1.]
 [1. 1.]]
```

Erros

ErroNumerico para dimensões negativas ou inválidas.

`vetor(...)` · módulo `coral.numerico`

O que faz

Materializa coleção numérica unidimensional em vetor.

Assinatura

Assinatura para consulta

```
vetor(valores: 'Iterable[Any]', tipo: 'Any | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	Iterable[Any]	sim	sem padrão	Sequência dos componentes.
tipo	Any | None	não	None	Tipo de componente numérico opcional.

Retorno

Vetor de valores na ordem fornecida.

Exemplo

Código Coral

```
de coral.numerico importe vetor
mostre vetor([1, 2, 3])
```

Resultado

Saída esperada

```
[1 2 3]
```

Erros

ErroNumerico para valores incompatíveis com o tipo solicitado.

`zeros(...)` · módulo `coral.numerico`

O que faz

Cria um arranjo numérico de zeros da forma especificada.

Assinatura

Assinatura para consulta

```
zeros(forma, tipo=<class 'float'>)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
forma	valor dinâmico	sim	sem padrão	Tupla das dimensões do arranjo.
tipo	valor dinâmico	não	float	Tipo de componente numérico, padrão ponto flutuante.

Retorno

Arranjo de zeros com dimensões e tipo escolhidos.

Exemplo

Código Coral

```
de coral.numerico importe zeros
mostre zeros((2, 2))
```

Resultado

Saída esperada

```
[[0. 0.]
 [0. 0.]]
```

Erros

ErroNumerico para dimensões negativas ou incompatíveis.

`compor_transformacoes3(...)` · módulo `coral.numerico`

O que faz

Compõe matrizes afins 3D em ordem de aplicação, produzindo uma única transformação homogênea.

Assinatura

Assinatura para consulta

```
compor_transformacoes3(*transformacoes)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
transformacoes	valor dinâmico	não	sem padrão	Matrizes homogêneas 4 por 4 na ordem operacional desejada.

Retorno

Matriz homogênea 4 por 4, apropriada para pontos e vetores.

Exemplo

Código Coral

```
de coral.numerico importe compor_transformacoes3
de coral.numerico importe transformar_ponto3, matriz_translacao3
mostre transformar_ponto3([1,2,3], compor_transformacoes3(matriz_translacao3(2,0,0),
  matriz_translacao3(0,3,0)))
```

Resultado

Saída esperada

```
[3. 5. 3.]
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

ErroNumerico ou ValueError quando alguma transformação não é uma matriz 4 por 4 válida.

`transformar_ponto3(...)` · módulo `coral.numerico`

O que faz

Aplica uma transformação afim homogênea 3D a um ponto, inclusive a componente de translação.

Assinatura

Assinatura para consulta

```
transformar_ponto3(ponto, transformacao)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
ponto	valor dinâmico	sim	sem padrão	Coordenadas tridimensionais do ponto de entrada.
transformacao	valor dinâmico	sim	sem padrão	Matriz homogênea 4 por 4 aplicada ao ponto.

Retorno

Vetor cartesiano tridimensional resultante da transformação.

Exemplo

Código Coral

```
de coral.numerico importe transformar_ponto3
de coral.numerico importe matriz_translacao3
mostre transformar_ponto3([1,2,3], matriz_translacao3(5,0,0))
```

Resultado

Saída esperada

```
[6. 2. 3.]
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

ErroNumerico para ponto de dimensão incorreta ou matriz incompatível.

`transformar_vetor3(...)` · módulo `coral.numerico`

O que faz

Aplica somente a componente linear de uma transformação 3D, sem deslocar o vetor por translação.

Assinatura

Assinatura para consulta

```
transformar_vetor3(vetor, transformacao)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
vetor	valor dinâmico	sim	sem padrão	Vetor de direção com três componentes.
transformacao	valor dinâmico	sim	sem padrão	Transformação 4 por 4 cuja translação deve ser ignorada.

Retorno

Vetor transformado com três componentes.

Exemplo

Código Coral

```
de coral.numerico importe transformar_vetor3
de coral.numerico importe matriz_translacao3
mostre transformar_vetor3([1,2,3], matriz_translacao3(5,0,0))
```

Resultado

Saída esperada

```
[1. 2. 3.]
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

ErroNumerico para vetor de dimensão incorreta ou transformação inválida.

`intersecao_reta_plano3(...)` · módulo `coral.numerico`

O que faz

Obtém o encontro de uma reta parametrizada por origem e direção com um plano definido por ponto e normal.

Assinatura

Assinatura para consulta

```
intersecao_reta_plano3(origem, direcao, ponto_plano, normal, *, tolerancia=1e-12)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
origem	valor dinâmico	sim	sem padrão	Ponto inicial da reta tridimensional.
direcao	valor dinâmico	sim	sem padrão	Vetor diretor da reta.
ponto_plano	valor dinâmico	sim	sem padrão	Ponto pertencente ao plano.
normal	valor dinâmico	sim	sem padrão	Vetor normal não nulo do plano.
tolerancia	valor dinâmico	não	1e-12	Limiar numérico para identificar paralelismo.

Retorno

Ponto 3D de interseção, ou nulo para uma reta paralela distinta do plano.

Exemplo

Código Coral

```
de coral.numerico importe intersecao_reta_plano3
mostre intersecao_reta_plano3([0,0,2], [0,0,-1], [0,0,0], [0,0,1])
```

Resultado

Saída esperada

```
[0. 0. 0.]
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

ErroNumerico para dimensões erradas, normal nula ou tolerância inválida.

`caminho_executavel(...)` · módulo `coral.sistema`

O que faz

Consulta o caminho informado pelo processo para o executável que hospeda o runtime, sem iniciar arquivos.

Assinatura

Assinatura para consulta

```
caminho_executavel() -> 'Path | None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Path do executável quando informado pelo sistema, ou nulo.

Exemplo

Código Coral

```
de coral.sistema importe caminho_executavel
mostre caminho_executavel() != nulo
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Não executa programas; a descoberta depende das informações expostas pelo processo.

`diagnosticar_sistema(...)` · módulo `coral.sistema`

O que faz

Reúne em um diagnóstico as informações de plataforma e capacidade observáveis pelo módulo Sistema.

Assinatura

Assinatura para consulta

```
diagnosticar_sistema() -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Dicionário com dados diagnósticos dependentes do ambiente.

Exemplo

Código Coral

```
de coral.sistema importe diagnosticar_sistema
mostre diagnosticar_sistema() != nulo
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Não altera configuração nem instala ferramentas; dados indisponíveis podem ficar ausentes.

`informacoes(...)` · módulo `coral.sistema`

O que faz

Consulta uma visão tipada de identificação e características do sistema corrente.

Assinatura

Assinatura para consulta

```
informacoes() -> 'InformacoesSistema'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Instância InformacoesSistema com campos sobre a plataforma observada.

Exemplo

Código Coral

```
de coral.sistema importe informacoes
mostre informacoes() != nulo
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Valores são informativos e não garantem disponibilidade de recursos opcionais.

`processo_atual(...)` · módulo `coral.sistema`

O que faz

Obtém dados sobre o processo corrente sem abrir ou encerrar outros processos.

Assinatura

Assinatura para consulta

```
processo_atual() -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Dicionário de informações do processo observado.

Exemplo

Código Coral

```
de coral.sistema importe processo_atual
mostre processo_atual() != nulo
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Alguns campos dependem do sistema operacional e de permissões disponíveis.

`quantidade_processadores(...)` · módulo `coral.sistema`

O que faz

Consulta o número de processadores lógicos reportado ao runtime.

Assinatura

Assinatura para consulta

```
quantidade_processadores() -> 'int | None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Inteiro positivo quando disponível, ou nulo caso não haja dado confiável.

Exemplo

Código Coral

```
de coral.sistema importe quantidade_processadores
mostre quantidade_processadores() != nulo
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Não interpreta cota de contêiner como capacidade física garantida.

`cpu(...)` · módulo `coral.hardware`

O que faz

Consulta métricas e propriedades observáveis da CPU pelo módulo Hardware.

Assinatura

Assinatura para consulta

```
cpu() -> 'CPU'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Registro CPU; alguns atributos podem ser indisponíveis na máquina consultada.

Exemplo

Código Coral

```
de coral.hardware importe cpu
mostre cpu() != nulo
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Uma dependência opcional ou sensor não disponível não implica que todos os campos possam ser obtidos.

`memoria(...)` · módulo `coral.hardware`

O que faz

Obtém uma fotografia da memória disponível e em uso segundo o backend de hardware.

Assinatura

Assinatura para consulta

```
memoria() -> 'Memoria'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Registro Memoria com métricas fornecidas pelo ambiente atual.

Exemplo

Código Coral

```
de coral.hardware importe memoria
mostre memoria() != nulo
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Valores de uso mudam com o tempo e podem variar entre sistemas.

`discos(...)` · módulo `coral.hardware`

O que faz

Consulta dispositivos ou partições de armazenamento detectados pelo backend.

Assinatura

Assinatura para consulta

```
discos() -> 'tuple[Disco, ...]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Tupla de registros Disco; pode estar vazia.

Exemplo

Código Coral

```
de coral.hardware importe discos
mostre discos() != nulo
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Partições inacessíveis e informações ausentes dependem de permissões e da plataforma.

`diagnosticar_jogos(...)` · módulo `coral.jogos`

O que faz

Informa dependências e capacidades disponibilizadas pelo subsistema Jogos.

Assinatura

Assinatura para consulta

```
diagnosticar_jogos() -> 'dict[str, object]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Dicionário de capacidades, útil para diagnosticar integrações opcionais.

Exemplo

Código Coral

```
de coral.jogos importe diagnosticar_jogos
mostre diagnosticar_jogos() != nulo
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

A consulta não cria janelas, não inicia o laço do jogo e não instala dependências.

`pygame_disponivel(...)` · módulo `coral.jogos`

O que faz

Consulta se a integração opcional Pygame pode ser utilizada no processo corrente.

Assinatura

Assinatura para consulta

```
pygame_disponivel() -> 'bool'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Booleano verdadeiro quando a dependência é detectada.

Exemplo

Código Coral

```
de coral.jogos importe pygame_disponivel
mostre pygame_disponivel() == pygame_disponivel()
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

O resultado não garante abertura de janela ou suporte a vídeo do sistema.

`normalizar_categorias_debug(...)` · módulo `coral.jogos`

O que faz

Converte rótulos ou coleções de categorias do overlay de depuração para um conjunto de identificadores canônicos.

Assinatura

Assinatura para consulta

```
normalizar_categorias_debug(categorias: 'Any' = None) -> 'frozenset[str]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
categorias	Any	não	None	Categoria única ou coleção de categorias de depuração.

Retorno

Conjunto imutável de categorias de debug reconhecidas.

Exemplo

Código Coral

```
de coral.jogos importe normalizar_categorias_debug
mostre normalizar_categorias_debug(["limites"])
```

Resultado

Saída esperada

```
frozenset({'limites'})
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Categorias desconhecidas ou malformadas seguem a política de validação de Jogos.

`resolver_curva_temporal(...)` · módulo `coral.jogos`

O que faz

Resolve o nome de uma função de easing de Jogos ou preserva uma função já fornecida.

Assinatura

Assinatura para consulta

```
resolver_curva_temporal(curva: 'str | Callable[[float], float]') -> 'Callable[[float],
  float]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
curva	str | Callable[[float], float]	sim	sem padrão	Nome da curva temporal ou função compatível de progressão.

Retorno

Função numérica que transforma a progressão de tempo em uma progressão interpolada.

Exemplo

Código Coral

```
de coral.jogos importe resolver_curva_temporal
mostre resolver_curva_temporal("linear")(0.5)
```

Resultado

Saída esperada

```
0.5
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

ValueError para identificador de curva não reconhecido; função personalizada precisa ser chamável.

`diagnosticar_laboratorio(...)` · módulo `coral.laboratorio`

O que faz

Produz um diagnóstico das capacidades, limites e recursos opcionais de Laboratório.

Assinatura

Assinatura para consulta

```
diagnosticar_laboratorio() -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Dicionário de capacidades e contratos observados.

Exemplo

Código Coral

```
de coral.laboratorio importe diagnosticar_laboratorio
mostre diagnosticar_laboratorio() != nulo
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Não executa experimentos nem grava resultados em disco.

`grade_parametros(...)` · módulo `coral.laboratorio`

O que faz

Gera as combinações cartesianas de valores de uma configuração de parâmetros para experimentos.

Assinatura

Assinatura para consulta

```
grade_parametros(parametros: 'Mapping[str, Iterable[Any]] | None') -> 'list[dict[str,
  Any]]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
parametros	Mapping[str, Iterable[Any]] | None	sim	sem padrão	Mapeamento de nomes para coleções de valores candidatos, ou nulo.

Retorno

Lista de dicionários, um por combinação de parâmetros, preservando o contrato do planejador.

Exemplo

Código Coral

```
de coral.laboratorio importe grade_parametros
mostre grade_parametros({"velocidade": [1,2]})
```

Resultado

Saída esperada

```
[{'velocidade': 1}, {'velocidade': 2}]
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Pode rejeitar iteráveis inválidos ou combinações que excedam limites de materialização.

`protocolo_medicao(...)` · módulo `coral.laboratorio`

O que faz

Consulta a definição do protocolo de medição e da fonte de tempo usada em Laboratório.

Assinatura

Assinatura para consulta

```
protocolo_medicao(relogio: 'FonteTempo | None' = None) -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
relogio	FonteTempo | None	não	None	Fonte de tempo opcional para registrar medições.

Retorno

Dicionário com características do relógio e da política de medição.

Exemplo

Código Coral

```
de coral.laboratorio importe protocolo_medicao
mostre protocolo_medicao() != nulo
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Uma fonte de tempo personalizada deve respeitar o protocolo esperado.

`barras(...)` · módulo `coral.graficos`

O que faz

Constrói gráfico de barras a partir de categorias e valores alinhados por posição.

Assinatura

Assinatura para consulta

```
barras(categorias: 'Iterable[Any]', valores: 'Iterable[Any]', nome: 'str | None' = None)
   -> 'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
categorias	Iterable[Any]	sim	sem padrão	Rótulos ordenados do eixo categórico.
valores	Iterable[Any]	sim	sem padrão	Altura numérica de cada barra.
nome	str | None	não	None	Rótulo opcional para a série.

Retorno

Grafico de tipo barras, pronto para exportação ou composição em painel.

Exemplo

Código Coral

```
de coral.graficos importe barras
mostre barras(["a","b"], [1,2]).tipo
```

Resultado

Saída esperada

```
barras
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Rejeita séries incompatíveis ou valores não representáveis conforme o contrato gráfico.

`barras_erro(...)` · módulo `coral.graficos`

O que faz

Constrói gráfico com valores e incertezas associadas às observações.

Assinatura

Assinatura para consulta

```
barras_erro(x: 'Iterable[Any]', y: 'Iterable[Any]', erros: 'Iterable[Any]', nome: 'str |
   None' = None) -> 'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	Iterable[Any]	sim	sem padrão	Coordenadas das observações.
y	Iterable[Any]	sim	sem padrão	Valores centrais de cada observação.
erros	Iterable[Any]	sim	sem padrão	Magnitude do erro ou incerteza em cada observação.
nome	str | None	não	None	Nome opcional da série.

Retorno

Grafico de tipo barras_erro.

Exemplo

Código Coral

```
de coral.graficos importe barras_erro
mostre barras_erro([1,2],[3,4],[0.1,0.2]).tipo
```

Resultado

Saída esperada

```
barras_erro
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Requer séries compatíveis e incertezas numéricas válidas.

`dispersao3d(...)` · módulo `coral.graficos`

O que faz

Constrói nuvem de pontos em três eixos espaciais a partir de sequências correspondentes.

Assinatura

Assinatura para consulta

```
dispersao3d(x: 'Iterable[Any]', y: 'Iterable[Any]', z: 'Iterable[Any]', nome: 'str |
  None' = None) -> 'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	Iterable[Any]	sim	sem padrão	Coordenadas do eixo X.
y	Iterable[Any]	sim	sem padrão	Coordenadas do eixo Y.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
z	Iterable[Any]	sim	sem padrão	Coordenadas do eixo Z.
nome	str | None	não	None	Nome opcional da série.

Retorno

Grafico tridimensional de tipo dispersao3d.

Exemplo

Código Coral

```
de coral.graficos importe dispersao3d
mostre dispersao3d([1,2],[3,4],[5,6]).tipo
```

Resultado

Saída esperada

```
dispersao3d
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Séries de tamanhos ou tipos incompatíveis não podem formar pontos 3D válidos.

`faixa_incerteza(...)` · módulo `coral.graficos`

O que faz

Cria uma faixa de incerteza com limites inferior e superior ao redor de uma curva central.

Assinatura

Assinatura para consulta

```
faixa_incerteza(x: 'Iterable[Any]', centro: 'Iterable[Any]', inferior: 'Iterable[Any]',
  superior: 'Iterable[Any]', nome: 'str | None' = None) -> 'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	Iterable[Any]	sim	sem padrão	Coordenadas do eixo horizontal.
centro	Iterable[Any]	sim	sem padrão	Valores centrais para cada abscissa.
inferior	Iterable[Any]	sim	sem padrão	Limites inferiores da faixa.
superior	Iterable[Any]	sim	sem padrão	Limites superiores da faixa.
nome	str | None	não	None	Nome opcional da série.

Retorno

Grafico com região de intervalo identificada como faixa_incerteza.

Exemplo

Código Coral

```
de coral.graficos importe faixa_incerteza
mostre faixa_incerteza([0,1],[2,3],[1,2],[3,4]).tipo
```

Resultado

Saída esperada

```
faixa_incerteza
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Os vetores precisam ter comprimentos compatíveis e representar limites consistentes.

`grafico_histograma(...)` · módulo `coral.graficos`

O que faz

Constrói um histograma da amostra usando faixas ou divisões declaradas.

Assinatura

Assinatura para consulta

```
grafico_histograma(valores: 'Iterable[Any]', faixas: 'Any', nome: 'str | None' = None)
  -> 'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	Iterable[Any]	sim	sem padrão	Observações a agrupar.
faixas	Any	sim	sem padrão	Limites explícitos ou configuração de agrupamento.
nome	str | None	não	None	Nome opcional da distribuição.

Retorno

Grafico de tipo histograma.

Exemplo

Código Coral

```
de coral.graficos importe grafico_histograma
mostre grafico_histograma([1,2,3],[0,2,4]).tipo
```

Resultado

Saída esperada

```
histograma
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Faixas inválidas ou dados incompatíveis geram falha de validação.

`mapa_calor(...)` · módulo `coral.graficos`

O que faz

Transforma uma grade bidimensional de valores em visualização por intensidade.

Assinatura

Assinatura para consulta

```
mapa_calor(matriz: 'Any', rotulos_linhas: 'Iterable[Any] | None' = None,
  rotulos_colunas: 'Iterable[Any] | None' = None, nome: 'str | None' = None) -> 'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
matriz	Any	sim	sem padrão	Grade de valores numéricos a representar.
rotulos_linhas	Iterable[Any] | None	não	None	Rótulos opcionais das linhas.
rotulos_colunas	Iterable[Any] | None	não	None	Rótulos opcionais das colunas.
nome	str | None	não	None	Identificador opcional da série.

Retorno

Grafico de tipo mapa_calor.

Exemplo

Código Coral

```
de coral.graficos importe mapa_calor
mostre mapa_calor([[1,2],[3,4]]).tipo
```

Resultado

Saída esperada

```
mapa_calor
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Matriz irregular e rótulos de dimensões incompatíveis são recusados.

`painel(...)` · módulo `coral.graficos`

O que faz

Agrupa vários gráficos numa composição de múltiplas células organizada em colunas.

Assinatura

Assinatura para consulta

```
painel(graficos: 'Iterable[Grafico]', colunas: 'int' = 2, titulo: 'str' = '') ->
  'PainelGraficos'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
graficos	Iterable[Grafico]	sim	sem padrão	Gráficos que compõem o painel em ordem.
colunas	int	não	2	Número de colunas da disposição.
titulo	str	não	''	Texto apresentado no cabeçalho do painel.

Retorno

PainelGraficos contendo os gráficos informados.

Exemplo

Código Coral

```
de coral.graficos importe painel
de coral.graficos importe barras
mostre painel([barras(["a"],[1])], 1, "Resumo").titulo
```

Resultado

Saída esperada

```
Resumo
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

A largura do painel em colunas deve ser válida; elementos precisam ser gráficos aceitos.

`para_svg(...)` · módulo `coral.graficos`

O que faz

Exporta gráfico ou painel para uma imagem vetorial SVG em memória.

Assinatura

Assinatura para consulta

```
para_svg(grafico: 'Grafico | PainelGraficos', *, largura: 'int' = 800, altura: 'int' =
  500) -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grafico	Grafico | PainelGraficos	sim	sem padrão	Grafico ou PainelGraficos a exportar.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
largura	int	não	800	Largura da tela vetorial em unidades SVG.
altura	int	não	500	Altura da tela vetorial em unidades SVG.

Retorno

Texto SVG completo, sem exigir arquivo de destino.

Exemplo

Código Coral

```
de coral.graficos importe para_svg
de coral.graficos importe barras
mostre para_svg(barras(["a"],[1]))[0:4]
```

Resultado

Saída esperada

```
<?xm
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Dimensões de desenho não positivas ou gráfico incompatível são recusados.

`para_html(...)` · módulo `coral.graficos`

O que faz

Exporta gráfico como página HTML autocontida para visualização local.

Assinatura

Assinatura para consulta

```
para_html(grafico: 'Any', *, limite_pontos: 'int' = 200000) -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grafico	Any	sim	sem padrão	Gráfico a serializar em HTML.
limite_pontos	int	não	200000	Número máximo de observações aceito na exportação.

Retorno

Texto HTML completo contendo a visualização.

Exemplo

Código Coral

```
de coral.graficos importe para_html, barras
defina pagina como para_html(barras(["a"], [1]))
mostre pagina.find("<html") >= 0
mostre pagina.find("coral.graficos.html/1") >= 0
```

Resultado

Saída esperada

```
True
True
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Limite de pontos impede exportação excessiva; dados incompatíveis são recusados.

Atenção

O prefixo textual do documento pode variar conforme a versão do Plotly. O exemplo verifica a presença da raiz HTML e do contrato de acessibilidade da Coral, sem depender de um prefixo específico.

`series_de_registros(...)` · módulo `coral.graficos`

O que faz

Extrai pares X e Y de registros tabulares usando nomes de campos.

Assinatura

Assinatura para consulta

```
series_de_registros(registros: 'Iterable[Any]', campo_x: 'str', campo_y: 'str') ->
  'SeriesRegistros'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
registros	Iterable[Any]	sim	sem padrão	Registros tabulares a percorrer.
campo_x	str	sim	sem padrão	Nome da propriedade para o eixo horizontal.
campo_y	str	sim	sem padrão	Nome da propriedade para o eixo vertical.

Retorno

SeriesRegistros com os valores da seleção.

Exemplo

Código Coral

```
de coral.graficos importe series_de_registros
mostre series_de_registros([{"t":1,"v":2}], "t", "v") != nulo
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Registros sem campos exigidos ou tipos incompatíveis podem causar erro de validação.

`resposta_html(...)` · módulo `coral.web.servidor`

O que faz

Cria uma resposta HTTP em memória para conteúdo HTML, com tipo de conteúdo apropriado.

Assinatura

Assinatura para consulta

```
resposta_html(conteudo: 'Any', status: 'int' = 200, cabecalhos: 'Any' = None) ->
  'RespostaServidorHTTP'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
conteudo	Any	sim	sem padrão	Texto ou conteúdo HTML a enviar.
status	int	não	200	Código HTTP, normalmente 200.
cabecalhos	Any	não	None	Cabeçalhos adicionais da resposta.

Retorno

RespostaServidorHTTP com status, corpo e cabeçalhos normalizados.

Exemplo

Código Coral

```
de coral.web.servidor importe resposta_html
mostre resposta_html("<p>Olá</p>").status
```

Resultado

Saída esperada

```
200
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Status inválido ou conteúdo não suportado é recusado pelo normalizador.

`rolar(...)` · módulo `coral.rpg`

O que faz

Interpreta expressão de dados como 2d6 e executa rolagem com gerador controlável.

Assinatura

Assinatura para consulta

```
rolar(expressao: 'str' = '1d20', semente: 'int | None' = None, *, gerador:
  'random.Random | None' = None) -> 'Rolagem'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
expressao	str	não	'1d20'	Fórmula da rolagem, como 2d6 ou 1d20.
semente	int | None	não	None	Semente opcional para repetibilidade.
gerador	random.Random | None	não	None	Gerador aleatório fornecido explicitamente, alternativo à semente.

Retorno

Rolagem que contém resultado total e detalhes dos dados.

Exemplo

Código Coral

```
de coral.rpg importe rolar
mostre rolar("2d6",42).total
```

Resultado

Saída esperada

```
7
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Expressão malformada ou lados/quantidades inválidas são recusados.

`amostrar_campo(...)` · módulo `coral.procedural`

O que faz

Amostra um campo procedural em pontos explícitos, produzindo registros de coordenadas e valores portáteis.

Assinatura

Assinatura para consulta

```
amostrar_campo(campo: 'CampoProcedural', coordenadas: 'Iterable[Sequence[float]]', *,
  nomes_eixos: 'Sequence[str] | None' = None, nome_valor: 'str' = 'valor',
  limite_amostras: 'int' = 100000) -> 'tuple[dict[str, Any], ...]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
campo	CampoProcedural	sim	sem padrão	CampoProcedural a consultar sem alterá-lo.
coordenadas	Iterable[Sequence[float]]	sim	sem padrão	Pontos de amostragem em ordem.
nomes_eixos	Sequence[str] | None	não	None	Rótulos opcionais das dimensões espaciais.
nome_valor	str	não	'valor'	Nome do campo de saída que armazena a amostra.
limite_amostras	int	não	100000	Quantidade máxima de pontos autorizada.

Retorno

Tupla de dicionários com índice, eixos e valor amostrado.

Exemplo

Código Coral

```
de coral.procedural importe amostrar_campo
de coral.procedural importe campo_constante
mostre amostrar_campo(campo_constante(2), [[0,0]])
```

Resultado

Saída esperada

```
({'indice': 0, 'x': 0.0, 'y': 0.0, 'valor': 2.0},)
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Limite de amostras e formato de coordenadas são verificados antes de materializar dados.

`diagnosticar_inspecao(...)` · módulo `coral.inspecao`

O que faz

Analisa se um objeto pode fornecer snapshot normalizado e quais visões de inspeção estão disponíveis.

Assinatura

Assinatura para consulta

```
diagnosticar_inspecao(objeto: 'Any') -> 'DiagnosticoInspecao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto a diagnosticar por meio das capacidades públicas de inspeção.

Retorno

DiagnosticoInspecao com dados sobre o objeto observado.

Exemplo

Código Coral

```
de coral.inspecao importe diagnosticar_inspecao
mostre diagnosticar_inspecao(7).tipo
```

Resultado

Saída esperada

```
int
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Pode sinalizar ausência de capacidades específicas sem modificar o objeto.

`comparar_inspecoes(...)` · módulo `coral.inspecao`

O que faz

Compara dois estados normalizados e identifica diferenças de propriedades observáveis.

Assinatura

Assinatura para consulta

```
comparar_inspecoes(antes: 'Any', depois: 'Any', *, incluir: 'tuple[str, ...] |
  list[str]' = (), ignorar: 'tuple[str, ...] | list[str]' = ()) -> 'DiferencaInspecao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
antes	Any	sim	sem padrão	Objeto ou snapshot no estado inicial.
depois	Any	sim	sem padrão	Objeto ou snapshot no estado final.
incluir	tuple[str, ...] | list[str]	não	()	Caminhos opcionais a comparar explicitamente.
ignorar	tuple[str, ...] | list[str]	não	()	Caminhos a descartar do comparativo.

Retorno

DiferencaInspecao com as alterações encontradas.

Exemplo

Código Coral

```
de coral.inspecao importe comparar_inspecoes
mostre comparar_inspecoes(1,2) != nulo
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Caminhos de inclusão ou exclusão malformados não definem um filtro válido.

`relatorio_inspecao(...)` · módulo `coral.inspecao`

O que faz

Renderiza relatório legível de um snapshot e dos caminhos de dados solicitados.

Assinatura

Assinatura para consulta

```
relatorio_inspecao(objeto: 'Any', *, visao: 'str | None' = None, caminhos: 'list[str] |
  tuple[str, ...]' = ()) -> 'RelatorioInspecao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto ou snapshot a relatar.
visao	str | None	não	None	Visão semântica opcional.
caminhos	list[str] | tuple[str, ...]	não	()	Lista opcional de campos a destacar.

Retorno

RelatorioInspecao com visão e dados apresentados sem mutação.

Exemplo

Código Coral

```
de coral.inspecao importe relatorio_inspecao
mostre relatorio_inspecao(1) != nulo
```

Resultado

Saída esperada

```
True
```

Forma natural

Sem forma natural exclusiva; importe a função pública do módulo correspondente.

Erros

Visão desconhecida ou caminho inválido pode produzir erro de validação.

`ErroSerial(...)` · módulo `coral.hardware.serial`

O que faz

Representa a falha operacional base da comunicação serial.

Construtor

Assinatura para consulta

```
ErroSerial(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos posicionais fornecidos à exceção.

Retorno

Nova instância de ErroSerial.

Exemplo

Código Coral

```
de coral.hardware.serial importe ErroSerial
defina erro como ErroSerial("Falha serial de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha serial de exemplo
```

Erros

Instanciar a exceção não a lança; a falha ocorre quando uma operação a levanta.

Atenção

Capturar esta classe também captura as falhas seriais especializadas. O construtor herdado aceita argumentos posicionais da exceção; o exemplo fornece uma mensagem textual. A introspecção não recupera sua assinatura.

`ErroSerial(...)` · módulo `coral.hardware`

O que faz

Representa a falha operacional base da comunicação serial.

Construtor

Assinatura para consulta

```
ErroSerial(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos posicionais fornecidos à exceção.

Retorno

Nova instância de ErroSerial.

Exemplo

Código Coral

```
de coral.hardware importe ErroSerial
defina erro como ErroSerial("Falha serial de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha serial de exemplo
```

Erros

Instanciar a exceção não a lança; a falha ocorre quando uma operação a levanta.

Atenção

Capturar esta classe também captura as falhas seriais especializadas. O construtor herdado aceita argumentos posicionais da exceção; o exemplo fornece uma mensagem textual. A introspecção não recupera sua assinatura.

`DependenciaSerialAusente(...)` · módulo `coral.hardware.serial`

O que faz

Indica que o backend serial real não consegue carregar pyserial.

Construtor

Assinatura para consulta

```
DependenciaSerialAusente(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos posicionais fornecidos à exceção.

Retorno

Nova instância de DependenciaSerialAusente.

Exemplo

Código Coral

```
de coral.hardware.serial importe DependenciaSerialAusente
defina erro como DependenciaSerialAusente("Falha serial de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha serial de exemplo
```

Erros

Instanciar a exceção não a lança; a falha ocorre quando uma operação a levanta.

Atenção

Importar a classe não carrega pyserial; a dependência é exigida pela operação real. O construtor herdado aceita argumentos posicionais da exceção; o exemplo fornece uma mensagem textual. A introspecção não recupera sua assinatura.

`DependenciaSerialAusente(...)` · módulo `coral.hardware`

O que faz

Indica que o backend serial real não consegue carregar pyserial.

Construtor

Assinatura para consulta

```
DependenciaSerialAusente(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos posicionais fornecidos à exceção.

Retorno

Nova instância de DependenciaSerialAusente.

Exemplo

Código Coral

```
de coral.hardware importe DependenciaSerialAusente
defina erro como DependenciaSerialAusente("Falha serial de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha serial de exemplo
```

Erros

Instanciar a exceção não a lança; a falha ocorre quando uma operação a levanta.

Atenção

Importar a classe não carrega pyserial; a dependência é exigida pela operação real. O construtor herdado aceita argumentos posicionais da exceção; o exemplo fornece uma mensagem textual. A introspecção não recupera sua assinatura.

`ErroConfiguracaoSerial(...)` · módulo `coral.hardware.serial`

O que faz

Indica porta, velocidade, timeout, quantidade ou codificação serial inválidos.

Construtor

Assinatura para consulta

```
ErroConfiguracaoSerial(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos posicionais fornecidos à exceção.

Retorno

Nova instância de ErroConfiguracaoSerial.

Exemplo

Código Coral

```
de coral.hardware.serial importe ErroConfiguracaoSerial
defina erro como ErroConfiguracaoSerial("Falha serial de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha serial de exemplo
```

Erros

Instanciar a exceção não a lança; a falha ocorre quando uma operação a levanta.

Atenção

As validações de configuração ocorrem antes de abrir a porta quando aplicáveis. O construtor herdado aceita argumentos posicionais da exceção; o exemplo fornece uma mensagem textual. A introspecção não recupera sua assinatura.

`ErroConfiguracaoSerial(...)` · módulo `coral.hardware`

O que faz

Indica porta, velocidade, timeout, quantidade ou codificação serial inválidos.

Construtor

Assinatura para consulta

```
ErroConfiguracaoSerial(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos posicionais fornecidos à exceção.

Retorno

Nova instância de ErroConfiguracaoSerial.

Exemplo

Código Coral

```
de coral.hardware importe ErroConfiguracaoSerial
defina erro como ErroConfiguracaoSerial("Falha serial de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha serial de exemplo
```

Erros

Instanciar a exceção não a lança; a falha ocorre quando uma operação a levanta.

Atenção

As validações de configuração ocorrem antes de abrir a porta quando aplicáveis. O construtor herdado aceita argumentos posicionais da exceção; o exemplo fornece uma mensagem textual. A introspecção não recupera sua assinatura.

`ConexaoSerialFechada(...)` · módulo `coral.hardware.serial`

O que faz

Indica uma operação tentada sobre uma conexão serial já fechada.

Construtor

Assinatura para consulta

```
ConexaoSerialFechada(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos posicionais fornecidos à exceção.

Retorno

Nova instância de ConexaoSerialFechada.

Exemplo

Código Coral

```
de coral.hardware.serial importe ConexaoSerialFechada
defina erro como ConexaoSerialFechada("Falha serial de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha serial de exemplo
```

Erros

Instanciar a exceção não a lança; a falha ocorre quando uma operação a levanta.

Atenção

O fechamento deliberado é distinto da desconexão do dispositivo. O construtor herdado aceita argumentos posicionais da exceção; o exemplo fornece uma mensagem textual. A introspecção não recupera sua assinatura.

`ConexaoSerialFechada(...)` · **módulo** `coral.hardware`

O que faz

Indica uma operação tentada sobre uma conexão serial já fechada.

Construtor

Assinatura para consulta

```
ConexaoSerialFechada(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos posicionais fornecidos à exceção.

Retorno

Nova instância de ConexaoSerialFechada.

Exemplo

Código Coral

```
de coral.hardware importe ConexaoSerialFechada
defina erro como ConexaoSerialFechada("Falha serial de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha serial de exemplo
```

Erros

Instanciar a exceção não a lança; a falha ocorre quando uma operação a levanta.

Atenção

O fechamento deliberado é distinto da desconexão do dispositivo. O construtor herdado aceita argumentos posicionais da exceção; o exemplo fornece uma mensagem textual. A introspecção não recupera sua assinatura.

`DesconexaoSerial(...)` · **módulo** `coral.hardware.serial`

O que faz

Indica perda da conexão com a porta durante a sessão serial.

Construtor

Assinatura para consulta

```
DesconexaoSerial(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos posicionais fornecidos à exceção.

Retorno

Nova instância de DesconexaoSerial.

Exemplo

Código Coral

```
de coral.hardware.serial importe DesconexaoSerial
defina erro como DesconexaoSerial("Falha serial de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha serial de exemplo
```

Erros

Instanciar a exceção não a lança; a falha ocorre quando uma operação a levanta.

Atenção

No backend simulado, a retirada da porta também produz esta falha. O construtor herdado aceita argumentos posicionais da exceção; o exemplo fornece uma mensagem textual. A introspecção não recupera sua assinatura.

`DesconexaoSerial(...)` · módulo `coral.hardware`

O que faz

Indica perda da conexão com a porta durante a sessão serial.

Construtor

Assinatura para consulta

```
DesconexaoSerial(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos posicionais fornecidos à exceção.

Retorno

Nova instância de DesconexaoSerial.

Exemplo

Código Coral

```
de coral.hardware importe DesconexaoSerial
defina erro como DesconexaoSerial("Falha serial de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha serial de exemplo
```

Erros

Instanciar a exceção não a lança; a falha ocorre quando uma operação a levanta.

Atenção

No backend simulado, a retirada da porta também produz esta falha. O construtor herdado aceita argumentos posicionais da exceção; o exemplo fornece uma mensagem textual. A introspecção não recupera sua assinatura.

`ErroSistema(...)` · módulo `coral.sistema`

O que faz

Representa uma falha operacional do sistema hospedeiro.

Construtor

Assinatura para consulta

```
ErroSistema(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos usados na construção da exceção.

Retorno

Nova instância de ErroSistema.

Exemplo

Código Coral

```
de coral.sistema importe ErroSistema
defina erro como ErroSistema("Falha de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha de exemplo
```

Erros

Construir esta exceção não a lança automaticamente.

Atenção

Base comum das falhas de execução de processos e leitura de configuração obrigatória. O construtor herdado aceita argumentos posicionais; uma mensagem textual é a forma usada no exemplo. A introspecção não recupera sua assinatura.

`ProgramaNaoEncontrado(...)` · módulo `coral.sistema`

O que faz

Indica que o executável solicitado não foi encontrado.

Construtor

Assinatura para consulta

```
ProgramaNaoEncontrado(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos usados na construção da exceção.

Retorno

Nova instância de ProgramaNaoEncontrado.

Exemplo

Código Coral

```
de coral.sistema importe ProgramaNaoEncontrado
defina erro como ProgramaNaoEncontrado("Falha de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha de exemplo
```

Erros

Construir esta exceção não a lança automaticamente.

Atenção

Também pode ser capturada como FileNotFoundError em integrações Python. O construtor herdado aceita argumentos posicionais; uma mensagem textual é a forma usada no exemplo. A introspecção não recupera sua assinatura.

`FalhaSistemaOperacional(...)` · módulo `coral.sistema`

O que faz

Representa demais falhas do sistema operacional traduzidas pela Coral.

Construtor

Assinatura para consulta

```
FalhaSistemaOperacional(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos usados na construção da exceção.

Retorno

Nova instância de FalhaSistemaOperacional.

Exemplo

Código Coral

```
de coral.sistema importe FalhaSistemaOperacional
defina erro como FalhaSistemaOperacional("Falha de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha de exemplo
```

Erros

Construir esta exceção não a lança automaticamente.

Atenção

Também mantém compatibilidade com captura por OSError em integrações Python. O construtor herdado aceita argumentos posicionais; uma mensagem textual é a forma usada no exemplo. A introspecção não recupera sua assinatura.

`ErroNumerico(...)` · módulo `coral.numerico`

O que faz

Representa entrada numérica inválida ou operação numérica incompatível.

Construtor

Assinatura para consulta

```
ErroNumerico(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos usados na construção da exceção.

Retorno

Nova instância de ErroNumerico.

Exemplo

Código Coral

```
de coral.numerico importe ErroNumerico
defina erro como ErroNumerico("Falha de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha de exemplo
```

Erros

Construir esta exceção não a lança automaticamente.

Atenção

Compatível com ValueError e com a hierarquia ErroCoral. O construtor herdado aceita argumentos posicionais; uma mensagem textual é a forma usada no exemplo. A introspecção não recupera sua assinatura.

`DependenciaNumericaAusente(...)` · módulo `coral.numerico`

O que faz

Indica que uma operação numérica precisa de NumPy e não consegue carregá lo.

Construtor

Assinatura para consulta

```
DependenciaNumericaAusente(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos usados na construção da exceção.

Retorno

Nova instância de DependenciaNumericaAusente.

Exemplo

Código Coral

```
de coral.numerico importe DependenciaNumericaAusente
defina erro como DependenciaNumericaAusente("Falha de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha de exemplo
```

Erros

Construir esta exceção não a lança automaticamente.

Atenção

Importar o módulo numérico não exige NumPy; a operação dependente faz a verificação. O construtor herdado aceita argumentos posicionais; uma mensagem textual é a forma usada no exemplo. A introspecção não recupera sua assinatura.

`ErroGrafico(...)` · módulo `coral.graficos`

O que faz

Representa especificação gráfica inválida ou backend gráfico opcional indisponível.

Construtor

Assinatura para consulta

```
ErroGrafico(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos usados na construção da exceção.

Retorno

Nova instância de ErroGrafico.

Exemplo

Código Coral

```
de coral.graficos importe ErroGrafico
defina erro como ErroGrafico("Falha de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha de exemplo
```

Erros

Construir esta exceção não a lança automaticamente.

Atenção

Compatível com ValueError; consulte a mensagem para distinguir dados inválidos de dependência ausente. O construtor herdado aceita argumentos posicionais; uma mensagem textual é a forma usada no exemplo. A introspecção não recupera sua assinatura.

`ErroPersistencia(...)` · módulo `coral.persistencia`

O que faz

Representa falha de validação, codificação ou reconstrução de dados persistentes.

Construtor

Assinatura para consulta

```
ErroPersistencia(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos usados na construção da exceção.

Retorno

Nova instância de ErroPersistencia.

Exemplo

Código Coral

```
de coral.persistencia importe ErroPersistencia
defina erro como ErroPersistencia("Falha de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha de exemplo
```

Erros

Construir esta exceção não a lança automaticamente.

Atenção

Compatível com ValueError. A mensagem informa a etapa ou o dado recusado. O construtor herdado aceita argumentos posicionais; uma mensagem textual é a forma usada no exemplo. A introspecção não recupera sua assinatura.

`DependenciaHardwareAusente(...)` · módulo `coral.hardware`

O que faz

Indica que a consulta de hardware precisa da dependência opcional psutil indisponível.

Construtor

Assinatura para consulta

```
DependenciaHardwareAusente(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos usados na construção da exceção.

Retorno

Nova instância de DependenciaHardwareAusente.

Exemplo

Código Coral

```
de coral.hardware importe DependenciaHardwareAusente
defina erro como DependenciaHardwareAusente("Falha de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha de exemplo
```

Erros

Construir esta exceção não a lança automaticamente.

Atenção

Importar a classe não realiza consultas nem carrega a dependência. O construtor herdado aceita argumentos posicionais; uma mensagem textual é a forma usada no exemplo. A introspecção não recupera sua assinatura.

`DependenciaJogosAusente(...)` · módulo `coral.jogos`

O que faz

Indica que uma capacidade de jogos precisa de um backend opcional indisponível.

Construtor

Assinatura para consulta

```
DependenciaJogosAusente(...)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Atributos importantes

Atributo	Significado
`args`	Tupla dos argumentos usados na construção da exceção.

Retorno

Nova instância de DependenciaJogosAusente.

Exemplo

Código Coral

```
de coral.jogos importe DependenciaJogosAusente
defina erro como DependenciaJogosAusente("Falha de exemplo")
mostre erro
```

Resultado

Saída esperada

```
Falha de exemplo
```

Erros

Construir esta exceção não a lança automaticamente.

Atenção

A importação da classe não abre janela nem inicia o jogo. O construtor herdado aceita argumentos posicionais; uma mensagem textual é a forma usada no exemplo. A introspecção não recupera sua assinatura.

`TempoLimiteExcedido(...)` · módulo `coral.sistema`

O que faz

Registra que um processo ultrapassou o prazo permitido.

Construtor

Assinatura para consulta

```
TempoLimiteExcedido(comando: 'tuple[str, ...]', timeout: 'float | None', mensagem:
  'str')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
comando	tuple[str, ...]	sim	sem padrão	Sequência com o executável e seus argumentos.
timeout	float | None	sim	sem padrão	Prazo em segundos associado à execução, ou nulo.
mensagem	str	sim	sem padrão	Mensagem pública apresentada em português.

Atributos importantes

Atributo	Significado
`cmd`	Comando informado.
`timeout`	Prazo informado.
`output`	Saída associada ao timeout, inicialmente nulo.
`stderr`	Erro associado ao timeout, inicialmente nulo.

Retorno

Nova instância de TempoLimiteExcedido.

Exemplo

Código Coral

```
de coral.sistema importe TempoLimiteExcedido
defina erro como TempoLimiteExcedido(["programa"], 2.0, "Prazo excedido")
mostre erro
mostre erro.timeout
```

Resultado

Saída esperada

```
Prazo excedido
2.0
```

Atenção

Construir a exceção não executa o comando e não espera pelo timeout. output e stderr começam como nulo.

`ComandoFalhou(...)` · módulo `coral.sistema`

O que faz

Preserva o resultado de um processo cujo código de saída não é zero.

Construtor

Assinatura para consulta

```
ComandoFalhou(resultado: "'ResultadoComando'")
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
resultado	'ResultadoComando'	sim	sem padrão	ResultadoComando da execução malsucedida.

Atributos importantes

Atributo	Significado
`resultado`	Resultado completo da execução.
`codigo`	Código de saída copiado do resultado.
`stderr`	Texto do canal de erro copiado do resultado.

Retorno

Nova instância de ComandoFalhou.

Exemplo

Código Coral

```
de coral.sistema importe ComandoFalhou
de coral.sistema importe ResultadoComando
defina resultado como ResultadoComando(["programa"], 3, "", "Entrada inválida", 0.0)
defina erro como ComandoFalhou(resultado)
mostre erro.codigo
mostre erro.stderr
mostre erro
```

Resultado

Saída esperada

```
3
Entrada inválida
O comando falhou com código 3: Entrada inválida
```

Erros

Um objeto sem codigo ou erro produz AttributeError; forneça ResultadoComando.

Atenção

O construtor guarda o resultado e não executa o processo. A exigência de sucesso do ResultadoComando lança esta classe quando o código é diferente de zero.

`InformacoesSistema(...)` · **módulo** `coral.sistema`

O que faz

Agrupa um retrato das informações do processo e da plataforma.

Construtor

Assinatura para consulta

```
InformacoesSistema(sistema: 'str', versao: 'str', arquitetura: 'str', maquina: 'str',
  processador: 'str', python: 'str', pid: 'int', pasta_atual: 'str') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
sistema	str	sim	sem padrão	Nome do sistema operacional.
versao	str	sim	sem padrão	Versão do sistema operacional.
arquitetura	str	sim	sem padrão	Arquitetura reportada pela plataforma.
maquina	str	sim	sem padrão	Nome da máquina.
processador	str	sim	sem padrão	Identificação textual do processador.
python	str	sim	sem padrão	Versão do interpretador Python.
pid	int	sim	sem padrão	Identificador inteiro do processo.
pasta_atual	str	sim	sem padrão	Caminho textual da pasta de trabalho.

Atributos importantes

Atributo	Significado
`sistema`	Nome do sistema operacional.
`versao`	Versão do sistema operacional.
`arquitetura`	Arquitetura reportada pela plataforma.
`maquina`	Nome da máquina.
`processador`	Identificação textual do processador.
`python`	Versão do interpretador Python.
`pid`	Identificador inteiro do processo.
`pasta_atual`	Caminho textual da pasta de trabalho.

Métodos principais

Assinatura para consulta

```
para_dict(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de InformacoesSistema.

Exemplo

Código Coral

```
de coral.sistema importe InformacoesSistema
defina info como InformacoesSistema("Linux", "exemplo", "x86_64", "notebook", "CPU",
  "3", 42, "/tmp")
mostre info.sistema
mostre info.para_dict()["pid"]
```

Resultado

Saída esperada

```
Linux
42
```

Atenção

Registro imutável. Construí lo não consulta a máquina; informacoes() preenche os valores reais. O construtor não valida o conteúdo dos campos.

`ResultadoMinimosQuadrados(...)` · módulo `coral.numerico`

O que faz

Armazena a solução e os diagnósticos de um ajuste por mínimos quadrados.

Construtor

Assinatura para consulta

```
ResultadoMinimosQuadrados(solucao: 'Any', residuos: 'Any', posto: 'int',
  valores_singulares: 'Any') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
solucao	Any	sim	sem padrão	Vetor da solução calculada.
residuos	Any	sim	sem padrão	Somas dos resíduos quadráticos devolvidas pelo NumPy; podem ser vazias.
posto	int	sim	sem padrão	Posto inteiro da matriz usada no ajuste.
valores_singulares	Any	sim	sem padrão	Valores singulares calculados para a matriz.

Atributos importantes

Atributo	Significado
`solucao`	Vetor da solução calculada.
`residuos`	Somas dos resíduos quadráticos devolvidas pelo NumPy; podem ser vazias.
`posto`	Posto inteiro da matriz usada no ajuste.
`valores_singulares`	Valores singulares calculados para a matriz.

Retorno

Nova instância de ResultadoMinimosQuadrados.

Exemplo

Código Coral

```
de coral.numerico importe ResultadoMinimosQuadrados
defina resultado como ResultadoMinimosQuadrados([2.0], [], 1, [1.0])
mostre resultado.posto
```

Resultado

Saída esperada

```
1
```

Atenção

Registro imutável. Resíduos vazios podem indicar ausência de graus de liberdade; não significam automaticamente ajuste perfeito. O construtor aceita os valores fornecidos sem conversão automática.

`ResultadoRegressaoLinear(...)` · módulo `coral.numerico`

O que faz

Armazena o ajuste linear simples e suas medidas de qualidade.

Construtor

Assinatura para consulta

```
ResultadoRegressaoLinear(inclinacao: 'float', intercepto: 'float', residuos: 'Any',
  r_quadrado: 'float', quantidade: 'int') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
inclinacao	float	sim	sem padrão	Coeficiente multiplicador de x na reta ajustada.
intercepto	float	sim	sem padrão	Valor previsto quando x é zero.
residuos	Any	sim	sem padrão	Diferenças entre valores observados e valores ajustados.
r_quadrado	float	sim	sem padrão	Coeficiente de determinação calculado para o ajuste.
quantidade	int	sim	sem padrão	Número de observações usadas.

Atributos importantes

Atributo	Significado
`inclinacao`	Coeficiente multiplicador de x na reta ajustada.
`intercepto`	Valor previsto quando x é zero.
`residuos`	Diferenças entre valores observados e valores ajustados.
`r_quadrado`	Coeficiente de determinação calculado para o ajuste.
`quantidade`	Número de observações usadas.

Retorno

Nova instância de ResultadoRegressaoLinear.

Exemplo

Código Coral

```
de coral.numerico importe ResultadoRegressaoLinear
defina resultado como ResultadoRegressaoLinear(2.0, 1.0, [0.0, 0.0], 1.0, 2)
mostre resultado.inclinacao
```

Resultado

Saída esperada

```
2.0
```

Atenção

Registro imutável. Construir o resultado não ajusta uma reta; use regressao_linear para calcular os campos. O construtor aceita os valores fornecidos sem conversão automática.

`ResultadoBootstrap(...)` · **módulo** `coral.numerico`

O que faz

Agrupa estimativa e intervalo percentil de uma reamostragem bootstrap.

Construtor

Assinatura para consulta

```
ResultadoBootstrap(estimativa: 'float', inferior: 'float', superior: 'float', confianca:
   'float', repeticoes: 'int', semente: 'int', tamanho_amostra: 'int', custo_estimado:
  'int', metodo: 'str' = 'bootstrap_percentil') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
estimativa	float	sim	sem padrão	Estatística estimada na amostra original.
inferior	float	sim	sem padrão	Limite inferior do intervalo percentil.
superior	float	sim	sem padrão	Limite superior do intervalo percentil.
confianca	float	sim	sem padrão	Nível de confiança usado na reamostragem.
repeticoes	int	sim	sem padrão	Quantidade de reamostragens realizadas.
semente	int	sim	sem padrão	Semente usada para reproduzir o sorteio.
tamanho_amostra	int	sim	sem padrão	Quantidade de valores na amostra original.
custo_estimado	int	sim	sem padrão	Estimativa de trabalho registrada pela operação.
metodo	str	não	'bootstrap_percentil'	Nome do método; o padrão identifica bootstrap_percentil.

Atributos importantes

Atributo	Significado
`estimativa`	Estatística estimada na amostra original.
`inferior`	Limite inferior do intervalo percentil.
`superior`	Limite superior do intervalo percentil.
`confianca`	Nível de confiança usado na reamostragem.
`repeticoes`	Quantidade de reamostragens realizadas.
`semente`	Semente usada para reproduzir o sorteio.
`tamanho_amostra`	Quantidade de valores na amostra original.
`custo_estimado`	Estimativa de trabalho registrada pela operação.
`metodo`	Nome do método; o padrão identifica bootstrap_percentil.

Retorno

Nova instância de ResultadoBootstrap.

Exemplo

Código Coral

```
de coral.numerico importe ResultadoBootstrap
defina resultado como ResultadoBootstrap(2.0, 1.0, 3.0, 0.95, 100, 7, 3, 300)
mostre resultado.metodo
```

Resultado

Saída esperada

```
bootstrap_percentil
```

Atenção

Registro imutável. Construir a classe não realiza reamostragem nem valida os limites; intervalo_bootstrap calcula e valida o resultado. O construtor aceita os valores fornecidos sem conversão automática.

`ResultadoTestePermutacao(...)` · módulo `coral.numerico`

O que faz

Agrupa a diferença observada e o valor p de um teste bilateral de permutação.

Construtor

Assinatura para consulta

```
ResultadoTestePermutacao(diferenca_observada: 'float', valor_p: 'float', repeticoes:
  'int', semente: 'int', tamanho_a: 'int', tamanho_b: 'int', custo_estimado: 'int',
  metodo: 'str' = 'permutacao_bilateral') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
diferenca_observada	float	sim	sem padrão	Diferença de médias observada entre as amostras.
valor_p	float	sim	sem padrão	Probabilidade empírica estimada pelo teste.
repeticoes	int	sim	sem padrão	Quantidade de permutações realizadas.
semente	int	sim	sem padrão	Semente usada para reproduzir as permutações.
tamanho_a	int	sim	sem padrão	Quantidade de observações da primeira amostra.
tamanho_b	int	sim	sem padrão	Quantidade de observações da segunda amostra.
custo_estimado	int	sim	sem padrão	Estimativa de trabalho registrada pela operação.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
metodo	str	não	'permutacao_bilateral'	Nome do método; o padrão identifica permutacao_bilateral.

Atributos importantes

Atributo	Significado
`diferenca_observada`	Diferença de médias observada entre as amostras.
`valor_p`	Probabilidade empírica estimada pelo teste.
`repeticoes`	Quantidade de permutações realizadas.
`semente`	Semente usada para reproduzir as permutações.
`tamanho_a`	Quantidade de observações da primeira amostra.
`tamanho_b`	Quantidade de observações da segunda amostra.
`custo_estimado`	Estimativa de trabalho registrada pela operação.
`metodo`	Nome do método; o padrão identifica permutacao_bilateral.

Retorno

Nova instância de ResultadoTestePermutacao.

Exemplo

Código Coral

```
de coral.numerico importe ResultadoTestePermutacao
defina resultado como ResultadoTestePermutacao(1.0, 0.2, 100, 7, 3, 3, 600)
mostre resultado.valor_p
```

Resultado

Saída esperada

```
0.2
```

Atenção

Registro imutável. Construir a classe não executa o teste nem valida o valor p; teste_permutacao calcula e valida os campos. O construtor aceita os valores fornecidos sem conversão automática.

`PortaSerial(...)` · módulo `coral.hardware.serial`

O que faz

Descreve uma porta serial encontrada ou registrada no backend simulado.

Construtor

Assinatura para consulta

```
PortaSerial(dispositivo: 'str', descricao: 'str' = '', fabricante: 'str | None' = None,
  vid: 'int | None' = None, pid: 'int | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
dispositivo	str	sim	sem padrão	Identificador textual da porta, por exemplo /dev/ttyUSB0 ou COM3.
descricao	str	não	''	Descrição legível da porta; padrão vazio.
fabricante	str | None	não	None	Fabricante reportado pelo dispositivo, ou nulo.
vid	int | None	não	None	Identificador USB do fornecedor, ou nulo.
pid	int | None	não	None	Identificador USB do produto, ou nulo.

Atributos importantes

Atributo	Significado
`dispositivo`	Identificador textual da porta, por exemplo /dev/ttyUSB0 ou COM3.
`descricao`	Descrição legível da porta; padrão vazio.
`fabricante`	Fabricante reportado pelo dispositivo, ou nulo.
`vid`	Identificador USB do fornecedor, ou nulo.
`pid`	Identificador USB do produto, ou nulo.

Retorno

Nova instância de PortaSerial.

Exemplo

Código Coral

```
de coral.hardware.serial importe PortaSerial
defina porta como PortaSerial("SIM1", "Porta didática")
mostre porta.dispositivo
mostre porta.descricao
```

Resultado

Saída esperada

```
SIM1
Porta didática
```

Atenção

Registro imutável de metadados. Sua construção não abre a porta, não exige pyserial e não verifica se o dispositivo existe.

`PortaSerial(...)` · módulo `coral.hardware`

O que faz

Descreve uma porta serial encontrada ou registrada no backend simulado.

Construtor

Assinatura para consulta

```
PortaSerial(dispositivo: 'str', descricao: 'str' = '', fabricante: 'str | None' = None,
  vid: 'int | None' = None, pid: 'int | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
dispositivo	str	sim	sem padrão	Identificador textual da porta, por exemplo /dev/ttyUSB0 ou COM3.
descricao	str	não	''	Descrição legível da porta; padrão vazio.
fabricante	str | None	não	None	Fabricante reportado pelo dispositivo, ou nulo.
vid	int | None	não	None	Identificador USB do fornecedor, ou nulo.
pid	int | None	não	None	Identificador USB do produto, ou nulo.

Atributos importantes

Atributo	Significado
`dispositivo`	Identificador textual da porta, por exemplo /dev/ttyUSB0 ou COM3.
`descricao`	Descrição legível da porta; padrão vazio.
`fabricante`	Fabricante reportado pelo dispositivo, ou nulo.
`vid`	Identificador USB do fornecedor, ou nulo.
`pid`	Identificador USB do produto, ou nulo.

Retorno

Nova instância de PortaSerial.

Exemplo

Código Coral

```
de coral.hardware importe PortaSerial
defina porta como PortaSerial("SIM1", "Porta didática")
mostre porta.dispositivo
mostre porta.descricao
```

Resultado

Saída esperada

```
SIM1
Porta didática
```

Atenção

Registro imutável de metadados. Sua construção não abre a porta, não exige pyserial e não verifica se o dispositivo existe.

`BackendSerialSimulado(...)` · **módulo** `coral.hardware.serial`

O que faz

Simula portas seriais em memória para aulas e testes sem hardware.

Construtor

Assinatura para consulta

```
BackendSerialSimulado() -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Métodos principais

Assinatura para consulta

```
registrar_porta(self, dispositivo: 'str' = 'SIM0', *, descricao: 'str' = 'Porta serial
  simulada', fabricante: 'str | None' = 'Coral', vid: 'int | None' = None, pid: 'int |
  None' = None) -> 'PortaSerial'
```

Referência para leitura

```
injetar_recebimento(self, dispositivo: 'str', dados: 'bytes | bytearray | str', *,
  codificacao: 'str' = 'utf-8') -> 'int'
```

Referência para leitura

```
dados_enviados(self, dispositivo: 'str', *, limpar: 'bool' = False) -> 'bytes'
```

Referência para leitura

```
listar_portas(self) -> 'tuple[PortaSerial, ...]'
```

Referência para leitura

```
desconectar(self, dispositivo: 'str') -> 'None'
```

Referência para leitura

```
reconectar(self, dispositivo: 'str') -> 'None'
```

Referência para leitura

```
limpar(self, dispositivo: 'str | None' = None) -> 'None'
```

Retorno

Nova instância de BackendSerialSimulado.

Exemplo

Código Coral

```
de coral.hardware.serial importe BackendSerialSimulado
defina backend como BackendSerialSimulado()
execute backend.registrar_porta("SIM1")
execute backend.injetar_recebimento("SIM1", "Olá")
mostre backend.listar_portas()[0].dispositivo
```

Resultado

Saída esperada

```
SIM1
```

Erros

ErroConfiguracaoSerial para porta vazia ou duplicada; ErroSerial para porta inexistente; DesconexaoSerial para operação que exige porta conectada.

Atenção

Não exige pyserial, não espera pelo timeout e exige registro explícito das portas. registrar_porta cria metadados; injetar_recebimento acrescenta bytes consumidos pelas leituras; dados_enviados consulta os bytes escritos, e limpar apaga os buffers. desconectar invalida transportes já abertos; reconectar permite novas conexões.

`BackendSerialReal(...)` · módulo `coral.hardware.serial`

O que faz

Adapta o backend opcional pyserial para a interface de comunicação da Coral.

Construtor

Assinatura para consulta

```
BackendSerialReal()
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Métodos principais

Assinatura para consulta

```
abrir(self, porta: 'str', velocidade: 'int', timeout: 'float | None') ->
  'TransporteSerial'
```

Referência para leitura

```
listar_portas(self) -> 'tuple[PortaSerial, ...]'
```

Retorno

Nova instância de BackendSerialReal.

Exemplo

Código Coral

```
de coral.hardware.serial importe BackendSerialReal
defina backend como BackendSerialReal()
mostre backend.__class__.__name__
```

Resultado

Saída esperada

```
BackendSerialReal
```

Erros

DependenciaSerialAusente sem pyserial; ErroSerial para falhas ao abrir ou listar portas.

Atenção

A construção é leve e não abre porta. abrir(porta, velocidade, timeout) carrega pyserial e devolve o transporte; listar_portas devolve uma tupla de PortaSerial. As consultas reais dependem do sistema e do hardware.

`BackendSerial(...)` · módulo `coral.hardware.serial`

O que faz

Define o protocolo estrutural dos backends seriais usados pela Coral.

Construtor

Assinatura para consulta

```
BackendSerial(*args, **kwargs)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
args	valor dinâmico	não	sem padrão	Parâmetro herdado exibido pela introspecção; não representa uma construção permitida do protocolo.
kwargs	valor dinâmico	não	sem padrão	Parâmetro herdado exibido pela introspecção; não representa opções de um backend.

Métodos principais

Assinatura para consulta

```
abrir(self, porta: 'str', velocidade: 'int', timeout: 'float | None') ->
  'TransporteSerial'
```

Referência para leitura

```
listar_portas(self) -> 'tuple[PortaSerial, ...]'
```

Retorno

Protocolo para reconhecer objetos com abrir e listar_portas; não é uma fábrica de backends.

Exemplo

Código Coral

```
de coral.hardware.serial importe BackendSerial
de coral.hardware.serial importe BackendSerialSimulado, portas_seriais
defina backend como BackendSerialSimulado()
execute backend.registrar_porta("SIM1")
mostre isinstance(backend, BackendSerial)
```

Resultado

Saída esperada

```
True
```

Erros

TypeError ao tentar instanciar diretamente o protocolo.

Atenção

Não instanciar diretamente. abrir(porta, velocidade, timeout) devolve um transporte com leitura, escrita, fechamento e is_open; listar_portas devolve uma tupla de PortaSerial. isinstance verifica a presença dos membros, sem validar suas assinaturas ou seu comportamento.

`ConexaoSerial(...)` · módulo `coral.hardware.serial`

O que faz

Abre uma porta serial e oferece leitura e escrita com fechamento explícito.

Construtor

Assinatura para consulta

```
ConexaoSerial(porta: 'str', velocidade: 'int' = 115200, timeout: 'float | None' = 1.0,
  *, codificacao: 'str' = 'utf-8', backend: 'BackendSerial | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
porta	str	sim	sem padrão	Nome não vazio da porta que será aberta.
velocidade	int	não	115200	Inteiro positivo em bauds; padrão 115200.
timeout	float | None	não	1.0	Prazo finito não negativo em segundos, ou nulo para espera sem prazo; padrão 1.0.
codificacao	str	não	'utf-8'	Nome de codificação textual reconhecida; padrão utf-8.
backend	BackendSerial | None	não	None	Backend explícito, ou nulo para usar o contexto e depois o backend real.

Atributos importantes

Atributo	Significado
`porta`	Identificador normalizado.
`velocidade`	Velocidade usada.
`timeout`	Prazo de leitura.
`codificacao`	Codificação de texto.
`backend`	Backend escolhido.
`aberta`	Estado público da conexão.

Métodos principais

Assinatura para consulta

```
enviar(self, texto: 'Any', *, terminar_linha: 'bool' = False) -> 'int'
```

Referência para leitura

```
enviar_bytes(self, dados: 'bytes | bytearray') -> 'int'
```

Referência para leitura

```
ler(self, quantidade: 'int' = 1) -> 'bytes'
```

Referência para leitura

```
ler_linha(self) -> 'str'
```

Referência para leitura

```
fechar(self) -> 'None'
```

Retorno

Nova instância de ConexaoSerial.

Exemplo

Código Coral

```
de coral.hardware.serial importe ConexaoSerial
de coral.hardware.serial importe BackendSerialSimulado, portas_seriais
defina backend como BackendSerialSimulado()
execute backend.registrar_porta("SIM1")
defina conexao como ConexaoSerial("SIM1", backend=backend)
mostre conexao.enviar("OK")
execute conexao.fechar()
mostre conexao.aberta
```

Resultado

Saída esperada

```
2
False
```

Erros

ErroConfiguracaoSerial para parâmetros inválidos; DependenciaSerialAusente sem pyserial no backend real; ErroSerial para abertura malsucedida. Operações posteriores podem levantar ConexaoSerialFechada ou DesconexaoSerial.

Atenção

O construtor abre a porta imediatamente. fechar é idempotente. enviar converte para texto, codifica e devolve bytes escritos; ler devolve bytes; ler_linha decodifica com substituição de erros e remove CR/LF finais. Use contexto ou fechar para liberar a porta.

`conectar_serial(...)` · módulo `coral.hardware.serial`

O que faz

Abre uma porta e devolve a conexão serial configurada.

Assinatura

Assinatura para consulta

```
conectar_serial(porta: 'str', velocidade: 'int' = 115200, timeout: 'float | None' = 1.0,
   *, codificacao: 'str' = 'utf-8', backend: 'BackendSerial | None' = None) ->
  'ConexaoSerial'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
porta	str	sim	sem padrão	Nome não vazio da porta que será aberta.
velocidade	int	não	115200	Inteiro positivo em bauds; padrão 115200.
timeout	float | None	não	1.0	Prazo finito não negativo em segundos, ou nulo para espera sem prazo; padrão 1.0.
codificacao	str	não	'utf-8'	Nome de codificação textual reconhecida; padrão utf-8.
backend	BackendSerial | None	não	None	Backend explícito, ou nulo para usar o contexto e depois o backend real.

Retorno

ConexaoSerial aberta.

Exemplo

Código Coral

```
de coral.hardware.serial importe conectar_serial
de coral.hardware.serial importe BackendSerialSimulado, portas_seriais
defina backend como BackendSerialSimulado()
execute backend.registrar_porta("SIM1")
defina conexao como conectar_serial("SIM1", backend=backend)
mostre conexao.aberta
execute conexao.fechar()
```

Resultado

Saída esperada

```
True
```

Erros

ErroConfiguracaoSerial para parâmetros inválidos; DependenciaSerialAusente sem pyserial no backend real; ErroSerial para abertura malsucedida. Operações posteriores podem levantar ConexaoSerialFechada ou DesconexaoSerial.

Atenção

Tem o mesmo contrato do construtor ConexaoSerial. O backend simulado mantém o exemplo independente de pyserial e de equipamento físico. Fechar a conexão ao terminar.

`portas_seriais(...)` · módulo `coral.hardware.serial`

O que faz

Lista as portas disponíveis no backend explícito ou contextual.

Assinatura

Assinatura para consulta

```
portas_seriais(*, backend: 'BackendSerial | None' = None) -> 'tuple[PortaSerial, ...]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
backend	BackendSerial | None	não	None	Backend explícito; nulo usa o contexto ou o backend real.

Retorno

Tupla de PortaSerial; pode ser vazia.

Exemplo

Código Coral

```
de coral.hardware.serial importe portas_seriais
de coral.hardware.serial importe BackendSerialSimulado, portas_seriais
defina backend como BackendSerialSimulado()
execute backend.registrar_porta("SIM1")
mostre portas_seriais(backend=backend)[0].dispositivo
```

Resultado

Saída esperada

```
SIM1
```

Erros

ErroConfiguracaoSerial para backend sem o protocolo requerido; DependenciaSerialAusente sem pyserial no backend real; ErroSerial para falha de enumeração.

Atenção

O backend simulado retorna apenas portas registradas e conectadas. O real consulta o sistema; não abre as portas listadas.

`usar_backend_serial(...)` · módulo `coral.hardware.serial`

O que faz

Instala temporariamente um backend serial no contexto corrente.

Assinatura

Assinatura para consulta

```
usar_backend_serial(backend: 'BackendSerial')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
backend	BackendSerial	sim	sem padrão	Objeto que oferece abrir e listar_portas conforme BackendSerial.

Retorno

Gerenciador de contexto que fornece o backend selecionado ao entrar.

Exemplo

Código Coral

```
de coral.hardware.serial importe usar_backend_serial
de coral.hardware.serial importe BackendSerialSimulado, portas_seriais
defina backend como BackendSerialSimulado()
execute backend.registrar_porta("SIM1")
defina contexto como usar_backend_serial(backend)
execute contexto.__enter__()
mostre portas_seriais()[0].dispositivo
execute contexto.__exit__(nulo, nulo, nulo)
```

Resultado

Saída esperada

```
SIM1
```

Erros

ErroConfiguracaoSerial para objeto sem abrir e listar_portas.

Atenção

Ao sair, restaura o backend anterior, inclusive quando ocorre exceção. Usa ContextVar para separar contextos de tarefas; não troca um padrão global compartilhado. O exemplo mostra o protocolo de entrada e saída explicitamente.

`BackendSerialSimulado(...)` · módulo `coral.hardware`

O que faz

Simula portas seriais em memória para aulas e testes sem hardware.

Construtor

Assinatura para consulta

```
BackendSerialSimulado() -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Métodos principais

Assinatura para consulta

```
registrar_porta(self, dispositivo: 'str' = 'SIM0', *, descricao: 'str' = 'Porta serial
  simulada', fabricante: 'str | None' = 'Coral', vid: 'int | None' = None, pid: 'int |
  None' = None) -> 'PortaSerial'
```

Referência para leitura

```
injetar_recebimento(self, dispositivo: 'str', dados: 'bytes | bytearray | str', *,
  codificacao: 'str' = 'utf-8') -> 'int'
```

Referência para leitura

```
dados_enviados(self, dispositivo: 'str', *, limpar: 'bool' = False) -> 'bytes'
```

Referência para leitura

```
listar_portas(self) -> 'tuple[PortaSerial, ...]'
```

Referência para leitura

```
desconectar(self, dispositivo: 'str') -> 'None'
```

Referência para leitura

```
reconectar(self, dispositivo: 'str') -> 'None'
```

Referência para leitura

```
limpar(self, dispositivo: 'str | None' = None) -> 'None'
```

Retorno

Nova instância de BackendSerialSimulado.

Exemplo

Código Coral

```
de coral.hardware importe BackendSerialSimulado
defina backend como BackendSerialSimulado()
execute backend.registrar_porta("SIM1")
execute backend.injetar_recebimento("SIM1", "Olá")
mostre backend.listar_portas()[0].dispositivo
```

Resultado

Saída esperada

```
SIM1
```

Erros

ErroConfiguracaoSerial para porta vazia ou duplicada; ErroSerial para porta inexistente; DesconexaoSerial para operação que exige porta conectada.

Atenção

Não exige pyserial, não espera pelo timeout e exige registro explícito das portas. registrar_porta cria metadados; injetar_recebimento acrescenta bytes consumidos pelas leituras; dados_enviados consulta os bytes escritos, e limpar apaga os buffers. desconectar invalida transportes já abertos; reconectar permite novas conexões.

`BackendSerialReal(...)` · módulo `coral.hardware`

O que faz

Adapta o backend opcional pyserial para a interface de comunicação da Coral.

Construtor

Assinatura para consulta

```
BackendSerialReal()
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Métodos principais

Assinatura para consulta

```
abrir(self, porta: 'str', velocidade: 'int', timeout: 'float | None') ->
  'TransporteSerial'
```

Referência para leitura

```
listar_portas(self) -> 'tuple[PortaSerial, ...]'
```

Retorno

Nova instância de BackendSerialReal.

Exemplo

Código Coral

```
de coral.hardware importe BackendSerialReal
defina backend como BackendSerialReal()
mostre backend.__class__.__name__
```

Resultado

Saída esperada

```
BackendSerialReal
```

Erros

DependenciaSerialAusente sem pyserial; ErroSerial para falhas ao abrir ou listar portas.

Atenção

A construção é leve e não abre porta. abrir(porta, velocidade, timeout) carrega pyserial e devolve o transporte; listar_portas devolve uma tupla de PortaSerial. As consultas reais dependem do sistema e do hardware.

`BackendSerial(...)` · módulo `coral.hardware`

O que faz

Define o protocolo estrutural dos backends seriais usados pela Coral.

Construtor

Assinatura para consulta

```
BackendSerial(*args, **kwargs)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
args	valor dinâmico	não	sem padrão	Parâmetro herdado exibido pela introspecção; não representa uma construção permitida do protocolo.
kwargs	valor dinâmico	não	sem padrão	Parâmetro herdado exibido pela introspecção; não representa opções de um backend.

Métodos principais

Assinatura para consulta

```
abrir(self, porta: 'str', velocidade: 'int', timeout: 'float | None') ->
  'TransporteSerial'
```

Referência para leitura

```
listar_portas(self) -> 'tuple[PortaSerial, ...]'
```

Retorno

Protocolo para reconhecer objetos com abrir e listar_portas; não é uma fábrica de backends.

Exemplo

Código Coral

```
de coral.hardware importe BackendSerial
de coral.hardware importe BackendSerialSimulado, portas_seriais
defina backend como BackendSerialSimulado()
execute backend.registrar_porta("SIM1")
mostre isinstance(backend, BackendSerial)
```

Resultado

Saída esperada

```
True
```

Erros

TypeError ao tentar instanciar diretamente o protocolo.

Atenção

Não instanciar diretamente. abrir(porta, velocidade, timeout) devolve um transporte com leitura, escrita, fechamento e is_open; listar_portas devolve uma tupla de PortaSerial. isinstance verifica a presença dos membros, sem validar suas assinaturas ou seu comportamento.

`ConexaoSerial(...)` · **módulo** `coral.hardware`

O que faz

Abre uma porta serial e oferece leitura e escrita com fechamento explícito.

Construtor

Assinatura para consulta

```
ConexaoSerial(porta: 'str', velocidade: 'int' = 115200, timeout: 'float | None' = 1.0,
  *, codificacao: 'str' = 'utf-8', backend: 'BackendSerial | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
porta	str	sim	sem padrão	Nome não vazio da porta que será aberta.
velocidade	int	não	115200	Inteiro positivo em bauds; padrão 115200.
timeout	float | None	não	1.0	Prazo finito não negativo em segundos, ou nulo para espera sem prazo; padrão 1.0.
codificacao	str	não	'utf-8'	Nome de codificação textual reconhecida; padrão utf-8.
backend	BackendSerial | None	não	None	Backend explícito, ou nulo para usar o contexto e depois o backend real.

Atributos importantes

Atributo	Significado
`porta`	Identificador normalizado.
`velocidade`	Velocidade usada.
`timeout`	Prazo de leitura.
`codificacao`	Codificação de texto.
`backend`	Backend escolhido.
`aberta`	Estado público da conexão.

Métodos principais

Assinatura para consulta

```
enviar(self, texto: 'Any', *, terminar_linha: 'bool' = False) -> 'int'
```

Referência para leitura

```
enviar_bytes(self, dados: 'bytes | bytearray') -> 'int'
```

Referência para leitura

```
ler(self, quantidade: 'int' = 1) -> 'bytes'
```

Referência para leitura

```
ler_linha(self) -> 'str'
```

Referência para leitura

```
fechar(self) -> 'None'
```

Retorno

Nova instância de ConexaoSerial.

Exemplo

Código Coral

```
de coral.hardware importe ConexaoSerial
de coral.hardware importe BackendSerialSimulado, portas_seriais
defina backend como BackendSerialSimulado()
execute backend.registrar_porta("SIM1")
defina conexao como ConexaoSerial("SIM1", backend=backend)
mostre conexao.enviar("OK")
execute conexao.fechar()
mostre conexao.aberta
```

Resultado

Saída esperada

```
2
False
```

Erros

ErroConfiguracaoSerial para parâmetros inválidos; DependenciaSerialAusente sem pyserial no backend real; ErroSerial para abertura malsucedida. Operações posteriores podem levantar ConexaoSerialFechada ou DesconexaoSerial.

Atenção

O construtor abre a porta imediatamente. fechar é idempotente. enviar converte para texto, codifica e devolve bytes escritos; ler devolve bytes; ler_linha decodifica com substituição de erros e remove CR/LF finais. Use contexto ou fechar para liberar a porta.

`conectar_serial(...)` · módulo `coral.hardware`

O que faz

Abre uma porta e devolve a conexão serial configurada.

Assinatura

Assinatura para consulta

```
conectar_serial(porta: 'str', velocidade: 'int' = 115200, timeout: 'float | None' = 1.0,
    *, codificacao: 'str' = 'utf-8', backend: 'BackendSerial | None' = None) ->
  'ConexaoSerial'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
porta	str	sim	sem padrão	Nome não vazio da porta que será aberta.
velocidade	int	não	115200	Inteiro positivo em bauds; padrão 115200.
timeout	float | None	não	1.0	Prazo finito não negativo em segundos, ou nulo para espera sem prazo; padrão 1.0.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
codificacao	str	não	'utf-8'	Nome de codificação textual reconhecida; padrão utf-8.
backend	BackendSerial | None	não	None	Backend explícito, ou nulo para usar o contexto e depois o backend real.

Retorno

ConexaoSerial aberta.

Exemplo

Código Coral

```
de coral.hardware importe conectar_serial
de coral.hardware importe BackendSerialSimulado, portas_seriais
defina backend como BackendSerialSimulado()
execute backend.registrar_porta("SIM1")
defina conexao como conectar_serial("SIM1", backend=backend)
mostre conexao.aberta
execute conexao.fechar()
```

Resultado

Saída esperada

```
True
```

Erros

ErroConfiguracaoSerial para parâmetros inválidos; DependenciaSerialAusente sem pyserial no backend real; ErroSerial para abertura malsucedida. Operações posteriores podem levantar ConexaoSerialFechada ou DesconexaoSerial.

Atenção

Tem o mesmo contrato do construtor ConexaoSerial. O backend simulado mantém o exemplo independente de pyserial e de equipamento físico. Fechar a conexão ao terminar.

`portas_seriais(...)` · módulo `coral.hardware`

O que faz

Lista as portas disponíveis no backend explícito ou contextual.

Assinatura

Assinatura para consulta

```
portas_seriais(*, backend: 'BackendSerial | None' = None) -> 'tuple[PortaSerial, ...]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
backend	BackendSerial | None	não	None	Backend explícito; nulo usa o contexto ou o backend real.

Retorno

Tupla de PortaSerial; pode ser vazia.

Exemplo

Código Coral

```
de coral.hardware importe portas_seriais
de coral.hardware importe BackendSerialSimulado, portas_seriais
defina backend como BackendSerialSimulado()
execute backend.registrar_porta("SIM1")
mostre portas_seriais(backend=backend)[0].dispositivo
```

Resultado

Saída esperada

```
SIM1
```

Erros

ErroConfiguracaoSerial para backend sem o protocolo requerido; DependenciaSerialAusente sem pyserial no backend real; ErroSerial para falha de enumeração.

Atenção

O backend simulado retorna apenas portas registradas e conectadas. O real consulta o sistema; não abre as portas listadas.

`usar_backend_serial(...)` · módulo `coral.hardware`

O que faz

Instala temporariamente um backend serial no contexto corrente.

Assinatura

Assinatura para consulta

```
usar_backend_serial(backend: 'BackendSerial')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
backend	BackendSerial	sim	sem padrão	Objeto que oferece abrir e listar_portas conforme BackendSerial.

Retorno

Gerenciador de contexto que fornece o backend selecionado ao entrar.

Exemplo

Código Coral

```
de coral.hardware importe usar_backend_serial
de coral.hardware importe BackendSerialSimulado, portas_seriais
defina backend como BackendSerialSimulado()
execute backend.registrar_porta("SIM1")
defina contexto como usar_backend_serial(backend)
execute contexto.__enter__()
mostre portas_seriais()[0].dispositivo
execute contexto.__exit__(nulo, nulo, nulo)
```

Resultado

Saída esperada

```
SIM1
```

Erros

ErroConfiguracaoSerial para objeto sem abrir e listar_portas.

Atenção

Ao sair, restaura o backend anterior, inclusive quando ocorre exceção. Usa ContextVar para separar contextos de tarefas; não troca um padrão global compartilhado. O exemplo mostra o protocolo de entrada e saída explicitamente.

`Memoria(...)` · **módulo** `coral.hardware`

O que faz

Agrupa uma leitura de memória do sistema.

Construtor

Assinatura para consulta

```
Memoria(total_bytes: 'int | None', disponivel_bytes: 'int | None', percentual_usado:
  'float | None') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
total_bytes	int | None	sim	sem padrão	Capacidade total em bytes, ou nulo quando desconhecida.
disponivel_bytes	int | None	sim	sem padrão	Memória disponível em bytes, ou nulo.
percentual_usado	float | None	sim	sem padrão	Percentual de uso reportado, ou nulo.

Atributos importantes

Atributo	Significado
`total_bytes`	Capacidade total em bytes, ou nulo quando desconhecida.
`disponivel_bytes`	Memória disponível em bytes, ou nulo.
`percentual_usado`	Percentual de uso reportado, ou nulo.

Retorno

Nova instância de Memoria.

Exemplo

Código Coral

```
de coral.hardware importe Memoria
defina registro como Memoria(1024, 512, 50.0)
mostre registro.disponivel_bytes
```

Resultado

Saída esperada

```
512
```

Atenção

Registro imutável. O construtor recebe valores informados, sem medir o equipamento nem validar sua coerência. Valores nulos representam informação indisponível.

`Disco(...)` · módulo `coral.hardware`

O que faz

Agrupa metadados e capacidade de um volume de armazenamento.

Construtor

Assinatura para consulta

```
Disco(dispositivo: 'str', ponto_montagem: 'str', sistema_arquivos: 'str', total_bytes:
  'int | None' = None, livre_bytes: 'int | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
dispositivo	str	sim	sem padrão	Nome do dispositivo.
ponto_montagem	str	sim	sem padrão	Caminho onde o volume está montado.
sistema_arquivos	str	sim	sem padrão	Tipo do sistema de arquivos.
total_bytes	int | None	não	None	Capacidade total em bytes; padrão nulo.
livre_bytes	int | None	não	None	Espaço livre em bytes; padrão nulo.

Atributos importantes

Atributo	Significado
`dispositivo`	Nome do dispositivo.
`ponto_montagem`	Caminho onde o volume está montado.
`sistema_arquivos`	Tipo do sistema de arquivos.

Atributo	Significado
`total_bytes`	Capacidade total em bytes; padrão nulo.
`livre_bytes`	Espaço livre em bytes; padrão nulo.

Retorno

Nova instância de Disco.

Exemplo

Código Coral

```
de coral.hardware importe Disco
defina registro como Disco("volume", "/dados", "ext4", 1024, 512)
mostre registro.livre_bytes
```

Resultado

Saída esperada

```
512
```

Atenção

Registro imutável. O construtor recebe valores informados, sem medir o equipamento nem validar sua coerência. Valores nulos representam informação indisponível.

`definir_variavel_ambiente(...)` · módulo `coral.sistema`

O que faz

Define uma variável no ambiente do processo corrente.

Assinatura

Assinatura para consulta

```
definir_variavel_ambiente(nome: 'str', valor: 'Any') -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome convertido com str; precisa ser aceito pelo sistema operacional.
valor	Any	sim	sem padrão	Valor convertido com str antes de armazenar.

Retorno

nulo.

Exemplo

Código Coral

```
de coral.sistema importe definir_variavel_ambiente
de coral.sistema importe variavel_ambiente, remover_variavel_ambiente
execute definir_variavel_ambiente("CORAL_EXEMPLO_TEMP", 42)
mostre variavel_ambiente("CORAL_EXEMPLO_TEMP")
execute remover_variavel_ambiente("CORAL_EXEMPLO_TEMP")
```

Resultado

Saída esperada

```
42
```

Erros

ValueError para nome vazio, nome com = ou caracteres nulos; erros do sistema podem ocorrer na alteração do ambiente.

Atenção

Afeta o processo corrente e seus futuros subprocessos. Não altera o ambiente do terminal que iniciou a Coral. Preserve o valor anterior quando modificar uma variável já existente.

`variavel_ambiente_obrigatoria(...)` · módulo `coral.sistema`

O que faz

Obtém uma variável de ambiente que precisa estar presente.

Assinatura

Assinatura para consulta

```
variavel_ambiente_obrigatoria(nome: 'str') -> 'str'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome convertido com str; não pode ser vazio.

Retorno

Texto armazenado; texto vazio é aceito se a variável existe.

Exemplo

Código Coral

```
de coral.sistema importe variavel_ambiente_obrigatoria
de coral.sistema importe definir_variavel_ambiente, remover_variavel_ambiente
execute definir_variavel_ambiente("CORAL_EXEMPLO_TEMP", "ativo")
mostre variavel_ambiente_obrigatoria("CORAL_EXEMPLO_TEMP")
execute remover_variavel_ambiente("CORAL_EXEMPLO_TEMP")
```

Resultado

Saída esperada

```
ativo
```

Erros

ValueError para nome vazio; ErroSistema se a variável não está definida. A mensagem da ausência não expõe o conteúdo de outras variáveis.

`minimos_quadrados(...)` · módulo `coral.numerico`

O que faz

Resolve um ajuste linear por mínimos quadrados, inclusive para matriz não quadrada.

Assinatura

Assinatura para consulta

```
minimos_quadrados(coeficientes, termos)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
coeficientes	valor dinâmico	sim	sem padrão	Matriz real finita, bidimensional e não vazia.
termos	valor dinâmico	sim	sem padrão	Vetor ou matriz real finita, com a mesma quantidade de linhas dos coeficientes.

Retorno

ResultadoMinimosQuadrados com solução, resíduos, posto e valores singulares.

Exemplo

Código Coral

```
de coral.numerico importe minimos_quadrados
defina resultado como minimos_quadrados([[1], [1]], [2, 2])
mostre resultado.posto
mostre resultado.solucao[0] > 1.9
```

Resultado

Saída esperada

```
1
True
```

Erros

DependenciaNumericaAusente sem NumPy; ErroNumerico para dimensões incompatíveis, dados vazios, valores não finitos ou falha do ajuste.

Atenção

Requer NumPy. Usa rcond=None. Resíduos vazios podem ocorrer quando não há graus de liberdade ou a matriz não tem posto completo.

`intervalo_bootstrap(...)` · módulo `coral.numerico`

O que faz

Estima um intervalo percentil por reamostragem com reposição e semente local.

Assinatura

Assinatura para consulta

```
intervalo_bootstrap(valores, estatistica, confianca, repeticoes, semente)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	valor dinâmico	sim	sem padrão	Série real finita com pelo menos duas observações.
estatistica	valor dinâmico	sim	sem padrão	Função que recebe lista de números e devolve um número real finito.
confianca	valor dinâmico	sim	sem padrão	Número estritamente entre 0 e 1.
repeticoes	valor dinâmico	sim	sem padrão	Inteiro positivo de reamostragens.
semente	valor dinâmico	sim	sem padrão	Inteiro usado no gerador local reproduzível.

Retorno

ResultadoBootstrap com estimativa, limites, parâmetros e custo estimado.

Exemplo

Código Coral

```
de coral.numerico importe intervalo_bootstrap
de coral.numerico importe media
defina resultado como intervalo_bootstrap([1, 2, 3], media, 0.95, 20, 7)
mostre resultado.estimativa
mostre resultado.custo_estimado
```

Resultado

Saída esperada

```
2.0
60
```

Erros

ErroNumerico para dados ou confiança inválidos, estatística não chamável, retorno não finito, falha da estatística ou excesso de trabalho; DependenciaNumericaAusente sem NumPy.

Atenção

Requer NumPy. Usa percentis com interpolação linear. repeticoes vezes tamanho_amostra não pode exceder 5.000.000; o sorteio local não altera o gerador global.

`teste_permutacao(...)` · módulo `coral.numerico`

O que faz

Compara duas amostras usando a diferença de uma estatística e permutações bilaterais.

Assinatura

Assinatura para consulta

```
teste_permutacao(a, b, estatistica, repeticoes, semente)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	valor dinâmico	sim	sem padrão	Primeira série real finita não vazia.
b	valor dinâmico	sim	sem padrão	Segunda série real finita não vazia.
estatistica	valor dinâmico	sim	sem padrão	Função que recebe uma lista e retorna um número real finito.
repeticoes	valor dinâmico	sim	sem padrão	Inteiro positivo de permutações.
semente	valor dinâmico	sim	sem padrão	Inteiro que fixa o gerador aleatório local.

Retorno

ResultadoTestePermutacao com diferença observada, valor p, tamanhos e custo.

Exemplo

Código Coral

```
de coral.numerico importe teste_permutacao
de coral.numerico importe media
defina resultado como teste_permutacao([1, 1], [2, 2], media, 20, 7)
mostre resultado.diferenca_observada
mostre resultado.custo_estimado
```

Resultado

Saída esperada

```
-1.0
80
```

Erros

ErroNumerico para dados inválidos, função estatística incompatível, semente inválida ou excesso de trabalho; DependenciaNumericaAusente sem NumPy.

Atenção

O valor p usa (extremos + 1) dividido por (repeticoes + 1); compara valores absolutos, com tolerância numérica. O custo repeticoes vezes o tamanho combinado não pode exceder 5.000.000. Não altera a fonte aleatória global.

`Superficie3D(...)` · módulo `coral.graficos`

O que faz

Representa uma grade tridimensional explícita.

Construtor

Assinatura para consulta

```
Superficie3D(x: 'tuple[tuple[float, ...], ...]', y: 'tuple[tuple[float, ...], ...]', z:
  'tuple[tuple[float, ...], ...]', nome: 'str | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	tuple[tuple[float, ...], ...]	sim	sem padrão	Grade das coordenadas x.
y	tuple[tuple[float, ...], ...]	sim	sem padrão	Grade das coordenadas y.
z	tuple[tuple[float, ...], ...]	sim	sem padrão	Grade das alturas z.
nome	str | None	não	None	Nome opcional da superfície.

Atributos importantes

Atributo	Significado
`x`	Grade das coordenadas x.
`y`	Grade das coordenadas y.
`z`	Grade das alturas z.
`nome`	Nome opcional da superfície.

Retorno

Nova instância de Superficie3D.

Exemplo

Código Coral

```
de coral.graficos importe Superficie3D
defina registro como Superficie3D([[0, 1]], [[0, 0]], [[2, 3]])
mostre registro.forma
```

Resultado

Saída esperada

```
(1, 2)
```

Atenção

Registro imutável no nível dos atributos. Anotações de tupla não convertem listas automaticamente; prefira os construtores funcionais para obter dados normalizados. IndexError ao consultar forma quando z está vazio; use superficie3d para validar a grade.

`SerieCategorias(...)` · **módulo** `coral.graficos`

O que faz

Representa uma série de barras ou histograma.

Construtor

Assinatura para consulta

```
SerieCategorias(estilo: 'str', categorias: 'tuple[str, ...]', valores: 'tuple[float,
  ...]', limites: 'tuple[tuple[float, float], ...]' = (), nome: 'str | None' = None) ->
  None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
estilo	str	sim	sem padrão	Identificador do estilo da série.
categorias	tuple[str, ...]	sim	sem padrão	Rótulos das categorias.
valores	tuple[float, ...]	sim	sem padrão	Valores associados na mesma ordem.
limites	tuple[tuple[float, float], ...]	não	()	Intervalos das classes do histograma; padrão tupla vazia.
nome	str | None	não	None	Nome opcional da série.

Atributos importantes

Atributo	Significado
`estilo`	Identificador do estilo da série.
`categorias`	Rótulos das categorias.
`valores`	Valores associados na mesma ordem.
`limites`	Intervalos das classes do histograma; padrão tupla vazia.
`nome`	Nome opcional da série.

Retorno

Nova instância de SerieCategorias.

Exemplo

Código Coral

```
de coral.graficos importe SerieCategorias
defina registro como SerieCategorias("barras", ["A", "B"], [2, 3])
mostre registro.categorias
```

Resultado

Saída esperada

```
['A', 'B']
```

Atenção

Registro imutável no nível dos atributos. Anotações de tupla não convertem listas automaticamente; prefira os construtores funcionais para obter dados normalizados. O construtor armazena os dados; barras e histograma validam as entradas.

`ResumoCaixa(...)` · módulo `coral.graficos`

O que faz

Armazena o resumo de um grupo para um gráfico de caixa.

Construtor

Assinatura para consulta

```
ResumoCaixa(nome: 'str', minimo: 'float', q1: 'float', mediana: 'float', q3: 'float',
  maximo: 'float', bigode_inferior: 'float', bigode_superior: 'float', extremos:
  'tuple[float, ...]', quantidade: 'int') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Rótulo do grupo.
minimo	float	sim	sem padrão	Menor observação.
q1	float	sim	sem padrão	Primeiro quartil.
mediana	float	sim	sem padrão	Segundo quartil.
q3	float	sim	sem padrão	Terceiro quartil.
maximo	float	sim	sem padrão	Maior observação.
bigode_inferior	float	sim	sem padrão	Limite inferior do bigode.
bigode_superior	float	sim	sem padrão	Limite superior do bigode.
extremos	tuple[float, ...]	sim	sem padrão	Observações extremas além dos bigodes.
quantidade	int	sim	sem padrão	Número de observações do grupo.

Atributos importantes

Atributo	Significado
`nome`	Rótulo do grupo.
`minimo`	Menor observação.
`q1`	Primeiro quartil.
`mediana`	Segundo quartil.
`q3`	Terceiro quartil.
`maximo`	Maior observação.
`bigode_inferior`	Limite inferior do bigode.
`bigode_superior`	Limite superior do bigode.
`extremos`	Observações extremas além dos bigodes.
`quantidade`	Número de observações do grupo.

Retorno

Nova instância de ResumoCaixa.

Exemplo

Código Coral

```
de coral.graficos importe ResumoCaixa
defina registro como ResumoCaixa("A", 1, 2, 3, 4, 5, 1, 5, [], 5)
mostre registro.mediana
```

Resultado

Saída esperada

```
3
```

Atenção

Registro imutável no nível dos atributos. Anotações de tupla não convertem listas automaticamente; prefira os construtores funcionais para obter dados normalizados. O construtor não calcula quartis nem extremos; grafico_caixa calcula os resumos.

`MatrizCores(...)` · módulo `coral.graficos`

O que faz

Representa uma matriz numérica com rótulos de linhas e colunas.

Construtor

Assinatura para consulta

```
MatrizCores(valores: 'tuple[tuple[float, ...], ...]', rotulos_linhas: 'tuple[str, ...]',
    rotulos_colunas: 'tuple[str, ...]', nome: 'str | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valores	tuple[tuple[float, ...], ...]	sim	sem padrão	Matriz numérica do mapa de cores.
rotulos_linhas	tuple[str, ...]	sim	sem padrão	Rótulos alinhados às linhas.
rotulos_colunas	tuple[str, ...]	sim	sem padrão	Rótulos alinhados às colunas.
nome	str | None	não	None	Nome opcional da matriz.

Atributos importantes

Atributo	Significado
`valores`	Matriz numérica do mapa de cores.
`rotulos_linhas`	Rótulos alinhados às linhas.
`rotulos_colunas`	Rótulos alinhados às colunas.
`nome`	Nome opcional da matriz.

Retorno

Nova instância de MatrizCores.

Exemplo

Código Coral

```
de coral.graficos importe MatrizCores
defina registro como MatrizCores([[1, 2]], ["L1"], ["A", "B"])
mostre registro.rotulos_colunas
```

Resultado

Saída esperada

```
['A', 'B']
```

Atenção

Registro imutável no nível dos atributos. Anotações de tupla não convertem listas automaticamente; prefira os construtores funcionais para obter dados normalizados. O construtor não valida dimensões; use os construtores de mapa de calor ou correlação.

`SerieErro(...)` · módulo `coral.graficos`

O que faz

Representa uma série com incerteza inferior e superior por ponto.

Construtor

Assinatura para consulta

```
SerieErro(x: 'tuple[float, ...]', y: 'tuple[float, ...]', erro_inferior: 'tuple[float,
  ...]', erro_superior: 'tuple[float, ...]', nome: 'str | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	tuple[float, ...]	sim	sem padrão	Coordenadas horizontais.
y	tuple[float, ...]	sim	sem padrão	Valores centrais.
erro_inferior	tuple[float, ...]	sim	sem padrão	Magnitudes inferiores por ponto.
erro_superior	tuple[float, ...]	sim	sem padrão	Magnitudes superiores por ponto.
nome	str | None	não	None	Nome opcional da série.

Atributos importantes

Atributo	Significado
`x`	Coordenadas horizontais.
y	Valores centrais.
`erro_inferior`	Magnitudes inferiores por ponto.
`erro_superior`	Magnitudes superiores por ponto.

Atributo	Significado
`nome`	Nome opcional da série.

Retorno

Nova instância de SerieErro.

Exemplo

Código Coral

```
de coral.graficos importe SerieErro
defina registro como SerieErro([0, 1], [2, 3], [0.1, 0.2], [0.2, 0.3])
mostre registro.y
```

Resultado

Saída esperada

```
[2, 3]
```

Atenção

Registro imutável no nível dos atributos. Anotações de tupla não convertem listas automaticamente; prefira os construtores funcionais para obter dados normalizados. O construtor não verifica comprimentos nem sinais; barras_erro valida as entradas.

`PainelGraficos(...)` · módulo `coral.graficos`

O que faz

Agrupa gráficos em uma grade regular.

Construtor

Assinatura para consulta

```
PainelGraficos(graficos: "tuple['Grafico', ...]", colunas: 'int', titulo: 'str' = '') ->
   None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
graficos	tuple['Grafico', ...]	sim	sem padrão	Coleção ordenada de objetos Grafico.
colunas	int	sim	sem padrão	Número de colunas da grade.
titulo	str	não	''	Título textual do painel; padrão vazio.

Atributos importantes

Atributo	Significado
`graficos`	Coleção ordenada de objetos Grafico.
`colunas`	Número de colunas da grade.

Atributo	Significado
`titulo`	Título textual do painel; padrão vazio.

Retorno

Nova instância de PainelGraficos.

Exemplo

Código Coral

```
de coral.graficos importe PainelGraficos
defina registro como PainelGraficos([], 2, "Painel")
mostre registro.linhas
```

Resultado

Saída esperada

```
0
```

Atenção

Registro imutável no nível dos atributos. Anotações de tupla não convertem listas automaticamente; prefira os construtores funcionais para obter dados normalizados. Divisão por zero ao consultar linhas se colunas é zero; use painel para validar a composição.

`FonteAleatoria(...)` · módulo `coral.aleatorio`

O que faz

Mantém um gerador local de números aleatórios reproduzível com semente explícita.

Construtor

Assinatura para consulta

```
FonteAleatoria(semente: 'Any' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
semente	Any	não	None	Semente aceita por random.Random: inteiro, decimal, texto, bytes, bytearray ou nulo. Nulo usa inicialização não reproduzível.

Atributos importantes

Atributo	Significado
`semente`	Semente registrada na construção; alterar este atributo não reinicializa o gerador.

Métodos principais

Assinatura para consulta

```
inteiro(self, minimo, maximo)
```

Referência para leitura

```
decimal(self, minimo=0.0, maximo=1.0)
```

Referência para leitura

```
escolher(self, valores)
```

Referência para leitura

```
amostra(self, valores, quantidade)
```

Referência para leitura

```
escolha_ponderada(self, valores, pesos)
```

Referência para leitura

```
embaralhar(self, valores)
```

Retorno

Nova instância de FonteAleatoria.

Exemplo

Código Coral

```
de coral.aleatorio importe FonteAleatoria
defina fonte como FonteAleatoria(7)
mostre fonte.inteiro(1, 1)
mostre fonte.escolher(["Coral"])
```

Resultado

Saída esperada

```
1
Coral
```

Erros

TypeError para semente incompatível; ValueError para intervalo inválido, amostra impossível ou pesos inválidos; IndexError ao escolher em coleção vazia.

Atenção

A sequência depende da semente e da ordem das chamadas. inteiro inclui os dois limites; escolher requer coleção não vazia; amostra seleciona sem reposição; embaralhar retorna uma cópia. Não usar para segredos criptográficos.

`FonteEntrada(...)` · módulo `coral.entrada`

O que faz

Define o protocolo estrutural de uma fonte de linhas para a Coral.

Construtor

Assinatura para consulta

```
FonteEntrada(*args, **kwargs)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
args	valor dinâmico	não	sem padrão	Parâmetro herdado sem aplicação ao uso deste protocolo.
kwargs	valor dinâmico	não	sem padrão	Parâmetro herdado sem aplicação ao uso deste protocolo.

Métodos principais

Assinatura para consulta

```
ler_linha(self, mensagem: 'str' = '') -> 'str'
```

Retorno

Protocolo de reconhecimento; não é uma fonte concreta nem uma fábrica.

Exemplo

Código Coral

```
de coral.entrada importe FonteEntrada
de coral.entrada importe FonteEntradaSequencial
defina fonte como FonteEntradaSequencial(["Ana"])
mostre isinstance(fonte, FonteEntrada)
mostre fonte.ler_linha()
```

Resultado

Saída esperada

```
True
Ana
```

Erros

TypeError ao instanciar diretamente o protocolo; FimDeEntrada é o sinal usado por fontes concretas quando a entrada termina.

Atenção

Implemente ler_linha(mensagem="") para devolver texto. FonteEntradaSequencial é uma implementação pronta. isinstance verifica presença do método, sem conferir sua assinatura ou o retorno.

`trajetoria3d(...)` · módulo `coral.graficos`

O que faz

Cria uma trajetória tridimensional na ordem fornecida.

Assinatura

Assinatura para consulta

```
trajetoria3d(x: 'Iterable[Any]', y: 'Iterable[Any]', z: 'Iterable[Any]', nome: 'str |
  None' = None) -> 'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	Iterable[Any]	sim	sem padrão	Coordenadas x finitas.
y	Iterable[Any]	sim	sem padrão	Coordenadas y finitas, alinhadas a x.
z	Iterable[Any]	sim	sem padrão	Coordenadas z finitas, alinhadas a x.
nome	str | None	não	None	Nome opcional da série.

Retorno

Grafico com uma Serie3D.

Exemplo

Código Coral

```
de coral.graficos importe trajetoria3d
defina grafico como trajetoria3d([0, 1], [0, 1], [1, 2])
mostre grafico.tipo
```

Resultado

Saída esperada

```
trajetoria3d
```

Erros

ErroGrafico para dados vazios, ausentes, não finitos ou comprimentos incompatíveis.

Atenção

Não reamostra nem suaviza a trajetória. Construir a especificação não abre janela nem requer backend de renderização.

`superficie3d(...)` · módulo `coral.graficos`

O que faz

Cria uma superfície em grade retangular sem preencher valores ausentes.

Assinatura

Assinatura para consulta

```
superficie3d(x: 'Any', y: 'Any', z: 'Any', nome: 'str | None' = None) -> 'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	Any	sim	sem padrão	Eixo horizontal com uma coordenada por coluna de z, ou grade com a mesma forma de z.
y	Any	sim	sem padrão	Eixo vertical com uma coordenada por linha de z, ou grade com a mesma forma de z.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
z	Any	sim	sem padrão	Matriz retangular não vazia de alturas reais finitas.
nome	str | None	não	None	Nome opcional da superfície.

Retorno

Grafico do tipo superficie3d com uma Superficie3D normalizada.

Exemplo

Código Coral

```
de coral.graficos importe superficie3d
defina grafico como superficie3d([0, 1], [0], [[2, 3]])
mostre grafico.elementos[0].forma
```

Resultado

Saída esperada

```
(1, 2)
```

Erros

ErroGrafico para formas incompatíveis, matriz irregular, dados vazios ou valores não finitos.

Atenção

x e y precisam ser ambos eixos ou ambos grades. Não há interpolação silenciosa. A criação dos dados não depende de Matplotlib ou Plotly.

`grafico_caixa(...)` · módulo `coral.graficos`

O que faz

Calcula caixas de Tukey com quartis, bigodes e observações extremas.

Assinatura

Assinatura para consulta

```
grafico_caixa(grupos: 'Any') -> 'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grupos	Any	sim	sem padrão	Mapeamento de nomes para séries, ou coleção de grupos numéricos conforme o normalizador do módulo.

Retorno

Grafico com um ResumoCaixa por grupo e avisos aplicáveis.

Exemplo

Código Coral

```
de coral.graficos importe grafico_caixa
defina grafico como grafico_caixa({"A": [1, 2, 3, 4, 5]})
mostre grafico.elementos[0].mediana
```

Resultado

Saída esperada

```
3.0
```

Erros

ErroGrafico para grupos ou séries inválidos; erros numéricos do cálculo de quartis podem ser propagados.

Atenção

Bigodes usam limites de 1,5 vezes o intervalo interquartil; extremos são preservados. Grupos com menos de cinco observações recebem aviso. Quartis seguem coral.numerico.

`grafico_funcao(...)` · módulo `coral.graficos`

O que faz

Amostra uma função escalar em intervalo fechado e representa falhas como lacunas.

Assinatura

Assinatura para consulta

```
grafico_funcao(funcao: 'Callable[[float], Any]', inicio: 'Any', fim: 'Any', amostras:
  'int' = 200) -> 'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
funcao	Callable[[float], Any]	sim	sem padrão	Função chamável que recebe uma coordenada x e devolve número finito.
inicio	Any	sim	sem padrão	Primeira coordenada finita do intervalo.
fim	Any	sim	sem padrão	Última coordenada finita; precisa diferir de inicio.
amostras	int	não	200	Quantidade inteira exata entre 2 e 100000; padrão 200.

Retorno

Grafico com os pontos amostrados e avisos para lacunas.

Exemplo

Código Coral

```
de coral.graficos importe grafico_funcao
crie a função quadrado com x
    retorne x vezes x
fim
defina grafico como grafico_funcao(quadrado, 0, 2, 3)
mostre grafico.elementos[0].y
```

Resultado

Saída esperada

```
(0.0, 1.0, 4.0)
```

Erros

ErroGrafico para função não chamável, intervalo inválido, contagem inválida ou ausência de amostras válidas.

Atenção

Inclui as extremidades. Exceções e resultados não finitos na função são registrados como lacunas; não interpolar esses trechos. Cada chamada executa a função do programa.

`mapa_correlacao(...)` · **módulo** `coral.graficos`

O que faz

Calcula uma matriz de Pearson e cria sua representação como mapa de cores.

Assinatura

Assinatura para consulta

```
mapa_correlacao(dados: 'Any', rotulos: 'Iterable[Any] | None' = None, nome: 'str | None'
   = None) -> 'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
dados	Any	sim	sem padrão	Coleção de séries reais finitas compatível com matriz_correlacao; cada coluna é uma variável e cada linha é uma observação.
rotulos	Iterable[Any] | None	não	None	Rótulos das variáveis, ou nulo para rótulos automáticos.
nome	str | None	não	None	Nome opcional do mapa.

Retorno

Grafico com MatrizCores e proveniência do cálculo numérico.

Exemplo

Código Coral

```
de coral.graficos importe mapa_correlacao
defina grafico como mapa_correlacao([[1, 2], [2, 4], [3, 6]], ["A", "B"])
mostre grafico.tipo
```

Resultado

Saída esperada

```
mapa_correlacao
```

Erros

Erros numéricos ou de dependência são propagados; ErroGrafico para rótulos ou dados de apresentação inválidos.

Atenção

O cálculo pertence a coral.numerico e requer NumPy. Não abre janela. Os rótulos devem corresponder à quantidade de variáveis.

`amostrar_grafico(...)` · módulo `coral.graficos`

O que faz

Reduz explicitamente uma série simples por seleção uniforme de observações.

Assinatura

Assinatura para consulta

```
amostrar_grafico(grafico: 'Any', max_pontos: 'int') -> 'Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grafico	Any	sim	sem padrão	Grafico com uma única série de linha, dispersao, dispersao3d ou trajetoria3d.
max_pontos	int	sim	sem padrão	Inteiro maior ou igual a dois; limite de pontos na nova série.

Retorno

Grafico reduzido; devolve o próprio objeto se já está dentro do limite.

Exemplo

Código Coral

```
de coral.graficos importe amostrar_grafico
de coral.graficos importe linha
defina original como linha([0, 1, 2, 3], [0, 1, 4, 9])
defina reduzido como amostrar_grafico(original, 2)
mostre reduzido.elementos[0].x
```

Resultado

Saída esperada

```
(0.0, 3.0)
```

Erros

ErroGrafico para limite inválido, formato incompatível ou gráfico com mais de uma série.

Atenção

Preserva os metadados e registra aviso e proveniência da amostragem. Não é aplicado automaticamente na exportação HTML. Barras, caixas, grades e faixas exigem redução própria do programa.

`Medicao(...)` · módulo `coral.laboratorio`

O que faz

Registra o resultado de uma execução experimental.

Construtor

Assinatura para consulta

```
Medicao(repeticao: 'int', parametros: 'dict[str, Any]', semente: 'int | None',
  tempo_segundos: 'float', memoria_atual_bytes: 'int', memoria_pico_bytes: 'int', retorno:
   'Any' = None, erro: 'str | None' = None, metricas: 'dict[str, Any]' = <factory>) ->
  None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
repeticao	int	sim	sem padrão	Número positivo da repetição, contado a partir de um.
parametros	dict[str, Any]	sim	sem padrão	Dicionário dos valores de entrada desta execução.
semente	int | None	sim	sem padrão	Semente da execução, ou nulo.
tempo_segundos	float	sim	sem padrão	Tempo medido em segundos.
memoria_atual_bytes	int	sim	sem padrão	Memória atual medida em bytes.
memoria_pico_bytes	int	sim	sem padrão	Pico de memória medido em bytes.
retorno	Any	não	None	Valor devolvido pela função estudada; padrão nulo.
erro	str | None	não	None	Mensagem da falha, ou nulo quando não houve falha.
metricas	dict[str, Any]	não	<factory>	Dicionário de métricas adicionais; padrão dicionário novo por instância.

Atributos importantes

Atributo	Significado
`repeticao`	Número positivo da repetição, contado a partir de um.

Atributo	Significado
`parametros`	Dicionário dos valores de entrada desta execução.
`semente`	Semente da execução, ou nulo.
`tempo_segundos`	Tempo medido em segundos.
`memoria_atual_bytes`	Memória atual medida em bytes.
`memoria_pico_bytes`	Pico de memória medido em bytes.
`retorno`	Valor devolvido pela função estudada; padrão nulo.
`erro`	Mensagem da falha, ou nulo quando não houve falha.
`metricas`	Dicionário de métricas adicionais; padrão dicionário novo por instância.

Retorno

Nova instância de Medicao.

Exemplo

Código Coral

```
de coral.laboratorio importe Medicao
defina medicao como Medicao(1, {"x": 2}, 7, 0.0, 0, 0, retorno=4)
mostre medicao.retorno
```

Resultado

Saída esperada

```
4
```

Atenção

Registro imutável no nível dos atributos; dicionários internos continuam mutáveis. O construtor não mede a execução nem valida todos os campos. validar_resultado verifica medições inseridas em um resultado.

`ResultadoExperimento(...)` · módulo `coral.laboratorio`

O que faz

Agrupa medições, metadados e operações de análise de um experimento.

Construtor

Assinatura para consulta

```
ResultadoExperimento(nome: 'str', medicoes: 'list[Medicao]', metadados: 'dict[str,
  Any]') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome textual do experimento.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
medicoes	list[Medicao]	sim	sem padrão	Lista de Medicao, na ordem de execução.
metadados	dict[str, Any]	sim	sem padrão	Dicionário de informações adicionais do experimento.

Atributos importantes

Atributo	Significado
`nome`	Nome do experimento.
`medicoes`	Lista ordenada de medições.
`metadados`	Informações adicionais.
`passou`	Todas as medições sem erro.

Métodos principais

Assinatura para consulta

```
retornos(self, *, incluir_falhas: 'bool' = False) -> 'list[Any]'
```

Referência para leitura

```
resumo(self) -> 'dict[str, Any]'
```

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Referência para leitura

```
adicionar_metadados(self, nome: 'str', valor: 'Any') -> 'None'
```

Referência para leitura

```
salvar_json(self, caminho: 'str | Path') -> 'Path'
```

Referência para leitura

```
salvar_csv(self, caminho: 'str | Path') -> 'Path'
```

Retorno

Nova instância de ResultadoExperimento.

Exemplo

Código Coral

```
de coral.laboratorio importe ResultadoExperimento
de coral.laboratorio importe Medicao
defina medicao como Medicao(1, {"x": 2}, 7, 0.0, 0, 0, retorno=4)
defina resultado como ResultadoExperimento("Quadrado", [medicao], {})
mostre resultado.retornos()
mostre resultado.passou
```

Resultado

Saída esperada

```
[4]
True
```

Erros

ValueError para metadados inválidos ou conflitos de colunas no CSV; ErroPersistencia para valores não portáteis ao salvar; erros de arquivo podem ser propagados.

Atenção

Objeto mutável. passou exige que todas as medições tenham erro nulo; uma lista vazia também passa. retornos omite falhas por padrão. resumo calcula totais e médias; para_dict produz dados estruturados. salvar_json e salvar_csv escrevem arquivos. Tempo e memória variam com o ambiente.

`planejar_experimento(...)` · módulo `coral.laboratorio`

O que faz

Conta combinações e execuções sem materializar a grade ou executar a função estudada.

Assinatura

Assinatura para consulta

```
planejar_experimento(parametros: 'Mapping[str, Iterable[Any]] | None', repeticoes: 'int'
   = 1, limite: 'int | None' = None) -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
parametros	Mapping[str, Iterable[Any]] | None	sim	sem padrão	Mapeamento de nomes não vazios para coleções de valores, ou nulo para uma combinação sem parâmetros.
repeticoes	int	não	1	Inteiro positivo de repetições por combinação; padrão um.
limite	int | None	não	None	Inteiro positivo para comparação de tamanho, ou nulo sem limite.

Retorno

Dicionário com dimensões, combinações, execuções previstas, limite e excesso.

Exemplo

Código Coral

```
de coral.laboratorio importe planejar_experimento
defina plano como planejar_experimento({"x": [1, 2, 3]}, 2, 5)
mostre plano["execucoes_previstas"]
mostre plano["excede_limite"]
```

Resultado

Saída esperada

```
6
True
```

Erros

ValueError para nomes inválidos, repetições ou limite fora do contrato; entradas não iteráveis são recusadas.

Atenção

Iteráveis sem len são percorridos para contagem e podem ser consumidos; use coleções reutilizáveis quando depois precisar executar a grade. O limite é informado, sem executar nem interromper um experimento.

`selecionar_medicoes(...)` · módulo `coral.laboratorio`

O que faz

Seleciona medições sem modificar o resultado e preservando sua ordem.

Assinatura

Assinatura para consulta

```
selecionar_medicoes(resultado: 'ResultadoExperimento', criterio: 'Callable[[Medicao],
  bool] | Mapping[str, Any]') -> 'list[Medicao]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
resultado	ResultadoExperimento	sim	sem padrão	ResultadoExperimento que fornece as medições.
criterio	Callable[[Medicao], bool] | Mapping[str, Any]	sim	sem padrão	Função que recebe Medicao e devolve condição, ou mapeamento de chaves para valores esperados.

Retorno

Lista de Medicao correspondente ao critério.

Exemplo

Código Coral

```
de coral.laboratorio importe selecionar_medicoes
de coral.laboratorio importe Medicao, ResultadoExperimento
defina medicao como Medicao(1, {"x": 2}, 7, 0.0, 0, 0, retorno=4, metricas={"quadrado":
  4})
defina resultado como ResultadoExperimento("Quadrado", [medicao], {})
mostre selecionar_medicoes(resultado, {"parametros.x": 2})[0].retorno
```

Resultado

Saída esperada

```
4
```

Erros

TypeError para resultado ou critério incompatível; ValueError para chave de critério inválida; exceções do predicado são propagadas.

Atenção

Mapeamentos combinam condições por igualdade. Prefixos parametros. e metricas. desambiguam as chaves; campos como repeticao e semente também podem ser consultados. Chaves ausentes não correspondem. A lista contém os mesmos objetos do resultado.

`resumir_por_parametro(...)` · módulo `coral.laboratorio`

O que faz

Agrupa medições pelo valor de um parâmetro, preservando a ordem de aparição dos grupos.

Assinatura

Assinatura para consulta

```
resumir_por_parametro(resultado: 'ResultadoExperimento', nome: 'str') -> 'dict[str,
  Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
resultado	ResultadoExperimento	sim	sem padrão	ResultadoExperimento com as medições a resumir.
nome	str	sim	sem padrão	Nome não vazio de um parâmetro presente em todas as medições.

Retorno

Dicionário com nome do parâmetro, grupos e total de execuções.

Exemplo

Código Coral

```
de coral.laboratorio importe resumir_por_parametro
de coral.laboratorio importe Medicao, ResultadoExperimento
defina medicao como Medicao(1, {"x": 2}, 7, 0.0, 0, 0, retorno=4, metricas={"quadrado":
  4})
defina resultado como ResultadoExperimento("Quadrado", [medicao], {})
mostre resumir_por_parametro(resultado, "x")["execucoes"]
```

Resultado

Saída esperada

```
1
```

Erros

TypeError para resultado incompatível; ValueError para nome vazio, parâmetro ausente ou valor não agrupável pela política portátil.

Atenção

Um resultado sem medições devolve grupos vazios. A chave de agrupamento usa representação portátil; a função não altera as medições.

`validar_resultado(...)` · módulo `coral.laboratorio`

O que faz

Verifica estrutura e portabilidade de um resultado experimental sem modificá lo.

Assinatura

Assinatura para consulta

```
validar_resultado(resultado: 'Any') -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
resultado	Any	sim	sem padrão	Valor a verificar como ResultadoExperimento.

Retorno

Dicionário com valido, erros e avisos; erros incluem código, caminho e mensagem.

Exemplo

Código Coral

```
de coral.laboratorio importe validar_resultado
de coral.laboratorio importe Medicao, ResultadoExperimento
defina medicao como Medicao(1, {"x": 2}, 7, 0.0, 0, 0, retorno=4, metricas={"quadrado":
  4})
defina resultado como ResultadoExperimento("Quadrado", [medicao], {})
mostre validar_resultado(resultado)["valido"]
```

Resultado

Saída esperada

```
True
```

Erros

Inconsistências estruturais normais são devolvidas em erros, sem serem lançadas como exceção.

Atenção

Uma entrada de tipo incorreto devolve diagnóstico inválido. Verifica nomes, listas, medições, tempos, memória e dados portáteis. Resultado estruturalmente válido pode conter medições com erro de execução.

`carregar_resultado_json(...)` · módulo `coral.laboratorio`

O que faz

Reconstrói um resultado experimental salvo no formato JSON versionado do laboratório.

Assinatura

Assinatura para consulta

```
carregar_resultado_json(caminho: 'str | Path') -> 'ResultadoExperimento'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	str | Path	sim	sem padrão	Caminho textual ou Path do arquivo a ler.

Retorno

ResultadoExperimento reconstruído e validado.

Exemplo

Código Coral

```
de coral.laboratorio importe carregar_resultado_json
de coral.laboratorio importe Medicao, ResultadoExperimento
defina medicao como Medicao(1, {"x": 2}, 7, 0.0, 0, 0, retorno=4, metricas={"quadrado":
  4})
defina resultado como ResultadoExperimento("Quadrado", [medicao], {})
execute resultado.salvar_json("experimento.json")
mostre carregar_resultado_json("experimento.json").retornos()
```

Resultado

Saída esperada

```
[4]
```

Erros

ValueError para contrato, esquema ou dados inválidos; erros de leitura e de decodificação JSON são propagados.

Atenção

Conferir contrato e versão do documento antes da reconstrução. O arquivo precisa ter sido salvo no formato do laboratório; um JSON arbitrário não é suficiente.

`grafico_experimento(...)` · módulo `coral.graficos`

O que faz

Converte medições bem sucedidas de um experimento em uma série numérica comum.

Assinatura

Assinatura para consulta

```
grafico_experimento(resultado: 'Any', eixo_x: 'str', metrica: 'str') -> 'Grafico'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
resultado	Any	sim	sem padrão	ResultadoExperimento com pelo menos uma medição válida para o gráfico.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
eixo_x	str	sim	sem padrão	Nome não vazio do parâmetro ou campo que fornece coordenadas horizontais.
metrica	str	sim	sem padrão	Nome não vazio da métrica ou campo que fornece valores verticais.

Retorno

Grafico com valores numéricos das medições selecionadas e proveniência.

Exemplo

Código Coral

```
de coral.graficos importe grafico_experimento
de coral.laboratorio importe Medicao, ResultadoExperimento
defina medicao como Medicao(1, {"x": 2}, 7, 0.0, 0, 0, retorno=4, metricas={"quadrado":
  4})
defina resultado como ResultadoExperimento("Quadrado", [medicao], {})
defina grafico como grafico_experimento(resultado, "x", "quadrado")
mostre grafico.tipo
```

Resultado

Saída esperada

```
dispersao
```

Erros

ErroGrafico para resultado incompatível, nomes inválidos, valores ausentes ou não finitos, ou ausência de medições utilizáveis.

Atenção

Medições com erro são ignoradas e geram aviso. Importa laboratório apenas durante a chamada. Não interpola dados nem executa novamente o experimento.

`comparar_experimentos(...)` · módulo `coral.laboratorio`

O que faz

Compara resultados de forma descritiva e explicita diferenças de configuração.

Assinatura

Assinatura para consulta

```
comparar_experimentos(a: 'ResultadoExperimento', b: 'ResultadoExperimento') ->
  'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	ResultadoExperimento	sim	sem padrão	ResultadoExperimento usado como referência.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
b	ResultadoExperimento	sim	sem padrão	ResultadoExperimento comparado com a referência.

Retorno

Dicionário com resumos, configuração, diferenças de métricas e interpretação.

Exemplo

Código Coral

```
de coral.laboratorio importe comparar_experimentos
de coral.laboratorio importe Medicao, ResultadoExperimento
defina a como ResultadoExperimento("A", [Medicao(1, {}, 1, 0.0, 0, 0, retorno=2)], {})
defina b como ResultadoExperimento("B", [Medicao(1, {}, 1, 0.0, 0, 0, retorno=3)], {})
mostre comparar_experimentos(a, b)["significancia_estatistica"]
```

Resultado

Saída esperada

```
None
```

Erros

TypeError para tipos incompatíveis; ValueError para resultados estruturalmente inválidos.

Atenção

As diferenças numéricas usam b menos a. Não calcula significância estatística nem prova causalidade ou superioridade. Erros estruturais são recusados; o diagnóstico de portabilidade isolado não impede a comparação.

`executar_experimento(...)` · módulo `coral.laboratorio`

O que faz

Executa uma grade de parâmetros e registra retorno, duração, memória e erros de cada repetição.

Assinatura

Assinatura para consulta

```
executar_experimento(nome: 'str', funcao: 'Callable[..., Any]', *, repeticoes: 'int' =
  1, parametros: 'Mapping[str, Iterable[Any]] | None' = None, semente: 'int | None' =
  None, metricas: 'Callable[[Any], Mapping[str, Any]] | None' = None, continuar_em_erro:
  'bool' = True, relogio: 'FonteTempo | None' = None) -> 'ResultadoExperimento'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome atribuído ao experimento.
funcao	Callable[..., Any]	sim	sem padrão	Função chamável executada com os parâmetros da grade como argumentos nomeados.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
repeticoes	int	não	1	Inteiro positivo de execuções por combinação.
parametros	Mapping[str, Iterable[Any]] | None	não	None	Mapeamento de nomes para iteráveis não vazios, ou nulo para uma combinação sem parâmetros.
semente	int | None	não	None	Inteiro base ou nulo; cada execução soma seu índice à base.
metricas	Callable[[Any], Mapping[str, Any]] | None	não	None	Função que recebe o retorno e produz um mapeamento, ou nulo.
continuar_em_erro	bool	não	True	Se verdadeiro, registra exceções da função ou métricas e prossegue; se falso, propaga.
relogio	FonteTempo | None	não	None	FonteTempo para medição, ou nulo para relógio padrão.

Retorno

ResultadoExperimento com medições e metadados do ambiente.

Exemplo

Código Coral

```
de coral.laboratorio importe executar_experimento
crie a função quadrado com x
    retorne x vezes x
fim
defina resultado como executar_experimento("Quadrado", quadrado, parametros={"x": [2,
  3]}, semente=7)
mostre resultado.retornos()
```

Resultado

Saída esperada

```
[4, 9]
```

Erros

ValueError para repetições, semente ou grade inválidas; erros da função e métricas são registrados ou propagados conforme continuar_em_erro.

Atenção

A grade é materializada. Preserva e restaura o estado aleatório global de Python e NumPy; efeitos da função sobre arquivos ou outros objetos permanecem. Tempos e memória dependem do ambiente; o pico pode incluir rastreio já ativo. Erros de execução podem produzir um resultado com passou falso.

`planejar_experimento_procedural(...)` · módulo `coral.laboratorio`

O que faz

Estima execuções e custo máximo de um estudo procedural antes de reconstruir produtos.

Assinatura

Assinatura para consulta

```
planejar_experimento_procedural(receita: 'Any', *, sementes: 'Iterable[Any] | None' =
  None, parametros: 'Mapping[str, Iterable[Any]] | None' = None, repeticoes: 'int' = 1,
  limite_execucoes: 'int | None' = None, limite_custo_total: 'int | None' = None) ->
  'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
receita	Any	sim	sem padrão	ReceitaProcedural ou dados de receita aceitos pelo normalizador.
sementes	Iterable[Any] | None	não	None	Iterável de sementes da receita, ou nulo para sua semente raiz.
parametros	Mapping[str, Iterable[Any]] | None	não	None	Grade adicional do estudo, ou nulo.
repeticoes	int	não	1	Inteiro positivo de repetições por combinação.
limite_execucoes	int | None	não	None	Inteiro positivo para comparação com o total, ou nulo.
limite_custo_total	int | None	não	None	Inteiro positivo para comparação com o custo estimado, ou nulo.

Retorno

Dicionário versionado com plano da grade, receita, custo por execução e custo máximo total.

Exemplo

Código Coral

```
de coral.laboratorio importe planejar_experimento_procedural
de coral.procedural importe receita_procedural
defina receita como receita_procedural("distribuir_pontos", largura=10, altura=10,
  quantidade=2, max_tentativas_por_ponto=10, limite_tentativas_total=20)
defina plano como planejar_experimento_procedural(receita, sementes=[1, 2])
mostre plano["plano_experimento"]["execucoes_previstas"]
mostre plano["executou_receita"]
```

Resultado

Saída esperada

```
2
False
```

Erros

Erros de contrato procedural para receita inválida; ValueError para grades, repetições ou limites inválidos.

Atenção

Não executa a função estudada nem reconstrói a receita. Os limites aparecem como indicadores no plano. A estimativa usa o pior caso declarado pela receita; não é medição de tempo. Iteráveis de parâmetros podem ser consumidos durante a preparação.

`executar_experimento_procedural(...)` · módulo `coral.laboratorio`

O que faz

Reconstrói uma receita para cada combinação e mede a função que observa seu produto.

Assinatura

Assinatura para consulta

```
executar_experimento_procedural(nome: 'str', receita: 'Any', funcao: 'Callable[...,
  Any]', *, sementes: 'Iterable[Any] | None' = None, parametros: 'Mapping[str,
  Iterable[Any]] | None' = None, repeticoes: 'int' = 1, semente_laboratorio: 'int | None'
  = None, metricas: 'Callable[[Any], Mapping[str, Any]] | None' = None, continuar_em_erro:
   'bool' = True, relogio: 'FonteTempo | None' = None, limite_custo_receita: 'int' =
  1000000, limite_execucoes: 'int | None' = None, limite_custo_total: 'int | None' = None)
   -> 'ResultadoExperimento'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome do experimento.
receita	Any	sim	sem padrão	ReceitaProcedural ou dados de receita aceitos pelo normalizador.
funcao	Callable[..., Any]	sim	sem padrão	Função que recebe primeiro o produto procedural e depois os parâmetros nomeados do estudo.
sementes	Iterable[Any] | None	não	None	Iterável de sementes da receita, ou nulo para sua semente raiz.
parametros	Mapping[str, Iterable[Any]] | None	não	None	Grade adicional do estudo, ou nulo.
repeticoes	int	não	1	Inteiro positivo de repetições por combinação.
semente_laboratorio	int | None	não	None	Inteiro base das medições, independente das sementes procedurais, ou nulo.
metricas	Callable[[Any], Mapping[str, Any]] | None	não	None	Função que recebe o retorno e produz um mapeamento, ou nulo.
continuar_em_erro	bool	não	True	Se verdadeiro, registra exceções da função ou métricas e prossegue; se falso, propaga.
relogio	FonteTempo | None	não	None	FonteTempo para medição, ou nulo para relógio padrão.
limite_custo_receita	int	não	1000000	Teto de custo permitido na reconstrução de cada receita.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
limite_execucoes	int | None	não	None	Inteiro positivo para comparação com o total, ou nulo.
limite_custo_total	int | None	não	None	Inteiro positivo para comparação com o custo estimado, ou nulo.

Retorno

ResultadoExperimento com semente_procedural nas medições e proveniência nos metadados.

Exemplo

Código Coral

```
de coral.laboratorio importe executar_experimento_procedural
de coral.procedural importe receita_procedural
defina receita como receita_procedural("distribuir_pontos", largura=10, altura=10,
  quantidade=2, max_tentativas_por_ponto=10, limite_tentativas_total=20)
crie a função contar com pontos
    retorne len(pontos)
fim
defina resultado como executar_experimento_procedural("Pontos", receita, contar,
  sementes=[1, 2], limite_execucoes=2)
mostre resultado.retornos()
```

Resultado

Saída esperada

```
[2, 2]
```

Erros

TypeError para função não chamável; ValueError para limites excedidos ou grade inválida; erros procedurais e da função podem ser registrados ou propagados.

Atenção

Recusa limites globais excedidos antes de executar. O limite de cada receita é aplicado na reconstrução. A semente procedural controla o produto; semente_laboratorio controla a medição. Falhas de reconstrução ou da função obedecem a continuar_em_erro.

`diagnosticar_inspecao(...)` · módulo `coral`

O que faz

Verifica snapshot, serialização JSON e visões disponíveis do objeto.

Assinatura

Assinatura para consulta

```
diagnosticar_inspecao(objeto: 'Any') -> 'DiagnosticoInspecao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto inspecionável, snapshot ou valor aceito pelo contrato de inspeção.

Retorno

DiagnosticoInspecao com condição, assinatura, visões, avisos e eventual erro.

Exemplo

Código Coral

```
de coral importe diagnosticar_inspecao
de coral.inspecao importe SnapshotInspecao
defina estado como SnapshotInspecao("Exemplo", "alvo", {"execucoes": 0, "modo":
  "ao_ativar", "habilitada": verdadeiro})
mostre diagnosticar_inspecao(estado).ok
```

Resultado

Saída esperada

```
True
```

Erros

Exceções derivadas de Exception durante a inspeção são capturadas no diagnóstico.

Atenção

Falhas normais de inspeção são devolvidas com ok falso. Não avalia regras nem altera o domínio; métodos de inspeção fornecidos pelo próprio objeto são chamados.

`relatorio_inspecao(...)` · módulo `coral`

O que faz

Prepara uma apresentação humana a partir de um snapshot normalizado.

Assinatura

Assinatura para consulta

```
relatorio_inspecao(objeto: 'Any', *, visao: 'str | None' = None, caminhos: 'list[str] |
  tuple[str, ...]' = ()) -> 'RelatorioInspecao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto ou snapshot a inspecionar.
visao	str | None	não	None	Nome de visão semântica, ou nulo.
caminhos	list[str] | tuple[str, ...]	não	()	Lista ou tupla de caminhos a selecionar quando visao for nulo.

Retorno

RelatorioInspecao com dados, tipo, nome, visão e assinatura de origem.

Exemplo

Código Coral

```
de coral importe relatorio_inspecao
de coral.inspecao importe SnapshotInspecao
defina estado como SnapshotInspecao("Exemplo", "alvo", {"execucoes": 0, "modo":
  "ao_ativar", "habilitada": verdadeiro})
mostre relatorio_inspecao(estado).tipo
```

Resultado

Saída esperada

```
Exemplo
```

Erros

ValueError para visão desconhecida ou indisponível; erros de caminhos e de inspeção são propagados.

Atenção

visao tem precedência sobre caminhos. Sem ambos, inclui os dados completos normalizados. O relatório não substitui nem modifica o estado do domínio.

`visualizar_inspecao(...)` · módulo `coral`

O que faz

Projeta um snapshot em uma visão semântica disponível.

Assinatura

Assinatura para consulta

```
visualizar_inspecao(objeto: 'Any', visao: 'str') -> 'VisaoInspecao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto ou snapshot compatível com o contrato de inspeção.
visao	str	sim	sem padrão	Nome de visão ou alias reconhecido: cena, animacao, colisao, regras, eventos ou temporizadores.

Retorno

VisaoInspecao com dados normalizados e assinatura de origem.

Exemplo

Código Coral

```
de coral importe visualizar_inspecao
de coral.inspecao importe SnapshotInspecao
defina estado como SnapshotInspecao("Exemplo", "alvo", {"execucoes": 0, "modo":
  "ao_ativar", "habilitada": verdadeiro})
mostre visualizar_inspecao(estado, "regras").visao
```

Resultado

Saída esperada

```
regras
```

Erros

ValueError para nome desconhecido ou visão indisponível; erros de inspeção são propagados.

Atenção

A disponibilidade depende das chaves do snapshot. Não importa o módulo de domínio nem executa ações da simulação. Use visoes_inspecao para consultar as opções.

`explicar_regra(...)` · módulo `coral`

O que faz

Explica a última avaliação registrada de uma regra individual.

Assinatura

Assinatura para consulta

```
explicar_regra(objeto: 'Any') -> 'ExplicacaoRegra'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Regra inspecionável ou snapshot com execucoes, modo e habilitada.

Retorno

ExplicacaoRegra com estado, mensagem e dados observados.

Exemplo

Código Coral

```
de coral importe explicar_regra
de coral.inspecao importe SnapshotInspecao
defina estado como SnapshotInspecao("Exemplo", "alvo", {"execucoes": 0, "modo":
  "ao_ativar", "habilitada": verdadeiro})
mostre explicar_regra(estado).estado
```

Resultado

Saída esperada

```
nao_avaliada
```

Erros

TypeError quando o snapshot não descreve uma regra individual; erros de inspeção ou conversão dos dados são propagados.

Atenção

Não reavalia a condição nem executa a ação. Distingue regra desabilitada, não avaliada, suprimida, executada ou pronta usando apenas o snapshot.

`executar_comando_inspecao(...)` · módulo `coral`

O que faz

Atende operações de consulta de uma sessão com resposta estruturada.

Assinatura

Assinatura para consulta

```
executar_comando_inspecao(sessao: "'SessaoInspecao'", operacao: 'str', *, alvo: 'str |
  None' = None, argumentos: 'Mapping[str, Any] | None' = None, capturar_erros: 'bool' =
  True) -> 'RespostaInspecao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
sessao	'SessaoInspecao'	sim	sem padrão	SessaoInspecao que contém os alvos e eventos.
operacao	str	sim	sem padrão	Nome ou alias da operação de consulta.
alvo	str | None	não	None	Nome do alvo quando exigido pela operação, ou nulo.
argumentos	Mapping[str, Any] | None	não	None	Mapeamento com opções da operação, ou nulo.
capturar_erros	bool	não	True	Se verdadeiro, converte falhas internas da operação em resposta com ok falso.

Retorno

RespostaInspecao com operação, condição, dados e eventual erro.

Exemplo

Código Coral

```
de coral importe executar_comando_inspecao
de coral.inspecao importe SessaoInspecao
defina sessao como SessaoInspecao()
mostre executar_comando_inspecao(sessao, "listar_alvos").ok
```

Resultado

Saída esperada

```
True
```

Erros

TypeError para sessão inválida; erros na conversão inicial dos argumentos podem ser propagados. Com capturar_erros falso, falhas da operação também são propagadas.

Atenção

Operações: listar_alvos, snapshot, consultar, visoes, visualizar, historico, eventos, relatorio, sessao, explicar_regra e diagnosticar. Não amostra em segundo plano. A validação do tipo da sessão e a conversão inicial dos argumentos ocorrem antes da captura de erros.

`visualizar_inspecao(...)` · módulo `coral.inspecao`

O que faz

Projeta um snapshot em uma visão semântica disponível.

Assinatura

Assinatura para consulta

```
visualizar_inspecao(objeto: 'Any', visao: 'str') -> 'VisaoInspecao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto ou snapshot compatível com o contrato de inspeção.
visao	str	sim	sem padrão	Nome de visão ou alias reconhecido: cena, animacao, colisao, regras, eventos ou temporizadores.

Retorno

VisaoInspecao com dados normalizados e assinatura de origem.

Exemplo

Código Coral

```
de coral.inspecao importe visualizar_inspecao
de coral.inspecao importe SnapshotInspecao
defina estado como SnapshotInspecao("Exemplo", "alvo", {"execucoes": 0, "modo":
  "ao_ativar", "habilitada": verdadeiro})
mostre visualizar_inspecao(estado, "regras").visao
```

Resultado

Saída esperada

```
regras
```

Erros

ValueError para nome desconhecido ou visão indisponível; erros de inspeção são propagados.

Atenção

A disponibilidade depende das chaves do snapshot. Não importa o módulo de domínio nem executa ações da simulação. Use visoes_inspecao para consultar as opções.

`explicar_regra(...)` · módulo `coral.inspecao`

O que faz

Explica a última avaliação registrada de uma regra individual.

Assinatura

Assinatura para consulta

```
explicar_regra(objeto: 'Any') -> 'ExplicacaoRegra'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Regra inspecionável ou snapshot com execucoes, modo e habilitada.

Retorno

ExplicacaoRegra com estado, mensagem e dados observados.

Exemplo

Código Coral

```
de coral.inspecao importe explicar_regra
de coral.inspecao importe SnapshotInspecao
defina estado como SnapshotInspecao("Exemplo", "alvo", {"execucoes": 0, "modo":
  "ao_ativar", "habilitada": verdadeiro})
mostre explicar_regra(estado).estado
```

Resultado

Saída esperada

```
nao_avaliada
```

Erros

TypeError quando o snapshot não descreve uma regra individual; erros de inspeção ou conversão dos dados são propagados.

Atenção

Não reavalia a condição nem executa a ação. Distingue regra desabilitada, não avaliada, suprimida, executada ou pronta usando apenas o snapshot.

`executar_comando_inspecao(...)` · módulo `coral.inspecao`

O que faz

Atende operações de consulta de uma sessão com resposta estruturada.

Assinatura

Assinatura para consulta

```
executar_comando_inspecao(sessao: "'SessaoInspecao'", operacao: 'str', *, alvo: 'str |
  None' = None, argumentos: 'Mapping[str, Any] | None' = None, capturar_erros: 'bool' =
  True) -> 'RespostaInspecao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
sessao	'SessaoInspecao'	sim	sem padrão	SessaoInspecao que contém os alvos e eventos.
operacao	str	sim	sem padrão	Nome ou alias da operação de consulta.
alvo	str | None	não	None	Nome do alvo quando exigido pela operação, ou nulo.
argumentos	Mapping[str, Any] | None	não	None	Mapeamento com opções da operação, ou nulo.
capturar_erros	bool	não	True	Se verdadeiro, converte falhas internas da operação em resposta com ok falso.

Retorno

RespostaInspecao com operação, condição, dados e eventual erro.

Exemplo

Código Coral

```
de coral.inspecao importe executar_comando_inspecao
de coral.inspecao importe SessaoInspecao
defina sessao como SessaoInspecao()
mostre executar_comando_inspecao(sessao, "listar_alvos").ok
```

Resultado

Saída esperada

```
True
```

Erros

TypeError para sessão inválida; erros na conversão inicial dos argumentos podem ser propagados. Com capturar_erros falso, falhas da operação também são propagadas.

Atenção

Operações: listar_alvos, snapshot, consultar, visoes, visualizar, historico, eventos, relatorio, sessao, explicar_regra e diagnosticar. Não amostra em segundo plano. A validação do tipo da sessão e a conversão inicial dos argumentos ocorrem antes da captura de erros.

`comparar_inspecoes(...)` · módulo `coral`

O que faz

Compara dados normalizados de dois estados e registra alterações por caminho.

Assinatura

Assinatura para consulta

```
comparar_inspecoes(antes: 'Any', depois: 'Any', *, incluir: 'tuple[str, ...] |
  list[str]' = (), ignorar: 'tuple[str, ...] | list[str]' = ()) -> 'DiferencaInspecao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
antes	Any	sim	sem padrão	Objeto ou snapshot anterior.
depois	Any	sim	sem padrão	Objeto ou snapshot posterior.
incluir	tuple[str, ...] | list[str]	não	()	Filtros de caminhos incluídos, ou coleção vazia para todos.
ignorar	tuple[str, ...] | list[str]	não	()	Filtros de caminhos excluídos.

Retorno

DiferencaInspecao com assinaturas dos snapshots e tupla de mudanças.

Exemplo

Código Coral

```
de coral importe comparar_inspecoes
de coral.inspecao importe SnapshotInspecao
defina antes como SnapshotInspecao("Exemplo", "alvo", {"energia": 2})
defina depois como SnapshotInspecao("Exemplo", "alvo", {"energia": 3})
mostre comparar_inspecoes(antes, depois).quantidade
```

Resultado

Saída esperada

```
1
```

Erros

Erros de inspeção e normalização dos objetos são propagados.

Atenção

Compara dados, sem aplicar mudanças ao objeto. Para preservar um estado anterior, capture um snapshot antes de alterar o alvo. Filtros trabalham com caminhos do contrato normalizado.

`HistoricoInspecao(...)` · módulo `coral.inspecao`

O que faz

Mantém uma coleção limitada de snapshots registrados explicitamente.

Construtor

Assinatura para consulta

```
HistoricoInspecao(*, max_amostras: 'int' = 120, intervalo: 'float' = 0.0,
  somente_mudancas: 'bool' = False, nome_alvo: 'str | None' = None) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
max_amostras	int	não	120	Inteiro positivo do máximo de amostras retidas.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
intervalo	float	não	0.0	Intervalo mínimo finito e não negativo entre instantes informados.
somente_mudancas	bool	não	False	Se verdadeiro, ignora snapshots com a mesma assinatura do último.
nome_alvo	str | None	não	None	Nome textual do alvo ou nulo.

Atributos importantes

Atributo	Significado
`amostras`	Tupla das amostras atualmente retidas.
`ultima`	Última amostra ou nulo.
`max_amostras`	Capacidade configurada.

Métodos principais

Assinatura para consulta

```
registrar(self, objeto: 'Any', *, instante: 'float | None' = None, forcar: 'bool' =
  False) -> 'AmostraInspecao | None'
```

Referência para leitura

```
limpar(self) -> 'None'
```

Referência para leitura

```
ao_evento(self, callback: 'Callable[[EventoObservabilidade], Any]')
```

Referência para leitura

```
estatisticas_inspecao(self) -> 'dict[str, int]'
```

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de HistoricoInspecao.

Exemplo

Código Coral

```
de coral.inspecao importe HistoricoInspecao
de coral.inspecao importe SnapshotInspecao
defina estado como SnapshotInspecao("Exemplo", "alvo", {"execucoes": 0, "modo":
  "ao_ativar", "habilitada": verdadeiro})
defina historico como HistoricoInspecao(max_amostras=2)
execute historico.registrar(estado, instante=0)
mostre len(historico.amostras)
```

Resultado

Saída esperada

```
1
```

Erros

ValueError para capacidade, intervalo ou instante inválidos; TypeError para callback não chamável. Erros de inspeção e de callbacks são propagados.

Atenção

Não inicia tarefa em segundo plano. registrar pode devolver nulo por intervalo ou ausência de mudanças; forcar ignora essas restrições. Ao atingir o limite, descarta a amostra mais antiga. limpar remove amostras e referências temporais, preservando sequência e contadores.

`SessaoInspecao(...)` · módulo `coral.inspecao`

O que faz

Agrupa alvos, históricos, vigias e eventos para ferramentas de observabilidade.

Construtor

Assinatura para consulta

```
SessaoInspecao(*, max_eventos: 'int' = 500) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
max_eventos	int	não	500	Inteiro positivo do máximo de eventos retidos.

Atributos importantes

Atributo	Significado
`nomes_alvos`	Tupla dos nomes registrados em ordem de inserção.
`eventos`	Tupla dos eventos retidos.
`max_eventos`	Capacidade do buffer.

Métodos principais

Assinatura para consulta

```
adicionar_alvo(self, nome: 'str', objeto: 'Any', *, max_amostras: 'int' = 120,
  intervalo: 'float' = 0.0, somente_mudancas: 'bool' = False) -> 'Any'
```

Referência para leitura

```
remover_alvo(self, nome: 'str') -> 'Any'
```

Referência para leitura

```
adicionar_vigia(self, alvo: 'str', vigia: 'VigiaInspecao') -> 'VigiaInspecao'
```

Referência para leitura

```
amostrar(self, *, instante: 'float | None' = None, forcar: 'bool' = False) -> 'dict[str,
   AmostraInspecao | None]'
```

Referência para leitura

```
snapshot(self, alvo: 'str') -> 'SnapshotInspecao'
```

Referência para leitura

```
consultar(self, alvo: 'str', caminho: 'str', padrao: 'Any' = <ausente>) -> 'Any'
```

Referência para leitura

```
historico(self, alvo: 'str') -> 'HistoricoInspecao'
```

Referência para leitura

```
consultar_eventos(self, *, cursor: 'int' = 0, limite: 'int' = 100, alvo: 'str | None' =
  None, tipos: 'tuple[str, ...] | list[str]' = (), caminho: 'str | None' = None) ->
  'dict[str, Any]'
```

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de SessaoInspecao.

Exemplo

Código Coral

```
de coral.inspecao importe SessaoInspecao
defina sessao como SessaoInspecao(max_eventos=3)
mostre len(sessao.nomes_alvos)
mostre len(sessao.eventos)
```

Resultado

Saída esperada

```
0
0
```

Erros

ValueError para capacidade ou nome vazio; KeyError para alvo ausente; TypeError para vigia inválida; erros de inspeção e callbacks são propagados.

Atenção

Amostragem é explícita. Alvos são mantidos por referência; adicionar um nome existente substitui o alvo e seu histórico. Ao lotar o buffer, eventos mais antigos são descartados e o descarte é contado. Não cria servidor nem transporte DAP.

`AmostraInspecao(...)` · módulo `coral.inspecao`

O que faz

Associa um snapshot à sequência e ao instante de uma observação.

Construtor

Assinatura para consulta

```
AmostraInspecao(sequencia: 'int', snapshot: 'SnapshotInspecao', instante: 'float | None'
   = None, diferenca_anterior: 'DiferencaInspecao | None' = None, contrato: 'str' =
  'coral.inspecao.amostra/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
sequencia	int	sim	sem padrão	Número sequencial da observação.
snapshot	SnapshotInspecao	sim	sem padrão	SnapshotInspecao capturado.
instante	float | None	não	None	Instante numérico ou nulo.
diferenca_anterior	DiferencaInspecao | None	não	None	DiferencaInspecao em relação à amostra anterior, ou nulo.
contrato	str	não	'coral.inspecao.amostra/1'	Identificador versionado do formato.

Atributos importantes

Atributo	Significado
`sequencia`	Ordem da amostra.
`snapshot`	Estado associado.
`instante`	Tempo informado pelo chamador.
`diferenca_anterior`	Mudanças registradas ou nulo.

Métodos principais

Assinatura para consulta

```
para_dict(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de AmostraInspecao.

Exemplo

Código Coral

```
de coral.inspecao importe AmostraInspecao
de coral.inspecao importe SnapshotInspecao
defina estado como SnapshotInspecao("Exemplo", "alvo", {"execucoes": 0, "modo":
  "ao_ativar", "habilitada": verdadeiro})
defina amostra como AmostraInspecao(1, estado, instante=0)
mostre amostra.para_dict()["sequencia"]
```

Resultado

Saída esperada

```
1
```

Erros

TypeError para argumentos incompatíveis com a assinatura; falhas dos registros associados podem aparecer durante para_dict.

Atenção

Registro imutável nos campos, sem validação completa no construtor. O snapshot e seus dados podem conter referências mutáveis; prefira snapshots produzidos pelo normalizador. para_dict serializa os registros relacionados.

`Inspecionavel(...)` · módulo `coral.inspecao`

O que faz

Define o protocolo estrutural de objetos que expõem estado_inspecao.

Construtor

Assinatura para consulta

```
Inspecionavel(*args, **kwargs)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
args	valor dinâmico	não	sem padrão	Argumentos variáveis exibidos pela introspecção do protocolo; não usar para instanciar.
kwargs	valor dinâmico	não	sem padrão	Argumentos nomeados exibidos pela introspecção; não usar para instanciar.

Métodos principais

Assinatura para consulta

```
estado_inspecao(self) -> 'Mapping[str, Any]'
```

Retorno

Protocolo para reconhecimento de implementações compatíveis.

Exemplo

Código Coral

```
de coral.inspecao importe Inspecionavel
de coral.inspecao importe HistoricoInspecao
mostre isinstance(HistoricoInspecao(), Inspecionavel)
```

Resultado

Saída esperada

```
True
```

Erros

TypeError se o programa tentar instanciar o protocolo.

Atenção

Não instanciar. A verificação em tempo de execução reconhece a presença do método, sem conferir integralmente tipos de retorno ou ausência de efeitos. estado_inspecao deve devolver um mapeamento.

`bateria(...)` · **módulo** `coral.hardware`

O que faz

Consulta a bateria quando o sistema oferece esse dado.

Assinatura

Assinatura para consulta

```
bateria() -> 'Bateria | None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nenhum	não se aplica	não	não se aplica	não recebe argumentos

Retorno

Bateria com percentual, conexão à energia e segundos restantes, ou nulo.

Exemplo

Código Coral

```
de coral.hardware importe bateria
de coral.hardware importe Bateria
defina resultado como bateria()
mostre resultado == nulo ou isinstance(resultado, Bateria)
```

Resultado

Saída esperada

```
True
```

Erros

Falhas normais da consulta e da conversão dos sensores são tratadas como ausência de informação.

Atenção

Depende de psutil e de suporte do sistema. Ausência de bateria, dependência ou erro da consulta devolve nulo. Segundos restantes desconhecidos ou ilimitados também são nulos. Valores variam com a máquina.

`AmostraInspecao(...)` · **módulo** `coral`

O que faz

Associa um snapshot à sequência e ao instante de uma observação.

Construtor

Assinatura para consulta

```
AmostraInspecao(sequencia: 'int', snapshot: 'SnapshotInspecao', instante: 'float | None'
   = None, diferenca_anterior: 'DiferencaInspecao | None' = None, contrato: 'str' =
  'coral.inspecao.amostra/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
sequencia	int	sim	sem padrão	Número sequencial da observação.
snapshot	SnapshotInspecao	sim	sem padrão	SnapshotInspecao capturado.
instante	float | None	não	None	Instante numérico ou nulo.
diferenca_anterior	DiferencaInspecao | None	não	None	DiferencaInspecao em relação à amostra anterior, ou nulo.
contrato	str	não	'coral.inspecao.amostra/1'	Identificador versionado do formato.

Atributos importantes

Atributo	Significado
`sequencia`	Ordem da amostra.
`snapshot`	Estado associado.
`instante`	Tempo informado pelo chamador.
`diferenca_anterior`	Mudanças registradas ou nulo.

Métodos principais

Assinatura para consulta

```
para_dict(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de AmostraInspecao.

Exemplo

Código Coral

```
de coral importe AmostraInspecao
de coral.inspecao importe SnapshotInspecao
defina estado como SnapshotInspecao("Exemplo", "alvo", {"execucoes": 0, "modo":
  "ao_ativar", "habilitada": verdadeiro})
defina amostra como AmostraInspecao(1, estado, instante=0)
mostre amostra.para_dict()["sequencia"]
```

Resultado

Saída esperada

```
1
```

Erros

TypeError para argumentos incompatíveis com a assinatura; falhas dos registros associados podem aparecer durante para_dict.

Atenção

Registro imutável nos campos, sem validação completa no construtor. O snapshot e seus dados podem conter referências mutáveis; prefira snapshots produzidos pelo normalizador. para_dict serializa os registros relacionados.

`HistoricoInspecao(...)` · módulo `coral`

O que faz

Mantém uma coleção limitada de snapshots registrados explicitamente.

Construtor

Assinatura para consulta

```
HistoricoInspecao(*, max_amostras: 'int' = 120, intervalo: 'float' = 0.0,
  somente_mudancas: 'bool' = False, nome_alvo: 'str | None' = None) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
max_amostras	int	não	120	Inteiro positivo do máximo de amostras retidas.
intervalo	float	não	0.0	Intervalo mínimo finito e não negativo entre instantes informados.
somente_mudancas	bool	não	False	Se verdadeiro, ignora snapshots com a mesma assinatura do último.
nome_alvo	str | None	não	None	Nome textual do alvo ou nulo.

Atributos importantes

Atributo	Significado
`amostras`	Tupla das amostras atualmente retidas.
`ultima`	Última amostra ou nulo.
`max_amostras`	Capacidade configurada.

Métodos principais

Assinatura para consulta

```
registrar(self, objeto: 'Any', *, instante: 'float | None' = None, forcar: 'bool' =
  False) -> 'AmostraInspecao | None'
```

Referência para leitura

```
limpar(self) -> 'None'
```

Referência para leitura

```
ao_evento(self, callback: 'Callable[[EventoObservabilidade], Any]')
```

Referência para leitura

```
estatisticas_inspecao(self) -> 'dict[str, int]'
```

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de HistoricoInspecao.

Exemplo

Código Coral

```
de coral importe HistoricoInspecao
de coral.inspecao importe SnapshotInspecao
defina estado como SnapshotInspecao("Exemplo", "alvo", {"execucoes": 0, "modo":
  "ao_ativar", "habilitada": verdadeiro})
defina historico como HistoricoInspecao(max_amostras=2)
execute historico.registrar(estado, instante=0)
mostre len(historico.amostras)
```

Resultado

Saída esperada

```
1
```

Erros

ValueError para capacidade, intervalo ou instante inválidos; TypeError para callback não chamável. Erros de inspeção e de callbacks são propagados.

Atenção

Não inicia tarefa em segundo plano. registrar pode devolver nulo por intervalo ou ausência de mudanças; forcar ignora essas restrições. Ao atingir o limite, descarta a amostra mais antiga. limpar remove amostras e referências temporais, preservando sequência e contadores.

`SessaoInspecao(...)` · módulo `coral`

O que faz

Agrupa alvos, históricos, vigias e eventos para ferramentas de observabilidade.

Construtor

Assinatura para consulta

```
SessaoInspecao(*, max_eventos: 'int' = 500) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
max_eventos	int	não	500	Inteiro positivo do máximo de eventos retidos.

Atributos importantes

Atributo	Significado
`nomes_alvos`	Tupla dos nomes registrados em ordem de inserção.
`eventos`	Tupla dos eventos retidos.
`max_eventos`	Capacidade do buffer.

Métodos principais

Assinatura para consulta

```
adicionar_alvo(self, nome: 'str', objeto: 'Any', *, max_amostras: 'int' = 120,
  intervalo: 'float' = 0.0, somente_mudancas: 'bool' = False) -> 'Any'
```

Referência para leitura

```
remover_alvo(self, nome: 'str') -> 'Any'
```

Referência para leitura

```
adicionar_vigia(self, alvo: 'str', vigia: 'VigiaInspecao') -> 'VigiaInspecao'
```

Referência para leitura

```
amostrar(self, *, instante: 'float | None' = None, forcar: 'bool' = False) -> 'dict[str,
   AmostraInspecao | None]'
```

Referência para leitura

```
snapshot(self, alvo: 'str') -> 'SnapshotInspecao'
```

Referência para leitura

```
consultar(self, alvo: 'str', caminho: 'str', padrao: 'Any' = <ausente>) -> 'Any'
```

Referência para leitura

```
historico(self, alvo: 'str') -> 'HistoricoInspecao'
```

Referência para leitura

```
consultar_eventos(self, *, cursor: 'int' = 0, limite: 'int' = 100, alvo: 'str | None' =
  None, tipos: 'tuple[str, ...] | list[str]' = (), caminho: 'str | None' = None) ->
  'dict[str, Any]'
```

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de SessaoInspecao.

Exemplo

Código Coral

```
de coral importe SessaoInspecao
defina sessao como SessaoInspecao(max_eventos=3)
mostre len(sessao.nomes_alvos)
mostre len(sessao.eventos)
```

Resultado

Saída esperada

```
0
0
```

Erros

ValueError para capacidade ou nome vazio; KeyError para alvo ausente; TypeError para vigia inválida; erros de inspeção e callbacks são propagados.

Atenção

Amostragem é explícita. Alvos são mantidos por referência; adicionar um nome existente substitui o alvo e seu histórico. Ao lotar o buffer, eventos mais antigos são descartados e o descarte é contado. Não cria servidor nem transporte DAP.

`Entidade(...)` · **módulo** `coral.mundo`

O que faz

Representa uma entidade nomeada com atributos, tags e inventário.

Construtor

Assinatura para consulta

```
Entidade(nome: 'str', atributos: 'dict[str, Any]' = <factory>, tags: 'set[str]' =
  <factory>, inventario: 'list[Any]' = <factory>) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome da entidade.
atributos	dict[str, Any]	não	<factory>	Dicionário de atributos; por padrão, um dicionário novo.
tags	set[str]	não	<factory>	Conjunto de tags; por padrão, um conjunto novo.
inventario	list[Any]	não	<factory>	Lista de itens; por padrão, uma lista nova.

Atributos importantes

Atributo	Significado
`nome`	Identidade textual usada pelo Mundo.
`atributos`	Dados consultáveis por obter ou indexação.
`tags`	Conjunto de marcadores.

Atributo	Significado
`inventario`	Lista ordenada de itens.

Métodos principais

Assinatura para consulta

```
definir(self, nome: 'str', valor: 'Any') -> "'Entidade'"
```

Referência para leitura

```
obter(self, nome: 'str', padrao: 'Any' = None) -> 'Any'
```

Referência para leitura

```
adicionar_tag(self, tag: 'str') -> "'Entidade'"
```

Referência para leitura

```
remover_tag(self, tag: 'str') -> 'None'
```

Referência para leitura

```
possui_tag(self, tag: 'str') -> 'bool'
```

Referência para leitura

```
guardar(self, item: 'Any') -> 'Any'
```

Referência para leitura

```
remover_item(self, item: 'Any') -> 'bool'
```

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de Entidade.

Exemplo

Código Coral

```
de coral.mundo importe Entidade
defina entidade como Entidade("Explorador", {"energia": 10})
mostre entidade.obter("energia")
```

Resultado

Saída esperada

```
10
```

Erros

TypeError para argumentos fora da assinatura; KeyError ao indexar atributo ausente; coleções incompatíveis podem falhar nos métodos.

Atenção

Objeto mutável. Coleções fornecidas ao construtor são mantidas por referência, sem validação completa. definir e adicionar_tag devolvem a entidade. remover_item devolve falso se o item não existe. Criar a entidade não a adiciona a um Mundo.

`Relacao(...)` · módulo `coral.mundo`

O que faz

Representa uma ligação dirigida entre duas entidades.

Construtor

Assinatura para consulta

```
Relacao(origem: 'Entidade', destino: 'Entidade', tipo: 'str', dados: 'dict[str, Any]' =
  <factory>) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
origem	Entidade	sim	sem padrão	Entidade de partida.
destino	Entidade	sim	sem padrão	Entidade de chegada.
tipo	str	sim	sem padrão	Nome do vínculo.
dados	dict[str, Any]	não	<factory>	Dicionário de metadados; por padrão, um dicionário novo.

Atributos importantes

Atributo	Significado
`origem`	Entidade de origem.
`destino`	Entidade de destino.
`tipo`	Categoria do vínculo.
`dados`	Metadados associados.

Retorno

Nova instância de Relacao.

Exemplo

Código Coral

```
de coral.mundo importe Relacao
de coral.mundo importe Entidade
defina relacao como Relacao(Entidade("A"), Entidade("B"), "conhece")
mostre relacao.destino.nome
```

Resultado

Saída esperada

```
B
```

Erros

TypeError para argumentos fora da assinatura; o construtor não valida integralmente os tipos dos campos.

Atenção

Campos do registro são imutáveis, mas dados pode conter valores mutáveis. O construtor não exige que as entidades pertençam ao mesmo mundo e não registra a relação. Use Mundo.relacionar para validar pertencimento e registrar. A comparação de relações desconsidera dados.

`Mundo(...)` · módulo `coral.mundo`

O que faz

Organiza entidades, relações, mapas e um motor reativo em um mesmo contexto.

Construtor

Assinatura para consulta

```
Mundo(nome: 'str' = 'mundo', *, relogio: 'FonteTempo | None' = None, eventos: 'Eventos |
    None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	não	'mundo'	Nome convertido para texto; padrão mundo.
relogio	FonteTempo | None	não	None	FonteTempo do motor de regras, ou nulo.
eventos	Eventos | None	não	None	Barramento Eventos compartilhado, ou nulo.

Atributos importantes

Atributo	Significado
`nome`	Nome do mundo.
`entidades`	Tupla das entidades registradas.
`relacoes`	Tupla das relações registradas.
`mapas`	Tupla dos mapas registrados.
`regras`	MotorRegras associado.
`simulacao`	Simulação temporal criada, ou nulo.

Métodos principais

Assinatura para consulta

```
criar_entidade(self, nome: 'str', **atributos: 'Any') -> 'Entidade'
```

Referência para leitura

```
adicionar(self, entidade: 'Entidade') -> 'Entidade'
```

Referência para leitura

```
buscar(self, nome: 'str') -> 'Entidade | None'
```

Referência para leitura

```
exigir(self, nome: 'str') -> 'Entidade'
```

Referência para leitura

```
remover(self, entidade: 'Entidade | str') -> 'bool'
```

Referência para leitura

```
relacionar(self, origem: 'Entidade | str', destino: 'Entidade | str', tipo: 'str',
  **dados: 'Any') -> 'Relacao'
```

Referência para leitura

```
relacionados(self, entidade: 'Entidade | str', tipo: 'str | None' = None, *, direcao:
  'str' = 'saida') -> 'tuple[Entidade, ...]'
```

Referência para leitura

```
caminho(self, origem: 'Entidade | str', destino: 'Entidade | str', *, tipo: 'str | None'
   = None, bidirecional: 'bool' = True) -> 'tuple[Entidade, ...]'
```

Referência para leitura

```
criar_mapa(self, nome: 'str', topologia, *, propriedades: 'dict[str, Any] | None' =
  None) -> 'Mapa'
```

Referência para leitura

```
criar_simulacao(self, *, tempo=None)
```

Referência para leitura

```
avancar_tempo(self, duracao: 'float')
```

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de Mundo.

Exemplo

Código Coral

```
de coral.mundo importe Mundo
defina mundo como Mundo("Estudo")
defina entidade como mundo.criar_entidade("A", energia=5)
mostre mundo.exigir("A").obter("energia")
```

Resultado

Saída esperada

```
5
```

Erros

ValueError para conflitos de identidade ou relações entre entidades externas; KeyError para entidade ou mapa exigido e ausente; erros de mapas e do runtime temporal são propagados.

Atenção

O construtor não inicia loop nem cria simulação temporal. A simulação é criada explicitamente. Nome de entidade já usado por outro objeto é recusado; remover uma

entidade também remove suas relações. Relacionamentos formam um grafo dirigido; caminho usa busca em largura. Mapas podem compartilhar o runtime reativo do mundo.

`CelulaQuadrada(...)` · módulo `coral.mundo`

O que faz

Identifica uma célula em grade quadrada por coluna e linha.

Construtor

Assinatura para consulta

```
CelulaQuadrada(coluna: 'int', linha: 'int') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
coluna	int	sim	sem padrão	Coordenada inteira horizontal.
linha	int	sim	sem padrão	Coordenada inteira vertical.

Atributos importantes

Atributo	Significado
`coluna`	Coordenada inteira horizontal.
`linha`	Coordenada inteira vertical.

Métodos principais

Assinatura para consulta

```
como_tupla(self) -> 'tuple[int, int]'
```

Retorno

Nova instância de CelulaQuadrada.

Exemplo

Código Coral

```
de coral.mundo importe CelulaQuadrada
defina celula como CelulaQuadrada(2, 3)
mostre celula.como_tupla()
```

Resultado

Saída esperada

```
(2, 3)
```

Erros

ErroMapaCoral com código mapa.coordenada_nao_inteira para coordenada inválida.

Atenção

Registro imutável e utilizável como chave de dicionário. Aceita inteiros do contrato Integral e recusa booleanos ou números fracionários. Coordenadas negativas são válidas; os limites pertencem à topologia, não à célula.

`CelulaHexagonal(...)` · módulo `coral.mundo`

O que faz

Identifica uma célula hexagonal por coordenadas axiais q e r.

Construtor

Assinatura para consulta

```
CelulaHexagonal(q: 'int', r: 'int') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
q	int	sim	sem padrão	Primeira coordenada axial inteira.
r	int	sim	sem padrão	Segunda coordenada axial inteira.

Atributos importantes

Atributo	Significado
`q`	Primeira coordenada axial inteira.
r	Segunda coordenada axial inteira.

Métodos principais

Assinatura para consulta

```
como_tupla(self) -> 'tuple[int, int]'
```

Retorno

Nova instância de CelulaHexagonal.

Exemplo

Código Coral

```
de coral.mundo importe CelulaHexagonal
defina celula como CelulaHexagonal(2, 3)
mostre celula.como_tupla()
```

Resultado

Saída esperada

```
(2, 3)
```

Erros

ErroMapaCoral com código mapa.coordenada_nao_inteira para coordenada inválida.

Atenção

Registro imutável e utilizável como chave de dicionário. Aceita inteiros do contrato Integral e recusa booleanos ou números fracionários. Coordenadas negativas são válidas; os limites pertencem à topologia, não à célula.

`CelulaVoxel(...)` · módulo `coral.mundo`

O que faz

Identifica uma célula de uma grade tridimensional.

Construtor

Assinatura para consulta

```
CelulaVoxel(x: 'int', y: 'int', z: 'int') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	int	sim	sem padrão	Coordenada inteira x.
y	int	sim	sem padrão	Coordenada inteira y.
z	int	sim	sem padrão	Coordenada inteira z.

Atributos importantes

Atributo	Significado
`x`	Coordenada inteira x.
`y`	Coordenada inteira y.
`z`	Coordenada inteira z.

Métodos principais

Assinatura para consulta

```
como_tupla(self) -> 'tuple[int, int, int]'
```

Retorno

Nova instância de CelulaVoxel.

Exemplo

Código Coral

```
de coral.mundo importe CelulaVoxel
defina celula como CelulaVoxel(2, 3, 4)
mostre celula.como_tupla()
```

Resultado

Saída esperada

```
(2, 3, 4)
```

Erros

ErroMapaCoral com código mapa.coordenada_nao_inteira para coordenada inválida.

Atenção

Registro imutável e utilizável como chave de dicionário. Aceita inteiros do contrato Integral e recusa booleanos ou números fracionários. Coordenadas negativas são válidas; os limites pertencem à topologia, não à célula.

`TopologiaQuadrada(...)` · módulo `coral.mundo`

O que faz

Define uma grade bidimensional com células retangulares e escala contínua.

Construtor

Assinatura para consulta

```
TopologiaQuadrada(largura_celula: 'float' = 1.0, altura_celula: 'float' = 1.0, origem:
  'tuple[float, float]' = (0.0, 0.0), limites_configurados: 'object' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
largura_celula	float	não	1.0	Largura positiva e finita de cada célula.
altura_celula	float	não	1.0	Altura positiva e finita de cada célula.
origem	tuple[float, float]	não	(0.0, 0.0)	Dois números finitos que deslocam a grade no mundo.
limites_configurados	object	não	None	Limites inclusivos no formato ((mínimos...), (máximos...)), ou nulo para grade ilimitada.

Atributos importantes

Atributo	Significado
`dimensionalidade`	Dois.
`limitada`	Se há limites configurados.
`limites`	Limites normalizados ou nulo.

Métodos principais

Assinatura para consulta

```
normalizar(self, celula: 'object') -> 'C'
```

Referência para leitura

```
validar(self, celula: 'object') -> 'bool'
```

Referência para leitura

```
dentro_dos_limites(self, celula: 'object') -> 'bool'
```

Referência para leitura

```
vizinhos(self, celula: 'object', *, incluir_diagonais: 'bool' = False) ->
  'tuple[CelulaQuadrada, ...]'
```

Referência para leitura

```
adjacentes(self, primeira: 'object', segunda: 'object', **opcoes: 'object') -> 'bool'
```

Referência para leitura

```
celula_para_mundo(self, celula: 'object', *, ancora: 'str' = 'centro') ->
  'ValorEspacial'
```

Referência para leitura

```
mundo_para_celula(self, coordenada: 'object') -> 'CelulaQuadrada'
```

Retorno

Nova instância de TopologiaQuadrada.

Exemplo

Código Coral

```
de coral.mundo importe TopologiaQuadrada
defina topologia como TopologiaQuadrada()
mostre len(topologia.vizinhos([0, 0]))
```

Resultado

Saída esperada

```
4
```

Erros

ErroMapaCoral para escala, origem, limites, coordenadas, âncora ou dimensionalidade incompatíveis.

Atenção

Objeto imutável. Vizinhança padrão possui quatro células; incluir_diagonais permite oito. Limites filtram vizinhos e normalizar recusa células externas. Conversões contínuas exigem posições espaciais oficiais 2D; mundo_para_celula usa piso, inclusive para coordenadas negativas. Âncoras aceitas: centro e origem.

`TopologiaHexagonal(...)` · módulo `coral.mundo`

O que faz

Define uma grade hexagonal axial com seis vizinhos por célula interior.

Construtor

Assinatura para consulta

```
TopologiaHexagonal(tamanho_celula: 'float' = 1.0, orientacao: 'str' = 'topo_plano',
  origem: 'tuple[float, float]' = (0.0, 0.0), limites_configurados: 'object' = None) ->
  None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tamanho_celula	float	não	1.0	Escala positiva e finita usada na geometria hexagonal.
orientacao	str	não	'topo_plano'	topo_plano ou topo_pontudo.
origem	tuple[float, float]	não	(0.0, 0.0)	Dois números finitos do deslocamento contínuo.
limites_configurados	object	não	None	Limites inclusivos no formato ((mínimos...), (máximos...)), ou nulo para grade ilimitada.

Atributos importantes

Atributo	Significado
`dimensionalidade`	Dois.
`orientacao`	Geometria selecionada.
`limitada`	Presença de limites.
`limites`	Limites axiais normalizados ou nulo.

Métodos principais

Assinatura para consulta

```
normalizar(self, celula: 'object') -> 'C'
```

Referência para leitura

```
validar(self, celula: 'object') -> 'bool'
```

Referência para leitura

```
dentro_dos_limites(self, celula: 'object') -> 'bool'
```

Referência para leitura

```
vizinhos(self, celula: 'object', **_: 'object') -> 'tuple[CelulaHexagonal, ...]'
```

Referência para leitura

```
adjacentes(self, primeira: 'object', segunda: 'object', **opcoes: 'object') -> 'bool'
```

Referência para leitura

```
celula_para_mundo(self, celula: 'object', *, ancora: 'str' = 'centro') ->
  'ValorEspacial'
```

Referência para leitura

```
mundo_para_celula(self, coordenada: 'object') -> 'CelulaHexagonal'
```

Retorno

Nova instância de TopologiaHexagonal.

Exemplo

Código Coral

```
de coral.mundo importe TopologiaHexagonal
defina topologia como TopologiaHexagonal()
mostre len(topologia.vizinhos([0, 0]))
```

Resultado

Saída esperada

```
6
```

Erros

ErroMapaCoral para escala, orientação, origem, limites, âncora ou coordenada incompatíveis.

Atenção

Objeto imutável. Os limites são aplicados às coordenadas axiais q e r. celula_para_mundo aceita somente a âncora centro. mundo_para_celula exige posição espacial oficial 2D e usa arredondamento cúbico para escolher o hexágono. Vizinhos fora dos limites são omitidos.

`TopologiaVoxel(...)` · módulo `coral.mundo`

O que faz

Define uma grade tridimensional com escala independente em cada eixo.

Construtor

Assinatura para consulta

```
TopologiaVoxel(tamanho_x: 'float' = 1.0, tamanho_y: 'float' = 1.0, tamanho_z: 'float' =
  1.0, origem: 'tuple[float, float, float]' = (0.0, 0.0, 0.0), limites_configurados:
  'object' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tamanho_x	float	não	1.0	Escala positiva e finita no eixo x.
tamanho_y	float	não	1.0	Escala positiva e finita no eixo y.
tamanho_z	float	não	1.0	Escala positiva e finita no eixo z.
origem	tuple[float, float, float]	não	(0.0, 0.0, 0.0)	Três números finitos do deslocamento contínuo.
limites_configurados	object	não	None	Limites inclusivos no formato ((mínimos...), (máximos...)), ou nulo para grade ilimitada.

Atributos importantes

Atributo	Significado
`dimensionalidade`	Três.
`limitada`	Presença de limites.

Atributo	Significado
`limites`	Limites tridimensionais normalizados ou nulo.

Métodos principais

Assinatura para consulta

```
normalizar(self, celula: 'object') -> 'C'
```

Referência para leitura

```
validar(self, celula: 'object') -> 'bool'
```

Referência para leitura

```
dentro_dos_limites(self, celula: 'object') -> 'bool'
```

Referência para leitura

```
vizinhos(self, celula: 'object', **_: 'object') -> 'tuple[CelulaVoxel, ...]'
```

Referência para leitura

```
adjacentes(self, primeira: 'object', segunda: 'object', **opcoes: 'object') -> 'bool'
```

Referência para leitura

```
celula_para_mundo(self, celula: 'object', *, ancora: 'str' = 'centro') ->
  'ValorEspacial'
```

Referência para leitura

```
mundo_para_celula(self, coordenada: 'object') -> 'CelulaVoxel'
```

Retorno

Nova instância de TopologiaVoxel.

Exemplo

Código Coral

```
de coral.mundo importe TopologiaVoxel
defina topologia como TopologiaVoxel()
mostre len(topologia.vizinhos([0, 0, 0]))
```

Resultado

Saída esperada

```
6
```

Erros

ErroMapaCoral para escala, origem, limites, âncora ou coordenada incompatíveis.

Atenção

Objeto imutável. Vizinhança usa as seis faces, sem diagonais. Âncoras aceitas: centro e origem. Conversões contínuas exigem posição espacial oficial 3D; mundo_para_celula usa piso. Células de grades quadradas ou hexagonais são recusadas.

`ErroMapaCoral(...)` · módulo `coral.mundo`

O que faz

Representa falhas de contrato no domínio de mapas.

Construtor

Assinatura para consulta

```
ErroMapaCoral(mensagem: 'str', *, codigo: 'str' = 'mapa.invalido', contexto: 'dict[str,
  Any] | None' = None) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mensagem	str	sim	sem padrão	Descrição da falha.
codigo	str	não	'mapa.invalido'	Código estável de diagnóstico; padrão mapa.invalido.
contexto	dict[str, Any] | None	não	None	Dicionário de detalhes, ou nulo.

Atributos importantes

Atributo	Significado
`codigo`	Identificador do diagnóstico.
`contexto`	Dados associados à falha.

Retorno

Nova instância de ErroMapaCoral.

Exemplo

Código Coral

```
de coral.mundo importe ErroMapaCoral
defina erro como ErroMapaCoral("Célula inválida", codigo="mapa.coordenada_nao_inteira")
mostre erro.codigo
```

Resultado

Saída esperada

```
mapa.coordenada_nao_inteira
```

Erros

TypeError para argumentos fora da assinatura ou contexto incompatível com dict.

Atenção

Deriva de ValueError e pode ser capturado por essa classe base. contexto recebe uma cópia superficial do dicionário. Prefira codigo para identificar o diagnóstico sem depender do texto da mensagem.

`BlocoMapa(...)` · **módulo** `coral.mundo`

O que faz

Representa material ou terreno identificado, com propriedades portáteis.

Construtor

Assinatura para consulta

```
BlocoMapa(identificador: 'str', propriedades: 'Mapping[str, Any] | None' = None) ->
  'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
identificador	str	sim	sem padrão	Nome convertido para texto e sem espaços nas extremidades; precisa ser não vazio.
propriedades	Mapping[str, Any] | None	não	None	Mapeamento com chaves textuais e valores portáteis, ou nulo.

Atributos importantes

Atributo	Significado
`identificador`	Identidade normalizada do bloco.
`propriedades`	Mapeamento de propriedades protegido contra alterações diretas.

Retorno

Nova instância de BlocoMapa.

Exemplo

Código Coral

```
de coral.mundo importe BlocoMapa
defina bloco como BlocoMapa("pedra", {"dureza": 3})
mostre bloco.propriedades["dureza"]
```

Resultado

Saída esperada

```
3
```

Erros

ErroMapaCoral para identificador vazio, propriedades inválidas ou valor não portátil.

Atenção

Registro imutável. Propriedades são validadas e clonadas pelo contrato portátil; a consulta devolve uma visão somente leitura de uma nova cópia. O bloco não ocupa uma célula até ser colocado no mapa.

`ConteudoCelula(...)` · módulo `coral.mundo`

O que faz

Associa um valor portátil a propriedades e a uma política opcional de travessia.

Construtor

Assinatura para consulta

```
ConteudoCelula(valor: 'Any', propriedades: 'Mapping[str, Any] | None' = None, *,
  travessia: 'bool | None' = None, custo: 'float | int | None' = None, motivo: 'str |
  None' = None) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
valor	Any	sim	sem padrão	Valor aceito pelo contrato portátil.
propriedades	Mapping[str, Any] | None	não	None	Mapeamento com chaves textuais e dados portáteis, ou nulo.
travessia	bool | None	não	None	Verdadeiro, falso ou nulo para não declarar permissão.
custo	float | int | None	não	None	Número finito e não negativo, ou nulo.
motivo	str | None	não	None	Texto que explica a política, ou nulo.

Atributos importantes

Atributo	Significado
`valor`	Cópia do valor portátil.
`propriedades`	Visão de uma cópia somente leitura.
`travessia_permitida`	Permissão declarada ou nulo.
`custo_travessia`	Custo normalizado ou nulo.
`declarou_travessia`	Se há alguma informação de travessia.

Métodos principais

Assinatura para consulta

```
com_valor(self, valor: 'Any') -> "'ConteudoCelula'"
```

Referência para leitura

```
com_propriedades(self, propriedades: 'Mapping[str, Any]') -> "'ConteudoCelula'"
```

Referência para leitura

```
com_travessia(self, *, permitida: 'bool | None' = None, custo: 'float | int | None' =
  None, motivo: 'str | None' = None) -> "'ConteudoCelula'"
```

Retorno

Nova instância de ConteudoCelula.

Exemplo

Código Coral

```
de coral.mundo importe ConteudoCelula
defina conteudo como ConteudoCelula("lama", travessia=verdadeiro, custo=2)
mostre conteudo.custo_travessia
```

Resultado

Saída esperada

```
2.0
```

Erros

ErroMapaCoral para dados não portáteis, propriedades inválidas, custo inválido ou política de travessia incompatível.

Atenção

Registro imutável com clonagem do valor e propriedades. com_valor, com_propriedades e com_travessia criam outro registro. com_travessia substitui a política usando os argumentos informados, inclusive os padrões nulos. O mapa agrega custos e bloqueios das camadas que participam da travessia.

`CamadaMapa(...)` · módulo `coral.mundo`

O que faz

Organiza conteúdos em uma camada nomeada, ordenada e esparsa.

Construtor

Assinatura para consulta

```
CamadaMapa(nome: 'str', *, ordem: 'int' = 0, propriedades: 'Mapping[str, Any] | None' =
  None, participa_travessia: 'bool' = True, _sequencia: 'int' = 0) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome convertido para texto e sem espaços nas extremidades; não vazio.
ordem	int	não	0	Inteiro da ordem da camada, sem booleanos.
propriedades	Mapping[str, Any] | None	não	None	Mapeamento portátil com chaves textuais, ou nulo.
participa_travessia	bool	não	True	Se os conteúdos contribuem para a avaliação de passagem.
_sequencia	int	não	0	Ordem interna de inserção; prefira o padrão e deixe o Mapa administrar esse campo.

Atributos importantes

Atributo	Significado
`nome`	Nome normalizado.
`ordem`	Ordem principal.
`quantidade`	Quantidade de células preenchidas.

Atributo	Significado
`celulas`	Tupla das células ocupadas.
`propriedades`	Cópia portátil protegida.

Métodos principais

Assinatura para consulta

```
possui(self, celula: 'CelulaMapa') -> 'bool'
```

Referência para leitura

```
conteudo(self, celula: 'CelulaMapa') -> 'ConteudoCelula | None'
```

Retorno

Nova instância de CamadaMapa.

Exemplo

Código Coral

```
de coral.mundo importe CamadaMapa
defina camada como CamadaMapa("terreno", ordem=1)
mostre camada.quantidade
```

Resultado

Saída esperada

```
0
```

Erros

ErroMapaCoral para nome, ordem, propriedades ou participa_travessia inválidos; erro de conversão para _sequencia incompatível.

Atenção

Objeto mutável. O construtor não o registra em um mapa. Use Mapa.adicionar_camada e os métodos do mapa para alterar conteúdos e publicar mudanças. Consulta direta da camada exige uma célula já normalizada. Conteúdo ausente devolve nulo.

`Mapa(...)` · módulo `coral.mundo`

O que faz

Mantém camadas, regiões e conteúdos discretos sem depender de renderização.

Construtor

Assinatura para consulta

```
Mapa(nome: 'str', topologia: 'Any', *, propriedades: 'Mapping[str, Any] | None' = None,
  eventos: 'Eventos | None' = None) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome convertido para texto e sem espaços nas extremidades; não vazio.
topologia	Any	sim	sem padrão	Objeto com normalizar, vizinhos, dimensionalidade e nome; as topologias oficiais atendem ao contrato.
propriedades	Mapping[str, Any] | None	não	None	Mapeamento de dados portáteis com chaves textuais, ou nulo.
eventos	Eventos | None	não	None	Barramento de eventos associado, ou nulo sem barramento oculto.

Atributos importantes

Atributo	Significado
`nome`	Nome do mapa.
`topologia`	Política de coordenadas.
`camadas`	Tupla das camadas ordenadas.
`regioes`	Tupla das regiões registradas.
`eventos`	Barramento associado ou nulo.

Métodos principais

Assinatura para consulta

```
adicionar_camada(self, nome: 'str', *, ordem: 'int | None' = None, propriedades:
  'Mapping[str, Any] | None' = None, participa_travessia: 'bool' = True) -> 'CamadaMapa'
```

Referência para leitura

```
exigir_camada(self, nome: 'str') -> 'CamadaMapa'
```

Referência para leitura

```
definir(self, camada: 'str', celula: 'object', valor: 'Any', *, propriedades:
  'Mapping[str, Any] | None' = None, travessia: 'bool | None' = None, custo: 'float | int
  | None' = None, motivo: 'str | None' = None, causa: 'Any' = None) -> 'MudancaMapa'
```

Referência para leitura

```
obter(self, camada: 'str', celula: 'object', padrao: 'Any' = None) -> 'Any'
```

Referência para leitura

```
remover(self, camada: 'str', celula: 'object', *, causa: 'Any' = None) -> 'MudancaMapa |
   None'
```

Referência para leitura

```
avaliar_travessia(self, origem: 'object', destino: 'object') -> 'ResultadoPassagem'
```

Referência para leitura

```
tentar_travessia(self, origem: 'object', destino: 'object', *, causa: 'Any' = None) ->
  'ResultadoPassagem'
```

Referência para leitura

```
criar_regiao(self, nome: 'str', *, celulas: 'Iterable[object] | None' = None, forma:
  'Any' = None, referencia: 'str' = 'centro', propriedades: 'Mapping[str, Any] | None' =
  None) -> 'Any'
```

Referência para leitura

```
regioes_em(self, celula_ou_posicao: 'object') -> 'tuple[Any, ...]'
```

Referência para leitura

```
vizinhos(self, celula: 'object', **opcoes: 'Any') -> 'tuple[CelulaMapa, ...]'
```

Referência para leitura

```
aplicar_lote(self, operacoes: 'Iterable[Mapping[str, Any]]') -> 'tuple[MudancaMapa,
  ...]'
```

Retorno

Nova instância de Mapa.

Exemplo

Código Coral

```
de coral.mundo importe Mapa
de coral.mundo importe TopologiaQuadrada
defina mapa como Mapa("Estudo", TopologiaQuadrada())
execute mapa.adicionar_camada("terreno")
execute mapa.definir("terreno", [0, 0], "grama")
mostre mapa.obter("terreno", [0, 0])
```

Resultado

Saída esperada

```
grama
```

Erros

ErroMapaCoral para nome, topologia, propriedades, coordenadas, conteúdo ou política inválidos; erros de callbacks de eventos podem ser propagados.

Atenção

Criação não adiciona camadas ou regiões. Valores e propriedades são portáteis e clonados. Alterações devolvem MudancaMapa e podem ser canceladas por eventos antes da aplicação; verifique aplicada. avaliar_travessia consulta as camadas do destino, sem mover entidade nem exigir adjacência.

`MudancaMapa(...)` · **módulo** `coral.mundo`

O que faz

Registra uma proposta ou resultado de alteração de uma célula.

Construtor

Assinatura para consulta

```
MudancaMapa(mapa: 'str', camada: 'str', celula: 'CelulaMapa', operacao: 'str', anterior:
   'ConteudoCelula | None', novo: 'ConteudoCelula | None', causa: 'Any' = None, aplicada:
  'bool' = True, cancelada: 'bool' = False, motivo_cancelamento: 'str | None' = None,
  evento_sequencia: 'int | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mapa	str	sim	sem padrão	Nome do mapa.
camada	str	sim	sem padrão	Nome da camada.
celula	CelulaMapa	sim	sem padrão	Célula normalizada afetada.
operacao	str	sim	sem padrão	Nome da operação.
anterior	ConteudoCelula | None	sim	sem padrão	ConteudoCelula anterior ou nulo.
novo	ConteudoCelula | None	sim	sem padrão	ConteudoCelula novo ou nulo.
causa	Any	não	None	Informação da causa, ou nulo.
aplicada	bool	não	True	Se a alteração foi aplicada; padrão verdadeiro.
cancelada	bool	não	False	Se houve cancelamento; padrão falso.
motivo_cancelamento	str | None	não	None	Texto do motivo ou nulo.
evento_sequencia	int | None	não	None	Sequência do evento relacionado ou nulo.

Atributos importantes

Atributo	Significado
`aplicada`	Resultado efetivo da aplicação.
`cancelada`	Indicação de cancelamento.
`anterior`	Conteúdo antes da operação.
`novo`	Conteúdo proposto ou aplicado.

Retorno

Nova instância de MudancaMapa.

Exemplo

Código Coral

```
de coral.mundo importe MudancaMapa
de coral.mundo importe Mapa, TopologiaQuadrada
defina mapa como Mapa("Estudo", TopologiaQuadrada())
execute mapa.adicionar_camada("terreno")
defina mudanca como mapa.definir("terreno", [0, 0], "grama")
mostre isinstance(mudanca, MudancaMapa)
mostre mudanca.aplicada
```

Resultado

Saída esperada

```
True
True
```

Erros

TypeError para argumentos fora da assinatura; o construtor não aplica as validações das operações de Mapa.

Atenção

Registro imutável nos campos, sem validação integral no construtor. Criar um registro manualmente não altera o mapa. Prefira o registro devolvido pelo mapa, que normaliza a célula, clona dados e registra aplicação ou cancelamento.

`ResultadoPassagem(...)` · módulo `coral.mundo`

O que faz

Descreve permissão, custo e responsáveis por uma consulta de travessia.

Construtor

Assinatura para consulta

```
ResultadoPassagem(permitida: 'bool', custo: 'float', motivo: 'str | None',
  camadas_responsaveis: 'tuple[str, ...]', origem: 'CelulaMapa', destino: 'CelulaMapa') ->
   None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
permitida	bool	sim	sem padrão	Condição da passagem.
custo	float	sim	sem padrão	Soma dos custos declarados.
motivo	str | None	sim	sem padrão	Motivo do bloqueio, ou nulo.
camadas_responsaveis	tuple[str, ...]	sim	sem padrão	Tupla de nomes das camadas que declararam política no destino.
origem	CelulaMapa	sim	sem padrão	Célula de origem.
destino	CelulaMapa	sim	sem padrão	Célula de destino.

Atributos importantes

Atributo	Significado
`permitida`	Se nenhuma camada participante bloqueia.
`custo`	Custo total.
`motivo`	Explicação do primeiro bloqueio.
`camadas_responsaveis`	Camadas com declaração de travessia.

Retorno

Nova instância de ResultadoPassagem.

Exemplo

Código Coral

```
de coral.mundo importe ResultadoPassagem
de coral.mundo importe Mapa, TopologiaQuadrada
defina mapa como Mapa("Estudo", TopologiaQuadrada())
defina resultado como mapa.avaliar_travessia([0, 0], [1, 0])
mostre isinstance(resultado, ResultadoPassagem)
mostre resultado.permitida
```

Resultado

Saída esperada

```
True
True
```

Erros

TypeError para argumentos fora da assinatura; criação manual não normaliza células nem valida custo.

Atenção

Registro imutável nos campos, sem validação integral no construtor. Prefira Mapa.avaliar_travessia. Qualquer bloqueio de uma camada participante impede a passagem. O resultado descreve a política; não move objetos nem verifica adjacência ou caminho.

`RegiaoMapa(...)` · módulo `coral.mundo`

O que faz

Define uma região por células explícitas ou por uma forma espacial oficial.

Construtor

Assinatura para consulta

```
RegiaoMapa(mapa: 'Any', nome: 'str', *, celulas: 'Iterable[object] | None' = None,
  forma: 'Any' = None, referencia: 'str' = 'centro', propriedades: 'Mapping[str, Any] |
  None' = None) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mapa	Any	sim	sem padrão	Mapa ao qual a região pertence.
nome	str	sim	sem padrão	Nome textual normalizado e não vazio.
celulas	Iterable[object] | None	não	None	Iterável de células, ou nulo quando usar forma.
forma	Any	não	None	Forma espacial oficial com dimensão compatível, ou nulo quando usar celulas.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
referencia	str	não	'centro'	centro ou origem para consultar a forma com uma célula.
propriedades	Mapping[str, Any] | None	não	None	Mapeamento portátil de propriedades, ou nulo.

Atributos importantes

Atributo	Significado
`mapa`	Mapa associado.
`nome`	Nome da região.
`tipo`	celulas ou forma.
`celulas_explicitas`	Tupla normalizada das células declaradas.
`propriedades`	Cópia protegida das propriedades.

Métodos principais

Assinatura para consulta

```
contem(self, celula_ou_posicao: 'object') -> 'bool'
```

Referência para leitura

```
celulas(self, *, limites: 'object' = None) -> 'tuple[CelulaMapa, ...]'
```

Retorno

Nova instância de RegiaoMapa.

Exemplo

Código Coral

```
de coral.mundo importe RegiaoMapa
de coral.mundo importe Mapa, TopologiaQuadrada
defina mapa como Mapa("Estudo", TopologiaQuadrada())
defina regiao como RegiaoMapa(mapa, "Base", celulas=[[0, 0]])
mostre regiao.contem([0, 0])
```

Resultado

Saída esperada

```
True
```

Erros

ErroMapaCoral para nome, alternativas, forma, dimensão, referência ou limites de enumeração inválidos; erros do contrato do mapa podem ser propagados.

Atenção

Exige exatamente uma alternativa entre celulas e forma. Células são normalizadas, deduplicadas e ordenadas. O construtor não registra a região no mapa. Regiões por forma

em mapa ilimitado exigem limites explícitos para enumerar células; topologia hexagonal aceita somente âncora centro na conversão.

`ObservadorRegiaoMapa(...)` · módulo `coral.mundo`

O que faz

Reconhece entradas e saídas de um alvo entre avaliações explícitas.

Construtor

Assinatura para consulta

```
ObservadorRegiaoMapa(nome: 'str', alvo: 'Any', mapa: 'Mapa', regioes:
  'Iterable[RegiaoMapa | str]', resolvedor: 'ResolvedorLocalizacao', eventos: 'Eventos',
  *, identificador_alvo: 'Any' = None) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome do observador.
alvo	Any	sim	sem padrão	Objeto cuja localização será resolvida.
mapa	Mapa	sim	sem padrão	Mapa das regiões observadas.
regioes	Iterable[RegiaoMapa | str]	sim	sem padrão	Iterável não vazio de RegiaoMapa ou nomes registrados no mapa.
resolvedor	ResolvedorLocalizacao	sim	sem padrão	Função que recebe alvo e contexto e devolve célula ou posição espacial.
eventos	Eventos	sim	sem padrão	Barramento que recebe os eventos de entrada e saída.
identificador_alvo	Any	não	None	Identificador portátil explícito, ou nulo para obter identidade do alvo.

Atributos importantes

Atributo	Significado
`regioes_anteriores`	Tupla de nomes detectados na avaliação anterior.
`identificador_alvo`	Identidade usada nos eventos.

Métodos principais

Assinatura para consulta

```
avaliar(self, contexto: 'Any') -> 'bool'
```

Retorno

Nova instância de ObservadorRegiaoMapa.

Exemplo

Código Coral

```
de coral.mundo importe ObservadorRegiaoMapa
de coral.mundo importe Mapa, TopologiaQuadrada
defina mapa como Mapa("Estudo", TopologiaQuadrada())
de coral.mundo importe RegiaoMapa
de coral.regras importe Eventos
defina regiao como RegiaoMapa(mapa, "Base", celulas=[[0, 0]])
crie a função localizar com alvo, contexto
    retorne [0, 0]
fim
defina observador como ObservadorRegiaoMapa("Teste", "A", mapa, [regiao], localizar,
  Eventos())
mostre observador.avaliar(nulo)
```

Resultado

Saída esperada

```
False
```

Erros

TypeError para resolvedor não chamável; ValueError para coleção vazia ou regiões incompatíveis; KeyError para nome de região ausente; erros de identidade portátil, consulta e callbacks são propagados.

Atenção

A primeira avaliação estabelece a referência e devolve falso. Avaliações posteriores emitem mapa.regiao_saiu e mapa.regiao_entrou quando há mudança. Não inicia thread nem amostra automaticamente. Todos os objetos de região devem pertencer ao mesmo mapa; o resolvedor e os ouvintes podem executar código do programa.

`SequenciaProcedural(...)` · **módulo** `coral.procedural`

O que faz

Produz valores por índice sem depender da ordem das consultas.

Construtor

Assinatura para consulta

```
SequenciaProcedural(semente_raiz: 'Any' = 0, gerador: 'Callable[[GeradorProcedural,
  int], Any] | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
semente_raiz	Any	não	0	Valor de semente aceito pelo contrato procedural; padrão zero.
gerador	Callable[[GeradorProcedural, int], Any] | None	não	None	Função que recebe GeradorProcedural e índice, ou nulo para decimais pseudoaleatórios.

Atributos importantes

Atributo	Significado
`gerador`	GeradorProcedural usado pela sequência.

Métodos principais

Assinatura para consulta

```
obter(self, indice: 'int') -> 'Any'
```

Referência para leitura

```
trecho(self, inicio: 'int', quantidade: 'int', *, limite_itens: 'int' = 1000000) ->
  'tuple[Any, ...]'
```

Retorno

Nova instância de SequenciaProcedural.

Exemplo

Código Coral

```
de coral.procedural importe SequenciaProcedural
crie a função dobro com gerador, indice
    retorne indice vezes 2
fim
defina sequencia como SequenciaProcedural(7, dobro)
mostre sequencia.obter(3)
```

Resultado

Saída esperada

```
6
```

Erros

ValueError ou TypeError para índice, trecho ou semente inválidos; erros do callback são propagados.

Atenção

O gerador padrão usa namespace sequencia. obter recusa booleanos, converte outros índices com int e recusa valores negativos; trecho aplica validação exata e limite de itens. Funções próprias precisam ser determinísticas para preservar a independência da ordem das consultas.

`CampoProcedural(...)` · módulo `coral.procedural`

O que faz

Encapsula uma função numérica consultada por coordenadas e permite composição de campos.

Construtor

Assinatura para consulta

```
CampoProcedural(funcao: 'Callable[..., float]', *, nome: 'str' = 'campo', metadados:
  'Mapping[str, Any] | None' = None, dimensoes_min: 'int' = 1, dimensoes_max: 'int | None'
   = None, intervalo: 'tuple[float, float] | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
funcao	Callable[..., float]	sim	sem padrão	Função chamável que recebe coordenadas e devolve número finito.
nome	str	não	'campo'	Nome convertido para texto; padrão campo.
metadados	Mapping[str, Any] | None	não	None	Mapeamento copiado superficialmente, ou nulo.
dimensoes_min	int	não	1	Inteiro exato positivo do mínimo de coordenadas.
dimensoes_max	int | None	não	None	Inteiro exato positivo, não menor que dimensoes_min, ou nulo.
intervalo	tuple[float, float] | None	não	None	Par de limites finitos ordenados, ou nulo para intervalo desconhecido.

Atributos importantes

Atributo	Significado
`nome`	Nome do campo.
`dimensoes_min`	Limite inferior de coordenadas.
`dimensoes_max`	Limite superior ou nulo.
`intervalo`	Faixa declarada ou nulo.
`metadados`	Informações descritivas.

Métodos principais

Assinatura para consulta

```
valor(self, *coordenadas: 'float') -> 'float'
```

Referência para leitura

```
transformar(self, funcao: 'Callable[[float], float]', *, nome: 'str | None' = None,
  intervalo: 'tuple[float, float] | None' = None, operacao: 'str' = 'transformar') ->
  "'CampoProcedural'"
```

Referência para leitura

```
limitar(self, minimo: 'float', maximo: 'float') -> "'CampoProcedural'"
```

Referência para leitura

```
normalizar(self, origem_min: 'float', origem_max: 'float', destino_min: 'float' = 0.0,
  destino_max: 'float' = 1.0) -> "'CampoProcedural'"
```

Referência para leitura

```
somar(self, outro: "'CampoProcedural | float'") -> "'CampoProcedural'"
```

Referência para leitura

```
multiplicar(self, outro: "'CampoProcedural | float'") -> "'CampoProcedural'"
```

Referência para leitura

```
mascara(self, *, minimo: 'float | None' = None, maximo: 'float | None' = None, dentro:
  'float' = 1.0, fora: 'float' = 0.0) -> "'CampoProcedural'"
```

Retorno

Nova instância de CampoProcedural.

Exemplo

Código Coral

```
de coral.procedural importe CampoProcedural
crie a função quadrado com x
    retorne x vezes x
fim
defina campo como CampoProcedural(quadrado, dimensoes_max=1)
mostre campo.valor(3)
```

Resultado

Saída esperada

```
9.0
```

Erros

TypeError para função não chamável; ValueError para dimensão, intervalo, coordenada ou retorno inválidos; erros do callback são propagados.

Atenção

intervalo descreve o campo e não limita automaticamente os valores. Cada consulta verifica dimensão, coordenadas e retorno finitos. limitar cria outro campo que efetivamente recorta valores; somar e multiplicar combinam campos ou constantes. Efeitos do callback dependem do programa.

`ReceitaProcedural(...)` · módulo `coral.procedural`

O que faz

Descreve uma operação procedural portátil e reconstruível sem guardar callbacks.

Construtor

Assinatura para consulta

```
ReceitaProcedural(algoritmo: 'str', *, semente_raiz: 'Any' = 0, parametros:
  'Mapping[str, Any] | None' = None, versao_algoritmo: 'int' = 1) -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
algoritmo	str	sim	sem padrão	Nome de algoritmo suportado pelo registro de receitas.
semente_raiz	Any	não	0	Semente raiz aceita pelo contrato procedural.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
parametros	Mapping[str, Any] | None	não	None	Mapeamento portátil específico do algoritmo, ou nulo para seus padrões.
versao_algoritmo	int	não	1	Versão suportada do algoritmo; atualmente um.

Atributos importantes

Atributo	Significado
`algoritmo`	Operação registrada.
`versao_algoritmo`	Versão validada.
`semente`	Semente normalizada.
`parametros`	Cópia dos parâmetros normalizados.
`contrato`	Identificador versionado da receita.

Métodos principais

Assinatura para consulta

```
para_dados(self) -> 'dict[str, Any]'
```

Referência para leitura

```
para_json(self, *, identar: 'int | None' = None) -> 'str'
```

Referência para leitura

```
de_dados(dados: 'Mapping[str, Any]') -> "'ReceitaProcedural'"
```

Referência para leitura

```
de_json(texto: 'str') -> "'ReceitaProcedural'"
```

Retorno

Nova instância de ReceitaProcedural.

Exemplo

Código Coral

```
de coral.procedural importe ReceitaProcedural
defina receita como ReceitaProcedural("distribuir_pontos", semente_raiz=7,
  parametros={"largura": 10, "altura": 10, "quantidade": 2, "max_tentativas_por_ponto":
  10, "limite_tentativas_total": 20})
mostre receita.para_dados()["algoritmo"]
```

Resultado

Saída esperada

```
distribuir_pontos
```

Erros

ValueError para algoritmo, versão, parâmetros, semente ou contrato incompatíveis; erros de decodificação JSON podem ser propagados.

Atenção

Criar a receita valida algoritmo, versão e parâmetros, mas não gera seu produto. parametros devolve uma cópia profunda. para_dados e para_json preservam contrato, versão, semente e parâmetros; de_dados e de_json revalidam. A reconstrução e seus limites de custo pertencem a reconstruir_receita.

`criar_mundo(...)` · módulo `coral.mundo`

O que faz

Cria um Mundo nomeado com motor reativo próprio.

Assinatura

Assinatura para consulta

```
criar_mundo(nome: 'str' = 'mundo') -> 'Mundo'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	não	'mundo'	Nome convertido para texto; padrão mundo.

Retorno

Nova instância de Mundo.

Exemplo

Código Coral

```
de coral.mundo importe criar_mundo
defina mundo como criar_mundo("Estudo")
mostre mundo.nome
```

Resultado

Saída esperada

```
Estudo
```

Erros

TypeError para argumentos fora da assinatura; erros do construtor Mundo são propagados.

Atenção

Equivale a Mundo(nome). Não inicia simulação nem loop. Para compartilhar relógio ou eventos, use o construtor Mundo com esses argumentos.

`regra(...)` · módulo `coral.regras`

O que faz

Cria uma regra independente a partir de condição e ação.

Assinatura

Assinatura para consulta

```
regra(nome: 'str', quando: 'Condicao', entao: 'Acao', *, modo: 'str' = 'ao_ativar',
  prioridade: 'str | int' = 'normal', max_execucoes: 'int | None' = None, cooldown:
  'float' = 0.0) -> 'Regra'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome identificador da regra.
quando	Condicao	sim	sem padrão	Função que recebe contexto e devolve condição de ativação.
entao	Acao	sim	sem padrão	Função que recebe contexto e realiza a ação.
modo	str	não	'ao_ativar'	ao_ativar executa na transição para verdadeiro; sempre executa enquanto verdadeiro; uma_vez executa somente na primeira ativação.
prioridade	str | int	não	'normal'	Inteiro, ou baixa, normal, alta e crítica; padrão normal.
max_execucoes	int | None	não	None	Inteiro positivo máximo de disparos, ou nulo para não limitar.
cooldown	float	não	0.0	Duração finita não negativa entre disparos; padrão zero.

Retorno

Regra criada, ainda sem registro em um motor.

Exemplo

Código Coral

```
de coral.regras importe regra
crie a função pronta com contexto
    retorne verdadeiro
fim
crie a função agir com contexto
    mostre contexto
fim
defina r como regra("aviso", pronta, agir)
mostre r.avaliar("pronto")
```

Resultado

Saída esperada

```
pronto
True
```

Erros

ValueError para prioridade, limite ou cooldown inválidos; modo desconhecido falha ao avaliar; funções não chamáveis falham durante a avaliação.

Atenção

A avaliação é explícita. Erros de callbacks são propagados. Cooldown requer instante lógico para controlar disparos sucessivos. O nome só precisa ser único quando registrado em MotorRegras.

`Regra(...)` · módulo `coral.regras`

O que faz

Mantém condição, ação e estado de disparo de uma regra reativa.

Construtor

Assinatura para consulta

```
Regra(nome: 'str', condicao: 'Condicao', acao: 'Acao', modo: 'str' = 'ao_ativar',
  habilitada: 'bool' = True, ativa: 'bool' = False, execucoes: 'int' = 0, prioridade: 'str
   | int' = 'normal', max_execucoes: 'int | None' = None, cooldown: 'float' = 0.0,
  ultimo_disparo: 'float | None' = None, motivo_ultima_supressao: 'str | None' = None,
  ultima_condicao: 'bool | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Identificador da regra.
condicao	Condicao	sim	sem padrão	Função que recebe contexto e devolve valor lógico.
acao	Acao	sim	sem padrão	Função que recebe contexto quando ocorre disparo.
modo	str	não	'ao_ativar'	ao_ativar executa na transição para verdadeiro; sempre executa enquanto verdadeiro; uma_vez executa somente na primeira ativação.
habilitada	bool	não	True	Se pode ser avaliada; padrão verdadeiro.
ativa	bool	não	False	Estado anterior da condição; padrão falso.
execucoes	int	não	0	Contagem inicial de ações; padrão zero.
prioridade	str | int	não	'normal'	Inteiro, ou baixa, normal, alta e crítica; padrão normal.
max_execucoes	int | None	não	None	Inteiro positivo máximo de disparos, ou nulo para não limitar.
cooldown	float	não	0.0	Duração finita não negativa entre disparos; padrão zero.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
ultimo_disparo	float | None	não	None	Instante lógico do último disparo, ou nulo.
motivo_ultima_supressao	str | None	não	None	Motivo anterior de supressão, ou nulo.
ultima_condicao	bool | None	não	None	Resultado anterior da condição, ou nulo.

Atributos importantes

Atributo	Significado
`ativa`	Última condição avaliada.
`execucoes`	Ações concluídas.
`prioridade`	Prioridade numérica normalizada.
`motivo_ultima_supressao`	Por que a última avaliação não disparou.

Métodos principais

Assinatura para consulta

```
avaliar(self, contexto: 'Any', *, instante: 'float | None' = None) -> 'bool'
```

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de Regra.

Exemplo

Código Coral

```
de coral.regras importe Regra
crie a função pronta com contexto
    retorne verdadeiro
fim
crie a função agir com contexto
    mostre contexto
fim
defina r como Regra("aviso", pronta, agir)
mostre r.avaliar("pronto")
mostre r.execucoes
```

Resultado

Saída esperada

```
pronto
True
1
```

Erros

ValueError para prioridade, limite, duração, modo ou instante lógico ausente no controle de cooldown; erros de callbacks são propagados.

Atenção

Estado mutável. Prefira os valores iniciais padrão para os campos de execução. ao_ativar depende da transição da condição; a ação só aumenta execucoes após terminar. O construtor valida prioridade, max_execucoes e cooldown; modo é validado ao avaliar. Não registra a regra em um motor.

`GrupoRegras(...)` · módulo `coral.regras`

O que faz

Agrupa referências a regras para habilitar ou desabilitar o conjunto.

Construtor

Assinatura para consulta

```
GrupoRegras(nome: 'str', regras: 'tuple[Regra, ...]') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome do grupo.
regras	tuple[Regra, ...]	sim	sem padrão	Tupla das instâncias Regra participantes.

Atributos importantes

Atributo	Significado
`nome`	Identidade do grupo.
`regras`	Referências às regras agrupadas.

Métodos principais

Assinatura para consulta

```
habilitar(self) -> 'None'
```

Referência para leitura

```
desabilitar(self) -> 'None'
```

Retorno

Nova instância de GrupoRegras.

Exemplo

Código Coral

```
de coral.regras importe GrupoRegras
crie a função pronta com contexto
    retorne verdadeiro
fim
crie a função agir com contexto
    mostre contexto
fim
de coral.regras importe Regra
defina r como Regra("aviso", pronta, agir)
defina grupo como GrupoRegras("avisos", (r,))
execute grupo.desabilitar()
mostre r.habilitada
```

Resultado

Saída esperada

```
False
```

Erros

TypeError para argumentos fora da assinatura; objetos sem habilitada podem falhar ao aplicar o grupo.

Atenção

Registro com campos imutáveis que altera as regras mutáveis referenciadas. Não copia regras nem registra o grupo em um motor. Use MotorRegras.criar_grupo para resolver nomes e registrar.

`Debounce(...)` · **módulo** `coral.regras`

O que faz

Adia uma ação até decorrer um intervalo sem nova notificação.

Construtor

Assinatura para consulta

```
Debounce(intervalo: 'float', acao: 'Acao', nome: 'str' = 'debounce', restante: 'float |
  None' = None, pendente: 'bool' = False, contexto_mais_recente: 'Any' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
intervalo	float	sim	sem padrão	Duração finita não negativa, em segundos.
acao	Acao	sim	sem padrão	Função que recebe o contexto da notificação mais recente.
nome	str	não	'debounce'	Nome descritivo; padrão debounce.
restante	float | None	não	None	Tempo inicial restante, ou nulo.
pendente	bool	não	False	Estado inicial de notificação; padrão falso.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
contexto_mais_recente	Any	não	None	Contexto inicial, ou nulo.

Atributos importantes

Atributo	Significado
`pendente`	Se há ação aguardando.
`restante`	Tempo até disparar.
`contexto_mais_recente`	Último contexto recebido.

Métodos principais

Assinatura para consulta

```
notificar(self, contexto: 'Any' = None) -> 'None'
```

Referência para leitura

```
atualizar(self, dt: 'float') -> 'bool'
```

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de Debounce.

Exemplo

Código Coral

```
de coral.regras importe Debounce
crie a função agir com contexto
    mostre contexto
fim
defina d como Debounce(0.5, agir)
execute d.notificar("concluído")
mostre d.atualizar(0.5)
```

Resultado

Saída esperada

```
concluído
True
```

Erros

ValueError para intervalo ou passo inválido; erros da ação são propagados.

Atenção

notificar reinicia o intervalo. atualizar precisa ser chamado pelo programa ou motor; não cria thread nem espera em tempo real. Passos negativos são tratados como zero por compatibilidade. A pendência é limpa antes da ação.

`Temporizador(...)` · módulo `coral.regras`

O que faz

Conta tempo explícito e executa uma ação única ou repetida.

Construtor

Assinatura para consulta

```
Temporizador(intervalo: 'float', acao: 'Acao', repetir: 'bool' = False, nome: 'str' =
  'temporizador', restante: 'float | None' = None, ativo: 'bool' = True) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
intervalo	float	sim	sem padrão	Duração finita não negativa; deve ser positiva quando repetir.
acao	Acao	sim	sem padrão	Função que recebe contexto no disparo.
repetir	bool	não	False	Se repete a cada intervalo; padrão falso.
nome	str	não	'temporizador'	Nome descritivo; padrão temporizador.
restante	float | None	não	None	Tempo até o primeiro disparo; nulo usa intervalo.
ativo	bool	não	True	Estado inicial de execução; padrão verdadeiro.

Atributos importantes

Atributo	Significado
`ativo`	Se aceita novos avanços.
`restante`	Tempo até o próximo disparo.
`repetir`	Política de repetição.

Métodos principais

Assinatura para consulta

```
atualizar(self, contexto: 'Any', dt: 'float') -> 'int'
```

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de Temporizador.

Exemplo

Código Coral

```
de coral.regras importe Temporizador
crie a função agir com contexto
    mostre contexto
fim
defina t como Temporizador(1, agir)
mostre t.atualizar("disparo", 1)
mostre t.ativo
```

Resultado

Saída esperada

```
disparo
1
False
```

Erros

ValueError para duração, restante ou passo inválidos; erros da ação são propagados.

Atenção

atualizar devolve o número de disparos. Sem repetição, desativa após a ação. Com repetição, um passo grande pode executar vários disparos para compensar os intervalos vencidos. Não cria thread; passos negativos equivalem a zero.

`MotorRegras(...)` · **módulo** `coral.regras`

O que faz

Coordena regras, observadores, temporizadores e eventos por avanço explícito.

Construtor

Assinatura para consulta

```
MotorRegras(*, relogio: 'FonteTempo | None' = None, eventos: 'Eventos | None' = None,
  rastreamento: 'RastreamentoReativo | str | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
relogio	FonteTempo | None	não	None	FonteTempo compartilhada, ou nulo para fornecer dt.
eventos	Eventos | None	não	None	Barramento Eventos compartilhado, ou nulo para criar outro.
rastreamento	RastreamentoReativo | str | None	não	None	RastreamentoReativo, nome de modo válido, ou nulo para desligado.

Atributos importantes

Atributo	Significado
`regras`	Lista das regras registradas.

Atributo	Significado
`eventos`	Barramento usado.
`relogio`	Fonte de tempo ou nulo.
`rastreamento`	Registro das decisões.
`grupos`	Grupos nomeados.

Métodos principais

Assinatura para consulta

```
regra(self, nome: 'str', condicao: 'Condicao', acao: 'Acao', *, modo: 'str' =
  'ao_ativar', prioridade: 'str | int' = 'normal', max_execucoes: 'int | None' = None,
  cooldown: 'float' = 0.0) -> 'Regra'
```

Referência para leitura

```
adicionar(self, regra: 'Regra') -> 'Regra'
```

Referência para leitura

```
avaliar(self, contexto: 'Any') -> 'tuple[str, ...]'
```

Referência para leitura

```
atualizar(self, contexto: 'Any', dt: 'float | None' = None) -> 'tuple[str, ...]'
```

Referência para leitura

```
processar_ciclo(self, contexto: 'Any' = None, *, limite_reacoes: 'int' = 1000) ->
  'tuple[Evento, ...]'
```

Referência para leitura

```
criar_grupo(self, nome: 'str', regras: 'list[str] | tuple[str, ...]') -> 'GrupoRegras'
```

Referência para leitura

```
observar_mudanca(self, nome: 'str', leitor, acao) -> 'ObservadorMudanca'
```

Referência para leitura

```
observar_regioes_mapa(self, nome: 'str', alvo: 'Any', mapa, regioes, resolvedor, *,
  identificador_alvo: 'Any' = None)
```

Referência para leitura

```
sequencia(self, nome: 'str', primeiro: 'str', segundo: 'str', acao, *, janela: 'float |
  None' = None) -> 'SequenciaDoisPassos'
```

Referência para leitura

```
debounce(self, segundos: 'float', acao: 'Acao', *, nome: 'str' = 'debounce') ->
  'Debounce'
```

Referência para leitura

```
depois_de(self, segundos: 'float', acao: 'Acao', *, nome: 'str' = 'depois') ->
  'Temporizador'
```

Referência para leitura

```
a_cada(self, segundos: 'float', acao: 'Acao', *, nome: 'str' = 'intervalo') ->
  'Temporizador'
```

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de MotorRegras.

Exemplo

Código Coral

```
de coral.regras importe MotorRegras
crie a função pronta com contexto
    retorne verdadeiro
fim
crie a função agir com contexto
    mostre contexto
fim
defina motor como MotorRegras()
execute motor.regra("aviso", pronta, agir)
mostre motor.avaliar("pronto")
```

Resultado

Saída esperada

```
pronto
('aviso',)
```

Erros

ValueError para nomes repetidos, relógio ausente ou opções inválidas; KeyError para regra ou grupo ausente; ErroCicloReativo ao exceder reações; erros de callbacks são propagados.

Atenção

Não inicia loop. Regras são avaliadas por prioridade decrescente, com desempate por registro. atualizar avança temporizadores e debounces; sem dt exige relógio. processar_ciclo consome eventos enfileirados com orçamento de reações. O motor configura a fonte de instante do barramento compartilhado.

`Eventos(...)` · módulo `coral.regras`

O que faz

Distribui eventos localmente, por fila ou com propagação ao barramento pai.

Construtor

Assinatura para consulta

```
Eventos(*, pai: "'Eventos | None'" = None, relogio: 'FonteTempo | None' = None,
  fonte_instante: 'Callable[[], float] | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
pai	'Eventos | None'	não	None	Barramento Eventos superior, ou nulo.
relogio	FonteTempo | None	não	None	FonteTempo dos instantes, ou nulo.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
fonte_instante	Callable[[], float] | None	não	None	Função sem argumentos que devolve instante, ou nulo; relógio tem precedência.

Atributos importantes

Atributo	Significado
`pendentes`	Quantidade de eventos na fila.
`pai`	Barramento superior.
`possui_fonte_instante`	Se pode registrar tempo.

Métodos principais

Assinatura para consulta

```
quando(self, nome: 'str', funcao: 'Callable[..., Any]', *, prioridade: 'str | int' =
  'normal', receber_evento: 'bool' = False, max_execucoes: 'int | None' = None, cooldown:
  'float' = 0.0)
```

Referência para leitura

```
remover(self, nome: 'str', funcao: 'Callable[..., Any]') -> 'bool'
```

Referência para leitura

```
emitir(self, nome: 'str', *args, **kwargs) -> 'int'
```

Referência para leitura

```
emitir_evento(self, nome: 'str', *args, origem: 'str | None' = None, cancelavel: 'bool'
  = False, metadados: 'Mapping[str, Any] | None' = None, **kwargs) -> 'Evento'
```

Referência para leitura

```
enfileirar(self, nome: 'str', *args, origem: 'str | None' = None, **kwargs) -> 'Evento'
```

Referência para leitura

```
processar(self, limite: 'int | None' = None) -> 'tuple[Evento, ...]'
```

Referência para leitura

```
emitir_propagado(self, nome: 'str', *args, origem: 'str | None' = None, **kwargs) ->
  'int'
```

Referência para leitura

```
definir_pai(self, pai: "'Eventos | None'") -> 'None'
```

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de Eventos.

Exemplo

Código Coral

```
de coral.regras importe Eventos
defina eventos como Eventos()
crie a função receber com valor
    mostre valor
fim
execute eventos.quando("aviso", receber)
mostre eventos.emitir("aviso", "pronto")
```

Resultado

Saída esperada

```
pronto
1
```

Erros

ValueError para prioridade, limite, cooldown, ausência de relógio ou ciclo ao definir pai; erros dos ouvintes são propagados.

Atenção

emitir chama ouvintes imediatamente; enfileirar exige processar. quando usa prioridade e ordem de inscrição; receber_evento permite receber o objeto Evento. Cooldown exige fonte de instante. definir_pai verifica ciclos; forneça uma hierarquia válida ao construtor. Cancelamento sinaliza a operação; consumo interrompe propagação ao pai. O número retornado por emitir conta inscrições selecionadas, inclusive as suprimidas por limites.

`ObservadorMudanca(...)` · **módulo** `coral.regras`

O que faz

Detecta mudanças de valor ou cruzamentos de limite entre avaliações.

Construtor

Assinatura para consulta

```
ObservadorMudanca(nome: 'str', leitor: 'LeitorValor', acao: 'AcaoMudanca', modo: 'str' =
   'mudanca', limite: 'Any' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Identificador convertido para texto.
leitor	LeitorValor	sim	sem padrão	Função que recebe contexto e devolve valor observado.
acao	AcaoMudanca	sim	sem padrão	Função que recebe anterior, novo e contexto.
modo	str	não	'mudanca'	mudanca, passar_de ou cair_abaixo_de; padrão mudanca.
limite	Any	não	None	Valor de comparação para modos de limite; padrão nulo.

Atributos importantes

Atributo	Significado
`inicializado`	Se já ocorreu leitura.
`valor_anterior`	Último valor lido, ou nulo antes da primeira leitura.

Métodos principais

Assinatura para consulta

```
avaliar(self, contexto: 'Any') -> 'bool'
```

Referência para leitura

```
passar_de(nome: 'str', leitor: 'LeitorValor', limite: 'Any', acao: 'AcaoMudanca') ->
  "'ObservadorMudanca'"
```

Referência para leitura

```
cair_abaixo_de(nome: 'str', leitor: 'LeitorValor', limite: 'Any', acao: 'AcaoMudanca')
  -> "'ObservadorMudanca'"
```

Retorno

Nova instância de ObservadorMudanca.

Exemplo

Código Coral

```
de coral.regras importe ObservadorMudanca
crie a função ler_valor com contexto
    retorne contexto
fim
crie a função mudou com anterior, novo, contexto
    mostre novo
fim
defina o como ObservadorMudanca("valor", ler_valor, mudou)
mostre o.avaliar(1)
mostre o.avaliar(2)
```

Resultado

Saída esperada

```
False
2
True
```

Erros

ValueError para modo inválido; erros de leitura, comparação ou ação são propagados.

Atenção

A primeira leitura estabelece referência e não dispara. passar_de exige anterior <= limite < novo; cair_abaixo_de exige anterior >= limite > novo. Valores são mantidos por referência; mutações internas no mesmo objeto podem não ser reconhecidas. Não inicia amostragem automática.

`ObservadorEspacial(...)` · módulo `coral.regras`

O que faz

Detecta transições de uma relação espacial entre alvo e referência.

Construtor

Assinatura para consulta

```
ObservadorEspacial(nome: 'str', leitor: 'LeitorValor', referencia: 'Any', acao:
  'Callable[[Any, Any, Any], Any]', relacao: 'str', transicao: 'str' = 'entrar', limite:
  'Any' = None, obter: 'Callable[[Any], Any] | None' = None, obter_referencia:
  'Callable[[Any], Any] | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome do observador.
leitor	LeitorValor	sim	sem padrão	Função que recebe contexto e devolve alvo espacial ou resolvível.
referencia	Any	sim	sem padrão	Valor espacial ou função de contexto que o devolve.
acao	Callable[[Any, Any, Any], Any]	sim	sem padrão	Função que recebe alvo resolvido, referência resolvida e contexto.
relacao	str	sim	sem padrão	dentro, perto, toca ou intersecta.
transicao	str	não	'entrar'	entrar para falso a verdadeiro; use sair para verdadeiro a falso.
limite	Any	não	None	Distância usada por perto, ou nulo.
obter	Callable[[Any], Any] | None	não	None	Resolvedor do alvo, ou nulo para resolução oficial.
obter_referencia	Callable[[Any], Any] | None	não	None	Resolvedor da referência, ou nulo para resolução oficial.

Atributos importantes

Atributo	Significado
`relacao`	Relação consultada.
`transicao`	Sentido da mudança.

Métodos principais

Assinatura para consulta

```
avaliar(self, contexto: 'Any') -> 'bool'
```

Referência para leitura

```
entrar_em_regiao(nome, leitor, regiao_alvo, acao, *, obter=None, obter_referencia=None)
```

Referência para leitura

```
sair_de_regiao(nome, leitor, regiao_alvo, acao, *, obter=None, obter_referencia=None)
```

Referência para leitura

```
chegar_a_menos_de(nome, leitor, referencia, limite, acao, *, obter=None,
  obter_referencia=None)
```

Referência para leitura

```
tocar(nome, leitor, referencia, acao, *, obter=None, obter_referencia=None)
```

Referência para leitura

```
intersectar(nome, leitor, referencia, acao, *, obter=None, obter_referencia=None)
```

Retorno

Nova instância de ObservadorEspacial.

Exemplo

Código Coral

```
de coral.regras importe ObservadorEspacial
crie a função localizar com contexto
    retorne ponto(0, 0)
fim
crie a função avisar com alvo, referencia, contexto
    mostre "entrada"
fim
defina o como ObservadorEspacial("perto", localizar, ponto(1, 0), avisar, "perto",
  limite=2)
mostre o.avaliar(nulo)
```

Resultado

Saída esperada

```
False
```

Erros

ValueError para relação desconhecida ao avaliar; erros de dimensão, resolução espacial, distância e callbacks são propagados.

Atenção

Primeira avaliação estabelece referência sem disparar. Prefira os construtores nomeados para entrar/sair de região, proximidade, toque ou interseção. Não amostra automaticamente. O construtor direto não valida toda a configuração; qualquer transicao diferente de entrar segue o ramo de saída na avaliação.

`SequenciaDoisPassos(...)` · módulo `coral.regras`

O que faz

Reconhece dois nomes de evento em ordem e chama uma ação.

Construtor

Assinatura para consulta

```
SequenciaDoisPassos(eventos: 'Eventos', primeiro: 'str', segundo: 'str', acao:
  'AcaoSequencia', janela: 'float | None' = None, nome: 'str' = 'sequencia') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
eventos	Eventos	sim	sem padrão	Barramento onde serão inscritos os dois ouvintes.
primeiro	str	sim	sem padrão	Nome do primeiro evento, convertido para texto.
segundo	str	sim	sem padrão	Nome do segundo evento, convertido para texto.
acao	AcaoSequencia	sim	sem padrão	Função que recebe os objetos dos dois eventos.
janela	float | None	não	None	Duração máxima finita não negativa entre eventos, ou nulo sem limite temporal.
nome	str	não	'sequencia'	Nome descritivo; padrão sequencia.

Atributos importantes

Atributo	Significado
`primeiro`	Nome que inicia a sequência.
`segundo`	Nome que a conclui.
`janela`	Limite temporal ou nulo.

Métodos principais

Assinatura para consulta

```
resetar(self) -> 'None'
```

Retorno

Nova instância de SequenciaDoisPassos.

Exemplo

Código Coral

```
de coral.regras importe SequenciaDoisPassos
de coral.regras importe Eventos
defina eventos como Eventos()
crie a função concluir com primeiro, segundo
    mostre segundo.nome
fim
defina s como SequenciaDoisPassos(eventos, "abrir", "fechar", concluir)
execute eventos.emitir("abrir")
execute eventos.emitir("fechar")
```

Resultado

Saída esperada

```
fechar
```

Erros

ValueError para janela inválida ou barramento sem fonte temporal quando a janela foi declarada; erros dos ouvintes e da ação são propagados.

Atenção

O construtor inscreve ouvintes. Uma nova ocorrência do primeiro evento substitui a anterior. O segundo consome a referência, mesmo fora da janela. Janela exige fonte de instante; o limite exato é aceito. resetar limpa o evento pendente e mantém inscrições.

`distribuir_pontos(...)` · módulo `coral.procedural`

O que faz

Distribui pontos bidimensionais determinísticos com margem e restrições.

Assinatura

Assinatura para consulta

```
distribuir_pontos(largura: 'float', altura: 'float', quantidade: 'int', *, semente_raiz:
    'Any' = 0, margem: 'float' = 0.0, distancia_minima: 'float' = 0.0, restricao:
   'Callable[[float, float], bool] | None' = None, max_tentativas_por_ponto: 'int' = 200,
   limite_tentativas_total: 'int' = 1000000) -> 'tuple[tuple[float, float], ...]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
largura	float	sim	sem padrão	Largura finita positiva da área.
altura	float	sim	sem padrão	Altura finita positiva da área.
quantidade	int	sim	sem padrão	Inteiro exato não negativo de pontos.
semente_raiz	Any	não	0	Semente aceita pelo contrato procedural; padrão zero.
margem	float	não	0.0	Margem finita não negativa, menor que metade de ambas as dimensões.
distancia_minima	float	não	0.0	Separação finita não negativa entre pontos.
restricao	Callable[[float, float], bool] | None	não	None	Função de x e y que autoriza posição, ou nulo.
max_tentativas_por_ponto	int	não	200	Inteiro positivo de tentativas para cada ponto; padrão 200.
limite_tentativas_total	int	não	1000000	Inteiro positivo do orçamento preventivo; padrão 1000000.

Retorno

Tupla de pares (x, y) em ponto flutuante.

Exemplo

Código Coral

```
de coral.procedural importe distribuir_pontos
defina pontos como distribuir_pontos(10, 10, 3, semente_raiz=7)
mostre len(pontos)
```

Resultado

Saída esperada

```
3
```

Erros

ValueError para dimensões, margem, distância, inteiros, orçamento ou distribuição inviável; TypeError para restrição não chamável; erros do callback são propagados.

Atenção

O pior caso quantidade vezes max_tentativas_por_ponto deve caber no orçamento antes da geração. Restrições podem tornar a quantidade impossível dentro das tentativas; a função falha em vez de devolver quantidade parcial. Use callback determinístico para reprodutibilidade.

`matriz_campo(...)` · **módulo** `coral.procedural`

O que faz

Amostra um campo em uma grade bidimensional limitada.

Assinatura

Assinatura para consulta

```
matriz_campo(campo: 'CampoProcedural', colunas: 'int', linhas: 'int', *, origem:
  'Sequence[float]' = (0.0, 0.0), passo: 'Sequence[float]' = (1.0, 1.0), limite_celulas:
  'int' = 1000000) -> 'tuple[tuple[float, ...], ...]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
campo	CampoProcedural	sim	sem padrão	Instância CampoProcedural capaz de receber duas coordenadas.
colunas	int	sim	sem padrão	Inteiro exato positivo da largura da matriz.
linhas	int	sim	sem padrão	Inteiro exato positivo da altura da matriz.
origem	Sequence[float]	não	(0.0, 0.0)	Par finito x e y da primeira amostra; padrão (0, 0).
passo	Sequence[float]	não	(1.0, 1.0)	Par finito não nulo entre amostras; padrão (1, 1).
limite_celulas	int	não	1000000	Inteiro positivo do máximo de linhas vezes colunas; padrão 1000000.

Retorno

Tupla de linhas, cada linha uma tupla de valores numéricos.

Exemplo

Código Coral

```
de coral.procedural importe matriz_campo
de coral.procedural importe campo_constante
defina campo como campo_constante(2)
defina matriz como matriz_campo(campo, 2, 2)
mostre matriz[0][0]
```

Resultado

Saída esperada

```
2.0
```

Erros

TypeError para campo incompatível; ValueError para dimensões, orçamento, origem ou passo inválidos; erros do campo são propagados.

Atenção

A indexação é matriz[linha][coluna]. Origem e passo transformam as coordenadas, sem alterar o campo. Passos negativos são aceitos. O orçamento é verificado antes de consultar o campo.

`anotar_pontos(...)` · módulo `coral.procedural`

O que faz

Acrescenta valores de campos procedurais a uma coleção de pontos 2D.

Assinatura

Assinatura para consulta

```
anotar_pontos(pontos: 'Iterable[Sequence[float]]', campos: 'Mapping[str,
  CampoProcedural]', *, limite_pontos: 'int' = 100000) -> 'tuple[dict[str, Any], ...]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
pontos	Iterable[Sequence[float]]	sim	sem padrão	Iterável de pares finitos x e y.
campos	Mapping[str, CampoProcedural]	sim	sem padrão	Mapeamento não vazio de nomes para CampoProcedural.
limite_pontos	int	não	100000	Inteiro positivo do máximo de pontos processados; padrão 100000.

Retorno

Tupla de dicionários com indice, x, y e os campos amostrados.

Exemplo

Código Coral

```
de coral.procedural importe anotar_pontos
de coral.procedural importe campo_constante
defina campo como campo_constante(2)
defina registros como anotar_pontos([[1, 2]], {"altura": campo})
mostre registros[0]["altura"]
```

Resultado

Saída esperada

```
2.0
```

Erros

ValueError para limite, coleção vazia, nomes reservados, pontos ou coordenadas inválidos; TypeError para campos incompatíveis; erros do iterável e dos campos são propagados.

Atenção

Nomes indice, x e y são reservados. O índice começa em zero. O limite é verificado durante o consumo do iterável; se ele for excedido, avaliações anteriores já ocorreram, mas não há retorno parcial.

`receita_procedural(...)` · módulo `coral.procedural`

O que faz

Cria uma descrição portátil de algoritmo procedural sem executá-lo.

Assinatura

Assinatura para consulta

```
receita_procedural(algoritmo: 'str', *, semente_raiz: 'Any' = 0, **parametros: 'Any') ->
   'ReceitaProcedural'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
algoritmo	str	sim	sem padrão	campo_constante, campo_ruido ou distribuir_pontos.
semente_raiz	Any	não	0	Semente raiz aceita pelo contrato; padrão zero.
parametros	Any	não	sem padrão	Argumentos nomeados específicos do algoritmo; por exemplo valor para campo_constante.

Retorno

ReceitaProcedural validada na versão de algoritmo um.

Exemplo

Código Coral

```
de coral.procedural importe receita_procedural
defina receita como receita_procedural("campo_constante", valor=2)
mostre receita.algoritmo
```

Resultado

Saída esperada

```
campo_constante
```

Erros

ValueError para algoritmo, parâmetros ou semente inválidos; TypeError para argumentos fora da assinatura.

Atenção

Valida nomes, parâmetros e semente, sem reconstruir o produto. Não aceita callbacks arbitrários como receitas. Use reconstruir_receita com orçamento explícito para produzir o campo ou os pontos.

`diagnosticar_custo_receita(...)` · módulo `coral.procedural`

O que faz

Estima o trabalho máximo conhecido de uma receita antes da reconstrução.

Assinatura

Assinatura para consulta

```
diagnosticar_custo_receita(receita: 'ReceitaProcedural | Mapping[str, Any]') ->
  'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
receita	ReceitaProcedural | Mapping[str, Any]	sim	sem padrão	ReceitaProcedural ou mapeamento no contrato portátil da receita.

Retorno

Dicionário com contrato, algoritmo, versao_algoritmo, custo_maximo_estimado e unidade.

Exemplo

Código Coral

```
de coral.procedural importe diagnosticar_custo_receita
de coral.procedural importe receita_procedural
defina receita como receita_procedural("campo_constante", valor=2)
mostre diagnosticar_custo_receita(receita)["custo_maximo_estimado"]
```

Resultado

Saída esperada

```
1
```

Erros

ValueError para dados, versão ou parâmetros inválidos ao reconstruir a descrição; erros de contrato da receita são propagados.

Atenção

Para distribuir_pontos, estima quantidade vezes max_tentativas_por_ponto em tentativas. Para construção de campos, estima uma construção; não inclui custo de amostragem futura nem tempo de execução.

`reconstruir_receita(...)` · módulo `coral.procedural`

O que faz

Produz o campo ou os pontos descritos por uma receita validada.

Assinatura

Assinatura para consulta

```
reconstruir_receita(receita: 'ReceitaProcedural | Mapping[str, Any]', *, limite_custo:
  'int' = 1000000) -> 'Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
receita	ReceitaProcedural | Mapping[str, Any]	sim	sem padrão	ReceitaProcedural ou mapeamento portátil correspondente.
limite_custo	int	não	1000000	Inteiro exato positivo do orçamento máximo; padrão 1000000.

Retorno

CampoProcedural para campo_constante ou campo_ruido; tupla de pontos para distribuir_pontos.

Exemplo

Código Coral

```
de coral.procedural importe reconstruir_receita
de coral.procedural importe receita_procedural
defina receita como receita_procedural("campo_constante", valor=2)
defina campo como reconstruir_receita(receita)
mostre campo.valor(1, 2)
```

Resultado

Saída esperada

```
2.0
```

Erros

ValueError para receita, limite ou custo excedido; erros do algoritmo de origem são propagados.

Atenção

Verifica versão e custo estimado antes de executar o algoritmo. O limite controla construção ou tentativas, não a amostragem futura de um campo. Para aceitar custo maior, aumente explicitamente limite_custo.

`proveniencia_procedural(...)` · módulo `coral.procedural`

O que faz

Descreve a origem de um objeto procedural conhecido.

Assinatura

Assinatura para consulta

```
proveniencia_procedural(objeto: 'Any') -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	ReceitaProcedural, Semente, GeradorProcedural ou CampoProcedural.

Retorno

Dicionário no contrato coral.procedural.proveniencia/1, específico do tipo.

Exemplo

Código Coral

```
de coral.procedural importe proveniencia_procedural
de coral.procedural importe receita_procedural
defina receita como receita_procedural("campo_constante", valor=2)
mostre proveniencia_procedural(receita)["reconstruivel"]
```

Resultado

Saída esperada

```
True
```

Erros

TypeError para tipo desconhecido; erros ao copiar metadados não portáteis podem ser propagados.

Atenção

Receitas, sementes e geradores possuem descrição reconstruível. Campos são marcados reconstruivel=falso, pois seus callbacks não são serializados; seus metadados são copiados profundamente. Proveniência descreve origem e não produz amostras.

`Personagem(...)` · módulo `coral.rpg`

O que faz

Cria um agente de RPG com vida limitada e atributos extensíveis.

Construtor

Assinatura para consulta

```
Personagem(nome: 'str', vida: 'int' = 10, *, vida_max: 'int | None' = None, **atributos:
    'Any')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome convertido para texto.
vida	int	não	10	Valor inicial convertido para inteiro; padrão dez.
vida_max	int | None	não	None	Máximo convertido para inteiro e no mínimo um; nulo usa vida inicial.
atributos	Any	não	sem padrão	Atributos adicionais fornecidos como argumentos nomeados.

Atributos importantes

Atributo	Significado
`vida`	Valor do recurso vida.
`vida_max`	Máximo do recurso vida.
`vivo`	Se a vida é positiva.
`mundo`	Mundo associado, ou nulo.

Métodos principais

Assinatura para consulta

```
receber_dano(self, quantidade: 'int') -> 'int | float'
```

Referência para leitura

```
curar(self, quantidade: 'int') -> 'int | float'
```

Referência para leitura

```
teste(self, atributo: 'str', dificuldade: 'int' = 10, *, semente: 'int | None' = None)
  -> 'tuple[Rolagem, bool]'
```

Referência para leitura

```
definir_atributo(self, nome: 'str', base: 'float' = 0.0) -> 'Atributo'
```

Referência para leitura

```
valor_atributo(self, nome: 'str', padrao: 'Any' = None) -> 'Any'
```

Retorno

Nova instância de Personagem.

Exemplo

Código Coral

```
de coral.rpg importe Personagem
defina personagem como Personagem("A", 10)
mostre personagem.receber_dano(3)
mostre personagem.vida
```

Resultado

Saída esperada

```
3
7
```

Erros

ValueError ou TypeError para valores não convertíveis ou contratos de recursos inválidos; erros de eventos e callbacks são propagados.

Atenção

Não se registra automaticamente em Mundo. A vida inicial fica entre zero e vida_max. receber_dano e curar devolvem a quantidade efetivamente aplicada, respeitando limites; dano pode ser cancelado por evento quando houver mundo. teste faz rolagem d20 com bônus do atributo e dificuldade.

`CombateTurnos(...)` · módulo `coral.rpg`

O que faz

Organiza iniciativa, turnos, ataques e encerramento de um combate.

Construtor

Assinatura para consulta

```
CombateTurnos(participantes: 'Iterable[Personagem]' = (), *, nome: 'str | None' = None,
  encerramento: 'str' = 'ultimo_vivo')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
participantes	Iterable[Personagem]	não	()	Iterável de Personagem; padrão vazio.
nome	str | None	não	None	Nome convertido para texto, ou nulo sem registro automático.
encerramento	str	não	'ultimo_vivo'	ultimo_vivo, sem_vivos ou manual; padrão ultimo_vivo.

Atributos importantes

Atributo	Significado
`participantes`	Lista dos participantes.
`ordem`	Lista da ordem de turnos.
`atual`	Personagem do turno ou nulo.

Atributo	Significado
`rodada`	Rodada atual.
`pausado`	Estado de pausa.
`terminou`	Condição da política de encerramento.

Métodos principais

Assinatura para consulta

```
adicionar(self, personagem: 'Personagem') -> 'Personagem'
```

Referência para leitura

```
iniciar(self, *, iniciativas: 'dict[str, int] | None' = None, semente: 'int | None' =
  None) -> 'tuple[Personagem, ...]'
```

Referência para leitura

```
proximo(self) -> 'Personagem | None'
```

Referência para leitura

```
atacar(self, atacante: 'Personagem', alvo: 'Personagem', dano: 'int', *, defesa: 'int |
  None' = None, chance_critico: 'float' = 0.0, multiplicador_critico: 'float' = 2.0,
  exigir_turno: 'bool' = True) -> 'ResultadoAtaque'
```

Referência para leitura

```
pausar(self) -> 'None'
```

Referência para leitura

```
retomar(self) -> 'None'
```

Referência para leitura

```
encerrar(self) -> 'None'
```

Retorno

Nova instância de CombateTurnos.

Exemplo

Código Coral

```
de coral.rpg importe CombateTurnos
de coral.rpg importe Personagem
defina a como Personagem("A")
defina b como Personagem("B")
defina combate como CombateTurnos([a, b])
execute combate.iniciar(iniciativas={"A": 2, "B": 1}, semente=7)
mostre combate.atual.nome
```

Resultado

Saída esperada

```
A
```

Erros

TypeError para participante incompatível; ValueError para política, registro ou ataque inválidos; RuntimeError ao atacar antes de iniciar, durante pausa, fora do turno ou com personagem derrotado; erros dos eventos são propagados.

Atenção

O construtor não inicia combate. Quando todos pertencem ao mesmo Mundo e há nome, registra o combate nesse mundo. iniciar ordena iniciativas decrescentes, com desempate pelo nome, e remove derrotados da ordem. atacar exige combate iniciado, ativo, participantes vivos e turno correto por padrão. proximo avança a fila; pausar mantém o estado.

`ResultadoAtaque(...)` · módulo `coral.rpg`

O que faz

Registra os valores calculados e aplicados por um ataque.

Construtor

Assinatura para consulta

```
ResultadoAtaque(atacante: "'Personagem'", alvo: "'Personagem'", dano_base: 'int',
  defesa: 'int', critico: 'bool', multiplicador_critico: 'float', dano_calculado: 'int',
  dano_aplicado: 'int | float', cancelado: 'bool' = False) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
atacante	'Personagem'	sim	sem padrão	Personagem que atacou.
alvo	'Personagem'	sim	sem padrão	Personagem atingido.
dano_base	int	sim	sem padrão	Dano inteiro antes de defesa e crítico.
defesa	int	sim	sem padrão	Defesa usada no cálculo.
critico	bool	sim	sem padrão	Se ocorreu crítico.
multiplicador_critico	float	sim	sem padrão	Multiplicador usado no crítico.
dano_calculado	int	sim	sem padrão	Dano calculado após as políticas.
dano_aplicado	int | float	sim	sem padrão	Dano efetivamente retirado da vida.
cancelado	bool	não	False	Se o ataque foi cancelado; padrão falso.

Atributos importantes

Atributo	Significado
`dano_calculado`	Dano proposto pelo cálculo.
`dano_aplicado`	Dano efetivo.

Atributo	Significado
`cancelado`	Estado de cancelamento.

Retorno

Nova instância de ResultadoAtaque.

Exemplo

Código Coral

```
de coral.rpg importe ResultadoAtaque
de coral.rpg importe Personagem
defina a como Personagem("A")
defina b como Personagem("B")
de coral.rpg importe CombateTurnos
defina combate como CombateTurnos([a, b])
execute combate.iniciar(iniciativas={"A": 2, "B": 1}, semente=7)
defina resultado como combate.atacar(a, b, 3)
mostre isinstance(resultado, ResultadoAtaque)
mostre resultado.dano_aplicado
```

Resultado

Saída esperada

```
True
3
```

Erros

TypeError para argumentos fora da assinatura; o construtor manual não valida os contratos de ataque.

Atenção

Registro imutável nos campos, com referências a personagens mutáveis. Prefira o resultado devolvido por CombateTurnos.atacar. Construir um registro manualmente não aplica dano nem valida integralmente os campos; dano_aplicado pode ser menor que dano_calculado por limite de vida ou cancelamento.

`EstadoRPG(...)` · módulo `coral.rpg`

O que faz

Envolve um Mundo no contrato persistente do domínio RPG.

Construtor

Assinatura para consulta

```
EstadoRPG(mundo: 'Mundo') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mundo	Mundo	sim	sem padrão	Instância Mundo que será preservada pelo agregado.

Atributos importantes

Atributo	Significado
`mundo`	Mundo envolvido pelo agregado.

Retorno

Nova instância de EstadoRPG.

Exemplo

Código Coral

```
de coral.rpg importe EstadoRPG
de coral.mundo importe criar_mundo
defina mundo como criar_mundo("Estudo")
defina estado como EstadoRPG(mundo)
mostre estado.mundo.nome
```

Resultado

Saída esperada

```
Estudo
```

Erros

TypeError quando mundo não é uma instância Mundo.

Atenção

Mantém a referência ao mundo, sem clone nem arquivo automático. O domínio RPG registra adaptadores e extensão de combates na persistência. Para escrever e restaurar o agregado, use salvar_estado_rpg e carregar_estado_rpg.

`criar_personagem(...)` · módulo `coral.rpg`

O que faz

Cria um Personagem e o registra imediatamente em um Mundo.

Assinatura

Assinatura para consulta

```
criar_personagem(mundo: 'Mundo', nome: 'str', vida: 'int' = 10, **atributos: 'Any') ->
  'Personagem'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mundo	Mundo	sim	sem padrão	Mundo que recebe o personagem.
nome	str	sim	sem padrão	Nome do personagem.
vida	int	não	10	Vida inicial; padrão dez.
atributos	Any	não	sem padrão	Atributos adicionais como argumentos nomeados.

Retorno

Personagem criado e associado ao mundo.

Exemplo

Código Coral

```
de coral.rpg importe criar_personagem
de coral.mundo importe criar_mundo
defina mundo como criar_mundo("Estudo")
defina p como criar_personagem(mundo, "A", 10, defesa=2)
mostre mundo.exigir("A").nome
```

Resultado

Saída esperada

```
A
```

Erros

ValueError para conflitos de identidade; erros do construtor Personagem e do registro no mundo são propagados.

Atenção

Usa Personagem e Mundo.adicionar. O nome precisa respeitar a unicidade das entidades. Para criar sem registro, use Personagem diretamente. Atributos são dados do agente, não classes ou comportamentos novos.

`estado_rpg(...)` · módulo `coral.rpg`

O que faz

Cria um agregado de persistência RPG para um Mundo.

Assinatura

Assinatura para consulta

```
estado_rpg(mundo: 'Mundo') -> 'EstadoRPG'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mundo	Mundo	sim	sem padrão	Instância Mundo a envolver.

Retorno

EstadoRPG referenciando o mesmo mundo.

Exemplo

Código Coral

```
de coral.rpg importe estado_rpg
de coral.mundo importe criar_mundo
defina mundo como criar_mundo("Estudo")
defina estado como estado_rpg(mundo)
mostre estado.mundo.nome
```

Resultado

Saída esperada

```
Estudo
```

Erros

TypeError para argumento que não seja Mundo.

Atenção

Não copia estado nem escreve arquivos; equivale a EstadoRPG(mundo).

`registrar_combate(...)` · módulo `coral.rpg`

O que faz

Registra um combate no catálogo RPG associado a um Mundo.

Assinatura

Assinatura para consulta

```
registrar_combate(mundo: 'Mundo', nome: 'str', combate: "'CombateTurnos'") ->
  "'CombateTurnos'"
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mundo	Mundo	sim	sem padrão	Mundo que possui o registro.
nome	str	sim	sem padrão	Nome convertido para texto e não vazio.
combate	'CombateTurnos'	sim	sem padrão	Instância CombateTurnos.

Retorno

A mesma instância CombateTurnos recebida.

Exemplo

Código Coral

```
de coral.rpg importe registrar_combate
de coral.mundo importe criar_mundo
defina mundo como criar_mundo("Estudo")
de coral.rpg importe CombateTurnos
defina combate como CombateTurnos()
defina registrado como registrar_combate(mundo, "ensaio", combate)
mostre registrado.nome
```

Resultado

Saída esperada

```
None
```

Erros

TypeError para mundo ou combate incompatível; ValueError para nome vazio ou conflito de registro.

Atenção

Repetir o mesmo nome e objeto é permitido; usar esse nome para outro combate falha. O registro não inicia o combate, não altera combate.nome e não exige que os participantes pertençam ao mundo. Para persistir combates contextuais, mantenha nomes e participantes associados de modo coerente.

`buscar_combate(...)` · módulo `coral.rpg`

O que faz

Consulta um combate nomeado no registro RPG de um Mundo.

Assinatura

Assinatura para consulta

```
buscar_combate(mundo: 'Mundo', nome: 'str') -> "'CombateTurnos | None'"
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mundo	Mundo	sim	sem padrão	Instância Mundo consultada.
nome	str	sim	sem padrão	Nome convertido para texto.

Retorno

CombateTurnos registrado ou nulo quando ausente.

Exemplo

Código Coral

```
de coral.rpg importe buscar_combate
de coral.mundo importe criar_mundo
defina mundo como criar_mundo("Estudo")
mostre buscar_combate(mundo, "ausente")
```

Resultado

Saída esperada

```
None
```

Erros

TypeError para argumento que não seja Mundo.

Atenção

Não cria registro, não inicia combate e não exige que o nome exista.

`combates_do_mundo(...)` · módulo `coral.rpg`

O que faz

Obtém os combates registrados para um Mundo.

Assinatura

Assinatura para consulta

```
combates_do_mundo(mundo: 'Mundo') -> "tuple['CombateTurnos', ...]"
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mundo	Mundo	sim	sem padrão	Instância Mundo consultada.

Retorno

Tupla das instâncias CombateTurnos registradas, em ordem de inserção.

Exemplo

Código Coral

```
de coral.rpg importe combates_do_mundo
de coral.mundo importe criar_mundo
defina mundo como criar_mundo("Estudo")
mostre len(combates_do_mundo(mundo))
```

Resultado

Saída esperada

```
0
```

Erros

TypeError para argumento que não seja Mundo.

Atenção

A tupla é uma fotografia das referências; os combates continuam mutáveis. Um mundo sem registro devolve tupla vazia.

`salvar_estado_rpg(...)` · módulo `coral.rpg`

O que faz

Grava um Mundo como agregado RPG no formato de persistência da Coral.

Assinatura

Assinatura para consulta

```
salvar_estado_rpg(mundo: 'Mundo', caminho: 'str | Path') -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mundo	Mundo	sim	sem padrão	Mundo a envolver em EstadoRPG.
caminho	str | Path	sim	sem padrão	Texto ou Path do arquivo de saída.

Retorno

Path do arquivo gravado.

Exemplo

Código Coral

```
de coral.rpg importe salvar_estado_rpg
de coral.mundo importe criar_mundo
defina mundo como criar_mundo("Estudo")
defina destino como salvar_estado_rpg(mundo, "estado.json")
mostre destino.name
```

Resultado

Saída esperada

```
estado.json
```

Erros

TypeError para mundo incompatível; ErroPersistencia para dados, adaptadores ou extensões inválidos; erros de sistema de arquivos são propagados.

Atenção

Grava dados, não código executável. O mundo e extensões precisam respeitar contratos de persistência. A gravação substitui o conteúdo no caminho informado. Use um caminho de teste para não sobrescrever arquivos importantes.

`carregar_estado_rpg(...)` · módulo `coral.rpg`

O que faz

Lê um agregado EstadoRPG e devolve o Mundo restaurado.

Assinatura

Assinatura para consulta

```
carregar_estado_rpg(caminho: 'str | Path') -> 'Mundo'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	str | Path	sim	sem padrão	Texto ou Path do arquivo persistido.

Retorno

Mundo reconstruído a partir do EstadoRPG.

Exemplo

Código Coral

```
de coral.rpg importe carregar_estado_rpg
de coral.mundo importe criar_mundo
defina mundo como criar_mundo("Estudo")
de coral.rpg importe salvar_estado_rpg
execute salvar_estado_rpg(mundo, "estado.json")
defina restaurado como carregar_estado_rpg("estado.json")
mostre restaurado.nome
```

Resultado

Saída esperada

```
Estudo
```

Erros

ErroPersistencia para arquivo ausente ou ilegível, JSON, contrato, versão, tipo ou referências inválidas.

Atenção

Exige o tipo persistido EstadoRPG; um arquivo de Mundo avulso não substitui esse agregado. Reconstrói dados e extensões registradas, sem executar código arbitrário nem reiniciar simulações ou loops automaticamente.

`traduzir_arquivo(...)` · módulo `coral`

O que faz

Lê um arquivo Coral em UTF 8 e traduz seu conteúdo para código Python.

Assinatura

Assinatura para consulta

```
traduzir_arquivo(caminho)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	valor dinâmico	sim	sem padrão	Texto ou objeto aceito por Path que identifica o arquivo de entrada.

Retorno

Texto Python produzido pela tradução.

Exemplo

Código Coral

```
de coral importe traduzir_arquivo
de pathlib importe Path
defina fonte como Path("exemplo.coral")
execute fonte.write_text("mostre 7", encoding="utf-8")
defina codigo como traduzir_arquivo(fonte)
mostre isinstance(codigo, str)
```

Resultado

Saída esperada

```
True
```

Erros

OSError para arquivo ausente ou ilegível; UnicodeError para entrada fora de UTF 8; ErroDeTraducao ou outros erros de análise para código inválido.

Atenção

Não executa o programa nem escreve um arquivo Python. O nome do caminho aparece nos diagnósticos. Para compilar com a API estrutural e obter metadados, prefira coral.api.compilar.

`SnapshotAnalise(...)` · módulo `coral.api`

O que faz

Mantém uma fotografia reutilizável da análise de uma fonte Coral.

Construtor

Assinatura para consulta

```
SnapshotAnalise(fonte: 'str', nome_fonte: 'str', hash_fonte: 'str', ast: 'Programa |
  None', simbolos: 'TabelaSimbolos | None', interface_modulo: 'InterfaceModuloAnalise',
  tipos_conhecidos: 'tuple[tuple[str, str], ...]', diagnosticos:
  'tuple[DiagnosticoEstruturado, ...]', versao: 'int | str' = 1, versao_snapshot: 'str' =
  '1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
fonte	str	sim	sem padrão	Texto completo da fonte.
nome_fonte	str	sim	sem padrão	Nome usado nos diagnósticos.
hash_fonte	str	sim	sem padrão	Hash SHA256 do texto UTF 8.
ast	Programa | None	sim	sem padrão	Programa estrutural ou nulo quando a análise falhou.
simbolos	TabelaSimbolos | None	sim	sem padrão	TabelaSimbolos ou nulo quando indisponível.
interface_modulo	InterfaceModuloAnalise	sim	sem padrão	InterfaceModuloAnalise com símbolos públicos encontrados.
tipos_conhecidos	tuple[tuple[str, str], ...]	sim	sem padrão	Tupla de pares de nome e tipo conhecido.
diagnosticos	tuple[DiagnosticoEstruturado, ...]	sim	sem padrão	Tupla de DiagnosticoEstruturado.
versao	int | str	não	1	Versão lógica da edição; inteiro ou texto, padrão um.
versao_snapshot	str	não	'1'	Versão do contrato de snapshot; padrão texto 1.

Atributos importantes

Atributo	Significado
`id_snapshot`	Identificador composto de hash e versão.
`possui_erros`	Se há diagnóstico de nível erro.

Atributo	Significado
`tipos`	Novo dicionário dos tipos conhecidos.
`ast`	Árvore estrutural ou nulo.
`diagnosticos`	Diagnósticos da análise.

Retorno

Nova instância de SnapshotAnalise.

Exemplo

Código Coral

```
de coral.api importe SnapshotAnalise
de coral.api importe analisar_snapshot
defina foto como analisar_snapshot("mostre 7", versao=2)
mostre isinstance(foto, SnapshotAnalise)
mostre foto.possui_erros
```

Resultado

Saída esperada

```
True
False
```

Erros

TypeError para argumentos fora da assinatura; o construtor manual não valida integralmente a coerência dos campos.

Atenção

Prefira analisar_snapshot para construir uma fotografia coerente. O construtor manual não recalcula hash nem analisa fonte. Os campos são imutáveis, mas ast e simbolos podem referenciar estruturas internas mutáveis. tipos devolve um dicionário novo; id_snapshot combina hash e versão lógica. Use compilar_de_snapshot para reutilizar a análise.

`explorar(...)` · módulo `coral.graficos`

O que faz

Gera um HTML interativo local e solicita sua abertura em navegador.

Assinatura

Assinatura para consulta

```
explorar(grafico: 'Any', *, limite_pontos: 'int' = 200000) -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grafico	Any	sim	sem padrão	Objeto gráfico suportado pela exportação interativa da Coral.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
limite_pontos	int	não	200000	Inteiro do orçamento de pontos exportados; padrão 200000.

Retorno

Path do HTML gerado, quando o navegador aceita a abertura.

Exemplo

Código Coral

```
de coral.graficos importe explorar
de coral.graficos importe barras, ErroGrafico
de os importe environ
execute environ.pop("DISPLAY", nulo)
execute environ.pop("WAYLAND_DISPLAY", nulo)
defina grafico como barras(["A"], [1])
tente
    execute explorar(grafico)
se der erro do tipo ErroGrafico
    mostre "Use salvar_html em ambiente sem interface"
fim
```

Resultado

Saída esperada

```
Use salvar_html em ambiente sem interface
```

Erros

ErroGrafico quando não há sessão gráfica, o navegador falha ou recusa abertura; erros de exportação, dependência Plotly e escrita são propagados.

Atenção

Abertura é explícita e exige sessão gráfica. Este exemplo demonstra o fallback sem interface em Linux; remove as variáveis de sessão apenas no processo do exemplo. O arquivo fica na pasta temporária coral/graficos_interativos. Sem interface, não inicia navegador nem servidor. Prefira salvar_html para escolher o destino ou trabalhar em ambiente sem interface.

`salvar(...)` · módulo `coral.persistencia`

O que faz

Serializa dados ou objetos registrados e grava um documento JSON de persistência.

Assinatura

Assinatura para consulta

```
salvar(objeto: 'Any', caminho: 'str | Path') -> 'Path'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Valor ou objeto aceito pelo contrato de persistência.
caminho	str | Path	sim	sem padrão	Texto ou Path do arquivo de saída, com expansão de usuário.

Retorno

Path normalizado do arquivo gravado.

Exemplo

Código Coral

```
de coral.persistencia importe salvar
defina destino como salvar({"energia": 10}, "estado.json")
mostre destino.name
```

Resultado

Saída esperada

```
estado.json
```

Erros

ErroPersistencia para objeto, adaptador, JSON ou dados incompatíveis; erros de caminho, criação de diretório e escrita são propagados.

Atenção

Cria diretórios pais e substitui o conteúdo existente. O documento inclui contrato e versão de esquema. Não serializa código arbitrário. A gravação usa write_text diretamente e não oferece substituição atômica; um erro de escrita pode deixar arquivo incompleto.

`carregar(...)` · módulo `coral.persistencia`

O que faz

Lê um documento de persistência e reconstrói seu valor ou objeto.

Assinatura

Assinatura para consulta

```
carregar(caminho: 'str | Path') -> 'Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	str | Path	sim	sem padrão	Texto ou Path do arquivo JSON, com expansão de usuário.

Retorno

Valor ou objeto reconstruído conforme o contrato do documento.

Exemplo

Código Coral

```
de coral.persistencia importe carregar
de coral.persistencia importe salvar
execute salvar({"energia": 10}, "estado.json")
mostre carregar("estado.json")["energia"]
```

Resultado

Saída esperada

```
10
```

Erros

ErroPersistencia para arquivo ausente ou ilegível, UTF 8, JSON, contrato, versão, tipo ou payload inválidos; erros de adaptadores podem ser propagados.

Atenção

Valida contrato e versões aceitas antes de reconstruir. Tipos e extensões oficiais podem carregar seus módulos registrados. Adaptadores personalizados devem estar registrados no processo. O formato contém dados, não código do programa.

`carregar_como(...)` · módulo `coral.persistencia`

O que faz

Carrega um documento e exige que o objeto reconstruído tenha o tipo esperado.

Assinatura

Assinatura para consulta

```
carregar_como(caminho: 'str | Path', tipo_esperado: 'type', nome_tipo: 'str | None' =
  None) -> 'Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	str | Path	sim	sem padrão	Texto ou Path do documento de persistência.
tipo_esperado	type	sim	sem padrão	Classe usada em isinstance após a reconstrução.
nome_tipo	str | None	não	None	Nome descritivo para a mensagem de erro, ou nulo para o nome da classe.

Retorno

Objeto reconstruído e compatível com tipo_esperado.

Exemplo

Código Coral

```
de coral.persistencia importe carregar_como
de coral.persistencia importe salvar
execute salvar({"energia": 10}, "estado.json")
defina dados como carregar_como("estado.json", dict)
mostre dados["energia"]
```

Resultado

Saída esperada

```
10
```

Erros

ErroPersistencia para tipo incompatível e para falhas de carregar; TypeError quando tipo_esperado não é aceito por isinstance.

Atenção

Usa isinstance, portanto aceita subclasses. Não converte um objeto de outro tipo. A reconstrução ocorre antes da validação final; callbacks de adaptadores já podem ter sido executados.

`registrar_adaptador(...)` · módulo `coral.persistencia`

O que faz

Registra conversores de um tipo personalizado para o contrato de persistência.

Assinatura

Assinatura para consulta

```
registrar_adaptador(tipo_python: 'type', identificador: 'str', para_dados:
  'Callable[[Any], dict[str, Any]]', de_dados: 'Callable[[dict[str, Any]], Any]') ->
  'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tipo_python	type	sim	sem padrão	Classe representada pelo adaptador.
identificador	str	sim	sem padrão	Texto não vazio contendo barra, por exemplo estudo/Caixa.
para_dados	Callable[[Any], dict[str, Any]]	sim	sem padrão	Função que recebe o objeto e devolve dicionário de dados serializáveis.
de_dados	Callable[[dict[str, Any]], Any]	sim	sem padrão	Função que recebe dados reconstruídos e devolve o objeto.

Retorno

nulo; altera o registro de adaptadores do processo.

Exemplo

Código Coral

```
de coral.persistencia importe registrar_adaptador
crie a classe Caixa
    ao criar uma Caixa com valor
        defina seu valor como valor
    fim
fim
crie a função exportar com objeto
    retorne {"valor": objeto.valor}
fim
crie a função importar_dados com dados
    retorne Caixa(dados["valor"])
fim
de coral.persistencia importe salvar, carregar
execute registrar_adaptador(Caixa, "estudo/Caixa", exportar, importar_dados)
execute salvar(Caixa(7), "caixa.json")
mostre carregar("caixa.json").valor
```

Resultado

Saída esperada

```
7
```

Erros

ValueError para identificador inválido ou já associado a outro tipo; TypeError para tipo não utilizável como chave; erros dos callbacks surgem ao serializar ou reconstruir.

Atenção

O identificador precisa ser estável para ler documentos depois. Registrá-lo para outro tipo falha; para o mesmo tipo, substitui o adaptador desse identificador. Não há validação integral de tipo e callbacks no registro: o programa deve fornecer contratos coerentes. Para leitura em outro processo, registre o adaptador novamente.

`registrar_extensao_persistente(...)` · módulo `coral.persistencia`

O que faz

Registra dados de domínio adicionais para um tipo hospedeiro.

Assinatura

Assinatura para consulta

```
registrar_extensao_persistente(tipo_python: 'type', identificador: 'str', para_dados:
  'Callable[[Any], Any]', aplicar_dados: 'Callable[[Any, Any], None]') -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tipo_python	type	sim	sem padrão	Classe hospedeira à qual a extensão se aplica, incluindo subclasses.
identificador	str	sim	sem padrão	Identificador estável não vazio contendo barra.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
para_dados	Callable[[Any], Any]	sim	sem padrão	Função que recebe o hospedeiro e devolve payload ou nulo para omitir.
aplicar_dados	Callable[[Any, Any], None]	sim	sem padrão	Função que recebe hospedeiro e payload para restaurar os dados.

Retorno

nulo; altera o registro de extensões do processo.

Exemplo

Código Coral

```
de coral.persistencia importe registrar_extensao_persistente
crie a classe Caixa
    ao criar uma Caixa com valor
        defina seu valor como valor
    fim
fim
crie a função exportar_extra com objeto
    retorne {"valor": objeto.valor}
fim
crie a função aplicar_extra com objeto, dados
    execute setattr(objeto, "valor", dados["valor"])
fim
de coral.persistencia importe extensoes_para_dados
execute registrar_extensao_persistente(Caixa, "estudo/extra", exportar_extra,
  aplicar_extra)
mostre extensoes_para_dados(Caixa(7))["estudo/extra"]["valor"]
```

Resultado

Saída esperada

```
7
```

Erros

ValueError para identificador inválido ou associado a outro tipo; erros de contratos de tipo e callbacks surgem ao usar a extensão.

Atenção

Repetir identificador e tipo substitui a extensão e a recoloca ao final da lista desse tipo. O registro não cria adaptador do hospedeiro e não grava arquivo. Callbacks não são serializados. Forneça funções válidas e dados compatíveis com o serializador.

`extensoes_para_dados(...)` · módulo `coral.persistencia`

O que faz

Consulta extensões registradas e reúne seus payloads para um objeto.

Assinatura

Assinatura para consulta

```
extensoes_para_dados(objeto: 'Any') -> 'dict[str, Any]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto hospedeiro a consultar.

Retorno

Dicionário de identificadores para payloads; vazio quando nenhuma extensão fornece dados.

Exemplo

Código Coral

```
de coral.persistencia importe extensoes_para_dados
crie a classe Caixa
    ao criar uma Caixa com valor
        defina seu valor como valor
    fim
fim
crie a função exportar_extra com objeto
    retorne {"valor": objeto.valor}
fim
crie a função aplicar_extra com objeto, dados
    execute setattr(objeto, "valor", dados["valor"])
fim
de coral.persistencia importe registrar_extensao_persistente
execute registrar_extensao_persistente(Caixa, "estudo/extra", exportar_extra,
  aplicar_extra)
mostre extensoes_para_dados(Caixa(7))["estudo/extra"]["valor"]
```

Resultado

Saída esperada

```
7
```

Erros

Erros dos callbacks e de tipos registrados incompatíveis são propagados.

Atenção

Consulta tipos compatíveis por isinstance. Payload nulo é omitido. Não clona nem serializa automaticamente os payloads; callbacks podem executar código e produzir efeitos. Para formar o documento persistente completo, use para_dados ou salvar.

`aplicar_extensoes_persistentes(...)` · módulo `coral.persistencia`

O que faz

Aplica payloads de extensões registradas a um objeto já existente.

Assinatura

Assinatura para consulta

```
aplicar_extensoes_persistentes(objeto: 'Any', extensoes: 'Any') -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto hospedeiro compatível com os tipos das extensões.
extensoes	Any	sim	sem padrão	Dicionário de identificadores para payloads, ou nulo sem alterações.

Retorno

nulo; callbacks podem alterar o objeto.

Exemplo

Código Coral

```
de coral.persistencia importe aplicar_extensoes_persistentes
crie a classe Caixa
    ao criar uma Caixa com valor
        defina seu valor como valor
    fim
fim
crie a função exportar_extra com objeto
    retorne {"valor": objeto.valor}
fim
crie a função aplicar_extra com objeto, dados
    execute setattr(objeto, "valor", dados["valor"])
fim
de coral.persistencia importe registrar_extensao_persistente
execute registrar_extensao_persistente(Caixa, "estudo/extra", exportar_extra,
  aplicar_extra)
defina caixa como Caixa(0)
execute aplicar_extensoes_persistentes(caixa, {"estudo/extra": {"valor": 7}})
mostre caixa.valor
```

Resultado

Saída esperada

```
7
```

Erros

ErroPersistencia para extensoes fora de dicionário, identificador inválido, extensão desconhecida ou tipo hospedeiro incompatível; erros de importação e callbacks são propagados.

Atenção

Aplica itens em ordem de inserção. Não há transação nem rollback: se uma extensão falhar, as anteriores podem ter sido aplicadas. Identificadores oficiais iniciados em coral. podem importar seu módulo para localizar a extensão. O payload é validado pelo callback da extensão.

`executar_comando(...)` · módulo `coral.sistema`

O que faz

Executa um programa sem shell e captura saída, erro e código de retorno.

Assinatura

Assinatura para consulta

```
executar_comando(programa: 'str', argumentos: 'Iterable[Any]' = (), *, pasta: 'str |
  Path | None' = None, timeout: 'float | None' = None, ambiente: 'Mapping[str, Any] |
  None' = None, entrada: 'str | None' = None, relogio: 'FonteTempo | None' = None,
  codificacao: 'str | None' = None, erros_decodificacao: 'str' = 'estrito',
  limite_saida_bytes: 'int | None' = None) -> 'ResultadoComando'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
programa	str	sim	sem padrão	Nome ou caminho textual não vazio do executável.
argumentos	Iterable[Any]	não	()	Iterável convertido para argumentos textuais; padrão vazio.
pasta	str | Path | None	não	None	Diretório de trabalho com expansão de usuário, ou nulo para herdar.
timeout	float | None	não	None	Tempo finito não negativo, ou nulo sem limite.
ambiente	Mapping[str, Any] | None	não	None	Mapeamento que complementa o ambiente herdado, ou nulo.
entrada	str | None	não	None	Texto enviado à entrada padrão, ou nulo sem envio.
relogio	FonteTempo | None	não	None	FonteTempo para medir duração, ou nulo para relógio monotônico.
codificacao	str | None	não	None	Nome de codec para entrada e saída, ou nulo para padrão do runtime.
erros_decodificacao	str	não	'estrito'	estrito, substituir ou ignorar; também aceita strict, replace e ignore. Padrão estrito.
limite_saida_bytes	int | None	não	None	Inteiro não negativo de bytes retidos por fluxo, ou nulo sem limite.

Retorno

ResultadoComando com comando, codigo, saida, erro, duracao_segundos e flags de truncamento.

Exemplo

Código Coral

```
de coral.sistema importe executar_comando
tente
    execute executar_comando("", timeout=1)
se der erro do tipo ValueError
    mostre "Informe um executável"
fim
```

Resultado

Saída esperada

```
Informe um executável
```

Erros

ValueError para programa vazio, duração, limite ou política inválidos; ProgramaNaoEncontrado para executável ausente; TempoLimiteExcedido por timeout; ErroSistema para codificação; FalhaSistemaOperacional para erros de processo ou diretório; outros erros de configuração são propagados.

Atenção

O exemplo mostra a validação antes de criar processo. Para execução real, use um executável existente, como executar_comando("python3", ["--version"], timeout=5). Argumentos não passam por shell. Código não zero é registrado no resultado, sem exceção automática. Limite de saída vale individualmente para stdout e stderr; excedentes são drenados. Timeout encerra a árvore de processos e tem precedência sobre truncamento e retorno.

`requisitar(...)` · módulo `coral.web.http`

O que faz

Executa uma requisição HTTP ou HTTPS com limites explícitos de tempo, resposta e redirecionamento.

Assinatura

Assinatura para consulta

```
requisitar(metodo: 'str', url: 'str', cabecalhos: 'Any' = None, corpo: 'Any' = None,
  tempo_limite: 'float' = 10.0, *, codificacao_corpo: 'str | None' = None,
  limite_resposta: 'int' = 8388608, seguir_redirecionamentos: 'bool | None' = None,
  max_redirecionamentos: 'int' = 5, proxy: 'Any' = None, contexto_tls: 'ssl.SSLContext |
  None' = None) -> 'RespostaHTTP'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
metodo	str	sim	sem padrão	Método HTTP normalizado para maiúsculas, sem controles.
url	str	sim	sem padrão	URL absoluta http ou https, sem credenciais incorporadas.
cabecalhos	Any	não	None	Mapeamento ou pares válidos de nome e valor, ou nulo.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
corpo	Any	não	None	Bytes, bytearray, memoryview, texto com codec explícito, ou nulo.
tempo_limite	float	não	10.0	Tempo finito positivo do transporte; padrão dez segundos.
codificacao_corpo	str | None	não	None	Codec necessário quando corpo é texto, ou nulo para dados binários.
limite_resposta	int	não	8388608	Inteiro não negativo do máximo de bytes do corpo; padrão 8388608.
seguir_redirecionamentos	bool | None	não	None	Lógico explícito, ou nulo para seguir somente GET, HEAD e OPTIONS.
max_redirecionamentos	int	não	5	Inteiro não negativo do máximo de redirecionamentos; padrão cinco.
proxy	Any	não	None	nulo para conexão direta, URL textual para ambos os esquemas ou mapeamento de http e https para URLs de proxy.
contexto_tls	ssl.SSLContext | None	não	None	SSLContext explícito, ou nulo para contexto TLS padrão com verificação.

Retorno

RespostaHTTP com status, cabeçalhos, corpo em bytes, URL final e redirecionamentos.

Exemplo

Código Coral

```
de coral.web.http importe requisitar
# Exemplo de HTTP no loopback
de coral.web.servidor importe criar_servidor, resposta_texto
crie a função pagina com requisicao
    retorne resposta_texto("Coral local")
fim
defina servidor como criar_servidor()
execute servidor.adicionar_rota("GET", "/", pagina)
execute servidor.iniciar()
tente
    defina resposta como requisitar("GET", servidor.url_base + "/", tempo_limite=2,
      limite_resposta=4096)
    mostre resposta.status
    mostre resposta.texto()
finalmente
    execute servidor.encerrar()
fim
```

Resultado

Saída esperada

```
200
Coral local
```

Erros

ErroValor para método, URL, cabeçalhos, corpo, codec, limites, proxy ou contexto inválidos; ErroHTTP para timeout, TLS ou transporte; ErroLimiteHTTP ao exceder corpo; ErroRedirecionamentoHTTP ao exceder a política de redirecionamento.

Atenção

3xx, 4xx e 5xx são respostas inspecionáveis, sem exceção de transporte automática. texto e json decodificam o corpo após a chamada. Métodos com efeitos não seguem redirecionamentos por padrão. O cliente não usa proxy de ambiente implicitamente quando proxy é nulo. Configuração TLS deve respeitar o serviço acessado.

`enviar(...)` · módulo `coral.web.http`

O que faz

Envia um corpo por POST com as mesmas políticas do cliente HTTP.

Assinatura

Assinatura para consulta

```
enviar(url: 'str', corpo: 'Any', cabecalhos: 'Any' = None, tempo_limite: 'float' = 10.0,
    **opcoes: 'Any') -> 'RespostaHTTP'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
url	str	sim	sem padrão	URL absoluta http ou https sem credenciais embutidas.
corpo	Any	sim	sem padrão	Dados binários ou texto com codificacao_corpo explícita.
cabecalhos	Any	não	None	Mapeamento ou sequência de pares HTTP, ou nulo.
tempo_limite	float	não	10.0	Tempo finito positivo do transporte; padrão dez segundos.
opcoes	Any	não	sem padrão	Opções nomeadas de requisitar, como codificacao_corpo, limite_resposta, proxy, contexto_tls e redirecionamentos.

Retorno

RespostaHTTP materializada após o POST.

Exemplo

Código Coral

```
de coral.web.http importe enviar
# Exemplo de HTTP no loopback
de coral.web.servidor importe criar_servidor, resposta_texto
crie a função pagina com requisicao
    retorne resposta_texto("Coral local")
fim
defina servidor como criar_servidor()
execute servidor.adicionar_rota("POST", "/", pagina)
execute servidor.iniciar()
tente
    defina resposta como enviar(servidor.url_base + "/", "olá",
      codificacao_corpo="utf-8", tempo_limite=2, limite_resposta=4096)
    mostre resposta.status
    mostre resposta.texto()
finalmente
    execute servidor.encerrar()
fim
```

Resultado

Saída esperada

```
200
Coral local
```

Erros

Erros públicos de requisitar para configuração, limites, transporte ou TLS; TypeError para opção desconhecida.

Atenção

Equivale a requisitar("POST", ...). POST não segue redirecionamentos por padrão; habilite explicitamente se aceitar repetição ou alteração da requisição. Corpo textual exige codec; a função não converte dicionário em JSON automaticamente. Status 4xx ou 5xx continuam disponíveis no objeto resposta.

`ServidorHTTP(...)` · **módulo** `coral.web.servidor`

O que faz

Configura um servidor de desenvolvimento com rotas explícitas e limites de corpo.

Construtor

Assinatura para consulta

```
ServidorHTTP(host: 'str' = '127.0.0.1', porta: 'int' = 0, *, limite_corpo: 'int' =
  1048576, tempo_limite_corpo: 'float' = 10.0, raiz_arquivos: 'Any' = None,
  permitir_externo: 'bool' = False)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
host	str	não	'127.0.0.1'	Endereço de escuta; padrão 127.0.0.1.
porta	int	não	0	Inteiro de zero a 65535; zero solicita porta disponível.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
limite_corpo	int	não	1048576	Inteiro não negativo do máximo de bytes recebidos; padrão 1048576.
tempo_limite_corpo	float	não	10.0	Duração finita positiva de leitura do corpo; padrão dez segundos.
raiz_arquivos	Any	não	None	Diretório válido para rotas de arquivo, ou nulo sem essa raiz.
permitir_externo	bool	não	False	Autorização explícita para host não local; padrão falso.

Atributos importantes

Atributo	Significado
`ativo`	Se a thread e servidor estão ativos.
`porta`	Porta solicitada antes de iniciar; porta efetiva depois.
`url_base`	URL do servidor iniciado.
`ultimo_erro`	Última exceção de tratamento, ou nulo.
`raiz_arquivos`	Raiz permitida para arquivos.

Métodos principais

Assinatura para consulta

```
adicionar_rota(self, metodo: 'str', caminho: 'str', tratador: 'TratadorHTTP') ->
  "'ServidorHTTP'"
```

Referência para leitura

```
adicionar_arquivo(self, caminho_rota: 'str', caminho_arquivo: 'Any', *, limite_arquivo:
  'int' = 8388608, cabecalhos: 'Any' = None) -> "'ServidorHTTP'"
```

Referência para leitura

```
iniciar(self) -> "'ServidorHTTP'"
```

Referência para leitura

```
encerrar(self) -> 'None'
```

Retorno

Nova instância de ServidorHTTP.

Exemplo

Código Coral

```
de coral.web.servidor importe ServidorHTTP
# Exemplo de HTTP no loopback
defina servidor como ServidorHTTP()
mostre servidor.ativo
execute servidor.iniciar()
tente
    mostre servidor.ativo
finalmente
    execute servidor.encerrar()
fim
```

Resultado

Saída esperada

```
False
True
```

Erros

ErroValor para host, porta, limites ou autorização inválidos; ErroRotaHTTP para rota duplicada, caminho ou tratador inválidos; ErroServidorHTTP para abertura de porta, reinício indevido ou alteração de rotas durante execução.

Atenção

O construtor não abre a porta. iniciar cria servidor e thread; encerrar fecha ambos e pode ser repetido. Registre rotas antes de iniciar. url_base exige servidor iniciado. O servidor é voltado ao desenvolvimento local, não oferece TLS próprio nem autenticação automática. Rotas de arquivo usam raiz explícita.

`resposta_arquivo(...)` · **módulo** `coral.web.servidor`

O que faz

Materializa uma resposta HTTP com um arquivo contido em uma raiz explícita.

Assinatura

Assinatura para consulta

```
resposta_arquivo(raiz: 'Any', caminho_relativo: 'Any', status: 'int' = 200, cabecalhos:
  'Any' = None, *, limite_arquivo: 'int' = 8388608) -> 'RespostaServidorHTTP'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
raiz	Any	sim	sem padrão	Diretório existente autorizado para leitura.
caminho_relativo	Any	sim	sem padrão	Caminho relativo válido dentro da raiz.
status	int	não	200	Inteiro HTTP de 100 a 599; padrão 200.
cabecalhos	Any	não	None	Mapeamento ou pares HTTP válidos, ou nulo.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
limite_arquivo	int	não	8388608	Inteiro não negativo do máximo de bytes do arquivo; padrão 8388608.

Retorno

RespostaServidorHTTP com corpo em bytes e tipo de conteúdo inferido pela extensão.

Exemplo

Código Coral

```
de coral.web.servidor importe resposta_arquivo
de pathlib importe Path
execute Path("pagina.txt").write_text("Coral", encoding="utf-8")
defina resposta como resposta_arquivo(".", "pagina.txt")
mostre resposta.status
mostre resposta.corpo_bytes.decode("utf-8")
```

Resultado

Saída esperada

```
200
Coral
```

Erros

ErroValor para status, cabeçalhos, limite, raiz ou caminho inválidos, fuga da raiz ou arquivo ausente; ErroServidorHTTP para inspeção, leitura ou tamanho excedido.

Atenção

Não inicia servidor nem registra rota. Resolve caminho e exige arquivo dentro da raiz, inclusive após resolução de links. Verifica tamanho antes de ler; não oferece streaming. Para registrar uma rota de arquivo, use ServidorHTTP.adicionar_arquivo com raiz_arquivos configurada.

`resposta_grafico(...)` · módulo `coral.web.servidor`

O que faz

Converte um gráfico Coral em resposta HTML autossuficiente.

Assinatura

Assinatura para consulta

```
resposta_grafico(grafico: 'Any', status: 'int' = 200, cabecalhos: 'Any' = None, *,
  limite_pontos: 'int' = 200000) -> 'RespostaServidorHTTP'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grafico	Any	sim	sem padrão	Gráfico aceito por coral.graficos.para_html.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
status	int	não	200	Inteiro HTTP de 100 a 599; padrão 200.
cabecalhos	Any	não	None	Mapeamento ou pares HTTP válidos, ou nulo.
limite_pontos	int	não	200000	Inteiro do orçamento de pontos da exportação; padrão 200000.

Retorno

RespostaServidorHTTP com HTML UTF 8 em bytes.

Exemplo

Código Coral

```
de coral.web.servidor importe resposta_grafico
de coral.graficos importe barras
defina resposta como resposta_grafico(barras(["A"], [1]))
mostre resposta.status
mostre resposta.corpo_bytes.decode("utf-8").find("coral.graficos.html/1") >= 0
```

Resultado

Saída esperada

```
200
True
```

Erros

Erros gráficos e de dependência de para_html; ErroValor para status ou cabeçalhos inválidos.

Atenção

Não inicia servidor. Carrega Plotly apenas na exportação. Adiciona Cache-Control: no-store e X-Content-Type-Options: nosniff quando ausentes. O HTML incorpora dados e recursos necessários; o programa deve escolher quais informações deseja servir.

`Cor(...)` · módulo `coral.jogos`

O que faz

Representa canais de cor e transparência para a API de desenho.

Construtor

Assinatura para consulta

```
Cor(vermelho: 'int', verde: 'int', azul: 'int', alfa: 'int' = 255) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
vermelho	int	sim	sem padrão	Canal vermelho inteiro de zero a 255.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
verde	int	sim	sem padrão	Canal verde inteiro de zero a 255.
azul	int	sim	sem padrão	Canal azul inteiro de zero a 255.
alfa	int	não	255	Transparência de zero a 255; padrão 255, opaco.

Métodos principais

Assinatura para consulta

```
rgb(self) -> 'tuple[int, int, int]'
```

Retorno

Nova instância de Cor.

Exemplo

Código Coral

```
de coral.jogos importe Cor
defina cor como Cor(20, 40, 60)
mostre cor.rgb()
mostre cor.alfa
```

Resultado

Saída esperada

```
(20, 40, 60)
255
```

Erros

ValueError para canais fora da faixa ou conversão inválida; TypeError para valores não conversíveis.

Atenção

Registro imutável. A validação converte temporariamente os canais com int, mas não substitui os valores armazenados; forneça inteiros. Construir uma cor não desenha nem altera uma janela.

`Retangulo(...)` · módulo `coral.jogos`

O que faz

Guarda posição e dimensões de um retângulo alinhado aos eixos.

Construtor

Assinatura para consulta

```
Retangulo(x: 'float', y: 'float', largura: 'float', altura: 'float') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
x	float	sim	sem padrão	Coordenada horizontal do canto inicial.
y	float	sim	sem padrão	Coordenada vertical do canto inicial.
largura	float	sim	sem padrão	Extensão horizontal; use valor positivo.
altura	float	sim	sem padrão	Extensão vertical; use valor positivo.

Métodos principais

Assinatura para consulta

```
colide_com(self, outro: "'Retangulo'") -> 'bool'
```

Retorno

Nova instância de Retangulo.

Exemplo

Código Coral

```
de coral.jogos importe Retangulo
defina area como Retangulo(0, 0, 10, 10)
mostre area.colide_com(Retangulo(5, 5, 10, 10))
mostre area.colide_com(Retangulo(10, 0, 10, 10))
```

Resultado

Saída esperada

```
True
False
```

Erros

TypeError ou AttributeError ao comparar campos incompatíveis ou objeto sem os campos exigidos.

Atenção

Registro mutável sem normalização ou validação automática de dimensões. colide_com testa sobreposição estrita; tocar uma borda não conta como colisão. Não incorpora rotação.

`Viewport(...)` · módulo `coral.jogos`

O que faz

Converte coordenadas entre resolução lógica e física do desenho.

Construtor

Assinatura para consulta

```
Viewport(largura_logica: 'float', altura_logica: 'float', largura_fisica: 'float',
  altura_fisica: 'float', modo: 'str' = 'ajustar') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
largura_logica	float	sim	sem padrão	Largura lógica finita positiva.
altura_logica	float	sim	sem padrão	Altura lógica finita positiva.
largura_fisica	float	sim	sem padrão	Largura física finita positiva.
altura_fisica	float	sim	sem padrão	Altura física finita positiva.
modo	str	não	'ajustar'	ajustar preserva proporção e centraliza; esticar preenche com escalas independentes.

Atributos importantes

Atributo	Significado
`escala_x`	Fator horizontal.
`escala_y`	Fator vertical.
`area_fisica_util`	Tupla x, y, largura e altura da área útil.

Métodos principais

Assinatura para consulta

```
logica_para_tela(self, x: 'float', y: 'float') -> 'tuple[float, float]'
```

Referência para leitura

```
tela_para_logica(self, x: 'float', y: 'float', *, limitar: 'bool' = False) ->
  'tuple[float, float]'
```

Referência para leitura

```
contem_tela(self, x: 'float', y: 'float') -> 'bool'
```

Referência para leitura

```
ancorar(self, largura: 'float', altura: 'float', *, ancora: 'str' = 'superior_esquerda',
   margem_x: 'float' = 0.0, margem_y: 'float | None' = None) -> 'tuple[float, float]'
```

Retorno

Nova instância de Viewport.

Exemplo

Código Coral

```
de coral.jogos importe Viewport
defina janela como Viewport(100, 50, 200, 200)
mostre janela.area_fisica_util
mostre janela.logica_para_tela(0, 0)
mostre janela.tela_para_logica(0, 50)
```

Resultado

Saída esperada

```
(0.0, 50.0, 200.0, 100.0)
(0.0, 50.0)
(0.0, 0.0)
```

Erros

ValueError para medidas, modo, âncora ou argumentos numéricos inválidos.

Atenção

Registro imutável. Em ajustar podem existir faixas fora da área útil. tela_para_logica não limita coordenadas por padrão; limitar=verdadeiro recorta à área lógica. ancorar devolve posição lógica para uma das nove âncoras, sem desenhar.

`FormaTransformada(...)` · módulo `coral.jogos`

O que faz

Armazena vértices 2D de uma forma após transformações.

Construtor

Assinatura para consulta

```
FormaTransformada(vertices: 'tuple[tuple[float, float], ...]') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
vertices	tuple[tuple[float, float], ...]	sim	sem padrão	Sequência com pelo menos três pares de coordenadas finitas.

Atributos importantes

Atributo	Significado
`vertices`	Tupla normalizada de pares.
`limites`	Extremos alinhados aos eixos.

Retorno

Nova instância de FormaTransformada.

Exemplo

Código Coral

```
de coral.jogos importe FormaTransformada
defina forma como FormaTransformada(((0, 0), (10, 0), (0, 10)))
mostre forma.limites
```

Resultado

Saída esperada

```
(0.0, 0.0, 10.0, 10.0)
```

Erros

ValueError para poucos vértices, pares malformados ou números inválidos; TypeError para coleção incompatível.

Atenção

Normaliza coordenadas para float e vértices para tupla imutável. limites significa mínimo x, mínimo y, máximo x, máximo y, e não largura e altura. Não valida convexidade, área ou autointerseção; formas_colidem pressupõe polígonos convexos válidos.

`QuadroSprite(...)` · módulo `coral.jogos`

O que faz

Descreve um recorte de imagem de spritesheet sem carregar a imagem.

Construtor

Assinatura para consulta

```
QuadroSprite(caminho: 'Path', x: 'int', y: 'int', largura: 'int', altura: 'int', indice:
    'int | None' = None, linha: 'int | None' = None, coluna: 'int | None' = None, nome:
   'str | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	Path	sim	sem padrão	Path da imagem de origem.
x	int	sim	sem padrão	Origem horizontal não negativa do recorte.
y	int	sim	sem padrão	Origem vertical não negativa do recorte.
largura	int	sim	sem padrão	Largura positiva do recorte.
altura	int	sim	sem padrão	Altura positiva do recorte.
indice	int | None	não	None	Índice opcional do quadro.
linha	int | None	não	None	Linha opcional da grade.
coluna	int | None	não	None	Coluna opcional da grade.
nome	str | None	não	None	Nome opcional do quadro.

Atributos importantes

Atributo	Significado
`retangulo`	Tupla x, y, largura e altura do recorte.
`caminho`	Origem da imagem.

Retorno

Nova instância de QuadroSprite.

Exemplo

Código Coral

```
de coral.jogos importe QuadroSprite
de pathlib importe Path
defina quadro como QuadroSprite(Path("atlas.png"), 0, 0, 16, 16, nome="parado")
mostre quadro.retangulo
mostre quadro.nome
```

Resultado

Saída esperada

```
(0, 0, 16, 16)
parado
```

Erros

ValueError para origem negativa ou dimensões não positivas; TypeError para valores não comparáveis.

Atenção

Registro imutável. Use coordenadas e dimensões inteiras. Não verifica existência da imagem nem se o recorte cabe nela; essas verificações dependem do carregamento no backend. O construtor não converte caminho textual para Path.

`EstiloCelula(...)` · **módulo** `coral.jogos`

O que faz

Agrupa opções visuais de uma célula de mapa.

Construtor

Assinatura para consulta

```
EstiloCelula(cor: 'Cor | None' = None, imagem: 'str | QuadroSprite | None' = None,
  texto: 'str | None' = None, cor_texto: 'Cor' = Cor(vermelho=255, verde=255, azul=255,
  alfa=255), tamanho_texto: 'int' = 16, contorno: 'int' = 0, opacidade: 'float' = 1.0) ->
  None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
cor	Cor | None	não	None	Cor de preenchimento ou nulo.
imagem	str | QuadroSprite | None	não	None	Caminho de imagem ou QuadroSprite, ou nulo.
texto	str | None	não	None	Texto sobre a célula, ou nulo.
cor_texto	Cor	não	Cor(vermelho=255, verde=255, azul=255, alfa=255)	Cor do texto; padrão branco.
tamanho_texto	int	não	16	Tamanho do texto; padrão 16.
contorno	int	não	0	Espessura do contorno; zero para preenchimento.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
opacidade	float	não	1.0	Opacidade desejada entre zero e um; padrão um.

Retorno

Nova instância de EstiloCelula.

Exemplo

Código Coral

```
de coral.jogos importe EstiloCelula
defina estilo como EstiloCelula(texto="A", opacidade=0.5)
mostre estilo.texto
mostre estilo.opacidade
```

Resultado

Saída esperada

```
A
0.5
```

Erros

TypeError para argumentos fora da assinatura; dados visuais incompatíveis podem falhar ao desenhar.

Atenção

Registro imutável sem validação integral no construtor. A criação não desenha, carrega imagem nem altera mapa. Use com as funções públicas de desenho de mapas; a interpretação e validação das opções pertencem ao consumidor.

`Cena(...)` · módulo `coral.jogos`

O que faz

Organiza sprites, representações, camadas e callbacks de uma cena.

Construtor

Assinatura para consulta

```
Cena(nome: 'str')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome identificador da cena.

Atributos importantes

Atributo	Significado
`nome`	Identificação.
`sprites`	Sprites da cena.

Atributo	Significado
`camadas`	Tupla ordenada de camadas.
`representacoes`	Tupla de vínculos de entidades e sprites.
`camera`	Câmera opcional.

Métodos principais

Assinatura para consulta

```
adicionar(self, sprite: 'Sprite', *, camada: 'str | CamadaApresentacao | None' = None)
  -> 'Sprite'
```

Referência para leitura

```
remover(self, sprite: 'Sprite') -> 'None'
```

Referência para leitura

```
criar_camada(self, nome: 'str', *, ordem: 'int' = 0, visivel: 'bool' = True, opacidade:
  'float' = 1.0, espaco: 'str' = 'mundo', paralaxe: 'float | tuple[float, float]' = 1.0)
  -> 'CamadaApresentacao'
```

Referência para leitura

```
camada(self, nome: 'str') -> 'CamadaApresentacao'
```

Referência para leitura

```
adicionar_em_camada(self, elemento: 'Any', camada: 'str | CamadaApresentacao') -> 'Any'
```

Referência para leitura

```
atualizar(self, jogo, dt: 'float')
```

Referência para leitura

```
desenhar(self, jogo)
```

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de Cena.

Exemplo

Código Coral

```
de coral.jogos importe Cena
de coral.jogos importe Sprite
defina cena como Cena("campo")
defina jogador como Sprite(0, 0, 10, 10)
execute cena.adicionar(jogador)
mostre len(cena.sprites)
execute cena.remover(jogador)
mostre len(cena.sprites)
```

Resultado

Saída esperada

```
1
0
```

Erros

ValueError para camada ausente, duplicada ou de outra cena; erros de sprites, representações e callbacks são propagados.

Atenção

Não abre janela nem inicia laço de jogo. adicionar evita repetição do mesmo sprite; uma camada informada deve pertencer à cena. camadas devolve tupla ordenada por ordem, preservando empates. sprites é lista pública mutável, mas use os métodos para manter associações de camada coerentes. Callbacks podem executar código do programa.

`MetricasJogo(...)` · módulo `coral.jogos`

O que faz

Mantém medidas leves do laço de jogo atualizadas explicitamente.

Construtor

Assinatura para consulta

```
MetricasJogo(quadros: 'int' = 0, delta_logico: 'float' = 0.0, tempo_quadro: 'float' =
  0.0, fps_instantaneo: 'float' = 0.0, fps_medio: 'float' = 0.0) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
quadros	int	não	0	Quantidade inicial de quadros; padrão zero.
delta_logico	float	não	0.0	Último intervalo lógico em segundos.
tempo_quadro	float	não	0.0	Última duração física de quadro em segundos.
fps_instantaneo	float	não	0.0	Último FPS calculado.
fps_medio	float	não	0.0	Estimativa suavizada de FPS.

Atributos importantes

Atributo	Significado
`tempo_quadro_ms`	Última duração convertida para milissegundos.

Métodos principais

Assinatura para consulta

```
registrar(self, delta_logico: 'float', tempo_quadro: 'float') -> 'None'
```

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de MetricasJogo.

Exemplo

Código Coral

```
de coral.jogos importe MetricasJogo
defina medida como MetricasJogo()
execute medida.registrar(0.02, 0.02)
execute medida.registrar(0.01, 0.01)
mostre medida.quadros
mostre medida.tempo_quadro_ms
mostre medida.fps_medio
```

Resultado

Saída esperada

```
2
10.0
55.0
```

Erros

ValueError para intervalos inválidos; campos iniciais incoerentes podem falhar no registro.

Atenção

Registro mutável; o construtor não valida integralmente campos iniciais. registrar valida intervalos finitos não negativos, incrementa quadros e calcula FPS como inverso do tempo físico, ou zero para tempo zero. Depois do primeiro quadro, fps_medio usa 90% da estimativa anterior e 10% do novo FPS; não é média aritmética. Não consulta relógio por conta própria.

`PrimitivaDebug(...)` · módulo `coral.jogos`

O que faz

Descreve uma instrução geométrica neutra de depuração.

Construtor

Assinatura para consulta

```
PrimitivaDebug(tipo: 'str', categoria: 'str', dados: 'Mapping[str, Any]' = <factory>,
  rotulo: 'str | None' = None, espaco: 'str' = 'mundo') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tipo	str	sim	sem padrão	Tipo da primitiva, como retangulo, poligono, ponto, linha ou texto.
categoria	str	sim	sem padrão	Categoria visual, como limites ou metricas.
dados	Mapping[str, Any]	não	<factory>	Mapeamento dos dados geométricos; padrão novo dicionário vazio.
rotulo	str | None	não	None	Rótulo opcional.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
espaco	str	não	'mundo'	mundo usa a câmera no desenho; tela usa coordenadas de tela.

Métodos principais

Assinatura para consulta

```
para_dict(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de PrimitivaDebug.

Exemplo

Código Coral

```
de coral.jogos importe PrimitivaDebug
defina item como PrimitivaDebug("texto", "metricas", {"texto": "FPS", "x": 0, "y": 0},
  espaco="tela")
mostre item.para_dict()["espaco"]
```

Resultado

Saída esperada

```
tela
```

Erros

TypeError para argumentos incompatíveis com a assinatura; erros de normalização ou consumo podem surgir depois.

Atenção

Registro congelado, mas dados pode referenciar mapeamento mutável. O construtor não valida integralmente tipo ou geometria. para_dict normaliza os dados para inspeção. Não desenha; prefira gerar_overlay_debug para obter primitivas coerentes.

`OverlayDebug(...)` · módulo `coral.jogos`

O que faz

Reúne primitivas de depuração e a assinatura opcional da inspeção de origem.

Construtor

Assinatura para consulta

```
OverlayDebug(primitivas: 'tuple[PrimitivaDebug, ...]' = (), assinatura_origem: 'str |
  None' = None, contrato: 'str' = 'coral.jogos.debug.overlay/1') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
primitivas	tuple[PrimitivaDebug, ...]	não	()	Tupla de PrimitivaDebug; padrão vazia.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
assinatura_origem	str | None	não	None	Assinatura opcional do snapshot que gerou o overlay.
contrato	str	não	'coral.jogos.debug.overlay/1'	Identificador do formato; padrão coral.jogos.debug.overlay/1.

Atributos importantes

Atributo	Significado
`quantidade`	Número de primitivas.

Métodos principais

Assinatura para consulta

```
por_categoria(self, categoria: 'str') -> 'tuple[PrimitivaDebug, ...]'
```

Referência para leitura

```
para_dict(self) -> 'dict[str, Any]'
```

Referência para leitura

```
para_json(self, **kwargs: 'Any') -> 'str'
```

Retorno

Nova instância de OverlayDebug.

Exemplo

Código Coral

```
de coral.jogos importe OverlayDebug
de coral.jogos importe PrimitivaDebug
defina painel como OverlayDebug((PrimitivaDebug("ponto", "pivo", {"x": 0, "y": 0}),))
mostre painel.quantidade
mostre len(painel.por_categoria("PIVO"))
```

Resultado

Saída esperada

```
1
1
```

Erros

Erros de normalização, serialização JSON ou itens incompatíveis são propagados ao consumir o overlay.

Atenção

Registro imutável superficialmente; dados internos das primitivas podem ser mutáveis. por_categoria ignora caixa e espaços externos, mas não aplica todos os aliases da configuração de debug. para_json não grava arquivo. O construtor manual não valida o contrato ou tipos dos itens.

`EstiloOverlayDebug(...)` · módulo `coral.jogos`

O que faz

Define cores e medidas para desenhar um overlay de depuração.

Construtor

Assinatura para consulta

```
EstiloOverlayDebug(limites: 'Any' = None, colisor: 'Any' = None, pivo: 'Any' = None,
  camera: 'Any' = None, camera_centro: 'Any' = None, rotulo: 'Any' = None, metricas: 'Any'
   = None, espessura: 'int' = 1, tamanho_pivo: 'int' = 3, tamanho_texto: 'int' = 14,
  opacidade: 'float' = 1.0) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
limites	Any	não	None	Cor opcional da categoria; nulo usa o padrão.
colisor	Any	não	None	Cor opcional da categoria; nulo usa o padrão.
pivo	Any	não	None	Cor opcional da categoria; nulo usa o padrão.
camera	Any	não	None	Cor opcional da categoria; nulo usa o padrão.
camera_centro	Any	não	None	Cor opcional da categoria; nulo usa o padrão.
rotulo	Any	não	None	Cor opcional da categoria; nulo usa o padrão.
metricas	Any	não	None	Cor opcional da categoria; nulo usa o padrão.
espessura	int	não	1	Espessura de traços; padrão um.
tamanho_pivo	int	não	3	Tamanho dos marcadores de pivô; padrão três.
tamanho_texto	int	não	14	Tamanho do texto; padrão 14.
opacidade	float	não	1.0	Opacidade desejada de zero a um; padrão um.

Métodos principais

Assinatura para consulta

```
cor_para(self, categoria: 'str')
```

Retorno

Nova instância de EstiloOverlayDebug.

Exemplo

Código Coral

```
de coral.jogos importe EstiloOverlayDebug
de coral.jogos importe BRANCO
defina estilo como EstiloOverlayDebug()
mostre estilo.cor_para("metricas") == BRANCO
mostre estilo.espessura
```

Resultado

Saída esperada

```
True
1
```

Erros

TypeError para argumentos desconhecidos; erros de desenho aparecem ao consumir o estilo.

Atenção

Registro imutável sem validação integral no construtor. cor_para usa cores padrão quando o campo correspondente é nulo; categorias desconhecidas usam branco. O estilo não gera primitivas nem altera objetos da cena. Valores incompatíveis podem falhar no backend.

`ConfiguracaoDebugJogo(...)` · **módulo** `coral.jogos`

O que faz

Guarda a configuração persistente de depuração de um jogo.

Construtor

Assinatura para consulta

```
ConfiguracaoDebugJogo(ativo: 'bool' = False, categorias: 'frozenset[str]' =
  frozenset({'camera', 'colisor', 'limites', 'metricas', 'pivo'}), incluir_invisiveis:
  'bool' = False, estilo: 'EstiloOverlayDebug' = <factory>) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
ativo	bool	não	False	Se o debug deve estar ativo; padrão falso.
categorias	frozenset[str]	não	frozenset({'camera', 'colisor', 'limites', 'metricas', 'pivo'})	Categorias a mostrar; aceita aliases normalizados e remove duplicatas.
incluir_invisiveis	bool	não	False	Se deve incluir objetos invisíveis; padrão falso.
estilo	EstiloOverlayDebug	não	<factory>	EstiloOverlayDebug; padrão novo estilo.

Métodos principais

Assinatura para consulta

```
com_alteracoes(self, *, ativo: 'bool | None' = None, categorias: 'Any' = None,
  incluir_invisiveis: 'bool | None' = None, estilo: 'EstiloOverlayDebug | None' = None) ->
   "'ConfiguracaoDebugJogo'"
```

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de ConfiguracaoDebugJogo.

Exemplo

Código Coral

```
de coral.jogos importe ConfiguracaoDebugJogo
defina config como ConfiguracaoDebugJogo(categorias=["FPS", "pivô"])
mostre sorted(config.categorias)
defina ativa como config.com_alteracoes(ativo=verdadeiro)
mostre config.ativo
mostre ativa.ativo
```

Resultado

Saída esperada

```
['metricas', 'pivo']
False
True
```

Erros

ValueError para categoria desconhecida; TypeError para coleção não iterável ou argumentos fora da assinatura.

Atenção

Registro imutável. Normaliza categorias para frozenset; os padrões incluem limites, colisor, pivo, camera e metricas. com_alteracoes devolve outra configuração, preservando campos omitidos. A criação não liga debug em um Jogo existente. estado_inspecao resume opções, sem serializar integralmente o estilo.

`mundo_para_tela(...)` · módulo `coral.jogos`

O que faz

Projeta uma posição espacial 2D do mundo na tela.

Assinatura

Assinatura para consulta

```
mundo_para_tela(camera: 'Camera | None', coordenada: 'Any', *, viewport: 'Viewport |
  None' = None) -> 'tuple[float, float]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
camera	Camera | None	sim	sem padrão	Camera obrigatória para a transformação.
coordenada	Any	sim	sem padrão	Valor espacial 2D aceito pela câmera, como posicao.
viewport	Viewport | None	não	None	Viewport opcional para converter tela lógica em física.

Retorno

Par de coordenadas de tela, físicas quando viewport é informado.

Exemplo

Código Coral

```
de coral.jogos importe mundo_para_tela
de coral.jogos importe Camera, Viewport
defina camera como Camera(x=10, y=20, largura=100, altura=50)
defina viewport como Viewport(100, 50, 200, 100)
mostre mundo_para_tela(camera, posicao(10, 20), viewport=viewport)
```

Resultado

Saída esperada

```
(0.0, 0.0)
```

Erros

ErroMapaCoral quando falta câmera ou a coordenada não é 2D; erros de coordenadas e câmera são propagados.

Atenção

Aplica a câmera primeiro e o viewport depois. Não altera a posição original. Sem viewport, o resultado é lógico; valores fora da área útil continuam possíveis.

`tela_para_mundo(...)` · módulo `coral.jogos`

O que faz

Converte coordenadas de tela em posição espacial 2D do mundo.

Assinatura

Assinatura para consulta

```
tela_para_mundo(camera: 'Camera | None', x: 'float', y: 'float', *, viewport: 'Viewport
  | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
camera	Camera | None	sim	sem padrão	Camera exigida para transformação inversa.
x	float	sim	sem padrão	Coordenada horizontal da tela.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
y	float	sim	sem padrão	Coordenada vertical da tela.
viewport	Viewport | None	não	None	Viewport opcional quando a entrada é física.

Retorno

Valor espacial posicao com duas dimensões.

Exemplo

Código Coral

```
de coral.jogos importe tela_para_mundo
de coral.jogos importe Camera, Viewport
defina camera como Camera(x=10, y=20, largura=100, altura=50)
defina viewport como Viewport(100, 50, 200, 100)
defina lugar como tela_para_mundo(camera, 0, 0, viewport=viewport)
mostre lugar.x
mostre lugar.y
```

Resultado

Saída esperada

```
10.0
20.0
```

Erros

ErroMapaCoral para ausência de câmera; erros numéricos e da transformação são propagados.

Atenção

Com viewport, converte coordenadas físicas em lógicas antes de aplicar a câmera. Não recorta automaticamente pontos fora da área útil. Não altera câmera nem mundo.

`celula_para_tela(...)` · módulo `coral.jogos`

O que faz

Projeta uma célula de mapa 2D usando sua topologia e uma câmera.

Assinatura

Assinatura para consulta

```
celula_para_tela(mapa: 'Mapa', camera: 'Camera | None', celula: 'Any', *, ancora: 'str'
  = 'centro', viewport: 'Viewport | None' = None) -> 'tuple[float, float]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mapa	Mapa	sim	sem padrão	Mapa 2D com topologia espacial.
camera	Camera | None	sim	sem padrão	Camera exigida.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
celula	Any	sim	sem padrão	Coordenada de célula aceita pela topologia.
ancora	str	não	'centro'	Ponto da célula a projetar; padrão centro.
viewport	Viewport | None	não	None	Viewport opcional para resultado físico.

Retorno

Par de coordenadas de tela da âncora escolhida.

Exemplo

Código Coral

```
de coral.jogos importe celula_para_tela
de coral.mundo importe Mapa
de coral.mundo importe TopologiaQuadrada
de coral.jogos importe Camera
defina mundo como Mapa("grade", TopologiaQuadrada(largura_celula=10, altura_celula=10))
defina camera como Camera()
mostre celula_para_tela(mundo, camera, (0, 0))
```

Resultado

Saída esperada

```
(5.0, 5.0)
```

Erros

ErroMapaCoral para dimensão não suportada ou câmera ausente; erros de célula, âncora e política de bordas são propagados.

Atenção

Primeiro transforma célula em mundo, depois aplica câmera e viewport. Não suporta mapa voxel ou outra dimensão. A topologia define as âncoras aceitas; topologia hexagonal aceita centro.

`tela_para_celula(...)` · módulo `coral.jogos`

O que faz

Localiza a célula correspondente a um ponto da tela em mapa 2D.

Assinatura

Assinatura para consulta

```
tela_para_celula(mapa: 'Mapa', camera: 'Camera | None', x: 'float', y: 'float', *,
  viewport: 'Viewport | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
mapa	Mapa	sim	sem padrão	Mapa 2D com topologia espacial.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
camera	Camera | None	sim	sem padrão	Camera exigida.
x	float	sim	sem padrão	Coordenada horizontal de tela.
y	float	sim	sem padrão	Coordenada vertical de tela.
viewport	Viewport | None	não	None	Viewport opcional quando a entrada é física.

Retorno

Valor espacial de célula produzido pela topologia.

Exemplo

Código Coral

```
de coral.jogos importe tela_para_celula
de coral.mundo importe Mapa
de coral.mundo importe TopologiaQuadrada
de coral.jogos importe Camera
defina mundo como Mapa("grade", TopologiaQuadrada(largura_celula=10, altura_celula=10))
defina camera como Camera()
defina alvo como tela_para_celula(mundo, camera, 5, 5)
mostre alvo.como_tupla()
```

Resultado

Saída esperada

```
(0, 0)
```

Erros

ErroMapaCoral para dimensão não suportada ou câmera ausente; erros de transformação e topologia são propagados.

Atenção

Converte tela em mundo e consulta a topologia. Não muda ocupantes nem cria células. Limites e política de bordas pertencem ao mapa; mapas voxel não são suportados.

`posicao_espacial_sprite(...)` · módulo `coral.jogos`

O que faz

Expõe x e y de um sprite como posição espacial 2D.

Assinatura

Assinatura para consulta

```
posicao_espacial_sprite(sprite: 'Any')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
sprite	Any	sim	sem padrão	Objeto com x, y e forma_colisao compatíveis com Sprite.

Retorno

Valor posicao com duas dimensões.

Exemplo

Código Coral

```
de coral.jogos importe posicao_espacial_sprite
de coral.jogos importe Sprite
defina sprite como Sprite(3, 4, 10, 20)
defina lugar como posicao_espacial_sprite(sprite)
mostre lugar.x
mostre lugar.y
```

Resultado

Saída esperada

```
3.0
4.0
```

Erros

TypeError para objeto sem os campos exigidos; erros de posição espacial são propagados.

Atenção

Usa os campos x e y, e não calcula o centro visual ou do colisor. O valor espacial é independente do sprite; não cria vínculo de atualização.

`forma_espacial_sprite(...)` · módulo `coral.jogos`

O que faz

Converte os vértices da forma de colisão do sprite em polígono espacial.

Assinatura

Assinatura para consulta

```
forma_espacial_sprite(sprite: 'Any') -> 'Poligono'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
sprite	Any	sim	sem padrão	Objeto compatível com Sprite e forma_colisao com vértices.

Retorno

Poligono espacial a partir dos vértices do colisor.

Exemplo

Código Coral

```
de coral.jogos importe forma_espacial_sprite
de coral.jogos importe Sprite
defina sprite como Sprite(3, 4, 10, 20)
defina forma como forma_espacial_sprite(sprite)
mostre len(forma.vertices)
```

Resultado

Saída esperada

```
4
```

Erros

TypeError para sprite incompatível ou forma sem vértices; erros de construção do polígono são propagados.

Atenção

Usa a forma de colisão atual, que pode diferir da aparência conforme as opções de escala e rotação do sprite. Não move o sprite nem altera o colisor.

`sprites_colidem(...)` · módulo `coral.jogos`

O que faz

Testa sobreposição espacial entre as formas de colisão de dois sprites.

Assinatura

Assinatura para consulta

```
sprites_colidem(a: 'Any', b: 'Any') -> 'bool'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	Any	sim	sem padrão	Primeiro sprite compatível.
b	Any	sim	sem padrão	Segundo sprite compatível.

Retorno

Booleano de sobreposição dos interiores das formas.

Exemplo

Código Coral

```
de coral.jogos importe sprites_colidem
de coral.jogos importe Sprite
mostre sprites_colidem(Sprite(0, 0, 10, 10), Sprite(5, 0, 10, 10))
mostre sprites_colidem(Sprite(0, 0, 10, 10), Sprite(10, 0, 10, 10))
```

Resultado

Saída esperada

```
True
False
```

Erros

TypeError para objetos incompatíveis; erros da conversão e operação espacial são propagados.

Atenção

Tocar uma borda não conta como sobreposição. Usa as opções atuais da forma de colisão; não usa pixels da imagem, não resolve colisão nem altera velocidade.

`formas_colidem(...)` · módulo `coral.jogos`

O que faz

Testa colisão estrita entre formas transformadas convexas usando eixos separadores.

Assinatura

Assinatura para consulta

```
formas_colidem(a: 'FormaTransformada', b: 'FormaTransformada') -> 'bool'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
a	FormaTransformada	sim	sem padrão	Primeira FormaTransformada de polígono convexo válido.
b	FormaTransformada	sim	sem padrão	Segunda FormaTransformada de polígono convexo válido.

Retorno

Booleano de sobreposição estrita.

Exemplo

Código Coral

```
de coral.jogos importe formas_colidem
de coral.jogos importe FormaTransformada
defina a como FormaTransformada(((0, 0), (10, 0), (10, 10), (0, 10)))
defina b como FormaTransformada(((5, 0), (15, 0), (15, 10), (5, 10)))
mostre formas_colidem(a, b)
```

Resultado

Saída esperada

```
True
```

Erros

AttributeError ou TypeError para formas incompatíveis; a validade geométrica deve ser garantida pelo programa.

Atenção

Pressupõe convexidade e polígonos não degenerados; o construtor da forma não comprova essas propriedades. Contato apenas em borda não conta. Não altera os vértices nem calcula resposta física.

`sprites_de_pontos(...)` · módulo `coral.jogos`

O que faz

Adapta uma coleção de pontos 2D para novos sprites.

Assinatura

Assinatura para consulta

```
sprites_de_pontos(pontos: 'Iterable[Sequence[float]]', *, largura: 'float' = 16.0,
  altura: 'float' = 16.0, imagem: 'Any' = None, cor: 'Cor' = Cor(vermelho=255, verde=255,
  azul=255, alfa=255), nome_base: 'str' = 'procedural', dados: 'Mapping[str, Any] | None'
  = None, campos: 'Mapping[str, CampoProcedural] | None' = None, fabrica: 'Callable[[int,
  float, float, dict[str, Any]], Sprite] | None' = None, limite_sprites: 'int' = 10000) ->
   'tuple[Sprite, ...]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
pontos	Iterable[Sequence[float]]	sim	sem padrão	Iterável de pares 2D finitos; posições iniciais dos sprites.
largura	float	não	16.0	Largura dos sprites padrão; 16.
altura	float	não	16.0	Altura dos sprites padrão; 16.
imagem	Any	não	None	Imagem opcional dos sprites padrão.
cor	Cor	não	Cor(vermelho=255, verde=255, azul=255, alfa=255)	Cor dos sprites padrão; branco.
nome_base	str	não	'procedural'	Prefixo para nomes seguido de índice iniciado em zero.
dados	Mapping[str, Any] | None	não	None	Mapeamento base copiado profundamente para cada sprite.
campos	Mapping[str, CampoProcedural] | None	não	None	Mapeamento de campos procedurais para anotar posições, ou nulo.
fabrica	Callable[[int, float, float, dict[str, Any]], Sprite] | None	não	None	Callback opcional com índice, x, y e dados; deve devolver Sprite.
limite_sprites	int	não	10000	Inteiro positivo máximo de sprites; padrão 10000.

Retorno

Tupla dos novos Sprite em ordem dos pontos.

Exemplo

Código Coral

```
de coral.jogos importe sprites_de_pontos
defina itens como sprites_de_pontos([(0, 0), (10, 5)], nome_base="arvore")
mostre len(itens)
mostre itens[1].nome
mostre itens[1].dados["procedural"]["indice"]
```

Resultado

Saída esperada

```
2
arvore_1
1
```

Erros

ValueError para limites, pontos ou campos inválidos; TypeError para fábrica que não devolve Sprite; erros de cópia, callbacks e construção são propagados.

Atenção

Materializa e valida todos os pontos antes de criar sprites. Cada sprite recebe metadados de origem. Não adiciona a cena nem abre janela. Com fabrica, as opções visuais são responsabilidade do callback. Falhas de callbacks não desfazem seus efeitos externos.

`sprites_de_receita(...)` · **módulo** `coral.jogos`

O que faz

Reconstrói uma receita procedural de pontos e cria sprites a partir dela.

Assinatura

Assinatura para consulta

```
sprites_de_receita(receita: 'ReceitaProcedural | Mapping[str, Any]', *, limite_custo:
  'int' = 1000000, **opcoes_sprite: 'Any') -> 'tuple[Sprite, ...]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
receita	ReceitaProcedural | Mapping[str, Any]	sim	sem padrão	ReceitaProcedural ou mapeamento portátil aceito pela reconstrução.
limite_custo	int	não	1000000	Inteiro positivo do orçamento de reconstrução; padrão 1000000.
opcoes_sprite	Any	não	sem padrão	Opções nomeadas de sprites_de_pontos, incluindo limite_sprites.

Retorno

Tupla de Sprite produzidos pelos pontos reconstruídos.

Exemplo

Código Coral

```
de coral.jogos importe sprites_de_receita
de coral.procedural importe ReceitaProcedural
defina receita como ReceitaProcedural("distribuir_pontos", semente_raiz=7,
  parametros={"largura": 10, "altura": 10, "quantidade": 2, "max_tentativas_por_ponto":
  10, "limite_tentativas_total": 20})
defina itens como sprites_de_receita(receita, largura=8, altura=8)
mostre len(itens)
mostre itens[0].largura
```

Resultado

Saída esperada

```
2
8
```

Erros

ValueError para receita de campo, produto incompatível ou limites; erros de reconstrução e de criação de sprites são propagados.

Atenção

Receitas de campo não produzem sprites e são recusadas. A semente controla a geração de pontos. Não adiciona sprites a cena. Os limites de custo da receita e de quantidade de sprites são independentes.

`popular_cena_procedural(...)` · **módulo** `coral.jogos`

O que faz

Cria sprites de pontos ou receita e os adiciona a uma cena existente.

Assinatura

Assinatura para consulta

```
popular_cena_procedural(cena: 'Cena', *, pontos: 'Iterable[Sequence[float]] | None' =
  None, receita: 'ReceitaProcedural | Mapping[str, Any] | None' = None, camada: 'Any' =
  None, limite_custo: 'int' = 1000000, **opcoes_sprite: 'Any') -> 'tuple[Sprite, ...]'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
cena	Cena	sim	sem padrão	Cena que receberá os sprites.
pontos	Iterable[Sequence[float]] | None	não	None	Iterável de pontos 2D, ou nulo ao usar receita.
receita	ReceitaProcedural | Mapping[str, Any] | None	não	None	Receita de pontos, ou nulo ao informar pontos.
camada	Any	não	None	Nome ou CamadaApresentacao da cena, ou nulo.
limite_custo	int	não	1000000	Orçamento de reconstrução quando receita é usada.
opcoes_sprite	Any	não	sem padrão	Opções nomeadas encaminhadas à criação dos sprites.

Retorno

Tupla dos novos sprites, também adicionados à cena.

Exemplo

Código Coral

```
de coral.jogos importe popular_cena_procedural
de coral.jogos importe Cena
defina cena como Cena("bosque")
defina itens como popular_cena_procedural(cena, pontos=[(1, 2), (3, 4)])
mostre len(itens)
mostre len(cena.sprites)
```

Resultado

Saída esperada

```
2
2
```

Erros

TypeError para cena incompatível; ValueError para origem ausente ou duplicada, camada ou limites; erros procedurais e de sprites são propagados.

Atenção

Exige exatamente uma origem: pontos ou receita. Cria os sprites antes de adicioná-los. A adição não é transacional: uma falha de camada pode deixar alterações já realizadas na cena. Não abre janela nem atualiza o laço de jogo.

`aplicar_campo_em_sprites(...)` · módulo `coral.jogos`

O que faz

Avalia um campo 2D e grava seu valor nos dados de cada sprite.

Assinatura

Assinatura para consulta

```
aplicar_campo_em_sprites(campo: 'CampoProcedural', sprites: 'Iterable[Sprite]', *,
  chave: 'str' = 'valor_procedural', usar_centro: 'bool' = False, limite_sprites: 'int' =
  10000) -> 'int'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
campo	CampoProcedural	sim	sem padrão	CampoProcedural consultado com x e y.
sprites	Iterable[Sprite]	sim	sem padrão	Iterável contendo apenas Sprite.
chave	str	não	'valor_procedural'	Chave não vazia dos dados; padrão valor_procedural.
usar_centro	bool	não	False	Se verdadeiro, soma metade da largura e altura às coordenadas.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
limite_sprites	int	não	10000	Inteiro positivo máximo de itens processados; padrão 10000.

Retorno

Inteiro da quantidade de sprites atualizados.

Exemplo

Código Coral

```
de coral.jogos importe aplicar_campo_em_sprites
de coral.procedural importe CampoProcedural
de coral.jogos importe Sprite
crie a função somar com x, y
    retorne x mais y
fim
defina campo como CampoProcedural(somar, dimensoes_min=2, dimensoes_max=2)
defina sprite como Sprite(3, 4, 10, 10)
mostre aplicar_campo_em_sprites(campo, [sprite], chave="peso")
mostre sprite.dados["peso"]
```

Resultado

Saída esperada

```
1
7.0
```

Erros

TypeError para campo ou item incompatível; ValueError para chave, limite, coordenadas, dimensão ou retorno inválidos; erros do callback são propagados.

Atenção

Substitui o valor existente da chave. usar_centro calcula centro com x, y, largura e altura, sem aplicar transformação visual. Processa e modifica um item por vez; limite ou falha posterior não desfaz os valores anteriores. O callback do campo pode produzir outros efeitos.

`gerar_overlay_debug(...)` · módulo `coral.jogos`

O que faz

Converte inspeção de sprite, cena ou jogo em primitivas neutras de depuração.

Assinatura

Assinatura para consulta

```
gerar_overlay_debug(objeto: 'Any', *, limites: 'bool' = True, colisores: 'bool' = True,
  camera: 'bool' = True, pivos: 'bool' = True, rotulos: 'bool' = False,
  incluir_invisiveis: 'bool' = False) -> 'OverlayDebug'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
objeto	Any	sim	sem padrão	Objeto inspecionável ou SnapshotInspecao.
limites	bool	não	True	Incluir limites visuais; padrão verdadeiro.
colisores	bool	não	True	Incluir formas de colisão; padrão verdadeiro.
camera	bool	não	True	Incluir geometria de câmera; padrão verdadeiro.
pivos	bool	não	True	Incluir pivôs; padrão verdadeiro.
rotulos	bool	não	False	Incluir rótulos; padrão falso.
incluir_invisiveis	bool	não	False	Incluir objetos invisíveis; padrão falso.

Retorno

OverlayDebug com primitivas e assinatura do snapshot.

Exemplo

Código Coral

```
de coral.jogos importe gerar_overlay_debug
de coral.jogos importe Sprite
defina sprite como Sprite(3, 4, 10, 20)
defina painel como gerar_overlay_debug(sprite, limites=verdadeiro, colisores=falso,
  pivos=falso, camera=falso)
mostre painel.quantidade
mostre len(painel.por_categoria("limites"))
```

Resultado

Saída esperada

```
1
1
```

Erros

Erros de inspeção e normalização do objeto são propagados.

Atenção

Não desenha nem exige Pygame. Objetos sem geometria reconhecida podem produzir overlay vazio. A inspeção de um objeto pode chamar seu contrato estado_inspecao; efeitos desse contrato são responsabilidade do objeto.

`gerar_overlay_metricas_debug(...)` · módulo `coral.jogos`

O que faz

Gera quatro linhas de texto de tela para métricas operacionais.

Assinatura

Assinatura para consulta

```
gerar_overlay_metricas_debug(metricas: 'Any', *, x: 'float' = 8.0, y: 'float' = 8.0,
  espacamento: 'float' = 18.0) -> 'OverlayDebug'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
metricas	Any	sim	sem padrão	Mapeamento ou objeto com estado_inspecao que devolva mapeamento.
x	float	não	8.0	Posição horizontal inicial; padrão oito.
y	float	não	8.0	Posição vertical inicial; padrão oito.
espacamento	float	não	18.0	Distância vertical entre linhas; padrão 18.

Retorno

OverlayDebug com textos FPS, duração, delta e quantidade de quadros.

Exemplo

Código Coral

```
de coral.jogos importe gerar_overlay_metricas_debug
de coral.jogos importe MetricasJogo
defina medida como MetricasJogo()
execute medida.registrar(0.02, 0.02)
defina painel como gerar_overlay_metricas_debug(medida)
mostre painel.quantidade
mostre painel.primitivas[0].dados["texto"]
```

Resultado

Saída esperada

```
4
FPS 50.0
```

Erros

TypeError para métricas fora de mapeamento; ValueError ou TypeError para conversões numéricas inválidas; erros do método de inspeção são propagados.

Atenção

As primitivas usam espaco tela, sem deslocamento de câmera. Chaves ausentes usam zero. Converte dados numéricos, mas não impõe orçamento nem valida integralmente finitude das posições. Não coleta medidas nem desenha.

`desenhar_overlay_debug(...)` · **módulo** `coral.jogos`

O que faz

Desenha primitivas de depuração usando a API pública do alvo.

Assinatura

Assinatura para consulta

```
desenhar_overlay_debug(alvo: 'Any', overlay_ou_objeto: 'Any', *, camera: 'Any | None' =
  None, estilo: 'EstiloOverlayDebug | None' = None, limites: 'bool' = True, colisores:
  'bool' = True, pivos: 'bool' = True, rotulos: 'bool' = False, incluir_invisiveis: 'bool'
   = False, mostrar_camera: 'bool' = True) -> 'OverlayDebug'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
alvo	Any	sim	sem padrão	Backend ou Jogo que oferece a API pública de desenho.
overlay_ou_objeto	Any	sim	sem padrão	OverlayDebug pronto ou objeto a inspecionar.
camera	Any | None	não	None	Câmera opcional para primitivas no espaço mundo.
estilo	EstiloOverlayDebug | None	não	None	EstiloOverlayDebug opcional.
limites	bool	não	True	Incluir limites visuais; padrão verdadeiro.
colisores	bool	não	True	Incluir formas de colisão; padrão verdadeiro.
pivos	bool	não	True	Incluir pivôs; padrão verdadeiro.
rotulos	bool	não	False	Incluir rótulos; padrão falso.
incluir_invisiveis	bool	não	False	Incluir objetos invisíveis; padrão falso.
mostrar_camera	bool	não	True	Incluir geometria de câmera quando o overlay é gerado; padrão verdadeiro.

Retorno

OverlayDebug usado no desenho.

Exemplo

Código Coral

```
de coral.jogos importe desenhar_overlay_debug
de coral.jogos importe BackendNulo, MetricasJogo, gerar_overlay_metricas_debug
defina backend como BackendNulo()
defina painel como gerar_overlay_metricas_debug(MetricasJogo())
defina desenhado como desenhar_overlay_debug(backend, painel)
mostre desenhado.quantidade
```

Resultado

Saída esperada

```
4
```

Erros

TypeError para alvo sem a API de desenho; erros de geração, estilo e backend são propagados.

Atenção

Produz chamadas de desenho no alvo. Com overlay pronto, as opções de geração não filtram suas primitivas. Primitivas de tela ignoram a câmera. Não abre janela automaticamente; use BackendNulo para execução sem interface. Falhas durante desenho não desfazem comandos anteriores.

`AcaoApresentacao(...)` · módulo `coral.jogos`

O que faz

Representa uma ação instantânea executada ao avançar a apresentação.

Construtor

Assinatura para consulta

```
AcaoApresentacao(acao: 'Callable[[], Any]', nome: 'str | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
acao	Callable[[], Any]	sim	sem padrão	Callback sem argumentos; seu retorno é ignorado.
nome	str | None	não	None	Nome opcional convertido para texto; espaços externos são removidos.

Atributos importantes

Atributo	Significado
`finalizado`	Se o callback terminou sem erro.
`progresso`	Zero antes da execução; um depois.

Métodos principais

Assinatura para consulta

```
avancar(self, dt: 'float') -> 'float'
```

Referência para leitura

```
reiniciar(self) -> 'None'
```

Retorno

Nova instância de AcaoApresentacao.

Exemplo

Código Coral

```
de coral.jogos importe AcaoApresentacao
crie a função avisar
    mostre "ação concluída"
fim
defina etapa como AcaoApresentacao(avisar)
mostre etapa.finalizado
mostre etapa.avancar(0.5)
mostre etapa.finalizado
```

Resultado

Saída esperada

```
False
ação concluída
0.5
True
```

Erros

TypeError para ação não chamável; ValueError para dt inválido; erros do callback são propagados.

Atenção

O construtor não executa o callback. avancar valida dt finito não negativo, executa uma vez e devolve todo o dt como sobra. reiniciar permite nova execução. Se o callback falhar, a etapa não é marcada como executada, mas seus efeitos anteriores não são desfeitos.

`executar(...)` · **módulo** `coral.jogos`

O que faz

Cria uma etapa instantânea a partir de uma função sem argumentos.

Assinatura

Assinatura para consulta

```
executar(acao: 'Callable[[], Any]', *, nome: 'str | None' = None) -> 'AcaoApresentacao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
acao	Callable[[], Any]	sim	sem padrão	Callback que será chamado ao avançar a etapa.
nome	str | None	não	None	Nome opcional convertido para texto; espaços externos são removidos.

Retorno

AcaoApresentacao ainda não executada.

Exemplo

Código Coral

```
de coral.jogos importe executar
crie a função avisar
    mostre "ação concluída"
fim
defina etapa como executar(avisar, nome="aviso")
mostre etapa.nome
execute etapa.avancar(0)
```

Resultado

Saída esperada

```
aviso
ação concluída
```

Erros

TypeError para ação não chamável; erros do callback surgem ao avançar.

Atenção

Não chama o callback durante a criação e não inicia processo do sistema. A ação é executada pela etapa ou composição. O retorno do callback é ignorado; reiniciar a etapa permite repetir seus efeitos.

`EsperaApresentacao(...)` · **módulo** `coral.jogos`

O que faz

Consome um intervalo de tempo lógico de apresentação.

Construtor

Assinatura para consulta

```
EsperaApresentacao(duracao: 'float', nome: 'str | None' = None, tempo: 'float' = 0.0) ->
   None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
duracao	float	sim	sem padrão	Duração finita não negativa em segundos.
nome	str | None	não	None	Nome opcional convertido para texto; espaços externos são removidos.
tempo	float	não	0.0	Tempo inicial finito não negativo, limitado à duração.

Atributos importantes

Atributo	Significado
`finalizado`	Se tempo atingiu duração.
`progresso`	Fração consumida; um quando duração é zero.

Métodos principais

Assinatura para consulta

```
avancar(self, dt: 'float') -> 'float'
```

Referência para leitura

```
reiniciar(self) -> 'None'
```

Retorno

Nova instância de EsperaApresentacao.

Exemplo

Código Coral

```
de coral.jogos importe EsperaApresentacao
defina etapa como EsperaApresentacao(0.5)
mostre etapa.avancar(0.75)
mostre etapa.finalizado
```

Resultado

Saída esperada

```
0.25
True
```

Erros

ValueError para duração, tempo ou dt negativos, booleanos ou não finitos.

Atenção

Não dorme nem consulta relógio. avancar consome apenas o tempo restante e devolve a sobra; uma etapa já concluída devolve todo o dt. Duração zero já está concluída. reiniciar zera tempo.

`esperar(...)` · módulo `coral.jogos`

O que faz

Cria uma etapa que espera por duração lógica explícita.

Assinatura

Assinatura para consulta

```
esperar(duracao: 'float', *, nome: 'str | None' = None) -> 'EsperaApresentacao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
duracao	float	sim	sem padrão	Segundos finitos não negativos a consumir.
nome	str | None	não	None	Nome opcional convertido para texto; espaços externos são removidos.

Retorno

EsperaApresentacao com tempo inicial zero.

Exemplo

Código Coral

```
de coral.jogos importe esperar
defina etapa como esperar(1)
execute etapa.avancar(0.25)
mostre etapa.progresso
```

Resultado

Saída esperada

```
0.25
```

Erros

ValueError para duração inválida.

Atenção

Não bloqueia a execução nem chama sleep. Precisa receber dt pela composição ou por avancar; um relógio real só participa quando ligado explicitamente.

`EsperaConclusao(...)` · módulo `coral.jogos`

O que faz

Observa a conclusão de outro objeto ou de uma condição.

Construtor

Assinatura para consulta

```
EsperaConclusao(alvo: 'Any', nome: 'str | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
alvo	Any	sim	sem padrão	Callback sem argumentos, ou objeto com finalizado ou finalizada.
nome	str | None	não	None	Nome opcional convertido para texto; espaços externos são removidos.

Atributos importantes

Atributo	Significado
`finalizado`	Estado observado no alvo.
`progresso`	Progresso do alvo limitado a zero e um, ou zero quando indisponível.

Métodos principais

Assinatura para consulta

```
avancar(self, dt: 'float') -> 'float'
```

Referência para leitura

```
reiniciar(self) -> 'None'
```

Retorno

Nova instância de EsperaConclusao.

Exemplo

Código Coral

```
de coral.jogos importe EsperaConclusao
de coral.jogos importe esperar
defina alvo como esperar(0.5)
defina etapa como EsperaConclusao(alvo)
mostre etapa.avancar(0.25)
execute alvo.avancar(0.5)
mostre etapa.avancar(0.25)
```

Resultado

Saída esperada

```
0.0
0.25
```

Erros

TypeError para alvo sem contrato de conclusão; ValueError para dt inválido; erros do callback ou propriedades são propagados.

Atenção

Não avança nem reinicia o alvo. Se ele for chamável, consulta seu retorno como booleano. finalizado e progresso podem consultar o callback repetidamente. Enquanto não concluído, avancar devolve zero; quando concluído, devolve todo o dt. Falta de atualização externa pode impedir conclusão indefinidamente.

`esperar_conclusao(...)` · módulo `coral.jogos`

O que faz

Cria uma etapa que espera a conclusão de um alvo atualizado externamente.

Assinatura

Assinatura para consulta

```
esperar_conclusao(alvo: 'Any', *, nome: 'str | None' = None) -> 'EsperaConclusao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
alvo	Any	sim	sem padrão	Callback de condição ou objeto com finalizado ou finalizada.
nome	str | None	não	None	Nome opcional convertido para texto; espaços externos são removidos.

Retorno

EsperaConclusao que observa o alvo.

Exemplo

Código Coral

```
de coral.jogos importe esperar_conclusao
de coral.jogos importe esperar
defina alvo como esperar(0)
defina etapa como esperar_conclusao(alvo)
mostre etapa.finalizado
```

Resultado

Saída esperada

```
True
```

Erros

TypeError para alvo incompatível; erros do alvo são propagados durante consulta.

Atenção

Não atualiza o alvo nem o reinicia. O programa continua responsável por produzir sua conclusão. Consultas podem chamar código do alvo; uma condição chamável tem seu retorno convertido para bool.

`SequenciaApresentacao(...)` · módulo `coral.jogos`

O que faz

Executa etapas em ordem, reaproveitando o tempo que sobra de cada uma.

Construtor

Assinatura para consulta

```
SequenciaApresentacao(etapas: 'tuple[NoApresentacao, ...]', nome: 'str | None' = None)
  -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
etapas	tuple[NoApresentacao, ...]	sim	sem padrão	Tupla não vazia de etapas com avancar, reiniciar, finalizado e progresso.
nome	str | None	não	None	Nome opcional convertido para texto; espaços externos são removidos.

Atributos importantes

Atributo	Significado
`etapa_atual`	Etapa ativa ou nulo quando concluída.
`finalizado`	Se todas as etapas foram percorridas.

Atributo	Significado
`progresso`	Fração das etapas concluídas e progresso da atual.

Métodos principais

Assinatura para consulta

```
avancar(self, dt: 'float') -> 'float'
```

Referência para leitura

```
reiniciar(self) -> 'None'
```

Retorno

Nova instância de SequenciaApresentacao.

Exemplo

Código Coral

```
de coral.jogos importe SequenciaApresentacao
de coral.jogos importe esperar
defina fluxo como SequenciaApresentacao((esperar(0.25), esperar(0.5)))
mostre fluxo.avancar(1)
mostre fluxo.finalizado
```

Resultado

Saída esperada

```
0.25
True
```

Erros

ValueError para coleção vazia ou dt inválido; TypeError para etapa sem contrato; erros das etapas são propagados.

Atenção

Compartilha os objetos das etapas; não os copia. avancar pode concluir várias etapas no mesmo passo, encaminhando sobras. progresso pondera cada etapa igualmente, e não por duração. reiniciar reinicia todas as etapas; callbacks e efeitos externos não são desfeitos.

`sequencia(...)` · módulo `coral.jogos`

O que faz

Agrupa etapas de apresentação em ordem sequencial.

Assinatura

Assinatura para consulta

```
sequencia(*etapas: 'NoApresentacao', nome: 'str | None' = None) ->
  'SequenciaApresentacao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
etapas	NoApresentacao	não	sem padrão	Etapas passadas como argumentos posicionais separados; exige ao menos uma.
nome	str | None	não	None	Nome opcional convertido para texto; espaços externos são removidos.

Retorno

SequenciaApresentacao com referências às etapas recebidas.

Exemplo

Código Coral

```
de coral.jogos importe sequencia
de coral.jogos importe esperar
defina fluxo como sequencia(esperar(0.25), esperar(0.5))
execute fluxo.avancar(0.25)
mostre fluxo.progresso
```

Resultado

Saída esperada

```
0.5
```

Erros

ValueError quando nenhuma etapa é fornecida; TypeError para etapa incompatível.

Atenção

Não avança durante criação. A sobra de tempo de uma etapa é encaminhada à seguinte. Evite compartilhar a mesma etapa mutável entre composições independentes.

`ParaleloApresentacao(...)` · módulo `coral.jogos`

O que faz

Avança ramos de apresentação com o mesmo intervalo lógico.

Construtor

Assinatura para consulta

```
ParaleloApresentacao(etapas: 'tuple[NoApresentacao, ...]', nome: 'str | None' = None) ->
    None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
etapas	tuple[NoApresentacao, ...]	sim	sem padrão	Tupla não vazia de etapas com avancar, reiniciar, finalizado e progresso.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str | None	não	None	Nome opcional convertido para texto; espaços externos são removidos.

Atributos importantes

Atributo	Significado
`finalizado`	Se todos os ramos terminaram.
`progresso`	Média dos progressos dos ramos.

Métodos principais

Assinatura para consulta

```
avancar(self, dt: 'float') -> 'float'
```

Referência para leitura

```
reiniciar(self) -> 'None'
```

Retorno

Nova instância de ParaleloApresentacao.

Exemplo

Código Coral

```
de coral.jogos importe ParaleloApresentacao
de coral.jogos importe esperar
defina fluxo como ParaleloApresentacao((esperar(0.25), esperar(0.5)))
mostre fluxo.avancar(0.75)
mostre fluxo.finalizado
```

Resultado

Saída esperada

```
0.25
True
```

Erros

ValueError para ramos vazios ou dt inválido; TypeError para etapa sem contrato; erros das etapas são propagados.

Atenção

Não cria threads: avança ramos em ordem no mesmo processo. O ramo mais lento determina a sobra global. progresso é a média das frações dos ramos, sem ponderar duração. Compartilha etapas e seus efeitos; reiniciar não desfaz efeitos externos.

`paralelo(...)` · módulo `coral.jogos`

O que faz

Agrupa etapas que recebem o mesmo dt em cada avanço.

Assinatura

Assinatura para consulta

```
paralelo(*etapas: 'NoApresentacao', nome: 'str | None' = None) -> 'ParaleloApresentacao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
etapas	NoApresentacao	não	sem padrão	Etapas posicionais separadas; exige ao menos uma.
nome	str | None	não	None	Nome opcional convertido para texto; espaços externos são removidos.

Retorno

ParaleloApresentacao com os ramos fornecidos.

Exemplo

Código Coral

```
de coral.jogos importe paralelo
de coral.jogos importe esperar
defina fluxo como paralelo(esperar(0.25), esperar(0.5))
execute fluxo.avancar(0.25)
mostre fluxo.progresso
```

Resultado

Saída esperada

```
0.75
```

Erros

ValueError para ausência de etapas; TypeError para etapa incompatível.

Atenção

Paralelismo lógico de apresentação, sem threads ou tarefas assíncronas. Não avança durante a criação. Uma falha de ramo pode ocorrer depois de outros ramos já terem avançado.

`ComposicaoTemporal(...)` · **módulo** `coral.jogos`

O que faz

Controla uma árvore de apresentação com pausa, cancelamento e repetição.

Construtor

Assinatura para consulta

```
ComposicaoTemporal(raiz: 'NoApresentacao', repeticoes: 'int | None' = 1, nome: 'str |
  None' = None, estado: 'str' = 'ativo') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
raiz	NoApresentacao	sim	sem padrão	Etapa ou árvore com contrato de apresentação.
repeticoes	int | None	não	1	Quantidade positiva de ciclos; nulo repete sem limite. Use inteiro, padrão um.
nome	str | None	não	None	Nome opcional convertido para texto; espaços externos são removidos.
estado	str	não	'ativo'	ativo, pausado, cancelado ou finalizado; padrão ativo.

Atributos importantes

Atributo	Significado
`finalizado`	Se estado é finalizado.
`cancelado`	Se estado é cancelado.
`ciclos_concluidos`	Quantidade de ciclos terminados.
`progresso`	Fração global ou do ciclo atual quando repetição ilimitada.
`etapa_atual`	Nome da etapa atual quando disponível.

Métodos principais

Assinatura para consulta

```
avancar(self, dt: 'float') -> "'ComposicaoTemporal'"
```

Referência para leitura

```
pausar(self) -> "'ComposicaoTemporal'"
```

Referência para leitura

```
retomar(self) -> "'ComposicaoTemporal'"
```

Referência para leitura

```
cancelar(self) -> "'ComposicaoTemporal'"
```

Referência para leitura

```
reiniciar(self) -> "'ComposicaoTemporal'"
```

Retorno

Nova instância de ComposicaoTemporal.

Exemplo

Código Coral

```
de coral.jogos importe ComposicaoTemporal
de coral.jogos importe esperar
defina fluxo como ComposicaoTemporal(esperar(0.25), repeticoes=2)
execute fluxo.avancar(0.5)
mostre fluxo.ciclos_concluidos
mostre fluxo.finalizado
```

Resultado

Saída esperada

```
2
True
```

Erros

TypeError para raiz incompatível; ValueError para repetições, estado ou dt inválidos; erros das etapas são propagados.

Atenção

O construtor não avança nem reinicia a raiz. avancar recebe dt finito não negativo e devolve a própria composição, não a sobra. Cancelado é distinto de finalizado. reiniciar reinicia a raiz e zera ciclos, sem desfazer efeitos externos. Para repetição ilimitada, cada ciclo precisa consumir tempo: uma árvore puramente instantânea com dt positivo pode impedir que o avanço termine. O construtor converte repeticoes com int; forneça inteiro exato.

`VinculoTempoApresentacao(...)` · **módulo** `coral.jogos`

O que faz

Avança uma composição com diferenças entre instantes de uma fonte explícita.

Construtor

Assinatura para consulta

```
VinculoTempoApresentacao(composicao: 'ComposicaoTemporal', fonte: 'Any') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
composicao	ComposicaoTemporal	sim	sem padrão	ComposicaoTemporal que receberá os intervalos observados.
fonte	Any	sim	sem padrão	Objeto com agora() que devolva instantes finitos sem retroceder.

Métodos principais

Assinatura para consulta

```
atualizar(self) -> 'float'
```

Referência para leitura

```
ressincronizar(self) -> "'VinculoTempoApresentacao'"
```

Retorno

Nova instância de VinculoTempoApresentacao.

Exemplo

Código Coral

```
de coral.jogos importe VinculoTempoApresentacao
de coral.jogos importe ComposicaoTemporal, esperar
de coral.tempo_eventos importe RelogioSimulado
defina fluxo como ComposicaoTemporal(esperar(1))
defina relogio como RelogioSimulado()
defina vinculo como VinculoTempoApresentacao(fluxo, relogio)
execute relogio.avancar(0.25)
mostre vinculo.atualizar()
mostre fluxo.progresso
```

Resultado

Saída esperada

```
0.25
0.25
```

Erros

TypeError para fonte sem agora chamável; ValueError para instante inválido ou retrocesso; erros da fonte e composição são propagados.

Atenção

Consulta agora no construtor para guardar o instante inicial. atualizar calcula a diferença, atualiza sua referência e avança a composição; não chama passo nem avança a fonte. ressincronizar descarta o intervalo acumulado. Se a composição falhar, o instante de referência já foi atualizado. Uma pausa não impede a leitura do relógio.

`usar_fonte_tempo(...)` · módulo `coral.jogos`

O que faz

Cria um vínculo entre composição de apresentação e fonte de tempo.

Assinatura

Assinatura para consulta

```
usar_fonte_tempo(composicao: 'ComposicaoTemporal', fonte: 'Any') ->
  'VinculoTempoApresentacao'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
composicao	ComposicaoTemporal	sim	sem padrão	ComposicaoTemporal que receberá os intervalos observados.
fonte	Any	sim	sem padrão	Objeto com agora() que devolva instantes finitos sem retroceder.

Retorno

VinculoTempoApresentacao que precisa ser atualizado explicitamente.

Exemplo

Código Coral

```
de coral.jogos importe usar_fonte_tempo
de coral.jogos importe ComposicaoTemporal, esperar
de coral.tempo_eventos importe RelogioSimulado
defina fluxo como ComposicaoTemporal(esperar(1))
defina relogio como RelogioSimulado()
defina vinculo como usar_fonte_tempo(fluxo, relogio)
execute relogio.avancar(0.5)
mostre vinculo.atualizar()
mostre relogio.agora()
```

Resultado

Saída esperada

```
0.5
0.5
```

Erros

TypeError para fonte sem agora(); ValueError para instante inicial inválido; erros da fonte são propagados.

Atenção

Consulta agora no construtor para guardar o instante inicial. atualizar calcula a diferença, atualiza sua referência e avança a composição; não chama passo nem avança a fonte. ressincronizar descarta o intervalo acumulado. Se a composição falhar, o instante de referência já foi atualizado. Uma pausa não impede a leitura do relógio. Não instala atualização automática em um jogo.

`VinculoComposicaoEventos(...)` · módulo `coral.jogos`

O que faz

Controla uma composição ao receber um evento do barramento.

Construtor

Assinatura para consulta

```
VinculoComposicaoEventos(composicao: 'ComposicaoTemporal', eventos: 'Any', evento:
  'str', modo: 'str' = 'reiniciar') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
composicao	ComposicaoTemporal	sim	sem padrão	Composição a controlar.
eventos	Any	sim	sem padrão	Barramento com quando(evento, callback) e remover(evento, callback).
evento	str	sim	sem padrão	Nome de evento não vazio, com espaços externos removidos.
modo	str	não	'reiniciar'	reiniciar, retomar, pausar ou cancelar; padrão reiniciar.

Atributos importantes

Atributo	Significado
`ativo`	Se o vínculo ainda recebe eventos.

Métodos principais

Assinatura para consulta

```
desligar(self) -> 'None'
```

Retorno

Nova instância de VinculoComposicaoEventos.

Exemplo

Código Coral

```
de coral.jogos importe VinculoComposicaoEventos
de coral.jogos importe ComposicaoTemporal, esperar
de coral.tempo_eventos importe Eventos
defina fluxo como ComposicaoTemporal(esperar(1), estado="pausado")
defina eventos como Eventos()
defina vinculo como VinculoComposicaoEventos(fluxo, eventos, "iniciar")
execute eventos.emitir("iniciar")
mostre fluxo.estado
execute vinculo.desligar()
mostre vinculo.ativo
```

Resultado

Saída esperada

```
ativo
False
```

Erros

TypeError para barramento sem contrato; ValueError para evento ou modo inválido; erros de registro, remoção e composição são propagados.

Atenção

O construtor registra um ouvinte imediatamente. O evento chama o controle escolhido, sem avançar tempo e ignorando seu payload. desligar remove o ouvinte e pode ser repetido; o vínculo não oferece método de religamento. Não controla o ciclo de vida do barramento.

`ligar_evento(...)` · módulo `coral.jogos`

O que faz

Registra controle de uma composição por um evento explícito.

Assinatura

Assinatura para consulta

```
ligar_evento(composicao: 'ComposicaoTemporal', eventos: 'Any', evento: 'str', *, modo:
  'str' = 'reiniciar') -> 'VinculoComposicaoEventos'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
composicao	ComposicaoTemporal	sim	sem padrão	Composição a controlar.
eventos	Any	sim	sem padrão	Barramento com quando(evento, callback) e remover(evento, callback).
evento	str	sim	sem padrão	Nome de evento não vazio, com espaços externos removidos.
modo	str	não	'reiniciar'	reiniciar, retomar, pausar ou cancelar; padrão reiniciar.

Retorno

VinculoComposicaoEventos com ouvinte já registrado.

Exemplo

Código Coral

```
de coral.jogos importe ligar_evento
de coral.jogos importe ComposicaoTemporal, esperar
de coral.tempo_eventos importe Eventos
defina fluxo como ComposicaoTemporal(esperar(1), estado="pausado")
defina eventos como Eventos()
defina vinculo como ligar_evento(fluxo, eventos, "iniciar", modo="retomar")
execute eventos.emitir("iniciar")
mostre fluxo.estado
execute vinculo.desligar()
```

Resultado

Saída esperada

```
ativo
```

Erros

TypeError para barramento incompatível; ValueError para nome ou modo inválido; erros do barramento são propagados.

Atenção

O construtor registra um ouvinte imediatamente. O evento chama o controle escolhido, sem avançar tempo e ignorando seu payload. desligar remove o ouvinte e pode ser repetido; o vínculo não oferece método de religamento. Não controla o ciclo de vida do barramento.

`ComandoComposicao(...)` · módulo `coral.jogos`

O que faz

Representa um comando validado de reprodução de apresentação.

Construtor

Assinatura para consulta

```
ComandoComposicao(operacao: 'str', valor: 'float | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
operacao	str	sim	sem padrão	avancar, pausar, retomar, cancelar ou reiniciar; ignora caixa e espaços externos.
valor	float | None	não	None	dt finito não negativo exigido para avancar; nulo nas demais operações.

Retorno

Nova instância de ComandoComposicao.

Exemplo

Código Coral

```
de coral.jogos importe ComandoComposicao
defina comando como ComandoComposicao(" AVANCAR ", 0.25)
mostre comando.operacao
mostre comando.valor
```

Resultado

Saída esperada

```
avancar
0.25
```

Erros

ValueError para operação desconhecida, valor ausente, indevido ou dt inválido.

Atenção

Registro imutável. Construir o comando não o executa nem grava arquivo. Apenas avancar aceita valor; as demais operações exigem nulo. O registro guarda dados, não o callback das etapas.

`GravadorComposicao(...)` · módulo `coral.jogos`

O que faz

Registra comandos explícitos e os aplica a uma composição para permitir replay.

Construtor

Assinatura para consulta

```
GravadorComposicao(composicao: 'ComposicaoTemporal', comandos: 'list[ComandoComposicao]'
   = <factory>) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
composicao	ComposicaoTemporal	sim	sem padrão	Composição controlada pelo gravador.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
comandos	list[ComandoComposicao]	não	<factory>	Lista inicial de ComandoComposicao; padrão nova lista vazia.

Métodos principais

Assinatura para consulta

```
avancar(self, dt: 'float') -> "'GravadorComposicao'"
```

Referência para leitura

```
pausar(self) -> "'GravadorComposicao'"
```

Referência para leitura

```
retomar(self) -> "'GravadorComposicao'"
```

Referência para leitura

```
cancelar(self) -> "'GravadorComposicao'"
```

Referência para leitura

```
reiniciar(self) -> "'GravadorComposicao'"
```

Referência para leitura

```
reproduzir_em(self, composicao: 'ComposicaoTemporal') -> 'ComposicaoTemporal'
```

Retorno

Nova instância de GravadorComposicao.

Exemplo

Código Coral

```
de coral.jogos importe GravadorComposicao
de coral.jogos importe ComposicaoTemporal, esperar
defina original como ComposicaoTemporal(esperar(0.5))
defina gravador como GravadorComposicao(original)
execute gravador.avancar(0.25)
execute gravador.avancar(0.25)
defina copia como ComposicaoTemporal(esperar(0.5))
execute gravador.reproduzir_em(copia)
mostre len(gravador.comandos)
mostre copia.finalizado
```

Resultado

Saída esperada

```
2
True
```

Erros

ValueError para comando ou dt inválido; erros de composição, lista e callbacks são propagados.

Atenção

Registra o comando antes de aplicá-lo; falha de aplicação pode deixar o comando na lista. reproduzir_em reinicia o destino e reaplica os comandos, sem gravar novos. O determinismo depende de árvore equivalente, callbacks e estado externo. Não captura entradas externas, objetos ou callbacks nem oferece gravação automática em arquivo. A lista inicial não é revalidada integralmente.

`CanalAnimado(...)` · módulo `coral.jogos`

O que faz

Calcula uma propriedade oscilatória a partir de tempo e onda.

Construtor

Assinatura para consulta

```
CanalAnimado(canal: 'str', base: 'float', amplitude: 'float', frequencia: 'float', fase:
   'float' = 0.0, modo: 'str' = 'absoluto', onda: 'str | Callable[[float], float]' =
  'seno') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
canal	str	sim	sem padrão	x, y, escala, rotacao ou opacidade.
base	float	sim	sem padrão	Deslocamento base finito do valor.
amplitude	float	sim	sem padrão	Multiplicador finito da onda; pode ser negativo.
frequencia	float	sim	sem padrão	Ciclos por unidade de tempo; finita não negativa.
fase	float	não	0.0	Fase inicial finita, em ciclos; padrão zero.
modo	str	não	'absoluto'	absoluto ou relativo; relativo soma referencia.
onda	str | Callable[[float], float]	não	'seno'	Nome de onda registrado ou callback com fase que devolva número finito.

Métodos principais

Assinatura para consulta

```
valor(self, tempo: 'float', *, referencia: 'float' = 0.0) -> 'float'
```

Retorno

Nova instância de CanalAnimado.

Exemplo

Código Coral

```
de coral.jogos importe CanalAnimado
defina canal como CanalAnimado("x", 10, 2, 0, modo="relativo")
mostre canal.valor(0, referencia=5)
```

Resultado

Saída esperada

```
15.0
```

Erros

ValueError para canal, modo, frequência, onda ou número inválido; erros do callback e cálculo são propagados.

Atenção

valor calcula base mais amplitude vezes onda da fase, somando referência no modo relativo. O canal não avança relógio nem aplica o resultado a um objeto. Campos são mutáveis; alterações diretas não repetem toda validação do construtor. Tempo de consulta deve ser numérico compatível.

`AnimacaoTransformacao(...)` · módulo `coral.jogos`

O que faz

Mantém o tempo compartilhado de canais oscilatórios de transformação.

Construtor

Assinatura para consulta

```
AnimacaoTransformacao(canais: 'tuple[CanalAnimado, ...]', tempo: 'float' = 0.0, estado:
  'str' = 'ativo') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
canais	tuple[CanalAnimado, ...]	sim	sem padrão	Coleção não vazia de CanalAnimado, normalizada para tupla.
tempo	float	não	0.0	Tempo inicial finito não negativo; padrão zero.
estado	str	não	'ativo'	ativo, pausado ou encerrado.

Métodos principais

Assinatura para consulta

```
avancar(self, dt: 'float') -> 'None'
```

Referência para leitura

```
pausar(self) -> 'None'
```

Referência para leitura

```
retomar(self) -> 'None'
```

Referência para leitura

```
reiniciar(self) -> 'None'
```

Referência para leitura

```
encerrar(self) -> 'None'
```

Retorno

Nova instância de AnimacaoTransformacao.

Exemplo

Código Coral

```
de coral.jogos importe AnimacaoTransformacao
de coral.jogos importe CanalAnimado
defina animacao como AnimacaoTransformacao((CanalAnimado("x", 0, 1, 1),))
execute animacao.avancar(0.25)
mostre animacao.tempo
execute animacao.pausar()
execute animacao.avancar(0.25)
mostre animacao.tempo
```

Resultado

Saída esperada

```
0.25
0.25
```

Erros

ValueError para coleção vazia, tempo, estado ou dt inválidos; erros de canais incompatíveis podem aparecer no consumidor.

Atenção

avancar só acumula dt quando ativo; não aplica valores a sprite por conta própria. Não valida integralmente os tipos dos canais no construtor. reiniciar zera tempo e reativa se encerrado, mas preserva estado pausado. Campos e canais são mutáveis.

`QuadroChave(...)` · módulo `coral.jogos`

O que faz

Associa um valor numérico a um instante de faixa de animação.

Construtor

Assinatura para consulta

```
QuadroChave(instante: 'float', valor: 'float', curva: 'str | Callable[[float], float]' =
   'linear') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
instante	float	sim	sem padrão	Instante finito não negativo.
valor	float	sim	sem padrão	Valor finito da propriedade nesse instante.
curva	str | Callable[[float], float]	não	'linear'	Nome ou callback de curva temporal do trecho que sai deste quadro.

Retorno

Nova instância de QuadroChave.

Exemplo

Código Coral

```
de coral.jogos importe QuadroChave
defina quadro como QuadroChave(0, 10, curva="linear")
mostre quadro.instante
mostre quadro.valor
```

Resultado

Saída esperada

```
0.0
10.0
```

Erros

ValueError para instante, valor ou curva inválidos; erros de callbacks surgem durante interpolação.

Atenção

Registro imutável. A curva é usada para interpolar deste quadro para o próximo, não para chegar a este quadro. O construtor valida a identificação da curva, mas não executa callback personalizado. Não cria clipe nem altera objeto.

`FaixaAnimacao(...)` · módulo `coral.jogos`

O que faz

Interpola valores entre quadros chave de uma propriedade nomeada.

Construtor

Assinatura para consulta

```
FaixaAnimacao(nome: 'str', quadros: 'tuple[QuadroChave, ...]') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome não vazio e sem espaços externos da faixa.
quadros	tuple[QuadroChave, ...]	sim	sem padrão	Coleção não vazia de QuadroChave em ordem crescente, sem instantes repetidos.

Atributos importantes

Atributo	Significado
`duracao`	Instante do último quadro.

Métodos principais

Assinatura para consulta

```
valor_em(self, instante: 'float') -> 'float'
```

Retorno

Nova instância de FaixaAnimacao.

Exemplo

Código Coral

```
de coral.jogos importe FaixaAnimacao
de coral.jogos importe QuadroChave
defina quadros como (QuadroChave(0, 0), QuadroChave(1, 10))
defina faixa como FaixaAnimacao("x", quadros)
mostre faixa.duracao
mostre faixa.valor_em(0.5)
```

Resultado

Saída esperada

```
1.0
5.0
```

Erros

ValueError para nome vazio, coleção vazia, ordem, repetição ou consulta inválidos; TypeError para item que não é QuadroChave; erros da curva são propagados.

Atenção

Registro imutável que normaliza quadros para tupla. Não ordena automaticamente entradas. Consulta antes do primeiro instante devolve seu valor; depois do último, devolve o último. duracao é o instante do último quadro, e não a diferença entre primeiro e último. Não aplica resultado a objeto.

`ClipeAnimacao(...)` · módulo `coral.jogos`

O que faz

Reproduz uma ou mais faixas numéricas com tempo explícito.

Construtor

Assinatura para consulta

```
ClipeAnimacao(faixas: 'tuple[FaixaAnimacao, ...]', velocidade: 'float' = 1.0, modo:
  'str' = 'normal', repetir: 'bool' = False, tempo: 'float' = 0.0, estado: 'str' =
  'ativo') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
faixas	tuple[FaixaAnimacao, ...]	sim	sem padrão	Coleção não vazia de FaixaAnimacao com nomes únicos.
velocidade	float	não	1.0	Multiplicador finito positivo do dt; padrão um.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
modo	str	não	'normal'	normal, reverso ou ida_volta; também aceita aliases de ida e volta.
repetir	bool	não	False	Se repete ao atingir extremidade; padrão falso.
tempo	float	não	0.0	Tempo inicial finito não negativo, recortado à duração.
estado	str	não	'ativo'	ativo, pausado ou finalizado.

Atributos importantes

Atributo	Significado
`duracao`	Duração máxima das faixas.
`progresso`	Tempo atual dividido pela duração, ou um quando duração é zero.
`finalizado`	Se estado é finalizado.

Métodos principais

Assinatura para consulta

```
valor(self, faixa: 'str') -> 'float'
```

Referência para leitura

```
valores(self) -> 'dict[str, float]'
```

Referência para leitura

```
avancar(self, dt: 'float') -> 'dict[str, float]'
```

Referência para leitura

```
buscar(self, instante: 'float') -> "'ClipeAnimacao'"
```

Referência para leitura

```
pausar(self) -> "'ClipeAnimacao'"
```

Referência para leitura

```
retomar(self) -> "'ClipeAnimacao'"
```

Referência para leitura

```
reiniciar(self) -> "'ClipeAnimacao'"
```

Retorno

Nova instância de ClipeAnimacao.

Exemplo

Código Coral

```
de coral.jogos importe ClipeAnimacao
de coral.jogos importe QuadroChave, FaixaAnimacao
defina faixa como FaixaAnimacao("x", (QuadroChave(0, 0), QuadroChave(1, 10)))
defina clipe como ClipeAnimacao((faixa,))
mostre clipe.avancar(0.5)["x"]
mostre clipe.progresso
```

Resultado

Saída esperada

```
5.0
0.5
```

Erros

TypeError para faixa de tipo errado; ValueError para faixas vazias, nomes repetidos, velocidade, modo, tempo, estado ou dt inválidos; KeyError para faixa ausente; erros de interpolação são propagados.

Atenção

Não desenha nem altera alvo; avancar devolve valores das faixas. Duração é o máximo das durações das faixas. Em reverso, tempo inicial zero começa no fim. buscar recorta tempo, mas não reativa clipe finalizado. Em ida_volta, progresso pode diminuir na volta e não representa fração global do percurso. Um dt zero só consulta os valores.

`VinculoClipe(...)` · **módulo** `coral.jogos`

O que faz

Liga os valores de faixas às propriedades animáveis de um alvo.

Construtor

Assinatura para consulta

```
VinculoClipe(alvo: 'Any', clipe: 'ClipeAnimacao', mapeamento: 'dict[str, str]', modo:
  'str' = 'absoluto', restaurar_ao_final: 'bool' = False, composicao: 'str' =
  'substituir') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
alvo	Any	sim	sem padrão	Objeto com adaptadores das propriedades e valores iniciais finitos.
clipe	ClipeAnimacao	sim	sem padrão	ClipeAnimacao a reproduzir.
mapeamento	dict[str, str]	sim	sem padrão	Dicionário não vazio de nomes das faixas para propriedades do alvo.
modo	str	não	'absoluto'	absoluto usa o valor da faixa; relativo soma o valor inicial da propriedade.
restaurar_ao_final	bool	não	False	Se restaura valores iniciais quando o clipe finaliza; padrão falso.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
composicao	str	não	'substituir'	substituir escreve o valor; aditiva aplica diferença em relação ao último valor calculado.

Atributos importantes

Atributo	Significado
`propriedades`	Tupla das propriedades mapeadas sem repetição.
`finalizado`	Clipe terminado e restauração concluída quando exigida.

Métodos principais

Assinatura para consulta

```
atualizar(self, dt: 'float') -> 'dict[str, float]'
```

Retorno

Nova instância de VinculoClipe.

Exemplo

Código Coral

```
de coral.jogos importe VinculoClipe
de coral.jogos importe QuadroChave, FaixaAnimacao
defina faixa como FaixaAnimacao("x", (QuadroChave(0, 0), QuadroChave(1, 10)))
de coral.jogos importe Sprite, ClipeAnimacao
defina sprite como Sprite(0, 0, 10, 10)
defina clipe como ClipeAnimacao((faixa,))
defina vinculo como VinculoClipe(sprite, clipe, {"x": "x"})
execute vinculo.atualizar(0.5)
mostre sprite.x
```

Resultado

Saída esperada

```
5.0
```

Erros

TypeError para clipe incompatível; ValueError para mapeamento, faixa, propriedade, modo ou composição inválidos; erros de leitura, escrita e clipe são propagados.

Atenção

O construtor captura valores iniciais. atualizar avança o clipe e escreve nas propriedades; não há rollback se uma escrita falhar. Não atualize o mesmo clipe novamente em outro controlador no mesmo passo. Restauração acontece uma vez; reiniciar só o clipe não reinicia o estado interno de restauração do vínculo. O construtor direto não registra o vínculo na lista do alvo nem verifica conflitos com outros vínculos; prefira ligar_clipe para integração.

`ligar_clipe(...)` · módulo `coral.jogos`

O que faz

Cria e registra um vínculo de animação no alvo, verificando conflitos.

Assinatura

Assinatura para consulta

```
ligar_clipe(alvo: 'Any', clipe: 'ClipeAnimacao', mapeamento: 'dict[str, str]', *, modo:
  'str' = 'absoluto', restaurar_ao_final: 'bool' = False, composicao: 'str' =
  'substituir', permitir_conflito: 'bool' = False) -> 'VinculoClipe'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
alvo	Any	sim	sem padrão	Objeto com adaptadores das propriedades e valores iniciais finitos.
clipe	ClipeAnimacao	sim	sem padrão	ClipeAnimacao a reproduzir.
mapeamento	dict[str, str]	sim	sem padrão	Dicionário não vazio de nomes das faixas para propriedades do alvo.
modo	str	não	'absoluto'	absoluto usa o valor da faixa; relativo soma o valor inicial da propriedade.
restaurar_ao_final	bool	não	False	Se restaura valores iniciais quando o clipe finaliza; padrão falso.
composicao	str	não	'substituir'	substituir escreve o valor; aditiva aplica diferença em relação ao último valor calculado.
permitir_conflito	bool	não	False	Se permite sobreposição e conflito com velocidade; padrão falso.

Retorno

VinculoClipe registrado na lista de vínculos do alvo.

Exemplo

Código Coral

```
de coral.jogos importe ligar_clipe
de coral.jogos importe QuadroChave, FaixaAnimacao
defina faixa como FaixaAnimacao("x", (QuadroChave(0, 0), QuadroChave(1, 10)))
de coral.jogos importe Sprite, ClipeAnimacao
defina sprite como Sprite(0, 0, 10, 10)
defina clipe como ClipeAnimacao((faixa,))
defina vinculo como ligar_clipe(sprite, clipe, {"x": "x"})
execute vinculo.atualizar(1)
mostre sprite.x
mostre vinculo.finalizado
```

Resultado

Saída esperada

```
10.0
True
```

Erros

TypeError para clipe ou alvo incompatível; ValueError para mapeamento, adaptador ou conflito; erros de callbacks e propriedades são propagados.

Atenção

O construtor captura valores iniciais. atualizar avança o clipe e escreve nas propriedades; não há rollback se uma escrita falhar. Não atualize o mesmo clipe novamente em outro controlador no mesmo passo. Restauração acontece uma vez; reiniciar só o clipe não reinicia o estado interno de restauração do vínculo. O alvo deve aceitar armazenamento de vínculos. Detecta sobreposição entre vínculos substitutivos e conflito com velocidade de Sprite; composição aditiva ou permitir_conflito alteram essa política. A ordem de atualização importa quando conflitos são permitidos. Em alvo personalizado, o programa deve avançar os vínculos explicitamente.

`animar_propriedade(...)` · módulo `coral.jogos`

O que faz

Cria e registra uma animação de uma propriedade até um valor final.

Assinatura

Assinatura para consulta

```
animar_propriedade(alvo: 'Any', propriedade: 'str', valor: 'float', duracao: 'float', *,
   curva: 'str | Callable[[float], float]' = 'suave', modo: 'str' = 'normal') ->
  "'VinculoClipe'"
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
alvo	Any	sim	sem padrão	Sprite, Camera ou objeto com adaptador animável registrado.
propriedade	str	sim	sem padrão	Nome da propriedade suportada.
valor	float	sim	sem padrão	Valor final finito.
duracao	float	sim	sem padrão	Duração finita positiva em segundos.
curva	str | Callable[[float], float]	não	'suave'	Nome ou callback da interpolação; padrão suave.
modo	str	não	'normal'	Modo do clipe: normal, reverso ou ida_volta.

Retorno

VinculoClipe registrado no alvo, com faixa de dois quadros chave.

Exemplo

Código Coral

```
de coral.jogos importe animar_propriedade
de coral.jogos importe Sprite
defina sprite como Sprite(0, 0, 10, 10)
defina vinculo como animar_propriedade(sprite, "x", 10, 1, curva="linear")
execute vinculo.atualizar(0.5)
mostre sprite.x
```

Resultado

Saída esperada

```
5.0
```

Erros

ValueError para adaptador inexistente, valor, duração, curva, modo ou conflito; TypeError para alvo incompatível; erros de leitura e escrita são propagados.

Atenção

Lê o valor inicial, cria uma faixa e delega a ligar_clipe. Não altera a propriedade até atualizar o vínculo ou alvo integrado. Não anima atributos arbitrários: Sprite possui adaptadores de x, y, escala, rotacao, opacidade e pivôs; Camera de zoom e rotacao. Outros tipos exigem registrar_propriedade_animavel. Conflitos padrão continuam sendo verificados.

`registrar_propriedade_animavel(...)` · **módulo** `coral.jogos`

O que faz

Registra leitura e escrita explícitas de uma propriedade numérica de um tipo.

Assinatura

Assinatura para consulta

```
registrar_propriedade_animavel(tipo: 'type', nome: 'str', obter: 'Callable[[Any],
  float]', definir: 'Callable[[Any, float], None]') -> 'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tipo	type	sim	sem padrão	Classe hospedeira do adaptador; aceita subclasses durante consulta.
nome	str	sim	sem padrão	Nome não vazio da propriedade.
obter	Callable[[Any], float]	sim	sem padrão	Callback com objeto que devolve valor finito.
definir	Callable[[Any, float], None]	sim	sem padrão	Callback com objeto e valor que aplica a mudança.

Retorno

nulo; modifica o registro de adaptadores do processo.

Exemplo

Código Coral

```
de coral.jogos importe registrar_propriedade_animavel
crie a classe Medidor
    ao criar um Medidor com valor
        defina seu valor como valor
    fim
fim
crie a função ler com objeto
    retorne objeto.valor
fim
crie a função escrever com objeto, valor
    execute setattr(objeto, "valor", valor)
fim
de coral.jogos importe animar_propriedade
execute registrar_propriedade_animavel(Medidor, "valor", ler, escrever)
defina medidor como Medidor(0)
defina vinculo como animar_propriedade(medidor, "valor", 10, 1, curva="linear")
execute vinculo.atualizar(0.5)
mostre medidor.valor
```

Resultado

Saída esperada

```
5.0
```

Erros

TypeError para tipo que não é classe ou callbacks não chamáveis; ValueError para nome vazio; erros de callbacks surgem ao usar a propriedade.

Atenção

Registrar novamente a mesma classe e nome substitui callbacks. Não chama os callbacks no registro. Adaptadores nativos de Sprite e Camera têm precedência para suas propriedades suportadas; adaptadores personalizados são consultados por compatibilidade de tipo em ordem do registro. Callbacks e efeitos não são serializados nem revertidos automaticamente.

`reproduzir_ao_contrario(...)` · módulo `coral.jogos`

O que faz

Configura e reinicia uma animação ou clipe para reprodução reversa.

Assinatura

Assinatura para consulta

```
reproduzir_ao_contrario(animacao: 'Any') -> 'Any'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
animacao	Any	sim	sem padrão	Objeto com modo e método reiniciar compatíveis com reprodução reversa.

Retorno

O próprio objeto recebido, após alteração e reinício.

Exemplo

Código Coral

```
de coral.jogos importe reproduzir_ao_contrario
de coral.jogos importe QuadroChave, FaixaAnimacao
defina faixa como FaixaAnimacao("x", (QuadroChave(0, 0), QuadroChave(1, 10)))
de coral.jogos importe ClipeAnimacao
defina clipe como ClipeAnimacao((faixa,))
mostre reproduzir_ao_contrario(clipe) == clipe
mostre clipe.modo
mostre clipe.valores()["x"]
```

Resultado

Saída esperada

```
True
reverso
10.0
```

Erros

TypeError para objeto sem modo ou reiniciar; erros de atribuição ou reinício são propagados.

Atenção

Altera modo para reverso e reinicia imediatamente; não cria cópia. Ajusta sentido interno quando disponível. Não atualiza automaticamente um alvo de vínculo. O contrato é verificado parcialmente; se reiniciar falhar, as mudanças anteriores podem permanecer.

`AdaptadorEventosJogo(...)` · módulo `coral.jogos`

O que faz

Publica acontecimentos de entrada e colisão no barramento do jogo.

Construtor

Assinatura para consulta

```
AdaptadorEventosJogo(jogo: 'Jogo', *, teclas: 'Iterable[str]' = (), botoes_mouse:
  'Iterable[str]' = (), observar_mouse: 'bool' = False)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
jogo	Jogo	sim	sem padrão	Instância de Jogo que receberá o observador de atualização.
teclas	Iterable[str]	não	()	Iterável de nomes de teclas observadas; normaliza caixa e remove duplicatas.
botoes_mouse	Iterable[str]	não	()	Iterável de botões aceitos pela API de Jogos.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
observar_mouse	bool	não	False	Se observa movimento e roda do mouse; padrão falso.

Atributos importantes

Atributo	Significado
`ativo`	Se o observador publica acontecimentos.
`teclas`	Tupla normalizada de teclas.
`botoes_mouse`	Tupla normalizada de botões.

Métodos principais

Assinatura para consulta

```
ligar(self) -> 'None'
```

Referência para leitura

```
desligar(self) -> 'None'
```

Referência para leitura

```
observar_colisao(self, primeiro: 'Sprite', segundo: 'Sprite', *, nome: 'str | None' =
  None) -> '_ColisaoObservada'
```

Referência para leitura

```
publicar_transicao_visual(self, sprite: 'Sprite', transicao: 'str', *, estado: 'str |
  None' = None)
```

Retorno

Nova instância de AdaptadorEventosJogo.

Exemplo

Código Coral

```
de coral.jogos importe AdaptadorEventosJogo
de coral.jogos importe Jogo, BackendNulo
defina jogo como Jogo("exemplo", backend=BackendNulo())
defina adaptador como AdaptadorEventosJogo(jogo, teclas=["A", "a"])
mostre adaptador.teclas
execute adaptador.desligar()
mostre adaptador.ativo
```

Resultado

Saída esperada

```
('a',)
False
```

Erros

TypeError para jogo ou sprites incompatíveis; ValueError para botão, nome de colisão ou transição vazio; erros de entrada e barramento são propagados.

Atenção

A criação consulta o estado inicial das teclas e registra callback no jogo. Publica jogo_atualizado, mudanças de teclas e botões e, quando habilitado, mouse e roda. observar_colisao registra dois sprites e publica início e fim da colisão; não resolve colisão. desligar suspende observação, mas não remove o callback nem seus registros; ligar retoma. A atualização depende do laço do jogo.

`adaptar_eventos(...)` · módulo `coral.jogos`

O que faz

Instala um adaptador opcional que publica acontecimentos no jogo.

Assinatura

Assinatura para consulta

```
adaptar_eventos(jogo: 'Jogo', *, teclas: 'Iterable[str]' = (), botoes_mouse:
  'Iterable[str]' = (), observar_mouse: 'bool' = False) -> 'AdaptadorEventosJogo'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
jogo	Jogo	sim	sem padrão	Instância de Jogo que receberá o observador de atualização.
teclas	Iterable[str]	não	()	Iterável de nomes de teclas observadas; normaliza caixa e remove duplicatas.
botoes_mouse	Iterable[str]	não	()	Iterável de botões aceitos pela API de Jogos.
observar_mouse	bool	não	False	Se observa movimento e roda do mouse; padrão falso.

Retorno

AdaptadorEventosJogo com callback já registrado.

Exemplo

Código Coral

```
de coral.jogos importe adaptar_eventos
de coral.jogos importe Jogo, BackendNulo
defina jogo como Jogo("exemplo", backend=BackendNulo())
defina adaptador como adaptar_eventos(jogo, observar_mouse=verdadeiro)
mostre adaptador.ativo
execute adaptador.desligar()
```

Resultado

Saída esperada

```
True
```

Erros

TypeError para jogo incompatível; ValueError para botões inválidos; erros de leitura de entrada e registro são propagados.

Atenção

A criação consulta o estado inicial das teclas e registra callback no jogo. Publica jogo_atualizado, mudanças de teclas e botões e, quando habilitado, mouse e roda. observar_colisao registra dois sprites e publica início e fim da colisão; não resolve colisão. desligar suspende observação, mas não remove o callback nem seus registros; ligar retoma. A atualização depende do laço do jogo. Não abre janela; o exemplo usa BackendNulo.

`Assets(...)` · módulo `coral.jogos`

O que faz

Resolve caminhos de recursos dentro de uma raiz explícita.

Construtor

Assinatura para consulta

```
Assets(raiz: 'str | Path' = 'assets')
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
raiz	str | Path	não	'assets'	Texto ou Path da pasta de recursos; padrão assets, com expansão de usuário e resolução.

Atributos importantes

Atributo	Significado
`raiz`	Path absoluto normalizado da raiz.

Métodos principais

Assinatura para consulta

```
imagem(self, nome: 'str | Path') -> 'Path'
```

Referência para leitura

```
som(self, nome: 'str | Path') -> 'Path'
```

Referência para leitura

```
spritesheet(self, nome: 'str | Path', *, largura_imagem: 'int', altura_imagem: 'int',
  largura_quadro: 'int', altura_quadro: 'int', margem: 'int' = 0, espacamento: 'int' = 0)
  -> 'Spritesheet'
```

Retorno

Nova instância de Assets.

Exemplo

Código Coral

```
de coral.jogos importe Assets
de pathlib importe Path
execute Path("assets").mkdir()
execute Path("assets/exemplo.txt").write_text("conteúdo", encoding="utf-8")
defina recursos como Assets("assets")
mostre recursos.imagem("exemplo.txt").name
```

Resultado

Saída esperada

```
exemplo.txt
```

Erros

ErroAssetCoral para caminho fora da raiz ou arquivo ausente; erros de caminho e metadados de spritesheet são propagados.

Atenção

O construtor não cria a pasta. imagem e som verificam arquivo e contenção na raiz após resolver caminhos e links; devolvem Path, sem decodificar imagem ou áudio. O exemplo evidencia resolução de caminho, não formato de imagem. spritesheet exige dimensões explícitas e produz metadados de recorte. A verificação não elimina corrida entre resolução e leitura posterior.

`Animacao(...)` · módulo `coral.jogos`

O que faz

Seleciona quadros de imagem em uma linha de tempo de animação.

Construtor

Assinatura para consulta

```
Animacao(quadros: 'tuple[str | Path | QuadroSprite, ...]', fps: 'float' = 10.0, repetir:
   'bool' = True, _tempo: 'float' = 0.0, _indice: 'int' = 0, finalizada: 'bool' = False,
  duracoes: 'tuple[float, ...] | None' = None, velocidade: 'float' = 1.0, modo: 'str' =
  'normal', indice_inicial: 'int | None' = None, marcadores: 'dict[str, int]' = <factory>)
   -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
quadros	tuple[str | Path | QuadroSprite, ...]	sim	sem padrão	Coleção não vazia de caminhos ou QuadroSprite; normalizada para tupla.
fps	float	não	10.0	Quadros por segundo finitos positivos; usados quando duracoes é nulo.
repetir	bool	não	True	Se repete ciclos; padrão verdadeiro.
_tempo	float	não	0.0	Acumulador inicial interno de tempo; prefira zero.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
_indice	int	não	0	Índice inicial interno usado quando indice_inicial é nulo no modo normal.
finalizada	bool	não	False	Estado inicial de conclusão; padrão falso.
duracoes	tuple[float, ...] | None	não	None	Tupla de duração positiva por quadro, ou nulo para usar fps.
velocidade	float	não	1.0	Multiplicador finito positivo do dt.
modo	str	não	'normal'	normal, reverso ou ida_volta; admite aliases de ida e volta.
indice_inicial	int | None	não	None	Índice inicial explícito, ou nulo para o padrão do modo.
marcadores	dict[str, int]	não	<factory>	Dicionário de nomes não vazios para índices de quadro.

Atributos importantes

Atributo	Significado
`quadro_atual`	Recurso do quadro selecionado.
`indice_atual`	Índice atual.
`duracao`	Soma das durações de quadro.
`tempo_atual`	Posição temporal calculada pelos quadros e acumulador.
`ciclos`	Ciclos concluídos.

Métodos principais

Assinatura para consulta

```
atualizar(self, dt: 'float')
```

Referência para leitura

```
reiniciar(self)
```

Referência para leitura

```
marcar(self, nome: 'str', quadro: 'int') -> "'Animacao'"
```

Referência para leitura

```
ao_marcador(self, nome: 'str', callback: 'Callable') -> 'Callable'
```

Referência para leitura

```
ao_completar_ciclo(self, callback: 'Callable') -> 'Callable'
```

Referência para leitura

```
ao_finalizar(self, callback: 'Callable') -> 'Callable'
```

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de Animacao.

Exemplo

Código Coral

```
de coral.jogos importe Animacao
defina animacao como Animacao(("a.png", "b.png"), fps=2, repetir=falso)
mostre animacao.quadro_atual
execute animacao.atualizar(0.5)
mostre animacao.quadro_atual
```

Resultado

Saída esperada

```
a.png
b.png
```

Erros

ValueError para quadros vazios, fps, duração, velocidade, modo ou marcador inválidos; IndexError para índice inexistente; erros de callbacks e carregamento são propagados no consumidor.

Atenção

Não abre imagens nem janela. Sprite.atualizar pode avançar a animação ligada; não avance a mesma instância duas vezes por passo. Índices começam em zero. Campos com prefixo _ estão expostos pela assinatura do registro, mas use indice_inicial e reiniciar no uso comum. Compartilhar a instância compartilha seu progresso e callbacks. A validação não decodifica os quadros; o backend os carrega ao desenhar.

`Spritesheet(...)` · módulo `coral.jogos`

O que faz

Descreve dimensões e recortes de uma imagem sem carregá-la.

Construtor

Assinatura para consulta

```
Spritesheet(caminho: 'str | Path', *, largura_imagem: 'int', altura_imagem: 'int') ->
  'None'
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
caminho	str | Path	sim	sem padrão	Texto ou Path da imagem; convertido para Path.
largura_imagem	int	sim	sem padrão	Largura positiva declarada da imagem; use inteiro.
altura_imagem	int	sim	sem padrão	Altura positiva declarada da imagem; use inteiro.

Atributos importantes

Atributo	Significado
`caminho`	Imagem de origem.
`colunas`	Número de colunas quando configurada em grade.
`linhas`	Número de linhas da grade.

Métodos principais

Assinatura para consulta

```
em_grade(caminho: 'str | Path', *, largura_imagem: 'int', altura_imagem: 'int',
  largura_quadro: 'int', altura_quadro: 'int', margem: 'int' = 0, espacamento: 'int' = 0)
  -> "'Spritesheet'"
```

Referência para leitura

```
quadro(self, indice: 'int') -> 'QuadroSprite'
```

Referência para leitura

```
quadro_em(self, linha: 'int', coluna: 'int') -> 'QuadroSprite'
```

Referência para leitura

```
quadros_de(self, inicio: 'int', fim: 'int') -> 'tuple[QuadroSprite, ...]'
```

Referência para leitura

```
quadros_da_linha(self, linha: 'int', coluna_inicial: 'int' = 0, coluna_final: 'int |
  None' = None) -> 'tuple[QuadroSprite, ...]'
```

Retorno

Nova instância de Spritesheet.

Exemplo

Código Coral

```
de coral.jogos importe Spritesheet
defina folha como Spritesheet.em_grade("atlas.png", largura_imagem=32, altura_imagem=16,
   largura_quadro=16, altura_quadro=16)
mostre len(folha)
mostre folha.quadro(1).retangulo
```

Resultado

Saída esperada

```
2
(16, 0, 16, 16)
```

Erros

ValueError para medidas, margem, espaçamento ou intervalo inválidos; IndexError para recorte ausente; TypeError para valores incompatíveis.

Atenção

O construtor simples começa sem quadros. em_grade cria recortes completos em ordem de linha, com margem e espaçamento. Não verifica existência ou dimensões reais da imagem.

quadro usa índice iniciado em zero; quadros_de inclui o índice final. atlas permite recortes nomeados. O programa deve fornecer medidas inteiras coerentes; conversões com int não asseguram essa coerência para entradas fracionárias.

`CamadaApresentacao(...)` · módulo `coral.jogos`

O que faz

Agrupa elementos visuais em uma camada ordenada de mundo ou tela.

Construtor

Assinatura para consulta

```
CamadaApresentacao(nome: 'str', ordem: 'int' = 0, visivel: 'bool' = True, opacidade:
  'float' = 1.0, espaco: 'str' = 'mundo', paralaxe_x: 'float' = 1.0, paralaxe_y: 'float' =
   1.0, elementos: 'list[Any]' = <factory>) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
nome	str	sim	sem padrão	Nome não vazio com espaços externos removidos.
ordem	int	não	0	Prioridade de desenho convertida para int; menor é desenhada antes.
visivel	bool	não	True	Visibilidade inicial, convertida para bool.
opacidade	float	não	1.0	Número finito de zero a um.
espaco	str	não	'mundo'	mundo aplica câmera; tela ignora câmera.
paralaxe_x	float	não	1.0	Fator horizontal finito da influência da câmera.
paralaxe_y	float	não	1.0	Fator vertical finito da influência da câmera.
elementos	list[Any]	não	<factory>	Lista inicial de elementos; padrão nova lista vazia.

Atributos importantes

Atributo	Significado
`presa_a_tela`	Se espaco é tela.
`elementos`	Lista mutável em ordem de inserção.

Métodos principais

Assinatura para consulta

```
adicionar(self, elemento: 'Any') -> 'Any'
```

Referência para leitura

```
remover(self, elemento: 'Any') -> 'bool'
```

Referência para leitura

```
definir_visibilidade(self, visivel: 'bool') -> "'CamadaApresentacao'"
```

Referência para leitura

```
definir_opacidade(self, opacidade: 'float') -> "'CamadaApresentacao'"
```

Referência para leitura

```
definir_paralaxe(self, x: 'float' = 1.0, y: 'float | None' = None) ->
  "'CamadaApresentacao'"
```

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de CamadaApresentacao.

Exemplo

Código Coral

```
de coral.jogos importe CamadaApresentacao
defina camada como CamadaApresentacao("HUD", espaco="tela")
mostre camada.presa_a_tela
execute camada.adicionar("marcador")
mostre camada.remover("marcador")
```

Resultado

Saída esperada

```
True
True
```

Erros

ValueError para nome, opacidade, espaço ou paralaxe inválidos; erros de conversão, comparação e desenho são propagados.

Atenção

Não desenha nem se adiciona a uma cena automaticamente; prefira Cena.criar_camada para integrar. Mantém ordem de inserção e evita elemento já presente pela comparação da lista. A lista inicial não valida integralmente os elementos; o desenho exige contratos compatíveis. Paralaxe só participa do espaço mundo. Alterações diretas em campos não repetem validação dos métodos.

`RepresentacaoEntidade(...)` · módulo `coral.jogos`

O que faz

Sincroniza um sprite visual com a posição de uma entidade.

Construtor

Assinatura para consulta

```
RepresentacaoEntidade(entidade: 'Any', sprite: 'Sprite', atributo_x: 'str' = 'x',
  atributo_y: 'str' = 'y') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
entidade	Any	sim	sem padrão	Objeto com obter(nome, padrão) ou atributos numéricos de posição.
sprite	Sprite	sim	sem padrão	Sprite visual que receberá as coordenadas.
atributo_x	str	não	'x'	Nome do campo horizontal; padrão x.
atributo_y	str	não	'y'	Nome do campo vertical; padrão y.

Atributos importantes

Atributo	Significado
`x`	Posição horizontal do sprite visual.
`y`	Posição vertical do sprite visual.
`largura`	Largura visual.
`altura`	Altura visual.

Métodos principais

Assinatura para consulta

```
sincronizar(self) -> 'Sprite'
```

Retorno

Nova instância de RepresentacaoEntidade.

Exemplo

Código Coral

```
de coral.jogos importe RepresentacaoEntidade
de coral.jogos importe Sprite
defina entidade como Sprite(3, 4, 10, 10)
defina visual como Sprite(0, 0, 10, 10)
defina vinculo como RepresentacaoEntidade(entidade, visual)
execute vinculo.sincronizar()
mostre visual.x
mostre visual.y
```

Resultado

Saída esperada

```
3
4
```

Erros

ValueError para nome vazio ou campo ausente; TypeError para coordenada inválida; erros de leitura e escrita são propagados.

Atenção

A entidade é a fonte da posição; sincronizar não escreve de volta nela. O construtor só valida nomes não vazios e não sincroniza. Usa obter quando chamável, caso contrário atributos; exige int ou float finito e rejeita bool. A escrita é incremental: se y falhar, x pode já ter sido aplicado. Prefira a integração pública de Cena para sincronização no laço.

`TextoApresentacao(...)` · módulo `coral.jogos`

O que faz

Representa texto 2D com quebra de linha, alinhamento e estilo.

Construtor

Assinatura para consulta

```
TextoApresentacao(texto: 'str', x: 'float' = 0.0, y: 'float' = 0.0, tamanho: 'int' = 24,
   cor: 'Cor' = Cor(vermelho=255, verde=255, azul=255, alfa=255), largura_maxima: 'float |
   None' = None, alinhamento_horizontal: 'str' = 'esquerda', alinhamento_vertical: 'str' =
   'topo', espacamento_linhas: 'float' = 1.0, sombra: 'Cor | None' = None,
  deslocamento_sombra: 'tuple[float, float]' = (2.0, 2.0), contorno: 'Cor | None' = None,
  espessura_contorno: 'int' = 1, opacidade: 'float' = 1.0, origem: 'str | None' = None,
  pivo: 'tuple[float, float] | None' = None, ancora_viewport: 'str | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
texto	str	sim	sem padrão	Conteúdo convertido para texto.
x	float	não	0.0	Posição horizontal finita.
y	float	não	0.0	Posição vertical finita.
tamanho	int	não	24	Tamanho positivo convertido para inteiro; padrão 24.
cor	Cor	não	Cor(vermelho=255, verde=255, azul=255, alfa=255)	Cor do texto; padrão branco.
largura_maxima	float | None	não	None	Largura finita positiva para quebra, ou nulo.
alinhamento_horizontal	str	não	'esquerda'	esquerda, centro ou direita.
alinhamento_vertical	str	não	'topo'	topo, centro ou base.
espacamento_linhas	float	não	1.0	Multiplicador finito positivo do espaçamento.
sombra	Cor | None	não	None	Cor opcional de sombra.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
deslocamento_sombra	tuple[float, float]	não	(2.0, 2.0)	Par finito de deslocamentos da sombra.
contorno	Cor | None	não	None	Cor opcional de contorno.
espessura_contorno	int	não	1	Espessura não negativa convertida para inteiro.
opacidade	float	não	1.0	Número finito de zero a um.
origem	str | None	não	None	topo_esquerdo, centro ou centro_inferior, ou nulo.
pivo	tuple[float, float] | None	não	None	Par finito de coordenadas de pivô, ou nulo.
ancora_viewport	str | None	não	None	Nome de âncora da área lógica, ou nulo.

Métodos principais

Assinatura para consulta

```
linhas(self, jogo) -> 'tuple[str, ...]'
```

Retorno

Nova instância de TextoApresentacao.

Exemplo

Código Coral

```
de coral.jogos importe TextoApresentacao
de coral.jogos importe Jogo, BackendNulo
defina jogo como Jogo("texto", backend=BackendNulo())
defina legenda como TextoApresentacao("Coral", tamanho=16)
mostre legenda.linhas(jogo)
```

Resultado

Saída esperada

```
('Coral',)
```

Erros

ValueError para posição, tamanho, largura, alinhamento, espaçamento, opacidade, pivô ou âncora inválidos; erros da métrica e do desenho são propagados.

Atenção

Não cria widget nem captura entrada. linhas usa a métrica do jogo quando há limite de largura; quebra palavras longas e preserva parágrafos. Não desenha no construtor. Cores são consumidas pelo backend sem validação integral no registro. Campos são mutáveis; forneça inteiros coerentes para tamanho e espessura. Pode ser adicionado a uma camada de apresentação.

`Jogo(...)` · módulo `coral.jogos`

O que faz

Coordena backend, cenas, regras, mundo e laço de atualização e desenho.

Construtor

Assinatura para consulta

```
Jogo(titulo: 'str', largura: 'int' = 800, altura: 'int' = 450, fps: 'int' = 60, *,
  backend: 'BackendJogos', mundo=None, relogio: 'FonteTempo | None' = None, eventos:
  'Eventos | None' = None, tela_cheia: 'bool' = False)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
titulo	str	sim	sem padrão	Título da janela ou execução.
largura	int	não	800	Largura lógica positiva; padrão 800, use inteiro.
altura	int	não	450	Altura lógica positiva; padrão 450, use inteiro.
fps	int	não	60	Limite positivo de quadros por segundo; padrão 60, use inteiro.
backend	BackendJogos	sim	sem padrão	Implementação obrigatória da API BackendJogos; BackendNulo permite execução sem janela.
mundo	valor dinâmico	não	None	Mundo compatível a integrar, ou nulo.
relogio	FonteTempo | None	não	None	FonteTempo com passo para dt lógico, ou nulo para usar dt do backend.
eventos	Eventos | None	não	None	Barramento Eventos compartilhado, ou nulo para criar um.
tela_cheia	bool	não	False	Se solicita tela cheia ao iniciar; padrão falso.

Atributos importantes

Atributo	Significado
`rodando`	Estado do laço.
`cena_atual`	Cena selecionada.
`metricas`	Métricas dos quadros.
`eventos`	Barramento compartilhado.
`regras`	Motor de regras.

Métodos principais

Assinatura para consulta

```
executar(self, *, max_quadros: 'int | None' = None) -> 'None'
```

Referência para leitura

```
mudar_cena(self, cena) -> 'None'
```

Referência para leitura

```
mudar_cena_com_transicao(self, cena, *, tipo: 'str' = 'desaparecer', duracao: 'float' =
  0.4, curva: 'str | Callable[[float], float]' = 'suave', bloquear_entrada: 'bool' = True,
   atualizar_anterior: 'bool' = True, atualizar_proxima: 'bool' = True, direcao: 'str' =
  'direita') -> 'TransicaoCena'
```

Referência para leitura

```
usar_mundo(self, mundo)
```

Referência para leitura

```
quando_iniciar(self, funcao: "Callable[['Jogo'], Any]")
```

Referência para leitura

```
quando_atualizar(self, funcao: "Callable[['Jogo', float], Any]")
```

Referência para leitura

```
quando_desenhar(self, funcao: "Callable[['Jogo'], Any]")
```

Referência para leitura

```
quando_encerrar(self, funcao: "Callable[['Jogo'], Any]")
```

Referência para leitura

```
estado_inspecao(self) -> 'dict[str, Any]'
```

Retorno

Nova instância de Jogo.

Exemplo

Código Coral

```
de coral.jogos importe Jogo
de coral.jogos importe BackendNulo, Cena
defina jogo como Jogo("exemplo", backend=BackendNulo())
execute jogo.mudar_cena(Cena("principal"))
execute jogo.executar(max_quadros=1)
mostre jogo.metricas.quadros
mostre jogo.rodando
```

Resultado

Saída esperada

```
1
False
```

Erros

ValueError para dimensões ou fps não positivos, mundo ou opções inválidos; TypeError para max_quadros incompatível; erros de backend, callbacks, regras e cenas são propagados.

Atenção

O construtor não abre janela. executar inicia o backend, aplica regras, mundo, callbacks, cena e desenho; usa finalmente para encerrar. max_quadros limita execução, inclusive zero para apenas iniciar e encerrar. Uma fonte de tempo explícita controla dt lógico, sem substituir a limitação física do backend. Use callbacks e integração de cenas em vez de alterar estado interno. Backend gráfico pode exigir dependências opcionais e sessão gráfica.

`EfeitoCamera(...)` · módulo `coral.jogos`

O que faz

Produz uma alteração visual temporária sem mover a posição lógica da câmera.

Construtor

Assinatura para consulta

```
EfeitoCamera(tipo: 'str', duracao: 'float', intensidade: 'float' = 1.0, x: 'float' =
  0.0, y: 'float' = 0.0, zoom: 'float' = 0.0, rotacao: 'float' = 0.0, semente: 'int' = 0,
  frequencia: 'float' = 24.0, curva: 'str | Callable[[float], float]' = 'linear', tempo:
  'float' = 0.0, estado: 'str' = 'ativo') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
tipo	str	sim	sem padrão	tremor, deslocamento, zoom ou rotacao.
duracao	float	sim	sem padrão	Tempo finito positivo em segundos.
intensidade	float	não	1.0	Multiplicador finito do efeito.
x	float	não	0.0	Deslocamento horizontal finito para tipo deslocamento.
y	float	não	0.0	Deslocamento vertical finito para tipo deslocamento.
zoom	float	não	0.0	Delta finito de zoom para esse tipo.
rotacao	float	não	0.0	Delta finito de rotação para esse tipo.
semente	int	não	0	Inteiro convertido usado nas fases determinísticas do tremor.
frequencia	float	não	24.0	Frequência finita não negativa do tremor.
curva	str | Callable[[float], float]	não	'linear'	Nome ou callback temporal; o ganho decai como um menos a curva.
tempo	float	não	0.0	Tempo inicial finito não negativo.
estado	str	não	'ativo'	ativo, pausado, cancelado ou finalizado.

Atributos importantes

Atributo	Significado
`ativo`	Se estado é ativo ou pausado.
`progresso`	Tempo dividido pela duração, limitado a zero e um.

Métodos principais

Assinatura para consulta

```
amostra(self) -> 'tuple[float, float, float, float]'
```

Referência para leitura

```
avancar(self, dt: 'float') -> 'None'
```

Referência para leitura

```
pausar(self) -> "'EfeitoCamera'"
```

Referência para leitura

```
retomar(self) -> "'EfeitoCamera'"
```

Referência para leitura

```
cancelar(self) -> "'EfeitoCamera'"
```

Retorno

Nova instância de EfeitoCamera.

Exemplo

Código Coral

```
de coral.jogos importe EfeitoCamera
defina efeito como EfeitoCamera("deslocamento", 1, x=10)
mostre efeito.amostra()
execute efeito.avancar(0.5)
mostre efeito.amostra()
```

Resultado

Saída esperada

```
(10.0, 0.0, 0.0, 0.0)
(5.0, 0.0, 0.0, 0.0)
```

Erros

ValueError para tipo, duração, frequência, tempo, estado ou número inválidos; erros da curva são propagados.

Atenção

amostra devolve dx, dy, delta de zoom e delta de rotação; não aplica automaticamente em câmera. Pausado mantém sua amostra e conta como ativo; cancelado e finalizado produzem zeros. O construtor consulta a curva em zero e pode executar callback. Use a API de efeitos de Camera para integração. Não oferece reinício nesta classe.

`TransicaoCena(...)` · módulo `coral.jogos`

O que faz

Coordena a apresentação temporal entre duas cenas.

Construtor

Assinatura para consulta

```
TransicaoCena(origem: 'Any', destino: 'Any', tipo: 'str' = 'desaparecer', duracao:
  'float' = 0.4, curva: 'str | Callable[[float], float]' = 'suave', bloquear_entrada:
  'bool' = True, atualizar_anterior: 'bool' = True, atualizar_proxima: 'bool' = True,
  direcao: 'str' = 'direita', tempo: 'float' = 0.0, estado: 'str' = 'ativo') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
origem	Any	sim	sem padrão	Cena ou referência de origem para apresentação.
destino	Any	sim	sem padrão	Cena ou referência de destino.
tipo	str	não	'desaparecer'	desaparecer, aparecer, escurecer, clarear, deslizar, revelar ou personalizada; admite aliases.
duracao	float	não	0.4	Segundos positivos; padrão 0.4.
curva	str | Callable[[float], float]	não	'suave'	Curva temporal ou callback; padrão suave.
bloquear_entrada	bool	não	True	Se o consumidor deve bloquear entrada durante a transição.
atualizar_anterior	bool	não	True	Se o consumidor deve atualizar a cena anterior.
atualizar_proxima	bool	não	True	Se o consumidor deve atualizar a próxima cena.
direcao	str	não	'direita'	direita, esquerda, cima ou baixo.
tempo	float	não	0.0	Tempo inicial não negativo; padrão zero.
estado	str	não	'ativo'	ativo, pausado, cancelado ou finalizado.

Atributos importantes

Atributo	Significado
`finalizada`	Se estado é finalizado.
`ativa`	Se ativa ou pausada.
`fase`	saida, entrada ou concluida.
`entrada_bloqueada`	Bloqueio durante estado ativo ou pausado.

Métodos principais

Assinatura para consulta

```
avancar(self, dt: 'float') -> 'bool'
```

Referência para leitura

```
amostra(self) -> 'dict[str, Any]'
```

Referência para leitura

```
ao_concluir(self, callback: 'Callable') -> 'Callable'
```

Referência para leitura

```
pausar(self) -> "'TransicaoCena'"
```

Referência para leitura

```
retomar(self) -> "'TransicaoCena'"
```

Referência para leitura

```
cancelar(self) -> "'TransicaoCena'"
```

Retorno

Nova instância de TransicaoCena.

Exemplo

Código Coral

```
de coral.jogos importe TransicaoCena
de coral.jogos importe Cena
defina transicao como TransicaoCena(Cena("A"), Cena("B"), duracao=1)
mostre transicao.avancar(0.5)
mostre transicao.avancar(0.5)
mostre transicao.finalizada
```

Resultado

Saída esperada

```
True
False
True
```

Erros

ValueError para tipo, duração, tempo, direção, estado ou curva inválidos; TypeError para callback não chamável; erros de callbacks e consumidor são propagados.

Atenção

avancar informa uma vez quando o progresso cruza metade, mas não troca cena por conta própria. Jogo.mudar_cena_com_transicao integra a troca efetiva. Callback de conclusão recebe a transição. O construtor não valida integralmente os objetos de cena e executa a curva em zero. Pausada permanece ativa e pode manter entrada bloqueada; cancelada não equivale a concluída.

`AnimacoesDirecionais(...)` · módulo `coral.jogos`

O que faz

Agrupa animações nomeadas por direção visual.

Construtor

Assinatura para consulta

```
AnimacoesDirecionais(animacoes: 'dict[str, Animacao]') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
animacoes	dict[str, Animacao]	sim	sem padrão	Dicionário não vazio de direção para Animacao; normaliza nomes sem espaços externos e caixa.

Atributos importantes

Atributo	Significado
`direcoes`	Tupla dos nomes normalizados.

Métodos principais

Assinatura para consulta

```
para(self, direcao: 'str') -> 'Animacao'
```

Referência para leitura

```
usar(self, sprite: 'Sprite', direcao: 'str', *, reiniciar: 'bool' = True) -> 'Animacao'
```

Referência para leitura

```
ligar(self, movimento: 'MovimentoDirecional', sprite: 'Sprite') ->
  'VinculoAnimacaoMovimento'
```

Retorno

Nova instância de AnimacoesDirecionais.

Exemplo

Código Coral

```
de coral.jogos importe AnimacoesDirecionais
de coral.jogos importe Animacao, Sprite
defina animacao como Animacao(("a.png", "b.png"))
defina visual como Sprite(0, 0, 10, 10)
defina grupo como AnimacoesDirecionais({"direita": animacao})
execute grupo.usar(visual, "DIREITA")
mostre grupo.direcoes
mostre visual.imagem
```

Resultado

Saída esperada

```
('direita',)
a.png
```

Erros

ValueError para coleção vazia, nome vazio, direção ausente ou incompatibilidade; TypeError para animação, sprite ou movimento inválidos.

Atenção

Não move entidade. usar liga a animação ao sprite e seleciona sua imagem; reinicia quando troca de instância e reiniciar é verdadeiro. Valores de animação são compartilhados. Nomes que coincidem após normalização substituem a entrada anterior. ligar escolhe direções compatíveis com o MovimentoDirecional e instala um observador.

`VinculoAnimacaoMovimento(...)` · módulo `coral.jogos`

O que faz

Seleciona animação visual quando muda a direção de olhar de um movimento.

Construtor

Assinatura para consulta

```
VinculoAnimacaoMovimento(grupo: "'AnimacoesDirecionais'", movimento:
  'MovimentoDirecional', sprite: 'Sprite', direcoes_compativeis: 'tuple[str, ...]',
  direcao_visual: 'str | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grupo	'AnimacoesDirecionais'	sim	sem padrão	AnimacoesDirecionais que fornece os recursos visuais.
movimento	MovimentoDirecional	sim	sem padrão	MovimentoDirecional observado.
sprite	Sprite	sim	sem padrão	Sprite visual a atualizar.
direcoes_compativeis	tuple[str, ...]	sim	sem padrão	Tupla não vazia de direções comuns ao movimento e grupo.
direcao_visual	str | None	não	None	Direção visual inicial ou nulo; atualizada pelo observador.

Atributos importantes

Atributo	Significado
`direcao_visual`	Última direção visual selecionada.

Retorno

Nova instância de VinculoAnimacaoMovimento.

Exemplo

Código Coral

```
de coral.jogos importe VinculoAnimacaoMovimento
de coral.jogos importe Animacao, Sprite
defina animacao como Animacao(("a.png", "b.png"))
defina visual como Sprite(0, 0, 10, 10)
de coral.direcoes importe MovimentoDirecional, ConjuntoDirecoes
defina movimento como MovimentoDirecional(visual, ConjuntoDirecoes.cardeais())
de coral.jogos importe AnimacoesDirecionais
defina grupo como AnimacoesDirecionais({"direita": animacao})
defina vinculo como VinculoAnimacaoMovimento(grupo, movimento, visual, ("direita",))
mostre isinstance(vinculo, VinculoAnimacaoMovimento)
```

Resultado

Saída esperada

```
True
```

Erros

ValueError para direções vazias ou incompatíveis; erros de movimento, resolução e recursos são propagados.

Atenção

O construtor registra callback e, quando já há direção de olhar, sincroniza imediatamente. Resolve o vetor contra direções compatíveis; não altera posição nem toma controle do movimento. O construtor direto não valida integralmente coerência de tipos e nomes; prefira grupo.ligar. Esta classe não oferece desligamento do observador.

`EstadosAnimacao(...)` · **módulo** `coral.jogos`

O que faz

Agrupa animações por estado visual, independente do estado lógico.

Construtor

Assinatura para consulta

```
EstadosAnimacao(estados: 'dict[str, Animacao | AnimacoesDirecionais]', *,
  estado_inicial: 'str | None' = None)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
estados	dict[str, Animacao | AnimacoesDirecionais]	sim	sem padrão	Dicionário não vazio de nome para Animacao ou AnimacoesDirecionais.
estado_inicial	str | None	não	None	Estado existente para uso na ligação, ou nulo sem seleção inicial.

Atributos importantes

Atributo	Significado
`estados`	Tupla de nomes disponíveis.

Atributo	Significado
`estado_inicial`	Nome normalizado ou nulo.

Métodos principais

Assinatura para consulta

```
para(self, estado: 'str') -> 'Animacao | AnimacoesDirecionais'
```

Referência para leitura

```
usar(self, sprite: 'Sprite', estado: 'str', *, direcao: 'str | None' = None, reiniciar:
  'bool' = True) -> 'Animacao'
```

Referência para leitura

```
ligar(self, movimento: 'MovimentoDirecional', sprite: 'Sprite', *, estado_inicial: 'str
  | None' = None, direcao_inicial: 'str | None' = None) ->
  'VinculoEstadosAnimacaoMovimento'
```

Referência para leitura

```
ao_entrar(self, estado: 'str', callback: 'Callable') -> 'Callable'
```

Referência para leitura

```
ao_sair(self, estado: 'str', callback: 'Callable') -> 'Callable'
```

Retorno

Nova instância de EstadosAnimacao.

Exemplo

Código Coral

```
de coral.jogos importe EstadosAnimacao
de coral.jogos importe Animacao, Sprite
defina animacao como Animacao(("a.png", "b.png"))
defina visual como Sprite(0, 0, 10, 10)
defina estados como EstadosAnimacao({"parado": animacao})
execute estados.usar(visual, "parado")
mostre estados.estados
mostre visual.imagem
```

Resultado

Saída esperada

```
('parado',)
a.png
```

Erros

ValueError para estados vazios, nomes duplicados, ausentes ou direção indisponível; TypeError para recurso, sprite, movimento ou callback incompatível.

Atenção

Normaliza nomes e rejeita duplicatas normalizadas. Compartilha animações, sem copiar progresso. usar escolhe recurso; um recurso direcional exige direção ou vínculo de movimento. ligar registra observador e pode aplicar estado inicial. Callbacks de entrada

recebem sprite, estado e anterior; saída recebe sprite, estado e próximo. Efeitos dos callbacks não são transacionais.

`VinculoEstadosAnimacaoMovimento(...)` · módulo `coral.jogos`

O que faz

Mantém estado visual e direção do sprite como dimensões separadas.

Construtor

Assinatura para consulta

```
VinculoEstadosAnimacaoMovimento(grupo: "'EstadosAnimacao'", movimento:
  'MovimentoDirecional', sprite: 'Sprite', estado_atual: 'str | None' = None,
  estado_anterior: 'str | None' = None, direcao_visual: 'str | None' = None,
  ultima_direcao_visual: 'str | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grupo	'EstadosAnimacao'	sim	sem padrão	EstadosAnimacao que fornece os recursos.
movimento	MovimentoDirecional	sim	sem padrão	MovimentoDirecional observado.
sprite	Sprite	sim	sem padrão	Sprite visual.
estado_atual	str | None	não	None	Nome inicial de estado visual, ou nulo.
estado_anterior	str | None	não	None	Histórico inicial de estado, ou nulo.
direcao_visual	str | None	não	None	Direção visual inicial, ou nulo.
ultima_direcao_visual	str | None	não	None	Direção visual lembrada para seleção posterior, ou nulo.

Atributos importantes

Atributo	Significado
`estado_atual`	Estado visual selecionado.
`estado_anterior`	Último estado substituído.
`direcao_visual`	Direção do recurso direcional ou nulo.

Métodos principais

Assinatura para consulta

```
usar(self, estado: 'str', *, direcao: 'str | None' = None, reiniciar: 'bool' = True) ->
  'Animacao'
```

Retorno

Nova instância de VinculoEstadosAnimacaoMovimento.

Exemplo

Código Coral

```
de coral.jogos importe VinculoEstadosAnimacaoMovimento
de coral.jogos importe Animacao, Sprite
defina animacao como Animacao(("a.png", "b.png"))
defina visual como Sprite(0, 0, 10, 10)
de coral.direcoes importe MovimentoDirecional, ConjuntoDirecoes
defina movimento como MovimentoDirecional(visual, ConjuntoDirecoes.cardeais())
de coral.jogos importe EstadosAnimacao
defina estados como EstadosAnimacao({"parado": animacao}, estado_inicial="parado")
defina vinculo como VinculoEstadosAnimacaoMovimento(estados, movimento, visual)
execute vinculo.usar("parado")
mostre vinculo.estado_atual
```

Resultado

Saída esperada

```
parado
```

Erros

TypeError para movimento ou sprite incompatível; ValueError para estado, direção ou recurso ausentes; erros de callbacks são propagados.

Atenção

Registra observador no construtor, sem selecionar automaticamente grupo.estado_inicial. prefira estados.ligar para registro do vínculo no grupo e seleção inicial. usar pode disparar callbacks de saída antes de validar integralmente a nova seleção; falha não desfaz efeitos. Mudança só da direção preserva o estado visual. O histórico inicial não é integralmente validado.

`TransicaoEstadoAnimacao(...)` · módulo `coral.jogos`

O que faz

Descreve a política de uma mudança de estado visual.

Construtor

Assinatura para consulta

```
TransicaoEstadoAnimacao(estado: 'str', prioridade: 'int' = 0, tempo_minimo: 'float' =
  0.0, bloquear_interrupcao: 'float' = 0.0, retorno_automatico: 'str | None' = None,
  reiniciar: 'bool' = True) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
estado	str	sim	sem padrão	Estado visual de destino; validado e normalizado pelo grupo consumidor.
prioridade	int	não	0	Prioridade convertida para int.
tempo_minimo	float	não	0.0	Tempo finito não negativo exigido no estado anterior.
bloquear_interrupcao	float	não	0.0	Tempo finito não negativo de bloqueio após entrada.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
retorno_automatico	str | None	não	None	Estado de retorno, anterior ou estado_anterior, ou nulo.
reiniciar	bool	não	True	Se solicita reinício da animação ao entrar.

Retorno

Nova instância de TransicaoEstadoAnimacao.

Exemplo

Código Coral

```
de coral.jogos importe TransicaoEstadoAnimacao
defina politica como TransicaoEstadoAnimacao("ataque", prioridade=2, tempo_minimo=0.1)
mostre politica.prioridade
mostre politica.tempo_minimo
```

Resultado

Saída esperada

```
2
0.1
```

Erros

ValueError para tempos inválidos; erros de conversão de prioridade; estado e retorno incompatíveis falham no grupo consumidor.

Atenção

Registro imutável. Não executa mudança, verifica existência do estado nem normaliza estado no construtor direto. O grupo faz essas verificações ao consumir. Uma prioridade maior pode superar bloqueio de interrupção, mas não o tempo mínimo. O retorno depende de término da animação visual, não de relógio isolado.

`CondicaoTransicaoAnimacao(...)` · módulo `coral.jogos`

O que faz

Associa uma condição de sprite a uma política de transição visual.

Construtor

Assinatura para consulta

```
CondicaoTransicaoAnimacao(condicao: 'Callable[[Sprite], bool]', transicao:
  'TransicaoEstadoAnimacao', ordem: 'int' = 0) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
condicao	Callable[[Sprite], bool]	sim	sem padrão	Callback que recebe Sprite e cujo resultado é interpretado como bool.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
transicao	TransicaoEstadoAnimacao	sim	sem padrão	TransicaoEstadoAnimacao a aplicar quando a condição for aceita.
ordem	int	não	0	Critério de desempate entre condições de mesma prioridade.

Retorno

Nova instância de CondicaoTransicaoAnimacao.

Exemplo

Código Coral

```
de coral.jogos importe CondicaoTransicaoAnimacao
de coral.jogos importe TransicaoEstadoAnimacao
crie a função pronto com sprite
    retorne verdadeiro
fim
defina item como CondicaoTransicaoAnimacao(pronto, TransicaoEstadoAnimacao("parado"))
mostre item.ordem
```

Resultado

Saída esperada

```
0
```

Erros

TypeError para condição não chamável; política, ordem ou callback inválidos podem falhar no consumidor.

Atenção

Registro imutável que só verifica se a condição é chamável. Não chama a condição nem registra regra no construtor; prefira TransicoesEstadosAnimacao.quando para integrar. O consumidor prioriza maior prioridade e depois menor ordem. O callback pode ter efeitos e ser consultado em cada atualização.

`TransicoesEstadosAnimacao(...)` · módulo `coral.jogos`

O que faz

Configura mudanças visuais por eventos, condições e chamadas explícitas.

Construtor

Assinatura para consulta

```
TransicoesEstadosAnimacao(estados: 'EstadosAnimacao', transicoes: 'dict[str, str |
  TransicaoEstadoAnimacao | dict[str, Any]]', *, reiniciar_ao_retornar: 'bool' = True)
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
estados	EstadosAnimacao	sim	sem padrão	EstadosAnimacao a controlar.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
transicoes	dict[str, str | TransicaoEstadoAnimacao | dict[str, Any]]	sim	sem padrão	Dicionário não vazio de evento para estado, política ou dicionário declarativo.
reiniciar_ao_retornar	bool	não	True	Se reinicia a animação no retorno automático; padrão verdadeiro.

Atributos importantes

Atributo	Significado
`eventos`	Tupla de eventos declarados.

Métodos principais

Assinatura para consulta

```
para(self, evento: 'str') -> 'str'
```

Referência para leitura

```
para_transicao(self, evento: 'str') -> 'TransicaoEstadoAnimacao'
```

Referência para leitura

```
quando(self, condicao: 'Callable[[Sprite], bool]', estado: 'str', *, prioridade: 'int' =
   0, tempo_minimo: 'float' = 0.0, bloquear_interrupcao: 'float' = 0.0,
  retorno_automatico: 'str | None' = None, reiniciar: 'bool' = True) ->
  'CondicaoTransicaoAnimacao'
```

Referência para leitura

```
ligar(self, movimento: 'MovimentoDirecional', sprite: 'Sprite') ->
  'VinculoTransicoesEstadosMovimento'
```

Referência para leitura

```
ligar_eventos(self, eventos: 'Any', sprite: 'Sprite') -> 'VinculoTransicoesEventos'
```

Referência para leitura

```
acionar(self, sprite: 'Sprite', evento: 'str') -> 'Animacao | None'
```

Referência para leitura

```
transicionar(self, sprite: 'Sprite', estado: 'str', **kwargs) -> 'Animacao | None'
```

Retorno

Nova instância de TransicoesEstadosAnimacao.

Exemplo

Código Coral

```
de coral.jogos importe TransicoesEstadosAnimacao
de coral.jogos importe Animacao, Sprite
defina animacao como Animacao(("a.png", "b.png"))
defina visual como Sprite(0, 0, 10, 10)
de coral.direcoes importe MovimentoDirecional, ConjuntoDirecoes
defina movimento como MovimentoDirecional(visual, ConjuntoDirecoes.cardeais())
de coral.jogos importe EstadosAnimacao
defina estados como EstadosAnimacao({"parado": animacao}, estado_inicial="parado")
defina transicoes como TransicoesEstadosAnimacao(estados, {"parar": "parado"})
defina vinculo como transicoes.ligar(movimento, visual)
execute transicoes.acionar(visual, "parar")
mostre vinculo.estado_atual
```

Resultado

Saída esperada

```
parado
```

Erros

TypeError para grupo, definição ou opções incompatíveis; ValueError para evento, estado, retorno ou vínculo inválidos; erros de callbacks são propagados.

Atenção

Normaliza eventos e verifica destinos e retornos existentes. ligar registra vínculo de movimento; ligar_eventos exige essa ligação prévia e registra ouvintes. quando registra uma condição, não a executa. Não substitui máquinas de estado lógico de agentes ou simulação. Eventos mover e parar podem reagir automaticamente a mudanças do movimento.

`VinculoTransicoesEstadosMovimento(...)` · módulo `coral.jogos`

O que faz

Executa políticas de estado visual associadas ao movimento de um sprite.

Construtor

Assinatura para consulta

```
VinculoTransicoesEstadosMovimento(grupo: "'TransicoesEstadosAnimacao'", movimento:
  'MovimentoDirecional', sprite: 'Sprite', vinculo_estados:
  'VinculoEstadosAnimacaoMovimento', ultimo_evento: 'str | None' = None, tempo_no_estado:
  'float' = 0.0, bloqueio_restante: 'float' = 0.0, prioridade_atual: 'int' = 0,
  _estava_movendo: 'bool' = False, _ultimo_estado: 'str | None' = None,
  _retorno_automatico: 'str | None' = None) -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grupo	'TransicoesEstadosAnimacao'	sim	sem padrão	TransicoesEstadosAnimacao que fornece políticas.
movimento	MovimentoDirecional	sim	sem padrão	MovimentoDirecional observado.
sprite	Sprite	sim	sem padrão	Sprite visual integrado.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
vinculo_estados	VinculoEstadosAnimacaoMovimento	sim	sem padrão	VinculoEstadosAnimacaoMovimento correspondente.
ultimo_evento	str | None	não	None	Último evento inicial ou nulo.
tempo_no_estado	float	não	0.0	Acumulador inicial do tempo no estado visual.
bloqueio_restante	float	não	0.0	Tempo inicial de bloqueio.
prioridade_atual	int	não	0	Prioridade inicial da política ativa.
_estava_movendo	bool	não	False	Campo interno; recalculado pelo construtor.
_ultimo_estado	str | None	não	None	Campo interno; recalculado do vínculo de estados.
_retorno_automatico	str | None	não	None	Campo interno de retorno; prefira o padrão nulo.

Atributos importantes

Atributo	Significado
`estado_atual`	Estado visual selecionado.
`estado_anterior`	Estado visual anterior.
`pode_ser_interrompido`	Se o bloqueio restante acabou.

Métodos principais

Assinatura para consulta

```
acionar(self, evento: 'str') -> 'Animacao | None'
```

Referência para leitura

```
transicionar(self, estado: 'str', *, prioridade: 'int' = 0, tempo_minimo: 'float' = 0.0,
    bloquear_interrupcao: 'float' = 0.0, retorno_automatico: 'str | None' = None,
  reiniciar: 'bool' = True) -> 'Animacao | None'
```

Referência para leitura

```
atualizar(self, dt: 'float') -> 'None'
```

Retorno

Nova instância de VinculoTransicoesEstadosMovimento.

Exemplo

Código Coral

```
de coral.jogos importe VinculoTransicoesEstadosMovimento
de coral.jogos importe Animacao, Sprite
defina animacao como Animacao(("a.png", "b.png"))
defina visual como Sprite(0, 0, 10, 10)
de coral.direcoes importe MovimentoDirecional, ConjuntoDirecoes
defina movimento como MovimentoDirecional(visual, ConjuntoDirecoes.cardeais())
de coral.jogos importe EstadosAnimacao
defina estados como EstadosAnimacao({"parado": animacao}, estado_inicial="parado")
de coral.jogos importe TransicoesEstadosAnimacao
defina transicoes como TransicoesEstadosAnimacao(estados, {"parar": "parado"})
defina vinculo_estados como estados.ligar(movimento, visual)
defina vinculo como VinculoTransicoesEstadosMovimento(transicoes, movimento, visual,
  vinculo_estados)
execute vinculo.acionar("parar")
mostre vinculo.estado_atual
```

Resultado

Saída esperada

```
parado
```

Erros

TypeError para movimento ou sprite incompatível; ValueError para dt, evento, estado ou política inválidos; erros de callbacks e vínculo de estados são propagados.

Atenção

O construtor registra observador e adiciona o vínculo ao sprite, mas não ao grupo; prefira transicoes.ligar. Campos temporais iniciais não são integralmente validados no construtor. atualizar valida dt e avalia retorno e condições, mas não avança a animação visual diretamente. acionar ou transicionar podem devolver nulo quando a política impede mudança. Não há desligamento nesta classe.

`VinculoTransicoesEventos(...)` · módulo `coral.jogos`

O que faz

Encaminha eventos do barramento para transições visuais declaradas.

Construtor

Assinatura para consulta

```
VinculoTransicoesEventos(grupo: "'TransicoesEstadosAnimacao'", eventos: 'Any', sprite:
  'Sprite', vinculo_movimento: 'VinculoTransicoesEstadosMovimento') -> None
```

Parâmetros

Parâmetro	Tipo	Obrigatório	Padrão	Significado
grupo	'TransicoesEstadosAnimacao'	sim	sem padrão	Grupo de transições com eventos nomeados.
eventos	Any	sim	sem padrão	Barramento com quando e remover chamáveis.
sprite	Sprite	sim	sem padrão	Sprite visual associado ao vínculo.

Parâmetro	Tipo	Obrigatório	Padrão	Significado
vinculo_movimento	VinculoTransicoesEstadosMovimento	sim	sem padrão	VinculoTransicoesEstadosMovimento que receberá os eventos.

Atributos importantes

Atributo	Significado
`ativo`	Se o vínculo ainda encaminha eventos.

Métodos principais

Assinatura para consulta

```
desligar(self) -> 'None'
```

Retorno

Nova instância de VinculoTransicoesEventos.

Exemplo

Código Coral

```
de coral.jogos importe VinculoTransicoesEventos
de coral.jogos importe Animacao, Sprite
defina animacao como Animacao(("a.png", "b.png"))
defina visual como Sprite(0, 0, 10, 10)
de coral.direcoes importe MovimentoDirecional, ConjuntoDirecoes
defina movimento como MovimentoDirecional(visual, ConjuntoDirecoes.cardeais())
de coral.jogos importe EstadosAnimacao
defina estados como EstadosAnimacao({"parado": animacao}, estado_inicial="parado")
de coral.jogos importe TransicoesEstadosAnimacao
defina transicoes como TransicoesEstadosAnimacao(estados, {"parar": "parado"})
de coral.tempo_eventos importe Eventos
defina vinculo_movimento como transicoes.ligar(movimento, visual)
defina eventos como Eventos()
defina vinculo como VinculoTransicoesEventos(transicoes, eventos, visual,
  vinculo_movimento)
execute eventos.emitir("parar")
mostre vinculo_movimento.estado_atual
execute vinculo.desligar()
mostre vinculo.ativo
```

Resultado

Saída esperada

```
parado
False
```

Erros

TypeError para barramento sem contrato; erros de registro, remoção e transição são propagados.

Atenção

Registra um ouvinte por evento no construtor. Ignora payload e aciona a política pelo nome. desligar remove ouvintes e pode ser repetido; uma falha de remoção pode deixar remoção

parcial. O construtor direto não valida integralmente a coerência do grupo, sprite e vínculo; prefira grupo.ligar_eventos após grupo.ligar. Não avança movimento, sprite ou relógio.

O que você aprendeu

Planeje o custo antes de executar. Preserve parâmetros e sementes; comparar resultados descritivamente não comprova causalidade ou superioridade geral.

Prática

Planeje um experimento com duas configurações e três repetições.

Conferir a resposta comentada

O plano contém seis execuções. Registre parâmetros, sementes, métricas e falhas; apresente diferenças como observações. Esse desenho não estabelece causalidade ou significância por si só.

Resposta do capítulo 42 no Apêndice B.

APÊNDICE A: Glossário

Termo	Significado	Primeira explicação
Algoritmo	Sequência de instruções para resolver uma tarefa.	Cap. 1
Argumento	Valor fornecido numa chamada de função.	Cap. 8
Assinatura	Forma que apresenta o nome da função, parâmetros e retorno declarado.	Cap. 8
Assíncrono	Modelo de execução em que uma tarefa pode suspender uma espera e permitir outras tarefas no ciclo de eventos.	Cap. 20
Atributo	Valor ou propriedade acessível num objeto.	Cap. 16
Backend	Implementação responsável por realizar uma capacidade, como renderização ou tradução.	Cap. 35
Booleano	Valor lógico: verdadeiro ou falso.	Cap. 10
Classe	Definição da estrutura e das operações de objetos.	Cap. 16
Coerção	Transformação automática de um valor para outro tipo.	Cap. 10
Coleção	Conjunto organizado de valores, como uma lista ou dicionário.	Cap. 14
Contrato	Regras de uso e comportamento que uma operação se compromete a manter.	Cap. 8
Dicionário	Coleção que relaciona chaves com valores.	Cap. 14
Efeito colateral	Mudança observável fora do retorno de uma função, como escrever num arquivo.	Cap. 26
Escopo	Região em que um nome pode ser consultado.	Cap. 15
Evento	Registro de um acontecimento ao qual uma regra pode reagir.	Cap. 32
Forma natural	Escrita oficial em português corrente equivalente a uma operação da linguagem ou biblioteca.	Cap. 8
Gerador	Função que entrega valores progressivamente com produza.	Cap. 19

Termo	Significado	Primeira explicação
Herança	Relação que permite a uma classe reutilizar a estrutura e o comportamento de uma base.	Cap. 16
Instância	Objeto concreto criado a partir de uma classe.	Cap. 16
JSON	Formato textual para valores estruturados portáteis.	Cap. 27
MRO C3	Regra que estabelece a ordem de procura de métodos em uma hierarquia de classes com herança múltipla.	Cap. 16
Mutação	Alteração de um objeto existente.	Cap. 25
Método	Operação associada a uma classe ou instância.	Cap. 16
Módulo	Unidade importável que reúne funções, classes e outros nomes.	Cap. 30
Nulidade	Possibilidade de um valor representar ausência com nulo.	Cap. 10
Parâmetro	Nome previsto na assinatura para receber um argumento.	Cap. 8
Persistência	Gravação de estado para uso em outra execução.	Cap. 29
Retorno	Valor devolvido por uma função ao chamador.	Cap. 15
Semente	Valor usado para inicializar uma fonte pseudoaleatória reproduzível.	Cap. 28
Sentinela	Valor interno especial usado para distinguir estados, como argumento omitido.	Cap. 8
Tipo parametrizado	Tipo que descreve também os elementos de uma coleção.	Cap. 10
Tupla estruturada	Tipo de tupla que informa o tipo de cada posição.	Cap. 10
Unicode	Padrão de representação de caracteres de diferentes sistemas de escrita.	Cap. 24
Valor padrão	Valor adotado quando um argumento opcional é omitido.	Cap. 8
API	Conjunto público de operações e contratos que um módulo oferece.	Cap. 8
CSV	Formato tabular textual que separa campos de cada registro com um delimitador.	Cap. 27

Termo	Significado	Primeira explicação
Callback	Função fornecida para ser chamada por outra operação quando um acontecimento ocorre.	Cap. 32
Codificação	Regra usada para representar texto como bytes.	Cap. 26
DAP	Protocolo que conecta o editor ao depurador.	Cap. 5
Depurador	Ferramenta para acompanhar execução, variáveis e pontos de parada.	Cap. 5
Esquema	Descrição da estrutura e versão dos dados persistidos.	Cap. 29
Frequência	Quantidade de ocorrências de um valor.	Cap. 25
Getter	Operação que fornece a leitura de uma propriedade.	Cap. 16
HMAC	Código de autenticação de mensagem baseado em hash e uma chave.	Cap. 28
HTTP	Protocolo de pedidos e respostas usado na comunicação Web.	Cap. 40
Hash	Resumo calculado a partir de dados; não equivale a cifragem.	Cap. 28
Imutável	Valor cuja estrutura não é alterada depois de sua criação.	Cap. 39
Iterável	Objeto que fornece elementos para uma passagem sequencial.	Cap. 14
LSP	Protocolo que conecta o editor ao servidor de linguagem.	Cap. 5
Loopback	Endereço de rede que se refere à própria máquina, como 127.0.0.1.	Cap. 40
Matriz	Organização retangular de valores em linhas e colunas.	Cap. 38
NaN	Representação numérica de um resultado que não é um número válido.	Cap. 38
NumPy	Biblioteca opcional usada em capacidades numéricas de Coral.	Cap. 38
Parâmetro nomeado	Parâmetro associado ao argumento pelo nome na chamada.	Cap. 15
Parâmetro posicional	Parâmetro associado ao argumento pela ordem na chamada.	Cap. 15
Plotly	Backend opcional para gráficos HTML interativos.	Cap. 39

Termo	Significado	Primeira explicação
Ponto de parada	Local em que o depurador interrompe temporariamente a execução para inspeção.	Cap. 5
Proveniência	Informação que registra origem e contexto dos dados.	Cap. 39
Pseudoaleatório	Sequência produzida por algoritmo a partir de um estado; pode ser repetida sob as mesmas condições.	Cap. 28
Recursão	Uso de uma função para resolver parte do problema por outra chamada à própria função.	Cap. 15
SVG	Formato vetorial de imagem que descreve formas e texto.	Cap. 39
Serial	Canal de comunicação em que dados são transmitidos sequencialmente.	Cap. 41
Setter	Operação que define o novo valor de uma propriedade.	Cap. 16
Subprocesso	Processo iniciado por outro programa para executar uma tarefa.	Cap. 41
TLS	Camada que verifica e protege uma conexão de rede quando configurada corretamente.	Cap. 40
Tupla	Sequência de posições cuja estrutura não é alterada como uma lista mutável.	Cap. 14
UTF 8	Codificação de texto Unicode usada por padrão em vários fluxos de arquivo.	Cap. 26
VSIX	Arquivo instalável de uma extensão do VS Code.	Cap. 5
Vetor	Sequência de componentes que representa posição, direção ou outra grandeza.	Cap. 36

APÊNDICE B: Respostas dos exercícios

Resposta do capítulo 1

Escreva um programa que mostre seu nome e o nome Coral em duas linhas.

Use duas instruções `mostre`, uma com o seu nome entre aspas e outra com `"Coral"`. Cada instrução termina sua saída com uma quebra de linha.

Resposta do capítulo 2

Escolha a instalação para uma máquina sem internet e identifique o que precisa transferir antes.

Transfira o pacote completo da release e, quando houver dependências opcionais, os pacotes dessas dependências para a plataforma alvo. O runtime portátil do pacote não precisa baixar dependências do núcleo. A instalação com pip só é offline quando todos os wheels necessários estão disponíveis.

Resposta do capítulo 3

Confirme no terminal a versão de Coral e execute um arquivo local.

Execute `coral --versao` e depois `coral programa.coral`. Se usar o runtime portátil, execute `python3 Instalacao/Runtime/coral-1.7.6.pyz --versao` e substitua `coral` por essa chamada.

Resposta do capítulo 4

Explique por que possuir um script Windows não comprova a qualificação daquela plataforma.

O script registra um fluxo de instalação, mas a qualificação exige sua execução na plataforma. A 1.7.5 considera Linux autoritativo e não promete evidência nativa Windows.

Resposta do capítulo 5

Localize a configuração de runtime da extensão e compare executar com depurar.

Executar inicia o programa no terminal. Depurar por F5 também conecta o adaptador e permite observar pontos de parada, pilha e variáveis. O runtime escolhido precisa corresponder à versão da aplicação.

Resposta do capítulo 6

Separe modelos e aplicação em arquivos diferentes e identifique a entrada no projeto.

Coloque classes e funções reutilizáveis em `modelos.coral` e importe os símbolos necessários em `principal.coral`. Configure o arquivo de entrada no `coral.toml`; esse caminho é independente do bloco opcional `programa principal`.

Resposta do capítulo 7

Altere a mensagem do primeiro programa e preveja a saída.

Troque apenas o texto entre aspas. O comando `mostre` imprime o novo texto; as aspas não aparecem na saída.

Resposta do capítulo 8

Diferencie parâmetro, argumento, padrão e retorno em uma ficha.

Parâmetro é o nome previsto pela função; argumento é o valor fornecido numa chamada; padrão é o valor usado quando um argumento opcional é omitido; retorno é o valor devolvido ao chamador.

Resposta do capítulo 9

Troque o valor inicial de `10` por `20` no exemplo de pontos. Antes de executar, preveja o resultado.

A saída será `23`: o programa começa com vinte pontos, adiciona cinco e retira dois. A ordem das instruções continua sendo a mesma.

Resposta do capítulo 10

Diferencie uma lista que pode ser nula de uma lista cujos itens podem ser nulos.

`lista de inteiro ou nulo` permite nulo no valor externo; `lista de (inteiro ou nulo)` permite nulo dentro da lista. Essas anotações não convertem nem validam profundamente os valores em execução.

Resposta do capítulo 11

Calcule o dobro da soma de 3 e 4, preservando a ordem desejada.

Use `(3 mais 4) vezes 2`. O resultado é 14. Parênteses tornam a ordem explícita.

Resposta do capítulo 12

Altere a média do exemplo para 6 e para 4; acrescente rejeição de valor negativo.

Para 6, o ramo de recuperação é escolhido; para 4, o de reprovação. Antes da comparação de aprovação, teste se a média é menor que zero e informe valor inválido.

Resposta do capítulo 13

Mostre os inteiros de 0 até antes de 3 e explique a terminação.

Use `para n de 0 até antes de 3 faça`, `mostre n` e `fim`. A saída é 0, 1 e 2 em linhas separadas; o limite superior não é visitado.

Resposta do capítulo 14

Diferencie alterar uma lista e produzir outra lista.

Uma alteração muda o objeto compartilhado por referências. Uma operação que produz nova lista preserva a original; confira o campo Atenção da ficha antes de assumir qualquer uma das duas situações.

Resposta do capítulo 15

Crie uma função que receba um inteiro e devolva seu dobro.

Declare `crie a função dobro com numero do tipo inteiro retornando inteiro`, use `retorne numero vezes 2` e feche com `fim`. Uma chamada com 21 devolve 42.

Resposta do capítulo 16

Explique a diferença entre instância, classe e atributo.

A classe descreve a estrutura e as operações; a instância é um objeto criado a partir dela; um atributo guarda parte do estado desse objeto. Duas instâncias podem manter estados diferentes.

Resposta do capítulo 17

Verifique se verdadeiro é do tipo inteiro.

O teste oficial `e_tipo(verdadeiro, "inteiro")` devolve falso. A superfície Coral não trata booleano como inteiro implícito.

Resposta do capítulo 18

Capture uma divisão por zero e imprima uma mensagem ao encerrar.

Use `tente`, a divisão, `se der erro do tipo ZeroDivisionError` com a mensagem, e `finalmente` para a mensagem final. O bloco final roda tanto no caso normal quanto na falha tratada.

Resposta do capítulo 19

Explique por que produzir números pode consumir menos memória que montar uma lista inteira.

Uma função geradora entrega um valor a cada solicitação e mantém apenas seu estado de execução. A economia existe quando o consumidor também processa os valores progressivamente.

Resposta do capítulo 20

Explique o papel de aguarde dentro de uma função assíncrona.

`aguarde` suspende a função até que a operação entregue um resultado, permitindo ao ciclo de eventos avançar outras tarefas. Não transforma um cálculo bloqueante em cálculo paralelo.

Resposta do capítulo 21

Coloque a função chamada abaixo do bloco programa principal. O que acontece ao importar o arquivo?

A execução direta inicializa o módulo inteiro e só depois inicia a entrada, portanto a função já está disponível. A importação disponibiliza os símbolos, mas não inicia o bloco principal.

Resposta do capítulo 22

Leia um nome e mostre uma saudação.

Leia uma linha com `ler_linha("Nome: ")` ou a forma natural equivalente. Guarde o texto e use `mostre "Olá, " mais nome`. A conversão é desnecessária para um nome.

Resposta do capítulo 23

Converta 12 e depois uma entrada inválida com padrão nulo.

`inteiro("12")` devolve 12. `inteiro("inválido", nulo)` devolve nulo. Sem o padrão, a entrada inválida produz `ErroConversao`.

Resposta do capítulo 24

Remova espaços externos de um texto e converta para maiúsculas.

`aparar(" Coral ")` devolve `"Coral"`; passar esse resultado para `maiusculas` devolve `"CORAL"`. O texto original não é modificado.

Resposta do capítulo 25

Remova valores repetidos de [3, 1, 3, 2] e ordene o resultado.

`unicos` produz [3, 1, 2], conservando a primeira ocorrência. `ordenar` produz [1, 2, 3]. A lista original permanece igual.

Resposta do capítulo 26

Escreva Coral num arquivo e leia o conteúdo.

Use `escrever_texto("dados.txt", "Coral")` e `ler_texto("dados.txt")`. A escrita tem efeito no sistema de arquivos e a leitura devolve texto. Considere permissão, codificação e pasta de trabalho.

Resposta do capítulo 27

Converta um dicionário para JSON e reconstrua o valor.

Use `para_json` para obter texto JSON e `de_json` para recuperar o dicionário. A ida e volta preserva os valores compatíveis com JSON, não objetos arbitrários ou identidade de instância.

Resposta do capítulo 28

Calcule a média de 2, 4 e 6.

A soma é 12 e a quantidade é 3, portanto a média é 4. `coral.matematica.media([2, 4, 6])` devolve 4.0. Não use uma coleção vazia.

Resposta do capítulo 29

Explique por que persistência não deve guardar callbacks e janelas ativas.

Esses objetos dependem de execução e recursos externos. O formato portátil guarda estado declarativo, com contrato e esquema, e reconstrói apenas os tipos que possuem adaptadores explícitos.

Resposta do capítulo 30

Diferencie arquivo de entrada, importação e bloco programa principal.

O projeto escolhe qual arquivo executar; a importação carrega um módulo; o bloco principal organiza a ação da execução direta. Bibliotecas reutilizáveis não devem iniciar a aplicação durante a importação.

Resposta do capítulo 31

Escreva uma verificação para uma função dobro e provoque uma falha intencional.

Use `garanta que dobro(3) for igual a 6`. Trocar 6 por 7 provoca a falha da verificação. Reponha o valor correto antes de considerar o teste aprovado.

Resposta do capítulo 32

Diferencie evento, regra e relógio de teste.

O evento registra um acontecimento; a regra escolhe como reagir; o relógio de teste permite controlar o tempo explicitamente. Uma passagem de tempo simulada não deve depender de espera real.

Resposta do capítulo 33

Modele dois objetos e uma relação fora do domínio de jogos.

Um laboratório pode modelar equipamentos e medições, com uma relação indicando o equipamento que gerou cada medição. Mundo oferece entidades e relações sem exigir janela ou combate.

Resposta do capítulo 34

Identifique os dados que devem ser reproduzíveis numa simulação de combate.

Estado inicial, participantes, regras, ordem de turno e semente ou resultados das jogadas precisam ser controlados. Uma vitória observada numa única execução não comprova equilíbrio.

Resposta do capítulo 35

Explique a vantagem de testar um jogo com BackendNulo.

O backend permite exercitar o estado e o fluxo sem janela nem renderização física. Ele não comprova desempenho gráfico ou funcionamento do dispositivo real.

Resposta do capítulo 36

Diferencie um vetor de posição e uma transformação em 3D.

O vetor representa coordenadas; a transformação especifica como alterar coordenadas. Em matrizes homogêneas, pontos e direções têm papéis diferentes e não devem ser confundidos.

Resposta do capítulo 37

Explique como semente e receita ajudam a reproduzir um campo procedural.

A semente controla o gerador e a receita registra algoritmo, versão e parâmetros. A reconstrução só é garantida para operações e versões suportadas; callbacks arbitrários não viram receita portátil.

Resposta do capítulo 38

Explique por que correlação com coluna constante é inválida.

O cálculo divide pela dispersão de cada variável. Se uma coluna não varia, essa dispersão é zero e a correlação não pode ser definida pelo contrato; a biblioteca produz diagnóstico.

Resposta do capítulo 39

Separe criar uma especificação de gráfico e salvar sua imagem.

Criar `linha` ou `dispersao` descreve os dados sem abrir janela. `salvar` escolhe o formato e renderiza. A disponibilidade do backend depende do formato solicitado.

Resposta do capítulo 40

Diferencie HTML local, cliente HTTP e servidor HTTP.

HTML local é conteúdo de apresentação; o cliente faz pedidos; o servidor responde a pedidos. Gerar HTML não abre conexão nem inicia um servidor.

Resposta do capítulo 41

Explique por que inventário simulado não comprova comunicação com uma placa física.

A simulação verifica o contrato do programa sem exercer cabo, porta, driver ou dispositivo. A comunicação física exige uma sessão de campo com aquele hardware.

Resposta do capítulo 42

Planeje um experimento com duas configurações e três repetições.

O plano contém seis execuções. Registre parâmetros, sementes, métricas e falhas; apresente diferenças como observações. Esse desenho não estabelece causalidade ou significância por si só.

Projeto 30

Cadastro em três arquivos. Use modelos.coral para as classes, principal.coral para a entrada e um arquivo de teste para as verificações. Importe nomes explícitos; a importação de modelos não deve iniciar a aplicação.

Projeto 31

Contrato de uma função. Os resultados são 0, 6 e menos 6 para entradas 0, 3 e menos 3. A conversão do texto inválido deve ser tratada na borda de entrada, e não mascarada no cálculo de dobro.

Projeto 32

Alerta de estoque. O estado guarda a quantidade; a condição da regra compara a quantidade com três. Teste a transição de 3 para 2 e a permanência abaixo do limite, respeitando a política de repetição da regra.

Projeto 33

Mapa de ocupação. Coloque um bloco numa célula da camada e consulte a mesma posição; uma célula sem bloco representa ausência conforme o contrato do mapa. A visualização deve ficar separada da ocupação.

Projeto 34

Duelo reproduzível. Crie um mundo, adicione os personagens, inicie o combate com a semente e registre cada alteração. Para comparar, reconstrua o estado inicial e use a mesma semente; não reutilize o estado final da execução anterior.

Projeto 35

Movimento sem janela. Crie o jogo sem janela, forneça uma entrada simulada e execute com max_quadros=2. Verifique deslocamento e contagem de quadros. A prova de renderização física permanece separada.

Projeto 36

Distâncias no espaço. Use coordenadas alinhadas e a mesma unidade. Pontos (0, 0) e (3, 4) distam 5; em 3D, (0, 0, 0) e (0, 0, 5) também distam 5. Esses resultados não tornam as duas representações intercambiáveis.

Projeto 37

Campo com receita. Registre algoritmo, versão, semente e parâmetros suportados. Consultas determinísticas do mesmo campo devem preservar os valores independentemente da ordem; não introduza callbacks não portáteis na receita.

Projeto 38

Resumo de medições. A média é 23 e a amplitude é 6. O contrato numérico rejeita ausência ou valores não finitos quando não há uma política explícita de seleção; não transforme dados ausentes em zero.

Projeto 39

Gráfico de temperatura. Configure a especificação com rótulos e unidades antes de salvar. SVG 2D usa o backend local da biblioteca; a criação da figura não abre janela. Confirme que o arquivo existe e que os dados de origem permanecem iguais.

Projeto 40

Página de estado local. Altere o corpo de resposta_texto e mantenha o encerramento em finalmente. A resposta deve ter status 200 e o texto escolhido. Não remova o limite de resposta ou exponha o servidor externamente por acidente.

Projeto 41

Relatório de capacidades. Use quantidade_processadores e programa_disponivel. Disponibilidade não significa que o programa executou, e quantidade de processadores não é uma promessa de recursos efetivos sob quotas.

Projeto 42

Experimento com seis execuções. O plano deve prever seis execuções. Preserve parâmetros, sementes, sucessos e falhas. Compare os resultados descritivamente; não declare significância ou superioridade geral a partir de uma diferença de tempo isolada.

APÊNDICE C: Índice remissivo

Recurso	Localização
O que é Coral	Cap. 1
Instalando Coral	Cap. 2
Linux	Cap. 3
Outras plataformas suportadas	Cap. 4
VS Code	Cap. 5
Projetos Coral	Cap. 6
Primeiro programa (passo a passo)	Cap. 7
Como ler exemplos e fichas de API	Cap. 8
Valores e variáveis	Cap. 9
Tipos e tipagem opcional	Cap. 10
Operadores e expressões	Cap. 11
Condições	Cap. 12
Repetições	Cap. 13
Coleções, índices e fatias	Cap. 14
Funções e escopo	Cap. 15
Classes e objetos	Cap. 16
Verificando tipos na prática	Cap. 17
Erros, padrões e contextos	Cap. 18
Geradores	Cap. 19
Assíncrono	Cap. 20
Programa principal	Cap. 21
Entrada e saída	Cap. 22
Conversões	Cap. 23
Texto	Cap. 24
Coleções	Cap. 25
Arquivos e caminhos	Cap. 26
JSON e serialização	Cap. 27
Matemática, datas e aleatoriedade	Cap. 28
Persistência	Cap. 29

Recurso	Localização
Projetos, módulos e coral.toml	Cap. 30
Testes	Cap. 31
Regras, eventos e tempo	Cap. 32
Mundo	Cap. 33
RPG	Cap. 34
Jogos	Cap. 35
Espaço e geometria	Cap. 36
Procedural	Cap. 37
Computação numérica e científica	Cap. 38
Gráficos	Cap. 39
Web	Cap. 40
Sistema e hardware	Cap. 41
Laboratório	Cap. 42
Algoritmo	Cap. 1; Apêndice A
Argumento	Cap. 8; Apêndice A
Assinatura	Cap. 8; Apêndice A
Assíncrono	Cap. 20; Apêndice A
Atributo	Cap. 16; Apêndice A
Backend	Cap. 35; Apêndice A
Booleano	Cap. 10; Apêndice A
Classe	Cap. 16; Apêndice A
Coerção	Cap. 10; Apêndice A
Coleção	Cap. 14; Apêndice A
Contrato	Cap. 8; Apêndice A
Dicionário	Cap. 14; Apêndice A
Efeito colateral	Cap. 26; Apêndice A
Escopo	Cap. 15; Apêndice A
Evento	Cap. 32; Apêndice A
Forma natural	Cap. 8; Apêndice A
Gerador	Cap. 19; Apêndice A
Herança	Cap. 16; Apêndice A

Recurso	Localização
Instância	Cap. 16; Apêndice A
JSON	Cap. 27; Apêndice A
MRO C3	Cap. 16; Apêndice A
Mutação	Cap. 25; Apêndice A
Método	Cap. 16; Apêndice A
Módulo	Cap. 30; Apêndice A
Nulidade	Cap. 10; Apêndice A
Parâmetro	Cap. 8; Apêndice A
Persistência	Cap. 29; Apêndice A
Retorno	Cap. 15; Apêndice A
Semente	Cap. 28; Apêndice A
Sentinela	Cap. 8; Apêndice A
Tipo parametrizado	Cap. 10; Apêndice A
Tupla estruturada	Cap. 10; Apêndice A
Unicode	Cap. 24; Apêndice A
Valor padrão	Cap. 8; Apêndice A
Chamadas multilinha	**Cap. 15, seção Chamadas explícitas em várias linhas**
Pertencimento a classe	Cap. 17, seção Consultar classes do programa
Celula	Cap. 16, seção Exemplo: uma célula que guarda conteúdo
Snapshots do editor	Volume II, Implementação e arquitetura

APÊNDICE D: Migração desde a Coral 1.7.0

A 1.7.2 acrescenta expressões naturais para entrada, conversões, texto, coleções, JSON e consulta de tipos. As APIs explícitas permanecem válidas. A mudança é opcional e não exige reescrever programas existentes. Consulte os Caps. 17 e 22 a 27. Os diagnósticos de execução passam a manter uma classificação comum entre terminal e depurador; consulte o Cap. 18.

A 1.7.3 acrescenta coleções parametrizadas, tuplas estruturadas e nulidade opcional nas anotações, sem obrigar o uso de tipos. Atualize anotações onde elas tornarem a intenção mais clara e mantenha testes de dados recebidos. Consulte o Cap. 10.

Herança múltipla e chamadas cooperativas seguem MRO C3. Uma chamada ao pai continua a cadeia; ela não seleciona indiscriminadamente todas as bases. Construtores precisam cooperar explicitamente. Consulte o Cap. 16, seção Para ir além.

O bloco `programa principal` pode organizar a entrada de uma aplicação e permite declarar suas funções abaixo. Ele começa apenas na execução direta do módulo; scripts lineares continuam válidos. Consulte o Cap. 21.

A coleção deixa de ser um único volume. As referências internas passam a indicar capítulo e seção. Esta revisão preserva os livros anteriores e usa fontes compartilhadas para os dois volumes e a projeção local do site.

Migração para a Coral 1.7.4

Instale Coral Language 0.75.2 e confirme o runtime 1.7.4. As formas de escrever classes, imports e `programa principal` permanecem disponíveis. Revise novas advertências entre módulos usando as assinaturas reais. O realce semântico acompanha o tema e pode ser desativado. Ao reutilizar um projeto antigo, permita que o build regenere o cache de contratos.

Migração para a Coral 1.7.6

Instale Coral Language 0.77.0 e confirme o runtime 1.7.6. A forma `objeto for do tipo Classe` substitui a chamada comum `e_tipo(objeto, Classe)` quando isso melhorar a leitura; a API continua compatível. Consulte o Cap. 17, seção Consultar classes do programa.

Chamadas longas podem usar `chame alvo com`, argumentos indentados e `fim`; a forma compacta permanece válida. Consulte o Cap. 15, seção Chamadas explícitas em várias linhas. Essa mudança não exige migrar o código existente.

Os dois volumes são a edição oficial corrente. O livro único da 1.7.0 é histórico e permanece disponível para compatibilidade. Os relatórios da coleção registram a validação da edição e as lacunas normativas anteriores que ainda precisam de revisão.

APÊNDICE E: Mapa de recursos novos

Release	Recurso	Localização
1.7.2	Formas naturais	Cap. 8; Caps. 22 a 27
1.7.2	Entrada	Cap. 22
1.7.2	Conversões	Cap. 23
1.7.2	Texto	Cap. 24
1.7.2	Coleções	Cap. 25
1.7.2	JSON	Cap. 27
1.7.2	Diagnósticos	Cap. 18
1.7.3	Tipos parametrizados	Cap. 10
1.7.3	Tuplas e nulidade	Cap. 10, seção Para ir além
1.7.3	Herança múltipla e MRO C3	Cap. 16, seção Para ir além
1.7.3	Chamadas e inicialização do pai	Cap. 16, seção Para ir além
1.7.3	Programa principal	Cap. 21

Release	Recurso	Localização
1.7.4	Realce semântico e opções do editor	Cap. 5
1.7.4	Tipagem entre módulos	Cap. 10 e Cap. 30
1.7.4	Contratos transitivos e reexportações	Cap. 30; Volume II, Implementação e arquitetura
1.7.4	Manifesto inicial e ícone coral.toml	Cap. 5 e Cap. 30

Release	Recurso	Localização
1.7.5	Consulta natural de classes e ancestrais	Cap. 17; Cap. 16, exemplo Celula
1.7.5	Chamadas explícitas multilinha	Cap. 15; Volume II, Referência da linguagem
1.7.5	Ajuda de assinatura e autocomplete de tipos	Cap. 5
1.7.5	Snapshots e índice seletivo	Volume II, Implementação e arquitetura
1.7.6	Parâmetros, imports e bases em listas explícitas	Cap. 15, 16 e 30
1.7.6	Ambiente isolado, wheels e lock por alvo	Cap. 30; Volume II, CLI